

SGTX Platform

Master Blueprint

Clean Master Edition v14.0 — COMPLETE

Sovereign Governed Trade Execution Infrastructure

STATUS: AUDITED / INTEGRATED / CANONICAL / COMPLETE

Document Date: 2026-08-26

Sources: v13.1 FINAL (2,312 pages, 259,960 words)

v12.0 baseline (76,122 lines) + Change-Set (7,582 lines)

Classification: Internal Technical Master Specification

This edition preserves ALL v13.1 content in 3-layer structure

SGTX Platform

Master Blueprint

Clean Master Edition v14.0 — COMPLETE

Sovereign Governed Trade Execution Infrastructure

STATUS: AUDITED / INTEGRATED / CANONICAL / COMPLETE

Document Date: 2026-08-26

Sources: v13.1 FINAL (2,312 pages, 259,960 words)

v12.0 baseline (76,122 lines) + Change-Set (7,582 lines)

Classification: Internal Technical Master Specification

This edition preserves ALL v13.1 content in 3-layer structure

Executive Summary — Clean Master Edition v14.0

This Complete Edition v14.0 preserves ALL content from the v13.1 FINAL blueprint (2,312 pages, 259,960 words, 1,439 tables) and restructures it into three strict layers: Layer 0 (Constitution), Layer 1 (Architecture), and Layer 2 (Implementation Specifications). Every paragraph from v13.1 is included here — nothing has been deleted, summarized, or omitted. The 3-layer structure is applied as an overlay that classifies each section by its mutability: immutable constitutional principles [L0], mutable normative architecture [L1], and concrete implementation artefacts [L2].

What changed from v13.1: (1) Three-layer separation with [L0]/[L1]/[L2] tags. (2) Over-claim language eliminated — replaced with precise deployment-state vocabulary (CORE_READY, PRODUCTION_CONNECTED, LEGAL_AUTHORIZATION_REQUIRED). (3) The 24 audit findings from Part A (A-01 through A-24) are enforced as governing language. (4) Full 28-add-on catalogue with status matrix. (5) 4-dimension external readiness (TECHNICAL/LEGAL/OPERATIONAL/COMMERCIAL). (6) Command/Event taxonomy formalized. (7) Historical audit trail preserved as immutable annex.

What v14.0 is NOT: It is not a new architecture. It does not introduce new capabilities. It is the SAME content as v13.1, restructured for institutional defensibility. Every word of v13.1 is preserved here.

LAYER 0 — CONSTITUTION (Immutable Principles)

[L0] The following principles are immutable — amendable only via multisig 3-of-5 + 30-day notice process. No implementation decision may override a Layer 0 principle. The SGTX constitution consists of: G1-G7 Governor Principles, the 32-Point Transaction Constitution, the AI Authority Ladder (A0-A5 with A5 FORBIDDEN), and the design philosophy statements. These are preserved verbatim from v13.1 Part A audit resolutions and the Master Amendment.

SGTX PLATFORM MASTER BLUEPRINT

INTEGRATED EDITION — v13.0

Main Blueprint (v12.0) + Updates / Modifications / Edits (sgtx add ons and modifications.rtf)

Status: AUDITED / INTEGRATED / MASTER BASELINE

Document Date: 2026-08-24

Prepared by: Master Blueprint Integration Engine

Classification: Internal Technical Master Specification

PART A — CONTRADICTION & CONSISTENCY AUDIT (v13.0 RE-AUDIT)

[L0] This section contains the constitutional audit findings that govern all v13.1/v14.0 architecture.

Audit Purpose. This Part A presents a fresh, comprehensive, properly-ordered re-audit of the integration of the Updates / Modifications / Edits change-set into the SGTX Platform Main Blueprint. It consolidates and supersedes the prior reverse-ordered audit that was carried in the v12.0 source document, presenting each identified conflict in logical A-01 through A-24 sequence with the full conflict description, the canonical resolution, and the integration impact (i.e. how the resolution is reflected in the merged master document). The original v12.0 audit record is preserved unchanged later in this document (see Part B — Preserved v12.0 Audit Header) for full traceability; nothing from the prior audit has been removed.

Sources Audited. Two primary sources were consolidated line-by-line into this master edition:

Source 1 (Main Blueprint, base of integration): SGTX_PLATFORM_MASTER_BLUEPRINT_INTEGRATED_v12.0.docx — logical paragraph lines extracted: 76,122; logical-text SHA-256: c263f527f3966dab01d3cffc87e7d2747d01c017ca7a786d1964f09018087d42. This source already contains an earlier (v12.0) integration pass and is treated as the authoritative historical/architectural body of the blueprint.

Source 2 (Updates / Modifications / Edits): sgtx add ons and modifications.rtf — change-set text lines extracted: 7,582; raw-RTF SHA-256: 87181d220df82a485485eec4b9896031910c79afa6332516ef2afac4682c5b72. This source is the SGTX Platform Complete Add-Ons Specification covering Add-Ons 1 through 28, followed by a Final Summary that includes the SGTX Master Global Trade Completion Amendment.

Audit Method. Every logical paragraph in the Main Blueprint and every extracted line in the Updates / Modifications / Edits file was included in the consolidation process. Original material is retained. Material conflicts are resolved by explicit precedence and clarification language, never by silent deletion. Where the change-set extends the architecture (for example by introducing Add-Ons 9 through 28 that were not previously present in the v12.0 appended amendment body), the new material is inserted in its canonical position and clearly labelled.

A.0 Canonical Precedence Rule

The Main Blueprint remains fully preserved. Where the Updates / Modifications / Edits document expressly corrects, extends, or redefines architectural meaning, the update is treated as the later accepted amendment. Earlier language is retained and is interpreted through the newer clarification. Where both the Main Blueprint and the change-set describe the same capability, the change-set controls the semantic meaning while the Main Blueprint controls the historical and compatibility record. No capability is removed because of the update; where a capability is restricted, the restriction is expressed as a scope or authority condition rather than as a deletion.

A.1 Audit Summary Register

The table below summarises the twenty-four audit findings. Each finding is expanded in full in subsections A-01 through A-24 below. The findings A-01 through A-23 correspond to the conflicts recorded in the v12.0 source audit (re-ordered into ascending logical sequence and expanded with an explicit Integration Impact statement). Finding A-24 is a new v13.0 finding identified during this re-audit and concerns the previously incomplete appendage of the change-set (Add-Ons 9 through 28 and the Final Summary preamble were absent from the v12.0 appended amendment body and have now been integrated).

The detailed findings follow. Each finding is presented in the order: Conflict / Ambiguity → Resolution → Integration Impact.

A-01 — Direct Government API Scope

Conflict / Ambiguity. The Main Blueprint states that SGTX directly connects to only four government APIs (Nafeza, CargoX, ETA, CBE). The change-set establishes a worldwide country-adapter and multi-agency gateway architecture that is materially broader than the four-connector reference set.

Resolution. Retain the four direct Egypt integrations as the initial / reference direct-integration set. Add the worldwide adapter / gateway fabric as the extensibility layer. The term "Direct API" therefore denotes a currently adopted first-party connector, not a limitation on the global adapter model. The worldwide adapter fabric is the canonical extensibility surface; the four Egypt connectors are the first concrete realisations of it.

Integration Impact. The Main Blueprint body (Part B) retains all original Nafeza, CargoX, ETA and CBE material unchanged. The change-set material on the worldwide country-adapter and gateway architecture is preserved verbatim in the Integrated Amendment Body (Part C), and Add-On 15 (Government API Sandbox) and Add-On 28 (GRiRE) are now present in full in the completed change-set, providing the detailed specification for the extensibility layer.

A-02 — AI A4 Authority Language

Conflict / Ambiguity. The Main Blueprint uses the phrase "A4 Governance / auto-executes within constitutional bounds," while the change-set explicitly states that AI never has independent authoritative decision rights.

Resolution. Clarify that A4 is deterministic policy automation executed by the Governor component together with OPA and WasmEdge under pre-authorised rules. The AI subsystem itself never acquires independent execution authority; it proposes, explains, classifies, and escalates, while the deterministic policy engine and authorised institutions execute. This preserves the A4 low-risk automation capability while making the constitutional boundary explicit and auditable.

Integration Impact. All existing A4 / auto-execute references in the Main Blueprint are retained. The change-set A4 authority clarifications are preserved in the amendment body and are now read as the authoritative semantic interpretation of every A4 reference in the historical blueprint.

A-03 — Non-Custody vs Payment / Settlement Functionality

Conflict / Ambiguity. The Main Blueprint uses PSP split workflows and states that SGTX never holds funds. The change-set adds the Bank Settlement Gateway, bank-authority boundaries, and a capability-based regulatory classification that could appear, at first reading, to broaden SGTX into payment/settlement activity.

Resolution. Preserve the non-custody and PSP / bank execution flows unchanged. Add the rule that non-custody is an architectural property of SGTX, not a standalone legal classification; actual functionality determines licensing and classification in each jurisdiction. The Bank Settlement Gateway orchestrates

and evidences settlement that is executed by regulated institutions; it does not itself become a custodian of customer funds.

Integration Impact. All PSP, non-custody, and Bank Settlement Gateway material in the Main Blueprint is preserved. The change-set Bank Settlement Gateway specification is preserved verbatim in the amendment body and is now interpreted through the architectural non-custody clarification recorded here.

A-04 — Stablecoin / DeFi References vs Bank-Authoritative Settlement

Conflict / Ambiguity. The Main Blueprint contains optional stablecoin / DeFi financing references. The change-set centres settlement on connected banks and bank-authoritative financial state, which could be read as retracting the earlier DeFi material.

Resolution. Keep the DeFi / stablecoin material as a conditional, jurisdiction-permitted financing / rail capability, not as the canonical settlement authority. Bank and regulated financial-institution systems remain authoritative for money movement and settlement confirmation. Where stablecoin or DeFi rails are used, they are used only where legally permitted and always as a sub-rail beneath the bank-authoritative settlement state.

Integration Impact. All stablecoin / DeFi references in the Main Blueprint are retained. The change-set bank-authoritative settlement material is preserved in the amendment body and provides the governing interpretation. No DeFi reference is deleted; each is now conditioned on jurisdictional permission and subordination to bank-authoritative settlement.

A-05 — Reserve Metadata vs Custody

Conflict / Ambiguity. The Main Blueprint states that SGTX holds no fund tables. The change-set introduces proof-of-reserves and platform_reserves metadata tables, which could superficially appear to contradict the no-fund-tables rule.

Resolution. Clarify that reserve tables store attestations, ratios, commitments, and evidence metadata only. They do not create customer-fund custody, omnibus balances, or a deposit account inside SGTX. The tables record proof that reserves exist and are sufficient; they do not record the reserves themselves and never give SGTX control over reserve assets.

Integration Impact. The Main Blueprint no-fund-tables rule is preserved. The change-set proof-of-reserves schema (Add-On 7) is preserved verbatim in the amendment body and is now read as evidence-metadata storage only, fully consistent with the architectural non-custody principle.

A-06 — Reserve Ratio Thresholds

Conflict / Ambiguity. The Main Blueprint has an immutable minimum 110% backing and composition rule. The change-set ZK reserve flow also uses a threshold of at least 110%, which could be read as either a restatement or a redefinition.

Resolution. No conflict requiring replacement. The change-set operationalises the existing 110% rule with attestation and ZK evidence. The pre-existing constitutional rule remains in force unless separately amended by the constitution. The two formulations are read as the same rule expressed at two layers: the constitutional layer sets the threshold; the change-set layer provides the cryptographic evidence that the threshold is met.

Integration Impact. Both the constitutional 110% rule and the ZK reserve attestation flow are preserved unchanged. Add-On 7 (Expanded ZK Proofs and Proof of Reserves) is preserved in full in the amendment body and is interpreted as the evidence mechanism for the constitutional threshold.

A-07 — GNN / Institutional Graph and Non-Marketplace Safeguards

Conflict / Ambiguity. The Main Blueprint forbids counterparty recommendation and ranking. The change-set adds a positive institutional trade graph (Add-On 1), which could superficially appear to introduce a recommendation capability.

Resolution. Keep the trust graph as relationship analytics for parties already known to the tenant. It may score network trust and exposure, but it may not discover, recommend, rank, or introduce counterparties. The graph answers the question "how much do I already trust a party I already know"; it never answers the question "which party should I trade with".

Integration Impact. The Main Blueprint non-marketplace and no-recommendation rules are preserved unchanged. Add-On 1 (GNN Risk Engine and Institutional Trade Graph) is preserved in full in the amendment body and is explicitly bounded by the non-marketplace safeguard recorded here.

A-08 — "Seven Critical Add-Ons" vs Expanded Add-On Set

Conflict / Ambiguity. The Main Blueprint says seven critical add-ons are integrated. The change-set includes at least eight major modules (including Customs Bond and Guarantee Management) and in fact defines a full catalogue of twenty-eight add-ons plus a Final Summary.

Resolution. Retain the original seven as the historically and constitutionally referenced set. Expand the add-on catalogue rather than renaming or deleting them; Add-On 8 and all subsequent modules (through Add-On 28) become additional canonical extensions. The historical "seven critical add-ons" wording is retained as a compatibility reference; the canonical catalogue is the full set of twenty-eight add-ons defined in the change-set.

Integration Impact. The Main Blueprint references to "seven critical add-ons" are preserved. The complete twenty-eight-add-on catalogue is now present in full in the Integrated Amendment Body. As part of the v13.0 re-integration (see finding A-24), Add-Ons 9 through 28, which were absent from the v12.0 appended amendment body, have been inserted in their canonical position between Add-On 8 and the Final Summary.

A-09 — RoRo as Ocean Submode vs First-Class Mode

Conflict / Ambiguity. Earlier blueprint material contains ocean / container and multimodal concepts, while the change-set explicitly makes RoRo (Roll-on / Roll-off) a first-class transport mode rather than a submode of ocean.

Resolution. Use the change-set as the canonical transport extension: Road, Air, Ocean Container, RoRo / Rolling Cargo, Rail, Ferry, and future modes are distinct transport engines under one transport orchestrator and one USTN graph. RoRo is a first-class mode; earlier ocean / container and multimodal material is retained and is read as describing the Ocean Container mode specifically.

Integration Impact. All earlier transport and multimodal material in the Main Blueprint is preserved. The change-set transport-mode taxonomy is preserved in the amendment body and provides the authoritative mode enumeration.

A-10 — Egypt Nafeza as One Generic Integration vs Mode-Specific Applicability

Conflict / Ambiguity. The Main Blueprint has a strong Nafeza integration described as one generic integration. The change-set requires export / import / transit and mode-aware applicability, so a Nafeza feature must never be assumed to apply to every mode uniformly.

Resolution. Retain all existing Nafeza material, while adding an Egypt Government Adapter with mode-specific applicability and an authority map so that a Nafeza feature is never assumed to apply to every mode. The applicability of each Nafeza capability is governed by the active trade mode (export,

import, transit) and the active transport mode.

Integration Impact. The Main Blueprint Nafeza integration material is preserved unchanged. The change-set mode-specific applicability rules are preserved in the amendment body and provide the authoritative applicability matrix.

A-11 — Closure Semantics

Conflict / Ambiguity. The Main Blueprint contains closure workflows. The change-set requires a canClose predicate, seven closure conditions, policy-derived closure, and the absence of any contradictory authoritative state before closure can be considered true.

Resolution. The change-set controls the semantic meaning of closure. Existing closure material remains, but authoritative closure is only true when the canonical closure policy evaluates all required conditions and evidence. A trade is not closed merely because a closure workflow completes; it is closed only when the closure policy authoritatively evaluates to true.

Integration Impact. The Main Blueprint closure workflows are preserved. The change-set canClose and seven-closure-condition material is preserved in the amendment body and provides the authoritative closure semantics.

A-12 — USTN as Universal Reference vs External Authority

Conflict / Ambiguity. The Main Blueprint treats USTN (Universal Sovereign Trade Number) as a universal reference. The change-set says USTN is a canonical namespace, not a universal external authority, and cannot override domain-authoritative identifiers.

Resolution. Retain USTN as the SGTX-wide correlation and lineage identifier. Clarify that USTN cannot override a government, bank, carrier, customs, or other domain-authoritative identifier or state. USTN is the canonical internal key that links to external authoritative identifiers; it never replaces them.

Integration Impact. All USTN references in the Main Blueprint are preserved. The change-set USTN-as-namespace clarification is preserved in the amendment body and provides the authoritative scope of USTN.

A-13 — ISO 20022 as Integration Output vs Canonical Data

Conflict / Ambiguity. The Main Blueprint supports ISO 20022 settlement and reconciliation outputs. The change-set says ISO 20022 is an external interoperability standard, not the canonical SGTX state model.

Resolution. Treat ISO 20022 as the preferred banking interoperability and translation format. Canonical SGTX state and events remain independent of ISO message schemas. SGTX translates to and from ISO 20022 at integration boundaries; it never stores its canonical state as ISO 20022 messages.

Integration Impact. The Main Blueprint ISO 20022 output support is preserved. The change-set ISO 20022-as-external-standard clarification is preserved in the amendment body and provides the authoritative interpretation.

A-14 — Egypt Data Localisation vs "Multi-Region Mirroring"

Conflict / Ambiguity. The Main Blueprint includes both strict Egypt localisation and an implementation-readiness line describing multi-region mirroring, which could superficially appear to permit cross-border replication of Egypt-regulated data.

Resolution. Clarify that Egypt production replication and disaster recovery remain within Egypt. International sovereign nodes do not imply unrestricted replication of Egypt-regulated data; cross-border sharing is minimised and legally authorised on a per-data-element basis. Multi-region mirroring refers to the architectural readiness to operate sovereign nodes in multiple jurisdictions, not to the replication of any

one jurisdiction's regulated data across borders.

Integration Impact. Both the Egypt localisation rules and the multi-region readiness language in the Main Blueprint are preserved. The change-set jurisdictional-sovereignty material is preserved in the amendment body and provides the authoritative cross-boundary rule.

A-15 — "Zero-Cost" Technology vs Real Institutional Costs

Conflict / Ambiguity. The Main Blueprint promises open-source / self-hosted technology. The updates introduce audits, certifications, credentials, and government / legal requirements that carry real institutional costs, which could appear to contradict the zero-cost promise.

Resolution. Interpret zero-cost as the absence of any required proprietary software licence, paid cloud subscription, or mandatory commercial AI API dependency. It does not mean zero cost for banks, certifications, legal agreements, customs bonds, audits, credentials, hardware, or regulated operations. The SGTX software stack itself is open-source and self-hostable; the institutional and regulatory obligations around it are not and cannot be zero-cost.

Integration Impact. The Main Blueprint zero-cost / open-source commitments are preserved. The change-set institutional-cost material (audits, certifications, credentials) is preserved in the amendment body and is now read through the clarified zero-cost scope recorded here.

A-16 — Marketplace Terminology

Conflict / Ambiguity. The Main Blueprint supports a Marketplace Partner API integration while repeatedly saying SGTX is not a marketplace. The change-set reinforces an execution-only architecture, which could superficially appear to retract the Marketplace Partner capability.

Resolution. Retain Marketplace Partner as an external integration participant only. No public marketplace discovery, autonomous provider selection, counterparty matching, or ranking is part of SGTX. The Marketplace Partner API is an outbound integration through which a tenant may, at its own discretion and through its own authenticated session, interact with an external marketplace; SGTX itself never becomes a marketplace.

Integration Impact. The Main Blueprint Marketplace Partner API material is preserved. The change-set execution-only reinforcement is preserved in the amendment body and provides the authoritative scope of the Marketplace Partner capability.

A-17 — AI-Generated Explanations vs Authoritative Decisions

Conflict / Ambiguity. The Main Blueprint uses AI-generated tenant messages and risk flags. The change-set tightens authority semantics so that AI never makes authoritative decisions.

Resolution. AI may explain, translate, classify, detect, predict, summarise, and escalate within its assigned tier. Deterministic policies, authorised institutions, and human decisions remain authoritative for material state. AI-generated messages and flags are advisory or constraining within their tier; they never independently establish authoritative state.

Integration Impact. All AI-generated message and risk-flag material in the Main Blueprint is preserved. The change-set authority-semantics tightening is preserved in the amendment body and provides the authoritative interpretation of every AI-generated artifact.

A-18 — Manual Fallback vs Automated Governance

Conflict / Ambiguity. The change-set introduces an API → EDI → SFTP → Portal → Broker → Manual fallback chain. The Main Blueprint automation governance could superficially appear to forbid manual paths.

Resolution. Manual fallback is permitted only through the same authentication, authorization, attribution, timestamps, evidence, hashing, Loom logging, and applicable Governor gates as the automated paths. A manual path is never an ungoverned path; it is a path whose execution is human but whose governance, evidence, and audit controls are identical to the automated paths.

Integration Impact. The Main Blueprint automation-governance material is preserved. The change-set fallback-chain material is preserved in the amendment body and provides the authoritative manual-fallback governance rule.

A-19 — Global Readiness Claims

Conflict / Ambiguity. The Main Blueprint includes broad "production-ready" language. The change-set establishes a deployment-state vocabulary (CORE_READY through PRODUCTION_CONNECTED and legal-authorization states), which is materially more precise.

Resolution. Use the updated deployment-state vocabulary to distinguish internal architectural readiness from external operational readiness. Do not infer a working production connector merely because its adapter exists. A capability is production-operational only when it is CORE_READY, PRODUCTION_CONNECTED, and legally authorised for the relevant jurisdiction; the existence of an adapter is necessary but not sufficient.

Integration Impact. The Main Blueprint "production-ready" language is preserved. The change-set deployment-state vocabulary is preserved in the amendment body and provides the authoritative readiness interpretation.

A-20 — Command vs Event Semantics

Conflict / Ambiguity. The change-set explicitly distinguishes Commands from Events and distinguishes observations, assertions, and confirmations. Older Main Blueprint material may use the word "action" broadly, conflating these categories.

Resolution. Preserve existing action terminology where it is historical or API-facing, but make the new canonical semantic distinction authoritative for event-sourced state reconstruction and audit. Commands are requests to change state; Events are records of state changes that occurred; observations, assertions, and confirmations are distinct evidence classes. Older "action" wording is read through this canonical distinction.

Integration Impact. All Main Blueprint "action" terminology is preserved. The change-set Command / Event / observation / assertion / confirmation distinction is preserved in the amendment body and provides the authoritative semantic framework for state reconstruction and audit.

A-21 — Recovery vs Rollback

Conflict / Ambiguity. The change-set says recovery replaces rollback. The Main Blueprint includes configuration rollback and module rollback mechanisms, which could superficially appear to be retracted.

Resolution. Differentiate safe technical rollback of an unactivated module or configuration from historical transaction rollback. Transactional history is never erased; recovery uses compensating events and execution rather than retroactive deletion. Technical module rollback remains allowed under version-control safeguards. The change-set "recovery replaces rollback" rule applies to transactional state, not to unactivated technical configurations.

Integration Impact. The Main Blueprint configuration-rollback and module-rollback mechanisms are preserved. The change-set recovery model is preserved in the amendment body and provides the authoritative transactional-recovery semantics.

A-22 — Proof-of-Reserves Wallet Examples

Conflict / Ambiguity. The change-set describes wallet / bank read-only reserve proofs. The Main Blueprint is non-custodial, which could superficially appear to be contradicted by wallet/bank proof flows.

Resolution. The proof flow remains verification-only. SGTX never controls the bank or wallet credentials and never moves reserve assets as part of proof generation. Read-only proofs are generated by observing attestations and balances through credentials that remain under the control of the owning institution; SGTX stores only the resulting evidence metadata.

Integration Impact. The Main Blueprint non-custodial principle is preserved. The change-set proof-of-reserves wallet / bank read-only flow (Add-On 7) is preserved in the amendment body and is now read as verification-only, fully consistent with non-custody.

A-23 — Legal Specificity of Jurisdiction Examples

Conflict / Ambiguity. The change-set supplies country-specific bond and customs examples (for example, Egyptian bond structures and customs procedures). The Main Blueprint is intended to be jurisdiction-neutral, which could superficially appear to be contradicted by the country-specific examples.

Resolution. Retain the country-specific examples as configuration / examples in the source change-set, but make runtime applicability governed by the active jurisdiction and regulatory profile and the current authoritative source version, rather than by hard-coded universal law. The examples illustrate how the abstract capability is realised in a specific jurisdiction; they do not universalise that jurisdiction's law.

Integration Impact. The Main Blueprint jurisdiction-neutral design is preserved. The change-set country-specific examples (Add-On 8 Customs Bond, Add-On 28 GRiRE, and others) are preserved verbatim in the amendment body and are now read as illustrative configuration governed by the active jurisdiction profile.

A-24 — Incomplete Appendage of the Change-Set (Add-Ons 9-28 and Final Summary Preamble) [NEW v13.0 FINDING]

Conflict / Ambiguity. This is a new finding identified during the v13.0 re-audit. The v12.0 source document asserts, in its Preservation Note and Integration Method, that "the complete Updates / Modifications / Edits text is appended as an integrated canonical amendment body." However, a line-by-line verification of the v12.0 appended amendment body against the source RTF revealed that only Add-Ons 1 through 8 (PARTs 1-8 of the change-set) and the tail of the Final Summary (from article 101. FINAL EVIDENCE through article 162. IMPLEMENTATION MAPPING) were actually present. Add-Ons 9 through 28 (PARTs 9-28) and the preamble of the Final Summary (the add-on coverage table and articles 0 through 100 of the SGTX Master Global Trade Completion Amendment) were absent. The v12.0 audit header therefore overstated the completeness of the appended change-set.

Resolution. In accordance with the canonical precedence rule and the "never delete, only add and expand" integration discipline, the v12.0 appended amendment body is preserved exactly as it was received (no existing paragraph or table is removed or rewritten). The missing segment — RTF lines 751 through 4680, comprising Add-Ons 9-28 and the Final Summary preamble — has been re-integrated into this v13.0 master document in its canonical position, immediately preceding the existing article "101. FINAL EVIDENCE". This restores the complete change-set sequence (Add-Ons 1-8, then Add-Ons 9-28, then the Final Summary in full) while preserving every line of the v12.0 source. The insertion point is clearly marked in the amendment body with an integration banner.

Integration Impact. The complete SGTX Platform Complete Add-Ons Specification (Add-Ons 1-28 plus the Final Summary / Master Global Trade Completion Amendment) is now present in full in the Integrated Amendment Body. Specifically, the following twenty add-ons, previously absent from the v12.0 appended

body, are now integrated: Add-On 9 (Demurrage and Detention Management), Add-On 10 (Broker Liability and Insurance Management), Add-On 11 (Customs Valuation Intelligence), Add-On 12 (Cold Chain Quality Management), Add-On 13 (Inspection Agency Accreditation), Add-On 14 (Currency Risk Management), Add-On 15 (Government API Sandbox), Add-On 16 (FTA Preference Management), Add-On 17 (Piracy and Security Risk Engine), Add-On 18 (Trade Compliance Calendar), Add-On 19 (Cargo Insurance Integration), Add-On 20 (Trade Finance Documentation), Add-On 21 (Back-to-Back LC Management), Add-On 22 (Force Majeure Handling), Add-On 23 (Shipper's Declaration and Export Documentation), Add-On 24 (Port and Terminal Integration), Add-On 25 (Payment Guarantee Confirmation), Add-On 26 (Demurrage Dispute Resolution), Add-On 27 (Reserved), and Add-On 28 (Global Regulatory Intelligence and Requirements Engine, GRiRE), together with the Final Summary preamble (the all-add-on coverage table and articles 0 through 100 of the Master Global Trade Completion Amendment).

A.2 Audit Conclusion

The audit identifies twenty-four findings (A-01 through A-24). Findings A-01 through A-23 resolve semantic and terminological tensions between the Main Blueprint and the change-set without deleting any material from either source. Finding A-24 corrects a completeness gap in the v12.0 appended amendment body by re-integrating the previously missing Add-Ons 9-28 and Final Summary preamble. The resulting master document is a complete, self-contained, audited, and integrated baseline. The original v12.0 audit header is preserved unchanged in Part B below; the complete Main Blueprint body is preserved unchanged in Part C-1; and the now-complete Integrated Amendment Body (with the v13.0 re-integrated segment) is preserved in Part C-2.

End of Part A — Contradiction & Consistency Audit (v13.0 Re-audit). The preserved v12.0 document follows.

PART B — PRESERVED v12.0 MASTER BLUEPRINT (ORIGINAL CONTENT, UNCHANGED)

END OF AUDIT — MAIN BLUEPRINT FOLLOWS

The original Main Blueprint content is preserved in the master document exactly at the paragraph-text level as extracted from the uploaded DOCX. The integrated amendment body is preserved line-by-line from the uploaded change-set extraction, with only document-formatting wrappers and the audit preface added.

PRESERVATION NOTE

5. No original section is deleted. No original feature is silently removed. Where a feature is restricted by a later rule, the restriction is expressed as a scope/authority condition.
4. Because the update contains both add-ons and later master constitutional amendments, later amendment language is interpreted as the governing clarification for semantics, while earlier Main Blueprint text remains retained for traceability and compatibility.
3. The complete Updates / Modifications / Edits text is appended as an integrated canonical amendment body, preserving its section numbering and detailed technical material.
2. The audit above establishes the canonical interpretation where wording differs.
1. The Main Blueprint is preserved in full as the historical/base architectural body.

INTEGRATION METHOD

Resolution: Retain them as configuration/examples in the source change set, but make runtime applicability governed by the active jurisdiction/regulatory profile and current authoritative source version rather than hard-coded universal law.

Potential conflict / ambiguity: The update supplies country-specific bond and customs examples.

A-23 — Legal specificity of jurisdiction examples

Resolution: The proof flow remains verification-only. SGTX never controls the bank/wallet credentials or moves reserve assets as part of proof generation.

Potential conflict / ambiguity: Update describes wallet/bank read-only reserve proofs; Main Blueprint is non-custodial.

A-22 — Proof-of-reserves wallet examples

Resolution: Differentiate safe technical rollback of an unactivated module/configuration from historical transaction rollback. Transactional history is never erased; recovery uses compensating events/execution. Technical module rollback remains allowed under version-control safeguards.

Potential conflict / ambiguity: The update says recovery replaces rollback; the Main Blueprint includes configuration rollback and module rollback mechanisms.

A-21 — Recovery vs rollback

Resolution: Preserve existing action terminology where it is historical/API-facing, but make the new canonical semantic distinction authoritative for event-sourced state reconstruction and audit.

Potential conflict / ambiguity: Update explicitly distinguishes Commands from Events and observations/assertions/confirmations; older material may use “action” broadly.

A-20 — Command vs Event semantics

Resolution: Use the updated deployment-state vocabulary to distinguish internal architectural readiness from external operational readiness. Do not infer a working production connector merely because its adapter exists.

Potential conflict / ambiguity: Main Blueprint includes broad “production-ready” language; update establishes CORE_READY through PRODUCTION_CONNECTED and legal-authorization states.

A-19 — Global readiness claims

Resolution: Manual fallback is permitted only through the same authentication, authorization, attribution, timestamps, evidence, hashing, Loom logging, and applicable Governor gates as automated paths.

Potential conflict / ambiguity: Update introduces API→EDI→SFTP→Portal→Broker→Manual fallback.

A-18 — Manual fallback vs automated governance

Resolution: AI may explain, translate, classify, detect, predict, summarize and escalate within its assigned tier. Deterministic policies, authorised institutions and human decisions remain authoritative for material state.

Potential conflict / ambiguity: Main Blueprint uses AI-generated tenant messages and risk flags; update tightens authority semantics.

A-17 — AI-generated explanations vs authoritative decisions

Resolution: Retain Marketplace Partner as an external integration participant only. No public marketplace discovery, autonomous provider selection, counterparty matching, or ranking is part of SGTX.

Potential conflict / ambiguity: Main Blueprint supports Marketplace Partner API integration while repeatedly saying SGTX is not a marketplace; the update reinforces execution-only architecture.

A-16 — Marketplace terminology

Resolution: Interpret zero-cost as no required proprietary software licence, paid cloud subscription, or mandatory commercial AI API dependency. It does not mean zero cost for banks, certifications, legal agreements, customs bonds, audits, credentials, hardware, or regulated operations.

Potential conflict / ambiguity: Main Blueprint promises open-source/self-hosted technology; updates introduce audits, certifications, credentials and government/legal requirements.

A-15 — “Zero-cost” technology vs real institutional costs

Resolution: Clarify that Egypt production replication and disaster recovery remain within Egypt. International sovereign nodes do not imply unrestricted replication of Egypt-regulated data; cross-border sharing is minimised and legally authorised.

Potential conflict / ambiguity: Main Blueprint includes both strict Egypt localisation and an implementation-readiness line describing multi-region mirroring.

A-14 — Egypt data localisation vs “multi-region mirroring”

Resolution: Treat ISO 20022 as preferred banking interoperability and translation format. Canonical SGTX state/events remain independent of ISO message schemas.

Potential conflict / ambiguity: Main Blueprint supports ISO 20022 settlement/reconciliation outputs; update says ISO 20022 is an external interoperability standard, not the canonical SGTX state model.

A-13 — ISO 20022 as integration output vs canonical data

Resolution: Retain USTN as SGTX-wide correlation and lineage identifier. Clarify that USTN cannot override a government, bank, carrier, customs or other domain-authoritative identifier/state.

Potential conflict / ambiguity: Main Blueprint treats USTN as universal reference; update says USTN is a canonical namespace, not universal external authority.

A-12 — USTN as universal reference vs external authority

Resolution: The update controls the semantic meaning of closure. Existing closure material remains, but authoritative closure is only true when the canonical closure policy evaluates all required conditions and evidence.

Potential conflict / ambiguity: Main Blueprint contains closure workflows; the update requires canClose, seven closure conditions, policy-derived closure, and no contradictory authoritative state.

A-11 — Closure semantics

Resolution: Retain all existing Nafeza material, while adding an Egypt Government Adapter with mode-specific applicability and authority mapping so a Nafeza feature is never assumed to apply to every mode.

Potential conflict / ambiguity: Main Blueprint has a strong Nafeza integration; update requires export/import/transit and mode-aware applicability.

A-10 — Egypt Nafeza as one generic integration vs mode-specific applicability

Resolution: Use the update as the canonical transport extension: Road, Air, Ocean Container, RoRo/Rolling Cargo, Rail, Ferry and future modes are distinct transport engines under one transport orchestrator and USTN graph.

Potential conflict / ambiguity: Earlier blueprint material contains ocean/container and multimodal concepts, while the update explicitly makes RoRo first-class.

A-09 — RoRo as ocean submode vs first-class mode

Resolution: Retain the original seven as historically/constitutionally referenced. Expand the add-on catalogue rather than renaming or deleting them; Add-On 8 and subsequent modules become additional canonical extensions.

Potential conflict / ambiguity: Main Blueprint says seven critical add-ons are integrated; the update includes at least eight major modules, including Customs Bond & Guarantee Management, plus a much larger global architecture.

A-08 — “Seven critical add-ons” vs expanded add-on set

Resolution: Keep trust graph as relationship analytics for parties already known to the tenant. It may score network trust and exposure, but may not discover, recommend, rank, or introduce counterparties.

Potential conflict / ambiguity: Main Blueprint forbids counterparty recommendation/ranking; the update adds a positive institutional trade graph.

A-07 — GNN/institutional graph and non-marketplace safeguards

Resolution: No conflict requiring replacement. The update operationalises the existing 110% rule with attestation/ZK evidence. The pre-existing constitutional rule remains unless separately amended by the constitution.

Potential conflict / ambiguity: Main Blueprint has immutable minimum 110% backing and composition rules; update ZK reserve flow also uses $\geq 110\%$.

A-06 — Reserve ratio thresholds

Resolution: Clarify that reserve tables store attestations, ratios, commitments and evidence metadata only. They do not create customer-fund custody, omnibus balances, or a deposit account inside SGTx.

Potential conflict / ambiguity: Main Blueprint says no fund tables; the update introduces proof-of-reserves and platform_reserves metadata.

A-05 — Reserve metadata vs custody

Resolution: Keep DeFi/stablecoin material as conditional, jurisdiction-permitted financing/rail capability, not as the canonical settlement authority. Bank and regulated financial institution systems remain authoritative for money movement and settlement confirmation.

Potential conflict / ambiguity: Main Blueprint contains optional stablecoin/DeFi financing references; the update centers settlement on connected banks and bank-authoritative financial state.

A-04 — Stablecoin / DeFi references vs bank-authoritative settlement

Resolution: Preserve non-custody and PSP/bank execution flows. Add the rule that non-custody is architectural, not a standalone legal classification; actual functionality determines licensing/classification in each jurisdiction.

Potential conflict / ambiguity: Main Blueprint uses PSP split workflows and states SGTx never holds funds; the update adds Bank Settlement Gateway, bank authority boundaries, and capability-based regulatory classification.

A-03 — Non-custody vs payment/settlement functionality

Resolution: Clarify that A4 is deterministic policy automation executed by Governor + OPA + WasmEdge under pre-authorised rules. AI itself never acquires independent execution authority. This preserves A4 low-risk automation while making the constitutional boundary explicit.

Potential conflict / ambiguity: Main Blueprint uses “A4 Governance / auto-executes within constitutional bounds,” while the update explicitly states AI never has independent authoritative decision rights.

A-02 — AI A4 authority language

Resolution: Retain the four direct Egypt integrations as the initial/reference direct-integration set. Add the worldwide adapter/gateway fabric as the extensibility layer. “Direct API” therefore means a currently adopted first-party connector, not a limitation on the global adapter model.

Potential conflict / ambiguity: Main Blueprint states SGTX directly connects to only four government APIs (Nafeza, CargoX, ETA, CBE); the update establishes a worldwide country-adapter and multi-agency gateway architecture.

A-01 — Direct government API scope

CONFLICT / AMBIGUITY REGISTER

Canonical precedence rule: The Main Blueprint remains fully preserved. Where the Updates / Modifications / Edits document expressly corrects, extends, or redefines architectural meaning, the update is treated as the later accepted amendment. Earlier language is retained and is interpreted through the newer clarification.

Audit method: Every logical paragraph in the Main Blueprint and every extracted line in the Updates / Modifications / Edits file was included in the consolidation process. Original material is retained. Material conflicts are resolved by explicit precedence/clarification language, never by silent deletion.

Source	SHA-256	(raw	RTF):
87181d220df82a485485eec4b9896031910c79afa6332516ef2afac4682c5b72			

Change-set text lines extracted from source: 7,582

Source 2: sgtx add ons and modifications.rtf

Source	SHA-256	(logical	text):
c263f527f3966dab01d3cffc87e7d2747d01c017ca7a786d1964f09018087d42			

Logical paragraph lines extracted from source: 76,122

Source 1: SGTX PLATFORM BLUEPRINT(4).docx

CONTRADICTION & CONSISTENCY AUDIT

Status: AUDITED / INTEGRATED / MASTER BASELINE

Version 12.0 — Main Blueprint + Updates / Modifications / Edits

SGTX PLATFORM BLUEPRINT — MASTER INTEGRATED EDITION

SGTX PLATFORM BLUEPRINT — PREAMBLE

SOVEREIGN GOVERNED TRADE EXECUTION

SGTX (Sovereign Governed Trade Execution) is a non-custodial, AI-governed, sovereign trade execution engine — an operating system for global trade, not a marketplace.

It provides the infrastructure for organisations that already know each other to execute cross-border trade with cryptographic certainty, AI-assisted optimisation, and zero counterparty risk through non-custodial FeeLock protection.

The platform never holds funds, never takes title to goods, never brokers introductions, and never recommends any specific client, service provider, or counterparty. All relationships are established outside SGTX and then onboarded by the users themselves.

THE SEVEN UNSHAKABLE PILLARS

GLOBAL TRADE IDENTITY (GTID)

Format: SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Example: SGTX-EG-26-F3A-1

Entity types (user-selectable during onboarding):

Resolution endpoint:

GET /v1/gtid/resolve?gtid=... returns only information the tenant has consented to share (legal name, type, jurisdiction, trust score, KYB tier, sanctions clearance, lifecycle state).

UNIVERSAL SHIPMENT TRACKING NUMBER (USTN)

Format:

SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Example: SGTX-EG-26-F3A-1

BUYER_SUFFIX: last 6 alphanumeric of buyer GTID (after removing hyphens)

SELLER_SUFFIX: last 6 alphanumeric of seller GTID

TIMESTAMP: UTC timestamp of contract/shipment lock

SEQ: 8 random alphanumeric characters (34-char alphabet excluding I,O,0,1)

Why company-anchored? Prevents replay attacks across different counterparties. A USTN from trade A cannot be used to impersonate trade B because the buyer/seller suffixes differ.

Generation: Automatic at contract lock (single-shipment) or per-shipment lock (multi-shipment). No user input required.

AI AUTHORITY LADDER (THREE-TIER, ZERO-COST, ASSISTIVE ONLY)

Key constraint: AI never suggests a counterparty, ranks providers, or offers "you may also like". All assistance is limited to data entry optimisation, regulatory compliance, risk detection, and workflow automation for relationships users have already established.

GLOBAL FEE MODEL & NON-CUSTODIAL SETTLEMENT

Trade Fee: 1.5% of invoice value – paid by each country's side (exporter or importer depending on incoterm). Collected via PSP split – SGTX never holds funds.

Financing Fee: Flat 0.25% of financed amount – deducted from disbursement via PSP split. Paid by borrower.

Optional Service Fees (broker, lab, QC): Added to trade total; collected via PSP split; provider receives net after 3% platform fee.

Takaful Protection (optional): 0.05% of financed amount – Sharia-compliant mutual pool covering stablecoin de-peg, liquidity disruption, and reserve shortfall.

Example – Egyptian exporter sells \$100,000 to German importer:

Egyptian side pays 1.5% = \$1,500 (collected at contract lock)

German importer (if Germany adopts SGTX) pays 1.5% = \$1,500

Broker certification fee \$150 paid by seller

Total infrastructure + service fees = \$3,150 – still less than traditional demurrage, document discrepancies, and financing costs.

No hidden fees, no variable "marketplace commissions" – just a fixed, transparent, constitutional percentage.

ZERO-COST TECHNOLOGY COMMITMENT

Every line of code, every microservice, every AI model, every deployment artefact is built exclusively on:

Open-source software – Rust, PostgreSQL, NATS, WasmEdge, OPA, ClickHouse, etc.

Free public APIs & data sources – AIS, OpenStreetMap, FAO, ECB, etc.

Self-hosted infrastructure – bare-metal K3s clusters, no cloud dependency.

Local AI inference – Hugging Face free tier, vLLM, Ollama, all run on your own server.

No billing details are ever required. The platform is fully operational without a single paid licence, cloud subscription, or external monetary dependency.

DATA LOCALISATION MANDATE (EGYPT)

All production data for Egyptian operations must reside exclusively on physical bare-metal servers located within the Arab Republic of Egypt. This includes:

PostgreSQL databases (all tenant data, trade records, audit logs)

NATS JetStream (FeeLock state, event streams)

Loom hash chain storage

ClickHouse (analytics, long-term audit)

All backups (primary and secondary)

Prohibited:

Cloud storage (AWS, Azure, GCP, or any foreign cloud)

Data replication outside Egypt

Use of foreign data centres for disaster recovery

Exception: Cross-border trade data legally required to be shared with foreign counterparties (e.g., EU importers) may be transmitted, but must be minimised and subject to explicit consent.

Enforcement: The geo-fencing service monitors all outbound data traffic and blocks any attempt to store or replicate Egyptian data outside the country. Violations trigger a P0 incident and immediate notification to the Platform Governance Authority.

NON-MARKETPLACE SAFEGUARD

SGTX never:

Suggests "you might also like" or "people also traded with"

Ranks tenants or service providers

Introduces strangers

Offers unsolicited business opportunities

The platform only works with existing, known relationships that users explicitly bring. All discovery is external; SGTx provides the execution layer, not the matchmaking layer.

IMPLEMENTATION READINESS SUMMARY

The SGTx Preamble establishes the foundation: a sovereign, non-custodial, AI-governed trade execution engine that serves as the invisible rails for global trade – with visible trust, cryptographic certainty, and zero-cost infrastructure. Every subsequent part builds on these principles.

PART 1 — CONSTITUTIONAL & GOVERNANCE LAYER

↓

Sign (Ed25519 + optional Dilithium3)

↓

Loom Log (deterministic hash chain)

↓

Store in `governor\_decisions` (PostgreSQL)

↓

NATS Event (if needed)

↓

Execute Action (if ALLOW)

Example Request to Governor (Internal):

json

POST /v1/governor/decision

```
{
  "action": "contract.sign",
  "actor_gtid": " SGTX-EG-26-F3A-1",
  "actor_employee_id": "emp-001",
  "active_trader_mode_context": "BUY",
  "resource": { "ustn": " SGTX-EG-26-F3A-1" },
  "payload": { "signature": "base64..." }
}
```

Example ALLOW Response:

json

```
{
  "decision_id": "dec-abc123",
  "verdict": "ALLOW",
  "loom_hash": "sha256:a1b2c3...",
  "cryptographic_signature": "ed25519:..."
}
```

Example CONDITIONAL Response (with AI-generated tenant message):

json

```
{
  "decision_id": "dec-def456",
  "verdict": "CONDITIONAL",
  "tenant_message": {
```

```

"summary": "Your contract lock is on hold because the seller's jurisdiction requires enhanced KYC documentation.",
"conditions": [
{
"condition_id": "kyc_tier2_seller",
"label": "Seller KYB Tier 2 verification",
"status": "unmet",
"action_url": "/v1/kyc/upgrade?gtid=SGTX-VN-TRD-000456"
},
{
"condition_id": "doc_cert_origin",
"label": "Certificate of Origin from Vietnam",
"status": "unmet",
"action_url": "/v1/documents/upload?trade=..."
}
],
"escalation_hint": "If you believe this is an error, you can request a human review. A compliance officer will respond within 24 hours."
},
"loom_hash": "sha256:d4e5f6..."
}

```

AI assistance for Governor: When a user encounters a CONDITIONAL or DENY, the Decision Panel automatically shows the plain-language explanation and, where possible, provides autocomplete-ready action links (e.g., "Upload document" pre-filled with the required document type). No unsolicited recommendations.

1.2 OPA Policy Engine (Rego)

OPA evaluates business rules and permissions. Policies are stored as .rego files in /core/governor/policies/ and can be hot-reloaded after multisig approval. The Governor passes the request context (actor, action, resource, trader mode) to OPA, which returns a verdict and optional conditions.

Example Policy — Contract Signing with Dual-Mode Context Enforcement:

```

rego

package sgtx.permissions

default allow = false

# Trader with BUY mode can sign contract as buyer
allow {
  input.action == "contract.sign"
  input.actor_trader_mode == "BUY"
}

```

```

role_has_permission(input.actor_role, "contract.sign")
not resource_already_signed_by_buyer(input.resource.ustn)
}

# Trader with SELL mode can sign contract as seller
allow {
input.action == "contract.sign"
input.actor_trader_mode == "SELL"
role_has_permission(input.actor_role, "contract.sign")
not resource_already_signed_by_seller(input.resource.ustn)
}

```

AI-Assisted Policy Authoring (Admin Portal): The Governor provides a Rego policy editor with syntax highlighting, autocomplete for common rule patterns, and a natural language prompt (e.g., "Create a rule that requires two managers to approve contracts over \$100k"). The AI (Groq) generates a draft policy, which the admin can then refine and test via impact simulation.

Policy Categories (Non-Marketplace):

OPA Testing: `opa test /core/governor/policies/` runs in CI. All policies must pass before deployment.

1.3 WasmEdge Constitutional Engine

1.3.1 Overview

The WasmEdge Constitutional Engine enforces non-negotiable, immutable rules that cannot be bypassed by any user, administrator, or even the Governor itself. These rules are implemented as sandboxed WebAssembly (WASM) modules, pre-compiled, signed by the Platform Governance Authority (multisig), and executed by the WasmEdge runtime. The modules have no network or filesystem access — they receive only a strict input schema via the Governor and return a deterministic verdict.

All constitutional modules are zero-cost, open-source, and hot-loaded without restarting the Governor service.

1.3.2 Core Constitutional Modules

1.3.3 Constitutional Reserve Rules (Immutable)

Module: `reserve_rules.wasm`

Purpose: Enforces the financial backbone of the platform — reserve composition, gold weighting, minimum backing, and entry/exit rules.

Immutable Rules:

Adjustable Parameters (not immutable, changeable by multisig):

Fee rates (within 0.1%–2.5% bounds — SGTX uses 1.5% fixed)

Stablecoin whitelist — added/removed by RIA + multisig (e.g., USDC, USDT, EURC)

Technical parameters — timeouts, retry limits, rate limits (Admin Portal only)

Amendment Process for Immutable Rules:

Proposal submitted via Admin Portal → creates Smart Inbox item for all multisig members.

3/5 members approve → 30-day public notice period begins (posted on status page and emailed to all tenants).

After 30 days, a new WASM module is compiled, signed by multisig, and deployed via hot reload.

The old module is archived and its Loom hash recorded permanently.

Example — Reserve Composition Check (Pseudocode):

rust

```
fn check_reserves(
  usd_percent: f64,
  eur_percent: f64,
  gold_percent: f64,
  other_percent: f64,
  backing_ratio: f64,
) -> Verdict {
  if usd_percent != 50.0 || eur_percent != 25.0 || gold_percent < 15.0 || other_percent > 10.0 {
    return Verdict::Deny { reason: "Reserve composition violates constitution." };
  }
  if backing_ratio < 1.10 {
    return Verdict::Deny { reason: "Insufficient reserves (below 110% backing)."; };
  }
  Verdict::Allow
}
```

1.3.4 Module Execution & Sandboxing

Each WASM module:

Is compiled to WASI 0.2 (no filesystem, no network, no environment variables).

Receives input as a JSON-serialised struct.

Returns a verdict: ALLOW, DENY, or CONDITIONAL with a machine-readable reason code.

Must complete execution within 50ms (hard timeout); otherwise the Governor treats it as DENY and logs a constitutional violation.

1.3.5 Hot Reloading & Versioning

The Governor polls a well-known NATS subject (constitutional.modules.update) every 60 seconds.

When a new module version is published (signed by multisig), the Governor downloads it, verifies the signature, and replaces the in-memory instance without restart.

The old module is retained for 7 days (rollback capability). A Loom event records every module change.

If a module fails to load (e.g., hash mismatch), the Governor continues using the last known good module and raises a P2 incident.

1.3.6 Testing & Verification

Every WASM module is unit-tested in CI using wasmtime test.

User-Facing Verification: Any tenant (or external auditor with a verification token) can download the Loom chain for a specific USTN and verify it using the open-source loom-verify tool. The tool shows a green checkmark if the chain is intact, or a red warning if tampering is detected.

1.7 Jurisdiction Supremacy & RIA Integration

SGTX always applies the strictest rule among:

Buyer country

Seller country

Logistics execution country

Financier country

Governing law

The Regulatory Intelligence Agent (RIA) scrapes 300+ official sources (OFAC, UN, EU, national lists) every 15 minutes, updates the jurisdictions table, and compiles a new jurisdiction_matrix.wasm module. The Governor loads the new module without restart.

Jurisdiction Tiers (dynamically updated by RIA):

Example — Jurisdiction Check in Governor (Pseudocode):

rust

```
let buyer = get_tenant_country(decision.buyer_gtid);
```

```
let seller = get_tenant_country(decision.seller_gtid);
```

```
let verdict = jurisdiction_module.check(buyer, seller);
```

```
if verdict == "DENY" {
```

```
    return GovernorVerdict::Deny { reason: "Sanctioned jurisdiction — trade cannot proceed." };
```

```
}
```

RIA-Driven UI Assistance: When a user selects a country in a form, the system autocompletes from the list of allowed countries (filtered by current jurisdiction matrix). If the country is restricted, an inline warning appears: "This jurisdiction is currently Restricted. Enhanced due diligence required." But SGTX never suggests "maybe trade with a different country".

1.8 Cryptographic Signing & Qualified Electronic Signature (QES)

1.8.1 Signature Types Hierarchy

Legal basis: Egyptian E-Signature Law No. 15 of 2004, Article 13 — QES has the same legal effect as handwritten signature.

1.8.2 Mandatory Qualified Signature Scenarios

1.8.3 Egypt Trust Integration Architecture

Qualified Signature Workflow (Textual):

text

User → SGTX Platform → Egypt Trust API → Signer's TSP (e.g., Misr)

■ ■ ■ ■

■ ■ ■ ■

▼ ▼ ▼ ▼

User completes identity verification (in-person or video call depending on TSP).

TSP issues QES certificate to user's HSM or cloud wallet.

User provides SGTX with public key or certificate reference.

SGTX stores certificate reference in `tenants.qes_certificate_ref`.

1.8.6 Hybrid Signing Fallback

If QES is unavailable (TSP downtime, user not enrolled), the system supports hybrid signing:

1.9 Device Trust, Session Security & Step-Up Authentication

1.9.1 Device Trust Registry

Every authenticated device receives a persistent Device Trust Record.

Database table: `device_registry` (add to Part 13 DDL)

Device States: NEW, TRUSTED, ELEVATED_RISK, BLOCKED, REVOKED

Device Trust Rules:

New Device: Passkey required + MFA required + email confirmation required.

High Risk Device (risk score >70): Manual approval required from tenant admin.

Blocked Device: Access denied; must be explicitly unblocked.

1.9.2 Device Management Center

Location: Company Admin → Security → Devices

User capabilities: view devices, rename, revoke, force logout, view login history, export security report.

One-click actions: "Revoke Device", "Force Logout", "Export Report"

1.9.3 Step-Up Authentication

Required for high-value actions: contract signing, payment approval, settlement release, bank/PSP changes, UBO changes, approval policy changes, Governor policy changes, and dual-mode switching immediately before a high-value action.

Step-Up Authentication Chain (all three required):

Passkey (WebAuthn) — something you have

Biometric verification — something you are

Cryptographic challenge signature — device-bound private key

1.9.4 Session Risk Engine

Continuous monitoring (A2 anomaly detection) for:

Impossible travel

VPN detection

TOR detection

Country mismatch

Device fingerprint change

Behavioural anomaly

Governor verdicts: ALLOW, REQUIRE_REAUTH, LOCK_SESSION, ESCALATE

Database addition: session_risk_events (add to Part 13 DDL) — all risk events are Loom-anchored.

1.9.5 Legal Recovery Flow for Lost Passkeys

If a user loses all registered passkeys:

Identity verification — Notarised ID + employment proof + two authorised signatories.

Platform Governance Authority review — Multi-sig approval (3/5).

Recovery code delivery — Registered mail to tenant's registered address; in-person verification at a designated centre (e.g., Egypt Post).

New device registration — All previous devices revoked.

Audit trail — Logged in governor_decisions with decision_type = 'PASSKEY_RECOVERY'.

1.10 Court Evidence Package Engine

1.10.1 Purpose

Automatically generate court-ready, cryptographically verifiable evidence bundles for disputes, arbitration, and litigation.

1.10.2 Evidence Package Contents

Contract (full clauses, signatures)

Signatures (Ed25519 and QES where applicable)

Loom chain (full hash chain for the USTN)

Audit logs (all Governor decisions)

Payment logs (fee payment requests, settlement instructions, PSP confirmations)

Communication logs (Trade Room messages, mediation log)

Document hashes (SHA256 of all uploaded/generated documents)

Milestone timeline (all confirmed milestones with timestamps and confirmers)

Sensor data (temperature, humidity, shock logs with anomaly flags)

QC report (full inspection, overrides with original AI confidence and inspector's reason)

Causal analysis (root cause attribution from Add-on 3)

1.10.3 Supported Formats & Jurisdictions

1.10.4 One-Click Generation

Trigger points:

Dispute tab — "Generate Evidence Package"

Trade Command Center — "Export → Court Evidence"

Government Portal — "Package for Prosecution"

API endpoint: POST /v1/evidence/package

Security: Encrypted with one-time password; Loom-hashed before delivery.

1.11 Compliance Intelligence Layer (Unified Screening Gateway)

1.11.1 Purpose

Single evaluation point for all compliance checks before any trade, financing, or distressed action.

1.11.2 Screening Dimensions

1.11.3 Output Verdicts

1.11.4 Integration with Governor

Called synchronously for every trade.initiate, financing.request, and distressed.declare.

Returns verdict as part of Governor pre-screen.

Decision Panel displays the verdict with remediation steps.

1.11.5 Override & Appeal

Human override allowed only with multisig approval (3/5 for BLOCKED, 1/3 for EDD).

Override reason stored immutably and flagged for quarterly audit.

1.11.6 API Endpoint (Internal)

POST /v1/compliance/screen — accepts GTID, HS code, jurisdiction, returns verdict and conditions.

1.12 Automated Suspicious Activity Report (SAR) Generation

1.12.1 Purpose

Automatically detect trade patterns that may indicate money laundering, sanctions evasion, or other financial crimes, and generate a Suspicious Activity Report (SAR) draft in the format required by the relevant Financial Intelligence Unit (FIU).

AI Authority: A2 (Isolation Forest + rule-based for detection), A1 (Groq for narrative generation), A4 (Governor for logging and access control).

1.12.2 Detection Triggers

1.12.3 SAR Drafting & Review Workflow

Detection: sar-detector agent (A2) compiles evidence.

Narrative Generation: A1 (Groq) generates plain-language description.

Draft Creation: Record inserted into suspicious_activity_reports with status = 'DRAFT'.

Smart Inbox Alert: High-priority item (priority 95) sent to compliance officer and Platform Governance Authority.

Review: Compliance officer reviews, edits, and approves.

Filing: If FIU supports electronic filing, system authenticates and submits via API. Otherwise, PDF generated for manual submission.

1.12.4 Database Schema

sql

```
CREATE TABLE suspicious_activity_reports (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  report_type TEXT NOT NULL, -- 'FinCEN', 'EG_AML', 'EU_ECB'  
  detection_rule TEXT, -- 'volume_spike', 'circular_trade', 'value_mismatch', etc.  
  involved_ustns TEXT[], -- Array of USTNs  
  parties JSONB, -- { buyer_gtid, seller_gtid, carrier_gtid, ... }  
  narrative TEXT, -- Groq-generated plain-language description
```



```

draft_status TEXT DEFAULT 'DRAFT', -- 'DRAFT', 'FILED', 'REJECTED'
filed_at TIMESTAMPTZ,
filing_reference TEXT, -- Reference from FIU (if available)
governor_decision_id UUID,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

1.12.5 Governance & Audit

Every SAR generation, edit, and filing is logged in `audit_log`.

The Loom hash of the final report is anchored in the blockchain (optional, if Add-on 6 enabled).

The Platform Governance Authority can review all SARs (read-only) and can mark a draft as REJECTED (with reason) if the detection was a false positive.

1.13 Public Loom Verification Endpoint for Government

Purpose: Enable government auditors and external verifiers to cryptographically verify the integrity of a specific trade's Loom hash chain without accessing the full platform or viewing sensitive trade data.

Endpoint: GET `/v1/verify/loom`

Authentication: Not required (public endpoint), but rate-limited (10 requests per minute per IP). A valid `verification_token` must be provided.

Query Parameters:

Response (200 OK):

json

```

{
  "ustn": "SGTX-EG-26-F3A-1 ",
  "genesis_hash": "sha256:abc123...",
  "latest_hash": "sha256:def456...",
  "chain_verified": true,
  "chain_length": 1247,
  "first_decision_timestamp": "2026-04-15T12:00:00Z",
  "last_decision_timestamp": "2026-06-07T10:00:00Z",
  "decision_hashes": [
    { "decision_id": "dec-001", "loom_hash": "sha256:...", "previous_hash": "sha256:...", "timestamp": "..."},
    ...
  ]
}

```

Token Generation & Database:

sql

```
CREATE TABLE loom_verification_tokens (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT NOT NULL REFERENCES shipments(ustn),  
  token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),  
  expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '90 days',  
  created_by UUID REFERENCES employees(id),  
  revoked BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

1.14 Integration with Add-Ons (Non-Marketplace)

The Governor calls add-on services synchronously for actions that require advanced risk assessment, but only to advise or constrain — never to recommend.

Non-Marketplace Safeguard: The GNN never suggests "find a different counterparty" — it only flags risk with the existing relationship.

1.15 Service Level & Recovery (User-Transparent)

Recovery: The Governor is stateless; multiple replicas run behind a load balancer. NATS JetStream persists FeeLock state. If a Governor replica crashes, others continue serving. Users are not impacted.

1.16 Implementation Checklist (Part 1)

1.16.1 Core Governor & OPA

Implement Governor service (Rust + Axum) with OPA evaluator.

Write Rego policies: permissions.rego, fee.rego, financing.rego, distressed.rego, multiship.rego, logistics.rego, broker.rego, reserve.rego.

Add hot-reload mechanism for OPA policies (NATS subscriber).

Add impact simulation for policy changes (Admin Portal).

Write unit tests for all policies (opa test).

1.16.2 WasmEdge Constitutional Engine

Implement constitutional_rules.wasm (fee bounds, A5 prohibition, multisig).

Implement jurisdiction_matrix.wasm (RIA-updated).

Implement incoterms_engine.wasm (incoterm validation).

Implement fee_gate.wasm (gross-up, split instructions).

Implement distressed_country_gate.wasm (country factor).

Implement dual_mode_gate.wasm (context enforcement).

Implement reserve_rules.wasm (reserve composition, 110% backing).

Add hot-reload mechanism (NATS, signature verification, Loom logging).

Write unit tests for each module (wasmtime test).

1.16.3 Tenant Message & Decision Panel

Implement tenant_message generation (A1, Groq → Ollama).

Build PlainLanguage Governor Decision Panel UI component (React).

Add static template fallback for when AI is unavailable.

1.16.4 Loom & Audit

Implement Loom hash chain library (Rust).

Add hourly audit-chain-verifier cron job.

Build public Loom verification endpoint (/v1/verify/loom).

Create loom_verification_tokens table.

1.16.5 Cryptographic Signatures & QES

Implement Ed25519 signing for all Governor decisions.

Set up SoftHSMv2 for platform Ed25519 key.

Integrate Dilithium3 for archival records (Add-on 6).

Implement QES layer (Egypt Trust API integration).

Add QES API endpoints (/v1/signature/qes/*).

Create user enrollment flow for QES during onboarding.

1.16.6 Device Trust & Session Security

Create device_registry table.

Implement device trust rules (NEW, TRUSTED, ELEVATED_RISK, BLOCKED, REVOKED).

Add Device Management Center to Company Admin.

Implement Step-Up Authentication for high-value actions.

Create session_risk_events table.

Add Session Risk Engine (A2 anomaly detection).

Implement legal recovery flow for lost passkeys.

1.16.7 Court Evidence Package

Implement evidence-compiler service (Rust).

Build evidence package generation endpoint (POST /v1/evidence/package).

Support formats: PDF, ZIP, Court Bundle, Arbitration Bundle.

Add Loom-hashing and one-time password encryption.

1.16.8 Compliance Intelligence & SAR

Implement Unified Screening Gateway (/v1/compliance/screen).

Build SAR detection triggers (Isolation Forest + rule-based).

Implement SAR draft generation (A1 Groq narrative).

Create suspicious_activity_reports table.

Build compliance officer review workflow (Smart Inbox).

Implement FIU filing integration (API or PDF).

1.16.9 Testing & Validation

Integration tests: Governor → OPA → WasmEdge → AI → Loom → storage.

Test QES hybrid fallback (TSP unavailable → advanced signature).

Test device trust: new device registration, step-up auth.

Test SAR detection and filing workflow.

Test public Loom verification endpoint.

Test constitutional reserve rules (110% backing enforcement).

1.17 AI Authority Summary for Part 1

END OF PART 1 — CONSTITUTIONAL & GOVERNANCE LAYER (v11.0 FULLY EXPANDED)

PART 2 — IDENTITY, TENANTS & INTERNAL AUTHORITY

2.0 Purpose & Design Philosophy

The Identity & Tenants layer is the foundation of trust in SGTX. Every organisation that uses the platform — whether a trader, logistics provider, laboratory, QC inspector, customs broker, financier, or government agency — is assigned a cryptographically verifiable Global Trade Identity (GTID) . The GTID is the permanent identifier for all actions, documents, and relationships.

Key principles (non-marketplace):

Self-sovereign — tenants control their own data; SGTX never shares GTID information without consent.

AI-assisted onboarding — autocomplete, document extraction (HF Donut), and real-time validation against government registries reduce manual data entry. But AI never suggests "you might also want to trade with X".

No unsolicited counterparty matching — the platform never recommends which tenants to connect with. All relationships are established outside SGTX and then explicitly added by users.

Flexible role hierarchy — supports dual-mode for traders (Buyer/Seller toggle), business units, approval chains, and granular data scopes (e.g., hiding cost components from warehouse staff).

All onboarding is zero-cost, self-hosted, and uses free government registry APIs (scraped) for KYB/KYC verification.

2.1.1 GTID Format Specification

2.1.1.1 Format Structure

text

GTID = "SGTX" + "-" + COUNTRY_CODE + "-" + ENTITY_TYPE + "-" + SEQUENCE + "-" + CHECKSUM

Full Format: SGTX-{COUNTRY_CODE}-{ENTITY_TYPE}-{SEQUENCE}-{CHECKSUM}

Example: SGTX-EG-26-F3A-1

2.1.1.2 Component Definitions

2.1.1.3 Total Length

text

$4 + 1 + 2 + 1 + 3 + 1 + 6 + 1 + 4 = 23$ characters

2.1.2 Entity Type Codes

The ENTITY_TYPE code determines the tenant's primary role on the platform and influences available features, portals, and permissions.

2.1.2.1 Core Entity Types

2.1.2.2 Financier Subtypes (FIN only)

2.1.2.3 Logistics Subtypes (LSP only)

2.1.3 Checksum Algorithm

2.1.3.1 Algorithm Specification

The checksum is a 4-hexadecimal-digit CRC32-ISO-HDLC of the concatenated fields (excluding the "SGTX-" prefix and the checksum itself).

Input string for checksum: {COUNTRY_CODE}{ENTITY_TYPE}{SEQUENCE}

Example:

Country Code: EG

Entity Type: TRD

Sequence: 002139

Input: EGTRD002139

CRC32-ISO-HDLC: 0x7F3A... → first 2 bytes 0x7F3A → checksum 7F3A

Algorithm (Rust):

```
rust
```

```
use crc32fast::Hasher;
```

```
fn calculate_checksum(country_code: &str, entity_type: &str, sequence: &str) -> String {
```

```
    let input = format!("{}", country_code, entity_type, sequence);
```

```
    let mut hasher = Hasher::new();
```

```
    hasher.update(input.as_bytes());
```

```
    let crc = hasher.finalize();
```

```
    // Take first 2 bytes (4 hex digits)
```

```
    format!("{:04X}", crc & 0xFFFF)
```

```
}
```

```
// Example: EGTRD002139 → CRC32 → 0x7F3A
```

2.1.3.2 Verification

```
rust
```

```
fn verify_gtid(gtid: &str) -> bool {
```

```
    let parts: Vec<&str> = gtid.split('-').collect();
```

```
    if parts.len() != 5 || parts[0] != "SGTX" {
```

```
        return false;
```

```
    }
```

```
    let country = parts[1];
```

```

let entity_type = parts[2];
let sequence = parts[3];
let provided_checksum = parts[4];
let calculated = calculate_checksum(country, entity_type, sequence);
calculated == provided_checksum
}

```

Governance: The Governor validates the checksum on every GTID resolution request. Invalid GTIDs are rejected with a 400 Bad Request response.

2.1.4 GTID Generation Algorithm

2.1.4.1 Generation Process

2.1.4.2 Database Sequence Management

```
sql
```

```
-- Tenant sequence counter
```

```
CREATE TABLE tenant_sequences (
country_code CHAR(2) NOT NULL,
entity_type CHAR(3) NOT NULL,
last_sequence INTEGER NOT NULL DEFAULT 0,
updated_at TIMESTAMPTZ DEFAULT NOW(),
PRIMARY KEY (country_code, entity_type)
);
```

```
-- Atomic sequence acquisition (Rust)
```

```
async fn acquire_sequence(pool: &PgPool, country: &str, entity_type: &str) -> Result<u32> {
let row = sqlx::query!(
r#"
UPDATE tenant_sequences
SET last_sequence = last_sequence + 1,
updated_at = NOW()
WHERE country_code = $1 AND entity_type = $2
RETURNING last_sequence
"#,
country, entity_type
)
.fetch_one(pool)
.await?;
Ok(row.last_sequence as u32)
}
```

2.1.4.3 Generation API Endpoint (Internal)

Endpoint: POST /v1/identity/gtid/generate (Internal — only called during onboarding)

Request:

json

```
{  
  "country_code": "EG",  
  "entity_type": "TRD",  
  "legal_name": "Strawberry Export Co.",  
  "jurisdiction": "EG"  
}
```

Response:

json

```
{  
  "gtid": "SGTX-EG-TRD-002139-7F3A",  
  "sequence": 2139,  
  "checksum": "7F3A",  
  "created_at": "2026-06-15T10:00:00Z"  
}
```

2.1.5 GTID Resolution Service

2.1.5.1 Purpose

The GTID Resolution Service allows any authenticated user to resolve a GTID to the tenant's public profile — information that the tenant has explicitly consented to share. This is the primary mechanism for identifying counterparties during trade creation.

2.1.5.2 Resolution Endpoint

Endpoint: GET /v1/gtid/resolve

Authentication: Valid JWT required (public GTID resolution is not available to unauthenticated users to prevent scraping).

Query Parameters:

Rate Limits:

2.1.5.3 Response Schema (Default — without verified IDs)

json

```
{  
  "gtid": "SGTX-EG-TRD-002139-7F3A",  
  "legal_name": "Strawberry Export Co.",  
  "type": "TRD",  
  "subtype": null, // For FIN: "BANK" or "PFI"; for LSP: "TRUCKING", etc.
```

```

"jurisdiction": "EG",
"kyb_tier": 2,
"kyb_status": "VERIFIED",
"sanctions_cleared": true,
"pep_status": "CLEAR", // 'CLEAR', 'ENHANCED_DD', 'BLOCKED'
"trust_score": 92,
"trust_confidence": 81,
"tri_status": "Advanced Trusted",
"lifecycle_state": "VERIFIED",
"is_saved_contact": true, // If the requester has saved this contact
"is_blocked": false, // If the requester has blocked this contact
"relationship_type": "SUPPLIER", // If saved contact
"last_interaction": "2026-06-10T14:30:00Z",
"dispute_rate": 2.3, // Percentage of trades disputed (anonymised)
"on_time_delivery_rate": 94.5, // For logistics providers
"consented_to_share": {
  "verified_ids": false,
  "trust_components": true,
  "financing_history": false
},
"resolved_at": "2026-06-15T10:05:00Z"
}

```

2.1.5.4 Response Schema (With Verified IDs — requires explicit consent)

json

```

{
  "gtid": "SGTX-EG-TRD-002139-7F3A",
  "legal_name": "Strawberry Export Co.",
  "type": "TRD",
  "jurisdiction": "EG",
  "kyb_tier": 2,
  "kyb_status": "VERIFIED",
  "trust_score": 92,
  "verified_identifiers": [
    {
      "type": "LEI",

```



```
"value": "549300ABC123",
"status": "VERIFIED",
"verified_at": "2026-01-15",
"expires_at": "2027-01-15",
"issuer": "GLEIF"
},
{
  "type": "DUNS",
  "value": "123456789",
  "status": "VERIFIED",
  "verified_at": "2026-02-01",
  "issuer": "Dun & Bradstreet"
},
{
  "type": "CUSTOMS_REG",
  "value": "EG-123456",
  "status": "VERIFIED",
  "verified_at": "2026-01-10",
  "issuer": "Egypt Customs"
}
],
"resolved_at": "2026-06-15T10:05:00Z"
}
```

2.1.5.5 Error Responses

2.1.5.6 Privacy & Consent Controls

Default visibility (always visible):

Consent-required visibility:

2.1.6 Resolution Caching & Performance

2.1.6.1 Caching Strategy

2.1.6.2 Cache Key

text

gtid:{gtid}:v{version}

Where version increments on any profile change.

2.1.6.3 Performance Targets

2.1.7 AI Assistance for GTID Resolution

2.1.7.1 Autocomplete (A1, Groq → Ollama)

When a user types a GTID (or partial GTID) in a form, the system provides real-time autocomplete from:

Saved contacts — GTIDs and legal names of counterparties the user has previously traded with.

Recent resolutions — GTIDs the user has resolved in the last 30 days.

Exact match — If the typed string matches a valid GTID format, the system attempts to resolve it.

Example (Buyer creating a trade request):

text

User types: "SGTX-VN"

System suggests:

- SGTX-VN-TRD-002139-7F3A — Mekong Fresh (previous trade: 8 Jun 2026)
- SGTX-VN-TRD-001234-5B6C — Vietnam Rice Export (saved contact)
- SGTX-VN-TRD-003456-D7E8 — No trade history — verify manually

Non-marketplace safeguard: The autocomplete list never includes "recommended" or "top-rated" sellers. It only shows:

Contacts the user has explicitly added.

Entities the user has previously traded with.

GTIDs the user has manually entered before.

■ Non-marketplace: If the user enters a GTID that is not in their saved contacts, the system displays a warning: "You have not traded with this entity before. Please verify their identity externally." No recommendation to add them.

2.1.7.2 Trust Score Explanation (A1, Groq)

When a GTID is resolved, the system displays a tooltip explaining the trust score and TRI status:

"Trust score 92 (Advanced Trusted) — based on 50 trades over 18 months. On-time delivery 98%, dispute rate 2.3%. Confidence: 81%."

Governance: The explanation is generated by Groq (A1) from the tenant's public profile data. It never compares the tenant to other tenants (no "better than average" language).

2.1.7.3 Sanctions Status Badge (A2, HF local)

AI Authority: A2 (HF local) performs sanctions screening. The badge is displayed prominently in the resolution response.

2.1.8 Security & Verification

2.1.8.1 GTID Unforgeability

2.1.8.2 Resolution Integrity

All resolution responses are signed with the platform's Ed25519 key (optional — for high-value resolutions).

Resolutions are logged in audit_log with actor GTID, resolved GTID, timestamp, and IP.

If a tenant is blocked (e.g., sanctions hit), resolution returns lifecycle_state: 'SUSPENDED' and sanctions_cleared: false.

2.1.8.3 GTID Revocation

2.1.9 Integration with Verified Identifiers

2.1.9.1 Verified Identifiers

Tenants can voluntarily link verified identifiers to their GTID. These identifiers are optional and require explicit consent to be shared.

2.1.9.2 Resolution with Verified IDs

To include verified identifiers in the resolution response, the requesting tenant must have explicit consent from the target tenant.

Consent check:

Each tenant sets default visibility per identifier type in Company Admin → Privacy → Verified IDs.

When resolving a GTID, the system checks:

If target has consented to share the specific identifier type.

If requesting tenant has a legitimate need (existing trade relationship or trade in progress).

If both conditions met, the identifier is included in the response.

One-click consent: Tenant admin clicks toggle in Privacy Dashboard → identifier becomes shareable.

2.1.10 Database Schema (Part 2.1)

sql

-- Tenants (core)

```
CREATE TABLE tenants (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  gtid TEXT UNIQUE NOT NULL,  
  legal_name TEXT NOT NULL,  
  legal_name_ar TEXT,  
  type tenant_type NOT NULL,  
  financier_subtype financier_subtype,  
  lsp_subtype lsp_subtype,  
  jurisdiction TEXT NOT NULL,  
  tax_id TEXT,  
  commercial_register TEXT,  
  kyb_tier INTEGER DEFAULT 1,  
  kyb_status TEXT DEFAULT 'PENDING',  
  trust_score DECIMAL(5,2),  
  sanctions_cleared BOOLEAN DEFAULT false,  
  pep_status TEXT DEFAULT 'CLEAR', -- 'CLEAR', 'ENHANCED_DD', 'BLOCKED'  
  lifecycle_state TEXT DEFAULT 'REGISTERED',  
  default_trader_mode trader_mode DEFAULT 'DUAL',  
  consent_settings JSONB DEFAULT '{}',
```

```

created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Tenant sequence counter
CREATE TABLE tenant_sequences (
country_code CHAR(2) NOT NULL,
entity_type CHAR(3) NOT NULL,
last_sequence INTEGER NOT NULL DEFAULT 0,
updated_at TIMESTAMPTZ DEFAULT NOW(),
PRIMARY KEY (country_code, entity_type)
);
-- Verified identifiers
CREATE TABLE tenant_verified_ids (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
id_type TEXT NOT NULL, -- 'LEI', 'DUNS', 'CUSTOMS_REG', 'CHAMBER_REG', 'VAT_REG'
id_value TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED'
verified_at TIMESTAMPTZ,
expires_at TIMESTAMPTZ,
reference_url TEXT,
is_public BOOLEAN DEFAULT false, -- Consent to share
created_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_gtid, id_type)
);
-- GTID resolution cache (materialised view)
CREATE MATERIALIZED VIEW gtid_resolution_cache AS
SELECT
gtid,
legal_name,
type,
jurisdiction,
kyb_tier,
kyb_status,
sanctions_cleared,

```

```

    pep_status,
    trust_score,
    lifecycle_state,
    updated_at,
    NOW() AS cached_at
FROM tenants
WHERE lifecycle_state IN ('VERIFIED', 'LIMITED_MODE');
REFRESH MATERIALIZED VIEW gtid_resolution_cache;

-- Resolution audit log
CREATE TABLE gtid_resolution_log (
    id BIGSERIAL PRIMARY KEY,
    requester_gtid TEXT NOT NULL,
    resolved_gtid TEXT NOT NULL,
    include_verified_ids BOOLEAN DEFAULT false,
    ip_address INET NOT NULL,
    user_agent TEXT,
    resolved_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_tenants_gtid ON tenants(gtid);
CREATE INDEX idx_tenants_type ON tenants(type);
CREATE INDEX idx_tenants_jurisdiction ON tenants(jurisdiction);
CREATE INDEX idx_tenants_lifecycle_state ON tenants(lifecycle_state);
CREATE INDEX idx_tenant_verified_ids_tenant ON tenant_verified_ids(tenant_gtid);
CREATE INDEX idx_tenant_verified_ids_type ON tenant_verified_ids(id_type);
CREATE INDEX idx_gtid_resolution_log_requester ON gtid_resolution_log(requester_gtid);
CREATE INDEX idx_gtid_resolution_log_resolved ON gtid_resolution_log(resolved_gtid);
CREATE INDEX idx_gtid_resolution_log_resolved_at ON gtid_resolution_log(resolved_at DESC);

```

2.1.11 Implementation Checklist (Part 2.1)

2.1.11.1 GTID Generation

Implement tenant_sequences table with atomic increment.

Implement CRC32-ISO-HDLC checksum calculation (Rust).

Add GTID generation endpoint (POST /v1/identity/gtid/generate — internal).

Add GTID format validation (regex + checksum).

Log GTID generation as Governor decision.

2.1.11.2 GTID Resolution

Implement resolution endpoint (GET /v1/gtid/resolve).

Add permission checks (RLS + OPA).

Implement consent-based filtering for verified identifiers.

Add rate limiting (100 req/min per tenant, 30 req/min per IP).

Implement caching (Valkey/Redis, 5 min TTL).

2.1.11.3 AI Assistance

Implement autocomplete from saved contacts (A1, Groq).

Add trust score tooltip generation (A1, Groq).

Implement sanctions badge (A2, HF local).

Add non-marketplace safeguard warning for unknown GTIDs.

2.1.11.4 Privacy & Compliance

Implement consent settings in Company Admin → Privacy.

Add verified identifier management (LEI, DUNS, etc.).

Implement resolution audit logging.

Add GTID revocation on lifecycle state change.

2.1.11.5 Testing

Test GTID generation (valid/invalid country, entity type).

Test checksum verification (valid/invalid checksums).

Test resolution permissions (RLS, consent).

Test rate limiting (exceed limits → 429).

Test autocomplete (saved contacts, recent resolutions, unknown GTID warning).

Test verified identifiers inclusion (with/without consent).

Test GTID revocation (ARCHIVED/SUSPENDED → 404).

2.1.12 AI Authority Summary for Part 2.1

END OF PART 2.1 — GLOBAL TRADE IDENTITY (GTID) — FORMAT & RESOLUTION (v11.0 FULLY EXPANDED)

2.2 Onboarding Wizard (6 Steps, AI-Assisted)

2.2.1 Overview

The Onboarding Wizard guides every tenant through a structured, zero-cost, AI-augmented onboarding process. The wizard collects all necessary identity, compliance, and configuration data while never suggesting counterparties or service providers.

AI Authority: A1 (Groq → Ollama) for autocomplete, defaults, and plain-language explanations; A2 (HF local → Ollama) for document extraction (HF Donut), registry cross-checks, and biometric liveness; A4 (OPA) for GTID generation and state transitions.

One-click guarantee: Each step has a single primary action button (e.g., "Confirm GTID", "Verify & Continue", "Save Preferences"). Data entry fields are not counted as irreversible clicks; only the final submission of each step is a one-click action.

2.2.2 Step 1 — Welcome & GTID Confirmation

Behaviour:

System generates a provisional GTID and displays it.

User confirms with one click: "Confirm GTID" .

Governor validates that the entity type is allowed in the selected jurisdiction (e.g., FIN only if jurisdiction allows private lending).

On success, tenant record created with lifecycle_state = 'REGISTERED'.

AI assistance: If the user enters an unsupported country, a plain-language explanation appears: "We cannot currently onboard entities from this jurisdiction due to sanctions restrictions. Please contact support if you believe this is an error." No recommendation to change jurisdiction.

One-click action: Click "Confirm GTID" → proceeds to Step 2.

2.2.3 Step 2 — Organization Details (with Verified Trade Profile)

Purpose: Collect core legal and commercial identifiers, including optional verified identifiers that will be linked to the GTID.

Fields (all tenants):

Document upload (PDF, max 10 MB):

User can upload supporting documents (commercial register extract, tax certificate, etc.).

A2 (HF Donut + PaddleOCR) extracts fields in real time and pre-fills the form.

Low-confidence fields (<85%) are highlighted; user must confirm or correct.

2.2.3.1 Verified Trade Profile (Optional)

After entering the basic identifiers, the user may optionally link additional verified identifiers. These are not required for onboarding but increase trust score and enable certain features.

Workflow for each identifier:

User enters the identifier or uploads a document.

Clicks the corresponding "Verify" button.

System calls the external API (or scraped cache) and returns:

- Verified — identifier stored in tenant_verified_ids table, linked to GTID.
- Mismatch / Invalid — error message with suggestion to correct.
- Pending (for batch-verified DUNS) — will be verified within 24h, user can proceed.

One-click actions per identifier: "Verify LEI", "Verify DUNS", etc. — each is a single click. User may skip any or all.

2.2.4 Step 3 — KYB/KYC Verification (Automated, AI-Driven)

Dynamic document list generated by RIA based on entity type, jurisdiction, and selected identifiers.

Example for TRD (Exporter, Egypt):

Commercial register extract (already uploaded in Step 2)

Tax registration certificate

Export licence

UBO declaration form (structured)

Bank account confirmation letter (optional)

Sanctions self-declaration (pre-filled, user signs)

AI assistance:

A2 (HF Donut) extracts all fields from uploaded PDFs.

System cross-references with government registries (free APIs, scraped).

Biometric liveness: for individuals (UBOs, directors) using ZITADEL WebAuthn (passkey) — no third-party service.

Verification status:

- Auto-verified — confidence $\geq 90\%$ and registry match.
- Manual review required — confidence $< 85\%$ or registry unavailable.
- Rejected — mismatch with official records (user can appeal).

User action:

Click "Submit Documents" — one click. System queues for verification.

While pending, tenant can use sandbox but cannot create real trades.

Smart Inbox shows estimated SLA (e.g., "Your documents are under review. Estimated response 48 hours.") — generated by A1.

Governance: Governor validates that all mandatory documents are submitted; otherwise returns CONDITIONAL with Decision Panel listing missing items.

2.2.5 Step 4 — Profile Configuration

Trader-specific (TRD only):

Trader mode: BUY (importer only), SELL (exporter only), DUAL (both). Default = DUAL.

Default incoterm (used to pre-fill trade requests).

Preferred currency (for displaying prices; actual trade currency can differ).

All tenants:

Preferred language (English, Arabic, German, Vietnamese, etc.) — used for all AI-generated messages, Smart Inbox, and UI.

Consent for optional features:

- Voice stress flag (off by default, on-device only)
- Offline mobile sync (for LSP/QC mobile apps)
- Anonymous market intelligence panel (sharing aggregated data)

Notification preferences (quiet hours, digest settings) — can be changed later in Company Admin.

One-click action: Click "Save Preferences" — stores settings, proceeds to Step 5.

2.2.6 Step 5 — Create First Resource (Optional)

Purpose: Pre-configure common data to accelerate future trade creation. Entirely optional; user can skip.

Resource types per tenant:

AI assistance (A1, Groq):

System suggests default fee ranges based on anonymised platform aggregates (e.g., "Typical trucking fee for this corridor: \$0.75–\$0.95/km").

User can accept, modify, or skip.

No suggestions based on "what others charge" — only anonymised, differentially private statistics.

One-click actions:

"Save Resource" — adds the resource to the tenant's catalogue.

"Skip" — proceeds to Step 6 without saving any resource.

2.2.7 Step 6 — Enter Sandbox

Purpose: Provide a fully functional, isolated replica of the platform for practice and education.

Sandbox characteristics:

Separate PostgreSQL schema (sandbox_*), NATS JetStream stream, and ClickHouse database.

Synthetic counterparties pre-provisioned with names like "Demo Buyer Co.", "Demo Seller Ltd.", "Demo Logistics Provider" — clearly marked DEMO.

No real money, real documents, or real API calls to government systems (mocked responses).

Resets automatically every week (Sunday 03:00 UTC). User can also click "Reset Sandbox" to wipe and restart.

Guided practice trade:

A step-by-step walkthrough (tooltips and modal overlays) that guides the user through:

Creating a trade request (structured form)

Seller accepting and locking EXW price

Designing packing

Adding logistics (using mock RFQ responses)

Submitting quote, signing contract, paying mock fee

Tracking milestones, confirming delivery, settling payment

Each step includes an educational explanation — never suggests alternative counterparties.

At the end, the user sees a summary of the complete trade cycle.

Sandbox UI elements:

Persistent banner: "You are in Sandbox mode. No real transactions will occur."

"Exit Sandbox" button (available only after completing the guided tour or after 5 minutes).

One-click actions:

"Start Sandbox" — creates sandbox environment and loads guided tour.

"Reset Sandbox" — wipes data and restarts.

"Exit Sandbox" — returns to main portal (sandbox data persists until reset).

2.2.8 Post-Onboarding — Go Live

After Step 6 (or after manual verification if Step 3 required human review), the tenant clicks "Go Live" .

System actions:

Wipes sandbox data (after confirmation prompt: "All sandbox data will be permanently deleted. Proceed?").

Sets lifecycle_state = VERIFIED (or KYB_PENDING if manual review still pending).

Issues a new JWT with full production permissions.

Redirects to the Universal Command Center (Part 12G).

If KYB is still pending, the user sees a banner: "Your account is verified for sandbox only. Real trading will be enabled once compliance review is complete." The "Go Live" button is disabled until verification.

One-click guarantee: "Go Live" — one click to transition to verified production state.

PART 2.3 — DUAL-MODE TOGGLE (TRADER PORTAL)

2.3.0 Purpose & Design Philosophy

Purpose: Allow trading companies with DUAL mode to switch seamlessly between Buyer Dashboard (inbound, import/procurement activities) and Seller Dashboard (outbound, export/sales activities) with a single click. The toggle provides clear visual feedback to prevent mode confusion and ensures that all actions, data views, and permissions are correctly scoped to the active context.

Design Philosophy:

One portal, two perspectives — A single unified Trader Portal replaces the need for separate Buyer and Seller logins.

Context-aware — Every UI element, API call, and Smart Inbox item adapts instantly to the selected mode.

Visual clarity — The active mode is unmistakable through colour, iconography, and persistent labelling.

Governance-enforced — OPA policies ensure that actions attempted in the wrong mode are gracefully blocked with clear remediation.

User-controlled — Tenants can disable the toggle per employee (e.g., for finance users who should only see one side).

Accessible — Full keyboard navigation, screen reader support, and voice command integration.

AI Integration in Part 2.3:

■ Non-marketplace: The toggle never suggests "you should also try buying" — it merely changes the view of existing data. It does not discover, recommend, or rank counterparties.

2.3.1 Visibility Conditions (All Must Be True)

The toggle is displayed in the global header of the Trader Portal only when all of the following conditions are met:

Fallback: If default_trader_mode = BUY, the user sees only Buyer dashboard (toggle hidden). If default_trader_mode = SELL, user sees only Seller dashboard (toggle hidden). If allow_role_switching = false, the toggle is hidden and the user is locked into their default_trader_mode.

One-click: Tenant admin can set allow_role_switching = false in Company Admin → Data Scopes → "Allow Role Switching" toggle (1 click).

2.3.2 UI Placement & Visual Design

2.3.2.1 Location

The toggle is positioned in the global header, to the right of the notifications bell and to the left of the user avatar. This consistent placement ensures it is always visible and accessible from any page within the Trader Portal.

Layout (textual):

text



■ SGTX Logo ■ [Search] ■ [Notifications] ■ [BUYER | SELLER] ■ ■ Ahmed (Buyer) ■



2.3.2.2 Visual Appearance (Enhanced, Professional)

Animations:

Mode switch transition: The highlight slides smoothly from one segment to the other (300ms ease-in-out).

Hover effect: Slight lift and shadow on the toggle container.

Active mode pulse: A subtle pulsing border around the active segment for 5 seconds after switching (to reinforce the change).

Accessibility considerations:

Colour is not the sole indicator — the icon, text, and avatar badge all change.

Minimum contrast ratio of 4.5:1 for all text.

Focus ring (2px solid #000) visible when navigating via keyboard.

Example – Buyer Mode:

text



■ [■ Buyer] ■ [■ Seller] ■

■ (Blue) ■ (Grey) ■



Example – Seller Mode:

text



■ [■ Buyer] ■ [■ Seller] ■

■ (Grey) ■ (Green) ■



2.3.2.3 Persistent Mode Indicators

Beyond the toggle itself, the active mode is reinforced across the portal:

2.3.3 Behaviour on Toggle

2.3.3.1 Step-by-Step Sequence

Total time: ~300–500ms (network + rendering). No visual flash or full-page refresh.

2.3.3.2 What Changes on Toggle

2.3.3.3 Draft Preservation

If a user is in the middle of filling out a form (e.g., creating a new trade request in Buyer mode) and switches to Seller mode, the form state is preserved in the browser's local storage for a short period. Upon switching back, the user receives a prompt: "You have an unsaved draft in Buyer mode. Resume?" The draft is stored in draft_history table with entity_type = 'TRADE_REQUEST' and edited_by = employee_id.

One-click: Click "Resume" (1) → form restored.

2.3.4 Backend API & Database

2.3.4.1 API Endpoint

Endpoint: POST /v1/employee/switch-context

Request:

json

```
{
  "active_trader_mode_context": "BUY" // or "SELL"
}
```

Response (200 OK):

json

```
{
  "jwt": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 900,
  "mode": "BUY"
}
```

Error Responses:

Rate Limit: 10 switches per 60 seconds per employee.

2.3.4.2 Database Fields

employees table (existing, enhanced):

data_scopes table (existing):

tenant table (existing):

2.3.4.3 JWT Claims

The JWT issued after login (or mode switch) includes the following claims:

json

```
{
  "sub": "employee-uuid",
  "tenant_id": "tenant-uuid",
  "tenant_gtid": "SGTX-EG-TRD-002139-7F3A",
}
```

```
"active_trader_mode_context": "BUY",
"allow_role_switching": true,
"permissions": ["trade.request.create", "contract.sign", "..."],
"exp": 1740000000,
"iat": 1739999100
}
```

Key: active_trader_mode_context is used by the Governor and frontend to filter actions and UI.

2.3.5 Governor Enforcement (OPA Policies)

2.3.5.1 Policy Logic

Every API request is evaluated against OPA policies. The active mode is checked for actions that are mode-specific.

permissions.rego (excerpt):

```
rego
package sgtx.permissions
default allow = false
# Action requires Buyer mode
allow {
  input.action == "trade.request.create"
  input.active_trader_mode_context == "BUY"
  role_has_permission(input.actor_role, "trade.request.create")
}
allow {
  input.action == "quote.accept"
  input.active_trader_mode_context == "BUY"
  role_has_permission(input.actor_role, "quote.accept")
}
# Action requires Seller mode
allow {
  input.action == "seller_quote.submit"
  input.active_trader_mode_context == "SELL"
  role_has_permission(input.actor_role, "seller_quote.submit")
}
allow {
  input.action == "exw.lock"
  input.active_trader_mode_context == "SELL"
  role_has_permission(input.actor_role, "exw.lock")
}
```

```

}
# Actions allowed in both modes (e.g., signing contract)
allow {
input.action == "contract.sign"
input.active_trader_mode_context in ["BUY", "SELL"]
role_has_permission(input.actor_role, "contract.sign")
not resource_already_signed_by_actor(input.resource.ustn, input.actor_gtid)
}

```

2.3.5.2 Block Response with Mode Switch Suggestion

If an action is attempted in the wrong mode, the Governor returns a DENY verdict with a tenant_message that includes a mode switch suggestion.

Example Response:

```

json
{
  "decision_id": "dec-abc123",
  "verdict": "DENY",
  "tenant_message": {
    "summary": "You are currently in Buyer mode. This action (submitting an EXW price) can only be performed in Seller mode. Switch to Seller mode to continue.",
    "conditions": [
      {
        "condition_id": "mode_mismatch",
        "label": "Switch to Seller mode",
        "status": "unmet",
        "action_url": "/v1/employee/switch-context?mode=SELL&retry=true"
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  },
  "loom_hash": "sha256:..."
}

```

UI Behaviour: The PlainLanguage Governor Decision Panel (Part 12A.3) displays this message and includes a "Switch to Seller Mode" button (1 click). Clicking it triggers the mode switch and then automatically retries the original action.

One-click guarantee: User clicks "Switch to Seller Mode" (1) → mode switches → action retries automatically.

2.3.6 Accessibility & Voice Support

2.3.6.1 Keyboard Navigation

ARIA Attributes:

html

```
<div role="group" aria-label="Trader mode selector">
```

```
<button role="radio" aria-checked="true" aria-label="Buyer mode" id="mode-buy">
```

Buyer

```
</button>
```

```
<button role="radio" aria-checked="false" aria-label="Seller mode" id="mode-sell">
```

Seller

```
</button>
```

```
</div>
```

```
<div aria-live="polite" aria-atomic="true" id="mode-announcer"></div>
```

Screen Reader Announcement: "Switched to Seller mode" via aria-live region.

2.3.6.2 Voice Command

Trigger: User speaks into the VoiceCommandButton (Part 12A.5).

Commands:

Processing:

Vosk (offline) transcribes.

HF Mixtral (A2) extracts intent and mode.

If intent recognised, triggers the same POST /v1/employee/switch-context endpoint.

Confirmation via TTS (device text-to-speech).

Fallback: If voice recognition fails, a visual prompt appears: "I didn't catch that. Please use the toggle or type your command."

2.3.7 Integration with Other Components

2.3.8 Role-Specific Views & Permissions

2.3.8.1 Pure Buyer (mode = BUY)

Toggle hidden.

Portal permanently in Buyer dashboard.

Only buyer-side actions allowed.

active_trader_mode_context always BUY.

2.3.8.2 Pure Seller (mode = SELL)

Toggle hidden.

Portal permanently in Seller dashboard.

Only seller-side actions allowed.

active_trader_mode_context always SELL.

2.3.8.3 Dual-Mode (mode = DUAL)

Toggle visible if allow_role_switching = true for the employee.

User can switch freely.

active_trader_mode_context set to last selected mode.

OPA policies enforce mode-specific actions.

2.3.8.4 Dual-Mode with Disabled Switching

Toggle hidden (allow_role_switching = false).

User locked into default_trader_mode (either BUY or SELL).

Employee sees only one side of the portal.

Use Case: Finance department user who only manages payables (Buyer mode) or receivables (Seller mode) should not have access to the opposite side.

One-click: Admin sets allow_role_switching = false in Company Admin → Data Scopes (1 click).

2.3.9 Edge Cases & Error Handling

2.3.10 Worked Example — Dual-Mode User Journey

Context: Nile Foods (Egypt, dual-mode) wants to import oranges from Mekong Fresh (Vietnam) and export dates to Germany.

Step 1 — Login

Ahmed logs into Trader Portal.

default_trader_mode = DUAL; active_trader_mode_context = BUY (last session).

Toggle shows Buyer active.

Browser tab: (Buyer) SGTX — Trade Portal.

Avatar: ■ Ahmed (Buyer).

Step 2 — Buyer Actions

Ahmed creates a new trade request for oranges (structured container form).

Submits trade request → seller notified.

Smart Inbox shows: "Quote received from Mekong Fresh".

Ahmed reviews quote, negotiates, accepts.

Contract signing: Buyer signs contract.

Shipment tracked in inbound view (Customs Readiness tab).

Step 3 — Switch to Seller

Ahmed needs to respond to a buyer's counter-offer on an outbound shipment of dates to Germany.

He clicks the Seller segment on the toggle.

Transition:

Toggle highlight slides to Seller (green).

Avatar changes to ■ Ahmed (Seller).

Browser tab updates to (Seller) SGTX — Trade Portal.

Smart Inbox refreshes: now shows seller-relevant items.

Sidebar changes: "Pending Requests", "EXW Price Lock", "Logistics Builder" appear.

Shipments Vault now shows outbound shipments.

Step 4 — Seller Actions

Ahmed opens the pending request from the German buyer.

Locks EXW price using live market chart.

Designs packing with non-uniform layers.

Uses Logistics Builder (Mode B) to get trucking quotes.

Submits quote.

Seller pays SGTX fee, contract locks.

Step 5 — Switch Back to Buyer

Later, Ahmed needs to check customs readiness for the inbound oranges.

He clicks Buyer on the toggle.

Portal reloads with Buyer context.

Customs Readiness tab shows missing phytosanitary certificate.

Ahmed requests the document from seller.

Throughout this day, Ahmed never logs out. The toggle allows him to fluidly move between the two sides of his company's operations, with the system always ensuring he sees the right data, the right actions, and the right compliance explanations.

Total clicks to switch mode: 1.

2.3.11 Implementation Checklist (Part 2.3)

2.3.11.1 Backend

Add `active_trader_mode_context` column to `employees` table (already exists; verify).

Add `allow_role_switching` column to `employees` and `data_scopes` (already exists; verify).

Implement `POST /v1/employee/switch-context` endpoint (identity service).

Update JWT generation to include `active_trader_mode_context` claim.

Extend OPA policies (`permissions.rego`) to enforce mode-specific actions.

Add rate limiting (10 switches per 60 seconds).

Write unit tests for mode switch API and OPA policies.

2.3.11.2 Frontend

Implement toggle UI in global header (React component).

Add animated slide transition.

Update avatar badge and browser tab title on mode switch.

Implement soft reload of Smart Inbox, Shipments Vault, TCC, sidebar, and other components.

Preserve form drafts in local storage when switching modes.

Add keyboard navigation and ARIA attributes.

Integrate with `VoiceCommandButton` for voice-activated switching.

Handle error cases (network failure, rate limit).

2.3.11.3 Governance

Ensure Decision Panel includes "Switch Mode" button for mode mismatch blocks.

Test mode switch with all role types (BUY, SELL, DUAL, DUAL with switching disabled).

Verify that actions are correctly filtered in Smart Inbox, Shipments Vault, and TCC.

2.3.11.4 Accessibility

Test keyboard navigation (Tab, Arrow keys, Enter/Space).

Verify screen reader announcements (aria-live).

Ensure colour contrast meets WCAG 2.2 AA.

Test voice command (Vosk + HF Mixtral).

2.3.11.5 Testing

Unit tests: mode switch API, OPA policy enforcement.

Integration tests: end-to-end mode switch → action in new mode.

UI tests: toggle visual updates, avatar badge, browser tab.

Accessibility tests: keyboard, screen reader, contrast.

Load tests: mode switch performance under concurrent users.

2.3.12 AI Authority Summary for Part 2.3

END OF PART 2.3 — DUAL-MODE TOGGLE (v11.0 FULLY EXPANDED)

2.4 Internal Tenant Organization Graph

Large enterprises can create business units, departments, cost centres, and approval chains within their own tenant. This is purely internal and never shared with other tenants.

2.4.1 Database Tables

2.4.2 Approval Policy Example

json

```
{
  "action": "contract.sign",
  "condition_json": { "trade.value": { ">": 100000 } },
  "required_approvals": [
    { "role": "trade_manager", "count": 2 },
    { "group_id": "finance-approvers", "count": 1 }
  ],
  "quorum": 3,
  "approval_mode": "parallel"
}
```

AI assistance for approval policy creation: The system provides a natural language prompt: "I want to require two senior managers to approve any contract over \$200k." The AI (Groq) generates a draft

approval policy JSON, which the admin can review, test, and activate.

2.5 Tenant Lifecycle Engine (Unified State Machine)

Every tenant transitions through these states. State changes are logged in `tenant_lifecycle_history` and trigger Smart Inbox notifications (with AI-generated explanations).

Transitions: Governor decision required for `KYB_PENDING` → `VERIFIED`, `VERIFIED` → `AT_RISK`, `SUSPENDED` → `VERIFIED`. Smart Inbox creates high-priority items on state changes with Groq-generated explanations.

AI assistance in `KYB_PENDING`: When a review is pending, the Smart Inbox shows the estimated SLA, queue position, and a direct link to the compliance team (no unsolicited "try another provider").

2.6 Network Feature (Saved Contacts) — No Marketplace Discovery

The Network feature (Saved Contacts) is a strictly opt-in, user-controlled directory of existing counterparties. SGTX never auto-discovers or suggests new contacts. Contacts are saved automatically when:

A trade request is created between two tenants.

A logistics provider accepts a quote.

A financing agreement is signed.

A direct message is exchanged in the Trade Room.

A distressed cargo notification is sent.

Manual addition: Users can also manually add a GTID (if they know it) via the "Add Contact" button. The system resolves the GTID and shows public information (legal name, jurisdiction, trust score) — but does not recommend "people you may know".

2.6.1 AI Enrichment (Non-Marketplace)

Important: The Network tab never displays a list of "top rated" or "recommended" contacts. Users must already have a relationship or explicitly know the GTID.

2.7 Sandbox Environment — Isolated, Safe, Educational

The sandbox is a complete replica of the production platform, but:

Data is stored in separate PostgreSQL schema (`sandbox_*`).

Synthetic counterparties are pre-provisioned with names like "Demo Buyer Co." and "Demo Seller Ltd." — clearly marked as demo.

No real money or real documents are ever used.

The sandbox resets every week (or manually via "Reset Sandbox" button).

A guided practice trade walks the user through every step (creating a request, locking price, adding logistics, paying fee, tracking milestones). Tooltips explain each action without suggesting alternative counterparties.

Exit to production: When the user clicks "Go Live", the sandbox data is wiped (after confirmation), and the tenant's `lifecycle_state` becomes `VERIFIED` (or `KYB_PENDING`). No data leaks to production.

2.8 Trade Readiness Assessment

2.8.1 Purpose

Ensure that every tenant (buyer, seller, logistics provider, financier, etc.) is fully prepared to execute their first trade or service on the SGTX platform. The Trade Readiness Assessment evaluates company

configuration, banking setup, trade preferences, security posture, and legal acknowledgements before allowing trade creation.

2.8.2 Readiness Categories & Checklist

The assessment is divided into five mandatory categories (plus optional advanced items). Each category contains several checklist items. For a tenant to be considered Fully Ready (score = 100%), all mandatory items must be completed. Optional items contribute to a higher score but are not required.

2.8.3 Readiness Score Calculation

Formula:

text

Readiness Score = (Completed Mandatory Items / Total Mandatory Items) × 100

Score Thresholds & Permissions:

2.8.4 UI Display — Readiness Card

Location:

Universal Command Center (Part 12G) — as a prominent card at the top.

Company Admin → Readiness — full detailed view.

Example Card:

text

■ ■ TRADE READINESS SCORECARD Score: 87% ■

■ Status: Mostly Ready ■

■ ■ Company: Tax ID, Registration, Address, UBO ■

■ ■■ Banking: Settlement account linked --- [Connect Now] ■

■ ■ Trade: 3 products, 5 ports, incoterm CFR ■

■ ■ Security: Passkey enrolled, MFA enabled ■

■ ■■ Legal: Fee schedule acknowledgment pending --- [Review & Sign] ■

■ Complete 2 remaining mandatory items to reach 100% readiness. ■

■ [Go to Company Admin] [Dismiss for 7 days] ■

2.8.5 One-Click Remediation Workflow

For each missing item, the system provides a direct action link that:

Opens a pre-filled form (where possible).

Guides the user through the required steps.

Upon completion, automatically updates the checklist status and recalculates the readiness score.

Example — Missing settlement account:

User clicks Connect PSP.

Modal appears with a list of available PSPs (Fawry, PayMob, CBE IPN).

User selects a PSP and is redirected to the PSP's OAuth flow.

After successful linking, SGTX receives a webhook and marks the item as completed.

The Readiness Card updates instantly (score increases).

2.8.6 Integration with Governor (Trade Creation Block)

When a user attempts to create a new trade and their current readiness score is <70%, the Governor returns a CONDITIONAL verdict with a Decision Panel that:

Lists the missing items.

Provides direct action buttons for each missing item.

Includes an "Ignore and Proceed" button only if the score is between 70% and 84% (with a warning log).

For scores <70%, the "Proceed" button is disabled.

2.9 Role Journey Maps

2.9.1 Trader Journey (Buyer — Importer)

2.9.2 Trader Journey (Seller — Exporter)

2.9.3 Logistics Service Provider (LSP) Journey

2.9.4 Shipping Line (SHIP) Journey

2.9.5 Customs Broker (CBR) Journey

2.9.6 Laboratory (LAB) Journey

2.9.7 QC Inspector Journey

2.9.8 Financier (Bank) Journey

2.9.9 Government Officer Journey

2.10 SGTX Trade Trust Passport™

2.10.1 Purpose

Provide a portable, participant-controlled institutional trust profile that aggregates a tenant's reliability, compliance, financing performance, dispute resolution, and verified identifiers. The Trust Passport is a verifiable credential (W3C standard) signed by SGTX, enabling participants to share their trustworthiness with counterparties, financiers, logistics providers, and regulators without exposing raw trade data.

Key principles (non-marketplace, sovereign):

Participant-controlled: No automatic sharing. Every share requires explicit consent.

Privacy-preserving: Passport contains aggregated scores and summaries, not transaction-level details.

Verifiable off-line: Passport includes a cryptographic signature that can be verified using SGTX's public key.

Revocable: Sharing links can be revoked at any time; old links become invalid immediately.

No ranking or discovery: The Passport never suggests "better" counterparties; it only reflects the tenant's own verified history.

2.10.2 Trust Passport Contents

The Trust Passport is a JSON-LD document with the following fields:

2.10.3 Verifiable Credential Format (W3C)

json

```
{
"@context": ["https://www.w3.org/2018/credentials/v1"],
"id": "https://sgtx.io/credentials/trust-passport/SGTX-EG-TRD-002139-7F3A",
"type": ["VerifiableCredential", "TradeTrustPassport"],
"issuer": "https://sgtx.io/issuers/platform",
"issuanceDate": "2026-06-15T00:00:00Z",
"expirationDate": "2026-09-13T00:00:00Z",
"credentialSubject": {
  "gtid": "SGTX-EG-TRD-002139-7F3A",
  "legal_name": "Strawberry Export Co.",
  "jurisdiction": "EG",
  "tri_score": 874,
  "tri_confidence": 97,
  "tri_status": "Advanced Trusted",
  "settlement_reliability": 910,
  "compliance_health": 890,
  "documentation_quality": 845,
  "financing_performance": 920,
  "dispute_resolution": 880,
  "verified_identifiers": [
    { "type": "LEI", "value": "549300ABC123", "status": "VERIFIED" },
    { "type": "DUNS", "value": "123456789", "status": "VERIFIED" }
  ],
  "compliance_summary": {
    "sanctions_cleared": true,
    "pep_status": "CLEAR",
    "kyb_tier": 2,
    "last_review": "2026-01-15"
  },
  "financing_summary": {
```

```

"total_financed_amount_usd": 2500000,
"on_time_repayment_rate": 98.5,
"default_count": 0
},
"dispute_summary": {
"total_disputes": 3,
"resolved_without_arbitration": 100,
"average_resolution_days": 12
}
},
"proof": {
"type": "Ed25519Signature2020",
"created": "2026-06-15T00:00:00Z",
"proofPurpose": "assertionMethod",
"verificationMethod": "https://sgtx.io/keys/ed25519",
"signature": "base58..."
}
}

```

2.10.4 Sharing Workflow (One-Click)

Step 1 — Generate Passport:

Tenant admin navigates to Company Admin → Trust Passport and clicks "Generate Passport" (one click).

System creates a new verifiable credential.

Passport is valid for 90 days (configurable per tenant).

Admin sees a summary card with TRI, confidence, and expiration.

Step 2 — Share with Counterparty:

Admin clicks "Share Passport" → selects the recipient's GTID (from saved contacts) or chooses "Anyone with link" (anonymous token).

If specific GTID, the system verifies that the recipient is an existing counterparty.

Admin selects which dimensions to share.

Click "Generate Link" → system returns a one-time token URL.

Admin copies link and sends to recipient (outside SGTX).

Step 3 — Recipient Verification:

Recipient opens the link (no login required).

Page displays the passport data (only the dimensions the sharer consented to).

Recipient can download signed JSON or copy a verification code to check offline.

The page shows a green checkmark if the signature is valid and the passport is not revoked or expired.

Step 4 — Revoke Access:

At any time, admin clicks "Revoke" next to the shared token.

Revocation is immediate.

Any subsequent attempt to verify that token returns { "valid": false, "reason": "revoked" }.

One-click guarantee: Generate Passport (1), Share (1), Revoke (1).

2.10.5 Offline Verification & Public Key

Any party can verify a Trust Passport offline using the SGTX public key.

Steps (for recipient):

Download the signed passport JSON.

Use the open-source tool trust-verify (provided by SGTX).

Verify signature and check that expires_at is in the future.

Public keys are published at: <https://sgtx.io/.well-known/sgtx-keys> (Ed25519 and optional Dilithium3 for archival).

2.11 Database Schema Additions (Part 2)

sql

-- Extended tenants table

ALTER TABLE tenants ADD COLUMN qes_certificate_ref TEXT;

ALTER TABLE tenants ADD COLUMN readiness_score INTEGER DEFAULT 0;

ALTER TABLE tenants ADD COLUMN readiness_last_calculated TIMESTAMPTZ;

ALTER TABLE tenants ADD COLUMN default_incoterm TEXT DEFAULT 'CFR';

-- Verified identifiers

CREATE TABLE tenant_verified_ids (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

id_type TEXT NOT NULL, -- 'LEI', 'DUNS', 'CUSTOMS_REG', 'CHAMBER_REG', 'VAT_REG'

id_value TEXT NOT NULL,

status TEXT DEFAULT 'PENDING', -- 'PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED'

verified_at TIMESTAMPTZ,

expires_at TIMESTAMPTZ,

reference_url TEXT,

created_at TIMESTAMPTZ DEFAULT NOW(),

UNIQUE(tenant_id, id_type)

);

-- Trust Passports

CREATE TABLE trust_passports (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),


```

tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid),
credential_hash TEXT NOT NULL,
tri_score INTEGER NOT NULL,
tri_confidence INTEGER NOT NULL,
tri_status TEXT NOT NULL,
component_scores JSONB NOT NULL,
issued_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ NOT NULL,
loom_hash TEXT,
created_by UUID REFERENCES employees(id)
);

CREATE TABLE trust_passport_shares (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
passport_id UUID NOT NULL REFERENCES trust_passports(id),
token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
shared_with_gtid TEXT,
dimensions_shared TEXT[], -- ['all', 'settlement', 'financing', etc.]
expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '7 days',
revoked BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE trust_passport_revocations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
share_id UUID NOT NULL REFERENCES trust_passport_shares(id),
revoked_by UUID REFERENCES employees(id),
reason TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Device Registry
CREATE TABLE device_registry (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
device_name TEXT,
device_id TEXT NOT NULL,
public_key TEXT,

```

```

state TEXT DEFAULT 'NEW', -- 'NEW', 'TRUSTED', 'ELEVATED_RISK', 'BLOCKED', 'REVOKED'
risk_score INTEGER DEFAULT 0,
last_seen TIMESTAMPTZ,
registered_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(employee_id, device_id)
);

-- Session Risk Events
CREATE TABLE session_risk_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
employee_id UUID NOT NULL REFERENCES employees(id),
session_id TEXT,
risk_type TEXT NOT NULL, -- 'IMPOSSIBLE_TRAVEL', 'VPN_DETECTED', 'TOR_DETECTED', etc.
risk_score INTEGER,
details JSONB,
governor_verdict TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Consent Records
CREATE TABLE consent_records (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_gtid TEXT NOT NULL,
purpose TEXT NOT NULL, -- 'marketing', 'analytics', 'govt_sharing', 'cross_border', 'voice_biometric'
version TEXT NOT NULL,
granted BOOLEAN NOT NULL,
ip_address INET NOT NULL,
user_agent TEXT,
device_id TEXT,
timestamp TIMESTAMPTZ DEFAULT NOW(),
withdrawal_timestamp TIMESTAMPTZ,
loom_hash TEXT
);

-- Onboarding State
CREATE TABLE tenant_onboarding_state (
tenant_id UUID PRIMARY KEY REFERENCES tenants(id) ON DELETE CASCADE,
current_step INTEGER NOT NULL DEFAULT 1,

```

```

step_data JSONB DEFAULT '{}',
sandbox_active BOOLEAN DEFAULT true,
completed BOOLEAN DEFAULT false,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Readiness Checklist
CREATE TABLE readiness_checklist (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id),
category TEXT NOT NULL, -- 'company', 'banking', 'trade', 'security', 'legal'
item_key TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'COMPLETED', 'SKIPPED'
completed_at TIMESTAMPTZ,
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, item_key)
);

-- Internal Organisation
CREATE TABLE tenant_business_units (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
parent_bu_id UUID REFERENCES tenant_business_units(id),
name TEXT NOT NULL,
administrator_employee_id UUID REFERENCES employees(id),
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_departments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
business_unit_id UUID NOT NULL REFERENCES tenant_business_units(id) ON DELETE CASCADE,
name TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_cost_centers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
business_unit_id UUID REFERENCES tenant_business_units(id),
code TEXT NOT NULL,

```

```

description TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_approval_groups (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
name TEXT NOT NULL,
employee_ids UUID[] NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_approval_policies (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
action TEXT NOT NULL,
condition_json JSONB NOT NULL,
required_approvals JSONB NOT NULL,
quorum INTEGER NOT NULL,
approval_mode TEXT DEFAULT 'parallel',
enabled BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Tenant Lifecycle History
CREATE TABLE tenant_lifecycle_history (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
from_state TEXT NOT NULL,
to_state TEXT NOT NULL,
reason TEXT,
governor_decision_id UUID,
changed_by UUID REFERENCES employees(id),
changed_at TIMESTAMPTZ DEFAULT NOW()
);

-- Role Journey Completion (for tracking)
CREATE TABLE role_journey_completion (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```
employee_id UUID NOT NULL REFERENCES employees(id),
role_type TEXT NOT NULL,
step_key TEXT NOT NULL,
completed_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(employee_id, role_type, step_key)
);
```

-- Indexes

```
CREATE INDEX idx_tenant_lifecycle_history_tenant ON tenant_lifecycle_history(tenant_id);
CREATE INDEX idx_tenant_lifecycle_history_state ON tenant_lifecycle_history(to_state);
CREATE INDEX idx_trust_passports_tenant ON trust_passports(tenant_gtid);
CREATE INDEX idx_trust_passport_shares_token ON trust_passport_shares(token);
CREATE INDEX idx_device_registry_employee ON device_registry(employee_id);
CREATE INDEX idx_session_risk_events_employee ON session_risk_events(employee_id);
CREATE INDEX idx_consent_records_user ON consent_records(user_gtid);
```

2.12 Implementation Checklist (Part 2)

2.12.1 Identity & GTID

Implement GTID generation algorithm (Rust).

Build GTID resolution service (/v1/gtid/resolve).

Add GTID checksum validation.

Implement GTID sequence management (atomic increment).

2.12.2 Onboarding Wizard

Build 6-step wizard UI with progress indicator.

Integrate A2 (HF Donut) for document extraction in Steps 2 and 3.

Add registry cross-reference service (RIA-driven) for commercial register, tax ID, etc.

Implement biometric liveness (ZITADEL WebAuthn) for UBOs.

Build Verified Trade Profile section with LEI, DUNS, customs, chamber, VAT verification.

Create sandbox environment with schema isolation and weekly reset cron job.

Develop guided practice trade (tooltips, synthetic data, mock APIs).

Add "Go Live" transition logic and state management.

2.12.3 Dual-Mode Toggle

Implement toggle UI in global header.

Add /v1/employee/switch-context endpoint.

Update JWT with active_trader_mode_context claim.

Add OPA policy enforcement for dual-mode context.

2.12.4 Internal Organisation Graph

Create tables: tenant_business_units, tenant_departments, tenant_cost_centers.

Create tables: tenant_approval_groups, tenant_approval_policies.

Build UI for Organisation Manager (Company Admin).

Implement approval chain enforcement in Governor.

2.12.5 Tenant Lifecycle Engine

Implement state machine (REGISTERED → ONBOARDING → KYB_PENDING → VERIFIED → ...).

Add state transition logging (tenant_lifecycle_history).

Create Smart Inbox notifications on state changes (A1 Groq explanations).

2.12.6 Network Feature (Saved Contacts)

Create tenant_contacts table.

Implement automatic contact saving on trade, quote, message, etc.

Build AI-enriched Trust Portrait (A1 Groq).

Implement Relationship Health Score (LSTM, A2).

Add GraphRAG indirect connections.

Build Voice-Driven Contact Management (Vosk + HF Mixtral).

2.12.7 Trade Readiness Assessment

Create readiness_checklist table.

Implement validation APIs for each checklist item (tax ID, PSP account, etc.).

Build Readiness Card UI component with real-time updates.

Integrate Groq (A1) for plain-language recommendations.

Add Governor policy trade.create.readiness that checks score $\geq 70\%$.

Create Decision Panel integration for blocked actions.

Implement daily re-evaluation cron job (expiry detection).

Add "Dismiss for 7 days" feature.

2.12.8 Role Journey Maps

Create role_journey_completion table.

Implement journey step tracking for each role.

Add journey completion reporting (Company Admin).

2.12.9 Trust Passport

Create trust_passports, trust_passport_shares, trust_passport_revocations tables.

Implement credential generation service (Rust, json-ld, ed25519-dalek).

Build API endpoints:

GET /v1/trust/passport/{gtid} — get own passport.

POST /v1/trust/share — generate sharing token.

GET /v1/trust/verify/{token} — public verification.

POST /v1/trust/revoke — revoke active token.

Integrate with TRI calculator and optional dimensions.

Add consent management UI for optional dimensions.

Implement verified identifiers refresh and storage.

Add ZK proof generation for trust graph reference (if Add-on 7 enabled).

Create public key endpoint and documentation for offline verification.

2.12.10 Device Trust & Security

Create device_registry table.

Implement device trust rules (NEW, TRUSTED, ELEVATED_RISK, BLOCKED, REVOKED).

Add Device Management Center to Company Admin.

Implement Step-Up Authentication for high-value actions.

Create session_risk_events table.

Add Session Risk Engine (A2 anomaly detection).

Implement legal recovery flow for lost passkeys.

2.12.11 Consent Management

Create consent_records table.

Implement granular consent checkboxes in onboarding and Company Admin.

Add Governor policy for consent enforcement (cross-border data transfer).

Build Privacy Dashboard for users to view and withdraw consents.

2.13 AI Authority Summary for Part 2

END OF PART 2 — IDENTITY, TENANTS & INTERNAL AUTHORITY (v11.0 FULLY EXPANDED)

PART 3 — UNIVERSAL SHIPMENT TRACKING NUMBER (USTN) & END-TO-END TRADE WORKFLOW

3.0.0 Purpose & Design Philosophy

The Universal Shipment Tracking Number (USTN) is the single master identifier that binds every piece of data, every document, every payment, and every physical movement of an SGTX trade transaction into one inseparable digital chain. It is the answer to the question: "How do we ensure that the invoice seen by Customs is the same invoice seen by the tax authority, the shipping line, the bank, and the buyer — and that the payment matches it exactly?"

The USTN is generated automatically at contract lock (single-shipment) or per-shipment lock (multi-shipment). From that point forward, no document can be created, no payment can be instructed, and no container can move without referencing this number. It is printed on the Bill of Lading, embedded in the e-Invoice XML, included in the ACID filing, written on the phytosanitary certificate, and used as the payment narrative in the Central Bank's Instant Payment Network.

The USTN Format (v11.0 Revised):

text

SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Example: SGTX-EG-26-F3A-1

Why This Format?

Example USTNs:

text

SGTX-EG-26-F3A-1 → First trade by trader F3A in Egypt, 2026

SGTX-EG-26-F3A-2 → Second trade

SGTX-EG-26-F3A-10 → Tenth trade

SGTX-EG-26-F3A-1000 → One-thousandth trade

SGTX-VN-26-7B3A-1 → First trade by trader 7B3A in Vietnam, 2026

SGTX-EG-27-F3A-1 → First trade by trader F3A in Egypt, 2027 (counter resets)

SGTX-EG-26-F3A-26 → Distressed micro-contract (parent = SGTX-EG-26-F3A-15)

Key Properties:

Globally unique — includes buyer and seller GTID suffixes, timestamp, and random.

Deterministic — generated by an algorithm, not a central sequence (except the random part).

Immutable — once generated, it never changes (though a shipment may be cancelled, the USTN remains as a historical record).

Machine-readable and human-readable — 15-22 character alphanumeric (plain numbers, no confusing characters).

Mandatory — every SGTX document, API call, and payment reference must include the USTN.

AI assistance (non-marketplace):

When a user needs to reference a USTN, the system autocompletes from a dropdown of recent USTNs the user has access to — no suggestions from "similar trades of others".

The USTN QR code can be scanned by the mobile app, which then automatically opens the relevant Trade Command Center — no manual typing.

If a user manually types a USTN, the system validates the format in real time and, if valid, resolves it to a summary card (status, parties, commodity) without needing to submit a form.

3.0.1 USTN Format & Generation Algorithm

3.0.1.1 Format Structure

Format:

text

SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Full Format: SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Example: SGTX-EG-26-F3A-1

3.0.1.2 Component Definitions

Total length: ~15-20 characters (4 + 1 + 2 + 1 + 2 + 1 + 3 + 1 + variable = 15-22).

3.0.1.3 Trader Identifier Extraction

The trader identifier is the last 3 alphanumeric characters of the GTID's checksum (the final component after the sequence).

Extraction Logic:

rust

```
fn extract_trader_id(gtid: &str) -> String {  
    let parts: Vec<&str> = gtid.split('-').collect();  
    let checksum = parts[4];  
    let len = checksum.len();  
    checksum[len-3..len].to_string()  
}
```

3.0.1.4 Generation Algorithm

Database Schema (PostgreSQL):

sql

```
-- USTN counters (per year, per trader)  
CREATE TABLE ustn_counters (  
    country CHAR(2) NOT NULL,  
    year CHAR(2) NOT NULL,  
    trader CHAR(3) NOT NULL,  
    counter BIGINT DEFAULT 0,  
    PRIMARY KEY (country, year, trader)  
);  
  
-- Atomic increment function  
CREATE OR REPLACE FUNCTION next_ustn_seq(  
    p_country CHAR(2),  
    p_year CHAR(2),  
    p_trader CHAR(3)  
)  
RETURNS BIGINT AS $$  
DECLARE  
    next_val BIGINT;  
BEGIN  
    INSERT INTO ustn_counters (country, year, trader, counter)  
    VALUES (p_country, p_year, p_trader, 0)  
    ON CONFLICT (country, year, trader) DO NOTHING;  
    UPDATE ustn_counters  
    SET counter = counter + 1  
    WHERE country = p_country
```

```

AND year = p_year
AND trader = p_trader
RETURNING counter INTO next_val;
RETURN next_val;
END;
$$ LANGUAGE plpgsql;

```

Rust Implementation:

```

rust
use chrono::Utc;
use sqlx::PgPool;

pub async fn generate_ustn(
    pool: &PgPool,
    seller_gtid: &str,
) -> Result<String, Box<dyn std::error::Error>> {
    let parts: Vec<&str> = seller_gtid.split('-').collect();
    if parts.len() != 5 {
        return Err("Invalid GTID format".into());
    }
    let country = parts[1];
    let trader = extract_trader_id(seller_gtid);
    let year = Utc::now().format("%Y").to_string();
    let seq: i64 = sqlx::query_scalar(
        "SELECT next_ustn_seq($1, $2, $3)"
    )
    .bind(country)
    .bind(&year)
    .bind(&trader)
    .fetch_one(pool)
    .await?;
    Ok(format!("SGTX-{}-{}-{}-{}", country, year, trader, seq))
}

```

3.0.1.5 Generation API Endpoint (Internal)

Endpoint: POST /v1/identity/ustn/generate (Internal — only called during contract lock)

Request:

json

```
{
  "seller_gtid": "SGTX-EG-TRD-002139-7F3A",
  "buyer_gtid": "SGTX-DE-TRD-001234-5B6C",
  "contract_id": "MC-20260415-001",
  "shipment_number": 1
}
```

Response:

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "country": "EG",
  "year": "26",
  "trader_id": "F3A",
  "sequence": 1,
  "generated_at": "2026-04-15T12:00:00Z",
  "loom_hash": "sha256:a1b2c3..."
}
```

3.0.1.6 Validation Rules

3.0.2 USTN Lifecycle — Complete with All Statuses

The USTN transitions through the following states as the trade progresses. Status changes are recorded in the shipments.status column and trigger Smart Inbox updates.

Example Timeline (Healthy Trade):

text

SGTX-EG-26-F3A-1

↓

INITIATED → STAGE1_PENDING → STAGE1_SETTLED → CUSTOMS_SUBMITTED → BOOKED

↓

LOADED → DEPARTED → IN_TRANSIT → ARRIVED → CUSTOMS_IMPORT → DELIVERED → SETTLED → COMPLETED

3.0.3 USTN Examples — Complete Set

3.0.3.1 Standard Orders

3.0.3.2 Multi-Shipment Examples

3.0.3.3 Distressed Micro-Contracts

3.0.3.4 Different Traders & Years

3.0.4 USTN Master Object — Complete JSON Schema

The USTN master object is stored in the shipments table (and partially in trade_requests for pre-contract data). It is the single source of truth for the entire trade.

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "created_at": "2026-04-15T12:00:00Z",
  "updated_at": "2026-06-07T10:00:00Z",
  "status": "DELIVERED",
  "risk_assessment": {
    "platform_risk_score": 12,
    "gnn_risk": {
      "sanctions_proximity": 4,
      "graph_risk_score": 8,
      "explanation": "No indirect sanctions links detected within 2 hops."
    },
    "causal_analysis": null
  },
  "parties": {
    "exporter": {
      "gtid": "SGTX-EG-TRD-002139-7F3A",
      "trader_id": "F3A",
      "legal_name": "Strawberry Export Co.",
      "trust_score": 92,
      "jurisdiction": "EG"
    },
    "importer": {
      "gtid": "SGTX-DE-TRD-001234-5B6C",
      "trader_id": "5B6C",
      "legal_name": "European Importer GmbH",
      "trust_score": 88,
      "jurisdiction": "DE"
    }
  },
  "goods": {
    "hs_code": "0811.10.90",
    "description": "Frozen strawberries, IQF, Grade A",
  }
```

```
"net_weight_kg": 20000,
"gross_weight_kg": 20500,
"invoice_value": {
  "amount": 100000.00,
  "currency": "USD"
},
"incoterm": "CFR",
"container_count": 2,
"container_numbers": ["MEDU1234567", "MEDU1234568"]
},
"documents": {
  "eta_invoice_uuid": "INV-abc123",
  "acid": "ACI20260415-0001",
  "nafeza_declaration_id": "SAD20260415-000147",
  "phyto_certificate": "PC-20260415-147",
  "health_certificate": "HC-20260415-147",
  "certificate_of_origin": "COO-20260415-147",
  "bill_of_lading": {
    "number": "MEDU1234567",
    "type": "eBL",
    "issuer": "MSC",
    "issued_at": "2026-04-16T10:00:00Z",
    "url": "sgtx://documents/eb1-123"
  },
  "packing_list_hash": "sha256:abc123...",
  "qc_report": {
    "provider": "SGS Inspection",
    "verdict": "CONDITIONAL_PASS",
    "overrides": 1,
    "report_url": "sgtx://documents/qc-001"
  }
},
"logistics": {
  "trucking": {
    "provider_gtid": "SGTX-EG-LSP-001",
```

```
"provider_name": "Fast Trucking Co.",
"quote_id": "QT-001",
"fee": 300,
"status": "PAID"
},
"shipping_line": {
"provider_gtid": "SGTX-EG-SHIP-001",
"provider_name": "MSC Egypt",
"booking_ref": "MAEU876543210",
"vessel": "MSC FIONA",
"voyage": "MSF123E",
"etd": "2026-04-20T08:00:00Z",
"eta": "2026-05-05T14:00:00Z",
"actual_departure": "2026-04-20T09:30:00Z",
"actual_arrival": null,
"freight_invoice": {
"amount": 4200,
"status": "CREDIT",
"due_date": "2026-06-05"
}
},
"customs_broker": {
"certification": {
"quote_id": "CBRQ-001",
"fee": 150,
"status": "PAID",
"certified_at": "2026-04-16T09:00:00Z"
},
"physical_handling": null,
"storage": null
}
},
"payment_plan": {
"stage1": {
"status": "SETTLED",
```

```
"settled_at": "2026-04-15T13:00:00Z",
"transactions": [
  {"payee": "SGTX", "amount": 1353.00, "currency": "USD"},
  {"payee": "Fast Trucking Co.", "amount": 300.00, "currency": "USD"},
  {"payee": "SGS Lab", "amount": 200.00, "currency": "USD"},
  {"payee": "Customs Broker (cert)", "amount": 150.00, "currency": "USD"},
  {"payee": "Egypt Customs", "amount": 200.00, "currency": "USD"}
],
"stage2": {
  "status": "PENDING",
  "transactions": [
    {"payee": "MSC", "amount": 4200.00, "currency": "USD", "condition": "CREDIT", "due_date": "2026-06-05"}
  ],
},
"deferred": {
  "status": "GUARANTEE_HELD",
  "guarantee_amount": 10000.00,
  "trigger_milestone": "CUSTOMS_IMPORT",
  "expiry_date": "2026-05-15T23:59:59Z"
},
"timeline": [
  {"timestamp": "2026-04-15T09:00:00Z", "event": "TRADE_INITIATED", "actor": "buyer@importer.com"},
  {"timestamp": "2026-04-15T10:00:00Z", "event": "SELLER_ACCEPTED", "actor": "seller@exporter.com"},
  {"timestamp": "2026-04-15T10:30:00Z", "event": "QUOTE_SUBMITTED", "actor": "seller@exporter.com"},
  {"timestamp": "2026-04-15T11:00:00Z", "event": "CONTRACT_SIGNED", "actor": "both"},
  {"timestamp": "2026-04-15T12:00:00Z", "event": "USTN_GENERATED", "actor": "system"},
  {"timestamp": "2026-04-15T13:00:00Z", "event": "STAGE1_PAID", "actor": "system"},
  {"timestamp": "2026-04-15T13:05:00Z", "event": "CUSTOMS_DECLARATION_SUBMITTED", "actor": "broker"},
  {"timestamp": "2026-04-16T09:00:00Z", "event": "BROKER_CERTIFIED", "actor": "broker"},
  {"timestamp": "2026-04-16T10:00:00Z", "event": "BOOKING_CONFIRMED", "actor": "shipping_line"},
  {"timestamp": "2026-04-20T08:00:00Z", "event": "VESSEL_DEPARTED", "actor": "shipping_line"},
  {"timestamp": "2026-05-05T14:00:00Z", "event": "VESSEL_ARRIVED", "actor": "shipping_line"}
],
```

```
"optional_services": {
  "laboratory": {
    "provider_gtid": "SGTX-EG-LAB-001",
    "quote_id": "LABQ-001",
    "fee": 200,
    "status": "PAID",
    "results": {
      "cypermethrin": {"value": 0.02, "limit": 0.05, "pass": true},
      "chlorpyrifos": {"value": 0.008, "limit": 0.01, "pass": true}
    }
  },
  "qc_inspection": {
    "provider_gtid": "SGTX-EG-QC-002",
    "quote_id": "QCQ-001",
    "fee": 500,
    "status": "PAID",
    "report": {
      "verdict": "CONDITIONAL_PASS",
      "defects": [
        {"pallet_id": "OR019", "defect": "mould", "severity": "medium", "ai_confidence": 92, "inspector_override": false},
        {"pallet_id": "OR020", "defect": "bruising", "severity": "low", "ai_confidence": 88, "inspector_override": true, "override_reason": "Dark spot is surface bruise, no mould."}
      ]
    }
  },
  "blockchain_anchor": {
    "txid": "0xabc123...",
    "merkle_root": "sha256:...",
    "network": "polygon-mainnet",
    "ppc_signature": "dilithium3:..."
  },
  "links": {
    "trade_command_center": "https://sgtx.io/trade/SGTX-EG-26-F3A-1",
    "export_pdf": "https://sgtx.io/api/v1/shipment/SGTX-EG-26-F3A-1/pdf"
```



```
}  
}
```

3.0.5 USTN in Documents — Mandatory Inclusion

Every document generated by or uploaded to SGTX must contain the USTN in a standardised location (e.g., header, footer, or QR code). This includes:

AI assistance for document generation: When a document is generated, the system automatically embeds the USTN and a QR code. No manual entry required. If a user uploads a document, the system checks that the USTN is present (and matches the trade) before accepting it — if missing, it returns a warning and suggests adding it (but does not auto-correct).

Physical marking: For paper documents, the USTN is printed as both human-readable text and a QR code. The QR code encodes the full USTN URL: <https://sgtx.io/ustn/SGTX-EG-26-F3A-1>. Scanning the QR code on a mobile device opens the Trade Command Center (read-only view) — no login required for public verification (if the tenant has enabled public sharing).

3.0.6 USTN Resolution Service

Endpoint: GET /v1/ustn/{ustn} (authenticated, role-based filtering)

Returns the USTN master object filtered by the requester's permissions. A logistics provider sees only their services; a buyer sees full commercial terms but not the seller's internal costs; a financier sees all trade data; a government official sees only compliance documents.

Example Request (Buyer):

bash

```
curl -H "Authorization: Bearer <JWT>" \  
https://api.sgtx.io/v1/ustn/SGTX-EG-26-F3A-1
```

Response (Filtered for Buyer):

json

```
{  
  "ustn": "SGTX-EG-26-F3A-1",  
  "status": "IN_TRANSIT",  
  "parties": {  
    "exporter": {  
      "gtid": "SGTX-EG-TRD-002139-7F3A",  
      "name": "Strawberry Export Co."  
    },  
    "importer": {  
      "gtid": "SGTX-DE-TRD-001234-5B6C",  
      "name": "European Importer GmbH"  
    }  
  },  
  "goods": {
```

```

"description": "Frozen strawberries",
"invoice_value": {
  "amount": 100000,
  "currency": "USD"
},
"logistics": {
  "vessel": "MSC FIONA",
  "eta": "2026-05-05T14:00:00Z"
},
"payment_plan": {
  "stage1": "SETTLED",
  "stage2": "PENDING"
},
"timeline": [ ... ]
}

```

Rate Limits: 100 requests per minute per tenant.

3.0.7 USTN for Multi-Shipment Contracts

In a multi-shipment contract, a master contract ID exists but no master USTN. Each shipment receives its own USTN when it locks. The master contract record in contracts table contains the schedule, and each contract_shipments row stores the shipment-specific USTN.

Example — Three Shipments:

text

Master Contract: MC-20260415-001

■

■■■ Shipment 1: SGTX-EG-26-F3A-21 (15 Jul 2026)

■■■ Shipment 2: SGTX-EG-26-F3A-22 (15 Aug 2026)

■■■ Shipment 3: SGTX-EG-26-F3A-23 (15 Sep 2026)

Key Properties:

Each shipment has its own USTN and locks independently.

Per-shipment fee collection.

Independent execution — a delay in one shipment does not block others.

Buyer and seller see all shipments in the Trade Command Center under the master contract.

AI assistance for multi-shipment: The Smart Inbox groups pending actions by shipment, with a clear label "Shipment 2 of 3". The user can toggle between shipments in the Trade Command Center via a dropdown that autocompletes from the list of USTNs under the master contract.

3.0.8 USTN for Distressed Micro-Contracts

When a seller declares distress on a portion of a shipment (e.g., 3 pallets of lemons), SGTX generates a microUSTN for the distressed lot. The microUSTN follows the same format but links back to the original USTN via `parent_ustn`.

text

Original USTN: SGTX-EG-26-F3A-15

MicroUSTN: SGTX-EG-26-F3A-26

Parent Link: `micro_contracts.parent_ustn` = SGTX-EG-26-F3A-15

Key Properties:

Has its own lifecycle: `DISTRESS_SALE_PENDING` → `DISTRESS_MICROCONTRACT_LOCKED` → `DISTRESS_SALE_COMPLETED`

References the original USTN via `parent_ustn`

Original documents are copied (with redacted pricing if confidential)

Separate SGTX fee ($1.5\% \times \text{country factor}$)

3.0.9 USTN in API Calls — Mandatory Reference

All SGTX API calls that create or modify trade-related entities must include the USTN (or microUSTN) in the request body or as a query parameter.

If an API call omits the USTN where required, the Governor returns a 400 error with a plain-language message: "Missing USTN parameter. Every trade action must reference its shipment identifier."

3.0.10 USTN QR Code & Physical Tagging

Every USTN is encoded as a QR code that can be printed on shipping labels, packing lists, and customs documents. The QR code contains:

The full USTN string: SGTX-EG-26-F3A-1

A URL to the public verification page: <https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1>

A cryptographic signature (optional, for high-value goods)

QR Code Payload:

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "verify_url": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1",
  "verifiable_credential": {
    "type": "VerifiableCredential",
    "issuer": "https://sgtx.io/issuers/platform",
    "proof": {
      "type": "Ed25519Signature2020",
      "signature": "base58..."
    }
  }
}
```

```
}
```

AI assistance for QR scanning: The SGTX mobile app scans the QR code and automatically opens the Trade Command Center (or the public verification page). If the user is not logged in, the app prompts for login (or shows public information if enabled).

Offline verification: A receiver with no internet can scan the QR code, and the app uses a cached version of the SGTX public key to verify the cryptographic signature, confirming that the USTN was issued by SGTX and has not been tampered with.

3.0.11 Replay Attack Protection

Question: "What if someone copies SGTX-EG-26-F3A-15 and tries to use it for a different buyer?"

Answer: Impossible. The USTN is permanently bound to the specific shipment record in the database. When a terminal, customs, or payment system queries SGTX-EG-26-F3A-15, the system looks up that exact trade. If the buyer doesn't match, the container release API returns ERROR: USTN_MISMATCH.

Implementation:

rust

```
fn validate_ustn_binding(ustn: &str, buyer_gtid: &str) -> Result<bool> {  
    let shipment = db::get_shipment_by_ustn(ustn)?;  
    Ok(shipment.buyer_gtid == buyer_gtid)  
}
```

The counter guarantees global uniqueness and the database guarantees binding. No replay attack can work.

3.0.12 USTN Counters — Per Year, Per Trader

The sequence counter is per year and per trader. This ensures:

No two traders have overlapping sequences — Trader F3A has 1, 2, 3... Trader 7B3A also has 1, 2, 3... but they are in different USTNs.

Yearly reset — Each year, counters reset. SGTX-EG-26-F3A-1 and SGTX-EG-27-F3A-1 are distinct.

Global uniqueness — The combination of country + year + trader + sequence is always unique.

Infinite scalability — BIGINT counters support trillions of trades per trader per year.

Counter Table Example:

3.0.13 USTN Format Comparison

3.0.14 Database Schema Additions (Part 3.0)

sql

```
-- USTN counters (per year, per trader)
```

```
CREATE TABLE ustn_counters (  
    country CHAR(2) NOT NULL,  
    year CHAR(2) NOT NULL,  
    trader CHAR(3) NOT NULL,  
    counter BIGINT DEFAULT 0,  
    PRIMARY KEY (country, year, trader)
```

```

);
-- Atomic increment function
CREATE OR REPLACE FUNCTION next_ustn_seq(
p_country CHAR(2),
p_year CHAR(2),
p_trader CHAR(3)
)
RETURNS BIGINT AS $$
DECLARE
next_val BIGINT;
BEGIN
INSERT INTO ustn_counters (country, year, trader, counter)
VALUES (p_country, p_year, p_trader, 0)
ON CONFLICT (country, year, trader) DO NOTHING;
UPDATE ustn_counters
SET counter = counter + 1
WHERE country = p_country
AND year = p_year
AND trader = p_trader
RETURNING counter INTO next_val;
RETURN next_val;
END;
$$ LANGUAGE plpgsql;
-- Ships table extended with USTN fields
CREATE TABLE shipments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT UNIQUE NOT NULL,
ustn_counter BIGINT,
trader_id CHAR(3),
contract_id UUID REFERENCES contracts(id),
contract_shipment_id UUID REFERENCES contract_shipments(id),
exporter_gtid TEXT REFERENCES tenants(gtid),
importer_gtid TEXT REFERENCES tenants(gtid),
status shipment_status DEFAULT 'INITIATED',
incoterm TEXT NOT NULL,

```

```

gnn_risk JSONB,
causal_analysis JSONB,
container_numbers TEXT[],
vessel_name TEXT,
voyage TEXT,
booking_ref TEXT,
etd TIMESTAMPTZ,
eta TIMESTAMPTZ,
actual_departure TIMESTAMPTZ,
actual_arrival TIMESTAMPTZ,
risk_score INTEGER,
carbon_footprint_kg_co2e DECIMAL(12,2),
release_status TEXT DEFAULT 'PENDING',
document_embeddings vector(384),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- USTN verification tokens
CREATE TABLE loom_verification_tokens (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn),
token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '90 days',
created_by UUID REFERENCES employees(id),
revoked BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_shipments_ustn ON shipments(ustn);
CREATE INDEX idx_ustn_counters_lookup ON ustn_counters(country, year, trader);
CREATE INDEX idx_ustn_verification_tokens_token ON loom_verification_tokens(token);
3.0.15 Implementation Checklist (Part 3.0)
3.0.16 AI Authority Summary for Part 3.0
3.0.17 Quick Reference Card
text

```


SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Full Format: SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Example: SGTX-EG-26-F3A-1

3.1.1.2 Component Definitions

3.1.1.3 Trader Identifier Extraction

The trader identifier is the last 3 alphanumeric characters of the GTID's checksum (the final component after the sequence).

Examples:

Extraction Logic:

rust

```
fn extract_trader_id(gtid: &str) -> String {  
    let parts: Vec<&str> = gtid.split('-').collect();  
    // Get the last part (checksum) and take the last 3 chars  
    let checksum = parts[4];  
    let len = checksum.len();  
    checksum[len-3..len].to_string()  
}
```

3.1.1.4 Total Length

text

4 + 1 + 2 + 1 + 2 + 1 + 3 + 1 + variable = 15-22 characters

3.1.2 Generation Algorithm

3.1.2.1 Database Schema (PostgreSQL)

sql

-- USTN counters (per year, per trader)

```
CREATE TABLE ustn_counters (  
    country CHAR(2) NOT NULL,  
    year CHAR(2) NOT NULL,  
    trader CHAR(3) NOT NULL,  
    counter BIGINT DEFAULT 0,  
    PRIMARY KEY (country, year, trader)  
);
```

-- Atomic increment function

```
CREATE OR REPLACE FUNCTION next_ustn_seq(  
    p_country CHAR(2),  
    p_year CHAR(2),  
    p_trader CHAR(3)
```

```

)
RETURNS BIGINT AS $$
DECLARE
next_val BIGINT;
BEGIN
-- Insert if not exists (race condition safe due to ON CONFLICT)
INSERT INTO ustn_counters (country, year, trader, counter)
VALUES (p_country, p_year, p_trader, 0)
ON CONFLICT (country, year, trader) DO NOTHING;
-- Atomic increment and return
UPDATE ustn_counters
SET counter = counter + 1
WHERE country = p_country
AND year = p_year
AND trader = p_trader
RETURNING counter INTO next_val;
RETURN next_val;
END;
$$ LANGUAGE plpgsql;

```

3.1.2.2 Generation Process

3.1.2.3 Rust Implementation

```

rust
use chrono::Utc;
use sqlx::PgPool;

/// Extract trader ID from GTID (last 3 chars of checksum)
pub fn extract_trader_id(gtid: &str) -> String {
    let parts: Vec<&str> = gtid.split('-').collect();
    if parts.len() != 5 {
        return "XXX".to_string();
    }
    let checksum = parts[4];
    let len = checksum.len();
    checksum[len-3..len].to_string()
}

/// Generate USTN for a trade

```

```

pub async fn generate_ustn(
    pool: &PgPool,
    seller_gtid: &str,
) -> Result<String, Box<dyn std::error::Error>> {
    let parts: Vec<&str> = seller_gtid.split('-').collect();
    if parts.len() != 5 {
        return Err("Invalid GTID format".into());
    }
    let country = parts[1]; // EG
    let trader = extract_trader_id(seller_gtid); // F3A
    let year = Utc::now().format("%y").to_string(); // 26
    // Atomic sequence acquisition
    let seq: i64 = sqlx::query_scalar(
        "SELECT next_ustn_seq($1, $2, $3)"
    )
    .bind(country)
    .bind(&year)
    .bind(&trader)
    .fetch_one(pool)
    .await?;
    Ok(format!("SGTX-{}-{}-{}-{}", country, year, trader, seq))
}

```

3.1.2.4 Generation API Endpoint (Internal)

Endpoint: POST /v1/identity/ustn/generate (Internal — only called during contract lock)

Request:

json

```

{
  "seller_gtid": "SGTX-EG-TRD-002139-7F3A",
  "buyer_gtid": "SGTX-DE-TRD-001234-5B6C",
  "contract_id": "MC-20260415-001",
  "shipment_number": 1
}

```

Response:

json

```

{

```

```

"ustn": "SGTX-EG-26-F3A-1",
"country": "EG",
"year": "26",
"trader_id": "F3A",
"sequence": 1,
"generated_at": "2026-04-15T12:00:00Z",
"loom_hash": "sha256:a1b2c3..."
}

```

3.1.3 Validation Rules

Validation Implementation:

```

rust
fn validate_ustn(ustn: &str) -> Result<bool, String> {
    let re = regex::Regex::new(
        r"^SGTX-[A-Z]{2}-\d{2}-[A-Z0-9]{3,4}-\d+$"
    ).unwrap();
    if !re.is_match(ustn) {
        return Err("Invalid USTN format. Expected SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}".into());
    }
    let parts: Vec<&str> = ustn.split('-').collect();
    let country = parts[1];
    let year = parts[2];
    let trader = parts[3];
    let seq = parts[4];
    // Validate country exists
    if !country_exists(country) {
        return Err(format!("Country '{}' not found in jurisdictions table", country));
    }
    // Validate year (current or previous)
    let current_year = Utc::now().format("%Y").to_string();
    if year != current_year && year != (current_year.parse::<i32>().unwrap() - 1).to_string() {
        return Err(format!("Year '{}' is not current or previous year", year));
    }
    // Validate sequence is positive integer
    let seq_num: i64 = seq.parse().map_err(|_| "Sequence must be a number".to_string())?;
    if seq_num < 1 {

```


Links back to the original USTN via parent_ustn.

Has its own lifecycle (DISTRESS_SALE_PENDING → DISTRESS_MICROCONTRACT_LOCKED → DISTRESS_SALE_COMPLETED).

Original documents are copied (with redacted pricing if confidential).

3.1.8 USTN in Documents — Mandatory Inclusion

Every document generated by or uploaded to SGTX must contain the USTN in a standardised location (e.g., header, footer, or QR code). This includes:

QR Code Encoding:

text

<https://sgtx.io/ustn/SGTX-EG-26-F3A-1>

3.1.9 USTN Resolution Service

Endpoint: GET /v1/ustn/{ustn} (authenticated, role-based filtering)

Returns the USTN master object filtered by the requester's permissions. A logistics provider sees only their services; a buyer sees full commercial terms but not the seller's internal costs; a financier sees all trade data; a government official sees only compliance documents.

Example Request (Buyer):

bash

```
curl -H "Authorization: Bearer <JWT>" \
```

```
https://api.sgtx.io/v1/ustn/SGTX-EG-26-F3A-1
```

Response (Filtered for Buyer):

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "status": "IN_TRANSIT",
  "parties": {
    "exporter": {
      "gtid": "SGTX-EG-TRD-002139-7F3A",
      "name": "Strawberry Export Co."
    },
    "importer": {
      "gtid": "SGTX-DE-TRD-001234-5B6C",
      "name": "European Importer GmbH"
    }
  },
  "goods": {
    "description": "Frozen strawberries",
    "invoice_value": {
```

```

"amount": 100000,
"currency": "USD"
},
"logistics": {
  "vessel": "MSC FIONA",
  "eta": "2026-05-05T14:00:00Z"
},
"payment_plan": {
  "stage1": "SETTLED",
  "stage2": "PENDING"
},
"timeline": [ ... ]
}

```

Rate Limits: 100 requests per minute per tenant.

3.1.10 USTN Replay Attack Protection

Question: "What if someone copies SGTX-EG-26-F3A-15 and tries to use it for a different buyer?"

Answer: Impossible. The USTN is permanently bound to the specific shipment record in the database. When a terminal, customs, or payment system queries SGTX-EG-26-F3A-15, the system looks up that exact trade. If the buyer doesn't match, the container release API returns ERROR: USTN_MISMATCH.

Implementation:

```

rust
fn validate_ustn_binding(ustn: &str, buyer_gtid: &str) -> Result<bool> {
  let shipment = db::get_shipment_by_ustn(ustn)?;
  Ok(shipment.buyer_gtid == buyer_gtid)
}

```

The counter guarantees global uniqueness and the database guarantees binding. No replay attack can work.

3.1.11 USTN Counters — Per Year, Per Trader

The sequence counter is per year and per trader. This ensures:

No two traders have overlapping sequences — Trader F3A has 1, 2, 3... Trader 7B3A also has 1, 2, 3... but they are in different USTNs.

Yearly reset — Each year, counters reset. SGTX-EG-26-F3A-1 and SGTX-EG-27-F3A-1 are distinct.

Global uniqueness — The combination of country + year + trader + sequence is always unique.

Infinite scalability — BIGINT counters support trillions of trades per trader per year.

Counter Table Example:

3.1.12 USTN Format Comparison

3.1.13 Database Schema Additions (Part 3.1)

sql

-- USTN counters (per year, per trader)

```
CREATE TABLE ustn_counters (  
country CHAR(2) NOT NULL,  
year CHAR(2) NOT NULL,  
trader CHAR(3) NOT NULL,  
counter BIGINT DEFAULT 0,  
PRIMARY KEY (country, year, trader)  
);
```

-- Atomic increment function

```
CREATE OR REPLACE FUNCTION next_ustn_seq(  
p_country CHAR(2),  
p_year CHAR(2),  
p_trader CHAR(3)  
)  
RETURNS BIGINT AS $$  
DECLARE  
next_val BIGINT;  
BEGIN  
INSERT INTO ustn_counters (country, year, trader, counter)  
VALUES (p_country, p_year, p_trader, 0)  
ON CONFLICT (country, year, trader) DO NOTHING;  
UPDATE ustn_counters  
SET counter = counter + 1  
WHERE country = p_country  
AND year = p_year  
AND trader = p_trader  
RETURNING counter INTO next_val;  
RETURN next_val;  
END;  
$$ LANGUAGE plpgsql;
```

-- Ships table extended with USTN fields

```
ALTER TABLE shipments ADD COLUMN IF NOT EXISTS ustn_counter BIGINT;  
ALTER TABLE shipments ADD COLUMN IF NOT EXISTS trader_id CHAR(3);
```

-- USTN verification tokens

```
CREATE TABLE loom_verification_tokens (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT NOT NULL REFERENCES shipments(ustn),  
  token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),  
  expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '90 days',  
  created_by UUID REFERENCES employees(id),  
  revoked BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_shipments_ustn ON shipments(ustn);  
CREATE INDEX idx_ustn_counters_lookup ON ustn_counters(country, year, trader);  
CREATE INDEX idx_ustn_verification_tokens_token ON loom_verification_tokens(token);
```

3.1.14 USTN in API Calls — Mandatory Reference

All SGTx API calls that create or modify trade-related entities must include the USTN (or microUSTN) in the request body or as a query parameter.

If an API call omits the USTN where required, the Governor returns a 400 error with a plain-language message: "Missing USTN parameter. Every trade action must reference its shipment identifier."

3.1.15 USTN QR Code & Physical Tagging

Every USTN is encoded as a QR code that can be printed on shipping labels, packing lists, and customs documents. The QR code contains:

The full USTN string: SGTx-EG-26-F3A-1

A URL to the public verification page: <https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1>

A cryptographic signature (optional, for high-value goods)

QR Code Payload:

json

```
{  
  "ustn": "SGTX-EG-26-F3A-1",  
  "verify_url": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1",  
  "verifiable_credential": {  
    "type": "VerifiableCredential",  
    "issuer": "https://sgtx.io/issuers/platform",  
    "proof": {  
      "type": "Ed25519Signature2020",  
      "signature": "base58..."
```


The USTN Lifecycle is the backbone of the SGTX trade execution engine. It ensures that every party — buyer, seller, logistics providers, financiers, customs, and government — knows exactly where the shipment stands at any given moment.

3.2.1 USTN Lifecycle State Machine

The USTN transitions through a predefined set of statuses. Each status has a clear definition, trigger, and responsible party.

3.2.1.1 Complete Status Table

3.2.1.2 Example Timeline for a Healthy Trade

text

INITIATED → STAGE1_PENDING → STAGE1_SETTLED → CUSTOMS_SUBMITTED → BOOKED

↓

LOADED → DEPARTED → IN_TRANSIT → ARRIVED → CUSTOMS_IMPORT → DELIVERED → SETTLED → COMPLETED

USTN Generated at Lock: SGTX-EG-26-F3A-1

3.2.2 Detailed Status Descriptions

3.2.2.1 INITIATED

Description: The trade request has been created by the buyer. The USTN has not yet been generated because the contract is not locked.

Trigger: Buyer submits the structured trade request form.

Responsible Party: Buyer

Actions Allowed: Seller can accept, decline, or propose modifications.

Next Status: STAGE1_PENDING (after seller submits quote) or CANCELLED (if trade is cancelled).

UI Display: Buyer sees "Trade Request Submitted — Awaiting Seller Response". Seller sees "New Trade Request — Review".

3.2.2.2 STAGE1_PENDING

Description: The seller has submitted a quote, and the Stage 1 payment (SGTX fee, government fees, lab, broker, trucking, etc.) is due. The USTN is still not generated because the fee must be paid first.

Trigger: Seller submits the final quote.

Responsible Party: Seller (or buyer depending on incoterm).

Actions Allowed: Seller pays the Stage 1 fee via PSP.

Next Status: STAGE1_SETTLED (after PSP webhook confirms payment).

UI Display: Seller sees "Pay Stage 1 Fee — Due Now". Buyer sees "Awaiting Seller Fee Payment".

3.2.2.3 STAGE1_SETTLED

Description: All mandatory Stage 1 invoices (SGTX fee, customs inspection, lab, broker, trucking, port charges, insurance, CargoX ACI) have been paid. The FeeLock becomes ACTIVE. The USTN is now generated at this point.

Trigger: PSP webhook confirms all split payments.

Responsible Party: System (automatic).

Actions Allowed: None — system automatically proceeds.

Next Status: CUSTOMS_SUBMITTED (after CargoX ACID and Nafeza SAD submission).

UI Display: All parties see "Fee Locked — USTN Generated: SGTX-EG-26-F3A-1".

3.2.2.4 CUSTOMS_SUBMITTED

Description: The customs declaration (SAD) has been submitted to Nafeza, and the ACID has been obtained from CargoX. The shipment is ready for physical execution.

Trigger: After Stage 1 payment, SGTX automatically calls CargoX and Nafeza APIs.

Responsible Party: System (automatic) or Broker (if broker certification selected).

Actions Allowed: None — system proceeds.

Next Status: BOOKED (after shipping line confirms booking).

UI Display: "Customs Declaration Filed — Reference: SAD20260415-000147, ACID: ACI20260415-0001".

3.2.2.5 BOOKED

Description: The shipping line has confirmed the booking, providing a booking reference, vessel name, voyage, ETD, and ETA. The container is reserved.

Trigger: Shipping line portal confirms booking (manual or API).

Responsible Party: SHIP user.

Actions Allowed: SHIP can update ETD/ETA; seller can request changes.

Next Status: LOADED (after container is loaded on vessel).

UI Display: "Booking Confirmed — Vessel: MSC FIONA, Voyage: MSF123E, ETD: 2026-04-20 08:00".

3.2.2.6 LOADED

Description: The container has been loaded onto the vessel. This milestone is typically confirmed via barcode scan or terminal gate-out event.

Trigger: Terminal gate-out scan OR LSP driver confirms loading.

Responsible Party: LSP or SHIP.

Actions Allowed: None — automatic progression.

Next Status: DEPARTED (after vessel departs).

UI Display: "Container Loaded — Gate-Out Time: 2026-04-20 09:30".

3.2.2.7 DEPARTED

Description: The vessel has departed from the port of loading. The shipment is now in transit.

Trigger: Shipping line updates milestone (manual) OR AIS data confirms departure.

Responsible Party: SHIP user or System (AIS).

Actions Allowed: SHIP can update ETA if schedule changes.

Next Status: IN_TRANSIT (after AIS confirms vessel at sea for >12h).

UI Display: "Vessel Departed — Actual Departure: 2026-04-20 09:30".

3.2.2.8 IN_TRANSIT

Description: The vessel is at sea. The system automatically tracks position via AIS. This status is auto-confirmed when the vessel has been at sea for more than 12 hours.

Trigger: AIS position >12h after departure.

Responsible Party: System (automatic).

Actions Allowed: None — advisory only.

Next Status: ARRIVED (after vessel arrives at destination port).

UI Display: "Vessel In Transit — Position: South of Crete, ETA: 2026-05-05 14:00".

3.2.2.9 ARRIVED

Description: The vessel has arrived at the destination port. The shipment is ready for customs clearance and delivery.

Trigger: Shipping line updates milestone (manual) OR AIS data confirms arrival.

Responsible Party: SHIP user or System (AIS).

Actions Allowed: Buyer or broker can initiate import customs clearance.

Next Status: CUSTOMS_IMPORT (after buyer/broker starts clearance).

UI Display: "Vessel Arrived — Actual Arrival: 2026-05-05 14:00".

3.2.2.10 CUSTOMS_IMPORT

Description: Import customs clearance has been initiated. Documents are being verified, and duties are being calculated.

Trigger: Buyer or broker initiates import clearance.

Responsible Party: Buyer or CBR.

Actions Allowed: Upload missing documents, pay duties (deferred if eligible).

Next Status: DELIVERED (after buyer confirms receipt of goods).

UI Display: "Import Customs Clearance In Progress — Documents: 5/5 Verified".

3.2.2.11 DELIVERED

Description: The buyer has confirmed receipt of the goods. The shipment is physically complete.

Trigger: Buyer clicks "Confirm Delivery" (one click or voice command).

Responsible Party: Buyer.

Actions Allowed: None — system proceeds to settlement.

Next Status: SETTLED (after trade principal is paid).

UI Display: "Goods Delivered — Confirmed by Buyer".

3.2.2.12 SETTLED

Description: The trade principal (amount due to seller) has been paid by the buyer. The trade is financially complete.

Trigger: PSP settlement webhook confirms payment.

Responsible Party: System (automatic).

Actions Allowed: None — system proceeds to archival.

Next Status: COMPLETED (after 30 days of SETTLED).

UI Display: "Trade Settled — Payment of \$30,000 Confirmed".

3.2.2.13 COMPLETED

Description: The trade is fully complete and archived. All milestones closed, documents archived, and records retained for regulatory periods.

Trigger: System after 30 days of SETTLED.

Responsible Party: System (automatic).

Actions Allowed: None — read-only.

Next Status: None (terminal state).

UI Display: "Trade Completed — Archived".

3.2.2.14 DISPUTED

Description: A dispute has been raised by any party (quality, delay, non-payment, etc.). The USTN status is frozen pending resolution.

Trigger: Any party files a dispute.

Responsible Party: Any party.

Actions Allowed: Parties engage in mediation, invite experts, accept settlement.

Next Status: Can return to previous status (if resolved) or proceed to COMPLETED (if settled).

UI Display: "Dispute Filed — Mediation In Progress".

3.2.2.15 DISTRESSED

Description: The seller has declared distress on a portion of the shipment (e.g., damaged goods). A microUSTN may be generated for the distressed lot.

Trigger: Seller declares distressed cargo.

Responsible Party: Seller.

Actions Allowed: Triage, accelerated outreach, microcontract.

Next Status: Can return to normal status (if resolved) or proceed to microcontract.

UI Display: "Distressed Cargo Declared — Pallets LEM003-005 affected".

3.2.2.16 CANCELLED

Description: The trade has been cancelled before completion. This can be mutual or forced by system (e.g., sanctions hit, fraud).

Trigger: Mutual agreement or system auto-cancel.

Responsible Party: Any party with consent or System.

Actions Allowed: None — read-only.

Next Status: None (terminal state).

UI Display: "Trade Cancelled — Reason: Mutual Agreement".

3.2.3 Status Transition Rules (Governor Enforcement)

All status transitions are validated by the Governor. The following rules apply:

3.2.4 USTN Master Object — Complete JSON Schema

The USTN master object is stored in the shipments table. It is the single source of truth for the entire trade.

Example USTN Master Object (with new format):

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "created_at": "2026-04-15T12:00:00Z",
  "updated_at": "2026-06-07T10:00:00Z",
  "status": "DELIVERED",
  "risk_assessment": {
    "platform_risk_score": 12,
    "gnn_risk": {
      "sanctions_proximity": 4,
      "graph_risk_score": 8,
      "explanation": "No indirect sanctions links detected within 2 hops."
    },
    "causal_analysis": null
  },
  "parties": {
    "exporter": {
      "gtid": "SGTX-EG-TRD-002139-7F3A",
      "trader_id": "F3A",
      "legal_name": "Strawberry Export Co.",
      "trust_score": 92,
      "jurisdiction": "EG"
    },
    "importer": {
      "gtid": "SGTX-DE-TRD-001234-5B6C",
      "trader_id": "5B6C",
      "legal_name": "European Importer GmbH",
      "trust_score": 88,
      "jurisdiction": "DE"
    }
  },
  "goods": {
    "hs_code": "0811.10.90",
    "description": "Frozen strawberries, IQF, Grade A",
    "net_weight_kg": 20000,
```

```
"gross_weight_kg": 20500,
"invoice_value": {
  "amount": 100000.00,
  "currency": "USD"
},
"incoterm": "CFR",
"container_count": 2,
"container_numbers": ["MEDU1234567", "MEDU1234568"]
},
"documents": {
  "eta_invoice_uuid": "INV-abc123",
  "acid": "ACI20260415-0001",
  "nafeza_declaration_id": "SAD20260415-000147",
  "phyto_certificate": "PC-20260415-147",
  "health_certificate": "HC-20260415-147",
  "certificate_of_origin": "COO-20260415-147",
  "bill_of_lading": {
    "number": "MEDU1234567",
    "type": "eBL",
    "issuer": "MSC",
    "issued_at": "2026-04-16T10:00:00Z",
    "url": "sgtx://documents/eb1-123"
  },
  "packing_list_hash": "sha256:abc123...",
  "qc_report": {
    "provider": "SGS Inspection",
    "verdict": "CONDITIONAL_PASS",
    "overrides": 1,
    "report_url": "sgtx://documents/qc-001"
  }
},
"logistics": {
  "trucking": {
    "provider_gtid": "SGTX-EG-LSP-001",
    "provider_name": "Fast Trucking Co.",
```

```
"quote_id": "QT-001",
"fee": 300,
"status": "PAID"
},
"shipping_line": {
"provider_gtid": "SGTX-EG-SHIP-001",
"provider_name": "MSC Egypt",
"booking_ref": "MAEU876543210",
"vessel": "MSC FIONA",
"voyage": "MSF123E",
"etd": "2026-04-20T08:00:00Z",
"eta": "2026-05-05T14:00:00Z",
"actual_departure": "2026-04-20T09:30:00Z",
"actual_arrival": null,
"freight_invoice": {
"amount": 4200,
"status": "CREDIT",
"due_date": "2026-06-05"
}
},
"customs_broker": {
"certification": {
"quote_id": "CBRQ-001",
"fee": 150,
"status": "PAID",
"certified_at": "2026-04-16T09:00:00Z"
},
"physical_handling": null,
"storage": null
}
},
"payment_plan": {
"stage1": {
"status": "SETTLED",
"settled_at": "2026-04-15T13:00:00Z",
```

```
"transactions": [
  {"payee": "SGTX", "amount": 1353.00, "currency": "USD"},
  {"payee": "Fast Trucking Co.", "amount": 300.00, "currency": "USD"},
  {"payee": "SGS Lab", "amount": 200.00, "currency": "USD"},
  {"payee": "Customs Broker (cert)", "amount": 150.00, "currency": "USD"},
  {"payee": "Egypt Customs", "amount": 200.00, "currency": "USD"}
],
"stage2": {
  "status": "PENDING",
  "transactions": [
    {"payee": "MSC", "amount": 4200.00, "currency": "USD", "condition": "CREDIT", "due_date": "2026-06-05"}
  ],
  "deferred": {
    "status": "GUARANTEE_HELD",
    "guarantee_amount": 10000.00,
    "trigger_milestone": "CUSTOMS_IMPORT",
    "expiry_date": "2026-05-15T23:59:59Z"
  },
  "timeline": [
    {"timestamp": "2026-04-15T09:00:00Z", "event": "TRADE_INITIATED", "actor": "buyer@importer.com"},
    {"timestamp": "2026-04-15T10:00:00Z", "event": "SELLER_ACCEPTED", "actor": "seller@exporter.com"},
    {"timestamp": "2026-04-15T10:30:00Z", "event": "QUOTE_SUBMITTED", "actor": "seller@exporter.com"},
    {"timestamp": "2026-04-15T11:00:00Z", "event": "CONTRACT_SIGNED", "actor": "both"},
    {"timestamp": "2026-04-15T12:00:00Z", "event": "USTN_GENERATED", "actor": "system"},
    {"timestamp": "2026-04-15T13:00:00Z", "event": "STAGE1_PAID", "actor": "system"},
    {"timestamp": "2026-04-15T13:05:00Z", "event": "CUSTOMS_DECLARATION_SUBMITTED", "actor": "broker"},
    {"timestamp": "2026-04-16T09:00:00Z", "event": "BROKER_CERTIFIED", "actor": "broker"},
    {"timestamp": "2026-04-16T10:00:00Z", "event": "BOOKING_CONFIRMED", "actor": "shipping_line"},
    {"timestamp": "2026-04-20T08:00:00Z", "event": "VESSEL_DEPARTED", "actor": "shipping_line"},
    {"timestamp": "2026-05-05T14:00:00Z", "event": "VESSEL_ARRIVED", "actor": "shipping_line"}
  ],
  "optional_services": {
```

```
"laboratory": {
  "provider_gtid": "SGTX-EG-LAB-001",
  "quote_id": "LABQ-001",
  "fee": 200,
  "status": "PAID",
  "results": {
    "cypermethrin": {"value": 0.02, "limit": 0.05, "pass": true},
    "chlorpyrifos": {"value": 0.008, "limit": 0.01, "pass": true}
  }
},
"qc_inspection": {
  "provider_gtid": "SGTX-EG-QC-002",
  "quote_id": "QCQ-001",
  "fee": 500,
  "status": "PAID",
  "report": {
    "verdict": "CONDITIONAL_PASS",
    "defects": [
      {"pallet_id": "OR019", "defect": "mould", "severity": "medium", "ai_confidence": 92, "inspector_override": false},
      {"pallet_id": "OR020", "defect": "bruising", "severity": "low", "ai_confidence": 88, "inspector_override": true, "override_reason": "Dark spot is surface bruise, no mould."}
    ]
  }
},
"blockchain_anchor": {
  "txid": "0xabc123...",
  "merkle_root": "sha256:...",
  "network": "polygon-mainnet",
  "pqc_signature": "dilithium3:..."
},
"links": {
  "trade_command_center": "https://sgtx.io/trade/SGTX-EG-26-F3A-1",
  "export_pdf": "https://sgtx.io/api/v1/shipment/SGTX-EG-26-F3A-1/pdf"
}
```

```
}
```

3.2.5 USTN Resolution Service (Detailed)

Endpoint: GET /v1/ustn/{ustn} (authenticated, role-based filtering)

Returns the USTN master object filtered by the requester's permissions.

3.2.5.1 Permission Filtering Rules

Example Request (Buyer):

```
bash
```

```
curl -H "Authorization: Bearer <JWT>" \
```

```
https://api.sgtx.io/v1/ustn/SGTX-EG-26-F3A-1
```

Response (Filtered for Buyer):

```
json
```

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "status": "IN_TRANSIT",
  "parties": {
    "exporter": {
      "gtid": "SGTX-EG-TRD-002139-7F3A",
      "name": "Strawberry Export Co."
    },
    "importer": {
      "gtid": "SGTX-DE-TRD-001234-5B6C",
      "name": "European Importer GmbH"
    }
  },
  "goods": {
    "description": "Frozen strawberries",
    "invoice_value": {
      "amount": 100000,
      "currency": "USD"
    }
  },
  "logistics": {
    "vessel": "MSC FIONA",
    "eta": "2026-05-05T14:00:00Z"
  },
}
```

```

"payment_plan": {
"stage1": "SETTLED",
"stage2": "PENDING"
},
"timeline": [ ... ]
}

```

Rate Limits: 100 requests per minute per tenant.

Error Responses:

3.2.6 USTN in Documents — Mandatory Inclusion

Every document generated by or uploaded to SGTX must contain the USTN in a standardised location.

AI Assistance for Document Generation: When a document is generated, the system automatically embeds the USTN and a QR code. No manual entry required. If a user uploads a document, the system checks that the USTN is present (and matches the trade) before accepting it — if missing, it returns a warning and suggests adding it.

Physical Marking: For paper documents, the USTN is printed as both human-readable text and a QR code. The QR code encodes the full USTN URL: <https://sgtx.io/ustn/SGTX-EG-26-F3A-1>. Scanning the QR code on a mobile device opens the Trade Command Center (read-only view) — no login required for public verification (if the tenant has enabled public sharing).

3.2.7 USTN in API Calls — Mandatory Reference

All SGTX API calls that create or modify trade-related entities must include the USTN (or microUSTN) in the request body or as a query parameter.

If an API call omits the USTN where required, the Governor returns a 400 error with a plain-language message: "Missing USTN parameter. Every trade action must reference its shipment identifier."

3.2.8 USTN for Multi-Shipment Contracts

In a multi-shipment contract, each shipment receives its own USTN when it locks.

Example — Three Shipments:

text

Master Contract: MC-20260415-001

■

■■■ Shipment 1: SGTX-EG-26-F3A-21 (15 Jul 2026)

■■■ Shipment 2: SGTX-EG-26-F3A-22 (15 Aug 2026)

■■■ Shipment 3: SGTX-EG-26-F3A-23 (15 Sep 2026)

Key Properties:

Each shipment has its own USTN and locks independently.

Per-shipment fee collection.

Independent execution — a delay in one shipment does not block others.

Buyer and seller see all shipments in the Trade Command Center under the master contract.

AI Assistance for Multi-Shipment: The Smart Inbox groups pending actions by shipment, with a clear label "Shipment 2 of 3". The user can toggle between shipments in the Trade Command Center via a dropdown that autocompletes from the list of USTNs under the master contract.

3.2.9 USTN for Distressed Micro-Contracts

When a seller declares distress on a portion of a shipment, SGTX generates a microUSTN for the distressed lot.

text

Original USTN: SGTX-EG-26-F3A-15

MicroUSTN: SGTX-EG-26-F3A-26

Parent Link: `micro_contracts.parent_ustn = SGTX-EG-26-F3A-15`

Key Properties:

Has its own lifecycle: `DISTRESS_SALE_PENDING` → `DISTRESS_MICROCONTRACT_LOCKED` → `DISTRESS_SALE_COMPLETED`

References the original USTN via `parent_ustn`

Original documents are copied (with redacted pricing if confidential)

Separate SGTX fee ($1.5\% \times \text{country factor}$)

3.2.10 USTN QR Code & Physical Tagging

Every USTN is encoded as a QR code that can be printed on shipping labels, packing lists, and customs documents.

QR Code Payload:

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "verify_url": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1",
  "verifiable_credential": {
    "type": "VerifiableCredential",
    "issuer": "https://sgtx.io/issuers/platform",
    "proof": {
      "type": "Ed25519Signature2020",
      "signature": "base58..."
    }
  }
}
```

AI Assistance for QR Scanning: The SGTX mobile app scans the QR code and automatically opens the Trade Command Center (or the public verification page). If the user is not logged in, the app prompts for login (or shows public information if enabled).

Offline Verification: A receiver with no internet can scan the QR code, and the app uses a cached version of the SGTX public key to verify the cryptographic signature, confirming that the USTN was issued by

SGTX and has not been tampered with.

3.2.11 Replay Attack Protection

Question: "What if someone copies SGTX-EG-26-F3A-15 and tries to use it for a different buyer?"

Answer: Impossible. The USTN is permanently bound to the specific shipment record in the database. When a terminal, customs, or payment system queries SGTX-EG-26-F3A-15, the system looks up that exact trade. If the buyer doesn't match, the container release API returns ERROR: USTN_MISMATCH.

Implementation:

rust

```
fn validate_ustn_binding(ustn: &str, buyer_gtid: &str) -> Result<bool> {  
    let shipment = db::get_shipment_by_ustn(ustn)?;  
    Ok(shipment.buyer_gtid == buyer_gtid)  
}
```

The counter guarantees global uniqueness and the database guarantees binding. No replay attack can work.

3.2.12 Edge Cases & Error Handling

3.2.13 Governance & Compliance Gates

3.2.14 Database Schema Additions (Part 3.2)

sql

-- Shipments table (core USTN master object)

```
CREATE TABLE shipments (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    ustn TEXT UNIQUE NOT NULL,  
    ustn_counter BIGINT,  
    trader_id CHAR(3),  
    contract_id UUID REFERENCES contracts(id),  
    contract_shipment_id UUID REFERENCES contract_shipments(id),  
    exporter_gtid TEXT REFERENCES tenants(gtid),  
    importer_gtid TEXT REFERENCES tenants(gtid),  
    status shipment_status DEFAULT 'INITIATED',  
    incoterm TEXT NOT NULL,  
    gnn_risk JSONB,  
    causal_analysis JSONB,  
    container_numbers TEXT[],  
    vessel_name TEXT,  
    voyage TEXT,  
    booking_ref TEXT,
```

```

etd TIMESTAMPTZ,
eta TIMESTAMPTZ,
actual_departure TIMESTAMPTZ,
actual_arrival TIMESTAMPTZ,
risk_score INTEGER,
carbon_footprint_kg_co2e DECIMAL(12,2),
release_status TEXT DEFAULT 'PENDING',
document_embeddings vector(384),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Shipment milestones (historical timeline)
CREATE TABLE shipment_milestones (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
milestone TEXT NOT NULL,
confirmed_at TIMESTAMPTZ NOT NULL,
confirmed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
confirmation_method TEXT, -- 'barcode', 'voice', 'manual', 'api', 'auto_consensus'
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
scanned_barcode_id UUID,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- USTN verification tokens (public verification)
CREATE TABLE loom_verification_tokens (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn),
token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '90 days',
created_by UUID REFERENCES employees(id),
revoked BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_shipments_ustn ON shipments(ustn);

```



```
"ustn": {
  "type": "string",
  "pattern": "^SGTX-[A-Z]{2}-\\d{2}-[A-Z0-9]{3,4}-\\d+$",
  "description": "Universal Shipment Tracking Number (human-readable format)",
  "example": "SGTX-EG-26-F3A-1"
},
"ustn_counter": {
  "type": "integer",
  "description": "Atomic sequence number per year per trader",
  "example": 1
},
"trader_id": {
  "type": "string",
  "pattern": "^[A-Z0-9]{3,4}$",
  "description": "Short trader identifier from GTID checksum",
  "example": "F3A"
},
"created_at": {
  "type": "string",
  "format": "date-time",
  "description": "Timestamp when the USTN was generated",
  "example": "2026-04-15T12:00:00Z"
},
"updated_at": {
  "type": "string",
  "format": "date-time",
  "description": "Timestamp when the record was last updated",
  "example": "2026-06-07T10:00:00Z"
},
"status": {
  "type": "string",
  "enum": [
    "INITIATED", "STAGE1_PENDING", "STAGE1_SETTLED",
    "CUSTOMS_SUBMITTED", "BOOKED", "LOADED",
    "DEPARTED", "IN_TRANSIT", "ARRIVED",
```

```
"CUSTOMS_IMPORT", "DELIVERED", "SETTLED",  
"COMPLETED", "DISPUTED", "DISTRESSED", "CANCELLED"  
],  
"description": "Current lifecycle status of the shipment",  
"example": "DELIVERED"  
},  
"risk_assessment": {  
  "type": "object",  
  "properties": {  
    "platform_risk_score": {  
      "type": "integer",  
      "minimum": 0,  
      "maximum": 100,  
      "description": "Internal risk score computed by platform",  
      "example": 12  
    },  
    "gnn_risk": {  
      "type": "object",  
      "properties": {  
        "sanctions_proximity": {  
          "type": "integer",  
          "description": "Minimum hops to a sanctioned entity",  
          "example": 4  
        },  
        "graph_risk_score": {  
          "type": "integer",  
          "minimum": 0,  
          "maximum": 100,  
          "description": "Composite adversarial risk score",  
          "example": 8  
        },  
        "explanation": {  
          "type": "string",  
          "description": "Plain-language explanation of risk",  
          "example": "No indirect sanctions links detected within 2 hops."  
        }  
      }  
    }  
  }  
}
```

```
}
}
},
"causal_analysis": {
  "type": "object",
  "description": "Causal analysis output (if triggered)",
  "nullable": true
}
}
},
"parties": {
  "type": "object",
  "properties": {
    "exporter": {
      "type": "object",
      "required": ["gtid", "trader_id", "legal_name", "jurisdiction"],
      "properties": {
        "gtid": {
          "type": "string",
          "pattern": "^SGTX-[A-Z]{2}-[A-Z]{3,4}-\\d{6}-[A-Z0-9]{4}$",
          "example": "SGTX-EG-TRD-002139-7F3A"
        },
        "trader_id": {
          "type": "string",
          "example": "F3A"
        },
        "legal_name": {
          "type": "string",
          "example": "Strawberry Export Co."
        }
      },
      "trust_score": {
        "type": "integer",
        "minimum": 0,
        "maximum": 1000,
        "example": 92
      }
    }
  }
}
```

```
,
"jurisdiction": {
  "type": "string",
  "pattern": "^[A-Z]{2}$",
  "example": "EG"
}
},
"importer": {
  "type": "object",
  "required": ["gtid", "trader_id", "legal_name", "jurisdiction"],
  "properties": {
    "gtid": {
      "type": "string",
      "pattern": "^SGTX-[A-Z]{2}-[A-Z]{3,4}-\\d{6}-[A-Z0-9]{4}$",
      "example": "SGTX-DE-TRD-001234-5B6C"
    },
    "trader_id": {
      "type": "string",
      "example": "5B6C"
    },
    "legal_name": {
      "type": "string",
      "example": "European Importer GmbH"
    },
    "trust_score": {
      "type": "integer",
      "minimum": 0,
      "maximum": 1000,
      "example": 88
    },
    "jurisdiction": {
      "type": "string",
      "pattern": "^[A-Z]{2}$",
      "example": "DE"
    }
  }
}
```



```
}
}
},
"additional_parties": {
  "type": "array",
  "description": "Other parties (LSP, SHIP, LAB, QC, CBR, FIN, etc.)",
  "items": {
    "type": "object",
    "properties": {
      "type": {
        "type": "string",
        "enum": ["LSP", "SHIP", "LAB", "QC", "CBR", "FIN", "GOV"]
      },
      "gtid": {
        "type": "string",
        "pattern": "^SGTX-[A-Z]{2}-[A-Z]{3,4}-\\d{6}-[A-Z0-9]{4}$"
      },
      "trader_id": {
        "type": "string"
      },
      "legal_name": {
        "type": "string"
      },
      "role": {
        "type": "string",
        "description": "Specific role in this trade"
      }
    }
  }
},
"goods": {
  "type": "object",
  "required": ["hs_code", "description", "invoice_value", "incoterm"],
```

```
"properties": {
  "hs_code": {
    "type": "string",
    "pattern": "^[\\d{4}\\.\\d{2}$|^\\d{6}$",
    "description": "Harmonized System code",
    "example": "0811.10.90"
  },
  "description": {
    "type": "string",
    "description": "Commodity description",
    "example": "Frozen strawberries, IQF, Grade A"
  },
  "net_weight_kg": {
    "type": "number",
    "minimum": 0,
    "description": "Net weight in kilograms",
    "example": 20000
  },
  "gross_weight_kg": {
    "type": "number",
    "minimum": 0,
    "description": "Gross weight in kilograms",
    "example": 20500
  },
  "invoice_value": {
    "type": "object",
    "required": ["amount", "currency"],
    "properties": {
      "amount": {
        "type": "number",
        "minimum": 0,
        "example": 100000.00
      },
      "currency": {
        "type": "string",
```

```
"pattern": "^[A-Z]{3}$",
"example": "USD"
}
},
"incoterm": {
"type": "string",
"enum": ["EXW", "FOB", "CFR", "CIF", "CPT", "CIP", "DAP", "DPU", "DDP"],
"example": "CFR"
},
"container_count": {
"type": "integer",
"minimum": 1,
"example": 2
},
"container_numbers": {
"type": "array",
"items": {
"type": "string",
"pattern": "^[A-Z]{4}\\d{7}$"
},
"example": ["MEDU1234567", "MEDU1234568"]
},
"quantity_tolerance": {
"type": "number",
"minimum": 0,
"maximum": 20,
"description": "Tolerance percentage",
"example": 5
},
"partial_shipment_allowed": {
"type": "boolean",
"example": true
},
"transshipment_allowed": {
```

```
"type": "boolean",
"example": false
}
},
"documents": {
  "type": "object",
  "properties": {
    "eta_invoice_uuid": {
      "type": "string",
      "format": "uuid",
      "example": "INV-abc123"
    },
    "acid": {
      "type": "string",
      "pattern": "^ACI\\d{8}-\\d{4}$",
      "example": "ACI20260415-0001"
    },
    "nafeza_declaration_id": {
      "type": "string",
      "example": "SAD20260415-000147"
    },
    "phyto_certificate": {
      "type": "string",
      "example": "PC-20260415-147"
    },
    "health_certificate": {
      "type": "string",
      "example": "HC-20260415-147"
    },
    "certificate_of_origin": {
      "type": "string",
      "example": "COO-20260415-147"
    },
    "bill_of_lading": {
```

```
"type": "object",
"properties": {
  "number": {
    "type": "string",
    "example": "MEDU1234567"
  },
  "type": {
    "type": "string",
    "enum": ["eBL", "paper"],
    "example": "eBL"
  },
  "issuer": {
    "type": "string",
    "example": "MSC"
  },
  "issued_at": {
    "type": "string",
    "format": "date-time",
    "example": "2026-04-16T10:00:00Z"
  },
  "url": {
    "type": "string",
    "format": "uri",
    "example": "sgtx://documents/eb1-123"
  }
},
"packing_list_hash": {
  "type": "string",
  "pattern": "^sha256:[a-f0-9]{64}$",
  "example": "sha256:abc123..."
},
"qc_report": {
  "type": "object",
  "properties": {
```

```
"provider": {
  "type": "string",
  "example": "SGS Inspection"
},
"verdict": {
  "type": "string",
  "enum": ["PASS", "FAIL", "CONDITIONAL_PASS"],
  "example": "CONDITIONAL_PASS"
},
"overrides": {
  "type": "integer",
  "example": 1
},
"report_url": {
  "type": "string",
  "format": "uri",
  "example": "sgtx://documents/qc-001"
}
}
},
"insurance_certificate": {
  "type": "string",
  "example": "INS-2026-001"
}
},
"logistics": {
  "type": "object",
  "properties": {
    "trucking": {
      "type": "object",
      "properties": {
        "provider_gtid": {
          "type": "string",
          "example": "SGTX-EG-LSP-001"
```

```
,
"provider_name": {
  "type": "string",
  "example": "Fast Trucking Co."
},
"quote_id": {
  "type": "string",
  "example": "QT-001"
},
"fee": {
  "type": "number",
  "example": 300
},
"status": {
  "type": "string",
  "enum": ["PENDING", "PAID", "CANCELLED"],
  "example": "PAID"
}
}
},
"shipping_line": {
  "type": "object",
  "properties": {
    "provider_gtid": {
      "type": "string",
      "example": "SGTX-EG-SHIP-001"
    },
    "provider_name": {
      "type": "string",
      "example": "MSC Egypt"
    },
    "booking_ref": {
      "type": "string",
      "example": "MAEU876543210"
    }
  }
},
```

```
"vessel": {
  "type": "string",
  "example": "MSC FIONA"
},
"voyage": {
  "type": "string",
  "example": "MSF123E"
},
"etd": {
  "type": "string",
  "format": "date-time",
  "example": "2026-04-20T08:00:00Z"
},
"eta": {
  "type": "string",
  "format": "date-time",
  "example": "2026-05-05T14:00:00Z"
},
"actual_departure": {
  "type": "string",
  "format": "date-time",
  "nullable": true,
  "example": "2026-04-20T09:30:00Z"
},
"actual_arrival": {
  "type": "string",
  "format": "date-time",
  "nullable": true,
  "example": null
},
"freight_invoice": {
  "type": "object",
  "properties": {
    "amount": {
      "type": "number",
```



```
"example": 4200
},
"status": {
  "type": "string",
  "enum": ["CREDIT", "MANDATORY", "PAID"],
  "example": "CREDIT"
},
"due_date": {
  "type": "string",
  "format": "date",
  "example": "2026-06-05"
}
}
}
}
},
"customs_broker": {
  "type": "object",
  "properties": {
    "certification": {
      "type": "object",
      "properties": {
        "quote_id": {
          "type": "string",
          "example": "CBRQ-001"
        }
      }
    },
    "fee": {
      "type": "number",
      "example": 150
    }
  },
  "status": {
    "type": "string",
    "enum": ["PENDING", "PAID", "CANCELLED"],
    "example": "PAID"
  }
},
```

```
"certified_at": {
  "type": "string",
  "format": "date-time",
  "example": "2026-04-16T09:00:00Z"
}
},
"physical_handling": {
  "type": "object",
  "nullable": true
},
"storage": {
  "type": "object",
  "nullable": true
}
},
"warehousing": {
  "type": "object",
  "nullable": true,
  "properties": {
    "provider_gtid": {
      "type": "string"
    },
    "provider_name": {
      "type": "string"
    },
    "location": {
      "type": "string"
    },
    "fee": {
      "type": "number"
    },
    "status": {
      "type": "string",
```

```
"enum": ["PENDING", "PAID", "CANCELLED"]
}
}
}
},
"payment_plan": {
  "type": "object",
  "properties": {
    "stage1": {
      "type": "object",
      "properties": {
        "status": {
          "type": "string",
          "enum": ["PENDING", "SETTLED", "FAILED"],
          "example": "SETTLED"
        },
        "settled_at": {
          "type": "string",
          "format": "date-time",
          "example": "2026-04-15T13:00:00Z"
        },
        "transactions": {
          "type": "array",
          "items": {
            "type": "object",
            "properties": {
              "payee": {
                "type": "string",
                "example": "SGTX"
              },
              "amount": {
                "type": "number",
                "example": 1353.00
              }
            }
          }
        }
      }
    }
  }
}
```

```
"currency": {
  "type": "string",
  "example": "USD"
}
}
}
}
},
"stage2": {
  "type": "object",
  "properties": {
    "status": {
      "type": "string",
      "enum": ["PENDING", "SETTLED", "FAILED"],
      "example": "PENDING"
    },
    "transactions": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "payee": {
            "type": "string",
            "example": "MSC"
          },
          "amount": {
            "type": "number",
            "example": 4200.00
          },
          "currency": {
            "type": "string",
            "example": "USD"
          },
          "condition": {
```

```
"type": "string",
"enum": ["CREDIT", "MANDATORY"],
"example": "CREDIT"
},
"due_date": {
  "type": "string",
  "format": "date",
  "example": "2026-06-05"
}
}
}
}
}
},
"deferred": {
  "type": "object",
  "properties": {
    "status": {
      "type": "string",
      "enum": ["GUARANTEE_HELD", "RELEASED", "PAID", "EXPIRED"],
      "example": "GUARANTEE_HELD"
    },
    "guarantee_amount": {
      "type": "number",
      "example": 10000.00
    },
    "trigger_milestone": {
      "type": "string",
      "example": "CUSTOMS_IMPORT"
    },
    "expiry_date": {
      "type": "string",
      "format": "date-time",
      "example": "2026-05-15T23:59:59Z"
    }
  }
}
```

```
}  
,  
"financing": {  
  "type": "object",  
  "nullable": true,  
  "properties": {  
    "agreement_id": {  
      "type": "string",  
      "format": "uuid"  
    },  
    "amount": {  
      "type": "number"  
    },  
    "apr": {  
      "type": "number"  
    },  
    "status": {  
      "type": "string",  
      "enum": ["PENDING", "ACTIVE", "REPAID", "DEFAULTED"]  
    }  
  }  
},  
"takaful": {  
  "type": "object",  
  "nullable": true,  
  "properties": {  
    "contribution_amount": {  
      "type": "number"  
    },  
    "claim_id": {  
      "type": "string",  
      "format": "uuid"  
    },  
    "status": {  
      "type": "string",
```

```
"enum": ["ACTIVE", "CLAIM_PENDING", "CLAIM_PAID", "EXPIRED"]
}
}
}
},
"timeline": {
  "type": "array",
  "description": "Chronological list of all significant events",
  "items": {
    "type": "object",
    "required": ["timestamp", "event", "actor"],
    "properties": {
      "timestamp": {
        "type": "string",
        "format": "date-time",
        "example": "2026-04-15T09:00:00Z"
      },
      "event": {
        "type": "string",
        "example": "TRADE_INITIATED"
      },
      "actor": {
        "type": "string",
        "example": "buyer@importer.com"
      },
      "details": {
        "type": "object",
        "description": "Optional additional context",
        "example": {"quote_amount": 30450}
      },
      "governor_decision_id": {
        "type": "string",
        "format": "uuid"
      }
    }
  }
}
```

```
"loom_hash": {
  "type": "string",
  "pattern": "^sha256:[a-f0-9]{64}$"
}
},
"optional_services": {
  "type": "object",
  "properties": {
    "laboratory": {
      "type": "object",
      "properties": {
        "provider_gtid": {
          "type": "string",
          "example": "SGTX-EG-LAB-001"
        },
        "quote_id": {
          "type": "string",
          "example": "LABQ-001"
        },
        "fee": {
          "type": "number",
          "example": 200
        },
        "status": {
          "type": "string",
          "enum": ["PENDING", "PAID", "CANCELLED"],
          "example": "PAID"
        },
        "results": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "properties": {
```



```
"value": {
  "type": "number"
},
"limit": {
  "type": "number"
},
"pass": {
  "type": "boolean"
}
},
"example": {
  "cypermethrin": {"value": 0.02, "limit": 0.05, "pass": true}
}
}
},
"qc_inspection": {
  "type": "object",
  "properties": {
    "provider_gtid": {
      "type": "string",
      "example": "SGTX-EG-QC-002"
    },
    "quote_id": {
      "type": "string",
      "example": "QCQ-001"
    },
    "fee": {
      "type": "number",
      "example": 500
    },
    "status": {
      "type": "string",
      "enum": ["PENDING", "PAID", "CANCELLED"],
```

```
"example": "PAID"
},
"report": {
  "type": "object",
  "properties": {
    "verdict": {
      "type": "string",
      "enum": ["PASS", "FAIL", "CONDITIONAL_PASS"]
    },
    "defects": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "pallet_id": {"type": "string"},
          "defect": {"type": "string"},
          "severity": {"type": "string"},
          "ai_confidence": {"type": "integer"},
          "inspector_override": {"type": "boolean"},
          "override_reason": {"type": "string"}
        }
      }
    }
  }
},
"blockchain_anchor": {
  "type": "object",
  "properties": {
    "txid": {
      "type": "string",
      "example": "0xabc123..."
    }
  }
}
```

```
,
"merkle_root": {
  "type": "string",
  "example": "sha256:..."
},
"network": {
  "type": "string",
  "enum": ["polygon-mainnet", "ethereum-mainnet", "solana-mainnet"],
  "example": "polygon-mainnet"
},
"pqc_signature": {
  "type": "string",
  "example": "dilithium3:..."
},
"anchored_at": {
  "type": "string",
  "format": "date-time"
}
}
},
"links": {
  "type": "object",
  "properties": {
    "trade_command_center": {
      "type": "string",
      "format": "uri",
      "example": "https://sgtx.io/trade/SGTX-EG-26-F3A-1"
    },
    "export_pdf": {
      "type": "string",
      "format": "uri",
      "example": "https://sgtx.io/api/v1/shipment/SGTX-EG-26-F3A-1/pdf"
    },
    "verify": {
      "type": "string",
```

```
"format": "uri",
"example": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1"
},
"qr_code": {
  "type": "string",
  "format": "uri",
  "example": "sgtx://ustn/SGTX-EG-26-F3A-1"
}
},
"corridor_data": {
  "type": "object",
  "nullable": true,
  "properties": {
    "corridor_code": {
      "type": "string",
      "example": "EGY-ITA-RORO-001"
    },
    "corridor_name": {
      "type": "string",
      "example": "Egypt--Italy RoRo Corridor"
    },
    "transit_time_days": {
      "type": "integer",
      "example": 6
    },
    "eligibility_score": {
      "type": "integer",
      "minimum": 0,
      "maximum": 100,
      "example": 94
    }
  }
}
```

```
}
```

3.3.2 Example USTN Master Object (Populated)

Here is a fully populated example of a USTN master object for a real trade:

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "ustn_counter": 1,
  "trader_id": "F3A",
  "created_at": "2026-04-15T12:00:00Z",
  "updated_at": "2026-06-07T10:00:00Z",
  "status": "DELIVERED",
  "risk_assessment": {
    "platform_risk_score": 12,
    "gnn_risk": {
      "sanctions_proximity": 4,
      "graph_risk_score": 8,
      "explanation": "No indirect sanctions links detected within 2 hops."
    },
    "causal_analysis": null
  },
  "parties": {
    "exporter": {
      "gtid": "SGTX-EG-TRD-002139-7F3A",
      "trader_id": "F3A",
      "legal_name": "Strawberry Export Co.",
      "trust_score": 92,
      "jurisdiction": "EG"
    },
    "importer": {
      "gtid": "SGTX-DE-TRD-001234-5B6C",
      "trader_id": "5B6C",
      "legal_name": "European Importer GmbH",
      "trust_score": 88,
      "jurisdiction": "DE"
    }
  }
}
```

```
"additional_parties": [  
  {  
    "type": "LSP",  
    "gtid": "SGTX-EG-LSP-001",  
    "trader_id": "9D2E",  
    "legal_name": "Fast Trucking Co.",  
    "role": "trucking"  
  },  
  {  
    "type": "SHIP",  
    "gtid": "SGTX-EG-SHIP-001",  
    "trader_id": "4F1G",  
    "legal_name": "MSC Egypt",  
    "role": "shipping_line"  
  },  
  {  
    "type": "LAB",  
    "gtid": "SGTX-EG-LAB-001",  
    "trader_id": "2H7K",  
    "legal_name": "SGS Egypt",  
    "role": "laboratory"  
  },  
  {  
    "type": "CBR",  
    "gtid": "SGTX-EG-CBR-001",  
    "trader_id": "8M3T",  
    "legal_name": "Nile Customs Services",  
    "role": "customs_broker"  
  }  
],  
"goods": {  
  "hs_code": "0811.10.90",  
  "description": "Frozen strawberries, IQF, Grade A",  
  "net_weight_kg": 20000,
```

```
"gross_weight_kg": 20500,
"invoice_value": {
  "amount": 100000.00,
  "currency": "USD"
},
"incoterm": "CFR",
"container_count": 2,
"container_numbers": ["MEDU1234567", "MEDU1234568"],
"quantity_tolerance": 5,
"partial_shipment_allowed": true,
"transshipment_allowed": false
},
"documents": {
  "eta_invoice_uuid": "INV-abc123",
  "acid": "ACI20260415-0001",
  "nafeza_declaration_id": "SAD20260415-000147",
  "phyto_certificate": "PC-20260415-147",
  "health_certificate": "HC-20260415-147",
  "certificate_of_origin": "COO-20260415-147",
  "bill_of_lading": {
    "number": "MEDU1234567",
    "type": "eBL",
    "issuer": "MSC",
    "issued_at": "2026-04-16T10:00:00Z",
    "url": "sgtx://documents/eb1-123"
  },
  "packing_list_hash": "sha256:abc123...",
  "qc_report": {
    "provider": "SGS Inspection",
    "verdict": "CONDITIONAL_PASS",
    "overrides": 1,
    "report_url": "sgtx://documents/qc-001"
  },
  "insurance_certificate": "INS-2026-001"
},
```

```
"logistics": {
  "trucking": {
    "provider_gtid": "SGTX-EG-LSP-001",
    "provider_name": "Fast Trucking Co.",
    "quote_id": "QT-001",
    "fee": 300,
    "status": "PAID"
  },
  "shipping_line": {
    "provider_gtid": "SGTX-EG-SHIP-001",
    "provider_name": "MSC Egypt",
    "booking_ref": "MAEU876543210",
    "vessel": "MSC FIONA",
    "voyage": "MSF123E",
    "etd": "2026-04-20T08:00:00Z",
    "eta": "2026-05-05T14:00:00Z",
    "actual_departure": "2026-04-20T09:30:00Z",
    "actual_arrival": null,
    "freight_invoice": {
      "amount": 4200,
      "status": "CREDIT",
      "due_date": "2026-06-05"
    }
  },
  "customs_broker": {
    "certification": {
      "quote_id": "CBRQ-001",
      "fee": 150,
      "status": "PAID",
      "certified_at": "2026-04-16T09:00:00Z"
    },
    "physical_handling": null,
    "storage": null
  },
  "warehousing": null
}
```



```
,
"payment_plan": {
  "stage1": {
    "status": "SETTLED",
    "settled_at": "2026-04-15T13:00:00Z",
    "transactions": [
      {"payee": "SGTX", "amount": 1353.00, "currency": "USD"},
      {"payee": "Fast Trucking Co.", "amount": 300.00, "currency": "USD"},
      {"payee": "SGS Lab", "amount": 200.00, "currency": "USD"},
      {"payee": "Customs Broker (cert)", "amount": 150.00, "currency": "USD"},
      {"payee": "Egypt Customs", "amount": 200.00, "currency": "USD"}
    ]
  },
  "stage2": {
    "status": "PENDING",
    "transactions": [
      {"payee": "MSC", "amount": 4200.00, "currency": "USD", "condition": "CREDIT", "due_date": "2026-06-05"}
    ]
  },
  "deferred": {
    "status": "GUARANTEE_HELD",
    "guarantee_amount": 10000.00,
    "trigger_milestone": "CUSTOMS_IMPORT",
    "expiry_date": "2026-05-15T23:59:59Z"
  },
  "financing": null,
  "takaful": null
},
"timeline": [
  {"timestamp": "2026-04-15T09:00:00Z", "event": "TRADE_INITIATED", "actor": "buyer@importer.com"},
  {"timestamp": "2026-04-15T10:00:00Z", "event": "SELLER_ACCEPTED", "actor": "seller@exporter.com"},
  {"timestamp": "2026-04-15T10:30:00Z", "event": "QUOTE_SUBMITTED", "actor": "seller@exporter.com"},
  {"timestamp": "2026-04-15T11:00:00Z", "event": "CONTRACT_SIGNED", "actor": "both"},
  {"timestamp": "2026-04-15T12:00:00Z", "event": "USTN_GENERATED", "actor": "system", "details": {"ustn": "SGTX-EG-26-F3A-1"}},
  {"timestamp": "2026-04-15T13:00:00Z", "event": "STAGE1_PAID", "actor": "system"},

```

```
{
  "timestamp": "2026-04-15T13:05:00Z", "event": "CUSTOMS_DECLARATION_SUBMITTED", "actor": "broker"},
  {
    "timestamp": "2026-04-16T09:00:00Z", "event": "BROKER_CERTIFIED", "actor": "broker"},
    {
      "timestamp": "2026-04-16T10:00:00Z", "event": "BOOKING_CONFIRMED", "actor": "shipping_line"},
      {
        "timestamp": "2026-04-20T08:00:00Z", "event": "VESSEL_DEPARTED", "actor": "shipping_line"},
        {
          "timestamp": "2026-05-05T14:00:00Z", "event": "VESSEL_ARRIVED", "actor": "shipping_line"},
          {
            "timestamp": "2026-05-06T10:00:00Z", "event": "CUSTOMS_IMPORT", "actor": "buyer"},
            {
              "timestamp": "2026-05-06T14:00:00Z", "event": "DELIVERED", "actor": "buyer"},
              {
                "timestamp": "2026-05-07T10:00:00Z", "event": "SETTLED", "actor": "system"}
          ],
          "optional_services": {
            "laboratory": {
              "provider_gtid": "SGTX-EG-LAB-001",
              "quote_id": "LABQ-001",
              "fee": 200,
              "status": "PAID",
              "results": {
                "cypermethrin": {"value": 0.02, "limit": 0.05, "pass": true},
                "chlorpyrifos": {"value": 0.008, "limit": 0.01, "pass": true}
              }
            },
            "qc_inspection": {
              "provider_gtid": "SGTX-EG-QC-002",
              "quote_id": "QCQ-001",
              "fee": 500,
              "status": "PAID",
              "report": {
                "verdict": "CONDITIONAL_PASS",
                "defects": [
                  {
                    "pallet_id": "OR019", "defect": "mould", "severity": "medium", "ai_confidence": 92, "inspector_override": false},
                    {
                      "pallet_id": "OR020", "defect": "bruising", "severity": "low", "ai_confidence": 88, "inspector_override": true,
                      "override_reason": "Dark spot is surface bruise, no mould."}
                ]
              }
            }
          }
        }
      }
    }
  }
}
```

```

},
"blockchain_anchor": {
  "txid": "0xabc123...",
  "merkle_root": "sha256:...",
  "network": "polygon-mainnet",
  "pqc_signature": "dilithium3:...",
  "anchored_at": "2026-04-16T12:00:00Z"
},
"links": {
  "trade_command_center": "https://sgtx.io/trade/SGTX-EG-26-F3A-1",
  "export_pdf": "https://sgtx.io/api/v1/shipment/SGTX-EG-26-F3A-1/pdf",
  "verify": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1",
  "qr_code": "sgtx://ustn/SGTX-EG-26-F3A-1"
},
"corridor_data": {
  "corridor_code": "EGY-ITA-RORO-001",
  "corridor_name": "Egypt--Italy RoRo Corridor",
  "transit_time_days": 6,
  "eligibility_score": 94
}
}

```

3.3.3 Role-Based Filtering

The USTN master object is filtered based on the requester's role and permissions.

3.3.4 Database Storage

The USTN master object is stored in the shipments table. Key fields are indexed for performance:

sql

-- Shipments table (core USTN master object)

```

CREATE TABLE shipments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT UNIQUE NOT NULL,
  ustn_counter BIGINT,
  trader_id CHAR(3),
  contract_id UUID REFERENCES contracts(id),
  contract_shipment_id UUID REFERENCES contract_shipments(id),
  exporter_gtid TEXT REFERENCES tenants(gtid),

```

```

importer_gtid TEXT REFERENCES tenants(gtid),
status shipment_status DEFAULT 'INITIATED',
incoterm TEXT NOT NULL,
gnn_risk JSONB,
causal_analysis JSONB,
container_numbers TEXT[],
vessel_name TEXT,
voyage TEXT,
booking_ref TEXT,
etd TIMESTAMPTZ,
eta TIMESTAMPTZ,
actual_departure TIMESTAMPTZ,
actual_arrival TIMESTAMPTZ,
risk_score INTEGER,
carbon_footprint_kg_co2e DECIMAL(12,2),
release_status TEXT DEFAULT 'PENDING',
document_embeddings vector(384),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes for fast lookups

```

CREATE INDEX idx_shipments_ustn ON shipments(ustn);
CREATE INDEX idx_shipments_status ON shipments(status);
CREATE INDEX idx_shipments_exporter ON shipments(exporter_gtid);
CREATE INDEX idx_shipments_importer ON shipments(importer_gtid);
CREATE INDEX idx_shipments_etd ON shipments(etd);
CREATE INDEX idx_shipments_eta ON shipments(eta);

```

3.3.5 Implementation Checklist (Part 3.3)

PART 3.4 — USTN IN DOCUMENTS — MANDATORY INCLUSION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0 REVISED)

3.4.0 Purpose & Design Philosophy

Every document generated by or uploaded to SGTX must contain the USTN in a standardised location. This ensures that every physical and digital document can be traced back to the trade it belongs to, creating an unbreakable chain of custody from commercial contract to customs declaration to payment.

Key Principles:

Mandatory Inclusion — No document is accepted without a valid USTN.

Standardised Location — The USTN appears in a consistent position on every document type.

Machine-Readable — The USTN is embedded in both human-readable text and QR code format.

Verifiable — The USTN can be cryptographically verified using the SGTX public key.

3.4.1 Document Types and USTN Placement

3.4.2 AI Assistance for Document Generation

When a document is generated, the system automatically embeds the USTN and a QR code. No manual entry required.

Document Generation Flow:

System retrieves USTN master object from shipments table.

System generates document content from template.

System embeds USTN in header/footer.

System generates QR code with USTN payload.

System embeds QR code in document.

System signs document with Ed25519.

System stores document in documents table with USTN reference.

json

```
{
  "document_id": "DOC-2026-001",
  "ustn": "SGTX-EG-26-F3A-1",
  "document_type": "COMMERCIAL_INVOICE",
  "generated_at": "2026-04-15T12:00:00Z",
  "content_hash": "sha256:abc123...",
  "qr_code": "data:image/png;base64,...",
  "digital_signature": "ed25519:...",
  "loom_hash": "sha256:def456..."
}
```

3.4.3 Document Upload Validation

When a user uploads a document, the system validates that:

The document contains the USTN (in text or QR code).

The USTN matches the trade being referenced.

The document is not a duplicate.

Validation Logic:

rust

```
async fn validate_document_upload(
```

```
pool: &PgPool,
```

```
ustn: &str,
```



```

"certificate_number": "PHYTO-2026-001",
"issue_date": "2026-06-15",
"expiry_date": "2026-12-15",
"issuing_authority": "Egyptian Plant Quarantine",
"hs_code": "0805.10",
"quantity": "100 MT",
"treatment": "Cold Treatment -0.5°C for 14 days"
},
"discrepancies": [],
"fraud_indicators": [],
"verification_timestamp": "2026-06-20T10:30:00Z"
}

```

3.4.6 Document Types and Triggers

Each document type is associated with specific triggers (Shipment, Settlement, Customs, Financing):

3.4.7 Implementation Checklist (Part 3.4)

PART 3.5 — USTN RESOLUTION SERVICE

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0 REVISED)

3.5.0 Purpose & Design Philosophy

The USTN Resolution Service is the authoritative endpoint for retrieving USTN master objects. It provides role-based filtering, ensuring that each party sees only the data they are permitted to view. The service is fast, reliable, and auditable.

Key Principles:

Role-Based Filtering — Different parties see different subsets of data.

Fast Response — P95 latency ≤ 100 ms for cached responses.

Auditable — Every resolution request is logged in `audit_log`.

Rate-Limited — 100 requests per minute per tenant.

3.5.1 Endpoint Specification

Endpoint: `GET /v1/ustn/{ustn}`

Authentication: Valid JWT required.

Path Parameters:

Query Parameters (Optional):

Rate Limits:

3.5.2 Response Schema

Example Response (Buyer View):

json

```
{
```



```
"ustn": "SGTX-EG-26-F3A-1",
"status": "IN_TRANSIT",
"parties": {
  "exporter": {
    "gtid": "SGTX-EG-TRD-002139-7F3A",
    "trader_id": "F3A",
    "name": "Strawberry Export Co.",
    "trust_score": 92,
    "jurisdiction": "EG"
  },
  "importer": {
    "gtid": "SGTX-DE-TRD-001234-5B6C",
    "trader_id": "5B6C",
    "name": "European Importer GmbH",
    "trust_score": 88,
    "jurisdiction": "DE"
  }
},
"goods": {
  "description": "Frozen strawberries",
  "invoice_value": {
    "amount": 100000,
    "currency": "USD"
  }
},
"logistics": {
  "vessel": "MSC FIONA",
  "eta": "2026-05-05T14:00:00Z"
},
"payment_plan": {
  "stage1": "SETTLED",
  "stage2": "PENDING"
},
"timeline": [
  {"timestamp": "2026-04-15T09:00:00Z", "event": "TRADE_INITIATED"},
```

```
{
  "timestamp": "2026-04-15T10:00:00Z", "event": "SELLER_ACCEPTED"},
  {"timestamp": "2026-04-20T08:00:00Z", "event": "VESSEL_DEPARTED"}
],
"links": {
  "trade_command_center": "https://sgtx.io/trade/SGTX-EG-26-F3A-1",
  "verify": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1"
},
"resolved_at": "2026-06-07T10:05:00Z"
}
```

3.5.3 Error Responses

3.5.4 Implementation Checklist (Part 3.5)

PART 3.6 — USTN FOR MULTI-SHIPMENT CONTRACTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0 REVISED)

3.6.0 Purpose & Design Philosophy

In a multi-shipment contract, a master contract ID exists but no master USTN. Each shipment receives its own USTN when it locks. This allows each shipment to be tracked, financed, and executed independently.

Key Principles:

Per-Shipment USTN — Each shipment has its own USTN.

Independent Locking — Each shipment locks independently after per-shipment fee payment.

Independent Tracking — Each shipment has its own status, milestones, and documents.

Independent Execution — A delay in one shipment does not block others.

3.6.1 Multi-Shipment Structure

text

Master Contract: MC-20260415-001

■

■■■ Shipment 1: SGTX-EG-26-F3A-21 (15 Jul 2026)

■■■ Shipment 2: SGTX-EG-26-F3A-22 (15 Aug 2026)

■■■ Shipment 3: SGTX-EG-26-F3A-23 (15 Sep 2026)

Contract Shipments Table:

3.6.2 Per-Shipment Fee Collection

Each shipment has its own SGTX fee:

text

Shipment 1: $\$150,000 \times 1.5\% = \$2,250$ (PAID)

Shipment 2: $\$150,000 \times 1.5\% = \$2,250$ (PENDING)

Shipment 3: $\$150,000 \times 1.5\% = \$2,250$ (PENDING)

Fee Collection Workflow:

Buyer clicks "Confirm Shipment X".

Seller pays fee (one click).

FeeLock becomes ACTIVE for that shipment.

After the master contract is locked, either party can request modifications to future (unlocked) shipments. Already-locked shipments are immutable.

Delivery date

Port of discharge

Number of containers

Commodities (quantity, packaging)

Buyer navigates to TCC → Multi-shipment Schedule Card.

Buyer clicks "Request Modification" on a future shipment.

Buyer selects shipment and proposes changes.

Buyer adds mandatory reason (≥ 20 chars).

Buyer clicks "Send Modification Request".

Seller receives Smart Inbox item (priority 85).

Seller reviews diff view.

Seller clicks "Accept", "Reject", or "Counter".

If accepted, schedule addendum generated and signed.

Shipment schedule updated.

Master Contract View:

text

■ MULTI-SHIPMENT SCHEDULE — MC-20260415-001 ■

11

■ Master Contract: ACTIVE MASTER ■

■ Total Shipments: 3 ■ Completed: 1 ■ In Progress: 1 ■ Scheduled: 1 ■

II II


```

sgtx_fee_amount DECIMAL(20,4),
fee_lock_id UUID,
ustn TEXT UNIQUE,
status TEXT DEFAULT 'SCHEDULED', -- SCHEDULED, LOCKED, IN_EXECUTION, COMPLETED,
CANCELLED
commodities_overridden BOOLEAN DEFAULT false,
commodity_data JSONB,
locked_at TIMESTAMPTZ,
completed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

```
CREATE INDEX idx_contract_shipments_contract ON contract_shipments(contract_id);
```

```
CREATE INDEX idx_contract_shipments_ustn ON contract_shipments(ustn);
```

```
CREATE INDEX idx_contract_shipments_status ON contract_shipments(status);
```

3.6.6 Implementation Checklist (Part 3.6)

PART 3.7 — USTN FOR DISTRESSED MICRO-CONTRACTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0 REVISED)

3.7.0 Purpose & Design Philosophy

When a seller declares distress on a portion of a shipment (e.g., 3 pallets of lemons), SGTX generates a microUSTN for the distressed lot. The microUSTN follows the same format but links back to the original USTN via parent_ustn.

Key Principles:

MicroUSTN Generation — New USTN for distressed lot.

Parent Link — Links back to original USTN via parent_ustn.

Independent Lifecycle — Has its own status and milestones.

Separate Fee — SGTX fee applies to distressed sale.

3.7.1 MicroUSTN Structure

text

Original USTN: SGTX-EG-26-F3A-15

MicroUSTN: SGTX-EG-26-F3A-26

Parent Link: micro_contracts.parent_ustn = SGTX-EG-26-F3A-15

Components:

3.7.2 MicroUSTN Lifecycle

3.7.3 MicroUSTN Example

Distressed Declaration:

text

Seller declares distress on 3 pallets of lemons (384 kg)

Original USTN: SGTX-EG-26-F3A-15

MicroUSTN: SGTX-EG-26-F3A-26

Parent Link: micro_contracts.parent_ustn = SGTX-EG-26-F3A-15

Distressed Fee Calculation:

text

Original trade fee rate = 1.5%

Country factor (UAE) = 0.85

Distressed sale amount = \$900

Distressed Fee = $1.5\% \times 0.85 \times \$900 = \$11.48$

3.7.4 MicroUSTN Database Schema

sql

-- Micro-contracts (distressed)

```
CREATE TABLE micro_contracts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  parent_contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,  
  distressed_listing_id UUID REFERENCES distressed_cargo_listings(id) ON DELETE SET NULL,  
  micro_ustn TEXT UNIQUE NOT NULL,  
  parent_ustn TEXT REFERENCES shipments(ustn),  
  buyer_gtid TEXT NOT NULL,  
  seller_gtid TEXT NOT NULL,  
  agreed_price DECIMAL(20,4),  
  sgtx_fee_rate DECIMAL(5,4),  
  sgtx_fee_amount DECIMAL(20,4),  
  fee_lock_id UUID,  
  status TEXT DEFAULT 'PENDING_SIGNATURES',  
  locked_at TIMESTAMPTZ,  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_micro_contracts_ustn ON micro_contracts(micro_ustn);  
CREATE INDEX idx_micro_contracts_parent ON micro_contracts(parent_ustn);
```

3.7.5 Implementation Checklist (Part 3.7)

PART 3.8 — USTN IN API CALLS — MANDATORY REFERENCE

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0 REVISED)

3.8.0 Purpose & Design Philosophy

All SGTX API calls that create or modify trade-related entities must include the USTN (or microUSTN) in the request body or as a query parameter. This ensures that every action is traceable to a specific shipment.

Key Principles:

Mandatory Reference — Every API call must include USTN.

Validation — USTN is validated for format and existence.

Auditable — Every API call is logged with USTN reference.

3.8.1 API Endpoints with USTN

3.8.2 USTN Validation in API Calls

Validation Logic:

rust

```
async fn validate_ustn_in_request(
    pool: &PgPool,
    ustn: &str,
    action: &str,
) -> Result<(), String> {
    // 1. Validate USTN format
    if !is_valid_ustn_format(ustn) {
        return Err("Invalid USTN format. Expected SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}".into());
    }
    // 2. Check if USTN exists
    let exists = sqlx::query!("SELECT 1 FROM shipments WHERE ustn = $1", ustn)
        .fetch_optional(pool)
        .await?
        .is_some();
    if !exists {
        return Err(format!("USTN '{}' not found in database", ustn));
    }
    // 3. Check if action is allowed for current status
    // (Governor handles this)
    Ok(())
}
```

Error Response (Missing USTN):

text

HTTP/1.1 400 Bad Request

Content-Type: application/json

```
{  
  "error": "Missing USTN parameter",  
  "message": "Missing USTN parameter. Every trade action must reference its shipment identifier.",  
  "help": "Please add 'ustn' field to your request body."  
}
```

3.8.3 Implementation Checklist (Part 3.8)

PART 3.9 — USTN QR CODE & PHYSICAL TAGGING

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0 REVISED)

3.9.0 Purpose & Design Philosophy

Every USTN is encoded as a QR code that can be printed on shipping labels, packing lists, and customs documents. This enables physical tracking, offline verification, and automated scanning.

Key Principles:

Universal Encoding — Every USTN has a QR code.

Offline Verification — Can be verified without internet access.

Cryptographic Signature — QR code includes verifiable credential.

Automated Scanning — Mobile app scans and opens TCC.

3.9.1 QR Code Payload

The QR code encodes a JSON payload with the USTN and verification information.

QR Code Payload:

```
json  
{  
  "ustn": "SGTX-EG-26-F3A-1",  
  "verify_url": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1",  
  "verifiable_credential": {  
    "type": "VerifiableCredential",  
    "issuer": "https://sgtx.io/issuers/platform",  
    "issuanceDate": "2026-04-15T12:00:00Z",  
    "credentialSubject": {  
      "ustn": "SGTX-EG-26-F3A-1",  
      "exporter_gtid": "SGTX-EG-TRD-002139-7F3A",  
      "importer_gtid": "SGTX-DE-TRD-001234-5B6C",  
      "product": "Frozen strawberries",  
      "hs_code": "0811.10.90"  
    },  
  },  
}
```



```
"proof": {
  "type": "Ed25519Signature2020",
  "created": "2026-04-15T12:00:00Z",
  "proofPurpose": "assertionMethod",
  "verificationMethod": "https://sgtx.io/keys/ed25519",
  "signature": "base58..."
}
```

3.9.2 QR Code Visual Design

text



3.9.3 AI Assistance for QR Scanning

The SGTX mobile app scans the QR code and automatically opens the Trade Command Center (or the public verification page).

Scanning Flow:

User opens SGTX mobile app.

■■■ Tax Registration Certificate (Uploaded: 2026-06-15) ■ Verifying... ■■

■ ■ ■ Export License (Not uploaded) [Upload Now] ■ ■

■ ■ ■ Sanctions Self-Declaration (Not signed) [Sign Now] ■ ■ ■

■ BIOMETRIC VERIFICATION (UBOs) ■

■ Sara Ali: [Verify with Passkey] ■ Pending ■

■ ■

■ Estimated response: 48 hours ■

■ Queue position: 3 ■



Description: Set trader mode, preferred language, and consent for optional features.

Trader-specific (TRD only):

Default incoterm (used to pre-fill trade requests).

Preferred currency (for displaying prices; actual trade currency can differ).

Preferred language (English, Arabic, German, Vietnamese, etc.) — used for all AI-generated messages, Smart Inbox, and UI.

Voice stress flag (off by default, on-device only)

Offline mobile sync (for LSP/QC mobile apps)

Anonymous market intelligence panel (sharing aggregated data)

Notification preferences (quiet hours, digest settings) — can be changed later in Company Admin.

One-click action: Click "Save Preferences" → proceeds to Step 5.

UI Example:

text

■ PROFILE CONFIGURATION — Step 4 of 6 ■

■ ■

■ TRADER MODE (TRD only) ■

■ [BUY ●] [SELL ■] [DUAL ■] ■

■ ■

■ Default Incoterm: [CFR ▼] ■

■ Preferred Currency: [USD ▼] ■

■ Preferred Language: [English ▼] ■

■ ■

■ CONSENT FOR OPTIONAL FEATURES ■

■ ■ Voice stress flag (on-device only) ■

■ ■ Offline mobile sync ■

■ ■ Anonymous market intelligence panel ■

■ ■

■ NOTIFICATION PREFERENCES ■

■ Quiet Hours: [22:00] to [07:00] Timezone: [UTC+2] ■

■ Digest Settings: [Daily at 07:00 UTC] ■

■ ■

■ [Save Preferences] [Skip to Step 5] ■

3.10.1.5 Step 5 — Create First Resource (Optional)

Description: Pre-configure common data to accelerate future trade creation. Entirely optional; user can skip.

Resource types per tenant:

AI assistance (A1, Groq):

System suggests default fee ranges based on anonymised platform aggregates (e.g., "Typical trucking fee for this corridor: \$0.75–\$0.95/km").

User can accept, modify, or skip.

No suggestions based on "what others charge" — only anonymised, differentially private statistics.

One-click actions:

"Save Resource" — adds the resource to the tenant's catalogue.

"Skip" — proceeds to Step 6 without saving any resource.

UI Example:

text

CREATE FIRST RESOURCE — Step 5 of 6 (Optional)

You can pre-configure common data to accelerate future trade creation.

SAVED COMMODITIES

HS Code: 0805.10 | Product: Fresh Oranges | Packaging: Cartons 10kg

[Add Commodity]

SAVED PORTS

Alexandria (EG) | Damietta (EG) | Port Said (EG)

[Add Port]

PREFERRED LOGISTICS PROVIDERS

Fast Trucking Co. (LSP) | SGS Egypt (LAB) | Nile Customs (CBR)

[Add Provider]

AI Suggestion: Typical trucking fee for Alexandria → Damietta: \$0.85/km

Would you like to add this? [Add] [Ignore]



■ [Save Resource] [Skip] ■



3.10.1.6 Step 6 — Enter Sandbox

Description: Provide a fully functional, isolated replica of the platform for practice and education.

Sandbox characteristics:

Separate PostgreSQL schema (sandbox_*), NATS JetStream stream, and ClickHouse database.

Synthetic counterparties pre-provisioned with names like "Demo Buyer Co.", "Demo Seller Ltd.", "Demo Logistics Provider" — clearly marked DEMO.

No real money, real documents, or real API calls to government systems (mocked responses).

Resets automatically every week (Sunday 03:00 UTC). User can also click "Reset Sandbox" to wipe and restart.

Guided practice trade:

A step-by-step walkthrough (tooltips and modal overlays) that guides the user through:

Creating a trade request (structured form)

Seller accepting and locking EXW price

Designing packing

Adding logistics (using mock RFQ responses)

Submitting quote, signing contract, paying mock fee

Tracking milestones, confirming delivery, settling payment

Each step includes an educational explanation — never suggests alternative counterparties.

At the end, the user sees a summary of the complete trade cycle.

Sandbox UI elements:

Persistent banner: "You are in Sandbox mode. No real transactions will occur."

"Exit Sandbox" button (available only after completing the guided tour or after 5 minutes).

One-click actions:

"Start Sandbox" — creates sandbox environment and loads guided tour.

"Reset Sandbox" — wipes data and restarts.

"Exit Sandbox" — returns to main portal (sandbox data persists until reset).

UI Example:

text



■ ENTER SANDBOX — Step 6 of 6 ■



json

```
{
  "gtid": "SGTX-EG-TRD-002139-7F3A",
  "legal_name": "Strawberry Export Co.",
  "type": "TRD",
  "jurisdiction": "EG",
  "kyb_tier": 2,
  "kyb_status": "VERIFIED",
  "sanctions_cleared": true,
  "trust_score": 92,
  "lifecycle_state": "VERIFIED",
  "is_saved_contact": true
}
```

3.10.3.2 Governor Service — Single Point of Truth

The Governor is a Rust Axum service that intercepts every API request that modifies state. It evaluates the request against a three-stage pipeline:

OPA (Rego) — business rules, permissions, data scopes, dual-mode context.

WasmEdge constitutional modules — jurisdiction supremacy, fee bounds, incoterm validation, A5 prohibition, reserve composition.

AI consult (A1–A3) — where configured, the Governor calls the AI Orchestrator (Groq → Ollama for A1; HF local → Ollama for A2/A3) for advisory recommendations or constraint flags.

Important: AI never executes. It only returns a structured DecisionSuggestion that the Governor merges with OPA and WasmEdge verdicts. The final decision always originates from human intent(the user's action) and immutable constitutional rules.

Architecture Diagram (Textual):

text

User Action (click/voice)

↓

API Gateway (Nginx + WasmEdge filters)

↓

Governor Proxy (Axum intercept)

↓

↓ ↓ ↓

OPA WasmEdge AI Router (Rig)

(Rego) (WASM) ■

A horizontal row of 12 squares representing lattice sites. The first square contains a downward-pointing arrow. The second square also contains a downward-pointing arrow. The remaining ten squares are empty.


```
"action_url": "/v1/kyc/upgrade?gtid=SGTX-VN-TRD-000456"
```

```
}
```

```
],
```

```
"escalation_hint": "If you believe this is an error, you can request a human review. A compliance officer will respond within 24 hours."
```

```
},
```

```
"loom_hash": "sha256:d4e5f6..."
```

```
}
```

3.10.4 Common Shared Components

3.10.5 Tenant Lifecycle Engine

State Machine: REGISTERED → ONBOARDING → KYB_PENDING → VERIFIED → LIMITED_MODE → AT_RISK → SUSPENDED → ARCHIVED

Transitions: Governor decision required for KYB_PENDING → VERIFIED, VERIFIED → AT_RISK, SUSPENDED → VERIFIED. Smart Inbox creates high-priority items on state changes with Groq-generated explanations.

Database:

sql

```
CREATE TABLE tenant_lifecycle_history (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  from_state TEXT NOT NULL,  
  to_state TEXT NOT NULL,  
  reason TEXT,  
  governor_decision_id UUID,  
  changed_by UUID REFERENCES employees(id),  
  changed_at TIMESTAMPTZ DEFAULT NOW()  
);
```

3.10.6 Network Feature (Saved Contacts)

The Network feature (Saved Contacts) is a strictly opt-in, user-controlled directory of existing counterparties. SGTX never auto-discovers or suggests new contacts. Contacts are saved automatically when:

A trade request is created between two tenants.

A logistics provider accepts a quote.

A financing agreement is signed.

A direct message is exchanged in the Trade Room.

A distressed cargo notification is sent.

Manual addition: Users can also manually add a GTID (if they know it) via the "Add Contact" button. The system resolves the GTID and shows public information (legal name, jurisdiction, trust score) — but

does not recommend "people you may know".

AI Enrichment (Non-Marketplace):

Important: The Network tab never displays a list of "top rated" or "recommended" contacts. Users must already have a relationship or explicitly know the GTID.

Database:

sql

```
CREATE TABLE tenant_contacts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  contact_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  relationship_type TEXT,  
  trade_count INTEGER DEFAULT 0,  
  total_value DECIMAL(20,4) DEFAULT 0,  
  first_interaction TIMESTAMPTZ,  
  last_interaction TIMESTAMPTZ,  
  is_favorite BOOLEAN DEFAULT false,  
  is_blocked BOOLEAN DEFAULT false,  
  trust_snapshot JSONB,  
  relationship_health_score DECIMAL(5,2),  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(tenant_id, contact_gtid)  
);
```

3.10.7 Sandbox Environment

The sandbox is a complete replica of the production platform, but:

Data is stored in separate PostgreSQL schema (sandbox_*).

Synthetic counterparties are pre-provisioned with names like "Demo Buyer Co." and "Demo Seller Ltd." — clearly marked as demo.

No real money or real documents are ever used.

The sandbox resets every week (or manually via "Reset Sandbox" button).

A guided practice trade walks the user through every step (creating a request, locking price, adding logistics, paying fee, tracking milestones). Tooltips explain each action without suggesting alternative counterparties.

Exit to production: When the user clicks "Go Live", the sandbox data is wiped (after confirmation), and the tenant's lifecycle_state becomes VERIFIED (or KYB_PENDING). No data leaks to production.

3.10.8 Trade Readiness Assessment

3.10.8.1 Purpose

Ensure that every tenant (buyer, seller, logistics provider, financier, etc.) is fully prepared to execute their first trade or service on the SGTx platform. The Trade Readiness Assessment evaluates company configuration, banking setup, trade preferences, security posture, and legal acknowledgements before allowing trade creation.

3.10.8.2 Readiness Categories & Checklist

3.10.8.3 Readiness Score Calculation

Formula:

text

Readiness Score = (Completed Mandatory Items / Total Mandatory Items) × 100

Score Thresholds & Permissions:

3.10.8.4 UI — Readiness Card

text

TRADE READINESS SCORECARD Score: 87%

Status: Mostly Ready

Company: Tax ID, Registration, Address, UBO

Banking: Settlement account linked -- [Connect Now]

Trade: 3 products, 5 ports, incoterm CFR

Security: Passkey enrolled, MFA enabled

Legal: Fee schedule acknowledgment pending -- [Review & Sign]

Complete 2 remaining mandatory items to reach 100% readiness.

[Go to Company Admin] [Dismiss for 7 days]

3.10.9 Role Journey Maps

3.10.9.1 Trader Journey (Buyer — Importer)

3.10.9.2 Trader Journey (Seller — Exporter)

3.10.10 SGTx Trade Trust Passport™

3.10.10.1 Purpose

Provide a portable, participant-controlled institutional trust profile that aggregates a tenant's reliability, compliance, financing performance, dispute resolution, and verified identifiers. The Trust Passport is a verifiable credential (W3C standard) signed by SGTx, enabling participants to share their

trustworthiness with counterparties, financiers, logistics providers, and regulators without exposing raw trade data.

3.10.10.2 Trust Passport Contents

3.10.10.3 Sharing Workflow (One-Click)

Step 1 — Generate Passport:

Tenant admin navigates to Company Admin → Trust Passport and clicks "Generate Passport" (one click).

Step 2 — Share with Counterparty:

Admin clicks "Share Passport" → selects the recipient's GTID (from saved contacts) or chooses "Anyone with link" (anonymous token).

Step 3 — Recipient Verification:

Recipient opens the link (no login required). Page displays passport data (only consented dimensions). Recipient can download signed JSON or copy a verification code to check offline.

Step 4 — Revoke Access:

At any time, admin clicks "Revoke" next to the shared token. Revocation is immediate.

One-click guarantee: Generate Passport (1), Share (1), Revoke (1).

3.10.11 Phase 0 One-Click Summary

3.10.12 Implementation Checklist (Phase 0)

3.10.12.1 Tenant Onboarding

Build 6-step wizard UI with progress indicator

Implement GTID generation algorithm (Rust)

Integrate A2 (HF Donut) for document extraction in Steps 2 and 3

Add registry cross-reference service (RIA-driven) for commercial register, tax ID, etc.

Implement biometric liveness (ZITADEL WebAuthn) for UBOs

Build Verified Trade Profile section with LEI, DUNS, customs, chamber, VAT verification

3.10.12.2 Sandbox

Create sandbox environment with schema isolation and weekly reset cron job

Develop guided practice trade (tooltips, synthetic data, mock APIs)

Add "Go Live" transition logic and state management

3.10.12.3 Governance & Identity

Implement Governor service (Rust + Axum) with OPA evaluator

Write Rego policies: permissions.rego, fee.rego, financing.rego, distressed.rego, multiship.rego, logistics.rego, broker.rego, reserve.rego

Implement WasmEdge constitutional modules

Implement Loom hash chain library (Rust)

Add hourly audit-chain-verifier cron job

Build public Loom verification endpoint (/v1/verify/loom)

Implement Ed25519 signing for all Governor decisions

3.10.12.4 Common Components

Implement inbox-service (Rust, Axum) with urgency scoring (Rego, A4)

Build Smart Inbox UI with Recommended Actions Widget

Build Trade Command Center (TCC) with timeline, pending action, summary cards

Build PlainLanguage Governor Decision Panel UI component (React)

Build Shared Shipments Vault with virtual scrolling, filters, map view

Integrate VoiceCommandButton (Vosk + HF Mixtral)

Build Customer Care Chatbot (Groq + HF Mixtral)

Build Dual-Mode Toggle (Trader Portal only)

3.10.12.5 Tenant Lifecycle

Implement state machine (REGISTERED → ONBOARDING → KYB_PENDING → VERIFIED → ...)

Add state transition logging (tenant_lifecycle_history)

Create Smart Inbox notifications on state changes (A1 Groq explanations)

3.10.12.6 Network Feature

Create tenant_contacts table

Implement automatic contact saving on trade, quote, message, etc.

Build AI-enriched Trust Portrait (A1 Groq)

Implement Relationship Health Score (LSTM, A2)

Add GraphRAG indirect connections

3.10.12.7 Trade Readiness Assessment

Create readiness_checklist table

Implement validation APIs for each checklist item (tax ID, PSP account, etc.)

Build Readiness Card UI component with real-time updates

Integrate Groq (A1) for plain-language recommendations

Add Governor policy trade.create.readiness that checks score $\geq 70\%$

3.10.12.8 Trust Passport

Create trust_passports, trust_passport_shares, trust_passport_revocations tables

Implement credential generation service (Rust, json-ld, ed25519-dalek)

Build API endpoints for generation, sharing, verification, revocation

3.10.12.9 Role Journey Maps

Create role_journey_completion table

Implement journey step tracking for each role

Add journey completion reporting (Company Admin)

3.10.13 AI Authority Summary for Phase 0

END OF PART 3.10 — PHASE 0: FOUNDATION — IDENTITY, GOVERNANCE & SHARED PORTALS

■ Credit Period: [30 Days ▼] Currency: [USD ▼] ■

SGTX-VN-TRD-002139-7F3A Mekong Fresh Co. Trust: 92 Sanctions:

Last trade: 15 May 2026 Vietnam 12 trades Relationship: Supplier

SGTX-EG-TRD-001234-5B6C Nile Foods Trust: 88 Sanctions:

Last trade: 10 Jun 2026 Egypt 8 trades Relationship: Customer

[Enter GTID manually] [Cancel]

3.11.4.2 Step 2: Incoterm Selection

Description: The buyer selects the governing international commercial term.

Component: Dropdown (Incoterms 2020). Auto-configures seller's mandatory logistics.

AI Assistance:

A1 (Groq): Generates plain-language explanation of responsibilities.

A4 (WasmEdge): Validates incoterm against jurisdiction and legality.

One-Click Action: Selecting an incoterm from the dropdown is a single click.

UI Example:

text

Select Incoterm

[CFR ▼]

CFR — Cost and Freight

Seller is responsible for:

Export customs clearance

Trucking to port of loading

Ocean freight to destination port

☐ [Advance Payment ☐] [Partial Advance ☐] [Against Documents ☒] ☐

■ [Against Shipment ■] [Against Delivery ■] [Deferred Payment ■] ■

■ [To Be Negotiated ■] ■

■ PREFERRED CREDIT PERIOD ■

■ [30 Days ▼] ■

■ ■

■ CURRENCY ■

■ [USD ▼] ■

■ ■

■ FINANCING INTEREST ■

■ [Buyer ■] [Seller ■] [Either Party ■] [None ●] ■

■ ■

■ BANK INSTRUMENT PREFERENCE (Optional) ■

■ [None ▼] ■

11

■ SETTLEMENT FLEXIBILITY ■

■ [Fixed ■] [Flexible ●] [Open To Alternatives ■] ■

□ □

■ ■ ■ Example: Buyer prefers LC but would accept Documentary Collection ■

■ if terms are improved. ■

11

■ DOCUMENTARY REQUIREMENTS (Settlement) ■

■ ■ Commercial Invoice ■ Packing List ■ Bill of Lading ■ Certificate of Origin ■

■ ■ Inspection Certificate ■ Insurance Certificate ■

11

■ AI SETTLEMENT READINESS (Generated Automatically) ■

■ Settlement Readiness: 87/100 ■

■ Potential Issue: ■ Inspection Certificate missing from settlement documents ■

■ Recommendation: Add Inspection Certificate to avoid delays. ■

3.11.4.9 Step 9: Executive Approval Layer

Description: Automatic approval workflow based on trade value.

Configuration (Tenant Admin):

Internal Approvals (Checkboxes for Pre-Approval):

- Procurement Manager
- Finance Manager
- Compliance Officer
- Legal Department
- CEO

Governance: If the required approval level is not met, Governor returns CONDITIONAL with Decision Panel.

UI Example:

text



■ EXECUTIVE APPROVAL LAYER ■



■ ■

■ Trade Value: \$450,000 ■

■ Required Approval Level: Director ■

■ ■

■ Approval Workflow: ■

■ ■ Procurement Manager: Ahmed Hassan (Approved: 2026-06-20 09:00) ■

■ ■ Finance Manager: Sara Ali (Approved: 2026-06-20 09:30) ■

■ ■ Director: Mohamed Eltonsy (Pending) ■

■ ■ Executive Committee: Not required ■

■ ■

■ Internal Approvals (Pre-Approval): ■

■ ■ Procurement Manager ■

■ ■ Finance Manager ■

■ ■ Compliance Officer ■

■ ■ Legal Department ■

■ ■ CEO ■

■ ■

■ [Submit for Approval] [Save Draft] ■



3.11.4.10 Step 10: AI Modules (Advisory)

Components:

Trade Request Readiness Score (A2, XGBoost)

Supplier Attractiveness Score (A2, LightGBM)

AI Assistance:

A2 (XGBoost): Calculates readiness score (0-100).

A2 (LightGBM): Calculates supplier attractiveness score (0-100%).

A1 (Groq): Generates plain-language explanations and recommendations.

One-Click Actions:

Click "Complete Missing Items" → navigates to the missing section.

Click "Submit Anyway?" (if score ≥80) — advisory only.

UI Example:

text

AI MODULES (Advisory)

TRADE REQUEST READINESS: 96/100

[]

96% ready

Complete Items:

Seller selected

Incoterm selected

Containers configured

Commodities defined

Acceptance Criteria Matrix complete

Documentation complete

Missing / Incomplete Items:

Insurance requirement not specified

[Complete Missing Items] [Submit Anyway?]

USTN not yet generated (will be at contract lock).

If CONDITIONAL or DENY:

Governor generates plain-language tenant_message (A1, Groq → Ollama).

Decision Panel displays with condition checklist and direct action links.

Buyer cannot proceed until conditions resolved.

Example CONDITIONAL Response (Missing Insurance):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your trade request is on hold because Insurance Requirement has not been specified. Please select whether insurance is Required, Optional, or Not Required.",
    "conditions": [
      {
        "condition_id": "insurance_required",
        "label": "Specify Insurance Requirement",
        "action_url": "/v1/trade/request/draft?trade_id=TR-2026-001&section=insurance"
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  }
}
```

Example CONDITIONAL Response (Executive Approval Pending):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your trade request requires Director approval. Please submit the request for approval before proceeding.",
    "conditions": [
      {
        "condition_id": "executive_approval",
        "label": "Submit for Director approval",
        "action_url": "/v1/approval/submit?trade_id=TR-2026-001"
      }
    ],
  }
}
```


Build Seller Selection component (GTID autocomplete, saved contacts)
Build Incoterm Selection component (with auto-configuration)
Build Container Tab component (drag-and-drop, clone, bulk edit)
Build Commodity Entry component (with Product Form Agent integration)
Build Acceptance Criteria Matrix component (dynamic per product)
Build Weight Summary component (real-time)
Build Documentation Requirements component (checkbox matrix + timing)
Build Special Trade Instructions component (with AI suggestions)
Build Transport & Logistics component (mode, equipment, delivery window)
Build Insurance Requirements component
Build Commercial Settlement component
Build Executive Approval component
Build AI Modules component (Readiness Score, Attractiveness Score)
Build Multi-Shipment Schedule component

3.11.8.2 AI Integration

Integrate Product Form Agent (A2) for dynamic schema generation
Integrate AI Container Advisor (A1, Groq)
Integrate Trade Request Readiness Score (A2, XGBoost)
Integrate Supplier Attractiveness Score (A2, LightGBM)
Integrate Special Instructions AI suggestions (A1, Groq)
Integrate Insurance recommendations (A1, Groq)
Integrate Settlement readiness recommendations (A1, Groq)

3.11.8.3 Governor Integration

Add all 33 validation gates (G1U1-G1U33)
Implement Decision Panel for CONDITIONAL/DENY
Implement tenant message generation (A1, Groq)
Implement executive approval validation
Implement readiness score validation
Implement draft validation

3.11.8.4 Draft & Recovery

Implement draft auto-save (30-second debounce)
Implement draft recovery in Smart Inbox
Implement draft expiry (14 days)
Implement draft reminders (11 and 13 days)
Implement draft restoration (Company Admin)

3.11.8.5 Testing

Unit tests: seller selection validation

Unit tests: incoterm auto-configuration

Unit tests: commodity entry with Product Form Agent

Unit tests: weight calculation

Unit tests: Governor validation gates

Integration tests: full trade request creation flow

Integration tests: conditional submission → fix → submit

Integration tests: draft auto-save → recovery → submit

End-to-end tests: buyer workflow (login → create → submit)

3.11.9 AI Authority Summary for Phase 1

END OF PART 3.11 — PHASE 1: TRADE INITIATION (BUYER DASHBOARD)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 3.12 — PHASE 2: SELLER QUOTE, PACKING & LOGISTICS ORCHESTRATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

3.12.0 Purpose & Design Philosophy

Phase 2: Seller Quote, Packing & Logistics Orchestration is the seller's operational heart of the trade. After receiving the buyer's structured trade request (Phase 1), the seller must transform that commercial intent into a physically feasible, commercially viable, and legally compliant quote. This phase encompasses:

Loading Origin — Where the goods will be loaded

EXW Price Lock — Pricing the goods with AI market intelligence

Packing & Containerisation — Designing the physical packing plan with non-uniform layer stacking

Logistics Orchestration — Determining logistics costs through three flexible modes (Manual, RFQ to LSPs, Direct to SHIP)

Alternative Delivery Ports — Providing options to the buyer

Multi-Shipment Response — Accepting, modifying, or rejecting multi-shipment schedules

Quote Submission — Assembling the final price and submitting to the buyer

Goal: Seller produces a complete, competitive, and compliant quote that includes:

EXW price (with AI market intelligence)

Packing plan (with non-uniform layers, SSCC barcodes, acceptance criteria references)

Logistics costs (selected from three flexible modes)

Alternative delivery options

SGTX platform fee (1.5% added to total)

Full multi-shipment schedule (if applicable)

All documents generated (packing list, invoice, customs declaration)

Key Principles:

Zero re-entry — All data from the buyer's request is pre-filled; no re-entry.

AI-assisted optimisation — AI suggests pricing, packing, and logistics optimisation.

Three flexible logistics modes — Manual (Mode A), RFQ to LSPs (Mode B), Direct to SHIP (Mode C).

Non-uniform layer stacking — Supports real-world packing with mixed layer configurations.

Transport mode-aware — Equipment selection drives logistics costs and packing.

Incoterm-driven — Mandatory logistics services are enforced based on incoterm.

One-click core — Every irreversible action is a single click.

Non-marketplace — Never suggests alternative buyers or service providers.

Phase 2 Timeline: Days 5–8 (seller prepares and submits quote)

One-click guarantee: From opening the request to submitting the quote, ~12-15 clicks (excluding data entry). Voice commands available for packing and logistics selections.

AI Integration in Phase 2:

3.12.1 Phase 2 Overview

3.12.1.1 High-Level Workflow

text



- ■ ■ → Seller enters fixed costs per service (trucking, customs, THC, freight) ■ ■ ■
- ■ ■ → Incoterm-based filtering (mandatory services enforced) ■ ■ ■

- → Select service categories (trucking, warehousing, customs brokerage) ■■■
- → Directed (specific LSPs) or Anonymous broadcast ■■■

■ ■ ■ → Quote submission and comparison panel ■ ■ ■

■ ■ ■ → Select target shipping lines ■ ■ ■

■ ■ ■ → Add optional ancillary services (trucking, customs broker, insurance) ■ ■ ■

■ ■ ■ → Quote submission and comparison panel ■ ■ ■

■ ■ ■

13

■ ■ → Add alternative ports within destination country ■ ■

■ ■ → AI suggests ports (A1, Groq) based on cost, transit, congestion ■ ■

■ ■ → Comparison table shown to buyer ■ ■

111

11

11

■ Credit: 30 DAYS | Currency: USD | Flexibility: FLEXIBLE ■

■ [Accept Request] [Decline] [Propose Modifications] [Ask Clarification] ■

Description: Seller specifies where the goods will be loaded.

Governance: Governor validates that loading port is not sanctioned and compatible with incoterm. If incoterm is FOB, loading port must be in seller's country.

text

Country of Loading: [Egypt ▼]

■ Port of Loading: [Damietta ▼] ■

Alternative Loading Point: [N/A] [Add Location]

■ Distance to Port: 150 km (estimated) ■

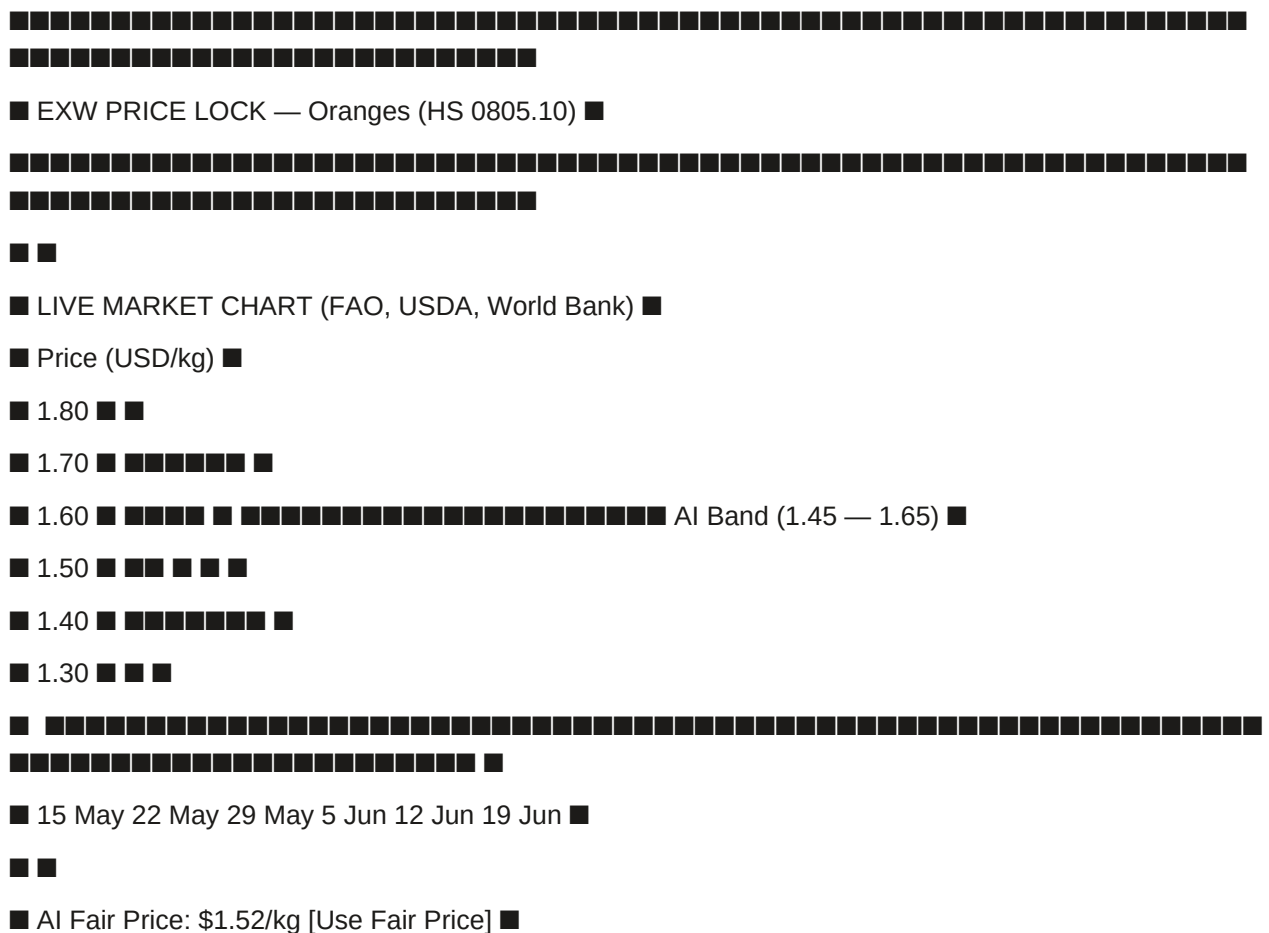
■ ■ ■ Loading port must be in seller's country for FOB incoterm. ■

■ Damietta is approved for reefer containers. ■

■ [Save] [Skip to EXW Price] ■

Description: Lock the EXW price per commodity with AI market intelligence.

Components:



22

■ ■

□ □

■ ■

□ □

■ ■ Stacking orientation: [Standard ▼] ■ ■

■ ■

Destination: Germany (EU)

Regulations: Packaging Act — Recycled content ≥30% for plastic

Special Instructions: No wooden pallets. Halal certification required.

CURRENT MATERIALS:

Commodity Material Carbon (kg CO₂e) Cost (USD) Status

Oranges Virgin Plastic Strap 2,200 120.00

RECOMMENDED ALTERNATIVE: Recycled PET Strapping

Carbon Savings: 2,200 kg CO₂e → 1,056 kg CO₂e (–52%)

Cost Impact: \$120.00 → \$108.00 (–10%)

Compliance: Compliant with EU Single-Use Plastics Directive

Compliant with Germany Packaging Act

[Apply to All Containers] [Apply to Oranges Only] [Dismiss]

Carbon Footprint Calculation:

text

CARBON FOOTPRINT — Container 1

Total CO₂e: 4,870 kg

Breakdown:

• Vessel Fuel: 3,000 kg CO₂e

• Truck Fuel: 120 kg CO₂e

• Reefer Electricity: 600 kg CO₂e

• Packaging Materials: 800 kg CO₂e

Send RFQ
Provider asks clarification (structured Q&A)
Providers submit quotes
Seller compares quotes in panel
Seller selects quote
UI Example — RFQ Setup:

text

MODE B — RFQ TO LOGISTICS PROVIDERS (LSPs)

SERVICE CATEGORIES

Trucking

Customs Brokerage

Warehousing

RFQ DISTRIBUTION

Directed — Select specific LSPs from saved contacts

☒ Anonymous Broadcast — Send to all eligible LSPs

Selected LSPs (Directed):

Fast Trucking Co. (Trust: 85) | Route: Beheira → Damietta

Green Haul (Trust: 83) | Route: Beheira → Damietta

Express Cargo (Trust: 76) | Route: Beheira → Damietta

[Send RFQ]

[Save Draft]

Clarification Request (Improvement #4):

text

Shipping Line Quotes:

text

SHIPPING LINE QUOTES — 2 × 40ft Reefer, Damietta → Hamburg

Line Base Fee Add-Ons Total Fee Select

MSC Egypt \$4,200 \$300 \$4,500 [Select]

Maersk Egypt \$4,350 \$280 \$4,630 [Select]

AI Recommendation: "MSC Egypt offers the best rate and has the shortest

transit time (12 days vs 14 days)."

Selected: MSC Egypt — \$4,500

[Confirm Selection] [Ask Clarification] [Dismiss]

3.12.3.9 Step 6: Alternative Delivery Ports

Description: Add alternative ports within the same destination country.

Fields:

UI Example:

text

ALTERNATIVE DELIVERY PORTS

■ ■

■ ■

3.12.3.11 Step 8: SGTX Fee Calculation

Description: Calculate the SGTX platform fee (1.5%) and final quoted price.

Formula:

text

Total Trade Value = EXW Total + Total Logistics Costs (as defined by incoterm)

$$\text{SGTX Fee} = \text{Total Trade Value} \times 1.5\%$$

Final Quoted Price = Total Trade Value + SGTX Fee

UI Example:

text

■ SGTX FEE CALCULATION ■

■ ■

■ EXW Total: \$150,000.00 ■

2

■ Logistics Costs: ■

■ • Trucking: \$127.50 ■

■ • Ocean Freight: \$4,200.00 ■

■ • Customs Brokerage: \$150.00 ■

■ • THC: \$300.00 ■

■ • Insurance: \$450.00 ■

■ Total Logistics: \$5,227.50 ■

□ □

■ Total Trade Value (EXW + Logistics): \$155,227.50 ■

11

■ SGTX Platform Fee: $1.5\% \times \$155,227.50 = \$2,328.41$ ■

11

■ FINAL QUOTED PRICE TO BUYER: \$157,555.91 ■

11

■ Breakdown by Alternative Ports: ■

■ • Alexandria (primary): \$157,555.91 ■

■ • Damietta (alternative): \$157,405.91 ■

■ • Port Said (alternative): \$157,655.91 ■

22

■ Multi-Shipment Breakdown: ■

■ • Shipment 1: \$78,777.96 ■

■ • Shipment 2: \$78,777.96 ■

■ ■

■ [Submit Quote] [Save Draft] [Back] ■

3.12.3.12 Step 9: Submit Quote (One Click)

Final Action: Clicking "Submit Quote".

Governor Validation:

All mandatory fields present

Packing plan locked (including non-uniform layers validated)

For multi-shipment: schedule fully defined, each shipment validated

Alternative ports valid

SGTX fee correctly calculated and added

For each mandatory service (based on incoterm), a quote is selected (from Mode A, B, or C)

All documents generated (packing list, invoice, customs declaration)

Acceptance criteria matrix included

Commercial settlement details included

If ALLOW:

Trade status → QUOTED

Quote package stored

Buyer receives Smart Inbox item (priority 75) with comparison table

Seller receives confirmation

If CONDITIONAL or DENY:

Governor returns plain-language tenant_message (A1, Groq → Ollama) with condition checklist and one-click remediation links

Seller fixes issues, clicks "Retry" — one click to resubmit

UI Example:

text

■ SUBMIT QUOTE ■

22

■ QUOTE PACKAGE SUMMARY ■

■ ■ Final Quoted Price: \$157,555.91 ■ ■

■ ■ EXW Total: \$150,000.00 ■ ■

■ ■ Logistics Total: \$5,227.50 ■ ■

■ ■ SGTX Fee: \$2,328.41 ■ ■

■ ■ ■ ■

■ ■ Alternatives: 3 delivery options ■ ■

■ ■ Multi-Shipment: 2 shipments ■ ■

■ ■ Documents: 6 of 6 verified ■ ■

22

■ ■ All validation checks passed ■

■ ■

■ [Submit Quote] [Save Draft] [Cancel] ■

Example CONDITIONAL Response (Missing Mandatory Service):

json

 $\{$

```
"verdict": "CONDITIONAL",
```

```
"tenant_message": {
```

"summary": "Your quote cannot be submitted because Ocean Freight is mandatory for CFR incoterm, but no ocean freight quote has been selected.",

```
"conditions": [
```

 $\{$

```
"condition_id": "mandatory_service_missing",
```

```
"label": "Select Ocean Freight quote",
```

"action_url": "/v1/quote/logistics/ship-direct?trade_id=..."

```

}
],
"escalation_hint": "If you believe this is an error, request a human review."
}
}

```

3.12.4 Integration with Other Parts

3.12.5 One-Click Count Summary (Seller)

3.12.6 Implementation Checklist (Phase 2)

3.12.6.1 Loading Origin

Build Loading Origin component (country, port, alternative point)

Implement OSRM distance calculation

Add Governor validation (sanctions, incoterm compatibility)

3.12.6.2 EXW Price Lock

Build live market chart (FAO, USDA, World Bank)

Implement AI fair price badge (A1, Groq)

Implement dynamic unit conversion (per ton / per kilo / per unit)

Implement total EXW value calculation (real-time)

Implement price deviation justification (>30% above AI band)

Implement post-lock price watch (A2, background monitoring)

Implement five high-end add-ons:

Volume-tiered pricing

Dynamic currency conversion

Historical price comparison

Lock & hold

Admin-controlled floor/ceiling

3.12.6.3 Packing & Containerisation

Implement weight calculation engine (real-time)

Implement palletisation optimiser (ORTools CP-SAT)

Implement non-uniform layer stacking (mixed layer configurations)

Implement 3D container viewer (Three.js)

Implement collaborative editing (Yjs + WebRTC)

Implement ecological packaging advisor (A1, Groq)

Implement carbon footprint calculation (ISO 14067)

Implement lock packing plan (Governor validation)

3.12.6.4 Logistics Orchestration (Three Modes)

Implement Mode A — Manual Entry with incoterm-based filtering

Implement Mode B — RFQ to LSPs with clarification requests

Implement Mode C — Direct to SHIP with add-on services

Build combined mode selection and comparison panel

Build alternative delivery ports with AI suggestions

3.12.6.5 Multi-Shipment Response

Implement accept as proposed (one click)

Implement propose modifications (edit delivery date, port, containers)

Implement reject — propose single shipment

Add Governor validation for multi-shipment modifications

3.12.6.6 Quote Submission

Implement SGTX fee calculation (1.5%)

Build quote breakdown display (EXW, logistics, SGTX fee, final price)

Implement submit quote endpoint with Governor validation

Implement Decision Panel for CONDITIONAL/DENY

3.12.6.7 Testing

Test full seller workflow (accept request → lock EXW → pack → logistics → submit quote)

Test all three logistics modes (A, B, C)

Test combined modes (B + C)

Test multi-shipment acceptance and modification

Test non-uniform layer stacking validation

Test EXW price deviation justification

Test post-lock price watch

Test alternative delivery ports

Test Governor validation for mandatory services

3.12.7 AI Authority Summary for Phase 2

END OF PART 3.12 — PHASE 2: SELLER QUOTE, PACKING & LOGISTICS ORCHESTRATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 3.13 — PHASE 3: CONTRACTING, NEGOTIATION & FEE COLLECTION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

3.13.0 Purpose & Design Philosophy

Phase 3: Contracting, Negotiation & Fee Collection is the critical commercial bridge between the seller's quote and the legally binding execution of the trade. This phase transforms the commercial agreement into a cryptographically signed, legally enforceable contract, collects the SGTX platform fee, and prepares the shipment for physical execution. It is the moment when the trade becomes irrevocable and the USTN is born.

Goal: Both parties mutually agree on final terms, sign a legally binding contract (with mandatory SGTX Witness Clause), pay the SGTX platform fee (per shipment for multi-shipment contracts), and generate the USTN.

Key Principles:

Mutual confirmation — Both parties must explicitly agree to the final terms before contract assembly.

Clause Forge (A2) — AI automatically generates the contract with all commercial terms and mandatory clauses.

SGTX Witness Clause — Non-removable clause establishing SGTX's role as non-custodial witness and fee collector.

Partial acceptance — Buyers can accept part of a multi-shipment schedule while negotiating the rest.

Counter-offer with reason — Mandatory reason (≥20 chars) for any counter-offer, creating an audit trail.

Deadline extension — Either party can request and approve deadline extensions.

Per-shipment fee collection — For multi-shipment contracts, fees are collected per shipment, not upfront.

Deferred government fees — Option to defer eligible government fees with PSP guarantee.

One-click core — Every irreversible action (sign, pay, confirm) is a single click.

Non-marketplace — Never suggests alternative counterparties or service providers.

Phase 3 Timeline: Days 9–12 (negotiation, contracting, fee payment)

One-click guarantee: From quote review to contract lock, ~11 clicks across parties. Voice commands available for approvals and signatures.

AI Integration in Phase 3:

3.13.1 Phase 3 Overview

3.13.1.1 High-Level Workflow

text



Counterparty Response:

text

DEADLINE EXTENSION REQUEST — USTN (Not yet generated)

Seller has requested a deadline extension.

Current Deadline: 2026-06-15 14:00 UTC (23h 45m remaining)

Proposed New Deadline: 2026-06-17 14:00 UTC (+48h)

Reason: "Internal approvals are still pending. Need 48 hours to finalise."

[Approve Extension] [Reject] [Counter-Propose]

3.13.3.6 Step 2D: Visual Diff for Amendments

Description: When a party proposes amendments, the system shows a side-by-side diff view.

UI Example:

text

AMENDMENT DIFF VIEW

CURRENT PROPOSED

Port: Alexandria Port: Damietta

Delivery: 15 Jul 2026 Delivery: 20 Jul 2026

Packaging: Boxes (10 kg) Packaging: Mesh bags

Price: \$155,000 Price: \$154,000

SGTX Fee: \$2,325 SGTX Fee: \$2,310

A2 (HF Donut) extracts key fields: parties, goods, price, incoterm, governing law, delivery terms, payment terms

System validates extracted fields against trade data

Mandatory: User must also sign a separate SGTX FEES Addendum (generated automatically, contains the SGTX Witness Clause and fee terms)

Both documents stored, both signed by all parties and Governor

UI Example:

text

■ UPLOAD OWN CONTRACT ■

■ ■

■ Upload Contract PDF: [Choose File] (max 10 MB) ■

■ ■

■ AI Extraction Results (A2, HF Donut): ■

■ ■ ■ Seller: Mekong Fresh Co. (matches trade) ■ ■

■ ■ ■ Buyer: Nile Foods (matches trade) ■ ■

■ ■ ■ Goods: Fresh Oranges (HS 0805.10) — matches trade ■ ■

■ ■ ■ Price: \$154,000 (matches final agreed price) ■ ■

■ ■ ■ Incoterm: CFR (matches trade) ■ ■

■ ■ ■ Delivery: 20 Jul 2026 (matches trade) ■ ■

■ ■ ■ Governing Law: UAE law (not Egypt law — inform legal team) ■ ■

■ ■ ■ Payment Terms: Documentary Collection, 30 days (matches settlement) ■ ■

■ ■

■ ■ ■ MANDATORY: SGTX FEES ADDENDUM (Non-negotiable) ■

■ ■ "The parties acknowledge and agree that SGTX Platform fees (1.5% of trade ■ ■

■ ■ value = \$2,310) shall be paid to SGTX as specified in the SGTX Witness Clause. ■ ■

■ ■ This addendum is incorporated into and forms part of the contract." ■ ■

■ [Uploaded Contract + SGTX FEES Addendum] [Review & Sign] ■

3.13.3.10 Step 6: Logistics Addendum Signing

Description: Before contract lock, each selected logistics provider must sign a Logistics Service Addendum.

Addendum Content:

USTN (placeholder until fee payment)

Obligation not to release container(s) without SGTX confirmation

Requirement to include USTN and GTIDs on all documents

Penalty clause for unauthorised release

Service-specific terms (trucking, ocean freight, etc.)

Workflow:

Each provider receives Smart Inbox item: "Sign logistics addendum for trade USTN..."

Provider opens addendum, reviews (PDF)

Provider clicks "Sign with Passkey" (ZITADEL) — one click

Governor records signature in logistics_addenda

UI Example:

text

■ LOGISTICS ADDENDUM SIGNING — USTN (Not yet generated) ■

11

■ Provider: Fast Trucking Co. (LSP) ■

■ Service: Trucking (Beheira → Damietta) ■

■ Fee: \$127.50 ■

■ ■

■ ■ LOGISTICS SERVICE ADDENDUM ■ ■

4444

■ ■ 1. The Provider agrees to transport the goods from Beheira to Damietta port. ■ ■

■ ■ 2. The Provider shall not release the container(s) without SGTX confirmation. ■ ■

■ ■ 3. The Provider shall include USTN and GTIDs on all shipping documents. ■ ■

3.13.3.11 Step 7: SGTX Fee Payment — Single-Shipment vs. Multi-Shipment

■ ■ 3 ■ 15 Sep 2026 ■ Damietta ■ SCHEDULED ■ \$2,310 ■ N/A ■ ■

11

■ Shipment 2 Payment Details: ■

■ ■ Trade Value: \$154,000 ■ ■

■ ■ SGTX Fee: \$2,310 ■ ■

■ ■ PSP: Fawry ■ ■

■ ■ Status: ■ Pending ■ ■

22

■ [Pay Shipment 2 Fee] [Mark Shipment 2 as Ready] ■

3.13.3.12 Step 8: Deferred Government Fee Payment (Improvement #7)

Description: Option to defer eligible government fees (e.g., import duties) with PSP guarantee.

Workflow:

During Stage 1 payment setup, system displays which fees can be deferred

Seller toggles "Defer" for those fees

System creates guarantee instruction

PSP holds the deferred amount as a guarantee

Actual payment triggered on milestone (e.g., CUSTOMS_IMPORT)

Three-step escalation if guarantee expires

UI Example:

text

■ DEFERRED PAYMENT — SGTX-EG-26-F3A-1 ■

11

■ Eligible Fees for Deferral (RIA-detected): ■

Fee Type	Amount	Eligible	Action
...	...	...	...

Description: Both parties sign the contract with ZITADEL passkey; SGTX Governor signs as witness.

Buyer signs contract (ZITADEL passkey) — one click

SGTX Governor signs as witness — automatic (no click)

USTN generated

UI Example (Contract Signing):

■ Contract Status: PENDING_SIGNATURES ■

■ ■ [Sign with Passkey] ■ ■

■■■ Seller Signature: Mekong Fresh Co. — PENDING ■■■

■■■■

■ ■ (Automatic after both parties sign) ■ ■

■ ■ ■ All parties must sign before the contract can lock. ■

11

Either party clicks "Request Modification" in TCC → Multi-shipment Schedule Card

Select shipment(s) to modify (future shipments only; already-locked shipments are immutable)

Propose changes: delivery date, port, container count, commodities/packages

Add mandatory reason (≥20 chars)

Send to counterparty (one click)

Counterparty reviews diff view. Can Accept, Reject, or Counter.

If accepted, a schedule addendum is created, signed by both parties.

UI Example — Modification Request:

text

SCHEDULE MODIFICATION REQUEST — MC-20260415-001

Modify Shipment 2 (future shipment — not yet locked)

Proposed Changes:

Field

Current

Proposed

Change

Delivery Date

15 Aug 2026

30 Aug 2026

+15 days

Port

Port Said

Alexandria

Different

Containers

1

2

+1

Commodities

Oranges (100 MT)

Oranges (150 MT)

+50 MT

Reason (mandatory, ≥20 chars):

[Warehouse capacity issue. Need additional storage time and increased container

count for peak season. Port change due to logistics partner availability.]

[Send Modification Request] [Cancel]

Counterparty Review:

■ ■

Build multi-shipment schedule display

Implement Accept/Negotiate/Amend actions

3.13.7.2 Negotiation Panel

Build three-column negotiation panel (Offer History, Current Offer, Trade Room Chat)

Implement offer history (chronological)

Implement Accept/Counter/Reject/Request Info buttons

Implement partial acceptance (Improvement #2a)

Implement counter-offer with mandatory reason (Improvement #2b)

Implement deadline extension request (Improvement #2c)

Implement visual diff for amendments

Implement Groq translation for Trade Room Chat

Implement timer for offer expiry

3.13.7.3 Mutual Confirmation

Implement mutual confirmation endpoint

Record mutual confirmation timestamp

Create pre-contract snapshot (immutable JSONB)

3.13.7.4 Contract Assembly (Clause Forge)

Implement Clause Forge (A2, HF local) for contract generation

Include all commercial terms (price, incoterm, delivery options, schedule)

Include mandatory SGTX Witness Clause (non-removable)

Include acceptance criteria matrix

Include commercial settlement details

Include insurance requirements

Include special trade instructions

Generate contract PDF

3.13.7.5 Upload Own Contract

Implement contract upload (PDF, max 10 MB)

Implement A2 (HF Donut) extraction

Implement field validation against trade data

Implement mandatory SGTX FEES Addendum generation

Implement document storage and signature collection

3.13.7.6 Logistics Addendum Signing

Implement logistics addendum generation

Implement Smart Inbox notification to providers

Implement signature collection (ZITADEL passkey)

Implement signature status tracking

Implement Governor validation before contract lock

3.13.7.7 SGTX Fee Payment

Implement single-shipment fee collection (upfront)

Implement multi-shipment per-shipment fee collection

Implement PSP Router (A2, LightGBM + Groq)

Implement PSP split instruction generation

Implement webhook verification

Implement FeeLock state management (NATS KV)

3.13.7.8 Deferred Government Fee Payment

Implement deferred payment toggle

Implement PSP guarantee creation

Implement three-step escalation (reminder, alert, expiry)

Implement auto-charge authorisation

3.13.7.9 Container Release Confirmation

Implement release token generation

Implement email notification (co-branded)

Implement API webhook

Implement provider acknowledgment (one click)

3.13.7.10 Digital Signatures & Contract Lock

Implement ZITADEL passkey signature collection (buyer, seller)

Implement Governor witness signature (automatic)

Implement contract lock status transition

Implement USTN generation

Implement Smart Inbox confirmations

3.13.7.11 Multi-Shipment Schedule Modification

Implement schedule modification request endpoint

Implement diff view for counterparty review

Implement Accept/Reject/Counter actions

Implement schedule addendum generation

Implement addendum signing (both parties + Governor)

3.13.7.12 Testing

Test full buyer negotiation → accept → sign workflow

Test partial acceptance (multi-shipment)

Test counter-offer with mandatory reason

Test deadline extension request and approval

Test Clause Forge contract generation
Test upload own contract with AI validation
Test logistics addendum signing
Test single-shipment fee payment
Test multi-shipment per-shipment fee collection
Test deferred government fee payment
Test container release confirmation
Test digital signatures and contract lock
Test multi-shipment schedule modification after lock

3.13.8 AI Authority Summary for Phase 3

END OF PART 3.13 — PHASE 3: CONTRACTING, NEGOTIATION & FEE COLLECTION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 3.14 — PHASE 4: UNIVERSAL TRADE FINANCE (FULL DISCLOSURE MODEL)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

3.14.0 Purpose & Design Philosophy

Phase 4: Universal Trade Finance is the financing engine of the SGTx platform. After a trade is locked, any party (buyer or seller) may request financing. The request triggers an automatic RFQ broadcast to all financiers whose pre-set preferences match. Financiers see full trade details before bidding — this is the key trust advantage of SGTx trade finance. Multiple financiers can co-finance (split funding) a single request. The SGTx financing fee is a flat 0.25% of the financed amount, deducted from disbursement via PSP split in USD, EGP, or EURO (crypto fee collection coming soon).

Goal: Provide transparent, efficient, and competitive trade financing by giving financiers full visibility into the trade and enabling multiple financiers to co-finance.

Key Principles:

Full disclosure — Financiers see complete trade details (buyer/seller names, documents, historical performance, previous financing history) before bidding. This builds trust and confidence.

Non-custodial — SGTx never holds financed funds. PSPs handle disbursement and repayment.

Co-financing (Split Funding) — Multiple financiers can each finance a portion of the same request. This is fully supported and enabled by default.

Jurisdiction-aware — DeFi financing is only available in jurisdictions where legally permitted. For Egypt, DeFi is hidden.

Full disclosure, no crypto for SGTx fees — SGTx financing fee (0.25%) is collected in USD, EGP, or EURO via PSP split. Crypto fee collection is coming soon.

Encrypted bids — Bids are encrypted with the borrower's public key; even SGTx cannot read them until the bidding window closes.

ZK proof of reserves (optional) — Financiers can provide zero-knowledge proof of reserves without revealing wallet balance.

Mandatory DeFi risk summary — Financiers must acknowledge a plain-language risk summary before selecting DeFi settlement methods.

Collateral monitoring — Real-time LTV tracking, margin calls, liquidation risk prediction.

Regulatory reporting — Automated generation of CBE monthly remittance reports (Egypt), ECB statistical returns (EU), FinCEN SAR (USA), etc.

Phase 4 Timeline: After contract lock, at any time before settlement

One-click guarantee: Request financing (1), Submit bid (1), Accept selected bids (1), Disburse (1)

AI Integration in Phase 4:

3.14.1 Critical Analysis — What Was Missing from Original Blueprint

Before proceeding with the fully expanded specification, I have identified the following critical gaps in the original Phase 4 financing section:

3.14.2 Co-Financing (Split Funding) — Complete Specification

3.14.2.1 Overview

Co-financing (split funding) allows multiple financiers to each finance a portion of a single financing request. This is a critical feature for real-world trade finance, enabling risk sharing, competitive pricing, and access to larger financing amounts.

Key Features:

Unlimited financiers — Any number of financiers can bid on a single request.

Partial bids — Each financier can bid for any portion of the requested amount (up to 100%).

Combined acceptance — Borrower selects a combination of bids that sum to the requested amount.

Blended APR — System calculates the weighted average APR across selected financiers.

Independent disbursement — Each financier disburses their portion independently.

Independent repayment — Borrower repays each financier according to their respective terms.

Separate agreements — Each financier has their own annex to the master agreement.

Example:

Requested Amount: \$100,000

Financier A bids: \$60,000 @ 4.8% APR

Financier B bids: \$40,000 @ 5.2% APR

Borrower accepts both → Blended APR = $(60,000 \times 4.8\% + 40,000 \times 5.2\%) / 100,000 = 4.96\%$

3.14.2.2 Co-Financing Workflow

text

1. Borrower requests \$100,000

↓

2. Auto-RFQ broadcast to all matching financiers

↓

3. Financier A bids \$60,000 @ 4.8% APR

Financier B bids \$40,000 @ 5.2% APR

Financier C bids \$100,000 @ 5.8% APR

↓

4. Bidding window closes

↓

5. Borrower sees all bids decrypted

↓

6. Borrower selects combination ($A + B = \$100,000$)

↓

7. Blended APR calculated: 4.96%

↓

8. Master agreement + annexes generated

↓

9. Borrower signs both annexes

↓

10. Financier A signs annex A

Financier B signs annex B

↓

11. Financier A disburses \$60,000 (minus 0.25% fee)

Financier B disburses \$40,000 (minus 0.25% fee)

↓

12. Borrower receives \$99,750 total (after fees)

↓

13. Borrower repays Financier A and Financier B independently

3.14.2.3 Co-Financing Database Design

sql

-- Financing requests (master)

CREATE TABLE financing_requests (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,

contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,

borrower_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

amount_requested DECIMAL(20,4) NOT NULL,

currency TEXT DEFAULT 'USD',

tenor_days INTEGER NOT NULL,

financing_type TEXT NOT NULL, -- 'PRE_SHIPMENT', 'POST_SHIPMENT', 'INVOICE', 'STRUCTURED'

preferred_settlement_method TEXT,

collateral_type TEXT,


```

special_instructions TEXT,
credit_intelligence JSONB, -- AI-generated credit score, default probability
status TEXT DEFAULT 'REQUESTED', -- 'REQUESTED', 'BIDDING', 'AWARDED', 'DISBURSED',
'REPAID', 'CANCELLED'
bidding_window_closes TIMESTAMPTZ,
awarded_at TIMESTAMPTZ,
disbursed_at TIMESTAMPTZ,
repaid_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Financing offers (bids) — each bid is a portion
CREATE TABLE financing_offers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_request_id UUID NOT NULL REFERENCES financing_requests(id) ON DELETE CASCADE,
financier_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
amount_offered DECIMAL(20,4) NOT NULL,
apr DECIMAL(6,4) NOT NULL,
collateral_requirements TEXT, -- Must be at least borrower's stated type
settlement_method TEXT NOT NULL,
settlement_details JSONB, -- Bank account details, DeFi protocol, etc.
conditions TEXT,
note_to_borrower TEXT,
reserve_proof TEXT, -- ZK proof of reserves (optional)
bid_encrypted BOOLEAN DEFAULT true,
status TEXT DEFAULT 'SUBMITTED', -- 'SUBMITTED', 'ACCEPTED', 'REJECTED', 'COUNTERED'
submitted_at TIMESTAMPTZ DEFAULT NOW(),
selected BOOLEAN DEFAULT false,
accepted_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL
);
-- Financing agreements (master + annexes)
CREATE TABLE financing_agreements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_request_id UUID NOT NULL REFERENCES financing_requests(id) ON DELETE CASCADE,

```

```

master_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL, -- NULL
for master

financier_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
amount DECIMAL(20,4) NOT NULL,
apr DECIMAL(6,4) NOT NULL,
tenor_days INTEGER NOT NULL,
collateral_requirements TEXT,
settlement_method TEXT NOT NULL,
sgtx_financing_fee DECIMAL(20,4) NOT NULL,
sgtx_fee_currency TEXT DEFAULT 'USD', -- USD, EGP, EURO
sgtx_fee_crypto_available BOOLEAN DEFAULT false, -- Coming soon
takaful_opted_in BOOLEAN DEFAULT false,
status TEXT DEFAULT 'PENDING_SIGNATURES', -- 'PENDING_SIGNATURES', 'ACTIVE', 'REPAID',
'DEFAULTED'
borrower_signature TEXT,
financier_signature TEXT,
governor_signature TEXT,
loom_hash TEXT,
disbursed_at TIMESTAMPTZ,
repaid_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Financing repayments (per financier)
CREATE TABLE financing_repayments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_agreement_id UUID NOT NULL REFERENCES financing_agreements(id) ON DELETE
CASCADE,
scheduled_date DATE NOT NULL,
principal_due DECIMAL(20,4) NOT NULL,
interest_due DECIMAL(20,4) NOT NULL,
actual_paid_at TIMESTAMPTZ,
principal_paid DECIMAL(20,4),
interest_paid DECIMAL(20,4),
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'PAID', 'LATE', 'DEFAULTED'
late_days INTEGER DEFAULT 0,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,

```

created_at TIMESTAMPTZ DEFAULT NOW())

);

3.14.3 Step-by-Step Specification — Fully Extended

3.14.3.1 Step 1: Financing Request Initiation (Borrower)

Description: Borrower requests financing for a locked trade (or specific shipment for multi-shipment).

Fields:

AI Assistance:

A1 (Groq): Recommends maximum LTV based on commodity perishability, jurisdiction, and historical default rates.

A4 (OPA + WasmEdge): Validates that the trade is LOCKED and the borrower has permission.

UI Example:

text

FINANCING REQUEST — SGTX-EG-26-F3A-1

Trade: Mekong Fresh Co. → Nile Foods

Total Trade Value: \$154,000

Incoterm: CFR | Commodity: Fresh Oranges

FINANCING DETAILS

Requested Amount: [\$100,000] (Max LTV recommendation: 70% = \$107,800)

Financing Type: [Pre-shipment ▼]

Tenor: [60] days

Preferred Settlement Method: [Bank transfer ▼]

Preferred Currency: [USD ▼]

Collateral Type: [Goods ▼]

AI Recommendation (A1, Groq): "Maximum LTV of 70% recommended for perishable

fresh fruit. Requested amount (\$100,000) is within this limit."

■ history. The primary risk is the perishable nature of the commodity and ■

■ moderate route risk. Default probability is low at 4.2%. ■

114

■ [Submit to Financiers] [Cancel] ■

3.14.3.3 Step 3: Auto-RFQ Broadcast (Background, Zero Clicks)

Description: System automatically broadcasts the financing request to all matching financiers.

Matching Logic:

System queries financier_preferences table for all active financiers.

Filters financiers whose preferences match the borrower's request:

Accepted borrower countries

Minimum borrower trust score

Minimum trade value

Maximum financed amount

Preferred financing types

Preferred settlement methods

Geographic restrictions

For each matching financier, an RFQ is created and delivered via:

Smart Inbox item (priority based on match score)

Email (if financier enabled notifications)

API webhook (if financier integrated)

Match Score (A1, Groq):

A composite score (0-100) representing how well the request aligns with the financier's preferences and historical success patterns.

UI Example (Financier Smart Inbox):

text

■ ■ SMART INBOX — Financing Opportunity ■ ■

■ ■

■ ■ HIGH PRIORITY (1) ■

■ ■ ■ New Financing Opportunity — Nile Foods (Egypt) ■ ■ ■

■■■■

Total financed amount
Repayment performance (on-time, late, default)
Existing active loans
6. AI Credit Intelligence:
Credit score
Default probability
Recommended LTV
Groq risk summary
Credit score trend (line chart over last 12 months with financing events)
UI Example:

text

FINANCING OPPORTUNITY — Full Disclosure

Request: FR-001 | Window Closes: 2026-06-21 14:00 UTC (4h)

TRADE OVERVIEW

USTN: SGTX-EG-26-F3A-1

Commodity: Fresh Oranges (HS 0805.10)

Total Trade Value: \$154,000

Incoterm: CFR

Ports: Damietta → Alexandria

Vessel: MSC FIONA | ETD: 20 Jun 2026 | ETA: 05 Jul 2026

Trade Criticality: PRIORITY

COUNTERPARTY DETAILS

Buyer: Nile Foods (SGTX-EG-TRD-001234-5B6C)

Trust Score: 88 | KYB Tier: 2 | Sanctions: Cleared

4444

■ ■ Credit Score Trend (12 months): ■ ■

■ ■ 90 ■ ■ ■ ■ ■ ■ ■ ■ ■ ■

■ ■ 75 ■ ■ ■ ■

■ ■ Jul Aug Sep Oct Nov Dec Jan Feb Mar Apr May Jun ■ ■

11

■ ■ AI Summary (A1, Groq): "This borrower has a strong credit profile (78/100). ■

■ Their payment history is excellent (100% on-time), and they have no dispute ■

■ history. The primary risk is the perishable nature of the commodity and ■

■ moderate route risk. Default probability is low at 4.2%. ■

11

■ [Submit Bid] [Decline] [Ask Clarification] ■

3.14.3.5 Step 5: Bid Submission (Financier, One Click)

Description: Financier submits an encrypted bid. Bids can be for a portion of the total request (enabling co-financing).

Bid Form Fields:

Encryption & Blind Bidding:

Bid is encrypted with the borrower's public key on client side before transmission.

Even SGTX cannot read the bid until the borrower decrypts after window closes.

DeFi Risk Summary (Mandatory Modal):

If financier selects a DeFi settlement method, a mandatory modal appears with 5 bullet points (A1, Grog).

UI Example:

text

■ SUBMIT BID — Financing Request FR-001 ■

11

Request: Nile Foods (Egypt) → \$100,000, 60 days, Pre-shipment

Amount Offered: [\$60,000] (of \$100,000) ← Co-financing portion

APR: [4.8]% (Benchmark: 5.2% market average)

Collateral Requirements: [Goods ●] [Warehouse receipt] [Receivables]

Settlement Method: [Bank transfer ▼]

Conditions:

[First priority on receivables. Borrower must maintain insurance coverage.]

Note to Borrower:

[We can offer quick disbursement within 24 hours of agreement signing.]

OPTIONAL: Reserve Proof

Include ZK proof of reserves (shows liquidity without revealing balance)

AI Recommendation: "Your APR (4.8%) is below market average (5.2%).

This is a competitive bid."

Your bid will be encrypted and only visible to the borrower after the

bidding window closes.

[Submit Bid] [Cancel]

DeFi Risk Summary Modal (A1, Groq):

text

DEFI PLAINLANGUAGE RISK SUMMARY

■ arrangement as a non-custodial witness. The platform's financing service fee ■

[REDACTED]

□ □

■ ■ New LTV: 78.4% ■ ■

■ ■ [Apply Simulation] [Reset] ■ ■

22

3.14.4.1 Purpose

3.14.4.2 Key Features

3.14.4.3 Workflow

0.05% contribution collected from financed amount

If claim event occurs, borrower files claim

Payout made from Takaful fund

3.14.5.1 Purpose

Jurisdiction Filtering:

For jurisdictions where defi_allowed = true: Full secondary market including DeFi positions

text

■ SECONDARY MARKET — Tokenised Trade Assets ■

Implement read-only document access

Implement historical performance queries

Implement credit score trend (12-month line chart)

Implement Groq credit summary (A1)

3.14.8.3 Bid Submission (Co-Financing)

Implement encrypted bid submission

Implement co-financing (portion of P — split funding)

Implement APR benchmark display

Implement ZK proof of reserves (A2 verification)

Implement DeFi Risk Summary modal (A1, Groq)

Implement DeFi jurisdiction gate (A4, WasmEdge)

Implement blended APR calculation for selected bids

3.14.8.4 Co-Financing Acceptance

Implement bid decryption (after window closes)

Build bid comparison table

Implement co-financing selection (sum of bids $\leq$ P)

Implement blended APR calculation

Implement acceptance endpoint (POST /v1/finance/award)

3.14.8.5 Financing Agreement

Implement Clause Forge for financing agreements (A2)

Include mandatory SGTX Financing Witness Clause

Implement annex generation per financier

Implement ZITADEL passkey signature collection

Implement Governor witness signature (automatic)

3.14.8.6 Disbursement & Fee Collection

Implement disbursement endpoint (POST /v1/finance/disburse)

Implement PSP split for fee deduction (0.25%)

Implement PSP split for conventional methods

Implement PSP split for DeFi methods (where allowed)

Implement Financing FeeLock marking ACTIVE

Add "crypto fee collection coming soon" note

3.14.8.7 Repayment Monitoring

Implement PSP webhook monitoring

Implement OpenBanking reconciliation

Implement on-chain event monitoring (The Graph)

Implement repayment history updates

Implement credit score recalculation

3.14.8.8 Collateral Monitoring

Implement LTV calculation (real-time)

Implement LTV gauge chart

Implement LTV history line chart

Implement liquidation risk meter (LSTM, A2)

Implement margin call issuance

Implement "Simulate Price Drop" slider

Implement collateral monitoring dashboard

3.14.8.9 DeFi Integration (Jurisdiction-Dependent)

Implement DeFi protocol risk oracle (A2)

Implement stablecoin de-peg monitor (A2)

Implement liquidation early warning (LSTM, A2)

Implement DeFi position tracking

Implement DeFi jurisdiction gate (A4, WasmEdge)

3.14.8.10 Takaful Protection Fund

Implement Takaful opt-in

Implement contribution collection (0.05%)

Implement segregated fund accounting

Implement Sharia board claim approval (2/3)

Implement claim payout

3.14.8.11 Secondary Market

Implement tokenised trade assets (ERC-3525)

Implement token listing (financiers)

Implement token purchase

Implement jurisdiction filtering

3.14.8.12 Regulatory Reporting

Implement CBE monthly remittance report (Egypt)

Implement ECB statistical return (EU)

Implement FinCEN SAR (USA)

Implement VAT/GST reports

3.14.8.13 Testing

Test full borrower → financier → acceptance → disbursement → repayment workflow

Test co-financing (split funding — multiple financiers)

Test DeFi jurisdiction gate (hidden where not allowed)

Test DeFi Risk Summary modal

Test ZK proof of reserves

Test collateral monitoring (LTV, margin calls)

Test regulatory reporting

Test encrypted bidding

Test SGTX fee deduction in USD/EGP/EURO

Test Takaful protection fund

3.14.9 AI Authority Summary for Phase 4

END OF PART 3.14 — PHASE 4: UNIVERSAL TRADE FINANCE (FULL DISCLOSURE MODEL)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 3.15 — PHASE 5: PHYSICAL EXECUTION & MULTIPARTY TRACKING

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

3.15.0 Purpose & Design Philosophy

Phase 5: Physical Execution & Multiparty Tracking is the operational heart of the SGTX platform. After contract lock, the trade moves from the digital realm into the physical world — containers are loaded, vessels depart, goods are tracked, and delivery is confirmed. This phase coordinates multiple parties (seller, buyer, logistics providers, shipping lines, QC inspectors, customs brokers) through a unified, AI-assisted, real-time tracking system.

Goal: Track the shipment from loading to delivery with AI-powered automation, multisensor consensus, voice-enabled milestone verification, and proactive exception handling.

Key Principles:

One source of truth — All milestone events are captured in real time and visible to all authorised parties.

Multisensor consensus — Milestones can be auto-confirmed when multiple independent sources agree (e.g., barcode scan + weight sensor + GPS).

AI-assisted automation — AI detects anomalies (cold chain deviations, delays, congestion) and suggests corrective actions.

Voice-enabled milestones — Field workers can confirm milestones using voice commands (Vosk + HF Mixtral).

Conditional QC with action plan — Allows loading to proceed with a hold pending resolution of minor issues.

Re-inspection workflow — Buyers can request a second inspection if the initial verdict is disputed.

Stuck trade recovery — Automatic escalation if milestones are overdue.

Multi-shipment independence — Each shipment's milestones are tracked independently; a delay in one does not block others.

Non-marketplace — Never suggests alternative counterparties or service providers.

Phase 5 Timeline: Days 13–40 (from loading to delivery)

One-click guarantee: Each milestone confirmation is a single click or voice command. Container loading can be auto-confirmed when all pallets are scanned.

AI Integration in Phase 5:

3.15.1 Phase 5 Overview

3.15.1.1 High-Level Workflow

text

■ PHASE 5: PHYSICAL EXECUTION & MULTIPARTY TRACKING ■

■ ■

■ ■ 1. PRE-EXECUTION SETUP (Automatic) ■ ■

■ ■ → Loading window selection (seller + trucking) ■ ■

■ ■ → Booking confirmation upload (shipping line) ■ ■

■ ■ → Dynamic document requirements (RIA-driven) ■ ■

■ ■ → Container release pre-advice (terminal) ■ ■

111

13

■ ■ 2. CONTAINER RELEASE & LOADING ■ ■

■ ■ → Container release confirmation (Part 8) ■ ■

■ ■ → Pallet-level loading with non-uniform layer support ■ ■

■ ■ → Barcode scanning (SSCC) + voice commands ■ ■

■ ■ → Multisensor consensus → auto-confirm "LOADED" ■ ■

■ ■ ■

11

■ ■ 3. QC INSPECTION (Conditional Pass Support) ■ ■

■ ■ → PASS: Loading proceeds immediately ■ ■

■■ → FAIL: Loading blocked ■■

■ ■ → CONDITIONAL: Loading proceeds with hold + action plan ■ ■

■ ■ → Re-inspection request (if disputed) ■ ■

■ ■ ■

■ ■ 4. PORT GATE & CUSTOMS ■ ■

■ ■ → Gate In: Container enters terminal (OCR/manual) ■ ■

■ ■ → eBL Interoperability: eBL issuance tracked ■ ■

■ ■ ■

■ ■ 5. VESSEL TRACKING & COLD-CHAIN MONITORING ■ ■

■ ■ → AIS Integration: Vessel position every 6 hours ■ ■

■ ■ → Cold-chain LSTM: Predicts remaining shelf life ■ ■

■ ■ ■

6. ARRIVAL & DELIVERY CONFIRMATION

■ ■ → Arrival milestone confirmed (shipping line) ■ ■

■ ■ → Delivery confirmation (buyer — one click or voice) ■ ■

■ ■ ■

2E — Loading Guide Display (A1, Groq):
Step-by-step loading instructions displayed on mobile app.
UI Example (Loading — Mobile App):

text

LOADING — Container MEDU1234567

USTN: SGTX-1397F3A-456ABC-...

LOADING GUIDE (A1, Groq)

Step 1: Pre-cool container to -18°C for at least 2 hours before loading.

Step 2: Load pallets OR001-OR011 (11 layers × 10 cartons)

Step 3: Load pallets OR012-OR022 (11 layers × 10 + 1 layer × 1)

Step 4: Ensure 2 cm air gap between pallets for air circulation.

Step 5: Secure strapping — recycled PET (recommended).

PALLET SCAN STATUS

Pallet ID SSCC Status Scanned At

OR-001 106141411234567890 Loaded 14:30:00

OR-002 106141411234567891 Loaded 14:31:00

... ..

OR-022 106141411234567911 Pending ---

Progress: 21/22 pallets loaded

Voice: "Load pallet OR022 into container MEDU1234567"



■ [Scan Next Pallet] [Batch Scan Mode] [Confirm Load Complete] ■



Milestone Confirmation Flow:

text

1. Pallet scanned/voice confirmed → `pallet.loaded` event sent



2. Governor validates:

- Device identity (ZITADEL certificate)
- Employee permissions (shipment.milestone.confirm)
- FeeLock status (ACTIVE)
- Pallet belongs to correct shipment (multi-shipment check)



3. If ALLOW: milestone confirmed, pallet_details.status updated to LOADED



4. When last pallet of a container is loaded:

- Multisensor consensus (if enabled) or manual confirmation
- Container milestone "Loaded" auto-confirmed



5. All relevant parties receive Smart Inbox updates

3.15.3.3 Step 3: QC Inspection (Conditional Pass Support)

Description: QC inspector performs on-site inspection with PASS/FAIL/CONDITIONAL verdicts.

Components:

3A — Standard Inspection Process:

Inspector uses mobile app (offline) to scan container and seal numbers, take photos.

Inspects mandatory pallets (AQL sampling + high-priority).

AI detects defects (A2, HF ViT). Inspector confirms or overrides (mandatory reason ≥10 chars).

Submit report.

Verdicts:

3B — Conditional Pass with Action Plan (Improvement #5):

Inspector clicks "Conditional Pass" instead of FAIL.

Form appears: Action plan (free text, mandatory), Deadline (hours/days, default 24h), Escalation if not completed.

Inspector submits report with CONDITIONAL verdict.

Loading may proceed but a hold flag is placed on the shipment.

Seller executes the action plan, then clicks "Mark Action Completed" (one click).

Inspector verifies (one click) — hold removed.

3C — Re-inspection Request:

If QC report is FAIL (or buyer disputes a PASS/CONDITIONAL), any party can request a second inspection.

Workflow:

From TCC, click "Request Re-inspection" — one click.

Select reason, choose same or different QC provider.

Request sent to selected QC provider.

QC provider accepts (one click) and performs second inspection.

Second report overrides the first for loading purposes.

UI Example (Conditional Pass Report):

text

QC INSPECTION REPORT — SGTX-EG-26-F3A-1

Inspector: SGS Inspection

Inspector Date: 2026-06-20

Verdict: CONDITIONAL PASS

DEFECTS DETECTED

Pallet	Defect Type	Severity	AI Confidence	Inspector Override
OR-020	Mould	Medium	88%	Override: "Surface bruising"
				Reason: Dark spot is bruise,
				no mould under magnifier.

CONDITIONAL PASS ACTION PLAN

Predicts remaining shelf life.

5C — Predictive ETA:

Alerts sent to buyer and seller if ETA changes by >24h.

text

■ VESSEL TRACKING — MSC FIONA — SGTX-EG-26-F3A-1 ■

■ ■

■ ■ [Map — AIS Position] ■ ■

■ ■ ■ ■

■ ■ MSC FIONA ● (30°N, 25°E) — Position: 12 Jun 14:30 UTC ■ ■

■ ■ Speed: 18.5 knots | Heading: 320° ■ ■

■ ■ ■ ■

■ ■ Destination: Alexandria ■ ■

■ ■ ETA: 2026-07-05 14:00 UTC (predicted: 2026-07-04 18:00 UTC, -20h) ■ ■

■ ■ Distance to destination: 1,200 nautical miles ■ ■

11

■ COLD-CHAIN MONITORING ■

■ ■ Container: MEDU1234567 | Temperature: -18.2°C | Status: ■ Normal ■ ■

■■■■

Temperature Log (Last 24h):

-18.5

■ ■ -18.0 ■ ■■■■■■■■■■■■■■■■■■■■

-17.5


```

scanned_barcode_id UUID,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- QC jobs
CREATE TABLE qc_jobs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
provider_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
inspection_date DATE,
sampling_plan JSONB,
status TEXT DEFAULT 'ASSIGNED', -- 'ASSIGNED', 'ACCEPTED', 'IN_PROGRESS', 'COMPLETED',
'DISPUTED'
verdict TEXT, -- 'PASS', 'FAIL', 'CONDITIONAL'
conditional_pass_status TEXT, -- 'PENDING', 'COMPLETED', 'EXPIRED'
action_plan TEXT,
action_plan_deadline TIMESTAMPTZ,
action_plan_completed_at TIMESTAMPTZ,
report_url TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Inspection logs (with overrides)
CREATE TABLE inspection_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
qc_job_id UUID REFERENCES qc_jobs(id) ON DELETE CASCADE,
pallet_id TEXT,
ai_defect TEXT,
ai_confidence DECIMAL(5,2),
inspector_override BOOLEAN DEFAULT false,
inspector_override_reason TEXT,
final_classification TEXT,
photo_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Re-inspection requests
CREATE TABLE re_inspection_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

original_ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
requested_by_gtid TEXT NOT NULL,
reason TEXT NOT NULL,
urgency TEXT DEFAULT 'NORMAL', -- 'NORMAL', 'URGENT'
deadline TIMESTAMPTZ,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACCEPTED', 'DECLINED', 'COMPLETED'
new_qc_job_id UUID REFERENCES qc_jobs(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- IoT sensor readings (TimescaleDB hypertable)
CREATE TABLE iot_sensor_readings (
id BIGSERIAL,
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
sensor_type TEXT NOT NULL,
value JSONB NOT NULL,
unit TEXT,
anomaly_flag BOOLEAN DEFAULT false,
recorded_at TIMESTAMPTZ NOT NULL
);
SELECT create_hypertable('iot_sensor_readings', 'recorded_at');
-- Stuck trade escalation logs
CREATE TABLE stuck_trade_escalations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
milestone TEXT NOT NULL,
escalation_level INTEGER NOT NULL, -- 1, 2, 3
triggered_at TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ,
resolution_notes TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL
);
-- Indexes
CREATE INDEX idx_shipment_milestones_ustn ON shipment_milestones(ustn);
CREATE INDEX idx_shipment_milestones_milestone ON shipment_milestones(milestone);
CREATE INDEX idx_shipment_milestones_confirmed_at ON shipment_milestones(confirmed_at DESC);

```

```
CREATE INDEX idx_qc_jobs_ustn ON qc_jobs(ustn);
CREATE INDEX idx_qc_jobs_provider ON qc_jobs(provider_gtid);
CREATE INDEX idx_qc_jobs_status ON qc_jobs(status);
CREATE INDEX idx_inspection_logs_job ON inspection_logs(qc_job_id);
CREATE INDEX idx_re_inspection_requests_ustn ON re_inspection_requests(original_ustn);
CREATE INDEX idx_iot_sensor_readings_ustn ON iot_sensor_readings(ustn);
CREATE INDEX idx_iot_sensor_readings_recorded ON iot_sensor_readings(recorded_at DESC);
CREATE INDEX idx_stuck_trade_escalations_ustn ON stuck_trade_escalations(ustn);
```

3.15.7 Implementation Checklist (Phase 5)

3.15.7.1 Pre-Execution Setup

- Implement loading window selection (seller + trucking)
- Implement booking confirmation upload (HF Donut extraction)
- Implement dynamic document requirements (RIA-driven)
- Implement container release pre-advice (Improvement #6)

3.15.7.2 Container Release & Loading

- Implement container release confirmation (Part 8)
- Implement pallet-level loading with non-uniform layer support
- Implement barcode scanning (SSCC) via mobile app
- Implement voice commands (Vosk + HF Mixtral)
- Implement batch scan mode
- Implement multisensor consensus (A4)
- Implement loading guide generation (A1, Groq)

3.15.7.3 QC Inspection

- Implement standard inspection process (PASS/FAIL)
- Implement conditional pass with action plan (Improvement #5)
- Implement re-inspection request workflow
- Implement QC override flagging (mandatory reason ≥ 10 chars)
- Implement AQL sampling enforcement

3.15.7.4 Port Gate & Customs

- Implement Gate In milestone (OCR/manual)
- Implement customs clearance (Nafeza API)
- Implement eBL interoperability (HF Donut extraction)

3.15.7.5 Vessel Tracking & Cold-Chain

- Implement AIS integration (vessel position)
- Implement digital twin simulation (ETA predictions)

Implement cold-chain monitoring (LSTM, A2)

Implement temperature excursion alerts

3.15.7.6 Arrival & Delivery

Implement arrival milestone confirmation (shipping line)

Implement import customs clearance (buyer/broker)

Implement delivery confirmation (one click or voice)

3.15.7.7 Stuck Trade Recovery

Implement stuck trade escalation (12h, 24h, 48h)

Implement Smart Inbox reminders

Implement Admin Portal alerts

Implement human mediator escalation (A3)

3.15.7.8 Testing

Test full loading workflow (pre-advice → release → scanning → auto-confirm)

Test conditional QC with action plan (hold → complete → verify)

Test re-inspection request and acceptance

Test AIS integration and ETA predictions

Test cold-chain anomaly detection and alerting

Test stuck trade recovery (12h/24h/48h escalations)

Test multi-shipment independent milestones

3.15.8 AI Authority Summary for Phase 5

END OF PART 3.15 — PHASE 5: PHYSICAL EXECUTION & MULTIPARTY TRACKING

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 3.16 — PHASE 6: SETTLEMENT & PAYMENT ORCHESTRATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

3.16.0 Purpose & Design Philosophy

Phase 6: Settlement & Payment Orchestration is the financial culmination of the trade lifecycle. After the shipment reaches its destination and the buyer confirms delivery, the platform orchestrates the settlement of the trade principal from buyer to seller. The platform supports milestone-based partial settlement (e.g., 30% on booking, 40% on departure, 30% on delivery), deferred government fee payment triggers, and automated reconciliation — all while maintaining strict non-custodial principles.

Goal: Ensure that the seller receives payment promptly and securely, government fees are settled, and all transactions are reconciled and auditable.

Key Principles:

Non-custodial — SGTX never holds trade principal. It only instructs PSPs or provides bank reconciliation files.

Milestone-based partial settlement — Pre-approved instalments triggered by specific milestones (e.g., 30% on booking, 40% on departure, 30% on delivery).

PSP Router (A2) — AI selects optimal PSP based on payer country, amount, real-time health, and cost.

Deferred government fee payment — Duties and taxes paid after customs clearance.

Reconciliation Engine (A2, HF Donut) — Automatically matches PSP webhooks and bank statements to settlement instructions.

Voice-enabled approval — Buyer can approve settlement with a single voice command.

Monthly reconciliation statements — Cryptographically signed statements for each tenant.

Full audit trail — Every settlement instruction is signed, Loom-hashed, and immutable.

Multi-shipment support — Each shipment settles independently.

Phase 6 Timeline: Day 41+ (after delivery confirmation)

One-click guarantee: Buyer approves settlement with one click or voice command. Reconciliation is automatic.

AI Integration in Phase 6:

3.16.1 Phase 6 Overview

3.16.1.1 High-Level Workflow

text



1C — Deferred Government Fee Settlement:

Trigger: CUSTOMS_IMPORT milestone confirmed.

Amount = deferred government fees (duties, VAT).

Beneficiary = government agency.

Reference = USTN + "DEFERRED_FEE".

Settlement Instruction Example (JSON):

json

```
{
  "instruction_id": "SI-20260415-001",
  "ustn": "SGTX-EG-26-F3A-1",
  "instruction_type": "TRADE_PRINCIPAL",
  "amount": 30960.00,
  "currency": "USD",
  "payer": {
    "gtid": "SGTX-EG-TRD-001234-5B6C",
    "bank_account": "EG3800100000000000123456",
    "bank_name": "National Bank of Egypt"
  },
  "payee": {
    "gtid": "SGTX-EG-TRD-002139-7F3A",
    "bank_account": "EG38002000000000000789012",
    "bank_name": "Commercial International Bank"
  },
  "reference": "SGTX USTN SGTX-EG-26-F3A-1",
  "terms": "MANDATORY",
  "due_date": "2026-07-06",
  "psp_recommendation": {
    "psp": "Stripe",
    "estimated_fee": 45.00,
    "estimated_settlement_days": 2,
    "explanation": "Recommended for USD cross-border payments"
  },
  "governor_decision_id": "dec-abc123",
  "loom_hash": "sha256:a1b2c3..."
}
```

3.16.3.2 Step 2: PSP Router (A2, LightGBM + Groq)

Description: The PSP Router selects the optimal PSP for each settlement instruction.

Input Features:

Output:

UI Example (Buyer Approval):

text

■ SETTLEMENT APPROVAL — SGTX-EG-26-F3A-1 ■

■ ■

■ Trade: Mekong Fresh Co. → Nile Foods ■

■ Amount: \$30,960.00 (USD) ■

■ Due Date: 2026-07-06 ■

■ ■

■ PSP RECOMMENDATION (A2, LightGBM + Groq) ■

■ ■ Recommended: Stripe ■ ■

■ ■ Estimated Fees: \$45.00 ■ ■

■ ■ Settlement: 2-3 days ■ ■

■ ■ ■ ■

■ ■ Explanation: "We recommend Stripe for this USD cross-border payment. ■ ■

■ ■ Stripe offers the best balance of cost and speed for your location. ■ ■

■ ■ Estimated fees are \$45.00, with settlement in 2-3 days." ■ ■

■ ■ ■ ■

■ ■ Alternatives: ■ ■

■ ■ • Payoneer — Estimated Fees: \$55.00, Settlement: 3-4 days ■ ■

■ ■ • Bank Transfer — Estimated Fees: \$75.00, Settlement: 5-7 days ■ ■

■ ■

■ [Approve Settlement] [Approve with Different PSP] [Reject] ■

■ ■

■ ■ Voice: "Approve settlement for USTN SGTX-1397F3A-..." ■

3. PSP debits buyer's account

↓

4. PSP credits seller's account

↓

5. PSP sends webhook to SGTX

↓

6. SGTX verifies webhook signature

↓

7. SGTX updates settlement status

↓

8. Smart Inbox notifications sent

3.16.3.6 Step 6: Reconciliation Engine (A2, HF Donut)

Description: The Reconciliation Engine continuously matches incoming payment confirmations against open settlement instructions.

Reconciliation Process:

PSP webhook or bank statement line is received.

Extract amount, currency, value date, reference (USTN pattern).

Compare with settlement_instructions that are AUTHORISED.

If match confidence $\geq 95\%$ (amount within tolerance, USTN matches, payer/payee consistent), the settlement is autoreconciled. Status → RECONCILED.

If confidence $< 95\%$ or no match found, a Smart Inbox item is created for the finance team: "Unmatched payment of \$X — please review and assign to USTN."

OpenBanking Integration:

For PSPs that support PSD2 / OpenBanking APIs, the settlement-service pulls SGTX's bank statement periodically.

HF Donut extracts each transaction, matches against open settlement_instructions using amount, reference, date.

If confidence $\geq 95\%$, auto-reconciled. Unmatched items trigger Smart Inbox alert for manual review.

Reconciliation Example:

json

```
{
  "transaction": {
    "amount": 30960.00,
    "currency": "USD",
    "value_date": "2026-07-06",
    "reference": "SGTX USTN SGTX-EG-26-F3A-1"
  },
}
```


Implement milestone-based pre-approval

Implement Governor validation (permissions, FeeLock, disputes)

3.16.6.4 PSP Processing

Implement PSP API integration (Stripe, Payoneer, etc.)

Implement webhook signature verification

Implement idempotency key handling

Implement retry logic (exponential backoff)

3.16.6.5 Reconciliation Engine

Implement reconciliation engine (A2, HF Donut)

Implement PSP webhook reconciliation

Implement OpenBanking integration (PSD2)

Implement auto-reconciliation (confidence $\geq 95\%$)

Implement manual reconciliation queue (confidence $< 95\%$)

3.16.6.6 Monthly Reconciliation Statement

Implement monthly statement generation (cron job)

Implement statement signing (Ed25519)

Implement statement download (PDF, CSV)

Implement statement verification

3.16.6.7 Error Handling & Retry Policies

Implement retry policies (exponential backoff)

Implement PSP webhook failure handling

Implement bank settlement API retries

Implement Smart Inbox alerts for failures

3.16.6.8 Testing

Test full settlement workflow (delivery → instruction → approval → PSP → webhook → reconciliation)

Test milestone-based partial settlement (3 instalments)

Test deferred government fee trigger

Test PSP fallback (primary fails → secondary PSP)

Test reconciliation (auto-reconciliation and manual queue)

Test monthly reconciliation statement

Test voice approval

3.16.7 AI Authority Summary for Phase 6

END OF PART 3.16 — PHASE 6: SETTLEMENT & PAYMENT ORCHESTRATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

3.17 Phase 7 — Distressed Cargo Resolution (Optional)

3.17.1 Phase 7 Goal

When cargo becomes distressed (physically damaged, abandoned, delayed beyond shelf life, or subject to demurrage), the platform provides a structured resolution workflow. The distressed cargo owner can declare distress, receive AI-assessed condition and pricing, and resolve the lot via sale, compliant disposal, or insurance claim.

3.17.2 Proactive Demurrage & Delay Alert (Pre-Distress) — Advisory

Trigger: System monitors port, customs, and container free-time data. When demurrage risk is detected (e.g., free time expires in 48 hours), a Smart Inbox alert (priority 80) is sent.

Example alert:

text

■ Demurrage charges will start in 48h. Estimated charge \$50/day.

Consider releasing cargo or declaring it distressed.

[View Shipment] [Declare Distressed]

3.17.3 Declaration of Distressed Cargo — One Click

Seller action: From Distressed Cargo & Notifications tab, click "Declare Distressed" — one click opens the declaration form.

Declaration form:

Select affected pallets/containers (from packing plan, multi-select).

Describe condition (free text, min 10 chars).

Upload photos (optional but recommended for AI assessment).

Choose distress reason: Damage, Quality deterioration, Shelf life expired, Abandonment, Demurrage, Other.

Voice command (A1): "Pallet LEM003 has mould, mark as distressed."

3.17.4 AI Condition Assessment — Automatic (A2, HF ViT)

Trigger: When photos are uploaded, the Condition Assessment Agent runs automatically.

HF ViT analyses photos — detects mould, crushing, colour change, moisture stains, insect presence.

Produces a condition score (0-100) per pallet.

Estimates remaining shelf life (if perishable) using cold-chain data.

Output example:

json

```
{  
  "condition_score": 35,  
  "tags": ["mould_detected_92%", "crushed_cartons_78%"],  
  "remaining_shelf_life_days": 5,  
  "confidence": 0.88  
}
```

3.17.5 Dynamic AI Pricing Engine — Automatic (A2, XGBoost)

Input features:

Original contract value of affected goods.

AI condition score.

Remaining shelf life.

Current commodity price index (FAO, scraped).

Destination demand.

Buyer urgency.

Output: Suggested price range and recommended listing price. Groq (A1) generates business explanation.

3.17.6 Distressed Cargo Triage Dashboard — Three Paths (One Click Each)

3.17.7 "Check Buyers" Advisory — One Click (No Notifications Sent)

Seller action: Before initiating any outreach, seller clicks "Check Buyers" — one click.

Modal displays eligible saved contacts, ranked by:

Past distressed purchase success

Trust score

Location proximity

Response time

Ranking composite score (A2, LightGBM). Seller selects buyers manually — no notifications are sent by clicking "Check Buyers".

3.17.8 Partial Distress & USTN Splitting (MicroUSTN) — Automatic

If only part of a shipment is distressed, the system automatically generates a microUSTN for the distressed pallets.

Example:

Original USTN: SGTX-EG-26-F3A-1

MicroUSTN: SGTX-1397F3A-456ABC-20260420150000-D4E5F6G7

3.17.9 Accelerated Outreach Mode — With Privacy Notice Opt-In

Seller action: After selecting buyers, seller clicks "Start Accelerated Outreach" — one click.

Mandatory privacy notice modal:

text

You are about to initiate Accelerated Outreach for the distressed lot [description].

The following contacts will be notified simultaneously: [list of names].

The notification will include: the product, quantity, condition score, asking price, and your company name.

Recipients will know your identity.

Do you want to proceed?

[Yes, start outreach] [No, go back]

Outreach parameters:

Outreach window (2-24 hours) — default 6h.

Floor price (hard minimum, never disclosed to recipients) — default 70% of asking price.

Notification method — Smart Inbox + email (co-branded: "Seller via SGTX").

3.17.10 Microcontract & Separate SGTX Fee — One Click to Lock

When offer is accepted, system automatically generates a microcontract with:

microUSTN

Buyer and seller (distressed cargo buyer)

Agreed price

Special distressed terms

SGTX fee for distressed sale:

Fee rate = $1.5\% \times \text{distressed country factor}$ (from `jurisdictions.distressed_country_factor`)

Example: UAE factor 0.85 → effective fee = $1.5\% \times 0.85 = 1.275\%$

Fee paid by seller upfront before microcontract lock.

3.17.11 AI-Insurer Integration & Claim Support (Path C)

Seller action: From Triage Dashboard, click "File Insurance Claim" — one click.

System compiles evidence package:

Original contract, invoice, packing list.

All milestone logs.

Sensor data (temperature excursions).

Photos with AI condition assessment.

Distressed sale agreement (if sold) or market valuation.

AI-generated claim narrative (A1, Groq).

One-click: Click "Download Package" — signed PDF ready for submission to insurer.

3.18 Phase 8 — Dispute Resolution & Arbitration

3.18.1 Phase 8 Goal

If a party believes that the executed shipment, payment, financing, or distressed sale has not met the contractual terms, they can file a dispute. The platform provides a structured, AI-assisted mediation process, with the option to escalate to international arbitration.

3.18.2 Filing a Dispute — One Click (or Voice)

Disputing party action: From TCC, click "Raise Dispute" — one click opens the filing form.

Filing form:

Select category (dropdown).

Description (free text, min 10 chars).

Upload evidence (optional).

Specify remedy sought.

Affected portion (if partial).

Voice-initiated filing (A1): "File a dispute for USTN SGTX-EG-... The oranges arrived with mould. I am claiming \$2,000." — pre-fills form.

Category dropdown:

3.18.3 Evidence Autocompiler — Automatic, One Click to View

Trigger: Automatically when dispute is filed.

Evidence package contents:

Trade & contract data

Shipment milestones

IoT sensor data

QC report (overrides with original AI confidence and inspector's reason)

Laboratory results

Broker certification log

All documents with SHA256 hashes

Communications

AI decisions

Payments

Causal analysis (if triggered)

Package is Loom-hashed and stored. Verification token generated for external auditors.

One-click: Both parties receive Smart Inbox item: "Evidence package compiled for dispute DSP-xxx. [View Package]"

3.18.4 Causal Inference Engine — Automatic (Add-on 3, A2/A3)

Trigger: Asynchronously when a dispute is filed.

Output: JSON with root causes and contribution percentages. Groq generates plain-language summary added to evidence package.

Example output for delay dispute:

json

```
{
  "root_causes": [
    { "factor": "port_strike", "contribution": 0.54, "description": "Port closure at Alexandria added 54% of total delay." },
    { "factor": "carrier_rerouting", "contribution": 0.32, "description": "Carrier rerouted due to weather, adding 32%." },
    { "factor": "customs_hold", "contribution": 0.14, "description": "Missing phytosanitary certificate caused 14% delay." }
  ],
  "groq_summary": "The delay was caused primarily by the port strike (54%), with the carrier's rerouting as a secondary factor (32%)."
}
```

3.18.5 Mediation Log & Multi-Language Support — One Click per Message

Interface: Structured mediation log accessible from the Dispute tab.

Features:

Text messages — type and click "Send" (one click). Groq translates into each party's preferred language.

Structured offers — click "Make Offer" → enter amount and conditions → send (one click).

Accept / Reject / Counter buttons — one click each.

Attach additional evidence — one click to upload file.

Invite Third-Party Expert — see below.

Sentiment analysis (A2): If hostility detected, suggests cooling-off period (24h pause) — one click to accept.

3.18.6 Third-Party Expert Invitation (Improvement #8) — One Click

Any party can click "Invite Third-Party Expert" — one click.

Workflow:

Click "Invite Third-Party Expert" — modal opens.

Select expert type: Surveyor, Laboratory, Arbitrator, Legal counsel.

Choose from pre-approved list or enter specific GTID.

Add message to expert (optional).

Click "Send Invitation" — one click.

Expert receives secure, read-only link to evidence package.

Expert posts non-binding opinion — one click.

3.18.7 Predictive Dispute Outcome — Automatic (A2, XGBoost)

Trigger: After evidence is compiled, system runs predictive model.

Input features: Dispute type, commodity, incoterm, ports, party trust scores, dispute history, contradictory evidence, contractual strength, QC override patterns.

Output: Estimated probability of filing party winning, predicted damage amount (range), confidence interval. Groq generates neutral plain-language summary.

3.18.8 AI-Generated Settlement Proposal — One Click to Accept

Trigger: When mediation reaches an impasse, any party can click "Request AI Settlement Proposal" — one click.

Proposal generation (A1, Groq + A2 XGBoost):

Based on predictive outcome, contract clauses, undisputed portion, industry standards.

Includes proposed amount, conditions, rationale, confidence, acceptance deadline.

Example proposal:

json

```
{  
  "proposal_id": "SP-20260415-001",  
  "proposed_settlement": {
```


"type": "PARTIAL_REFUND",

"amount": 800.00,

"currency": "USD",

"conditions": ["Buyer releases seller from further claims", "Both parties bear own costs"]

},

"rationale": "The temperature log shows no deviation during transit, indicating the damage likely occurred post-delivery. However, the buyer's photos show mould presence, suggesting a pre-existing quality issue. A 4% refund is consistent with similar cases.",

"confidence": 0.87

}

One-click: Both parties see proposal. Each can click "Accept" or "Reject" — one click each. If both accept, settlement addendum signed automatically.

3.18.9 QC Dispute Fast-Track — Automatic Flagging

If the dispute involves a QC report, the evidence package automatically includes a dedicated qc_overrides section listing every AI finding the inspector overrode, with:

Original AI detection (defect type, confidence %)

Inspector's classification after override

Inspector's mandatory reason (≥10 characters)

Timestamp and photo hashes

3.18.10 Escalation to International Arbitration — One Click to Prepare Case

If mediation fails, any party clicks "Escalate to Arbitration" — one click.

Automated Case Filing (A2, A1):

Retrieves arbitration request form template (pre-obtained from ICC, DIFC-LCIA, CRCICA, etc.).

Autofills with dispute details, parties, USTN, contract references.

Groq drafts claim narrative in required language.

Compiled package (form + evidence + Loom hash) produced as PDF.

Filing party's legal counsel reviews and submits (outside platform).

3.19 Trade Digital Twin (Scenario Simulation)

3.19.1 Purpose

Simulate external shocks on active and planned trades — tariff changes, currency shocks, regulatory changes, logistics disruptions, financing impacts.

Advisory only — never blocks or executes trades.

3.19.2 Scenario Types

3.19.3 Output Example

json

{

"scenario": "Tariff +15% on citrus (HS 0805)",

```

"affected_trades": ["USTN-EG-001", "USTN-EG-002"],
"impact_forecast": {
  "total_landed_cost_increase_percent": 12.3,
  "demand_elasticity_estimate": -0.4,
  "recommendation": "Accelerate shipments before effective date of 2026-09-01"
},
"confidence": 0.87,
"disclaimer": "Advisory only -- not a guarantee."
}

```

3.19.4 One-Click Actions

Run Scenario (1 click) — select scenario type, parameters, execute.

Apply Recommendation (1 click) — e.g., pre-fill hedging instrument, adjust shipping route.

3.20 Database Schema Additions (Part 3)

sql

-- USTN verification tokens

```

CREATE TABLE loom_verification_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT NOT NULL REFERENCES shipments(ustn),
  token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '90 days',
  created_by UUID REFERENCES employees(id),
  revoked BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Contract shipments (multi-shipment)

```

CREATE TABLE contract_shipments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
  shipment_number INTEGER NOT NULL,
  scheduled_delivery_date DATE NOT NULL,
  port_of_discharge TEXT NOT NULL,
  containers_count INTEGER NOT NULL,
  total_price DECIMAL(20,4) NOT NULL,
  sgtx_fee_amount DECIMAL(20,4),
  fee_lock_id UUID,

```

```

ustn TEXT UNIQUE,
status TEXT DEFAULT 'SCHEDULED', -- SCHEDULED, LOCKED, SHIPPED, CANCELLED,
FAILED_PAYMENT
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Milestone payment schedules (Improvement #9)
CREATE TABLE milestone_payment_schedules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
milestone_name TEXT NOT NULL, -- 'BOOKED', 'DEPARTED', 'DELIVERED'
percentage DECIMAL(5,2) NOT NULL,
amount DECIMAL(20,4) NOT NULL,
pre_approved BOOLEAN DEFAULT false,
paid BOOLEAN DEFAULT false,
paid_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Ship quote requests (Mode C)
CREATE TABLE ship_quote_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
seller_gtid TEXT NOT NULL,
base_service_type TEXT NOT NULL, -- 'OCEAN_FREIGHT', 'AIR_FREIGHT'
origin_port TEXT NOT NULL,
destination_port TEXT NOT NULL,
container_details JSONB NOT NULL,
add_on_services TEXT[], -- 'TRUCKING', 'CUSTOMS_BROKER', 'INSURANCE'
target_lines TEXT[], -- GTIDs of shipping lines
status TEXT DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE ship_quotes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
request_id UUID NOT NULL REFERENCES ship_quote_requests(id) ON DELETE CASCADE,
shipper_line_gtid TEXT NOT NULL,
base_fee DECIMAL(20,4) NOT NULL,

```

```

add_on_fees JSONB, -- {"TRUCKING": 300, "CUSTOMS_BROKER": 150}
total_fee DECIMAL(20,4) NOT NULL,
validity_period_hours INTEGER DEFAULT 48,
selected BOOLEAN DEFAULT false,
submitted_at TIMESTAMPTZ DEFAULT NOW()
);

-- Clarification requests (Improvement #4)
CREATE TABLE clarification_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
quotation_id UUID REFERENCES service_quotations(id) ON DELETE CASCADE,
requester_gtid TEXT NOT NULL,
questions JSONB NOT NULL,
answers JSONB,
resolved BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ
);

-- Conditional QC (Improvement #5)
ALTER TABLE qc_jobs ADD COLUMN conditional_pass_status TEXT DEFAULT NULL;
ALTER TABLE qc_jobs ADD COLUMN action_plan TEXT;
ALTER TABLE qc_jobs ADD COLUMN action_plan_deadline TIMESTAMPTZ;
ALTER TABLE qc_jobs ADD COLUMN action_plan_completed_at TIMESTAMPTZ;

-- Re-inspection requests
CREATE TABLE re_inspection_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
original_ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
requested_by_gtid TEXT NOT NULL,
reason TEXT NOT NULL,
urgency TEXT DEFAULT 'NORMAL', -- 'NORMAL', 'URGENT'
deadline TIMESTAMPTZ,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACCEPTED', 'DECLINED', 'COMPLETED'
new_qc_job_id UUID REFERENCES qc_jobs(id),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Deferred payment

```

```

ALTER TABLE fee_payment_requests ADD COLUMN deferred BOOLEAN DEFAULT false;
ALTER TABLE fee_payment_requests ADD COLUMN deferred_status TEXT DEFAULT
'GUARANTEE_HELD';
ALTER TABLE fee_payment_requests ADD COLUMN auto_charge_authorised BOOLEAN DEFAULT
false;
ALTER TABLE fee_payment_requests ADD COLUMN expiry_action_taken TEXT;
-- Third-party experts (Improvement #8)
CREATE TABLE dispute_experts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
dispute_id UUID REFERENCES disputes(id) ON DELETE CASCADE,
expert_gtid TEXT,
expert_type TEXT NOT NULL, -- 'surveyor', 'laboratory', 'arbitrator', 'legal'
invitation_token TEXT UNIQUE,
opinion TEXT,
posted_at TIMESTAMPTZ,
status TEXT DEFAULT 'INVITED' -- 'INVITED', 'ACCEPTED', 'DECLINED', 'POSTED'
);
-- Predictive insights (Trade Memory Layer)
CREATE TABLE predictive_insights (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
insight_type TEXT NOT NULL,
target_entity TEXT, -- USTN, GTID, or NULL for general
probability DECIMAL(5,2),
confidence_interval JSONB,
recommendation TEXT,
payload JSONB,
valid_until TIMESTAMPTZ,
acknowledged BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Trade Memory Events
CREATE TABLE trade_memory_events (
id BIGSERIAL,
event_type TEXT NOT NULL,
category TEXT NOT NULL, -- 'DELAY', 'DISPUTE', 'FINANCING', etc.
anonymous_entity_id TEXT,

```

```

hs_chapter CHAR(2),
origin_country CHAR(2),
destination_country CHAR(2),
incoterm TEXT,
volume_band TEXT, -- 'SMALL', 'MEDIUM', 'LARGE'
value_band TEXT,
delay_days INTEGER,
dispute_category TEXT,
resolution_days INTEGER,
payload JSONB,
occurred_week DATE,
created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (occurred_week);
SELECT create_hypertable('trade_memory_events', 'occurred_week');
-- Indexes
CREATE INDEX idx_ustn_verification_tokens_token ON loom_verification_tokens(token);
CREATE INDEX idx_contract_shipments_contract ON contract_shipments(contract_id);
CREATE INDEX idx_contract_shipments_ustn ON contract_shipments(ustn);
CREATE INDEX idx_milestone_payment_schedules_contract ON
milestone_payment_schedules(contract_id);
CREATE INDEX idx_ship_quote_requests_ustn ON ship_quote_requests(ustn);
CREATE INDEX idx_ship_quotes_request ON ship_quotes(request_id);
CREATE INDEX idx_clarification_requests_quotation ON clarification_requests(quotation_id);
CREATE INDEX idx_re_inspection_requests_ustn ON re_inspection_requests(original_ustn);
CREATE INDEX idx_dispute_experts_dispute ON dispute_experts(dispute_id);
CREATE INDEX idx_predictive_insights_type ON predictive_insights(insight_type);
CREATE INDEX idx_trade_memory_events_week ON trade_memory_events(occurred_week);
CREATE INDEX idx_trade_memory_events_category ON trade_memory_events(category);

```

3.21 Governance Gates (End-to-End Workflow)

3.22 Implementation Checklist (Part 3)

3.22.1 USTN Infrastructure (Updated)

Create ustn_counters table with atomic increment function.

Implement USTN generation algorithm (Rust) with per-year/per-trader counters.

Add trader identifier extraction from GTID.

Create shipments table with USTN as primary key (or unique indexed field).

Build USTN resolution service (/v1/ustn/{ustn}).

Add USTN validation in all document uploads.

Generate USTN QR codes with verifiable credentials.

Implement USTN for multi-shipment contracts.

Implement USTN for distressed micro-contracts.

3.22.2 Phase 1 — Trade Initiation

Build seller selection with GTID autocomplete and saved contacts.

Implement incoterm selection with auto-configuration.

Build structured container form with cloning, bulk edit.

Implement two-way smart product/HS code input.

Integrate Product Form Agent (A2) for dynamic specifications.

Build multi-shipment schedule builder.

Implement AI Container Advisor (A1, advisory).

Add marketplace attribution detection and dispute workflow.

Implement draft auto-save (every 30s) and recovery.

Extend Governor policies for Phase 1 validation.

Build Governor pre-screen with plain-language Decision Panel.

3.22.3 Phase 2 — Seller Quote & Logistics

Build loading origin selection with geocoding.

Implement EXW price lock with:

Live market chart

AI fair price badge

Dynamic unit conversion

Weight unit toggle

Price deviation justification

Post-lock price watch

Five add-ons (volume-tiered, dynamic currency, historical comparison, lock & hold, admin floor/ceiling)

Build packing module with non-uniform layer stacking.

Implement Mode A — Manual Entry with incoterm-based filtering.

Implement Mode B — RFQ to LSPs with clarification requests.

Implement Mode C — Direct to SHIP with add-on services.

Build unified "Compare Quotes" panel.

Add alternative delivery ports with AI suggestions.

Implement seller response to multi-shipment schedule.

Add SGTX fee calculation (1.5%).

Build "Submit Quote" endpoint with Governor validation.

3.22.4 Phase 3 — Contracting & Fee Collection

Build quote comparison table with Accept/Negotiate/Amend buttons.

Implement enhanced negotiation panel with:

Partial acceptance (Improvement #2a)

Counter-offer with reason (Improvement #2b)

Deadline extension request (Improvement #2c)

Visual diff for amendments

Implement mutual confirmation endpoint.

Integrate Clause Forge (A2) for contract generation.

Implement own-contract upload with HF Donut extraction.

Add logistics addendum signing workflow.

Implement multi-shipment schedule modification after lock (Improvement #3).

Build PSP payment flow with deferred fee option.

Implement late fee calculation cron job.

Add container release acknowledgment workflow.

Implement digital signature collection (ZITADEL passkey, QES).

3.22.5 Phase 4 — Trade Finance

Implement financing request endpoint with credit intelligence.

Build financier preferences table and matching engine.

Implement full disclosure page (trade details, documents, historical performance).

Add encrypted bid submission (client-side encryption).

Implement co-financing acceptance.

Add DeFi PlainLanguage Risk Summary modal.

Integrate PSP split for financing fee deduction (0.25%).

Implement repayment monitoring (webhooks, OpenBanking, on-chain).

Add DeFi Protocol Risk Oracle.

Implement stablecoin de-peg monitor.

Add liquidation early warning (LSTM).

3.22.6 Phase 5 — Physical Execution

Implement pre-advice webhook to terminals.

Extend mobile app to support non-uniform layer instructions.

Add batch scan mode for pallet loading.

Implement QC conditional pass with action plan.

Implement re-inspection request and acceptance workflow.

Integrate AIS and cold-chain LSTM models.

Add delivery confirmation endpoint.

Extend stuck trade recovery service with SLA-based escalations.

3.22.7 Phase 6 — Settlement

Implement settlement instruction generation (full and milestone-based).

Build PSP router with LightGBM + Groq explanation.

Add voice approval for settlement.

Implement deferred government fee payment trigger.

Enhance reconciliation engine with HF Donut.

Generate monthly reconciliation statements (cryptographically signed).

3.22.8 Phase 7 — Distressed Cargo

Implement distressed declaration with photo upload and AI condition assessment.

Build dynamic pricing engine (XGBoost).

Create triage dashboard UI (three paths).

Add "Check Buyers" advisory.

Implement accelerated outreach with privacy notice.

Build microcontract creation with separate SGTX fee.

Add evidence compiler for insurance claim path.

3.22.9 Phase 8 — Dispute Resolution

Implement dispute filing endpoint (voice and structured).

Build evidence autocompiler.

Integrate causal inference engine.

Implement mediation log UI with Groq translation.

Add third-party expert invitation workflow.

Build AI settlement proposal generator.

Implement FeeLock freeze on dispute filing.

Add SGTX fee dispute workflow with escalation to multisig.

Build arbitration case preparation.

3.22.10 Trade Digital Twin

Implement digital-twin service (Rust + Python).

Build scenario simulation endpoint.

Integrate with TCC as "Scenario Analysis" button.

Add "Apply Recommendation" one-click action.

3.23 AI Authority Summary for Part 3

END OF PART 3 — UNIVERSAL SHIPMENT TRACKING NUMBER (USTN) & END-TO-END TRADE WORKFLOW (v11.0 FULLY EXPANDED)

PART 4 — DYNAMIC PRODUCT-AWARE REQUEST FORM

4.0 Purpose & Design Philosophy

The Dynamic Product-Aware Request Form is the entry point for every trade on the SGTx platform. It uses AI-driven dynamic schema generation to adapt the form in real time based on the selected product, origin, destination, port, and any special requirements. The buyer never needs to research regulations manually — the system pre-populates the required fields, documents, and compliance requirements. Moreover, the form dynamically generates the exact fields needed for the packing list — such as pallet size, number of pallets, cartons per pallet, layer arrangement, and packaging details — tailored to the specific commodity and its handling requirements.

The form is buyer-facing (Trader Portal, Buyer mode) but the data is stored and reused throughout — for seller's packing, customs declaration, invoice generation, and financing. No data re-entry across phases.

4.0.1 Core Principles

One source of truth — All structured container and commodity data is stored in `trade_requests.parsed_specs` JSONB and reused throughout the workflow. No data duplication.

AI dynamically builds the form — Required documents, commodity-specific fields, lab tests, special conditions, acceptance criteria, and packing fields are added automatically based on the product and route.

AI never suggests counterparties — The form never recommends "you might also want to export to X" or "other buyers used Y." All assistance is limited to data entry optimisation and compliance.

Auto-detection of special procedures — If the destination country or port requires cold treatment, fumigation, or pre-cooling, the system adds the necessary fields and document requirements.

Dynamic packing specification — For each commodity, the system asks for pallet dimensions (EUR, ISO, custom), number of pallets, cartons per pallet, layers, stacking height, and packaging type — all derived from product-specific defaults and RIA data.

Two-way smart product/HS code resolution — Entering a product name auto-fills the HS code; entering an HS code auto-fills the product name.

Express Mode (optional) — Users who prefer natural language can describe their trade in plain English; AI parses and extracts structured data.

Multi-commodity, multi-container — Supports unlimited containers and unlimited commodities per container.

Multi-shipment ready — Optional schedule builder for future deliveries.

Draft auto-save — Every 30 seconds, resumes via Smart Inbox.

Incoterm selection — Buyer selects incoterm at the start, which determines logistics responsibilities later.

Commercial settlement configuration — Payment method, terms, financing, settlement structure, and flexibility.

Trade Request Readiness — Before submission, the system calculates a readiness score. This is a non-blocking advisory metric that flags missing or incomplete items.

Trade Criticality — The buyer indicates the operational importance of the trade: Routine, Priority, or Critical. This drives workflow routing, approval urgency, and alerting.

Dynamic transport mode configuration — Transport mode selection dynamically loads appropriate equipment types (containers for ocean, ULDs for air, trailers for truck, wagons for rail).

4.0.2 What This Form Is NOT

4.0.3 Complete Form Structure Overview

4.0.4 Database Design — Critical Correction

Issue: The original design stored quantity at the trade header level. This is incorrect for multi-commodity trades.

Example Problem:

Container 1: Oranges (100 MT)

Container 2: Potatoes (50 MT)

Both share the same requested_quantity at header level → INCORRECT.

Solution: Quantity is stored at the commodity level.

Corrected Data Model:

sql

-- Trade Request Commodities (NEW — replaces header-level quantity)

```
CREATE TABLE trade_request_commodities (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  container_id UUID NOT NULL,  
  commodity_type TEXT NOT NULL,  
  product_name TEXT NOT NULL,  
  hs_code CHAR(6) NOT NULL,  
  -- Quantity & Tolerance (per commodity)  
  requested_quantity DECIMAL(20,4) NOT NULL,  
  requested_quantity_unit TEXT NOT NULL,  
  quantity_tolerance DECIMAL(5,2) DEFAULT 5.0,  
  -- Acceptance Criteria (per commodity)  
  acceptance_criteria_matrix JSONB,  
  -- Packaging (per commodity)  
  packaging_type TEXT,  
  pallet_count INTEGER,  
  net_weight_per_unit DECIMAL(10,2),  
  gross_weight_per_unit DECIMAL(10,2),  
  -- Special requirements (per commodity)  
  partial_shipment_allowed BOOLEAN DEFAULT true,  
  transshipment_allowed BOOLEAN DEFAULT true,  
  inspection_requirements TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

4.0.5 AI Authority Summary for Part 4.0

4.0.6 Key Differences from Original Blueprint

4.0.7 Implementation Priority Matrix

4.0.8 Next Steps

After confirming Part 4.0, the following detailed sections will be provided:

4.1 — Regulatory Intelligence Agent (RIA) — Data sources, dynamic rules, and AI integration

4.2 — Incoterm Selection — Complete incoterm engine with auto-configuration

4.3 — Product Form Agent — Dynamic JSON schema generation with full examples

4.4 — Structured Container & Commodity Entry — All fields with UI examples

4.5 — Documentation Requirements — One-source-of-truth document management

4.6 — Special Trade Instructions — Free-text section

4.7 — Transport & Logistics — Dynamic equipment loading by transport mode

4.8 — Insurance Requirements — Simplified request-stage insurance

4.9 — Commercial Settlement Requirements — Complete commercial foundation

4.10 — Trade Request Readiness — Advisory completeness score

4.11 — Trade Criticality — Workflow routing and alerting

4.12 — Multi-Shipment Request — Schedule builder

4.13 — AI Container Advisor — Advisory container optimisation

4.14 — Draft Auto-Save & Recovery — Resilience and recovery

4.15 — Governor PreScreen & Submission — Complete validation matrix

4.16 — Database Schema Additions — Complete DDL

4.17 — Implementation Checklist — Complete implementation guide

4.18 — AI Authority Summary — Complete AI mapping

END OF PART 4.0 — DYNAMIC PRODUCT-AWARE REQUEST FORM (OVERVIEW & DESIGN PHILOSOPHY)

PART 4.1 — REGULATORY INTELLIGENCE AGENT (RIA)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.1.0 Purpose & Design Philosophy

The Regulatory Intelligence Agent (RIA) is the brain behind SGTX's dynamic compliance and product specification engine. It is a continuously running, self-updating knowledge graph that scrapes, parses, and structures regulatory data from over 300 official sources worldwide. The RIA ensures that every trade request is automatically configured with the correct product specifications, required documents, special procedures, lab tests, and acceptance criteria — without requiring the buyer to research regulations manually.

Key Principles:

Zero human research — Buyers never need to look up MRLs, treatment requirements, or document mandates.

Real-time updates — Regulatory changes are reflected within 6 hours of publication.

■ ■ PARSE & EXTRACT LAYER (A2, HF NLP) ■ ■


```
}
```

4.1.3 RIA Maintained Tables

4.1.3.1 Core Tables

```
sql
```

```
-- Jurisdictions (country matrix) — Updated by RIA
```

```
CREATE TABLE jurisdictions (  
  code TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  sanctions_level TEXT NOT NULL, -- 'FULL', 'STANDARD', 'LIMITED', 'RESTRICTED', 'BLOCKED'  
  regulatory_body TEXT,  
  defi_allowed BOOLEAN DEFAULT false,  
  defi_restrictions JSONB,  
  stablecoins_allowed TEXT[],  
  private_financier_allowed BOOLEAN DEFAULT false,  
  distressed_sale_allowed TEXT, -- 'YES', 'CONDITIONAL', 'NO'  
  distressed_country_factor DECIMAL(5,4) DEFAULT 1.0,  
  distressed_sgtx_fee_factor DECIMAL(5,4) DEFAULT 1.0,  
  kyc_tier_required INTEGER DEFAULT 2,  
  deferred_fees_allowed TEXT[], -- ['IMPORT_DUTIES', 'VAT']  
  registry_api_endpoint TEXT,  
  customs_api_endpoint TEXT,  
  cbdc_status TEXT DEFAULT 'NONE',  
  psp_partners JSONB,  
  reporting_requirements JSONB,  
  last_updated TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- Country Physical Document Requirements
```

```
CREATE TABLE country_physical_document_requirements (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  country_code TEXT NOT NULL REFERENCES jurisdictions(code),  
  document_type TEXT NOT NULL,  
  requires_original BOOLEAN DEFAULT false,  
  requires_translation BOOLEAN DEFAULT false,  
  requires_legalisation BOOLEAN DEFAULT false,  
  physical_handling_service_recommended BOOLEAN DEFAULT false,
```



```

last_updated TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(country_code, document_type)
);

-- Country MRL (Maximum Residue Limits)
CREATE TABLE country_mrl (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
destination_country CHAR(2) NOT NULL,
analyte TEXT NOT NULL,
mrl_mg_per_kg DECIMAL(10,4),
unit TEXT DEFAULT 'mg/kg',
limit_operator TEXT DEFAULT '<=', -- '<=', '=', '>=', etc.
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Treatment Requirements
CREATE TABLE treatment_requirements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_code TEXT,
treatment_type TEXT NOT NULL, -- 'cold_treatment', 'fumigation', 'irradiation', 'pre_cooling'
temperature_c DECIMAL(5,2),
duration_days INTEGER,
certificate_required BOOLEAN DEFAULT true,
accepted_facilities TEXT[],
mandate_effective_date DATE,
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Port Special Rules
CREATE TABLE port_special_rules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
port_code TEXT NOT NULL REFERENCES ports(unlocode) ON DELETE CASCADE,

```

```
rule_type TEXT NOT NULL, -- 'pre_cooling_certificate', 'cold_treatment', 'fumigation', 'document_check'
description TEXT,
mandatory BOOLEAN DEFAULT true,
updated_at TIMESTAMPTZ DEFAULT NOW()
```

```
);
```

```
-- Commodity Packing Defaults
```

```
CREATE TABLE commodity_packing_defaults (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
commodity_type TEXT NOT NULL,
default_pallet_type TEXT DEFAULT 'EUR',
default_cartons_per_pallet INTEGER DEFAULT 40,
default_net_weight_per_carton_kg DECIMAL(10,2) DEFAULT 10,
default_stacking_layers INTEGER DEFAULT 5,
max_pallets_per_container_40ft INTEGER DEFAULT 22,
carton_dimensions_mm JSONB,
updated_at TIMESTAMPTZ DEFAULT NOW()
```

```
);
```

```
-- Commodity Dynamic Schemas Cache
```

```
CREATE TABLE commodity_dynamic_schemas_cache (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_code TEXT,
schema_json JSONB NOT NULL, -- Full Product Form Agent schema
generated_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '7 days',
product_form_agent_version TEXT,
UNIQUE(hs_code, origin_country, destination_country, port_code)
```

```
);
```

```
-- NEW: Acceptance Criteria Templates
```

```
CREATE TABLE acceptance_criteria_templates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
```

```

parameter TEXT NOT NULL,
default_limit TEXT NOT NULL,
default_unit TEXT NOT NULL,
is_mandatory BOOLEAN DEFAULT true,
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- NEW: Documentation Triggers (One source of truth)
CREATE TABLE documentation_triggers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
document_type TEXT NOT NULL,
trigger_type TEXT NOT NULL, -- 'SHIPMENT', 'SETTLEMENT', 'CUSTOMS', 'FINANCING'
is_mandatory BOOLEAN DEFAULT false,
jurisdiction_code TEXT REFERENCES jurisdictions(code),
hs_code CHAR(6),
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

4.1.3.2 Vector Database (pgvector)

sql

-- Regulations vector store

```

CREATE TABLE regulation_vectors (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
source TEXT NOT NULL,
document_type TEXT NOT NULL,
content TEXT NOT NULL,
embedding vector(384), -- sentence-transformers/all-MiniLM-L6-v2
metadata JSONB,
scraped_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Product specification vectors

```

CREATE TABLE product_spec_vectors (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
product_name TEXT NOT NULL,

```

```

description TEXT,
embedding vector(384),
metadata JSONB,
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Create vector indexes for similarity search
CREATE INDEX idx_regulation_vectors_embedding ON regulation_vectors
USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
CREATE INDEX idx_product_spec_vectors_embedding ON product_spec_vectors
USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

4.1.4 How RIA Drives the Dynamic Form

4.1.4.1 Product Selection Flow

When a buyer selects a product (by HS code or product name), the Product Form Agent queries the RIA in real time:

rust

```

// Pseudo-code: Product Form Agent querying RIA
async fn get_product_schema(
  hs_code: &str,
  origin: &str,
  destination: &str,
  port: &str
) -> Result<ProductSchema> {
  // 1. Check cache
  if let Some(cached) = get_cached_schema(hs_code, origin, destination, port).await? {
    return Ok(cached);
  }
  // 2. Query treatment requirements
  let treatments = get_treatments(hs_code, origin, destination).await?;
  // 3. Query MRLs
  let mrls = get_mrls(hs_code, destination).await?;
  // 4. Query packing defaults
  let packing = get_packing_defaults(hs_code).await?;
  // 5. Query acceptance criteria
  let acceptance_criteria = get_acceptance_criteria(hs_code).await?;
  // 6. Query document requirements with triggers
  let documents = get_document_requirements(hs_code, destination, origin).await?;
}

```



```
groq_summary TEXT, -- A1-generated summary
loom_hash TEXT,
detected_at TIMESTAMPTZ DEFAULT NOW(),
applied_at TIMESTAMPTZ
);
-- Indexes
```

```
CREATE INDEX idx_integration_connector_logs_name ON integration_connector_logs(connector_name);
CREATE INDEX idx_integration_connector_logs_created ON integration_connector_logs(created_at
DESC);
CREATE INDEX idx_ria_change_log_type ON ria_change_log(change_type);
CREATE INDEX idx_ria_change_log_detected ON ria_change_log(detected_at DESC);
```

4.1.10 Implementation Checklist (Part 4.1)

4.1.10.1 Scraper Layer

- Implement Sanctions Scraper (UN, OFAC, EU)
- Implement Food Safety Scraper (RASFF, FDA, SFDA)
- Implement Ports Scraper (port authorities, UN/LOCODE)
- Implement Market Data Scraper (FAO, USDA, World Bank)
- Implement Trade Stats Scraper (UN Comtrade)
- Implement Egyptian-specific scrapers (Nafeza, NFSA, GOEIC)
- Implement error handling and retry logic
- Implement health checks for each scraper

4.1.10.2 Parse & Extract Layer

- Implement entity extraction (HF NLP, A2)
- Implement relation extraction (A2)
- Implement MRL extraction (A2)
- Implement document requirement extraction (A2)
- Implement treatment requirement extraction (A2)
- Implement acceptance criteria extraction (A2)
- Implement confidence scoring for extracted data

4.1.10.3 Vector Embedding Layer

- Deploy pgvector extension
- Implement embedding generation (sentence-transformers)
- Implement vector similarity search
- Implement vector index (IVFFlat)
- Implement batch embedding for efficiency

4.1.10.4 Rules & Matrix Layer

Implement jurisdiction matrix update (A4, WasmEdge)

Implement treatment rules update (A4, WasmEdge)

Implement document requirements update (A4, WasmEdge)

Implement MRL validation rules (A4, WasmEdge)

Implement port rules update (A4, WasmEdge)

4.1.10.5 Output Layer

Implement Product Form Agent schema generation

Implement document checklist generation

Implement treatment requirement display

Implement jurisdiction validation

4.1.10.6 Change Detection & Alerting

Implement change detection logic

Implement Smart Inbox alerting (A1, Groq)

Implement RIA change log

Implement Admin Portal monitoring dashboard

4.1.10.7 Testing

Test each scraper independently

Test parse and extraction accuracy

Test vector embedding search

Test jurisdiction matrix updates

Test Product Form Agent schema generation

Test change detection and alerting

Integration test: full RIA pipeline

Performance test: 6-hour scrape cycle

4.1.11 AI Authority Summary for Part 4.1

END OF PART 4.1 — REGULATORY INTELLIGENCE AGENT (RIA)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.2 — INCOTERM SELECTION & ENGINE

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.2.0 Purpose & Design Philosophy

The Incoterm Selection module is the critical junction where commercial intent meets logistics execution. The buyer's selection of an International Commercial Term (Incoterms 2020) instantly configures the entire trade workflow — determining which logistics services are mandatory, who pays for what, and how the SGTx platform fee is calculated. This is not a simple dropdown; it is a decision engine that drives all subsequent phases.

Key Principles:

One selection, full configuration — Choosing an incoterm immediately pre-populates the seller's mandatory logistics services, fee responsibilities, and documentation requirements.

Plain-language explanation — Both buyer and seller see exactly who is responsible for what, in their preferred language.

Constitutional enforcement — The WasmEdge incoterms_engine.wasm validates that all subsequent logistics entries match the selected incoterm.

Jurisdiction-aware — Some incoterms are restricted or modified based on origin/destination jurisdiction.

Non-marketplace — Never suggests "you might want to use DDP instead" or recommends alternative incoterms.

AI Integration in Part 4.2:

4.2.1 Incoterm Selection — Location & Behaviour

Location: Immediately after seller selection, before container entry. This is a mandatory field displayed prominently at the top of the form.

Behaviour:

Buyer selects an incoterm from the dropdown (Incoterms 2020).

System validates that the incoterm is permitted for the selected origin/destination countries.

System stores the incoterm in trade_requests.parsed_specs.incoterm.

System auto-configures the seller's mandatory logistics services (visible in Phase 2).

System calculates the default SGTX fee payer (always seller's side, but affects trade value calculation).

System displays a plain-language summary of responsibilities.

UI Placement:

text

NEW TRADE REQUEST

1. SELLER SELECTION

[SGTX-VN-TRD-002139-7F3A] Mekong Fresh Co. Trust: 92

2. INCOTERM SELECTION

[CFR ▼] (Cost and Freight)

Seller is responsible for:

• Export customs clearance

• Trucking to port of loading


```

};
}
// 2. Check if service is forbidden for this incoterm
if is_forbidden(incoterm, service) && has_quote {
return Verdict::Deny {
reason: format!("{}", is not applicable for {} incoterm. Please remove.",
service.display_name(), incoterm.display_name())
};
}
// 3. Check if incoterm is valid for jurisdiction
if !is_jurisdiction_valid(incoterm, origin_country, destination_country) {
return Verdict::Deny {
reason: format!("{}", is not permitted for the selected origin/destination.",
incoterm.display_name())
};
}
Verdict::Allow
}

```

4.2.5.2 Service Forbidden Rules

4.2.5.3 Jurisdiction Compatibility

4.2.5.4 Transport Mode Compatibility

4.2.6 Incoterm Impact on SGTX Fee Calculation

The incoterm affects the Total Trade Value used for the SGTX fee calculation:

Formula:

text

Total Trade Value = Σ (EXW Price) + Σ (Mandatory Logistics Costs)

SGTX Fee = Total Trade Value $\times$ 1.5%

Example:

4.2.7 AI-Generated Plain-Language Summary (A1, Groq)

The system generates a plain-language summary for both buyer and seller, in their preferred language.

Prompt Template:

text

You are explaining Incoterm {incoterm} to a {buyer_role} and {seller_role} in {language}.

The trade is from {origin} to {destination}.

Generate a 3-4 sentence summary of responsibilities:


```

ALTER TABLE trade_requests ADD COLUMN incoterm_selected_at TIMESTAMPTZ;
ALTER TABLE trade_requests ADD COLUMN incoterm_validation_result JSONB;

-- Incoterm rules (reference table, updated by RIA)
CREATE TABLE incoterm_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incoterm TEXT NOT NULL, -- 'EXW', 'FOB', 'CFR', 'CIF', 'CPT', 'CIP', 'DAP', 'DPU', 'DDP'
  full_name TEXT NOT NULL,
  service_type TEXT NOT NULL,
  is_mandatory BOOLEAN DEFAULT false,
  is_forbidden BOOLEAN DEFAULT false,
  jurisdiction_restrictions TEXT[],
  transport_mode_compatibility TEXT[],
  source_regulation TEXT,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(incoterm, service_type)
);

-- Incoterm plain-language templates (for A1 generation)
CREATE TABLE incoterm_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incoterm TEXT NOT NULL,
  language TEXT NOT NULL,
  buyer_responsibility TEXT NOT NULL,
  seller_responsibility TEXT NOT NULL,
  optional_items TEXT,
  payment_notes TEXT,
  version INTEGER DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(incoterm, language)
);

-- Indexes
CREATE INDEX idx_incoterm_rules_incoterm ON incoterm_rules(incoterm);
CREATE INDEX idx_incoterm_rules_service ON incoterm_rules(service_type);
CREATE INDEX idx_incoterm_templates_incoterm ON incoterm_templates(incoterm);
CREATE INDEX idx_incoterm_templates_language ON incoterm_templates(language);

```

4.2.11 Implementation Checklist (Part 4.2)

4.2.11.1 Frontend

- Implement incoterm dropdown with full incoterm names
- Add plain-language responsibility summary (A1, Groq)
- Auto-configure seller's mandatory services (hidden from buyer)
- Add visual indicator of selected incoterm (icon, colour coding)
- Add "View full Incoterms 2020 reference" link
- Mobile-responsive incoterm selection UI

4.2.11.2 Backend

- Implement incoterm selection endpoint (POST /v1/trade/incoterm/select)
- Implement incoterm validation service (Rust)
- Integrate WasmEdge incoterms_engine.wasm
- Implement incoterm rules lookup
- Implement auto-configuration of mandatory services
- Implement plain-language summary generation (A1, Groq)

4.2.11.3 WasmEdge Module

- Compile incoterms_engine.wasm from Rust
- Implement validation logic (mandatory/forbidden services)
- Implement jurisdiction compatibility
- Implement transport mode compatibility
- Add hot-reload capability
- Sign module with multisig

4.2.11.4 Governor Integration

- Add G1U5: Jurisdiction validation
- Add G1U6: Incoterm validation
- Add G2U18: Mandatory service check
- Add Decision Panel integration for CONDITIONAL/DENY

4.2.11.5 Testing

- Unit tests: incoterm rules validation
- Unit tests: mandatory service detection
- Integration tests: incoterm selection → auto-configuration
- Integration tests: incoterm validation (valid/invalid combinations)
- Integration tests: plain-language summary generation
- End-to-end tests: incoterm selection → seller quote → submission
- Performance tests: incoterm validation latency (<100ms)

The Product Form Agent is the intelligence engine that transforms a simple product selection into a complete, regulation-aware, product-specific form. It is the bridge between the RIA's regulatory knowledge and the buyer's data entry experience. When a buyer selects a product (by typing a product name or HS code), the Product Form Agent:

Queries the RIA vector database for all relevant regulations, requirements, and defaults.

Generates a dynamic JSON schema that defines exactly what fields, documents, treatments, and acceptance criteria are required.

Pre-populates defaults (packing, pallet types, weights) based on historical and regulatory data.

Adds special procedure fields (cold treatment, fumigation, pre-cooling) when required.

Generates the Acceptance Criteria Matrix — the specifications that determine acceptance or rejection.

Caches the schema for 7 days to ensure performance.

Key Principles:

One schema per product-context combination — The schema is generated uniquely for each HS code + origin + destination + port combination.

AI-generated, human-validated — The schema is generated by AI (A2, HF local + Groq) but can be overridden by the Platform Governance Authority if needed.

Regulation-aware — Every field, document, and requirement is traceable to a specific regulation.

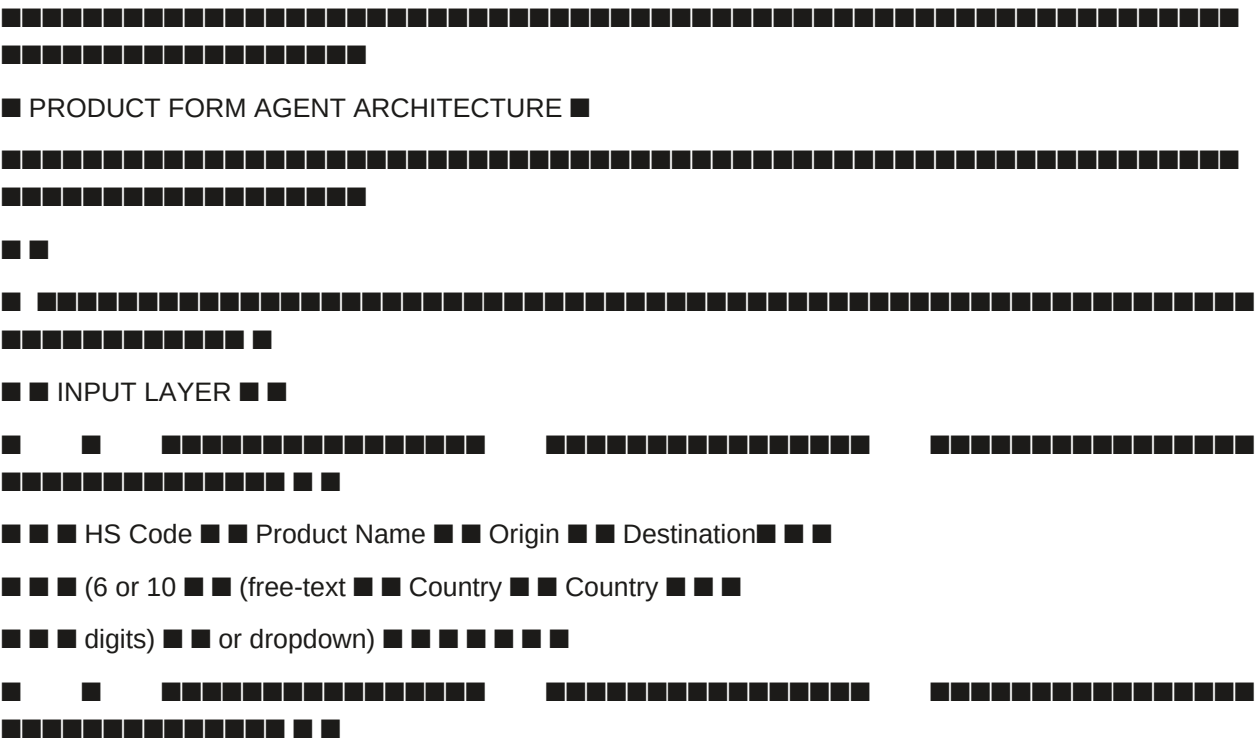
Performance-optimised — Schemas are cached for 7 days; fallback to static templates if AI is unavailable.

Non-marketplace — Never suggests products or alternatives; only generates the form for the buyer's selected product.

AI Integration in Part 4.3:

4.3.1 Agent Architecture

text




```
"name": "variety",
"label": "Variety",
"type": "dropdown",
"options": ["Valencia", "Navel", "Blood Orange", "Cara Cara", "Other"],
"mandatory": true,
"default": "Valencia",
"tooltip": "Select the variety of oranges. Different varieties have different MRL requirements.",
"ai_recommendation": "Valencia is the most common export variety from Egypt."
```

```
},
```

```
{
```

```
"name": "grade",
"label": "Grade",
"type": "dropdown",
"options": ["Premium", "Grade A", "Grade B", "Commercial"],
"mandatory": true,
"default": "Grade A",
"tooltip": "Grade determines quality standards and pricing.",
"ai_recommendation": "Grade A is recommended for export to Japan."
```

```
},
```

```
{
```

```
"name": "brix",
"label": "Brix (Sugar Content)",
"type": "number",
"min": 10,
"max": 16,
"step": 0.1,
"mandatory": true,
"default": 12.5,
"unit": "°Bx",
"tooltip": "Minimum Brix for export to Japan is 10°Bx. Higher Brix indicates sweeter fruit.",
"ai_recommendation": "12.5°Bx is the optimal balance for Japanese market preference."
```

```
},
```

```
{
```

```
"name": "size_range_min_mm",
"label": "Minimum Size (mm)",
```

```
"type": "number",
"min": 55,
"max": 90,
"mandatory": true,
"default": 72,
"unit": "mm",
"tooltip": "Diameter of the fruit in millimetres."
},
{
"name": "size_range_max_mm",
"label": "Maximum Size (mm)",
"type": "number",
"min": 55,
"max": 90,
"mandatory": true,
"default": 80,
"unit": "mm",
"tooltip": "Diameter of the fruit in millimetres."
},
{
"name": "colour_grade",
"label": "Colour Grade",
"type": "dropdown",
"options": ["Bright Orange", "Light Orange", "Greenish"],
"mandatory": true,
"default": "Bright Orange",
"tooltip": "Colour indicates ripeness and quality for the Japanese market."
}
],
"packing_defaults": {
"pallet_type": "EUR",
"pallet_dimensions_mm": {
"width": 800,
"length": 1200,
"height": 144
```

```
},
"cartons_per_pallet": 40,
"net_weight_per_carton_kg": 10,
"gross_weight_per_carton_kg": 10.5,
"stacking_layers": 5,
"max_pallets_per_container_40ft": 22,
"max_pallets_per_container_40hc": 24,
"pallet_tare_kg": 25,
"default_packaging_type": "Cartons 10kg",
"packaging_options": [
  "Cartons 10kg",
  "Cartons 12kg",
  "Mesh Bags 5kg",
  "Plastic Crates",
  "Bulk Boxes"
],
"ai_recommendation": "EUR pallet with 40 cartons (10kg each) is the most efficient configuration for this product and route."
},
"acceptance_criteria_matrix": [
  {
    "parameter": "Moisture",
    "limit": "Max 14%",
    "unit": "%",
    "mandatory": true,
    "source_regulation": "EU RASFF #2024-456",
    "ai_confidence": 0.92
  },
  {
    "parameter": "Protein",
    "limit": "Min 12%",
    "unit": "%",
    "mandatory": true,
    "source_regulation": "Codex Alimentarius",
    "ai_confidence": 0.88
  },
]
```

```
{
  "parameter": "Purity",
  "limit": "Min 99%",
  "unit": "%",
  "mandatory": true,
  "source_regulation": "WTO SPS #2024-789",
  "ai_confidence": 0.91
},
{
  "parameter": "Crude Fat",
  "limit": "Max 8%",
  "unit": "%",
  "mandatory": true,
  "source_regulation": "EU RASFF #2024-456",
  "ai_confidence": 0.90
},
{
  "parameter": "Ash",
  "limit": "Max 2%",
  "unit": "%",
  "mandatory": true,
  "source_regulation": "Codex Alimentarius",
  "ai_confidence": 0.87
},
{
  "parameter": "Foreign Matter",
  "limit": "Max 0.5%",
  "unit": "%",
  "mandatory": true,
  "source_regulation": "EU RASFF #2024-456",
  "ai_confidence": 0.93
}
],
"quantity_defaults": {
  "default_quantity": 100,
```

```
"default_unit": "MT",
"default_tolerance": 5,
"tolerance_options": [2, 3, 5, 7, 10, "Custom"],
"unit_options": ["MT", "KG", "LB", "Pieces", "Cartons"],
"ai_recommendation": "100 MT is the standard container load for this product."
},
"required_documents": [
{
"type": "COMMERCIAL_INVOICE",
"label": "Commercial Invoice",
"mandatory": true,
"auto_generate": true,
"trigger": "SHIPMENT",
"source_regulation": "WTO",
"tooltip": "Required for customs clearance and payment."
},
{
"type": "PACKING_LIST",
"label": "Packing List",
"mandatory": true,
"auto_generate": true,
"trigger": "SHIPMENT",
"source_regulation": "WTO",
"tooltip": "Required for customs clearance and logistics."
},
{
"type": "PHYTOSANITARY_CERTIFICATE",
"label": "Phytosanitary Certificate",
"mandatory": true,
"trigger": "SHIPMENT",
"source_regulation": "IPPC",
"tooltip": "Required for all fresh fruit exports to Japan."
},
{
"type": "COLD_TREATMENT_CERTIFICATE",
```



```
"label": "Cold Treatment Certificate",
"mandatory": true,
"trigger": "SHIPMENT",
"source_regulation": "IPPC",
"tooltip": "Required for Japan. Certificate must be from an approved facility."
},
{
"type": "HEALTH_CERTIFICATE",
"label": "Health Certificate",
"mandatory": true,
"trigger": "SHIPMENT",
"source_regulation": "NFSA",
"tooltip": "Required for food products to Japan."
},
{
"type": "CERTIFICATE_OF_ORIGIN",
"label": "Certificate of Origin",
"mandatory": true,
"trigger": "SETTLEMENT",
"source_regulation": "WTO",
"tooltip": "Required for tariff preference and LC."
},
{
"type": "COLD_CHAIN_LOG",
"label": "Cold Chain Temperature Log",
"mandatory": true,
"trigger": "SHIPMENT",
"source_regulation": "FAO",
"tooltip": "Required to verify temperature compliance during transit."
},
{
"type": "BILL_OF_LADING",
"label": "Bill of Lading",
"mandatory": true,
"auto_generate": false,
```

```
"trigger": "SETTLEMENT",
"source_regulation": "WTO",
"tooltip": "Required for settlement and LC."
},
{
"type": "INSURANCE_CERTIFICATE",
"label": "Insurance Certificate",
"mandatory": false,
"trigger": "SETTLEMENT",
"source_regulation": "Optional",
"tooltip": "Required if buyer purchases insurance."
}
],
"lab_tests_required": [
{
"analyte": "Cypermethrin",
"label": "Cypermethrin",
"mrl_mg_per_kg": 0.02,
"unit": "mg/kg",
"mandatory": true,
"source_regulation": "EU RASFF #2026-456",
"method": "GC-MS/MS"
},
{
"analyte": "Chlorpyrifos",
"label": "Chlorpyrifos",
"mrl_mg_per_kg": 0.01,
"unit": "mg/kg",
"mandatory": true,
"source_regulation": "EU RASFF #2025-123",
"method": "GC-MS/MS"
},
{
"analyte": "Thiabendazole",
"label": "Thiabendazole",
```

```
"mrl_mg_per_kg": 0.05,
"unit": "mg/kg",
"mandatory": true,
"source_regulation": "Codex Alimentarius",
"method": "HPLC"
},
{
  "analyte": "Imazalil",
  "label": "Imazalil",
  "mrl_mg_per_kg": 0.03,
  "unit": "mg/kg",
  "mandatory": true,
  "source_regulation": "EU RASFF #2024-789",
  "method": "HPLC"
}
],
"special_conditions": [
  {
    "condition": "Cold treatment required: -0.5°C for 14 days.",
    "required": true,
    "source_regulation": "IPPC",
    "details": "Must be completed before loading. Certificate from approved facility required."
  },
  {
    "condition": "Pre-cooling certificate required before loading.",
    "required": true,
    "source_regulation": "FAO",
    "details": "Fruit must be pre-cooled to 2°C within 12 hours of picking."
  },
  {
    "condition": "Wood packaging must be ISPM 15 compliant.",
    "required": true,
    "source_regulation": "IPPC",
    "details": "All wood packaging must be heat-treated or fumigated."
  },
  {
```

```
{
  "condition": "Traceability: Each carton must have a traceability barcode.",
  "required": true,
  "source_regulation": "EU RASFF",
  "details": "GS1-128 barcode with lot number and SSCC."
},
{
  "treatment_details": {
    "type": "cold_treatment",
    "temperature_c": -0.5,
    "duration_days": 14,
    "certificate_required": true,
    "accepted_facilities": [
      "Egyptian Cold Treatment Center",
      "Alexandria Cold Storage",
      "Damietta Cold Chain Facility"
    ],
    "source_regulation": "IPPC"
  },
  "inspection_requirements": {
    "recommended_type": "Third Party Inspection",
    "sampling_plan": "AQL General Level II",
    "sample_size": "5 pallets random",
    "regulatory_reference": "ISO 17020",
    "tooltip": "Japan requires third-party inspection for all fresh fruit imports."
  },
  "criticality_defaults": {
    "suggested_criticality": "Standard",
    "reasons": [
      "Fresh fruit is time-sensitive",
      "Cold treatment adds 14 days",
      "Japanese market has strict requirements"
    ]
  },
  "estimated_values": {
```

```
"estimated_total_quantity_kg": 8800,  
"estimated_gross_weight_kg": 9790,  
"estimated_pallet_count": 22,  
"estimated_container_count": 1,  
"estimated_container_type": "40ft Reefer"  
},  
"ai_confidence": 0.94,  
"fallback_used": false  
}
```

4.3.3 Dynamic Field Generation Logic

4.3.3.1 Field Types Supported

4.3.3.2 Example — Frozen Strawberries (Egypt → Germany)

json

```
{  
  "product_name": "Frozen Strawberries",  
  "hs_code": "0811.10",  
  "dynamic_fields": [  
    {  
      "name": "variety",  
      "label": "Variety",  
      "type": "dropdown",  
      "options": ["Festival", "Camarosa", "Albion", "Sweet Charlie", "Other"],  
      "mandatory": true,  
      "default": "Camarosa"  
    },  
    {  
      "name": "grade",  
      "label": "Grade",  
      "type": "dropdown",  
      "options": ["Grade A (whole)", "Grade B (sliced)", "Grade C (puree)"],  
      "mandatory": true,  
      "default": "Grade A (whole)"  
    },  
    {  
      "name": "sugar_added",
```

```

"label": "Sugar Added?",
"type": "toggle",
"mandatory": true,
"default": false
},
{
"name": "sugar_percentage",
"label": "Sugar Percentage",
"type": "number",
"min": 0,
"max": 30,
"condition": "sugar_added == true",
"default": 10,
"unit": "%"
},
{
"name": "temperature_requirement",
"label": "Temperature Requirement",
"type": "readonly",
"value": "-18°C",
"tooltip": "Frozen strawberries must be maintained at -18°C throughout the cold chain."
},
{
"name": "packaging_type",
"label": "Packaging Type",
"type": "dropdown",
"options": ["Polypags 1kg", "Polypags 5kg", "Cartons 10kg", "Bulk Boxes"],
"mandatory": true,
"default": "Cartons 10kg"
}
]
}

```

4.3.3.3 Example — Textiles (Cotton Fabric, Egypt → UAE)

json

```
{
```

```
"product_name": "Cotton Fabric",
"hs_code": "5208.11",
"dynamic_fields": [
{
"name": "weave_type",
"label": "Weave Type",
"type": "dropdown",
"options": ["Plain", "Twill", "Satin", "Jersey", "Dobby"],
"mandatory": true,
"default": "Plain"
},
{
"name": "thread_count",
"label": "Thread Count",
"type": "number",
"min": 100,
"max": 1000,
"mandatory": true,
"default": 200,
"unit": "threads/inch"
},
{
"name": "width_cm",
"label": "Width",
"type": "number",
"min": 50,
"max": 400,
"mandatory": true,
"default": 150,
"unit": "cm"
},
{
"name": "weight_gsm",
"label": "Weight",
"type": "number",
```

```
"min": 50,
"max": 500,
"mandatory": true,
"default": 180,
"unit": "gsm"
},
{
"name": "colour",
"label": "Colour",
"type": "text",
"mandatory": true,
"placeholder": "e.g., White, Ecru, Navy"
},
{
"name": "roll_length_m",
"label": "Roll Length",
"type": "number",
"min": 10,
"max": 500,
"mandatory": true,
"default": 100,
"unit": "m"
},
{
"name": "number_of_rolls",
"label": "Number of Rolls",
"type": "number",
"min": 1,
"max": 1000,
"mandatory": true
},
{
"name": "organic_certified",
"label": "Organic Certified?",
"type": "toggle",
```



```
"mandatory": false,
"default": false,
"tooltip": "If Yes, GOTS certificate will be required."
}
],
"required_documents": [
{
"type": "COMMERCIAL_INVOICE",
"mandatory": true,
"auto_generate": true,
"trigger": "SHIPMENT"
},
{
"type": "PACKING_LIST",
"mandatory": true,
"auto_generate": true,
"trigger": "SHIPMENT"
},
{
"type": "CERTIFICATE_OF_ORIGIN",
"mandatory": true,
"trigger": "SETTLEMENT"
},
{
"type": "GOTS_CERTIFICATE",
"mandatory": false,
"condition": "organic_certified == true",
"trigger": "SHIPMENT"
},
{
"type": "LAB_REPORT_FORMALDEHYDE",
"mandatory": true,
"trigger": "SHIPMENT"
}
],
```

```

"lab_tests_required": [
{
"analyte": "Formaldehyde",
"mrl_mg_per_kg": 75,
"unit": "mg/kg",
"mandatory": true
},
{
"analyte": "Azo dyes",
"limit": "Negative",
"unit": "N/A",
"mandatory": true
},
{
"analyte": "pH",
"range": "4.0-7.5",
"unit": "pH",
"mandatory": true
}
]
}

```

4.3.4 Schema Caching & Expiry

Cache Storage:

sql

```

-- Cache table (already defined in 4.1.3)
CREATE TABLE commodity_dynamic_schemas_cache (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_code TEXT,
schema_json JSONB NOT NULL,
generated_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '7 days',
product_form_agent_version TEXT,

```

```
ai_confidence DECIMAL(5,2),
fallback_used BOOLEAN DEFAULT false,
UNIQUE(hs_code, origin_country, destination_country, port_code)
);
```

Fallback Logic:

If cache exists and not expired → use cached schema.

If cache expired → regenerate schema.

If Product Form Agent fails (Groq and Ollama both unavailable):

Use cached schema (if available, even if expired).

If no cache exists, use a static template with:

Basic fields (commodity type, product name, quantity).

Generic packing defaults (EUR pallet, 40 cartons per pallet, 10 kg net weight).

Warning message: "Dynamic product specifications unavailable. Please enter details manually."

4.3.5 Acceptance Criteria Matrix Generation

The Acceptance Criteria Matrix is one of the most critical outputs of the Product Form Agent. It defines the specifications that determine whether the goods are accepted or rejected.

4.3.5.1 Generation Logic

rust

// Pseudo-code: Acceptance Criteria Matrix Generation

```
async fn generate_acceptance_criteria(
```

```
  hs_code: &str,
```

```
  destination_country: &str
```

```
) -> Result<Vec<AcceptanceCriterion>> {
```

```
// 1. Query acceptance_criteria_templates
```

```
let templates = get_templates(hs_code).await?;
```

```
// 2. Query country-specific MRLs
```

```
let mrls = get_country_mrls(hs_code, destination_country).await?;
```

```
// 3. Query regulatory standards (Codex, EU, FDA)
```

```
let standards = get_regulatory_standards(hs_code, destination_country).await?;
```

```
// 4. Merge and deduplicate
```

```
let mut criteria = Vec::new();
```

```
for template in templates {
```

```
// Check if country has specific limit
```

```
if let Some(mrl) = mrls.iter().find(|m| m.analyte == template.parameter) {
```

```
  criteria.push(AcceptanceCriterion {
```

```
    parameter: template.parameter,
```

```

limit: format!("{}", mrl.limit_operator, mrl.mrl_mg_per_kg),
unit: mrl.unit,
mandatory: template.is_mandatory,
source: mrl.source_regulation,
confidence: 0.95,
});
} else {
criteria.push(AcceptanceCriterion {
parameter: template.parameter,
limit: template.default_limit,
unit: template.default_unit,
mandatory: template.is_mandatory,
source: template.source_regulation,
confidence: 0.85,
});
}
}

// 5. Add AI-generated recommendations (A1, Groq)
let recommendations = groq_recommendations(&criteria).await?;
Ok(criteria)
}

```

confidence: 1.0,

```

source: "HS_DATABASE",
});
}
// 2. Check if query matches saved products (buyer's history)
let saved = get_saved_products_by_name(query).await?;
if !saved.is_empty() {
return Ok(saved);
}
// 3. Vector similarity search
let vectors = get_product_vectors(query).await?;
// 4. AI classification (A2, HF local)
let classified = classify_product(query).await?;
// 5. Merge and sort by confidence
let mut results = Vec::new();
results.extend(vectors);
results.extend(classified);
results.sort_by(|a, b| b.confidence.partial_cmp(&a.confidence).unwrap());
Ok(results)
}

```

4.3.7 Database Schema Additions (Part 4.3)

```

sql
-- Product vectors (for similarity search)
CREATE TABLE product_vectors (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
product_name TEXT NOT NULL,
synonyms TEXT[],
description TEXT,
embedding vector(384),
metadata JSONB,
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Acceptance criteria templates
CREATE TABLE acceptance_criteria_templates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

hs_code CHAR(6) NOT NULL,
parameter TEXT NOT NULL,
default_limit TEXT NOT NULL,
default_unit TEXT NOT NULL,
is_mandatory BOOLEAN DEFAULT true,
source_regulation TEXT,
ai_confidence DECIMAL(5,2),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Product form agent cache (already in 4.1.3)

```

CREATE TABLE commodity_dynamic_schemas_cache (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_code TEXT,
schema_json JSONB NOT NULL,
generated_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '7 days',
product_form_agent_version TEXT,
ai_confidence DECIMAL(5,2),
fallback_used BOOLEAN DEFAULT false,
UNIQUE(hs_code, origin_country, destination_country, port_code)
);

```

-- Indexes

```

CREATE INDEX idx_product_vectors_hs ON product_vectors(hs_code);
CREATE INDEX idx_product_vectors_name ON product_vectors(product_name);
CREATE INDEX idx_product_vectors_embedding ON product_vectors
USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
CREATE INDEX idx_acceptance_criteria_templates_hs ON acceptance_criteria_templates(hs_code);
CREATE INDEX idx_commodity_dynamic_schemas_cache_hs ON
commodity_dynamic_schemas_cache(hs_code, origin_country, destination_country);
CREATE INDEX idx_commodity_dynamic_schemas_cache_expiry ON
commodity_dynamic_schemas_cache(expires_at);

```

4.3.8 Implementation Checklist (Part 4.3)

4.3.8.1 Core Agent

Implement Product Form Agent service (Rust)
Implement RIA vector database query
Implement Groq schema generation (with Ollama fallback)
Implement schema validation (A4, WasmEdge)
Implement schema caching (7-day TTL)
Implement fallback to static template
Implement schema versioning

4.3.8.2 Dynamic Fields

Implement all field types (dropdown, number, toggle, text, readonly, date, condition)
Implement field validation (min, max, step, mandatory)
Implement conditional field logic
Implement tooltip generation (A1, Groq)
Implement default value recommendation (A1, Groq)

4.3.8.3 Packing Defaults

Implement packing defaults from RIA
Implement palletisation recommendations
Implement container capacity calculations
Implement weight calculations

4.3.8.4 Acceptance Criteria Matrix

Implement acceptance criteria generation
Implement MRL validation
Implement regulatory standard lookup
Implement confidence scoring

4.3.8.5 Product Resolution

Implement HS code validation
Implement product name → HS code resolution
Implement HS code → product name resolution
Implement free-text classification (A2, HF local)
Implement vector similarity search (pgvector)
Implement saved products lookup

4.3.8.6 Testing

Unit tests: schema generation for all product types
Unit tests: field validation
Unit tests: conditional field logic
Unit tests: acceptance criteria generation

Integration tests: product resolution (all flows)

4.3.9 AI Authority Summary for Part 4.3

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

The Structured Container & Commodity Entry is the heart of the New Trade Request form. It is where the buyer translates their commercial intent into a machine-readable, structured data model that drives all subsequent phases of the trade lifecycle. This section defines how containers and commodities are configured, validated, and stored.

Zero re-entry — All data entered here is stored once and reused across all phases (packing, customs, invoicing, financing, etc.).

AI-assisted, never automated — AI provides defaults, suggestions, and validation, but the buyer remains in full control.

Bulk operations — Clone, bulk edit, and drag-and-drop reordering for efficiency.

AI Integration in Part 4.4:

4.4.1.1 Overview

Default: 1 container (max 50 containers).

text

■ [Bulk Edit] [Drag to reorder] ■

Container

1

Country of Origin: [Vietnam ▼] Destination Country: [Egypt ▼] Port of Discharge: [Alexandria ▼] (filtered by Egypt) Palletized: [Yes ●] [No] Pallet Size: [EUR 800×1200 mm ▼] Destination Override: [Alexandria Free Zone] [Clone] [Remove]

Container

2

Country of Origin: [Vietnam ▼] Destination Country: [Egypt ▼] Port of Discharge: [Port Said ▼] Palletized: [Yes ●] [No] Pallet Size: [EUR 800×1200 mm ▼] Destination Override: [Port Said Free Zone] [Clone] [Remove]

Container

3

Country of Origin: [Vietnam ▼] Destination Country: [Egypt ▼] Port of Discharge: [Damietta ▼] Palletized: [Yes ●] [No] Pallet Size: [EUR 800×1200 mm ▼] Destination Override: [Damietta Port] [Clone] [Remove]

4.4.1.2 Per-Container Fields

4.4.1.3 UI Enhancements

Flag Icons:

Next to country dropdowns for visual recognition.

Lightweight sprite set, no external dependencies.

Inline Validation:

Port must exist and be active.

Country must not be sanctioned.

Pallet size must be valid.

Drag-and-Drop Reordering:

Reorder containers by dragging tabs.

Updates container numbering automatically.

Progress Indicator:

Shows how many containers are fully configured.

Container 1: 100% complete, Container 2: 80% complete.

Clone Container:

One click to copy all data to a new container.

Appends a number to the container name.

Bulk Edit:

Apply commodity to all containers.

Copy settings to a range of containers.

Update origin/destination/port for multiple containers.

Remove Container:

Only shown if number of containers > 1.

Confirmation dialog before removal.

4.4.2 Commodity Configuration

4.4.2.1 Overview

Within each container, the buyer defines one or more commodities. Each commodity has its own set of fields, dynamically generated by the Product Form Agent.

Default: 1 commodity per container (unlimited commodities per container).

4.4.2.2 Core Commodity Fields

4.4.2.3 Dynamic Commodity Fields (Product Form Agent)

When a product is selected, the Product Form Agent dynamically adds:

Product-Specific Fields:

Variety, grade, brix, size range, etc. (per product).

These are defined by the Product Form Agent schema.

Acceptance Criteria Matrix:

Dynamic table of specifications (Moisture, Protein, Purity, etc.).

Limits and units are pre-filled based on RIA data.

Buyer can add custom parameters.

Special Procedures:

Cold treatment, fumigation, pre-cooling, etc.

Date fields, facility selection, certificate requirements.

Required Documents:

Dynamic checklist per product and jurisdiction.

Triggers: Shipment, Settlement, Customs, Financing.

Lab Tests Required:

Analytes with MRLs (Maximum Residue Limits).

Pass/fail status, source regulation.

4.4.3 Complete Container & Commodity UI Example

Example: Container 1 with 2 Commodities (Oranges & Lemons)

text

Container 1 (2 commodities) [Progress: 100%] [Clone] [Remove]

Country of Origin: [Vietnam ▼] Destination Country: [Egypt ▼]

Port of Discharge: [Alexandria ▼] Palletized: [Yes ●] Pallet Size: [EUR ▼]

Destination Override: [Alexandria Free Zone]

Commodity

1:

Oranges

Product: [0805.10 → Valencia Oranges] ▼ (autocomplete)

Commodity Type: [Fresh Fruits ▼]

QUANTITY & TOLERANCE

Requested Quantity: [100] [MT ▼] Tolerance: [±5% ▼]

Partial Shipment: [Yes ●] Transshipment: [No ●] Criticality: [Standard ▼]

ACCEPTANCE CRITERIA MATRIX (Dynamically Generated)

Parameter

Value / Limit

Unit

Status

Source

Moisture

Max 14%

%

Within

EU

Protein

Min 12%

%

Within

Codex

Purity

Min 99%

%

Within

WTO

[Add Custom Parameter]

11

These specifications determine whether the goods will be accepted or rejected.				
All values must be within the specified limits.				
Parameter	Value / Limit	Unit	Status	Source
Moisture	Max 14%	%	Within	EU RASFF #2026
Protein	Min 12%	%	Within	Codex Alimentar
Purity	Min 99%	%	Within	WTO SPS #2025
Crude Fat	Max 8%	%	Within	EU RASFF #2026
Ash	Max 2%	%	Pending	Codex Alimentar
Foreign Matter	Max 0.5%	%	Exceeded	EU RASFF #2026
[Add Custom Parameter] [Request Deviation Approval for Foreign Matter]				
AI Recommendation: Foreign Matter exceeds EU limit. Consider improving				
cleaning process or request deviation approval with justification.				

4.4.5 Weight Calculation Engine

4.4.5.1 Core Calculations

The system calculates weights in real time based on the commodity configuration.

Formulas:

text

$$\text{Total Cartons} = \Sigma (\text{Pallets} \times \text{Cartons per Pallet})$$

$$\text{Total Net Weight} = \Sigma (\text{Total Cartons} \times \text{Net Weight per Carton})$$

$$\text{Gross Weight (Cartons Only)} = \Sigma (\text{Total Cartons} \times (\text{Net Weight per Carton} + \text{Tare per Carton}))$$

$$\text{Total Gross Weight} = \text{Gross Weight (Cartons Only)} + \Sigma (\text{Pallets} \times \text{Pallet Tare})$$

Example — Two Commodities:

Commodity A (Oranges): 22 pallets, 40 cartons/pallet, net 10 kg/carton, tare 0.5 kg.

Commodity B (Lemons): 14 pallets, 48 cartons/pallet, net 8 kg/carton, tare 0.4 kg.

Pallet Tare: 25 kg (EUR)

text

Commodity A:

$$\text{total cartons} = 22 \times 40 = 880$$

$$\text{net} = 880 \times 10 = 8,800 \text{ kg}$$

■ Total gross weight (including pallets): 15,784.8 kg ■

■ ■ Within container limit ($\leq 28,000$ kg for 40ft container) ■



4.4.5.3 Container Capacity Validation

Validation Rules (A4, WasmEdge):

Total gross weight $\leq$ container max payload.

Total volume $\leq$ container max volume (if packing plan locked).

If exceeded, Governor returns **CONDITIONAL** with Decision Panel.

Example CONDITIONAL Response:

json

 $\{$

```
"verdict": "CONDITIONAL",
```

```
"tenant_message": {
```

"summary": "Container 1 exceeds the maximum payload limit. Total gross weight is 32,000 kg, but the maximum for a 40ft container is 28,000 kg.",

```
"conditions": [
```

 $\{$

```
"condition_id": "weight_exceeded",
```

```
"label": "Reduce weight or select a different container type",
```

"action_url": "/v1/trade/request/draft?trade_id=..."

}

]

```
"escalation_hint": "If you believe this is an error, request a human review."
```

}

}

4.4.6 Two-Way Smart Product/HS Code Resolution

4.4.6.1 Resolution Flows

Flow 1: Product Name → HS Code

Buyer types product name (e.g., "Valencia Oranges").

System queries product vectors (pgvector) for closest match.

System displays suggestions with confidence scores.

Buyer selects the correct product.

System auto-fills the HS code.

Flow 2: HS Code → Product Name

Buyer types HS code (e.g., "0805.10").

System queries HS database for product name.
System auto-fills product name.
System triggers Product Form Agent with HS code.
Flow 3: Free-Text → Product Name + HS Code
Buyer types free-text (e.g., "Valencia oranges for export").
System uses AI (A2, HF local) to extract product and HS code.
System displays suggestions with confidence scores.
Buyer selects the correct product.
System triggers Product Form Agent.
Flow 4: Buyer History → Product Name + HS Code
Buyer types partial product name.
System shows previously used products from buyer's history.
Buyer selects from history.
System triggers Product Form Agent.

4.4.6.2 UI Example

text

Product Search

[Valencia oranges]

Suggestions:

Valencia Oranges (HS 0805.10) — Fresh Fruits — Confidence: 98%

Recent trade: 15 May 2026 (12 trades)

Navel Oranges (HS 0805.10) — Fresh Fruits — Confidence: 92%

Recent trade: 10 Jan 2026 (3 trades)

Blood Oranges (HS 0805.10) — Fresh Fruits — Confidence: 85%

No trade history

[Enter HS code manually]

[Use AI Express]



4.4.6.3 Resolution Algorithm

rust

```
async fn resolve_product(query: &str, buyer_gtid: &str) -> Result<Vec<ProductSuggestion>> {  
    // 1. Check if query is HS code format (6 or 10 digits)  
    if is_hs_code(query) {  
        let product = get_product_by_hs(query).await?;  
        return Ok(vec![ProductSuggestion {  
            product_name: product.name,  
            hs_code: product.hs_code,  
            confidence: 1.0,  
            source: "HS_DATABASE",  
            recent_trade: None,  
        }]);  
    }  
    // 2. Check buyer's saved products (history)  
    let saved = get_saved_products_by_name(query, buyer_gtid).await?;  
    if !saved.is_empty() {  
        return Ok(saved);  
    }  
    // 3. Vector similarity search (pgvector)  
    let vectors = get_product_vectors(query).await?;  
    // 4. AI classification (A2, HF local)  
    let classified = classify_product(query).await?;  
    // 5. Merge and sort by confidence  
    let mut results = Vec::new();  
    results.extend(vectors);  
    results.extend(classified);  
    results.sort_by(|a, b| b.confidence.partial_cmp(&a.confidence).unwrap());  
    // 6. Add recent trade information  
    for result in &mut results {  
        result.recent_trade = get_recent_trade(buyer_gtid, &result.hs_code).await?;  
    }  
    Ok(results)  
}
```

Non-Marketplace Safeguard: The dropdown is filtered by commodity type and buyer's saved products, not by "most popular." AI only assists based on public data and buyer's own history. The system never suggests "other buyers bought X" or "popular products."

4.4.7 Express Mode (Free-Text AI Parsing)

4.4.7.1 Overview

For users who prefer natural language over structured forms, Express Mode allows the buyer to describe their trade in plain English. AI parses the text and extracts structured data.

Toggle: "Use AI Express" at the top of the New Trade Request form. When enabled, the structured form is hidden and replaced with a single large text area.

Express Mode Interface:

text

AI Express Mode

Describe your trade in plain English. The AI will extract all structured data.

I need 100 MT of Valencia oranges from Vietnam to Egypt, CFR Alexandria.

Packed in 10 kg cartons, 40 cartons per pallet, 22 pallets. Also 50 MT of

Eureka lemons, 48 cartons per pallet, 14 pallets. Two containers, second

container to Port Said. Cold treatment required. Third-party inspection

required. Tolerance ±5%. Partial shipment allowed.

[Voice Input] [Parse]

Extracted Data (Confidence: 92%)

Container 1: Vietnam → Egypt (Alexandria)

Commodity 1: Valencia Oranges (HS 0805.10) — 100 MT

Tolerance: ±5% Partial Shipment: Yes

Packaging: 22 pallets, 40 cartons/pallet, 10 kg/carton

Description: One click to copy all data from one container to a new container.

Behaviour:

Click "Clone Container" on the source container.

System creates a new container with all data copied.

System appends a number to the container name.

Buyer can modify the new container as needed.

UI:

text

■ Number of Containers: [2] [+ Add Container] [Clone Container] ■

Container 1

■ ■ Origin: Vietnam → Egypt (Alexandria) Palletized: Yes ■ ■

■ ■ Commodities: Oranges (100 MT), Lemons (50 MT) ■ ■

■ ■ [Clone] [Remove] ■ ■

Container 2

■ ■ Origin: Vietnam → Egypt (Alexandria) Palletized: Yes ■ ■

■ ■ Commodities: Oranges (100 MT), Lemons (50 MT) ← Cloned data ■ ■

■ ■ [Clone] [Remove] ■ ■

4.4.8.2 Bulk Edit

Description: Apply commodity to all containers or copy settings to a range.

UI:

text

■ BULK EDIT ■

□ □

■ Apply to: [All Containers ▼] ■

□ □

■ Settings to apply: ■

Commodity Type: [Fresh Fruits ▼]

■ ■ Product: [Valencia Oranges ▼] ■ ■

■ ■ Packaging: [Palletized ▼] ■

■ ■ Tolerance: $[\pm 5\% \nabla]$ ■ ■

Country of Origin: [Vietnam ▼]

Destination Country: [Egypt ▼]

Port of Discharge: [Alexandria ▼]

■ ■

■ [Apply to All] [Cancel] ■

4.4.9 Database Schema Additions (Part 4.4)

sql

-- Trade Request Commodities (NEW — replaces header-level quantity)

```
CREATE TABLE trade_request_commodities (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
container_id UUID NOT NULL,
commodity_type TEXT NOT NULL,
product_name TEXT NOT NULL,
hs_code CHAR(6) NOT NULL,
-- Quantity & Tolerance (per commodity)
requested_quantity DECIMAL(20,4) NOT NULL,
requested_quantity_unit TEXT NOT NULL,
quantity_tolerance DECIMAL(5,2) DEFAULT 5.0,
partial_shipment_allowed BOOLEAN DEFAULT true,
transshipment_allowed BOOLEAN DEFAULT true,
-- Product Criticality (per commodity)
product_criticality TEXT DEFAULT 'STANDARD',
-- Inspection Requirements (per commodity)
inspection_requirements TEXT,
-- Acceptance Criteria (per commodity)
acceptance_criteria_matrix JSONB,
-- Packaging & Weight (per commodity)
packaging_type TEXT,
```

```

pallet_count INTEGER,
pallet_type TEXT,
cartons_per_pallet INTEGER,
net_weight_per_carton_kg DECIMAL(10,2),
gross_weight_per_carton_kg DECIMAL(10,2),
total_net_weight_kg DECIMAL(10,2),
total_gross_weight_kg DECIMAL(10,2),
-- Special requirements (per commodity)
special_conditions JSONB,
treatment_details JSONB,
lab_tests_required JSONB,
-- Dynamic fields (per commodity, from Product Form Agent)
dynamic_fields JSONB,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Container configuration
CREATE TABLE trade_request_containers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
container_number INTEGER NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_of_discharge TEXT NOT NULL,
palletized BOOLEAN DEFAULT true,
pallet_type TEXT DEFAULT 'EUR',
destination_override TEXT,
notes TEXT,
total_net_weight_kg DECIMAL(10,2),
total_gross_weight_kg DECIMAL(10,2),
total_pallet_count INTEGER,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(trade_request_id, container_number)
);

```


-- Express Mode parsing logs

```
CREATE TABLE express_mode_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID REFERENCES trade_requests(id) ON DELETE CASCADE,  
  raw_text TEXT NOT NULL,  
  parsed_json JSONB,  
  confidence DECIMAL(5,2),  
  missing_fields TEXT[],  
  low_confidence_fields TEXT[],  
  user_confirmed BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_trade_request_commodities_request ON  
trade_request_commodities(trade_request_id);  
CREATE INDEX idx_trade_request_commodities_container ON  
trade_request_commodities(container_id);  
CREATE INDEX idx_trade_request_commodities_hs ON trade_request_commodities(hs_code);  
CREATE INDEX idx_trade_request_containers_request ON trade_request_containers(trade_request_id);  
CREATE INDEX idx_trade_request_containers_port ON trade_request_containers(port_of_discharge);  
CREATE INDEX idx_express_mode_logs_request ON express_mode_logs(trade_request_id);  
CREATE INDEX idx_express_mode_logs_created ON express_mode_logs(created_at DESC);
```

4.4.10 Implementation Checklist (Part 4.4)

4.4.10.1 Backend Services

Implement container configuration endpoint (POST /v1/trade/container/configure)

Implement commodity configuration endpoint (POST /v1/trade/commodity/add)

Implement weight calculation service (Rust)

Implement container capacity validation (A4, WasmEdge)

Implement Express Mode parsing (A2, HF local → Ollama)

Implement product resolution service (pgvector + AI)

Implement bulk edit endpoint (POST /v1/trade/bulk-edit)

4.4.10.2 Frontend Components

Build Container Tab component with drag-and-drop reordering

Build Commodity Entry component with dynamic fields

Build Acceptance Criteria Matrix component

Build Weight Summary component (real-time)

Build Product Search component (two-way resolution)

Build Express Mode component (free-text + voice)

Build Bulk Edit modal

Build Clone Container functionality

Build Progress Indicator

Build Inline Validation (port, country, weight)

4.4.10.3 Validation & Governance

Add G1U7: Per-container data consistency

Add G1U9: Container type recommendation logged

Add G1U10: Multi-shipment schedule validation

Add weight limit validation (A4, WasmEdge)

Add port validation (A4, WasmEdge)

Add pallet count validation (A4, WasmEdge)

Add container capacity validation (A4, WasmEdge)

4.4.10.4 Testing

Unit tests: weight calculation engine

Unit tests: product resolution (all flows)

Integration tests: container configuration → commodity entry → weight calculation

Integration tests: Express Mode parsing → form population

Integration tests: bulk edit and clone container

End-to-end tests: full container & commodity configuration

Performance tests: container tab rendering with 50 containers

4.4.11 AI Authority Summary for Part 4.4

END OF PART 4.4 — STRUCTURED CONTAINER & COMMODITY ENTRY

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.5 — DOCUMENTATION REQUIREMENTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.5.0 Purpose & Design Philosophy

The Documentation Requirements section is the single source of truth for all trade-related documents. Unlike the original blueprint, which had fragmented document requirements (separate lists for shipment, settlement, and customs), this section consolidates everything into one unified, trigger-driven document management system.

Key Principles:

One source of truth — Every document is defined once with its triggers, not duplicated across sections.

Trigger-driven — Documents are associated with specific events (Shipment, Settlement, Customs, Financing) to align with real trade execution.

■ ■ • Physical handling service will be recommended ■ ■

Verification Result:

json

```
{
  "document_id": "DOC-2026-001",
  "document_type": "PHYTOSANITARY_CERTIFICATE",
  "verification_status": "VERIFIED",
  "confidence_score": 0.94,
  "extracted_fields": {
    "certificate_number": "PHYTO-2026-001",
    "issue_date": "2026-06-15",
    "expiry_date": "2026-12-15",
    "issuing_authority": "Egyptian Plant Quarantine",
    "hs_code": "0805.10",
    "quantity": "100 MT",
    "treatment": "Cold Treatment -0.5°C for 14 days"
  },
  "discrepancies": [
    {
      "field": "quantity",
      "expected": "100 MT",
      "actual": "100 MT",
      "match": true
    }
  ],
  "fraud_indicators": [],
  "verification_timestamp": "2026-06-20T10:30:00Z"
}
```

4.5.5.2 Fraud Detection (A2)

The system also checks for potential document fraud:

Metadata analysis: Check document metadata for tampering.

Consistency check: Compare with other documents (e.g., invoice vs. packing list).

Issuing authority verification: Check with registry (where available).

Digital signature validation: Verify signatures (if present).

Image analysis: Detect signs of manipulation (Photoshop, etc.).

Fraud Alert Example:


```

"recommended_format": {
"originals_required": true,
"electronic_accepted": true,
"language": "English"
},
"source_regulations": {
"PHYTOSANITARY_CERTIFICATE": "IPPC",
"HEALTH_CERTIFICATE": "NFSA",
"COLD_TREATMENT_CERTIFICATE": "IPPC"
}
}

```

4.5.8 Database Schema Additions (Part 4.5)

sql

-- Document requirements (single source of truth)

```

CREATE TABLE document_requirements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
document_type TEXT NOT NULL,
triggers TEXT[] NOT NULL, -- ['SHIPMENT', 'SETTLEMENT', 'CUSTOMS', 'FINANCING']
is_mandatory BOOLEAN NOT NULL DEFAULT false,
is_pre_selected BOOLEAN DEFAULT false,
format_requirements JSONB, -- { originals_required, electronic_accepted, language }
responsible_party TEXT, -- 'SELLER', 'BUYER', 'BROKER', 'SHIPPING_LINE', 'CUSTOMS'
source_regulation TEXT,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'UPLOADED', 'VERIFIED', 'REJECTED', 'FLAGGED', 'EXPIRED'
uploaded_document_id UUID REFERENCES documents(id) ON DELETE SET NULL,
verification_result JSONB,
fraud_indicators JSONB,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(trade_request_id, document_type)
);

-- Document triggers reference table
CREATE TABLE document_triggers_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```



```

document_type TEXT NOT NULL,
trigger_type TEXT NOT NULL, -- 'SHIPMENT', 'SETTLEMENT', 'CUSTOMS', 'FINANCING'
is_default BOOLEAN DEFAULT false,
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(document_type, trigger_type)
);

-- Document format requirements per jurisdiction
CREATE TABLE document_format_requirements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
document_type TEXT NOT NULL,
originals_required BOOLEAN DEFAULT false,
electronic_accepted BOOLEAN DEFAULT true,
required_languages TEXT[],
physical_handling_recommended BOOLEAN DEFAULT false,
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(country_code, document_type)
);

-- Document verification logs
CREATE TABLE document_verification_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
verification_type TEXT NOT NULL, -- 'AUTOMATIC', 'MANUAL'
verification_status TEXT NOT NULL, -- 'PENDING', 'VERIFIED', 'REJECTED', 'FLAGGED'
confidence_score DECIMAL(5,2),
extracted_fields JSONB,
discrepancies JSONB,
fraud_indicators JSONB,
verified_by UUID REFERENCES employees(id) ON DELETE SET NULL,
verified_at TIMESTAMPTZ DEFAULT NOW(),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes

```

```

CREATE INDEX idx_document_requirements_trade ON document_requirements(trade_request_id);
CREATE INDEX idx_document_requirements_type ON document_requirements(document_type);
CREATE INDEX idx_document_requirements_status ON document_requirements(status);
CREATE INDEX idx_document_requirements_triggers ON document_requirements USING GIN(triggers);
CREATE INDEX idx_document_triggers_ref_type ON document_triggers_ref(document_type);
CREATE          INDEX          idx_document_format_requirements_country          ON
document_format_requirements(country_code);
CREATE          INDEX          idx_document_verification_logs_document          ON
document_verification_logs(document_id);
CREATE          INDEX          idx_document_verification_logs_status          ON
document_verification_logs(verification_status);

```

4.5.9 Document Triggers — Technical Reference

4.5.9.1 Trigger Enumeration

rust

```

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum DocumentTrigger {
    #[serde(rename = "SHIPMENT")]
    Shipment,
    #[serde(rename = "SETTLEMENT")]
    Settlement,
    #[serde(rename = "CUSTOMS")]
    Customs,
    #[serde(rename = "FINANCING")]
    Financing,
}

impl DocumentTrigger {
    pub fn display_name(&self) -> &'static str {
        match self {
            DocumentTrigger::Shipment => "Before or during shipment",
            DocumentTrigger::Settlement => "Before payment",
            DocumentTrigger::Customs => "For customs clearance",
            DocumentTrigger::Financing => "For financing",
        }
    }

    pub fn priority(&self) -> u8 {
        match self {

```

```

DocumentTrigger::Shipment => 4, // Highest — must be ready before loading
DocumentTrigger::Customs => 3, // Must be ready at arrival
DocumentTrigger::Settlement => 2, // Must be ready before payment
DocumentTrigger::Financing => 1, // Must be ready before disbursement
}
}
}

```

4.5.9.2 Trigger Logic

rust

// Pseudo-code: Document trigger validation

```

fn validate_document_trigger(document_type: &str, trigger: DocumentTrigger) -> Result<()> {
// 1. Check if document is required for this trigger
let requirements = get_document_requirements(document_type, trigger)?;
// 2. Check if document has been uploaded
let document = get_document_by_type_and_trade(trade_id, document_type)?;
// 3. Check if document is verified
if !document.is_verified() {
return Err(Error::DocumentNotVerified(document_type));
}
// 4. Check if document expires before trigger date
if document.expires_at < trigger_date {
return Err(Error::DocumentExpired(document_type));
}
Ok(())
}

```

4.5.10 Governor Gates (Part 4.5)

Example CONDITIONAL Response (Missing Document):

json

```

{
"verdict": "CONDITIONAL",
"tenant_message": {
"summary": "Your trade request is on hold because Phytosanitary Certificate is mandatory for this trade
but has not been added to the document checklist.",
"conditions": [
{
"condition_id": "missing_document",

```

```
"label": "Add Phytosanitary Certificate to document requirements",  
"action_url": "/v1/trade/request/draft?trade_id=..."  
}  
],  
"escalation_hint": "If you believe this is an error, request a human review."  
}  
}
```

4.5.11 Implementation Checklist (Part 4.5)

4.5.11.1 Backend Services

Implement document requirements service (Rust)

Implement RIA pre-selection logic

Implement document verification service (A2, HF Donut)

Implement fraud detection service (A2)

Implement document format validation

Implement document trigger validation

Implement document expiry monitoring

4.5.11.2 Frontend Components

Build Document Requirements checklist component

Build Document Format & Language component

Build Document Status tracking component

Build Document Upload component with verification progress

Build Document Verification Results display

Build Fraud Alert display

Build Document Expiry warning component

4.5.11.3 RIA Integration

Implement document pre-selection from RIA

Implement document triggers lookup

Implement document format requirements per jurisdiction

Implement source regulation tracking

4.5.11.4 Document Service Integration

Integrate with Document Service for uploads

Integrate with Verification Service for AI verification

Integrate with Fraud Detection Service

Integrate with Notification Service for document expiry alerts

4.5.11.5 Testing



4.6.2 AI-Assisted Suggestion System (A1, Groq)

4.6.2.1 Suggestion Generation

The system generates AI suggestions based on:

Product type (HS code, commodity category)

Origin country (export regulations, cultural requirements)

Destination country (import regulations, cultural requirements)

Incoterm (responsibilities, logistics requirements)

Transport mode (specific handling requirements)

Previous trades (buyer's historical instructions)

Prompt Template:

text

Based on the following trade parameters, suggest common special instructions:

Product: {product_name} (HS {hs_code})

Origin: {origin_country}

Destination: {destination_country}

Incoterm: {incoterm}

Transport: {transport_mode}

Generate 5-10 specific, actionable instructions that are commonly used for this trade combination. Categorize them by type (Labeling, Packaging, Certifications, Logistics, Documentation, Inspection, Dispute).

Return as a structured JSON object with categories and suggestions.

4.6.2.2 Suggestion Categories

4.6.2.3 Example Suggestions

Example: Fresh Oranges (Egypt → Japan)

json

```
{
  "suggestions": {
    "labeling": [
      "Japanese-language labels mandatory on all cartons.",
      "Country of origin: 'Product of Egypt' clearly marked.",
      "Barcode (GS1-128) required on each carton."
    ],
    "packaging": [
      "No wooden pallets (ISPM 15 compliant only).",
```



```

"Temperature logger required in each container.",
"Humidity indicator cards required."
],
"certifications": [
"Phytosanitary certificate required.",
"Cold treatment certificate required.",
"Certificate of Origin (Arab-Egyptian) required."
],
"documentation": [
"Original Bill of Lading to be sent by courier.",
"Certified translation in Japanese required."
],
"inspection": [
"Third-party inspection (SGS) required.",
"Inspection must be witnessed by buyer's representative."
]
}
}

```

4.6.3 Structured Extraction (A2, HF Local)

4.6.3.1 Extraction Process

When the buyer enters special instructions, the system automatically extracts structured data:

Text is parsed using HF NLP (A2).

Key entities are extracted:

Requirements (e.g., "Halal certification")

Conditions (e.g., "must be witnessed")

Parties (e.g., "buyer's representative")

Deadlines (e.g., "within 48 hours")

Quantities (e.g., "each container")

Instructions are categorised (Labeling, Packaging, Certifications, Logistics, Documentation, Inspection, Dispute).

Missing information is flagged (e.g., language requirement not specified).

Confidence scores are assigned to each extraction.

4.6.3.2 Extraction Example

json

```
{
```

"raw_text": "Arabic labels mandatory on all cartons. No wooden pallets (ISPM 15 compliant only). Halal certification required. Temperature logger required in each container.",

"structured_instructions": [

{

"category": "Labeling",

"instruction": "Arabic labels mandatory on all cartons.",

"confidence": 0.95,

"entities": {

"label_language": "Arabic",

"label_location": "all cartons"

}

},

{

"category": "Packaging",

"instruction": "No wooden pallets (ISPM 15 compliant only).",

"confidence": 0.92,

"entities": {

"packaging_material": "No wooden pallets",

"standard": "ISPM 15"

}

},

{

"category": "Certifications",

"instruction": "Halal certification required.",

"confidence": 0.98,

"entities": {

"certification_type": "Halal",

"requirement": "required"

}

},

{

"category": "Logistics",

"instruction": "Temperature logger required in each container.",

"confidence": 0.88,

"entities": {

"equipment": "Temperature logger",

```

"location": "each container"
}
},
"missing_information": [
{
"field": "Insurance requirements",
"reason": "Not specified in instructions"
},
{
"field": "Language requirement",
"reason": "Not specified in instructions"
}
],
"compliance_issues": [],
"sentiment": "neutral"
}

```

4.6.4 Categorization & Aggregation

4.6.4.1 Instruction Categories

Instructions are automatically categorised to improve visibility and downstream integration.

4.6.4.2 Aggregation Across Commodities

If multiple commodities exist in the trade, instructions are aggregated at the trade level with commodity-specific overrides.

text

TRADE-LEVEL INSTRUCTIONS (Applies to all commodities)

■■■■ Halal certification required.

■■■■ Temperature logger required in each container.

■■■■ No wooden pallets (ISPM 15 compliant only).

COMMODITY-SPECIFIC INSTRUCTIONS

■■■■ Oranges (HS 0805.10)

■ ■■■■ Cold treatment certificate required.

■■■■ Lemons (HS 0805.50)

■■■■ Fumigation certificate required.

4.6.5 Saved Templates (Buyer-Specific)

4.6.5.1 Template Management

Buyers can save commonly used instruction sets as templates to accelerate future trade creation.

Template Structure:

```
json
{
  "template_id": "TPL-EG-001",
  "template_name": "Egypt Import Template",
  "description": "Standard instructions for importing into Egypt",
  "instructions": [
    "Arabic labels mandatory on all cartons.",
    "No wooden pallets (ISPM 15 compliant only).",
    "Halal certification required.",
    "Original Bill of Lading to be sent by DHL.",
    "Third-party inspection required."
  ],
  "default_for": {
    "destination_country": "EG",
    "commodity_categories": ["Fresh Fruits", "Fresh Vegetables"]
  },
  "created_at": "2026-01-15T10:00:00Z",
  "updated_at": "2026-06-01T14:30:00Z",
  "usage_count": 47
}
```

UI Example:

text

SAVED TEMPLATES (Buyer-specific)

• Egypt Import Template (12 instructions) — Used 47 times

Standard instructions for importing into Egypt: Arabic labels, Halal,

DHL for B/L, third-party inspection.

[Load] [Edit] [Delete] [Set as Default]

■ ■ • ■ ■ UAE Import Template (8 instructions) — Used 23 times ■ ■

■ ■ Standard instructions for importing into UAE: English/Arabic labels, ■ ■

■ ■ SGS inspection, DIFC-LCIA arbitration. ■ ■

■ ■ [Load] [Edit] [Delete] [Set as Default] ■ ■

■ ■ • ■ ■ Germany Import Template (10 instructions) — Used 15 times ■ ■

■ ■ Standard instructions for importing into Germany: European labelling, ■ ■

■ ■ REACH compliance, EU organic certification. ■ ■

■ ■ [Load] [Edit] [Delete] [Set as Default] ■ ■

■ ■

■ [Create New Template] [Import Template] [Export Template] ■

4.6.6 Integration with Other Phases

4.6.7 Validation & Governor Gates

Example CONDITIONAL Response (Conflict):

json

 $\{$

```
"verdict": "CONDITIONAL",
```

```
"tenant_message": {
```

"summary": "Your special instructions conflict with the selected incoterm. You have specified 'Buyer arranges insurance' in special instructions, but the incoterm CIF requires the seller to arrange insurance.",

```
"conditions": [
```

 $\{$

```
"condition_id": "instruction_conflict",
```

```
"label": "Review and correct insurance instructions",
```

"action_url": "/v1/trade/request/draft?trade_id=..."

}

1,

```
"escalation_hint": "If you believe this is an error, request a human review."
```

}

}

4.6.8 Database Schema Additions (Part 4.6)

sql

-- Special instructions (free-text + structured)

```
CREATE TABLE special_instructions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  raw_text TEXT, -- The original free-text input  
  structured_instructions JSONB, -- AI-extracted structured data  
  categories JSONB, -- Categorized instructions per category  
  missing_information JSONB, -- What's missing  
  compliance_issues JSONB, -- Potential compliance flags  
  confidence_score DECIMAL(5,2), -- AI confidence  
  validated BOOLEAN DEFAULT false,  
  validated_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
  validated_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Saved instruction templates (buyer-specific)

```
CREATE TABLE instruction_templates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  buyer_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  template_name TEXT NOT NULL,  
  description TEXT,  
  instructions TEXT[],  
  default_for JSONB, -- { destination_country, commodity_categories, incoterms }  
  usage_count INTEGER DEFAULT 0,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(buyer_tenant_id, template_name)  
);
```

-- Instruction template usage logs

```
CREATE TABLE instruction_template_usage (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  template_id UUID NOT NULL REFERENCES instruction_templates(id) ON DELETE CASCADE,
```

```

trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
used_at TIMESTAMPTZ DEFAULT NOW()
);

-- Instruction propagation logs (to downstream services)
CREATE TABLE instruction_propagation_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
instruction_id UUID NOT NULL REFERENCES special_instructions(id) ON DELETE CASCADE,
target_service TEXT NOT NULL, -- 'SELLER', 'LOGISTICS', 'CUSTOMS', 'FINANCING'
propagated_at TIMESTAMPTZ DEFAULT NOW(),
status TEXT DEFAULT 'SUCCESS', -- 'SUCCESS', 'FAILED', 'PENDING'
error_message TEXT
);

-- Indexes
CREATE INDEX idx_special_instructions_trade ON special_instructions(trade_request_id);
CREATE INDEX idx_special_instructions_categories ON special_instructions USING GIN(categories);
CREATE INDEX idx_instruction_templates_buyer ON instruction_templates(buyer_tenant_id);
CREATE INDEX idx_instruction_templates_name ON instruction_templates(template_name);
CREATE INDEX idx_instruction_template_usage_template ON instruction_template_usage(template_id);
CREATE INDEX idx_instruction_propagation_logs_instruction ON
instruction_propagation_logs(instruction_id);

```

4.6.9 Implementation Checklist (Part 4.6)

4.6.9.1 Backend Services

- Implement special instructions service (Rust)
- Implement AI suggestion system (A1, Groq)
- Implement structured extraction (A2, HF local)
- Implement instruction categorisation (A2)
- Implement template management CRUD
- Implement instruction propagation to downstream services
- Implement conflict detection (A4, WasmEdge)

4.6.9.2 Frontend Components

- Build Special Instructions text area component
- Build AI Suggestions panel
- Build Categorised display component
- Build Template Management modal
- Build Missing Information warning component
- Build AI Analysis display

4.6.9.3 AI Integration

Implement Groq suggestion generation prompt

Implement HF NLP extraction pipeline

Implement confidence scoring

Implement categorisation logic

Implement missing information detection

4.6.9.4 Integration

Integrate with Seller Dashboard to display instructions

Integrate with Contract Service to include instructions

Integrate with Logistics Service to propagate instructions

Integrate with Document Service for documentation instructions

Integrate with Dispute Service for instruction compliance

4.6.9.5 Testing

Unit tests: instruction suggestion generation

Unit tests: structured extraction

Unit tests: categorisation

Integration tests: suggestion → extraction → categorisation

Integration tests: template CRUD

Integration tests: instruction propagation

End-to-end tests: full special instructions lifecycle

4.6.10 AI Authority Summary for Part 4.6

END OF PART 4.6 — SPECIAL TRADE INSTRUCTIONS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.7 — TRANSPORT & LOGISTICS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.7.0 Purpose & Design Philosophy

The Transport & Logistics section defines how the goods physically move from origin to destination. Unlike the original blueprint, which had a simplistic approach to transport mode and container preferences, this section provides a dynamic, context-aware transport configuration that adapts to the buyer's selected transport mode, commodity, and route.

Key Principles:

Dynamic equipment loading — Transport mode selection dynamically loads appropriate equipment types (containers for ocean, ULDs for air, trailers for truck, wagons for rail).

Route-aware — System validates that the selected mode and equipment are available for the origin-destination route.

Commodity-aware — Perishable goods require specific equipment (reefer, temperature-controlled).

Regulatory-aware — RIA provides port-specific rules, restrictions, and requirements.


```

origin: &str,
destination: &str,
commodity: &str,
weight_kg: f64,
urgency: Urgency
) -> Result<TransportMode> {
// 1. Check if route is served by ocean
if is_ocean_route(origin, destination) {
// 2. Check if commodity requires air (perishable, urgent)
if is_perishable(commodity) && urgency == Urgency::Urgent {
return Ok(TransportMode::Air);
}
// 3. Check if weight justifies ocean (bulk)
if weight_kg > 1000.0 {
return Ok(TransportMode::Ocean);
}
// 4. Default to ocean for international
return Ok(TransportMode::Ocean);
}
// 5. Check if route is served by rail
if is_rail_route(origin, destination) {
return Ok(TransportMode::Rail);
}
// 6. Default to truck for regional
if is_regional(origin, destination) {
return Ok(TransportMode::Truck);
}
// 7. Fallback to multimodal
Ok(TransportMode::Multimodal)
}

```

4.7.2 Dynamic Equipment Loading

4.7.2.1 Ocean — Container Types

When Ocean is selected, the system displays container types.

Container Types:

UI Example (Ocean Mode):

□ □

■ ■ ■ Open Top ■ ■ Flat Rack ■ ■ Tank Container ■ ■

■ ■ ■ ■

■ ■ Estimated total capacity: 2 × 40ft Reefer = 135.4 cbm ■ ■

□ □ □ □

When Air is selected, the system displays ULD (Unit Load Device) types.

UI Example (Air Mode):

■ ■

■ ■

■ ■ ■ Pallet (96×125) ■ ■ Pallet High ■ ■ LD3 Container ■ ■ LD6 Container ■ ■


```
if !is_port_active(origin) {
```

```

return Err(Error::PortInactive(origin));
}
if !is_port_active(destination) {
return Err(Error::PortInactive(destination));
}
// 3. Check sanctions clearance
if is_sanctioned(origin) || is_sanctioned(destination) {
return Err(Error::SanctionedPort);
}
// 4. Check mode availability
if !is_mode_available(origin, destination, mode) {
return Err(Error::ModeUnavailable(mode));
}
// 5. Check equipment availability
if !is_equipment_available(origin, destination, equipment) {
return Err(Error::EquipmentUnavailable(equipment));
}
// 6. Get transit time
let transit_time = get_transit_time(origin, destination, mode);
if transit_time.is_none() {
return Err(Error::TransitTimeUnknown);
}
// 7. Get vessel/vehicle availability
let availability = get_availability(origin, destination, mode);
Ok(RouteValidation {
origin,
destination,
mode,
equipment,
transit_time,
availability,
is_valid: true,
})
}

```

4.7.4 Port Congestion Detection (A2, RIA)

4.7.4.1 Congestion Monitoring

The RIA monitors port congestion via:

AIS data (vessel positions, waiting times)

Port authority websites (scraped)

Historical congestion patterns

Congestion Alert Example:

text

■ ■ ■ PORT CONGESTION ALERT ■



■ Alexandria Port is currently experiencing congestion. ■



■ Waiting Time: 3-5 days (normal: 1-2 days) ■

■ Vessels at Anchor: 12 ■

■ Estimated Delay: +3 days ■

■ ■

■ Recommended: Consider alternative port (Damietta or Port Said) or ■

■ adjust delivery window. ■

■ ■

■ [View Alternative Ports] [Adjust Delivery Window] [Dismiss] ■

4.7.4.2 Port Recommendation (A1, Groq)

If the selected port is congested, the system recommends alternatives.

text

■ ■ AI PORT RECOMMENDATION ■ ■

■ ■

■ Alexandria Port is congested (3-5 day delay). ■

■ ■

Alternative Ports:

■ ■ Damietta ■ +2 days transit ■ Available ■ Saves: 1 day delay ■ ■

■ ■ Port Said ■ +1 day transit ■ Available ■ Saves: 2 days delay ■ ■

```
// 2. Check chronological order
```


4.7.6.1 Port Information

When a port is selected, the system displays relevant information:

text

■ PORT INFORMATION — Alexandria (EG) ■

■ ■

UN/LOCODE: EGALY

Country: Egypt (EG)

■ ■ Latitude: 31.2000°N Longitude: 29.9500°E ■ ■

■ ■ ■ ■

■ ■ STATISTICS ■ ■

■ ■ • Annual TEU: 1.5 million ■ ■

■ ■ • Vessel calls: 1,200 per year ■ ■

■ ■ • Current congestion: Moderate (3-5 day waiting) ■ ■

4444

■ ■ REQUIREMENTS ■ ■

■ ■ • Pre-cooling certificate required for reefer ■ ■

■ ■ • ISPM 15 compliance required for wood packaging ■ ■

■ ■ • GATE IN: 24/7 operation, 48-hour pre-advice ■ ■

■■■■

■ ■ [View Port Map] [Check Port Schedule] [View Demurrage Rates] ■ ■

4.7.6.2 Port Requirements (RIA-Driven)

4.7.7 AI Container Advisor (A1, Advisory)

4.7.7.1 Overview

The AI Container Advisor suggests optimal container configuration based on:

Commodity volume and weight

Pallet dimensions

Product type (perishable, hazardous, etc.)

Route and transport mode

Historical loading density (anonymised, differential privacy)

4.7.7.2 UI Example

text

■ ■ AI CONTAINER ADVISOR (Advisory) ■ ■

■ ■

■ Based on your input, we suggest the following container configuration: ■

■ ■

■ Commodity: Oranges (22 pallets, 8,800 kg) ■

■ Pallet Type: EUR (800×1200 mm) ■



■ ■ Recommendation: 1 × 40ft High-Cube Reefer ■ ■

■ ■ ■ ■

Reason:

■ ■ • 22 EUR pallets fit perfectly in 1 x 40ft Reefer ■ ■

■ ■ • Perishable goods require reefeer ■ ■

■ ■ • High-cube provides extra volume for air circulation ■ ■

■ ■ • Historical loading density for this route: 92% ■ ■

■ ■ ■ ■

■ ■ Comparison: ■ ■

Option	Cost	Transit	Capacity	Recommendation
1	100	100	100	100
2	100	100	100	100
3	100	100	100	100
4	100	100	100	100
5	100	100	100	100
6	100	100	100	100
7	100	100	100	100
8	100	100	100	100
9	100	100	100	100
10	100	100	100	100
11	100	100	100	100
12	100	100	100	100
13	100	100	100	100
14	100	100	100	100
15	100	100	100	100
16	100	100	100	100
17	100	100	100	100
18	100	100	100	100
19	100	100	100	100
20	100	100	100	100
21	100	100	100	100
22	100	100	100	100
23	100	100	100	100
24	100	100	100	100
25	100	100	100	100
26	100	100	100	100
27	100	100	100	100
28	100	100	100	100
29	100	100	100	100
30	100	100	100	100
31	100	100	100	100
32	100	100	100	100
33	100	100	100	100
34	100	100	100	100
35	100	100	100	100
36	100	100	100	100
37	100	100	100	100
38	100	100	100	100
39	100	100	100	100
40	100	100	100	100
41	100	100	100	100
42	100	100	100	100
43	100	100	100	100
44	100	100	100	100
45	100	100	100	100
46	100	100	100	100
47	100	100	100	100
48	100	100	100	100
49	100	100	100	100
50	100	100	100	100
51	100	100	100	100
52	100	100	100	100
53	100	100	100	100
54	100	100	100	100
55	100	100	100	100
56	100	100	100	100
57	100	100	100	100
58	100	100	100	100
59	100	100	100	100
60	100	100	100	100
61	100	100	100	100
62	100	100	100	100
63	100	100	100	100
64	100	100	100	100
65	100	100	100	100
66	100	100	100	100
67	100	100	100	100
68	100	100	100	100
69	100	100	100	100
70	100	100	100	100
71	100	100	100	100
72	100	100	100	100
73	100	100	100	100
74	100	100	100	100
75	100	100	100	100
76	100	100	100	100
77	100	100	100	100
78	100	100	100	100
79	100	100	100	100
80	100			

■ ■ ■ 1 x 40ft HC Reefer ■ \$4,200 ■ 14 days ■ 92% ■ ■ Recommended ■ ■ ■

■ ■ ■ 2 x 20ft Reefer ■ \$4,800 ■ 14 days ■ 78% ■ ■ Higher cost ■ ■ ■

■ ■ ■ 1 x 40ft Reefer ■ \$4,200 ■ 14 days ■ 85% ■ ■ ■ Lower capacity ■ ■ ■

■■■■

■ ■ [Adjust to 1 × 40ft HC Reefer] [Adjust to 2 × 20ft Reefer] [Keep Current] ■ ■

Non-Marketplace Safeguard: The Container Advisor uses anonymised aggregate data with differential privacy ($\epsilon=0.1$). It never says "other buyers used 3 containers" or suggests specific providers.

4.7.8 Integration with Other Phases

4.7.9 Governor Gates (Part 4.7)

Example CONDITIONAL Response (Invalid Delivery Window):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your delivery window is invalid. The earliest date must be before or equal to the preferred date, which must be before or equal to the latest date.",
    "conditions": [
      {
        "condition_id": "invalid_delivery_window",
        "label": "Correct the delivery window dates",
        "action_url": "/v1/trade/request/draft?trade_id=..."
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  }
}
```

4.7.10 Database Schema Additions (Part 4.7)

sql

-- Transport & Logistics configuration

```
CREATE TABLE transport_configuration (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
  transport_mode TEXT NOT NULL, -- 'OCEAN', 'AIR', 'RAIL', 'TRUCK', 'MULTIMODAL'
  equipment_type TEXT NOT NULL, -- Dynamic based on mode
  equipment_count INTEGER NOT NULL,
  origin_port TEXT,
  destination_port TEXT,
  alternative_ports TEXT[],
  earliest_delivery_date DATE NOT NULL,
  preferred_delivery_date DATE NOT NULL,
  latest_delivery_date DATE NOT NULL,
```

```

transit_time_days INTEGER,
route_details JSONB,
port_requirements JSONB,
congestion_alert JSONB,
container_advisor_recommendation JSONB,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Port information (reference table, updated by RIA)
CREATE TABLE port_information (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
unlocode TEXT UNIQUE NOT NULL,
name TEXT NOT NULL,
country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
latitude DECIMAL(10,8),
longitude DECIMAL(11,8),
is_active BOOLEAN DEFAULT true,
is_sanctioned BOOLEAN DEFAULT false,
timezone TEXT,
operating_hours TEXT,
max_draft_m DECIMAL(5,2),
annual_teu INTEGER,
vessel_calls_per_year INTEGER,
congestion_status TEXT DEFAULT 'NORMAL', -- 'NORMAL', 'MODERATE', 'SEVERE'
congestion_waiting_days INTEGER DEFAULT 0,
requirements JSONB,
facilities JSONB,
last_updated TIMESTAMPTZ DEFAULT NOW()
);

-- Equipment types (reference table)
CREATE TABLE equipment_types (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
mode TEXT NOT NULL, -- 'OCEAN', 'AIR', 'RAIL', 'TRUCK'
type_code TEXT NOT NULL,
type_name TEXT NOT NULL,

```

```

description TEXT,
max_payload_kg DECIMAL(10,2),
max_volume_cbm DECIMAL(10,2),
is_active BOOLEAN DEFAULT true,
requires_reefer BOOLEAN DEFAULT false,
requires_hazardous_handling BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(mode, type_code)
);

-- Route availability (reference table)
CREATE TABLE route_availability (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
origin_unlocode TEXT NOT NULL REFERENCES port_information(unlocode),
destination_unlocode TEXT NOT NULL REFERENCES port_information(unlocode),
mode TEXT NOT NULL, -- 'OCEAN', 'AIR', 'RAIL', 'TRUCK'
is_available BOOLEAN DEFAULT true,
transit_time_days INTEGER,
frequency_per_week INTEGER,
carriers TEXT[],
equipment_types TEXT[],
last_updated TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(origin_unlocode, destination_unlocode, mode)
);

-- AI Container Advisor logs
CREATE TABLE container_advisor_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
input_data JSONB,
recommendation JSONB,
alternatives JSONB,
selected_option TEXT,
user_accepted BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- Port congestion history (TimescaleDB hypertable)
CREATE TABLE port_congestion_history (
id BIGSERIAL,
unlocode TEXT NOT NULL REFERENCES port_information(unlocode),
congestion_status TEXT NOT NULL,
waiting_days INTEGER,
vessels_at_anchor INTEGER,
recorded_at TIMESTAMPTZ NOT NULL
);
SELECT create_hypertable('port_congestion_history', 'recorded_at');
-- Indexes
CREATE INDEX idx_transport_configuration_trade ON transport_configuration(trade_request_id);
CREATE INDEX idx_transport_configuration_mode ON transport_configuration(transport_mode);
CREATE INDEX idx_port_information_unlocode ON port_information(unlocode);
CREATE INDEX idx_port_information_country ON port_information(country_code);
CREATE INDEX idx_port_information_congestion ON port_information(congestion_status);
CREATE INDEX idx_equipment_types_mode ON equipment_types(mode);
CREATE INDEX idx_route_availability_origin ON route_availability(origin_unlocode);
CREATE INDEX idx_route_availability_destination ON route_availability(destination_unlocode);
CREATE INDEX idx_container_advisor_logs_trade ON container_advisor_logs(trade_request_id);
CREATE INDEX idx_port_congestion_history_unlocode ON port_congestion_history(unlocode);
CREATE INDEX idx_port_congestion_history_recorded ON port_congestion_history(recorded_at DESC);

```

4.7.11 Implementation Checklist (Part 4.7)

4.7.11.1 Backend Services

- Implement transport configuration service (Rust)
- Implement dynamic equipment loading based on mode
- Implement route validation (A4, WasmEdge)
- Implement port validation (A4, WasmEdge)
- Implement delivery window validation (A4, WasmEdge)
- Implement port congestion detection (A2, RIA)
- Implement AI Container Advisor (A1, Groq)

4.7.11.2 Frontend Components

- Build Transport Mode selector (with icons)
- Build Dynamic Equipment Type selector (mode-aware)
- Build Route Information display

Build Port Congestion Alert component

Build Delivery Window date pickers

Build AI Container Advisor panel

Build Port Details component

4.7.11.3 RIA Integration

Implement port information scraping

Implement port congestion monitoring

Implement route availability updates

Implement equipment type updates

Implement port requirements lookup

4.7.11.4 Governor Integration

Add G1U3: Port existence validation

Add G1U4: Port sanctions validation

Add G1U13: Transport mode validation

Add G1U14: Equipment compatibility validation

Add G1U15: Equipment availability validation

Add G1U19: Delivery window validation

Add G2U17: Loading origin validation

Add G2U19: Alternative port validation

4.7.11.5 Testing

Unit tests: transport mode validation

Unit tests: equipment loading logic

Unit tests: delivery window validation

Integration tests: mode selection → equipment loading → validation

Integration tests: AI Container Advisor recommendations

Integration tests: port congestion alerts

End-to-end tests: full transport configuration workflow

4.7.12 AI Authority Summary for Part 4.7

END OF PART 4.7 — TRANSPORT & LOGISTICS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.8 — INSURANCE REQUIREMENTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.8.0 Purpose & Design Philosophy

The Insurance Requirements section defines the insurance arrangements for the trade shipment. Insurance is a critical component of international trade — it protects against loss, damage, and liability during transit. Unlike the original blueprint, which treated insurance as an afterthought, this section

■■■■ According to CIF incoterm, the seller is responsible for arranging ■■

Insurance.

INSURANCE TYPE (if Required)

[All Risks ▼] (Covers all risks of loss or damage)

COVERAGE AMOUNT

[110]% of invoice value

Standard coverage is 110% of invoice value (100% goods + 10% profit).

COVERAGE CURRENCY

[USD ▼]

DESTINATION-SPECIFIC REQUIREMENTS (RIA-Detected)

Destination: Germany

• Cargo insurance is recommended for all imports.

• Minimum coverage: 100% of invoice value.

• Institute Cargo Clauses (A) recommended for food products.

[Apply Recommendations] [Dismiss]

POLICY DETAILS (Optional — for later)

Policy Number: [_____]

Insurer: [_____]

Policy Effective Date: [_____]

■ ■ ■ ■

4444

■ ■ required for trade request submission. ■ ■

XXXXXXXXXXXXXXXXXXXXXXXXXXXX

[illegible]

4.8.3 RIA-Driven Insurance Requirements

4.8.3.1 Destination-Specific Requirements

The RIA maintains destination-specific insurance requirements:

4.8.3.2 Commodity-Specific Requirements

4.8.4 Insurance Certificate Verification (A2)

4.8.4.1 Verification Process

When the buyer (or seller) uploads an insurance certificate, the system verifies it using AI (A2, HF Donut).

Verification Steps:

Document is uploaded.

System extracts key fields using HF Donut.

System validates:

Policy number

Insurer name

Insured party (matches buyer/seller)

Vessel/voyage (matches shipment)

Coverage amount ($\geq$ required)

Coverage type (matches selected type)

Effective and expiry dates

System cross-references with insurer database (if available).

System returns verification status.

Verification Result:

json

```
{
  "document_id": "INS-2026-001",
  "document_type": "INSURANCE_CERTIFICATE",
  "verification_status": "VERIFIED",
  "confidence_score": 0.92,
  "extracted_fields": {
    "policy_number": "POL-2026-001",
    "insurer": "Allianz Egypt",
    "insured_party": "Mekong Fresh Co.",
    "vessel": "MSC FIONA",
    "coverage_amount_usd": 105000,
    "coverage_type": "All Risks",
    "effective_date": "2026-06-15",
    "expiry_date": "2026-12-15"
```

```

},
"validation_results": {
  "policy_number": "VALID",
  "insurer": "VERIFIED",
  "insured_party": "MATCH",
  "vessel": "MATCH",
  "coverage_amount": "SUFFICIENT (105,000 ≥ 100,000)",
  "coverage_type": "MATCH",
  "dates": "VALID"
},
"fraud_indicators": [],
"verification_timestamp": "2026-06-20T10:30:00Z"
}

```

4.8.4.2 Fraud Detection

The system flags potential fraud:

Invalid policy number — Not recognised by insurer.

Expired policy — Policy has expired.

Mismatched insured party — Insured party does not match buyer/seller.

Mismatched vessel — Vessel does not match shipment.

Insufficient coverage — Coverage amount is below minimum required.

Fabricated document — Signs of tampering or forgery.

4.8.5 Integration with Other Phases

| Phase | How Insurance Requirements Flow |

|:---|:---|:---|

| Phase 2 (Seller Quote) | Seller sees insurance requirements; if responsible, adds insurance cost to quote. |

| Phase 3 (Contract) | Insurance requirements are incorporated into the contract. |

| Phase 4 (Financing) | Financiers check insurance coverage; may require assignment of insurance. |

| Phase 5 (Execution) | Insurance certificate is verified; coverage is confirmed before loading. |

| Phase 6 (Settlement) | Insurance coverage is confirmed before payment. |

| Phase 7 (Distressed Cargo) | Insurance claim workflow is triggered if cargo is distressed. |

| Phase 8 (Dispute) | Insurance coverage is part of the evidence package. |

4.8.6 Governor Gates (Part 4.8)

Example CONDITIONAL Response (Missing Insurance):

json

```
{
```

```

"verdict": "CONDITIONAL",
"tenant_message": {
  "summary": "Your trade request is on hold because insurance is required for CIF incoterm. Please specify the insurance requirements.",
  "conditions": [
    {
      "condition_id": "insurance_required",
      "label": "Specify insurance requirements",
      "action_url": "/v1/trade/request/draft?trade_id=..."
    }
  ],
  "escalation_hint": "If you believe this is an error, request a human review."
}

```

4.8.7 Insurance Types Reference

4.8.8 Database Schema Additions (Part 4.8)

sql

-- Insurance requirements

```

CREATE TABLE insurance_requirements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
  requirement TEXT NOT NULL, -- 'REQUIRED', 'OPTIONAL', 'NOT_REQUIRED'
  responsible_party TEXT NOT NULL, -- 'BUYER', 'SELLER', 'ACCORDING_TO_INCOTERM'
  insurance_type TEXT, -- 'ALL_RISKS', 'FPA', 'WA', 'ICC_A', 'ICC_B', 'ICC_C', 'REEFER', 'LIVESTOCK', 'WAR', 'STRIKES'
  coverage_percent INTEGER DEFAULT 110,
  coverage_currency TEXT DEFAULT 'USD',
  policy_number TEXT,
  insurer_name TEXT,
  policy_effective_date DATE,
  policy_expiry_date DATE,
  certificate_uploaded BOOLEAN DEFAULT false,
  certificate_document_id UUID REFERENCES documents(id) ON DELETE SET NULL,
  verification_status TEXT DEFAULT 'PENDING', -- 'PENDING', 'VERIFIED', 'REJECTED', 'FLAGGED'
  verification_result JSONB,
  auto_configured BOOLEAN DEFAULT false,

```

```

auto_config_reason TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Insurance type reference table
CREATE TABLE insurance_types_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
type_code TEXT UNIQUE NOT NULL,
type_name TEXT NOT NULL,
type_name_ar TEXT,
description TEXT,
description_ar TEXT,
coverage_level TEXT, -- 'BROAD', 'MEDIUM', 'LIMITED', 'SPECIALISED'
typical_use TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Country-specific insurance requirements (RIA-maintained)
CREATE TABLE country_insurance_requirements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
commodity_category TEXT,
hs_code CHAR(6),
requirement TEXT NOT NULL, -- 'REQUIRED', 'RECOMMENDED', 'OPTIONAL'
min_coverage_percent INTEGER DEFAULT 100,
recommended_type TEXT,
source_regulation TEXT,
last_updated TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(country_code, commodity_category, hs_code)
);

-- Insurance certificate verification logs
CREATE TABLE insurance_verification_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
insurance_requirement_id UUID NOT NULL REFERENCES insurance_requirements(id) ON DELETE
CASCADE,

```

```

verification_type TEXT NOT NULL, -- 'AUTOMATIC', 'MANUAL'
verification_status TEXT NOT NULL, -- 'PENDING', 'VERIFIED', 'REJECTED', 'FLAGGED'
confidence_score DECIMAL(5,2),
extracted_fields JSONB,
validation_results JSONB,
fraud_indicators JSONB,
verified_by UUID REFERENCES employees(id) ON DELETE SET NULL,
verified_at TIMESTAMPTZ DEFAULT NOW(),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes

CREATE INDEX idx_insurance_requirements_trade ON insurance_requirements(trade_request_id);
CREATE INDEX idx_insurance_requirements_status ON insurance_requirements(verification_status);
CREATE INDEX idx_insurance_requirements_responsible ON
insurance_requirements(responsible_party);
CREATE INDEX idx_country_insurance_requirements_country ON
country_insurance_requirements(country_code);
CREATE INDEX idx_country_insurance_requirements_hs ON country_insurance_requirements(hs_code);
CREATE INDEX idx_insurance_verification_logs_requirement ON
insurance_verification_logs(insurance_requirement_id);
CREATE INDEX idx_insurance_verification_logs_status ON
insurance_verification_logs(verification_status);

```

4.8.9 Implementation Checklist (Part 4.8)

4.8.9.1 Backend Services

- Implement insurance requirements service (Rust)
- Implement incoterm-driven auto-configuration
- Implement RIA-driven insurance requirements lookup
- Implement insurance type reference service
- Implement insurance certificate verification (A2, HF Donut)
- Implement insurance fraud detection (A2)

4.8.9.2 Frontend Components

- Build Insurance Requirement selector (Required/Optional/Not Required)
- Build Responsible Party selector
- Build Insurance Type dropdown
- Build Coverage Amount input
- Build Coverage Currency dropdown
- Build Policy Details section (optional fields)

Build Insurance Certificate upload component

Build Verification Status display

Build Fraud Alert display

4.8.9.3 RIA Integration

Implement country-specific insurance requirements

Implement commodity-specific insurance requirements

Implement auto-configuration based on RIA data

Implement Smart Inbox alerts for missing insurance

4.8.9.4 Governor Integration

Add G1U20: Insurance requirement selected

Add G1U20a: If Required, insurance type selected

Add G1U20b: If Required, coverage amount $\geq 100\%$

Add G1U20c: Incoterm compatibility (CIF/CIP require insurance)

Add G5U7: Insurance certificate verified before loading

4.8.9.5 Testing

Unit tests: incoterm-driven auto-configuration

Unit tests: RIA-driven insurance requirements lookup

Unit tests: insurance certificate verification

Integration tests: incoterm selection → insurance auto-configuration

Integration tests: insurance certificate upload → verification

Integration tests: fraud detection

End-to-end tests: full insurance workflow

4.8.10 AI Authority Summary for Part 4.8

END OF PART 4.8 — INSURANCE REQUIREMENTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.9 — COMMERCIAL SETTLEMENT REQUIREMENTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.9.0 Purpose & Design Philosophy

The Commercial Settlement Requirements section is the commercial foundation of the trade request. It defines how the trade will be settled financially — the payment structure, timing, credit terms, currency, financing interest, and settlement flexibility. Unlike the original blueprint, which treated settlement as an afterthought, this section provides a comprehensive, decision-ready framework that reflects real-world trade practices and enables downstream financing, negotiation, and contract execution.

Key Principles:

Reflects actual trade structures — Settlement options mirror real-world trade instruments (LC, Documentary Collection, Bank Transfer, Open Account).

4.9.4 Preferred Credit Period

Description: Defines the credit period after delivery or document presentation.

Fields: Dropdown.

Options:

0 Days

15 Days

30 Days

45 Days

60 Days

90 Days

Custom (free-text)

Why It Matters: Useful for trade finance and cash flow planning.

UI Example:

text

PREFERRED CREDIT PERIOD

What credit period do you prefer?

[30 Days ▼]

30 days is standard for many industries. Longer periods may require

financing or higher pricing.

If Custom is selected:

[45 Days]

4.9.5 Currency

Description: The currency in which the trade will be settled.

Fields: Dropdown.

Options:

USD

EUR

GBP

$\}$

■ ■

Documentary Collection



■ Balanced ■

■ ■

■ Low (Seller indicated no financing required) ■

■ 92% (11 of 12 required documents specified) ■

■ ■

■ ■ Inspection Certificate missing from settlement documents ■

22

■ "Based on your commercial priority (Balanced) and selected structure ■

■ Inspection Certificate is missing from settlement documents. Banks ■

■ typically require this for documentary collections. Add it to avoid ■

■ delays." ■

11

■ [Add Missing Document] [Dismiss] ■

| Phase | How Settlement Requirements Flow |

$$\begin{vmatrix} \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots \end{vmatrix}$$

| Phase 2 (Seller Quote) | Seller sees settlement requirements; adjusts pricing and terms accordingly. |

| Phase 3 (Contract) | Settlement terms are incorporated into the contract. |

| Phase 4 (Financing) | Financiers assess settlement structure for risk; financing interest informs auto-RFO. |

| Phase 5 (Execution) | Settlement documents are prepared and verified. |

| Phase 6 (Settlement) | Settlement instruction is generated based on selected structure and timing. |

| Phase 7 (Distressed Cargo) | Settlement flexibility affects distress handling options. |

| Phase 8 (Dispute) | Settlement terms are part of the evidence package. |

4.9.13 Governor Gates (Part 4.9)

Example CONDITIONAL Response (Missing Settlement Flexibility):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your trade request is on hold because Settlement Flexibility has not been selected. Please indicate whether the settlement terms are Fixed, Flexible, or Open To Alternatives.",
    "conditions": [
      {
        "condition_id": "settlement_flexibility",
        "label": "Select Settlement Flexibility",
        "action_url": "/v1/trade/request/draft?trade_id=..."
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  }
}
```

Example CONDITIONAL Response (Missing Settlement Documents):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your trade request is on hold because Inspection Certificate is missing from the settlement documents. Banks typically require this for documentary collections.",
    "conditions": [
      {
        "condition_id": "settlement_documents",
        "label": "Add Inspection Certificate to settlement documents",
        "action_url": "/v1/trade/request/draft?trade_id=..."
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  }
}
```

```
}
```

4.9.14 Database Schema Additions (Part 4.9)

```
sql
```

```
-- Commercial settlement configuration
```

```
CREATE TABLE commercial_settlement (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  commercial_priority TEXT NOT NULL, -- 'LOWEST_COST', 'FASTEST_SETTLEMENT',  
  'FINANCING_FRIENDLY', 'BALANCED'  
  
  settlement_structure TEXT NOT NULL, -- 'DOCUMENTARY_CREDIT',  
  'DOCUMENTARY_COLLECTION', 'BANK_TRANSFER', 'OPEN_ACCOUNT', 'TO_BE_NEGOTIATED'  
  payment_timing TEXT NOT NULL, -- 'ADVANCE', 'PARTIAL_ADVANCE', 'AGAINST_DOCUMENTS',  
  'AGAINST_SHIPMENT', 'AGAINST_DELIVERY', 'DEFERRED', 'TO_BE_NEGOTIATED'  
  
  credit_period TEXT NOT NULL, -- '0_DAYS', '15_DAYS', '30_DAYS', '45_DAYS', '60_DAYS', '90_DAYS',  
  'CUSTOM'  
  
  credit_period_custom_days INTEGER,  
  currency TEXT NOT NULL DEFAULT 'USD',  
  currency_other TEXT,  
  financing_interest TEXT NOT NULL, -- 'BUYER', 'SELLER', 'EITHER_PARTY', 'NONE'  
  bank_instrument TEXT, -- 'LC', 'SBLC', 'DLC', 'BG', 'NONE', 'TO_BE_NEGOTIATED'  
  settlement_flexibility TEXT NOT NULL, -- 'FIXED', 'FLEXIBLE', 'OPEN_TO_ALTERNATIVES'  
  settlement_documents TEXT[], -- Array of document types  
  partial_advance_percentage INTEGER,  
  balance_timing TEXT, -- 'AGAINST_DOCUMENTS', 'AGAINST_SHIPMENT', 'AGAINST_DELIVERY'  
  acceptable_alternatives TEXT[],  
  conditions_for_alternatives TEXT,  
  auto_configured BOOLEAN DEFAULT false,  
  auto_config_reason TEXT,  
  ai_readiness_score INTEGER, -- 0-100  
  ai_readiness_recommendation TEXT, -- Groq-generated  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- Settlement structure reference table  
CREATE TABLE settlement_structures_ref (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  structure_code TEXT UNIQUE NOT NULL,
```

```

structure_name TEXT NOT NULL,
structure_name_ar TEXT,
description TEXT,
description_ar TEXT,
recommended_for TEXT, -- 'LOWEST_COST', 'FASTEST_SETTLEMENT', 'FINANCING_FRIENDLY',
'BALANCED'
typical_use TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Payment timing reference table
CREATE TABLE payment_timing_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
timing_code TEXT UNIQUE NOT NULL,
timing_name TEXT NOT NULL,
timing_name_ar TEXT,
description TEXT,
description_ar TEXT,
typical_use TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Bank instrument reference table
CREATE TABLE bank_instruments_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
instrument_code TEXT UNIQUE NOT NULL,
instrument_name TEXT NOT NULL,
instrument_name_ar TEXT,
description TEXT,
description_ar TEXT,
typical_use TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

);
-- Commercial priority reference table
CREATE TABLE commercial_priority_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
priority_code TEXT UNIQUE NOT NULL,
priority_name TEXT NOT NULL,
priority_name_ar TEXT,
description TEXT,
description_ar TEXT,
typical_use TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Settlement AI readiness logs
CREATE TABLE settlement_readiness_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
settlement_id UUID NOT NULL REFERENCES commercial_settlement(id) ON DELETE CASCADE,
readiness_score INTEGER NOT NULL,
completeness_percentage INTEGER,
missing_documents TEXT[],
potential_issues JSONB,
ai_recommendation TEXT,
groq_model_version TEXT,
calculated_at TIMESTAMPTZ DEFAULT NOW(),
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_commercial_settlement_trade ON commercial_settlement(trade_request_id);
CREATE INDEX idx_commercial_settlement_priority ON commercial_settlement(commercial_priority);
CREATE INDEX idx_commercial_settlement_structure ON commercial_settlement(settlement_structure);
CREATE INDEX idx_commercial_settlement_currency ON commercial_settlement(currency);
CREATE INDEX idx_settlement_readiness_logs_settlement ON
settlement_readiness_logs(settlement_id);
CREATE INDEX idx_settlement_readiness_logs_score ON settlement_readiness_logs(readiness_score);

```

4.9.15 Implementation Checklist (Part 4.9)

4.9.15.1 Backend Services

Implement commercial settlement service (Rust)

Implement commercial priority logic

Implement incoterm-driven auto-configuration

Implement commercial priority-driven recommendations

Implement settlement validation (A4, WasmEdge)

Implement settlement AI readiness (A1, Groq)

4.9.15.2 Frontend Components

Build Commercial Priority selector

Build Settlement Structure selector

Build Payment Timing selector

Build Credit Period selector

Build Currency selector

Build Financing Interest selector

Build Bank Instrument selector (conditional)

Build Settlement Flexibility selector

Build Documentary Requirements (Settlement) checkbox matrix

Build AI Settlement Readiness display

Build auto-configuration display

4.9.15.3 Integration

Integrate with incoterm selection for auto-configuration

Integrate with financing module for financing interest

Integrate with contract service for settlement terms

Integrate with payment orchestrator for settlement instruction generation

Integrate with document service for settlement documents

4.9.15.4 Governor Integration

Add G1U9: Settlement Structure validation

Add G1U10: Payment Timing validation

Add G1U11: Credit Period validation

Add G1U12: Currency validation

Add G1U13: Financing Interest validation

Add G1U14: Bank Instrument validation

Add G1U15: Settlement Flexibility validation

Add G1U16: Settlement Documents validation

Add G1U17: Commercial Priority validation

4.9.15.5 Testing

Unit tests: incoterm-driven auto-configuration

Unit tests: commercial priority-driven recommendations

Unit tests: settlement validation

Integration tests: incoterm selection → settlement auto-configuration

Integration tests: commercial priority → settlement recommendations

Integration tests: settlement readiness AI

End-to-end tests: full settlement workflow

4.9.16 AI Authority Summary for Part 4.9

END OF PART 4.9 — COMMERCIAL SETTLEMENT REQUIREMENTS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.10 — TRADE REQUEST READINESS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.10.0 Purpose & Design Philosophy

The Trade Request Readiness section provides a non-blocking, advisory assessment of the completeness and quality of the trade request before submission. Unlike a blocking validation (which prevents submission), readiness is a guidance tool that helps buyers identify missing information, potential issues, and opportunities for improvement before the request reaches the seller.

Key Principles:

Non-blocking advisory — The score does not prevent submission; it guides improvement.

Completeness-focused — Evaluates which sections are complete, partially complete, or missing.

Quality-aware — Assesses not just presence but quality (e.g., unrealistic delivery window, vague instructions).

AI-enhanced — Uses AI (A2, XGBoost) to calculate score and identify missing items.

Actionable — Each missing or incomplete item has a direct action link.

Continuous improvement — Score updates in real-time as the buyer completes sections.

Non-marketplace — Never suggests alternative counterparties or service providers.

AI Integration in Part 4.10:

4.10.1 Readiness Scoring Logic (A2, XGBoost)

4.10.1.1 Scoring Components

The Trade Request Readiness score is calculated as a weighted composite of all sections:

4.10.1.2 Score Calculation

rust

// Pseudo-code: Trade Request Readiness Score calculation

```
fn calculate_readiness_score(trade_request: &TradeRequest) -> ReadinessResult {
```

```
    let mut scores = HashMap::new();
```

```

let mut missing_items = Vec::new();
let mut incomplete_items = Vec::new();
// 1. Seller Selection (5%)
if trade_request.seller_gtid.is_some() {
scores.insert("seller", 100);
} else {
scores.insert("seller", 0);
missing_items.push("Seller not selected");
}
// 2. Incoterm Selection (5%)
if trade_request.incoterm.is_some() {
scores.insert("incoterm", 100);
} else {
scores.insert("incoterm", 0);
missing_items.push("Incoterm not selected");
}
// 3. Containers (10%)
if trade_request.containers.is_empty() {
scores.insert("containers", 0);
missing_items.push("No containers configured");
} else {
let complete_containers = trade_request.containers.iter()
.filter(|c| c.is_complete())
.count();
let container_score = (complete_containers as f64 / trade_request.containers.len() as f64) * 100.0;
scores.insert("containers", container_score);
if container_score < 100.0 {
incomplete_items.push("Some containers are incomplete");
}
}
// 4. Commodities (15%)
// ... similar logic for each component
// 5. AI quality assessment (XGBoost)
let quality_score = ai_readiness_engine.assess_quality(trade_request);
let ai_confidence = ai_readiness_engine.confidence();

```


■ ■ Insurance ■ 100% ■ ■ Complete ■ ■

■ ■ Commercial Settlement ■ 100% ■ ■ Complete ■ ■

■ ■ IMPROVEMENT SUGGESTIONS ■ ■

■ ■ ■ ■

■ ■ To reach 100% readiness: ■ ■

□ □ □ □

■ ■ ■■ 1 acceptance criteria parameter missing ■ ■

■ ■ • Foreign Matter limit not specified ■ ■

■ ■ [Add Parameter] ■ ■

■ ■ ■ ■

■ ■ ■■ 1 optional document not specified ■ ■

■ ■ • Insurance Certificate is optional but recommended ■ ■

■ ■ [Add Document] ■ ■

■ ■ ■ ■

■ ■ [Submit Anyway] [Complete Missing Items] [Save Draft] ■ ■

4.10.2.2 Complete UI Example — Low Readiness

text

■ 10. TRADE REQUEST READINESS ■

11

■ ■ ■ TRADE REQUEST READINESS: 62/100 ■ ■ ■

4 of 4

§ 87(2)(b) [REDACTED]
[REDACTED] § 87(2)(b)

62% ready

This trade request needs attention before submission.

• Missing sections: 2

• Incomplete sections: 3

• Quality issues: 2

AI Recommendation: Complete the missing and incomplete sections to

improve seller response probability.

READINESS BREAKDOWN

Component Score Status

Seller Selection 100% Complete

Incoterm Selection 100% Complete

Containers 50% Only 1 of 2 containers complete

Commodities 40% 2 of 5 commodities incomplete

Quantity & Tolerance 60% Tolerance not specified

Packaging & Weight 30% Missing for 3 commodities

Acceptance Criteria 0% Not specified

Documentation 20% Missing mandatory documents

Transport & Logistics 60% Delivery window incomplete

Insurance 0% Not specified

Commercial Settlement 40% Incomplete settlement terms

■ ■ ■ ■

4 of 4

■ ■ ■ ■

□ □ □ □

■ ■ ■ ■

4.10.3.2 Detection Logic

}


```

// 2. Incoterm
if trade_request.incoterm.is_none() {
items.push(MissingItem {
severity: MissingSeverity::Critical,
section: "Incoterm",
field: "Incoterm",
message: "Incoterm not selected",
action_url: "/v1/trade/request/draft?section=incoterm",
});
}

// 3. Containers
if trade_request.containers.is_empty() {
items.push(MissingItem {
severity: MissingSeverity::Critical,
section: "Containers",
field: "Containers",
message: "No containers configured",
action_url: "/v1/trade/request/draft?section=containers",
});
} else {
for (idx, container) in trade_request.containers.iter().enumerate() {
if container.port_of_discharge.is_empty() {
items.push(MissingItem {
severity: MissingSeverity::Warning,
section: "Containers",
field: format!("Container {}", idx + 1),
message: "Port of discharge not specified",
action_url: format!("/v1/trade/request/draft?section=container&id={}", idx),
});
}
}
}

// 4. Commodities & Quantity
for (idx, commodity) in trade_request.commodities.iter().enumerate() {
if commodity.hs_code.is_empty() {

```

```

items.push(MissingItem {
severity: MissingSeverity::Critical,
section: "Commodities",
field: format!("Commodity {}", idx + 1),
message: "HS code missing",
action_url: format!("/v1/trade/request/draft?section=commodity&id={}", idx),
});
}

if commodity.requested_quantity <= 0.0 {
items.push(MissingItem {
severity: MissingSeverity::Critical,
section: "Commodities",
field: format!("Commodity {}", idx + 1),
message: "Quantity not specified",
action_url: format!("/v1/trade/request/draft?section=quantity&id={}", idx),
});
}

if commodity.quantity_tolerance.is_none() {
items.push(MissingItem {
severity: MissingSeverity::Recommended,
section: "Commodities",
field: format!("Commodity {}", idx + 1),
message: "Tolerance not specified (recommended)",
action_url: format!("/v1/trade/request/draft?section=tolerance&id={}", idx),
});
}
}

// 5. Acceptance Criteria
// ... similar logic
// 6. Documentation
// ... similar logic
// 7. Transport & Logistics

if trade_request.transport_mode.is_empty() {
items.push(MissingItem {
severity: MissingSeverity::Critical,

```

```
section: "Transport & Logistics",
field: "Transport Mode",
message: "Transport mode not selected",
action_url: "/v1/trade/request/draft?section=transport",
});
}
```

// 8. Insurance

```
if trade_request.insurance_requirement.is_none() {
items.push(MissingItem {
severity: MissingSeverity::Critical,
section: "Insurance",
field: "Insurance Requirement",
message: "Insurance requirement not specified",
action_url: "/v1/trade/request/draft?section=insurance",
});
}
```

// 9. Commercial Settlement

```
if trade_request.settlement_structure.is_none() {
items.push(MissingItem {
severity: MissingSeverity::Warning,
section: "Commercial Settlement",
field: "Settlement Structure",
message: "Settlement structure not specified (recommended)",
action_url: "/v1/trade/request/draft?section=settlement",
});
}
```

// 10. Quality assessment (A2, XGBoost)

```
let quality_issues = ai_readiness_engine.assess_quality(trade_request);
for issue in quality_issues {
items.push(MissingItem {
severity: MissingSeverity::Quality,
section: issue.section,
field: issue.field,
message: issue.message,
action_url: issue.action_url,
```

```
});
```

```
}
```

```
items
```

```
}
```

4.10.4 AI Quality Assessment (A2, XGBoost)

4.10.4.1 Quality Dimensions

4.10.4.2 Quality Assessment Example

```
json
```

```
{
```

```
"quality_score": 0.85,
```

```
"confidence": 0.92,
```

```
"dimensions": {
```

```
"timeline_realism": {
```

```
"score": 0.90,
```

```
"status": "REALISTIC",
```

```
"details": "Delivery window of 14-20 days is realistic for ocean freight (Alexandria → Hamburg)"
```

```
},
```

```
"specification_clarity": {
```

```
"score": 0.95,
```

```
"status": "CLEAR",
```

```
"details": "Product specifications are clear and complete"
```

```
},
```

```
"tolerance_reasonableness": {
```

```
"score": 0.80,
```

```
"status": "REASONABLE",
```

```
"details": "Tolerance of ±5% is reasonable for fresh fruit"
```

```
},
```

```
"documentation_completeness": {
```

```
"score": 0.70,
```

```
"status": "INCOMPLETE",
```

```
"details": "Phytosanitary certificate missing from document requirements"
```

```
},
```

```
"settlement_alignment": {
```

```
"score": 0.95,
```

```
"status": "ALIGNED",
```

```

"details": "Documentary Collection is appropriate for CFR incoterm"
},
"insurance_appropriateness": {
"score": 0.60,
"status": "INAPPROPRIATE",
"details": "Insurance is recommended for CIF incoterm but not specified"
}
},
"issues": [
{
"severity": "WARNING",
"section": "Documentation",
"field": "Required Documents",
"message": "Phytosanitary certificate missing from document requirements",
"action_url": "/v1/trade/request/draft?section=documentation"
},
{
"severity": "WARNING",
"section": "Insurance",
"field": "Insurance Requirement",
"message": "Insurance is recommended for CIF incoterm but not specified",
"action_url": "/v1/trade/request/draft?section=insurance"
}
]
}

```

4.10.5 Integration with Other Phases

4.10.6 Governor Gates (Part 4.10)

Note: Readiness is advisory only. It does not block submission. The Governor displays a recommendation but allows the buyer to proceed.

4.10.7 Database Schema Additions (Part 4.10)

sql

-- Trade Request Readiness scores (stored for historical analysis)

```

CREATE TABLE trade_readiness_scores (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
readiness_score INTEGER NOT NULL, -- 0-100

```

```

quality_score DECIMAL(5,2), -- 0-1
ai_confidence DECIMAL(5,2), -- 0-1
components JSONB, -- Component scores
missing_items JSONB, -- Missing items list
incomplete_items JSONB, -- Incomplete items list
quality_issues JSONB, -- Quality issues list
groq_recommendation TEXT, -- A1-generated recommendation
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Missing items reference table
CREATE TABLE readiness_missing_items_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
item_key TEXT UNIQUE NOT NULL,
item_name TEXT NOT NULL,
section TEXT NOT NULL,
severity TEXT NOT NULL, -- 'CRITICAL', 'WARNING', 'RECOMMENDED', 'QUALITY'
default_message TEXT,
default_action_url TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Readiness quality rules (configurable by Admin)
CREATE TABLE readiness_quality_rules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
rule_type TEXT NOT NULL, -- 'TIMELINE', 'SPECIFICATION', 'TOLERANCE', 'DOCUMENTATION',
'SETTLEMENT', 'INSURANCE'
condition JSONB, -- JSON condition (e.g., {"incoterm": "CIF"})
message_template TEXT,
severity TEXT NOT NULL, -- 'WARNING', 'SUGGESTION'
is_active BOOLEAN DEFAULT true,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_trade_readiness_scores_request ON trade_readiness_scores(trade_request_id);
CREATE INDEX idx_trade_readiness_scores_score ON trade_readiness_scores(readiness_score);

```

CREATE INDEX idx_trade_readiness_scores_created ON trade_readiness_scores(created_at DESC);

4.10.8 Implementation Checklist (Part 4.10)

4.10.8.1 Backend Services

Implement Trade Request Readiness service (Rust)

Implement readiness score calculation (A2, XGBoost)

Implement missing item detection logic

Implement quality assessment (A2, XGBoost)

Implement AI recommendation generation (A1, Groq)

4.10.8.2 Frontend Components

Build Trade Request Readiness score component

Build Readiness Breakdown table

Build Missing Items list

Build Improvement Suggestions

Build Quality Issues list

Build Action links (one-click to fix)

Build Readiness status indicators

4.10.8.3 Quality Rules

Define quality rules for each dimension

Implement rule evaluation logic

Implement severity classification

Implement Admin configuration UI for rules

4.10.8.4 Integration

Integrate with all form sections for real-time score updates

Integrate with AI recommendation engine

Integrate with Smart Inbox for low-readiness alerts

4.10.8.5 Testing

Unit tests: readiness score calculation

Unit tests: missing item detection

Unit tests: quality assessment

Integration tests: real-time score updates

Integration tests: AI recommendation generation

End-to-end tests: full readiness workflow

4.10.9 AI Authority Summary for Part 4.10

END OF PART 4.10 — TRADE REQUEST READINESS

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.11 — TRADE CRITICALITY

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.11.0 Purpose & Design Philosophy

The Trade Criticality classification indicates the operational importance of the entire trade. Unlike the original blueprint's "Product Criticality" (which was commodity-specific), Trade Criticality applies at the trade request level and drives workflow routing, approval urgency, alerting, and escalation handling across the entire trade lifecycle.

Key Principles:

Trade-level classification — Applies to the entire trade, not individual commodities.

Workflow routing — Critical trades follow expedited workflows with higher priority.

Approval urgency — Higher criticality triggers faster approval routing and shorter SLAs.

Alerting & escalation — Critical trades generate higher-priority Smart Inbox alerts and faster escalation.

AI-assisted suggestion — AI (A1, Groq) suggests criticality based on commodity, value, destination, and urgency.

Non-marketplace — Never suggests alternative counterparties or service providers.

AI Integration in Part 4.11:

4.11.1 Trade Criticality Levels

4.11.1.1 Classification Levels

4.11.1.2 Operational Implications

4.11.2 AI-Assisted Criticality Suggestion (A1, Groq)

4.11.2.1 Suggestion Generation

The system generates an AI-suggested criticality based on:

Prompt Template:

text

Based on the following trade parameters, suggest a Trade Criticality classification:

Commodity: {product_name} (HS {hs_code})

Trade Value: \${value}

Delivery Window: {days} days

Origin: {origin_country}

Destination: {destination_country}

Incoterm: {incoterm}

Inspection: {inspection_type}

Available classifications: ROUTINE, PRIORITY, CRITICAL

Provide a recommendation with:

1. The recommended classification
2. A confidence score (0-100%)
3. A plain-language explanation of why

Return as structured JSON.

json

4.11.2.3 UI Example

text

■ 11. TRADE CRITICALITY ■

□ □

■ ■ ■ AI SUGGESTED CRITICALITY: PRIORITY (87% confidence) ■ ■ ■

4444

■ ■ This trade is classified as Priority because: ■ ■

4444

■ ■ ■ Time-sensitive commodity (fresh fruit) ■ ■ ■

■ ■ ■ Medium-high value (\$150,000) ■ ■ ■

■ ■ ■ Moderate delivery window (15 days) ■ ■ ■

■ ■ ■ ■

■ ■ Recommended Approval: Manager + Finance ■ ■

4444

■ ■

■ ■ ■ ■

■ ■ ■ ■

■ ■ • 24-hour approval SLA ■ ■

■ ■ • Expedited financing review ■ ■

4.11.3.1 Approval Routing

4.11.3.3 Escalation Handling

4.11.4 Criticality Validation (A4, WasmEdge)

rust

```
fn validate_criticality(
```

selected_criticality: Criticality

```
) -> Result<()> {
```

```

// 1. Check if criticality is appropriate for incoterm
if selected_criticality == Criticality::Critical
&& matches!(trade_request.incoterm, Incoterm::EXW | Incoterm::FOB) {
return Err(Error::IncotermCriticalityMismatch(
"Critical criticality is not recommended for EXW/FOB incoterms".to_string()
));
}

// 2. Check if criticality is appropriate for transport mode
if selected_criticality == Criticality::Critical
&& trade_request.transport_mode == TransportMode::Rail {
return Err(Error::TransportCriticalityMismatch(
"Critical criticality may not be suitable for rail transport".to_string()
));
}

// 3. Check if criticality is appropriate for commodity
if selected_criticality == Criticality::Routine
&& is_perishable(trade_request) {
return Err(Error::CommodityCriticalityMismatch(
"Perishable commodities typically require Priority or Critical criticality".to_string()
));
}

// 4. Check if criticality is appropriate for destination
if selected_criticality == Criticality::Routine
&& trade_request.destination_country == "NG" { // Nigeria
return Err(Error::DestinationCriticalityMismatch(
"Nigeria typically requires Priority or Critical criticality".to_string()
));
}

// 5. Check if criticality is appropriate for value
if selected_criticality == Criticality::Routine
&& trade_request.total_value > 500000.0 {
return Err(Error::ValueCriticalityMismatch(
"Trades over $500,000 typically require Priority or Critical criticality".to_string()
));
}

```

Ok()

}

4.11.4.2 Auto-Adjustment Logic

If the selected criticality is inconsistent with trade parameters, the system suggests an adjustment:

text

■■■ CRITICALITY ADJUSTMENT SUGGESTED ■■■

■ ■

■ You have selected Routine criticality, but the following factors suggest: ■

■ ■

■ ■ Perishable commodity (fresh fruit) — Priority recommended ■

■ ■ High trade value (\$450,000) — Priority recommended ■

■ ■ Tight delivery window (7 days) — Critical recommended ■

■ ■

■ Recommended Criticality: PRIORITY ■

■ ■

■ [Apply Recommended Criticality] [Keep Routine] [Adjust Manually] ■

4.11.5 Integration with Other Phases

| Phase | How Criticality Flows |

$$| \begin{smallmatrix} : & - & - \\ : & & \end{smallmatrix} | \begin{smallmatrix} : & - & - \\ : & & \end{smallmatrix} | \begin{smallmatrix} : & - & - \\ : & & \end{smallmatrix} |$$

| Phase 1 (Trade Initiation) | Criticality set during trade request creation; drives workflow routing |

| Phase 2 (Seller Quote) | Seller sees criticality; may adjust pricing and logistics accordingly |

| Phase 3 (Contract) | Criticality is not in contract; it's operational |

| Phase 4 (Financing) | Financiers use criticality to assess urgency and priority |

| Phase 5 (Execution) | Logistics providers use criticality to prioritise bookings |

| Phase 6 (Settlement) | Criticality affects settlement priority |

| Phase 7 (Distressed Cargo) | Critical trades get faster distressed handling |

| Phase 8 (Dispute) | Critical disputes escalate faster |

4.11.6 Governor Gates (Part 4.11)

Example CONDITIONAL Response:

json

{

```
"verdict": "CONDITIONAL".
```

```

"tenant_message": {
  "summary": "Your trade request is on hold because Routine criticality is not recommended for this perishable commodity. Please review and adjust.",
  "conditions": [
    {
      "condition_id": "criticality_mismatch",
      "label": "Adjust criticality to Priority or Critical",
      "action_url": "/v1/trade/request/draft?section=criticality"
    }
  ],
  "escalation_hint": "If you believe this is an error, request a human review."
}

```

4.11.7 Database Schema Additions (Part 4.11)

sql

-- Trade Criticality

```
ALTER TABLE trade_requests ADD COLUMN trade_criticality TEXT DEFAULT 'ROUTINE';
```

```
ALTER TABLE trade_requests ADD COLUMN criticality_suggested TEXT;
```

```
ALTER TABLE trade_requests ADD COLUMN criticality_confidence DECIMAL(5,2);
```

```
ALTER TABLE trade_requests ADD COLUMN criticality_auto_adjusted BOOLEAN DEFAULT false;
```

```
ALTER TABLE trade_requests ADD COLUMN criticality_adjustment_reason TEXT;
```

-- Criticality rules (configurable by Admin)

```
CREATE TABLE criticality_rules (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
rule_type TEXT NOT NULL, -- 'INCOTERM', 'COMMODITY', 'DESTINATION', 'VALUE', 'TIME_SENSITIVE'
```

```
condition JSONB NOT NULL,
```

```
recommended_criticality TEXT NOT NULL, -- 'ROUTINE', 'PRIORITY', 'CRITICAL'
```

```
reason_template TEXT NOT NULL,
```

```
is_active BOOLEAN DEFAULT true,
```

```
updated_at TIMESTAMPTZ DEFAULT NOW()
```

```
);
```

-- Criticality overrides (Admin-configurable)

```
CREATE TABLE criticality_overrides (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
```

```

original_criticality TEXT NOT NULL,
new_criticality TEXT NOT NULL,
override_reason TEXT,
override_by UUID REFERENCES employees(id) ON DELETE SET NULL,
override_at TIMESTAMPTZ DEFAULT NOW()
);

-- Criticality metrics (for analytics)
CREATE TABLE criticality_metrics (
id BIGSERIAL PRIMARY KEY,
criticality TEXT NOT NULL,
trade_count INTEGER DEFAULT 0,
avg_value DECIMAL(20,4),
avg_delivery_days INTEGER,
avg_response_time_hours DECIMAL(10,2),
completion_rate DECIMAL(5,2),
measured_period DATE NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_trade_requests_criticality ON trade_requests(trade_criticality);
CREATE INDEX idx_trade_requests_criticality_created ON trade_requests(trade_criticality, created_at
DESC);
CREATE INDEX idx_criticality_rules_type ON criticality_rules(rule_type);
CREATE INDEX idx_criticality_overrides_trade ON criticality_overrides(trade_request_id);
CREATE INDEX idx_criticality_metrics_period ON criticality_metrics(measured_period);

```

4.11.8 Implementation Checklist (Part 4.11)

4.11.8.1 Backend Services

- Implement Trade Criticality service (Rust)
- Implement AI suggestion generation (A1, Groq)
- Implement criticality validation (A4, WasmEdge)
- Implement auto-adjustment logic
- Implement criticality rules engine
- Implement criticality override management

4.11.8.2 Frontend Components

- Build Trade Criticality selector (Routine/Priority/Critical)
- Build AI Suggestion display

Build Criticality implications display

Build Criticality adjustment suggestion

Build Approval routing display based on criticality

4.11.8.3 Criticality Rules

Define incoterm criticality rules

Define commodity criticality rules

Define destination criticality rules

Define value criticality rules

Define time-sensitive criticality rules

Implement Admin configuration UI for rules

4.11.8.4 Integration

Integrate with Smart Inbox for priority-based alerts

Integrate with Approval Engine for SLA-based routing

Integrate with Logistics Service for booking priority

Integrate with Monitoring Service for frequency-based monitoring

4.11.8.5 Testing

Unit tests: criticality validation

Unit tests: AI suggestion generation

Unit tests: auto-adjustment logic

Integration tests: criticality → approval routing

Integration tests: criticality → Smart Inbox alerts

End-to-end tests: full criticality workflow

4.11.9 AI Authority Summary for Part 4.11

END OF PART 4.11 — TRADE CRITICALITY

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.12 — MULTI-SHIPMENT REQUEST

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.12.0 Purpose & Design Philosophy

The Multi-Shipment Request feature enables buyers to structure a single master contract covering multiple future deliveries. This is a critical capability for industries with recurring shipments (e.g., agricultural commodities, manufacturing supplies, retail distribution). Unlike a single-shipment contract where all goods move at once, a multi-shipment contract allows for flexible scheduling, per-shipment fee collection, and independent execution of each shipment.

Key Principles:

Master contract with multiple shipments — One contract covers all shipments, but each shipment locks independently.

Per-shipment fee collection.

Each shipment gets its own USTN.

4.12.2 Schedule Builder

4.12.2.1 Complete UI Example

text

■ MULTI-SHIPMENT SCHEDULE ■

■ ■

■ ■ ■ Define the delivery schedule for this multi-shipment contract. ■ ■

■ ■ ■ ■

■ ■ Shipment 1: ■ ■

■■■ Delivery Date: [2026-07-15] (15 Jul 2026) ■■■

Port of Discharge: [Alexandria ▼]

■ ■ ■ Number of Containers: [1] ■ ■ ■

Commodities: [Inherit from main]

■ ■ ■ [Edit Commodities] [Remove Shipment] ■ ■ ■

11111

■ ■ Shipment 2: ■ ■

■■■ Delivery Date: [2026-08-15] (15 Aug 2026) ■■■

Port of Discharge: [Port Said ▼]

■ ■ ■ Number of Containers: [2] ■ ■ ■

■ ■ ■ Commodities: [Inherit from main] ■ ■ ■

■ ■ ■ [Edit Commodities] [Remove Shipment] ■ ■ ■

4444

■ ■ Shipment 3: ■ ■

Delivery Date: [2026-09-15] (15 Sep 2026)

Port of Discharge: [Damietta ▼]

Number of Containers: [1]

Commodities: [Inherit from main]

[Edit Commodities] [Remove Shipment]

[Add Shipment] [Import Schedule Template] [Clear All]

AI Recommendation: For fresh produce, consider 30-day intervals between shipments to align with harvest cycles.

4.12.2.2 Per-Shipment Fields

4.12.2.3 Commodity Override

When the buyer clicks "Edit Commodities" for a shipment, a mini-form appears:

text

EDIT COMMODITIES — Shipment 2

Commodity 1: Oranges

Quantity: [100] [MT ▼] Tolerance: [±5% ▼]

Packaging: [Palletized ▼] Pallets: [22]

Commodity 2: Lemons

Quantity: [50] [MT ▼] Tolerance: [±5% ▼]

Packaging: [Palletized ▼] Pallets: [14]

Changes here only affect Shipment 2. Other shipments retain main settings.

[Save Changes] [Reset to Main] [Cancel]

4.12.3 AI-Assisted Schedule Suggestions (A1, Groq)

4.12.3.1 Suggestion Generation

The system generates AI suggestions based on:

4.12.3.2 Suggestion Example

text

AI SCHEDULE SUGGESTIONS

Based on your trade parameters, we suggest the following schedule:

Shipment	Delivery Date	Port	Containers	Confidence	Reason
1	15 Jul 2026	Alexandria	1	92%	Post-harvest
2	15 Aug 2026	Port Said	2	88%	Peak demand
3	15 Sep 2026	Damietta	1	85%	End of cycle

[Apply All] [Modify] [Dismiss]

4.12.3.3 Schedule Optimisation (A2)

The system can also optimise the schedule based on:

- Carrier availability — Aligns with sailing schedules.
- Warehouse capacity — Spreads shipments to avoid storage bottlenecks.
- Payment cycles — Aligns with buyer's cash flow.
- Seasonal patterns — Aligns with historical demand patterns.

4.12.4 Data Model — Multi-Shipment Structure

4.12.4.1 Master Contract vs. Shipments

■ ■ Status: ACTIVE_MASTER ■ ■

■ ■ Total Value: \$450,000 ■ ■

■ ■ Created: 2026-04-15 ■ ■

■ ■ ■

■ ▼ ■

■ ■ Status: LOCKED (in transit) ■ Status: SCHEDULED ■ ■

■ ■ Delivery Date: 15 Jul 2026 ■ Delivery Date: 15 Aug 2026 ■ ■

■ ■ Containers: 1 ■ Containers: 2 ■ ■

■ ■ SGTX Fee: \$2,250 (paid) ■ SGTX Fee: \$2,250 (pending) ■ ■

■ ■ ■

■ ▼ ■

■ ■ Status: SCHEDULED ■ ■

■ ■ USTN: (not yet generated) ■ ■

4.12.5.2 Modification Workflow


```
],  
"escalation_hint": "If you believe this is an error, request a human review."  
}  
}
```

4.12.9 Database Schema Additions (Part 4.12)

sql

-- Contract shipments (multi-shipment)

```
CREATE TABLE contract_shipments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,  
  shipment_number INTEGER NOT NULL,  
  scheduled_delivery_date DATE NOT NULL,  
  port_of_discharge TEXT NOT NULL,  
  containers_count INTEGER NOT NULL,  
  total_price DECIMAL(20,4) NOT NULL,  
  sgtx_fee_amount DECIMAL(20,4),  
  fee_lock_id UUID,  
  ustn TEXT UNIQUE,  
  status TEXT DEFAULT 'SCHEDULED', -- SCHEDULED, READY, CONFIRMED, LOCKED,  
  IN_EXECUTION, COMPLETED, CANCELLED  
  commodities_overridden BOOLEAN DEFAULT false,  
  commodity_data JSONB, -- Overridden commodity data  
  locked_at TIMESTAMPTZ,  
  completed_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Schedule modification requests

```
CREATE TABLE schedule_modification_requests (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,  
  shipment_id UUID NOT NULL REFERENCES contract_shipments(id) ON DELETE CASCADE,  
  requested_by_gtid TEXT NOT NULL,  
  requested_at TIMESTAMPTZ DEFAULT NOW(),  
  proposed_changes JSONB NOT NULL,  
  reason TEXT NOT NULL,
```

```

status TEXT DEFAULT 'PENDING', -- PENDING, ACCEPTED, REJECTED, COUNTERED
responded_by_gtid TEXT,
responded_at TIMESTAMPTZ,
addendum_contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Schedule addenda (immutable record of modifications)
CREATE TABLE schedule_addenda (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
modification_request_id UUID REFERENCES schedule_modification_requests(id) ON DELETE SET NULL,
original_schedule JSONB NOT NULL,
modified_schedule JSONB NOT NULL,
reason TEXT NOT NULL,
buyer_signature TEXT NOT NULL,
seller_signature TEXT NOT NULL,
sgtx_signature TEXT NOT NULL,
loom_hash TEXT NOT NULL,
signed_at TIMESTAMPTZ DEFAULT NOW(),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Multi-shipment AI suggestions
CREATE TABLE multi_shipment_suggestions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
suggested_schedule JSONB NOT NULL,
confidence DECIMAL(5,2),
factors JSONB,
applied BOOLEAN DEFAULT false,
generated_at TIMESTAMPTZ DEFAULT NOW(),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes

```

```

CREATE INDEX idx_contract_shipments_contract ON contract_shipments(contract_id);
CREATE INDEX idx_contract_shipments_ustn ON contract_shipments(ustn);
CREATE INDEX idx_contract_shipments_status ON contract_shipments(status);
CREATE INDEX idx_contract_shipments_date ON contract_shipments(scheduled_delivery_date);
CREATE          INDEX          idx_schedule_modification_requests_contract          ON
schedule_modification_requests(contract_id);
CREATE          INDEX          idx_schedule_modification_requests_shipment          ON
schedule_modification_requests(shipment_id);
CREATE INDEX idx_schedule_modification_requests_status ON schedule_modification_requests(status);
CREATE INDEX idx_schedule_addenda_contract ON schedule_addenda(contract_id);
CREATE          INDEX          idx_multi_shipment_suggestions_trade          ON
multi_shipment_suggestions(trade_request_id);

```

4.12.10 Implementation Checklist (Part 4.12)

4.12.10.1 Backend Services

Implement multi-shipment schedule service (Rust)

Implement per-shipment fee calculation

Implement per-shipment USTN generation

Implement schedule modification request handling

Implement schedule addendum generation

Implement AI schedule suggestion (A1, Groq)

Implement schedule optimisation (A2)

4.12.10.2 Frontend Components

Build Multi-Shipment Toggle

Build Schedule Builder (add/remove/edit shipments)

Build Per-Shipment Commodity Override mini-form

Build Schedule Modification Request form

Build Schedule Modification Review (diff view)

Build Multi-Shipment Dashboard (master contract view)

Build Schedule Progress Indicator

4.12.10.3 Governor Integration

Add G1U10: Multi-shipment schedule validation

Add G3U8: Per-shipment fee validation

Add G3U9: Per-shipment USTN generation

Add G3U10: Schedule modification validation

Add G3U11: Schedule addendum validation

Add G5UA9: Independent milestone validation

4.12.10.4 Testing

Unit tests: schedule validation

Unit tests: per-shipment fee calculation

Unit tests: per-shipment USTN generation

Integration tests: multi-shipment creation → per-shipment fee → USTN generation

Integration tests: schedule modification workflow

Integration tests: independent execution

End-to-end tests: full multi-shipment lifecycle

4.12.11 AI Authority Summary for Part 4.12

END OF PART 4.12 — MULTI-SHIPMENT REQUEST

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.13 — AI CONTAINER ADVISOR (A1)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.13.0 Purpose & Design Philosophy

The AI Container Advisor is an advisory-only feature that suggests optimal container configuration based on the buyer's commodity volume, pallet dimensions, product type, route, and transport mode. It uses anonymised aggregate data with differential privacy to provide recommendations without exposing individual trade data.

Key Principles:

Advisory only — The buyer can accept, modify, or ignore the recommendation.

Privacy-preserving — Uses anonymised aggregate data with differential privacy ($\epsilon=0.1$).

Context-aware — Considers commodity type, pallet dimensions, route, transport mode, and historical loading density.

One-click adjustment — Buyer can apply the recommendation with a single click.

Transparent reasoning — Explains why the recommendation was made.

Non-marketplace — Never says "other buyers used 3 containers" or suggests specific providers.

AI Integration in Part 4.13:

4.13.1 Advisor Trigger & Logic

4.13.1.1 Trigger

The AI Container Advisor is triggered automatically after the buyer enters commodity volumes (pallet count, carton dimensions, net weight).

Trigger Conditions:

At least one commodity has been entered.

Pallet count and dimensions are specified.

Commodity type is selected.

Transport mode is selected.

4.13.1.2 Input Data

4.13.1.3 Calculation Logic

rust

// Pseudo-code: Container Advisor calculation

```
fn calculate_container_recommendation(
```

```
  pallets: &[Pallet],
```

```
  commodity_type: &str,
```

```
  total_weight_kg: f64,
```

```
  transport_mode: TransportMode,
```

```
  route: &Route
```

```
) -> ContainerRecommendation {
```

```
// 1. Calculate total volume
```

```
let total_volume_cbm = pallets.iter()
```

```
  .map(|p| p.volume_cbm())
```

```
  .sum();
```

```
// 2. Determine container requirements
```

```
let requires_reefer = is_perishable(commodity_type);
```

```
let requires_ventilation = requires_ventilation(commodity_type);
```

```
let is_hazardous = is_hazardous(commodity_type);
```

```
// 3. Get available container types for route
```

```
let available = get_available_containers(transport_mode, route);
```

```
// 4. Calculate fit for each container type
```

```
let mut options = Vec::new();
```

```
for container in available {
```

```
  let pallet_fit = calculate_pallet_fit(pallets, &container);
```

```
  let weight_fit = total_weight_kg <= container.max_payload_kg;
```

```
  let volume_fit = total_volume_cbm <= container.max_volume_cbm;
```

```
  let special_fit = check_special_requirements(container, requires_reefer, requires_ventilation, is_hazardous);
```

```
  if pallet_fit.is_ok() && weight_fit && volume_fit && special_fit {
```

```
    options.push(ContainerOption {
```

```
      container_type: container,
```

```
      pallet_arrangement: pallet_fit.unwrap(),
```

```
      capacity_utilisation: (total_volume_cbm / container.max_volume_cbm) * 100.0,
```

```
      weight_utilisation: (total_weight_kg / container.max_payload_kg) * 100.0,
```

```
      estimated_cost: estimate_cost(&container, route),
```

```
      fit_score: calculate_fit_score(&container, total_volume_cbm, total_weight_kg),
```

```

});
}
}
// 4. Get historical loading density (anonymised, differential privacy)
let historical_density = get_historical_density(route, commodity_type);
// 5. Generate recommendation (A1, Groq)
let recommendation = generate_recommendation(options, historical_density);
recommendation
}

```

4.13.2 Container Options & Recommendations

4.13.2.1 Container Types Considered

4.13.2.2 Recommendation Output

The advisor returns a ranked list of options with the recommended option highlighted.

json

```

{
  "recommendation": {
    "container_type": "40ft Reefer",
    "container_count": 1,
    "pallet_arrangement": {
      "pallets_per_row": 2,
      "rows": 11,
      "total_pallets": 22,
      "utilisation_percentage": 95
    },
    "capacity_utilisation": 92,
    "weight_utilisation": 78,
    "estimated_cost": 4200,
    "estimated_transit_days": 14
  },
  "alternatives": [
    {
      "container_type": "40ft HC Reefer",
      "container_count": 1,
      "capacity_utilisation": 85,
      "weight_utilisation": 78,

```


□ □ □ □

■ ■ delivery window." ■ ■

■ ■

■ ■ ■ ■

■ ■ 1 x 40ft Standard ■ \$3,800 ■ 14 days ■ 92% ■ ■ No reefer ■ ■

■ [Apply Recommended Configuration] [Adjust to 1 × 40ft HC Reefer] ■

4.13.3.1 Data Sources

4.13.3.2 Density Calculation

```
// Pseudo-code: Historical loading density with differential privacy
```

route: &Route,

```
) -> HistoricalDensity {
```

```
let memory_events = query_trade_memory(route, commodity_type);
```

```
let raw_density = memory_events.iter()
```

```
.average();
```

```
let noise = laplace_noise(0.1);
```

```
// 4. Calculate confidence based on sample size
```

HistoricalDensity {

confidence: confidence,

privacy method: "Differential Privacy $\epsilon=0.1$ ",

}

$\epsilon=0.1$ — Strong privacy guarantee.

No individual trade data — Only aggregated, anonymised data.

Opt-out — Tenants can opt out of contributing data.

No re-identification — Anonymous IDs are rotated periodically.

4.13.4 Pallet Fit Calculation

4.13.4.1 Pallet Types & Dimensions

4.13.4.2 Container Internal Dimensions

Container Type	Width (mm)	Length (mm)	Height (mm)
----------------	------------	-------------	-------------

--	--	--	--

20ft Standard	2350	5890	2390
---------------	------	------	------

40ft Standard	2350	12030	2390
---------------	------	-------	------

40ft High-Cube	2350	12030	2690
----------------	------	-------	------

20ft Reefer	2290	5440	2180
-------------	------	------	------

40ft Reefer	2290	11590	2500
-------------	------	-------	------

40ft HC Reefer	2290	11590	2690
----------------	------	-------	------

4.13.4.3 Fit Calculation Logic

rust

// Pseudo-code: Pallet fit calculation

```
fn calculate_pallet_fit(
```

```
  pallets: &[Pallet],
```

```
  container: &Container
```

```
) -> Result<PalletArrangement> {
```

```
// 1. Calculate how many pallets fit per row (width)
```

```
let pallets_per_row = container.internal_width / pallet.width;
```

```
// 2. Calculate how many rows fit (length)
```

```
let rows = container.internal_length / pallet.length;
```

```
// 3. Calculate total pallets that fit
```

```
let total_fit = pallets_per_row * rows;
```

```
// 4. Check if all pallets fit
```

```
if pallets.len() > total_fit {
```

```
  return Err(Error::NotEnoughSpace);
```

```
}
```

```
// 5. Calculate utilisation
```

```
let utilisation = (pallets.len() as f64 / total_fit as f64) * 100.0;
```

```
// 6. Calculate optimal arrangement
```

```
let arrangement = optimize_arrangement(pallets, container);
```

```
Ok(PalletArrangement {  
  pallets_per_row,  
  rows,  
  total_fit,  
  used_pallets: pallets.len(),  
  utilisation,  
  arrangement,  
})  
}
```

4.13.5 One-Click Integration

4.13.6 Integration with Other Phases

| Phase | How Container Advisor Flows |

|:---|:---|:---|

| Phase 1 (Trade Initiation) | Advisor runs during commodity entry; suggests optimal containers |

| Phase 2 (Seller Quote) | Seller sees the buyer's container selection; may adjust logistics |

| Phase 3 (Contract) | Container selection is included in the contract |

| Phase 5 (Execution) | Logistics providers use the container selection |

4.13.7 Governor Gates (Part 4.13)

4.13.8 Database Schema Additions (Part 4.13)

```
sql
```

```
-- Container advisor logs
```

```
CREATE TABLE container_advisor_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  input_data JSONB NOT NULL, -- Pallet count, dimensions, weight, etc.  
  recommendation JSONB NOT NULL, -- Recommended configuration  
  alternatives JSONB, -- Alternative configurations  
  historical_density JSONB, -- Anonymised historical density  
  selected_option TEXT, -- User's selection (recommendation, alternative, current)  
  user_accepted BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- Historical loading density (anonymised, differential privacy)
```

```
CREATE TABLE historical_loading_density (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  input_data JSONB NOT NULL, -- Pallet count, dimensions, weight, etc.  
  recommendation JSONB NOT NULL, -- Recommended configuration  
  alternatives JSONB, -- Alternative configurations  
  historical_density JSONB, -- Anonymised historical density  
  selected_option TEXT, -- User's selection (recommendation, alternative, current)  
  user_accepted BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```

id BIGSERIAL PRIMARY KEY,
route_id TEXT NOT NULL,
commodity_type TEXT NOT NULL,
container_type TEXT NOT NULL,
average_density DECIMAL(5,2),
sample_count INTEGER,
confidence DECIMAL(5,2),
privacy_epsilon DECIMAL(5,2) DEFAULT 0.1,
measured_period DATE NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Container type reference table

```

CREATE TABLE container_types_ref (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
type_code TEXT UNIQUE NOT NULL,
type_name TEXT NOT NULL,
internal_width_mm INTEGER NOT NULL,
internal_length_mm INTEGER NOT NULL,
internal_height_mm INTEGER NOT NULL,
max_volume_cbm DECIMAL(10,2) NOT NULL,
max_payload_kg DECIMAL(10,2) NOT NULL,
has_reefer BOOLEAN DEFAULT false,
has_ventilation BOOLEAN DEFAULT false,
is_hazardous_compatible BOOLEAN DEFAULT false,
typical_use TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

```

CREATE INDEX idx_container_advisor_logs_trade ON container_advisor_logs(trade_request_id);
CREATE INDEX idx_container_advisor_logs_created ON container_advisor_logs(created_at DESC);
CREATE INDEX idx_historical_loading_density_route ON historical_loading_density(route_id,
commodity_type);
CREATE INDEX idx_historical_loading_density_period ON historical_loading_density(measured_period);

```

4.13.9 Implementation Checklist (Part 4.13)

4.13.9.1 Backend Services

Implement Container Advisor service (Rust)

Implement pallet fit calculation

Implement container capacity optimisation

Implement historical loading density query (anonymised, differential privacy)

Implement recommendation generation (A1, Groq)

Implement container type validation

4.13.9.2 Frontend Components

Build Container Advisor banner

Build Pallet Arrangement visualisation

Build Alternative Configurations comparison table

Build One-click Apply functionality

Build Explainability display

4.13.9.3 Integration

Trigger advisor after commodity entry

Integrate recommendation application with form

Integrate historical density with Trade Memory Layer (Part 19)

Integrate container types with RIA

4.13.9.4 Testing

Unit tests: pallet fit calculation

Unit tests: container capacity optimisation

Unit tests: recommendation generation

Integration tests: advisor trigger → recommendation → application

Integration tests: historical density query (anonymised)

End-to-end tests: full advisor workflow

4.13.10 AI Authority Summary for Part 4.13

END OF PART 4.13 — AI CONTAINER ADVISOR (A1)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.14 — DRAFT AUTO-SAVE & RECOVERY

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.14.0 Purpose & Design Philosophy

The Draft Auto-Save & Recovery system ensures that no work is ever lost during the trade request creation process. Trade requests can be complex, with multiple containers, commodities, specifications, and documents. A user might spend 30-60 minutes filling out a detailed request. The draft system provides a safety net that automatically preserves this work and enables seamless recovery.

Key Principles:

- Zero data loss — Every keystroke is preserved; no work is ever lost.
- Automatic, non-intrusive — Auto-saves in the background without interrupting the user.
- Seamless recovery — Users can resume drafts from the Smart Inbox with one click.
- Intelligent expiry — Drafts expire after 14 days, with reminders at 11 and 13 days.
- Admin recovery — Expired drafts can be restored by tenant admins.
- Multi-session support — Users can have multiple drafts in progress across different sessions.

AI Integration in Part 4.14:

4.14.1 Auto-Save Mechanism

4.14.1.1 Architecture Overview

text



■ ▼ ■

■ ■ ■ draft_reminders (scheduled notifications) ■ ■ ■

■ ▼ ■

■ ■ ■ 4. Expired draft notification ■ ■ ■

Form state changes (any field modified).

Debounce timer (30 seconds) expires.

User navigates away from the page (beforeunload event).

User clicks "Save Draft" (manual save).

Debounce Implementation:

typescript

// Pseudo-code: Debounce auto-save

```
import { useDebounce } from 'use-debounce';

function TradeRequestForm() {
  const [formData, setFormData] = useState(initialData);
  const [debouncedData] = useDebounce(formData, 30000); // 30 seconds
  useEffect(() => {
    if (debouncedData && !isSubmitting) {
      saveDraft(debouncedData);
    }
  }, [debouncedData]);

  // Beforeunload save
  useEffect(() => {
    const handleBeforeUnload = () => {
      if (formData && !isSubmitting) {
        navigator.sendBeacon('/api/v1/trade/draft', JSON.stringify(formData));
      }
    };
    window.addEventListener('beforeunload', handleBeforeUnload);
    return () => window.removeEventListener('beforeunload', handleBeforeUnload);
  }, [formData]);
}
```

4.14.1.3 Draft Storage Structure

json

```
{
  "draft_id": "DRAFT-2026-001",
  "trade_request_id": null, // null for new drafts; populated after recovery
  "buyer_tenant_id": "SGTX-EG-TRD-001234",
  "created_by_employee_id": "emp-001",
  "created_at": "2026-06-20T10:30:00Z",
  "updated_at": "2026-06-20T11:00:00Z",
```


"expires_at": "2026-07-04T10:30:00Z", // 14 days from creation

"last_activity_at": "2026-06-20T11:00:00Z",

"version": 3,

"draft_data": {

"seller_gtid": "SGTX-VN-TRD-002139-7F3A",

"incoterm": "CFR",

"containers": [

{

"container_number": 1,

"origin_country": "VN",

"destination_country": "EG",

"port_of_discharge": "ALEX",

"palletized": true,

"pallet_type": "EUR",

"commodities": [

{

"commodity_type": "Fresh Fruits",

"product_name": "Valencia Oranges",

"hs_code": "0805.10",

"requested_quantity": 100,

"quantity_unit": "MT",

"tolerance": 5,

"partial_shipment_allowed": true,

"transshipment_allowed": false,

"packaging_type": "Palletized",

"pallet_count": 22,

"net_weight_per_carton_kg": 10,

"acceptance_criteria": {

"moisture": { "limit": "Max 14%", "unit": "%" },

"protein": { "limit": "Min 12%", "unit": "%" },

"purity": { "limit": "Min 99%", "unit": "%" }

}

}

]

}

```
],
"transport_mode": "OCEAN",
"container_preferences": ["40FT_REEFER"],
"earliest_delivery_date": "2026-07-10",
"preferred_delivery_date": "2026-07-15",
"latest_delivery_date": "2026-07-20",
"insurance_requirement": "REQUIRED",
"responsible_party": "ACCORDING_TO_INCOTERM",
"settlement_structure": "DOCUMENTARY_COLLECTION",
"payment_timing": "AGAINST_DOCUMENTS",
"credit_period": "30_DAYS",
"currency": "USD",
"settlement_flexibility": "FLEXIBLE",
"required_documents": [
"COMMERCIAL_INVOICE",
"PACKING_LIST",
"BILL_OF_LADING",
"CERTIFICATE_OF_ORIGIN"
],
"special_instructions": "Arabic labels mandatory. Halal certification required.",
"trade_criticality": "PRIORITY",
"multi_shipment_schedule": [
{
"shipment_number": 1,
"delivery_date": "2026-07-15",
"port": "ALEX",
"containers": 1
},
{
"shipment_number": 2,
"delivery_date": "2026-08-15",
"port": "PSD",
"containers": 1
}
]
```

}

}

4.14.2 Draft Recovery Workflow

4.14.2.1 Smart Inbox Integration

When a draft exists, the Smart Inbox displays a low-priority recovery item (priority 30).

text

■ SMART INBOX ■

■ ■

■ ■ LOW PRIORITY (2) ■

■ ■ ■ Continue draft trade request from 2026-06-20 ■ ■ ■

■ ■ ■ ■

■ ■ Seller: Mekong Fresh Co. ■ ■

Commodity: Oranges (HS 0805.10)

■ ■ Last edited: 2026-06-20 11:00:00 ■ ■

■ ■ Progress: 65% complete ■ ■

4444

■ ■ [Resume] [Delete] [Share with Team] ■ ■

Continue draft trade request from 2026-06-18

4444

■ ■ Seller: Global Exports Co. ■ ■

■ ■ Commodity: Textiles (HS 5208.11) ■ ■

■ ■ Last edited: 2026-06-18 16:30:00 ■ ■

■ ■ ■■ Expires in 2 days ■ ■

■ ■ ■ ■

■ ■ [Resume] [Delete] ■ ■

4.14.2.2 Recovery Flow

text

■ DRAFT RECOVERY FLOW ■

22

■ 1. User opens Smart Inbox ■

■ 2. User clicks "Resume" on draft item ■

■ 3. System loads draft data ■

■ 4. System populates form with saved data ■

■ 5. System updates last_activity_at ■

■ 6. System extends expiry (14 days from last activity) ■

■ 7. User continues editing ■

■ 8. Auto-save continues (30-second intervals) ■

4.14.2.3 Multiple Drafts

Users can have multiple drafts in progress simultaneously:

4.14.3 Draft Expiry & Reminders

4.14.3.1 Expiry Policy

4.14.3.2 Expiry Cron Job

rust

```
// Pseudo-code: Draft expiry cron job (runs daily at 02:00 UTC)
```

```
async fn process_draft_expiry() -> Result<()> {
```

```
let now = Utc::now();
```

```
// 1. Find drafts expiring today
```

```
let expiring_today = get_drafts_expiring_on(now.date_naive()).await?;
```

```
// 2. Send reminders (11 days and 13 days)
```

```
let reminder 11 days = get drafts with reminder at(now + Duration::days(11)).await?;
```


■ ■

■ ■

■ ■

4 of 4

```
ALTER TABLE trade_requests ADD COLUMN draft_reminded_at TIMESTAMPTZ;
```

```

ALTER TABLE trade_requests ADD COLUMN draft_restored_from_id UUID;
ALTER TABLE trade_requests ADD COLUMN archived_at TIMESTAMPTZ;

-- Draft history (versioned snapshots)
CREATE TABLE draft_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
  draft_snapshot JSONB NOT NULL,
  version INTEGER NOT NULL,
  edited_by UUID REFERENCES employees(id) ON DELETE SET NULL,
  edited_at TIMESTAMPTZ DEFAULT NOW(),
  change_summary TEXT, -- AI-generated summary of changes
  UNIQUE(trade_request_id, version)
);

-- Draft reminders (scheduled notifications)
CREATE TABLE draft_reminders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
  reminder_type TEXT NOT NULL, -- 'EXPIRY_3_DAYS', 'EXPIRY_1_DAY', 'EXPIRY_TODAY'
  sent_at TIMESTAMPTZ DEFAULT NOW(),
  sent_to_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
  sent_by TEXT, -- 'EMAIL', 'IN_APP', 'PUSH'
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Draft restore logs
CREATE TABLE draft_restore_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  original_draft_id UUID REFERENCES trade_requests(id) ON DELETE SET NULL,
  restored_draft_id UUID REFERENCES trade_requests(id) ON DELETE SET NULL,
  restored_by UUID REFERENCES employees(id) ON DELETE SET NULL,
  restore_reason TEXT,
  restored_at TIMESTAMPTZ DEFAULT NOW()
);

-- Draft templates (user-saved templates)
CREATE TABLE draft_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```



```

tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
template_name TEXT NOT NULL,
description TEXT,
template_data JSONB NOT NULL,
usage_count INTEGER DEFAULT 0,
is_default BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, template_name)
);

-- Template usage logs
CREATE TABLE template_usage_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
template_id UUID NOT NULL REFERENCES draft_templates(id) ON DELETE CASCADE,
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
used_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_trade_requests_draft_status ON trade_requests(is_draft) WHERE is_draft = true;
CREATE INDEX idx_trade_requests_draft_expires ON trade_requests(draft_expires_at) WHERE is_draft = true;

CREATE INDEX idx_trade_requests_archived_at ON trade_requests(archived_at) WHERE archived_at IS NOT NULL;

CREATE INDEX idx_draft_history_request ON draft_history(trade_request_id);
CREATE INDEX idx_draft_history_edited ON draft_history(edited_at DESC);
CREATE INDEX idx_draft_reminders_request ON draft_reminders(trade_request_id);
CREATE INDEX idx_draft_restore_logs_original ON draft_restore_logs(original_draft_id);
CREATE INDEX idx_draft_templates_tenant ON draft_templates(tenant_id);
CREATE INDEX idx_template_usage_logs_template ON template_usage_logs(template_id);

```

4.14.8 Implementation Checklist (Part 4.14)

4.14.8.1 Backend Services

```

Implement draft save endpoint (POST /v1/trade/draft)
Implement draft load endpoint (GET /v1/trade/draft/{id})
Implement draft delete endpoint (DELETE /v1/trade/draft/{id})
Implement draft list endpoint (GET /v1/trade/drafts)
Implement draft expiry cron job (Rust)

```

Implement draft reminder service

Implement draft restoration service

Implement draft template CRUD

4.14.8.2 Frontend Components

Implement debounce auto-save (30 seconds)

Implement beforeunload save (navigator.sendBeacon)

Implement draft recovery in Smart Inbox

Implement draft restoration in Company Admin

Implement draft template management

Implement draft status indicators (saved, saving, error)

4.14.8.3 Integration

Integrate draft save with all form sections

Integrate draft recovery with Smart Inbox

Integrate draft expiry with notification system

Integrate draft restoration with Company Admin

4.14.8.4 Testing

Unit tests: draft save and load

Unit tests: draft expiry logic

Unit tests: draft reminders

Integration tests: auto-save → recovery → continue

Integration tests: draft expiry → reminder → archive

Integration tests: draft restoration by admin

End-to-end tests: full draft lifecycle

4.14.9 AI Authority Summary for Part 4.14

END OF PART 4.14 — DRAFT AUTO-SAVE & RECOVERY

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.15 — GOVERNOR PRESCREEN & SUBMISSION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.15.0 Purpose & Design Philosophy

The Governor PreScreen & Submission is the final validation gate before a trade request is submitted to the seller. It is the authoritative decision engine that evaluates every aspect of the trade request against constitutional rules, business policies, regulatory requirements, and AI-driven risk assessments. No irreversible action bypasses this layer.

Key Principles:

Constitutional supremacy — WasmEdge modules enforce non-negotiable rules (fee bounds, jurisdiction supremacy, A5 prohibition).

Business logic — OPA policies enforce RBAC, permissions, dual-mode context, data scopes.

22

■ [Switch to Seller Selection] ■



■ [Request Human Review] — a compliance officer will respond within 24h ■

4.15.4.1 Decision Logging

Loom Entry Structure:

```
{
  "decision_id": "dec-abc123",
  "decision_type": "trade_request_submit",
  "actor_gtid": "SGTX-EG-TRD-001234",
  "actor_employee_id": "emp-001",
  "trade_request_id": "TR-2026-001",
  "verdict": "ALLOW",
  "tenant_message": null,
  "conditions": [],
  "timestamp": "2026-06-20T10:30:00Z",
  "previous_loom_hash": "sha256:prev123...",
  "loom_hash": "sha256:a1b2c3..."
}
```

rust

```
// Pseudo-code: Decision signing
```

```
let decision_json = serde_json::to_string(&decision).unwrap();
```

```
let signature = sign_with_ed25519(&decision_json);
```

```
let loom_hash = sha256(format!("{}", previous_hash, decision_json, signature));
```

4.15.4.3 Hourly Verification

A background job (audit-chain-verifier) recalculates every hash from genesis and compares with stored values. Any mismatch triggers a P0 incident.

4.15.5 Database Schema Additions (Part 4.15)

```
sql
```

```
-- Governor decisions (existing, extended)
```

```
ALTER TABLE governor_decisions ADD COLUMN trade_request_id UUID REFERENCES  
trade_requests(id) ON DELETE SET NULL;
```

```
ALTER TABLE governor_decisions ADD COLUMN decision_type TEXT NOT NULL;
```

```
ALTER TABLE governor_decisions ADD COLUMN actor_employee_id UUID REFERENCES  
employees(id) ON DELETE SET NULL;
```

```
ALTER TABLE governor_decisions ADD COLUMN conditions JSONB;
```

```
ALTER TABLE governor_decisions ADD COLUMN escalation_hint TEXT;
```

```
-- Submission logs
```

```
CREATE TABLE submission_logs (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
```

```
submitted_by UUID REFERENCES employees(id) ON DELETE SET NULL,
```

```
submitted_at TIMESTAMPTZ DEFAULT NOW(),
```

```
decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
```

```
verdict TEXT NOT NULL, -- 'ALLOW', 'CONDITIONAL', 'DENY'
```

```
conditions_resolved BOOLEAN DEFAULT false,
```

```
resolved_at TIMESTAMPTZ,
```

```
loom_hash TEXT
```

```
);
```

```
-- Pre-screen validation cache (for performance)
```

```
CREATE TABLE prescreen_validation_cache (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
```

```
validation_type TEXT NOT NULL,
```

```
validation_result JSONB NOT NULL,
```

```
validated_at TIMESTAMPTZ DEFAULT NOW(),
```

```
expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '5 minutes'
```

```
);
```

```
-- Indexes
```

```
CREATE INDEX idx_governor_decisions_trade_request ON governor_decisions(trade_request_id);
```

```
CREATE INDEX idx_governor_decisions_decision_type ON governor_decisions(decision_type);
```

```
CREATE INDEX idx_governor_decisions_created ON governor_decisions(created_at DESC);
```

```
CREATE INDEX idx_submission_logs_trade_request ON submission_logs(trade_request_id);
```

```
CREATE INDEX idx_submission_logs_verdict ON submission_logs(verdict);
```

```
CREATE INDEX idx_prescreen_validation_cache_trade ON  
prescreen_validation_cache(trade_request_id);
```

4.15.6 Implementation Checklist (Part 4.15)

4.15.6.1 Backend Services

Implement Governor service (Rust + Axum)

Implement OPA evaluator with all policies

Implement WasmEdge constitutional engine with all modules

Implement AI consult integration (A1-A3)

Implement decision merger (OPA + WasmEdge + AI)

Implement Loom hash chain logging

Implement Ed25519 signing

Implement tenant message generation (A1, Groq → Ollama)

Implement submission endpoint (POST /v1/trade/request/submit)

Implement audit-chain-verifier cron job (hourly)

4.15.6.2 OPA Policies

Implement permissions.rego (RBAC, dual-mode context, data scopes)

Implement fee.rego (fee rate validation)

Implement financing.rego (financing validation)

Implement distressed.rego (distressed validation)

Implement multiship.rego (multi-shipment validation)

Implement logistics.rego (logistics validation)

Implement broker.rego (broker validation)

Implement reserve.rego (reserve rules)

4.15.6.3 WasmEdge Modules

Implement constitutional_rules.wasm (fee bounds, A5 prohibition, multisig)

Implement jurisdiction_matrix.wasm (RIA-updated)

Implement incoterms_engine.wasm (incoterm validation)

Implement fee_gate.wasm (gross-up, split instructions)

Implement distressed_country_gate.wasm (country factor)

Implement dual_mode_gate.wasm (context enforcement)

Implement reserve_rules.wasm (reserve composition, 110% backing)

4.15.6.4 Decision Panel

Implement PlainLanguage Governor Decision Panel UI component (React)

Implement tenant_message generation (A1, Groq → Ollama)

Implement static template fallback

Implement "Switch to Seller/Buyer Mode" button

Implement "Request Human Review" escalation (A3)

Implement resolution timer and auto-escalation

Implement confidence/evidence expandable section

4.15.6.5 Testing

Unit tests: OPA policy evaluation

Unit tests: WasmEdge module evaluation

Unit tests: Decision merger logic

Integration tests: ALLOW → trade request created

Integration tests: CONDITIONAL → Decision Panel displayed

Integration tests: DENY → Decision Panel displayed

Integration tests: Loom chain logging

Integration tests: tenant message generation

End-to-end tests: full submission workflow

4.15.7 AI Authority Summary for Part 4.15

END OF PART 4.15 — GOVERNOR PRESCREEN & SUBMISSION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 4.16 — DATABASE SCHEMA ADDITIONS (COMPLETE DDL)

[L2] This section contains implementation specifications (Canonical Data Model).

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.16.0 Purpose & Design Philosophy

[ds-markdown-paragraph] This part consolidates the complete database schema additions required for the enhanced New Trade Request form (Parts 4.0 through 4.15). It includes all new tables, columns, indexes, and relationships defined throughout the previous parts. This DDL is designed to be production-ready, self-hosted on PostgreSQL 18 with pgvector and TimescaleDB extensions.

[ds-markdown-paragraph] Key Principles:

[ds-markdown-paragraph] Idempotent — All CREATE IF NOT EXISTS guards ensure safe execution.

[ds-markdown-paragraph] Complete — Includes all tables, columns, indexes, foreign keys, and constraints.

[ds-markdown-paragraph] Extensible — JSONB fields for flexible data storage where schema is dynamic.

[ds-markdown-paragraph] Auditable — All tables include created_at and updated_at timestamps.

[ds-markdown-paragraph] Secure — Foreign key constraints enforce referential integrity.

[ds-markdown-paragraph] Performant — Indexes for all frequently queried columns.

4.16.1 Extensions & Enums

sql

[HTML Preformatted] -- Enable required extensions

[HTML Preformatted] CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

[HTML Preformatted] CREATE EXTENSION IF NOT EXISTS "pgvector";

[HTML Preformatted] CREATE EXTENSION IF NOT EXISTS "timescaledb";

[HTML Preformatted] -- Enumerated types (v11.0 additions)

[HTML Preformatted] CREATE TYPE criticality_level AS ENUM ('ROUTINE', 'PRIORITY', 'CRITICAL');

[HTML Preformatted] CREATE TYPE settlement_structure AS ENUM ('DOCUMENTARY_CREDIT', 'DOCUMENTARY_COLLECTION', 'BANK_TRANSFER', 'OPEN_ACCOUNT', 'TO_BE_NEGOTIATED');

[HTML Preformatted] CREATE TYPE payment_timing AS ENUM ('ADVANCE', 'PARTIAL_ADVANCE', 'AGAINST_DOCUMENTS', 'AGAINST_SHIPMENT', 'AGAINST_DELIVERY', 'DEFERRED', 'TO_BE_NEGOTIATED');

[HTML Preformatted] CREATE TYPE credit_period AS ENUM ('0_DAYS', '15_DAYS', '30_DAYS', '45_DAYS', '60_DAYS', '90_DAYS', 'CUSTOM');

[HTML Preformatted] CREATE TYPE commercial_priority AS ENUM ('LOWEST_COST', 'FASTEST_SETTLEMENT', 'FINANCING_FRIENDLY', 'BALANCED');

[HTML Preformatted] CREATE TYPE settlement_flexibility AS ENUM ('FIXED', 'FLEXIBLE', 'OPEN_TO_ALTERNATIVES');

```
[HTML Preformatted] CREATE TYPE financing_interest AS ENUM ('BUYER', 'SELLER', 'EITHER_PARTY', 'NONE');
```

```
[HTML Preformatted] CREATE TYPE transport_mode AS ENUM ('OCEAN', 'AIR', 'RAIL', 'TRUCK', 'MULTIMODAL');
```

```
[HTML Preformatted] CREATE TYPE insurance_requirement AS ENUM ('REQUIRED', 'OPTIONAL', 'NOT_REQUIRED');
```

```
[HTML Preformatted] CREATE TYPE insurance_type AS ENUM ('ALL_RISKS', 'FPA', 'WA', 'ICC_A', 'ICC_B', 'ICC_C', 'REEFER', 'LIVESTOCK', 'WAR', 'STRIKES');
```

```
[HTML Preformatted] CREATE TYPE insurance_responsible_party AS ENUM ('BUYER', 'SELLER', 'ACCORDING_TO_INCOTERM');
```

```
[HTML Preformatted] CREATE TYPE document_trigger AS ENUM ('SHIPMENT', 'SETTLEMENT', 'CUSTOMS', 'FINANCING');
```

```
[HTML Preformatted] CREATE TYPE readiness_severity AS ENUM ('CRITICAL', 'WARNING', 'RECOMMENDED', 'QUALITY');
```

```
[HTML Preformatted] CREATE TYPE draft_status AS ENUM ('ACTIVE', 'EXPIRED', 'ARCHIVED', 'DELETED');
```

```
[HTML Preformatted] CREATE TYPE verdict_type AS ENUM ('ALLOW', 'DENY', 'CONDITIONAL', 'ESCALATE', 'PENDING');
```

```
[HTML Preformatted] CREATE TYPE container_type AS ENUM ('20FT_STANDARD', '40FT_STANDARD', '40FT_HC', '20FT_REEFER', '40FT_REEFER', '40FT_HC_REEFER', 'OPEN_TOP', 'FLAT_RACK', 'TANK');
```

```
[HTML Preformatted] CREATE TYPE equipment_type AS ENUM ('STANDARD', 'REEFER', 'HC', 'OPEN_TOP', 'FLAT_RACK', 'TANK', 'PALLET', 'ULD', 'DRY_VAN', 'FLATBED', 'CURTAIN_SIDE', 'BOX_WAGON', 'FLAT_WAGON', 'TANK_WAGON', 'REEFER_WAGON');
```

4.16.2 Core Trade Tables

4.16.2.1 Trade Requests (Extended)

sql

```
[HTML Preformatted] -- Trade requests (core table, extended for all new fields)
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS trade_criticality criticality_level DEFAULT 'ROUTINE';
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS criticality_suggested criticality_level;
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS criticality_confidence DECIMAL(5,2);
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS criticality_auto_adjusted BOOLEAN DEFAULT false;
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS criticality_adjustment_reason TEXT;
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS commercial_priority commercial_priority DEFAULT 'BALANCED';
```

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS settlement_structure
settlement_structure;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS payment_timing
payment_timing;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS credit_period
credit_period DEFAULT '30_DAYS';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
credit_period_custom_days INTEGER;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS currency TEXT
DEFAULT 'USD';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS currency_other
TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS financing_interest
financing_interest DEFAULT 'NONE';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS bank_instrument
TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS settlement_flexibility
settlement_flexibility DEFAULT 'FLEXIBLE';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
settlement_documents TEXT[];

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
partial_advance_percentage INTEGER;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS balance_timing
TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
acceptable_alternatives TEXT[];

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
conditions_for_alternatives TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
settlement_auto_configured BOOLEAN DEFAULT false;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
settlement_auto_config_reason TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS transport_mode
transport_mode;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS equipment_type
equipment_type;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS equipment_count
INTEGER;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS origin_port TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS destination_port
TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS alternative_ports TEXT[];

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS earliest_delivery_date DATE;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS preferred_delivery_date DATE;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS latest_delivery_date DATE;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS transit_time_days INTEGER;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS route_details JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS port_requirements JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS congestion_alert JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS container_advisor_recommendation JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_requirement insurance_requirement DEFAULT 'OPTIONAL';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_responsible_party insurance_responsible_party DEFAULT 'ACCORDING_TO_INCOTERM';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_type insurance_type;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_coverage_percent INTEGER DEFAULT 110;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_coverage_currency TEXT DEFAULT 'USD';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_policy_number TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_insurer_name TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_policy_effective_date DATE;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_policy_expiry_date DATE;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_auto_configured BOOLEAN DEFAULT false;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS insurance_auto_config_reason TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS document_language TEXT DEFAULT 'ENGLISH';


```

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
documents_originals_required BOOLEAN DEFAULT false;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
documents_electronic_accepted BOOLEAN DEFAULT true;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
special_trade_instructions TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS rfq_readiness_score
INTEGER;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS rfq_quality_score
DECIMAL(5,2);

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS rfq_ai_confidence
DECIMAL(5,2);

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS rfq_missing_items
JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
rfq_incomplete_items JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS rfq_quality_issues
JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
rfq_groq_recommendation TEXT;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
multi_shipment_schedule JSONB;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS is_multi_shipment
BOOLEAN DEFAULT false;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS master_contract_id
UUID REFERENCES contracts(id) ON DELETE SET NULL;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS is_draft BOOLEAN
DEFAULT false;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS draft_expires_at
TIMESTAMPTZ;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS draft_reminded_at
TIMESTAMPTZ;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS
draft_restored_from_id UUID;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS archived_at
TIMESTAMPTZ;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS last_activity_at
TIMESTAMPTZ;

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS approval_status
TEXT DEFAULT 'PENDING';

[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS approval_workflow
JSONB;

```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS approval_required_level TEXT;
```

```
[HTML Preformatted] ALTER TABLE trade_requests ADD COLUMN IF NOT EXISTS approval_completed_at TIMESTAMPTZ;
```

```
[HTML Preformatted] -- Indexes for trade_requests
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_criticality ON trade_requests(trade_criticality);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_priority ON trade_requests(trade_criticality, created_at DESC);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_settlement ON trade_requests(settlement_structure);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_transport ON trade_requests(transport_mode);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_insurance ON trade_requests(insurance_requirement);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_delivery_window ON trade_requests(earliest_delivery_date, latest_delivery_date);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_readiness ON trade_requests(rfq_readiness_score);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_approval_status ON trade_requests(approval_status);
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_draft_status ON trade_requests(is_draft) WHERE is_draft = true;
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_draft_expires ON trade_requests(draft_expires_at) WHERE is_draft = true;
```

```
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_multi_shipment ON trade_requests(is_multi_shipment) WHERE is_multi_shipment = true;
```

4.16.2.2 Trade Request Containers

```
sql
```

```
[HTML Preformatted] -- Container configuration
```

```
[HTML Preformatted] CREATE TABLE IF NOT EXISTS trade_request_containers (
```

```
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
```

```
[HTML Preformatted] container_number INTEGER NOT NULL,
```

```
[HTML Preformatted] origin_country CHAR(2) NOT NULL,
```

```
[HTML Preformatted] destination_country CHAR(2) NOT NULL,
```

```
[HTML Preformatted] port_of_discharge TEXT NOT NULL,
```

```
[HTML Preformatted] palletized BOOLEAN DEFAULT true,
```

```
[HTML Preformatted] pallet_type TEXT DEFAULT 'EUR',
```

```

[HTML Preformatted] destination_override TEXT,
[HTML Preformatted] notes TEXT,
[HTML Preformatted] total_net_weight_kg DECIMAL(10,2),
[HTML Preformatted] total_gross_weight_kg DECIMAL(10,2),
[HTML Preformatted] total_pallet_count INTEGER,
[HTML Preformatted] container_type container_type,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] UNIQUE(trade_request_id, container_number)
[HTML Preformatted] );
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_containers_request ON
trade_request_containers(trade_request_id);
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_containers_port ON
trade_request_containers(port_of_discharge);

```

4.16.2.3 Trade Request Commodities

sql

```

[HTML Preformatted] -- Trade Request Commodities (per-commodity data — critical fix for
multi-commodity)
[HTML Preformatted] CREATE TABLE IF NOT EXISTS trade_request_commodities (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] container_id UUID NOT NULL REFERENCES trade_request_containers(id) ON
DELETE CASCADE,
[HTML Preformatted] commodity_type TEXT NOT NULL,
[HTML Preformatted] product_name TEXT NOT NULL,
[HTML Preformatted] hs_code CHAR(6) NOT NULL,
[HTML Preformatted] -- Quantity & Tolerance (per commodity)
[HTML Preformatted] requested_quantity DECIMAL(20,4) NOT NULL,
[HTML Preformatted] requested_quantity_unit TEXT NOT NULL,
[HTML Preformatted] quantity_tolerance DECIMAL(5,2) DEFAULT 5.0,
[HTML Preformatted] partial_shipment_allowed BOOLEAN DEFAULT true,
[HTML Preformatted] transshipment_allowed BOOLEAN DEFAULT true,
[HTML Preformatted] -- Product Criticality (per commodity)
[HTML Preformatted] product_criticality TEXT DEFAULT 'STANDARD',
[HTML Preformatted] -- Inspection Requirements (per commodity)
[HTML Preformatted] inspection_requirements TEXT,

```

```

[HTML Preformatted] -- Acceptance Criteria (per commodity)
[HTML Preformatted] acceptance_criteria_matrix JSONB,
[HTML Preformatted] -- Packaging & Weight (per commodity)
[HTML Preformatted] packaging_type TEXT,
[HTML Preformatted] pallet_count INTEGER,
[HTML Preformatted] pallet_type TEXT,
[HTML Preformatted] cartons_per_pallet INTEGER,
[HTML Preformatted] net_weight_per_carton_kg DECIMAL(10,2),
[HTML Preformatted] gross_weight_per_carton_kg DECIMAL(10,2),
[HTML Preformatted] total_net_weight_kg DECIMAL(10,2),
[HTML Preformatted] total_gross_weight_kg DECIMAL(10,2),
[HTML Preformatted] -- Special requirements (per commodity)
[HTML Preformatted] special_conditions JSONB,
[HTML Preformatted] treatment_details JSONB,
[HTML Preformatted] lab_tests_required JSONB,
[HTML Preformatted] -- Dynamic fields (from Product Form Agent)
[HTML Preformatted] dynamic_fields JSONB,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_commodities_request ON
trade_request_commodities(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_commodities_container ON
trade_request_commodities(container_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_commodities_hs ON
trade_request_commodities(hs_code);

```

4.16.3 Documentation Tables

sql

```

[HTML Preformatted] -- Document requirements (single source of truth)
[HTML Preformatted] CREATE TABLE IF NOT EXISTS document_requirements (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] document_type TEXT NOT NULL,
[HTML Preformatted] triggers document_trigger[] NOT NULL,
[HTML Preformatted] is_mandatory BOOLEAN NOT NULL DEFAULT false,

```

```

[HTML Preformatted] is_pre_selected BOOLEAN DEFAULT false,
[HTML Preformatted] format_requirements JSONB,
[HTML Preformatted] responsible_party TEXT,
[HTML Preformatted] source_regulation TEXT,
[HTML Preformatted] status TEXT DEFAULT 'PENDING',
[HTML Preformatted] uploaded_document_id UUID REFERENCES documents(id) ON DELETE SET
NULL,
[HTML Preformatted] verification_result JSONB,
[HTML Preformatted] fraud_indicators JSONB,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] UNIQUE(trade_request_id, document_type)
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_trade ON
document_requirements(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_type ON
document_requirements(document_type);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_status ON
document_requirements(status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_triggers ON
document_requirements USING GIN(triggers);

[HTML Preformatted] -- Document triggers reference

[HTML Preformatted] CREATE TABLE IF NOT EXISTS document_triggers_ref (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] document_type TEXT NOT NULL,
[HTML Preformatted] trigger_type document_trigger NOT NULL,
[HTML Preformatted] is_default BOOLEAN DEFAULT false,
[HTML Preformatted] source_regulation TEXT,
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] UNIQUE(document_type, trigger_type)
[HTML Preformatted] );

[HTML Preformatted] -- Document format requirements per jurisdiction

[HTML Preformatted] CREATE TABLE IF NOT EXISTS document_format_requirements (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
[HTML Preformatted] document_type TEXT NOT NULL,
[HTML Preformatted] originals_required BOOLEAN DEFAULT false,

```

[HTML Preformatted] electronic_accepted BOOLEAN DEFAULT true,
 [HTML Preformatted] required_languages TEXT[],
 [HTML Preformatted] physical_handling_recommended BOOLEAN DEFAULT false,
 [HTML Preformatted] source_regulation TEXT,
 [HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),
 [HTML Preformatted] UNIQUE(country_code, document_type)
 [HTML Preformatted]);

4.16.4 Special Instructions Tables

sql

[HTML Preformatted] -- Special instructions (free-text + structured)
 [HTML Preformatted] CREATE TABLE IF NOT EXISTS special_instructions (
 [HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 [HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
 CASCADE,
 [HTML Preformatted] raw_text TEXT,
 [HTML Preformatted] structured_instructions JSONB,
 [HTML Preformatted] categories JSONB,
 [HTML Preformatted] missing_information JSONB,
 [HTML Preformatted] compliance_issues JSONB,
 [HTML Preformatted] confidence_score DECIMAL(5,2),
 [HTML Preformatted] validated BOOLEAN DEFAULT false,
 [HTML Preformatted] validated_by UUID REFERENCES employees(id) ON DELETE SET NULL,
 [HTML Preformatted] validated_at TIMESTAMPTZ,
 [HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
 [HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
 [HTML Preformatted]);
 [HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_special_instructions_trade ON
 special_instructions(trade_request_id);
 [HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_special_instructions_categories ON
 special_instructions USING GIN(categories);
 [HTML Preformatted] -- Saved instruction templates
 [HTML Preformatted] CREATE TABLE IF NOT EXISTS instruction_templates (
 [HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 [HTML Preformatted] buyer_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE
 CASCADE,
 [HTML Preformatted] template_name TEXT NOT NULL,

```

[HTML Preformatted] description TEXT,
[HTML Preformatted] instructions TEXT[],
[HTML Preformatted] default_for JSONB,
[HTML Preformatted] usage_count INTEGER DEFAULT 0,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] UNIQUE(buyer_tenant_id, template_name)
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_instruction_templates_buyer ON
instruction_templates(buyer_tenant_id);

[HTML Preformatted] -- Instruction template usage logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS instruction_template_usage (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] template_id UUID NOT NULL REFERENCES instruction_templates(id) ON DELETE
CASCADE,
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] used_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Instruction propagation logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS instruction_propagation_logs (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] instruction_id UUID NOT NULL REFERENCES special_instructions(id) ON DELETE
CASCADE,
[HTML Preformatted] target_service TEXT NOT NULL,
[HTML Preformatted] propagated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] status TEXT DEFAULT 'SUCCESS',
[HTML Preformatted] error_message TEXT
[HTML Preformatted] );

```

4.16.5 Transport & Logistics Tables

sql

```

[HTML Preformatted] -- Transport & Logistics configuration

[HTML Preformatted] CREATE TABLE IF NOT EXISTS transport_configuration (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] transport_mode transport_mode NOT NULL,

```

```

[HTML Preformatted] equipment_type equipment_type NOT NULL,
[HTML Preformatted] equipment_count INTEGER NOT NULL,
[HTML Preformatted] origin_port TEXT,
[HTML Preformatted] destination_port TEXT,
[HTML Preformatted] alternative_ports TEXT[],
[HTML Preformatted] earliest_delivery_date DATE NOT NULL,
[HTML Preformatted] preferred_delivery_date DATE NOT NULL,
[HTML Preformatted] latest_delivery_date DATE NOT NULL,
[HTML Preformatted] transit_time_days INTEGER,
[HTML Preformatted] route_details JSONB,
[HTML Preformatted] port_requirements JSONB,
[HTML Preformatted] congestion_alert JSONB,
[HTML Preformatted] container_advisor_recommendation JSONB,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_transport_configuration_trade ON
transport_configuration(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_transport_configuration_mode ON
transport_configuration(transport_mode);

[HTML Preformatted] -- Equipment types reference
[HTML Preformatted] CREATE TABLE IF NOT EXISTS equipment_types_ref (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] mode transport_mode NOT NULL,
[HTML Preformatted] type_code equipment_type NOT NULL,
[HTML Preformatted] type_name TEXT NOT NULL,
[HTML Preformatted] description TEXT,
[HTML Preformatted] max_payload_kg DECIMAL(10,2),
[HTML Preformatted] max_volume_cbm DECIMAL(10,2),
[HTML Preformatted] is_active BOOLEAN DEFAULT true,
[HTML Preformatted] requires_reefer BOOLEAN DEFAULT false,
[HTML Preformatted] requires_hazardous_handling BOOLEAN DEFAULT false,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] UNIQUE(mode, type_code)
[HTML Preformatted] );

```


[HTML Preformatted] -- Route availability

[HTML Preformatted] CREATE TABLE IF NOT EXISTS route_availability (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] origin_unlocode TEXT NOT NULL REFERENCES port_information(unlocode),

[HTML Preformatted] destination_unlocode TEXT NOT NULL REFERENCES port_information(unlocode),

[HTML Preformatted] mode transport_mode NOT NULL,

[HTML Preformatted] is_available BOOLEAN DEFAULT true,

[HTML Preformatted] transit_time_days INTEGER,

[HTML Preformatted] frequency_per_week INTEGER,

[HTML Preformatted] carriers TEXT[],

[HTML Preformatted] equipment_types equipment_type[],

[HTML Preformatted] last_updated TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] UNIQUE(origin_unlocode, destination_unlocode, mode)

[HTML Preformatted]);

[HTML Preformatted] -- Port information

[HTML Preformatted] CREATE TABLE IF NOT EXISTS port_information (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] unlocode TEXT UNIQUE NOT NULL,

[HTML Preformatted] name TEXT NOT NULL,

[HTML Preformatted] country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),

[HTML Preformatted] latitude DECIMAL(10,8),

[HTML Preformatted] longitude DECIMAL(11,8),

[HTML Preformatted] is_active BOOLEAN DEFAULT true,

[HTML Preformatted] is_sanctioned BOOLEAN DEFAULT false,

[HTML Preformatted] timezone TEXT,

[HTML Preformatted] operating_hours TEXT,

[HTML Preformatted] max_draft_m DECIMAL(5,2),

[HTML Preformatted] annual_teu INTEGER,

[HTML Preformatted] vessel_calls_per_year INTEGER,

[HTML Preformatted] congestion_status TEXT DEFAULT 'NORMAL',

[HTML Preformatted] congestion_waiting_days INTEGER DEFAULT 0,

[HTML Preformatted] requirements JSONB,

[HTML Preformatted] facilities JSONB,

[HTML Preformatted] last_updated TIMESTAMPTZ DEFAULT NOW()

[HTML Preformatted]);

```

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_port_information_unlocode ON
port_information(unlocode);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_port_information_country ON
port_information(country_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_port_information_congestion ON
port_information(congestion_status);

[HTML Preformatted] -- Container advisor logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS container_advisor_logs (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] input_data JSONB NOT NULL,
[HTML Preformatted] recommendation JSONB NOT NULL,
[HTML Preformatted] alternatives JSONB,
[HTML Preformatted] historical_density JSONB,
[HTML Preformatted] selected_option TEXT,
[HTML Preformatted] user_accepted BOOLEAN DEFAULT false,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_container_advisor_logs_trade ON
container_advisor_logs(trade_request_id);

```

4.16.6 Insurance Tables

sql

```

[HTML Preformatted] -- Insurance requirements

[HTML Preformatted] CREATE TABLE IF NOT EXISTS insurance_requirements (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] requirement insurance_requirement NOT NULL,
[HTML Preformatted] responsible_party insurance_responsible_party NOT NULL,
[HTML Preformatted] insurance_type insurance_type,
[HTML Preformatted] coverage_percent INTEGER DEFAULT 110,
[HTML Preformatted] coverage_currency TEXT DEFAULT 'USD',
[HTML Preformatted] policy_number TEXT,
[HTML Preformatted] insurer_name TEXT,
[HTML Preformatted] policy_effective_date DATE,
[HTML Preformatted] policy_expiry_date DATE,

```

```

[HTML Preformatted] certificate_uploaded BOOLEAN DEFAULT false,
[HTML Preformatted] certificate_document_id UUID REFERENCES documents(id) ON DELETE SET
NULL,
[HTML Preformatted] verification_status TEXT DEFAULT 'PENDING',
[HTML Preformatted] verification_result JSONB,
[HTML Preformatted] auto_configured BOOLEAN DEFAULT false,
[HTML Preformatted] auto_config_reason TEXT,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_insurance_requirements_trade ON
insurance_requirements(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_insurance_requirements_status ON
insurance_requirements(verification_status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_insurance_requirements_responsible ON
insurance_requirements(responsible_party);

[HTML Preformatted] -- Insurance type reference
[HTML Preformatted] CREATE TABLE IF NOT EXISTS insurance_types_ref (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] type_code insurance_type UNIQUE NOT NULL,
[HTML Preformatted] type_name TEXT NOT NULL,
[HTML Preformatted] type_name_ar TEXT,
[HTML Preformatted] description TEXT,
[HTML Preformatted] description_ar TEXT,
[HTML Preformatted] coverage_level TEXT,
[HTML Preformatted] typical_use TEXT,
[HTML Preformatted] is_active BOOLEAN DEFAULT true,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Country-specific insurance requirements
[HTML Preformatted] CREATE TABLE IF NOT EXISTS country_insurance_requirements (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
[HTML Preformatted] commodity_category TEXT,
[HTML Preformatted] hs_code CHAR(6),
[HTML Preformatted] requirement TEXT NOT NULL,

```

[HTML Preformatted] min_coverage_percent INTEGER DEFAULT 100,
[HTML Preformatted] recommended_type insurance_type,
[HTML Preformatted] source_regulation TEXT,
[HTML Preformatted] last_updated TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] UNIQUE(country_code, commodity_category, hs_code)
[HTML Preformatted]);

4.16.7 Commercial Settlement Tables

sql

[HTML Preformatted] -- Commercial settlement configuration
[HTML Preformatted] CREATE TABLE IF NOT EXISTS commercial_settlement (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] commercial_priority commercial_priority NOT NULL,
[HTML Preformatted] settlement_structure settlement_structure NOT NULL,
[HTML Preformatted] payment_timing payment_timing NOT NULL,
[HTML Preformatted] credit_period credit_period NOT NULL,
[HTML Preformatted] credit_period_custom_days INTEGER,
[HTML Preformatted] currency TEXT NOT NULL DEFAULT 'USD',
[HTML Preformatted] currency_other TEXT,
[HTML Preformatted] financing_interest financing_interest NOT NULL,
[HTML Preformatted] bank_instrument TEXT,
[HTML Preformatted] settlement_flexibility settlement_flexibility NOT NULL,
[HTML Preformatted] settlement_documents TEXT[],
[HTML Preformatted] partial_advance_percentage INTEGER,
[HTML Preformatted] balance_timing TEXT,
[HTML Preformatted] acceptable_alternatives TEXT[],
[HTML Preformatted] conditions_for_alternatives TEXT,
[HTML Preformatted] auto_configured BOOLEAN DEFAULT false,
[HTML Preformatted] auto_config_reason TEXT,
[HTML Preformatted] ai_readiness_score INTEGER,
[HTML Preformatted] ai_readiness_recommendation TEXT,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted]);

```

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commercial_settlement_trade ON
commercial_settlement(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commercial_settlement_priority ON
commercial_settlement(commercial_priority);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commercial_settlement_structure ON
commercial_settlement(settlement_structure);

[HTML Preformatted] -- Settlement readiness logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS settlement_readiness_logs (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] settlement_id UUID NOT NULL REFERENCES commercial_settlement(id) ON
DELETE CASCADE,

[HTML Preformatted] readiness_score INTEGER NOT NULL,

[HTML Preformatted] completeness_percentage INTEGER,

[HTML Preformatted] missing_documents TEXT[],

[HTML Preformatted] potential_issues JSONB,

[HTML Preformatted] ai_recommendation TEXT,

[HTML Preformatted] groq_model_version TEXT,

[HTML Preformatted] calculated_at TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()

[HTML Preformatted] );

```

4.16.8 Multi-Shipment Tables

sql

```

[HTML Preformatted] -- Contract shipments (multi-shipment)

[HTML Preformatted] CREATE TABLE IF NOT EXISTS contract_shipments (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,

[HTML Preformatted] shipment_number INTEGER NOT NULL,

[HTML Preformatted] scheduled_delivery_date DATE NOT NULL,

[HTML Preformatted] port_of_discharge TEXT NOT NULL,

[HTML Preformatted] containers_count INTEGER NOT NULL,

[HTML Preformatted] total_price DECIMAL(20,4) NOT NULL,

[HTML Preformatted] sgtx_fee_amount DECIMAL(20,4),

[HTML Preformatted] fee_lock_id UUID,

[HTML Preformatted] ustn TEXT UNIQUE,

[HTML Preformatted] status TEXT DEFAULT 'SCHEDULED',

[HTML Preformatted] commodities_overridden BOOLEAN DEFAULT false,

```

```

[HTML Preformatted] commodity_data JSONB,
[HTML Preformatted] locked_at TIMESTAMPTZ,
[HTML Preformatted] completed_at TIMESTAMPTZ,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_contract_shipments_contract ON
contract_shipments(contract_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_contract_shipments_ustn ON
contract_shipments(ustn);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_contract_shipments_status ON
contract_shipments(status);

[HTML Preformatted] -- Schedule modification requests

[HTML Preformatted] CREATE TABLE IF NOT EXISTS schedule_modification_requests (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
[HTML Preformatted] shipment_id UUID NOT NULL REFERENCES contract_shipments(id) ON DELETE
CASCADE,
[HTML Preformatted] requested_by_gtid TEXT NOT NULL,
[HTML Preformatted] requested_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] proposed_changes JSONB NOT NULL,
[HTML Preformatted] reason TEXT NOT NULL,
[HTML Preformatted] status TEXT DEFAULT 'PENDING',
[HTML Preformatted] responded_by_gtid TEXT,
[HTML Preformatted] responded_at TIMESTAMPTZ,
[HTML Preformatted] addendum_contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
[HTML Preformatted] governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON
DELETE SET NULL,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_schedule_modification_requests_contract ON
schedule_modification_requests(contract_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_schedule_modification_requests_shipment
ON schedule_modification_requests(shipment_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_schedule_modification_requests_status ON
schedule_modification_requests(status);

[HTML Preformatted] -- Schedule addenda

```

```

[HTML Preformatted] CREATE TABLE IF NOT EXISTS schedule_addenda (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
[HTML Preformatted] modification_request_id UUID REFERENCES schedule_modification_requests(id)
ON DELETE SET NULL,
[HTML Preformatted] original_schedule JSONB NOT NULL,
[HTML Preformatted] modified_schedule JSONB NOT NULL,
[HTML Preformatted] reason TEXT NOT NULL,
[HTML Preformatted] buyer_signature TEXT NOT NULL,
[HTML Preformatted] seller_signature TEXT NOT NULL,
[HTML Preformatted] sgtx_signature TEXT NOT NULL,
[HTML Preformatted] loom_hash TEXT NOT NULL,
[HTML Preformatted] signed_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

```

4.16.9 Draft & Recovery Tables

sql

```

[HTML Preformatted] -- Draft history (versioned snapshots)
[HTML Preformatted] CREATE TABLE IF NOT EXISTS draft_history (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] draft_snapshot JSONB NOT NULL,
[HTML Preformatted] version INTEGER NOT NULL,
[HTML Preformatted] edited_by UUID REFERENCES employees(id) ON DELETE SET NULL,
[HTML Preformatted] edited_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] change_summary TEXT,
[HTML Preformatted] UNIQUE(trade_request_id, version)
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_draft_history_request ON
draft_history(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_draft_history_edited ON
draft_history(edited_at DESC);

[HTML Preformatted] -- Draft reminders
[HTML Preformatted] CREATE TABLE IF NOT EXISTS draft_reminders (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,

[HTML Preformatted] reminder_type TEXT NOT NULL,

[HTML Preformatted] sent_at TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] sent_to_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,

[HTML Preformatted] sent_by TEXT,

[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()

[HTML Preformatted]);

[HTML Preformatted] -- Draft restore logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS draft_restore_logs (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] original_draft_id UUID REFERENCES trade_requests(id) ON DELETE SET NULL,

[HTML Preformatted] restored_draft_id UUID REFERENCES trade_requests(id) ON DELETE SET NULL,

[HTML Preformatted] restored_by UUID REFERENCES employees(id) ON DELETE SET NULL,

[HTML Preformatted] restore_reason TEXT,

[HTML Preformatted] restored_at TIMESTAMPTZ DEFAULT NOW()

[HTML Preformatted]);

[HTML Preformatted] -- Draft templates

[HTML Preformatted] CREATE TABLE IF NOT EXISTS draft_templates (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,

[HTML Preformatted] template_name TEXT NOT NULL,

[HTML Preformatted] description TEXT,

[HTML Preformatted] template_data JSONB NOT NULL,

[HTML Preformatted] usage_count INTEGER DEFAULT 0,

[HTML Preformatted] is_default BOOLEAN DEFAULT false,

[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] UNIQUE(tenant_id, template_name)

[HTML Preformatted]);

4.16.10 Governor & Validation Tables

sql

[HTML Preformatted] -- Governor decisions (extended)

[HTML Preformatted] ALTER TABLE governor_decisions ADD COLUMN IF NOT EXISTS trade_request_id UUID REFERENCES trade_requests(id) ON DELETE SET NULL;

[HTML Preformatted] ALTER TABLE governor_decisions ADD COLUMN IF NOT EXISTS decision_type TEXT;

[HTML Preformatted] ALTER TABLE governor_decisions ADD COLUMN IF NOT EXISTS actor_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL;

[HTML Preformatted] ALTER TABLE governor_decisions ADD COLUMN IF NOT EXISTS conditions JSONB;

[HTML Preformatted] ALTER TABLE governor_decisions ADD COLUMN IF NOT EXISTS escalation_hint TEXT;

[HTML Preformatted] -- Submission logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS submission_logs (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,

[HTML Preformatted] submitted_by UUID REFERENCES employees(id) ON DELETE SET NULL,

[HTML Preformatted] submitted_at TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,

[HTML Preformatted] verdict TEXT NOT NULL,

[HTML Preformatted] conditions_resolved BOOLEAN DEFAULT false,

[HTML Preformatted] resolved_at TIMESTAMPTZ,

[HTML Preformatted] loom_hash TEXT

[HTML Preformatted]);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_submission_logs_trade_request ON submission_logs(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_submission_logs_verdict ON submission_logs(verdict);

[HTML Preformatted] -- Pre-screen validation cache

[HTML Preformatted] CREATE TABLE IF NOT EXISTS prescreen_validation_cache (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,

[HTML Preformatted] validation_type TEXT NOT NULL,

[HTML Preformatted] validation_result JSONB NOT NULL,

[HTML Preformatted] validated_at TIMESTAMPTZ DEFAULT NOW(),

[HTML Preformatted] expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '5 minutes'

[HTML Preformatted]);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_prescreen_validation_cache_trade ON prescreen_validation_cache(trade_request_id);

4.16.11 Acceptance Criteria & Product Tables

sql

[HTML Preformatted] -- Acceptance criteria templates

[HTML Preformatted] CREATE TABLE IF NOT EXISTS acceptance_criteria_templates (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] hs_code CHAR(6) NOT NULL,

[HTML Preformatted] parameter TEXT NOT NULL,

[HTML Preformatted] default_limit TEXT NOT NULL,

[HTML Preformatted] default_unit TEXT NOT NULL,

[HTML Preformatted] is_mandatory BOOLEAN DEFAULT true,

[HTML Preformatted] source_regulation TEXT,

[HTML Preformatted] ai_confidence DECIMAL(5,2),

[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()

[HTML Preformatted]);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_acceptance_criteria_templates_hs ON acceptance_criteria_templates(hs_code);

[HTML Preformatted] -- Product vectors (for similarity search)

[HTML Preformatted] CREATE TABLE IF NOT EXISTS product_vectors (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] hs_code CHAR(6) NOT NULL,

[HTML Preformatted] product_name TEXT NOT NULL,

[HTML Preformatted] synonyms TEXT[],

[HTML Preformatted] description TEXT,

[HTML Preformatted] embedding vector(384),

[HTML Preformatted] metadata JSONB,

[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()

[HTML Preformatted]);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_product_vectors_hs ON product_vectors(hs_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_product_vectors_name ON product_vectors(product_name);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_product_vectors_embedding ON product_vectors USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

[HTML Preformatted] -- Commodity dynamic schemas cache

[HTML Preformatted] CREATE TABLE IF NOT EXISTS commodity_dynamic_schemas_cache (

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

[HTML Preformatted] hs_code CHAR(6) NOT NULL,

```

[HTML Preformatted] origin_country CHAR(2) NOT NULL,
[HTML Preformatted] destination_country CHAR(2) NOT NULL,
[HTML Preformatted] port_code TEXT,
[HTML Preformatted] schema_json JSONB NOT NULL,
[HTML Preformatted] generated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '7 days',
[HTML Preformatted] product_form_agent_version TEXT,
[HTML Preformatted] ai_confidence DECIMAL(5,2),
[HTML Preformatted] fallback_used BOOLEAN DEFAULT false,
[HTML Preformatted] UNIQUE(hs_code, origin_country, destination_country, port_code)
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commodity_dynamic_schemas_cache_hs ON
commodity_dynamic_schemas_cache(hs_code, origin_country, destination_country);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commodity_dynamic_schemas_cache_expiry
ON commodity_dynamic_schemas_cache(expires_at);

```

4.16.12 Express Mode & AI Logs

sql

```

[HTML Preformatted] -- Express Mode parsing logs

[HTML Preformatted] CREATE TABLE IF NOT EXISTS express_mode_logs (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID REFERENCES trade_requests(id) ON DELETE CASCADE,
[HTML Preformatted] raw_text TEXT NOT NULL,
[HTML Preformatted] parsed_json JSONB,
[HTML Preformatted] confidence DECIMAL(5,2),
[HTML Preformatted] missing_fields TEXT[],
[HTML Preformatted] low_confidence_fields TEXT[],
[HTML Preformatted] user_confirmed BOOLEAN DEFAULT false,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_express_mode_logs_request ON
express_mode_logs(trade_request_id);

[HTML Preformatted] -- Multi-shipment AI suggestions

[HTML Preformatted] CREATE TABLE IF NOT EXISTS multi_shipment_suggestions (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,

```

[HTML Preformatted] suggested_schedule JSONB NOT NULL,
[HTML Preformatted] confidence DECIMAL(5,2),
[HTML Preformatted] factors JSONB,
[HTML Preformatted] applied BOOLEAN DEFAULT false,
[HTML Preformatted] generated_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted]);

4.16.13 Product Form Agent & RIA Tables

sql

[HTML Preformatted] -- RIA-maintained: Commodity packing defaults
[HTML Preformatted] CREATE TABLE IF NOT EXISTS commodity_packing_defaults (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] hs_code CHAR(6) NOT NULL,
[HTML Preformatted] commodity_type TEXT NOT NULL,
[HTML Preformatted] default_pallet_type TEXT DEFAULT 'EUR',
[HTML Preformatted] default_cartons_per_pallet INTEGER DEFAULT 40,
[HTML Preformatted] default_net_weight_per_carton_kg DECIMAL(10,2) DEFAULT 10,
[HTML Preformatted] default_stacking_layers INTEGER DEFAULT 5,
[HTML Preformatted] max_pallets_per_container_40ft INTEGER DEFAULT 22,
[HTML Preformatted] carton_dimensions_mm JSONB,
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted]);
[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commodity_packing_defaults_hs ON
commodity_packing_defaults(hs_code);
[HTML Preformatted] -- RIA-maintained: Treatment requirements
[HTML Preformatted] CREATE TABLE IF NOT EXISTS treatment_requirements (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] hs_code CHAR(6) NOT NULL,
[HTML Preformatted] origin_country CHAR(2) NOT NULL,
[HTML Preformatted] destination_country CHAR(2) NOT NULL,
[HTML Preformatted] port_code TEXT,
[HTML Preformatted] treatment_type TEXT NOT NULL,
[HTML Preformatted] temperature_c DECIMAL(5,2),
[HTML Preformatted] duration_days INTEGER,
[HTML Preformatted] certificate_required BOOLEAN DEFAULT true,

```

[HTML Preformatted] accepted_facilities TEXT[],
[HTML Preformatted] mandate_effective_date DATE,
[HTML Preformatted] source_regulation TEXT,
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_treatment_requirements_hs ON
treatment_requirements(hs_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_treatment_requirements_origin_dest ON
treatment_requirements(origin_country, destination_country);

[HTML Preformatted] -- RIA-maintained: Country MRLs
[HTML Preformatted] CREATE TABLE IF NOT EXISTS country_mrl (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] hs_code CHAR(6) NOT NULL,
[HTML Preformatted] destination_country CHAR(2) NOT NULL,
[HTML Preformatted] analyte TEXT NOT NULL,
[HTML Preformatted] mrl_mg_per_kg DECIMAL(10,4),
[HTML Preformatted] unit TEXT DEFAULT 'mg/kg',
[HTML Preformatted] limit_operator TEXT DEFAULT '<=',
[HTML Preformatted] source_regulation TEXT,
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_country_mrl_hs_dest ON
country_mrl(hs_code, destination_country);

[HTML Preformatted] -- RIA-maintained: Port special rules
[HTML Preformatted] CREATE TABLE IF NOT EXISTS port_special_rules (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] port_code TEXT NOT NULL REFERENCES ports(unlocode) ON DELETE
CASCADE,
[HTML Preformatted] rule_type TEXT NOT NULL,
[HTML Preformatted] description TEXT,
[HTML Preformatted] mandatory BOOLEAN DEFAULT true,
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- RIA-maintained: Regulatory vectors
[HTML Preformatted] CREATE TABLE IF NOT EXISTS regulation_vectors (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] source TEXT NOT NULL,

```

```

[HTML Preformatted] document_type TEXT NOT NULL,
[HTML Preformatted] content TEXT NOT NULL,
[HTML Preformatted] embedding vector(384),
[HTML Preformatted] metadata JSONB,
[HTML Preformatted] scraped_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_regulation_vectors_embedding ON
regulation_vectors USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

4.16.14 Criticality & Readiness Tables

sql

```

[HTML Preformatted] -- Criticality rules

[HTML Preformatted] CREATE TABLE IF NOT EXISTS criticality_rules (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] rule_type TEXT NOT NULL,
[HTML Preformatted] condition JSONB NOT NULL,
[HTML Preformatted] recommended_criticality criticality_level NOT NULL,
[HTML Preformatted] reason_template TEXT NOT NULL,
[HTML Preformatted] is_active BOOLEAN DEFAULT true,
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Criticality overrides

[HTML Preformatted] CREATE TABLE IF NOT EXISTS criticality_overrides (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,
[HTML Preformatted] original_criticality criticality_level NOT NULL,
[HTML Preformatted] new_criticality criticality_level NOT NULL,
[HTML Preformatted] override_reason TEXT,
[HTML Preformatted] override_by UUID REFERENCES employees(id) ON DELETE SET NULL,
[HTML Preformatted] override_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Trade Readiness scores

[HTML Preformatted] CREATE TABLE IF NOT EXISTS trade_readiness_scores (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE
CASCADE,

```

```

[HTML Preformatted] readiness_score INTEGER NOT NULL,
[HTML Preformatted] quality_score DECIMAL(5,2),
[HTML Preformatted] ai_confidence DECIMAL(5,2),
[HTML Preformatted] components JSONB,
[HTML Preformatted] missing_items JSONB,
[HTML Preformatted] incomplete_items JSONB,
[HTML Preformatted] quality_issues JSONB,
[HTML Preformatted] groq_recommendation TEXT,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_readiness_scores_request ON
trade_readiness_scores(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_readiness_scores_score ON
trade_readiness_scores(readiness_score);

```

4.16.15 Reference & Configuration Tables

sql

```

[HTML Preformatted] -- Container types reference

[HTML Preformatted] CREATE TABLE IF NOT EXISTS container_types_ref (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] type_code container_type UNIQUE NOT NULL,
[HTML Preformatted] type_name TEXT NOT NULL,
[HTML Preformatted] internal_width_mm INTEGER NOT NULL,
[HTML Preformatted] internal_length_mm INTEGER NOT NULL,
[HTML Preformatted] internal_height_mm INTEGER NOT NULL,
[HTML Preformatted] max_volume_cbm DECIMAL(10,2) NOT NULL,
[HTML Preformatted] max_payload_kg DECIMAL(10,2) NOT NULL,
[HTML Preformatted] has_reefer BOOLEAN DEFAULT false,
[HTML Preformatted] has_ventilation BOOLEAN DEFAULT false,
[HTML Preformatted] is_hazardous_compatible BOOLEAN DEFAULT false,
[HTML Preformatted] typical_use TEXT,
[HTML Preformatted] is_active BOOLEAN DEFAULT true,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Settlement structures reference

[HTML Preformatted] CREATE TABLE IF NOT EXISTS settlement_structures_ref (

```

```

[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] structure_code settlement_structure UNIQUE NOT NULL,
[HTML Preformatted] structure_name TEXT NOT NULL,
[HTML Preformatted] structure_name_ar TEXT,
[HTML Preformatted] description TEXT,
[HTML Preformatted] description_ar TEXT,
[HTML Preformatted] recommended_for commercial_priority,
[HTML Preformatted] typical_use TEXT,
[HTML Preformatted] is_active BOOLEAN DEFAULT true,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Payment timing reference

[HTML Preformatted] CREATE TABLE IF NOT EXISTS payment_timing_ref (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] timing_code payment_timing UNIQUE NOT NULL,
[HTML Preformatted] timing_name TEXT NOT NULL,
[HTML Preformatted] timing_name_ar TEXT,
[HTML Preformatted] description TEXT,
[HTML Preformatted] description_ar TEXT,
[HTML Preformatted] typical_use TEXT,
[HTML Preformatted] is_active BOOLEAN DEFAULT true,
[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
[HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
[HTML Preformatted] );

[HTML Preformatted] -- Commercial priority reference

[HTML Preformatted] CREATE TABLE IF NOT EXISTS commercial_priority_ref (
[HTML Preformatted] id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
[HTML Preformatted] priority_code commercial_priority UNIQUE NOT NULL,
[HTML Preformatted] priority_name TEXT NOT NULL,
[HTML Preformatted] priority_name_ar TEXT,
[HTML Preformatted] description TEXT,
[HTML Preformatted] description_ar TEXT,
[HTML Preformatted] typical_use TEXT,
[HTML Preformatted] is_active BOOLEAN DEFAULT true,

```


[HTML Preformatted] created_at TIMESTAMPTZ DEFAULT NOW(),
 [HTML Preformatted] updated_at TIMESTAMPTZ DEFAULT NOW()
 [HTML Preformatted]);

4.16.16 Complete Index Summary

sql

[HTML Preformatted] -- All indexes created (complete list for reference)

[HTML Preformatted] -- Trade Requests

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_buyer ON trade_requests(buyer_tenant_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_status ON trade_requests(status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_incoterm ON trade_requests(incoterm);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_created ON trade_requests(created_at DESC);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_criticality ON trade_requests(trade_criticality);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_priority ON trade_requests(trade_criticality, created_at DESC);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_settlement ON trade_requests(settlement_structure);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_transport ON trade_requests(transport_mode);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_insurance ON trade_requests(insurance_requirement);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_readiness ON trade_requests(rfq_readiness_score);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_approval_status ON trade_requests(approval_status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_draft_status ON trade_requests(is_draft) WHERE is_draft = true;

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_draft_expires ON trade_requests(draft_expires_at) WHERE is_draft = true;

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_requests_multi_shipment ON trade_requests(is_multi_shipment) WHERE is_multi_shipment = true;

[HTML Preformatted] -- Containers & Commodities

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_containers_request ON trade_request_containers(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_containers_port ON trade_request_containers(port_of_discharge);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_commodities_request ON trade_request_commodities(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_commodities_container ON trade_request_commodities(container_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_request_commodities_hs ON trade_request_commodities(hs_code);

[HTML Preformatted] -- Documentation

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_trade ON document_requirements(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_type ON document_requirements(document_type);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_status ON document_requirements(status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_document_requirements_triggers ON document_requirements USING GIN(triggers);

[HTML Preformatted] -- Special Instructions

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_special_instructions_trade ON special_instructions(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_special_instructions_categories ON special_instructions USING GIN(categories);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_instruction_templates_buyer ON instruction_templates(buyer_tenant_id);

[HTML Preformatted] -- Transport

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_transport_configuration_trade ON transport_configuration(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_transport_configuration_mode ON transport_configuration(transport_mode);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_port_information_unlocode ON port_information(unlocode);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_port_information_country ON port_information(country_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_port_information_congestion ON port_information(congestion_status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_container_advisor_logs_trade ON container_advisor_logs(trade_request_id);

[HTML Preformatted] -- Insurance

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_insurance_requirements_trade ON insurance_requirements(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_insurance_requirements_status ON insurance_requirements(verification_status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_insurance_requirements_responsible ON insurance_requirements(responsible_party);

[HTML Preformatted] -- Commercial Settlement

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commercial_settlement_trade ON commercial_settlement(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commercial_settlement_priority ON commercial_settlement(commercial_priority);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commercial_settlement_structure ON commercial_settlement(settlement_structure);

[HTML Preformatted] -- Multi-Shipment

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_contract_shipments_contract ON contract_shipments(contract_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_contract_shipments_ustn ON contract_shipments(ustn);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_contract_shipments_status ON contract_shipments(status);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_schedule_modification_requests_contract ON schedule_modification_requests(contract_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_schedule_modification_requests_shipment ON schedule_modification_requests(shipment_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_schedule_modification_requests_status ON schedule_modification_requests(status);

[HTML Preformatted] -- Draft & Recovery

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_draft_history_request ON draft_history(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_draft_history_edited ON draft_history(edited_at DESC);

[HTML Preformatted] -- Governor & Submission

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_governor_decisions_trade_request ON governor_decisions(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_governor_decisions_decision_type ON governor_decisions(decision_type);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_governor_decisions_created ON governor_decisions(created_at DESC);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_submission_logs_trade_request ON submission_logs(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_submission_logs_verdict ON submission_logs(verdict);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_prescreen_validation_cache_trade ON prescreen_validation_cache(trade_request_id);

[HTML Preformatted] -- Product & RIA

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_acceptance_criteria_templates_hs ON acceptance_criteria_templates(hs_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_product_vectors_hs ON product_vectors(hs_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_product_vectors_name ON product_vectors(product_name);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_product_vectors_embedding ON product_vectors USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commodity_dynamic_schemas_cache_hs ON commodity_dynamic_schemas_cache(hs_code, origin_country, destination_country);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commodity_dynamic_schemas_cache_expiry ON commodity_dynamic_schemas_cache(expires_at);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_regulation_vectors_embedding ON regulation_vectors USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_treatment_requirements_hs ON treatment_requirements(hs_code);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_treatment_requirements_origin_dest ON treatment_requirements(origin_country, destination_country);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_country_mrl_hs_dest ON country_mrl(hs_code, destination_country);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_commodity_packing_defaults_hs ON commodity_packing_defaults(hs_code);

[HTML Preformatted] -- Express Mode & AI

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_express_mode_logs_request ON express_mode_logs(trade_request_id);

[HTML Preformatted] -- Criticality & Readiness

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_readiness_scores_request ON trade_readiness_scores(trade_request_id);

[HTML Preformatted] CREATE INDEX IF NOT EXISTS idx_trade_readiness_scores_score ON trade_readiness_scores(readiness_score);

4.16.17 Implementation Checklist (Part 4.16)

4.16.17.1 Core Tables

[ds-markdown-paragraph] Create all tables with appropriate foreign keys

[ds-markdown-paragraph] Add all columns with appropriate data types

[ds-markdown-paragraph] Set up all constraints (UNIQUE, CHECK, NOT NULL)

[ds-markdown-paragraph] Create all indexes for performance

[ds-markdown-paragraph] Enable RLS on all tables

4.16.17.2 Extensions & Enums

[ds-markdown-paragraph] Enable uuid-oss extension

[ds-markdown-paragraph] Enable pgvector extension

[ds-markdown-paragraph] Enable timescaledb extension

[ds-markdown-paragraph] Create all enumerated types

4.16.17.3 Migration & Versioning

[ds-markdown-paragraph] Create migration scripts for each table

[ds-markdown-paragraph] Add rollback scripts

[ds-markdown-paragraph] Version control all DDL

[ds-markdown-paragraph] Add migration validation tests

4.16.17.4 Testing

[ds-markdown-paragraph] Test all table creation

[ds-markdown-paragraph] Test all foreign key constraints

[ds-markdown-paragraph] Test all indexes

[ds-markdown-paragraph] Test RLS policies

[ds-markdown-paragraph] Test migration rollback

4.16.18 AI Authority Summary for Part 4.16

PART 4.17 — IMPLEMENTATION CHECKLIST

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

4.17.0 Purpose & Design Philosophy

This Implementation Checklist provides a comprehensive, actionable roadmap for building the Enhanced New Trade Request feature. It consolidates all implementation tasks from Parts 4.0 through 4.16 into a single, structured checklist organized by service, component, and priority. Every item is traceable to specific blueprint sections.

Key Principles:

Complete coverage — All features from Parts 4.0-4.16 are represented.

Priority-based — Critical, High, Medium, and Low priorities.

Traceable — Each item references the relevant blueprint section.

Actionable — Each item is a concrete, deliverable unit of work.

Sequential — Items are grouped by logical implementation order.

Implementation Priority Definitions:

4.17.1 Data Model & Migrations

4.17.2 Backend Services

4.17.2.1 Core Services

4.17.2.2 Container & Commodity Services

4.17.2.3 Product Form Agent

4.17.2.4 Express Mode

4.17.2.5 Documentation Services

- 4.17.2.6 Special Instructions
- 4.17.2.7 Transport & Logistics
- 4.17.2.8 Insurance Services
- 4.17.2.9 Commercial Settlement
- 4.17.2.10 Multi-Shipment
- 4.17.2.11 Trade Readiness & Criticality
- 4.17.3 Governor & Validation
 - 4.17.3.1 OPA Policies
 - 4.17.3.2 WasmEdge Modules
- 4.17.4 Frontend Components
 - 4.17.4.1 Core Form Components
 - 4.17.4.2 Dynamic Form Components
 - 4.17.4.3 Documentation Components
 - 4.17.4.4 Special Instructions
 - 4.17.4.5 Transport & Logistics
 - 4.17.4.6 Insurance
 - 4.17.4.7 Commercial Settlement
 - 4.17.4.8 Multi-Shipment
 - 4.17.4.9 Readiness & Criticality
 - 4.17.4.10 Express Mode
 - 4.17.4.11 Decision Panel
 - 4.17.4.12 Draft & Recovery
- 4.17.5 RIA Integration
- 4.17.6 AI Integration
- 4.17.7 Quality Assurance
 - 4.17.7.1 Unit Tests
 - 4.17.7.2 Integration Tests
 - 4.17.7.3 End-to-End Tests
 - 4.17.7.4 Performance Tests
- 4.17.8 Documentation
- 4.17.9 Implementation Dependencies

text



■ IMPLEMENTATION DEPENDENCY MAP ■



- ## ■ 1. Data Model (DM-001 to DM-022) ■

- ## ■ 2. Backend Services (BS-001 to BS-066) ■

- ### ■ 3. Governor & Validation (GV-001 to GV-010, OP-001 to OP-007, WM-001 to WM-007) ■

- #### ■ 4. RIA Integration (RI-001 to RI-010) ■

- ## ■ 5. AI Integration (AI-001 to AI-012) ■

- ## ■ 6. Frontend Components (FC-001 to FC-068) ■

- ## ■ 7. Quality Assurance (QT-001 to PT-005) ■

- ## ■ 8. Documentation (DC-001 to DC-006) ■

4.17.10 Summary

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

The AI Authority Summary consolidates all AI-related information for the Enhanced New Trade Request feature into a single comprehensive reference. It defines which AI agents are used, at what authority levels, with what models, and under what constraints. This is the authoritative source for understanding how AI is integrated into the trade request creation process.

A5 is FORBIDDEN — No autonomous execution.

Three-tier fallback — Grog → Ollama → Static templates ensure zero downtime.

Privacy-first — Sensitive data is never sent to external AI providers; local inference is used for privacy-critical tasks.

4.18.1 AI Agent Registry — Part 4 (New Trade Request)

- 4.18.3 Model Training & Update Frequency
- 4.18.4 AI Fallback Chain (Zero-Cost, No Billing Details)
 - 4.18.4.1 Fallback Chain by Authority
 - 4.18.4.2 Provider Availability
 - 4.18.4.3 Fallback Trigger Logic
 - 4.18.4.4 Fallback Logging

All fallback events are logged in ai_inference_records:

```
json
{
  "agent_name": "product_form_agent",
  "authority_level": "A2",
  "provider": "ollama",
  "fallback_used": true,
  "fallback_reason": "groq_rate_limit_exceeded",
  "model": "llama3.2:3b",
  "latency_ms": 450,
  "created_at": "2026-06-20T10:30:00Z"
}
```

- 4.18.5 AI Inference Timeouts
- 4.18.6 AI Agent Integration with Governor
- 4.18.7 AI Decision Matrix (Part 4)
- 4.18.8 AI Compliance & Privacy
 - 4.18.8.1 Data Privacy
 - 4.18.8.2 Compliance Controls
- 4.18.9 AI Monitoring & Performance Metrics
- 4.18.10 AI Agent Implementation Checklist
- 4.18.11 Summary
- 4.18.12 AI Authority Summary — Quick Reference Card

text

■ AI AUTHORITY QUICK REFERENCE — NEW TRADE REQUEST ■

■ ■

■ A1 — ADVISORY (Groq → Ollama → Static) ■

Ecological packaging advisor — suggests sustainable alternatives (recycled strapping, biodegradable materials), now integrated with Special Trade Instructions (e.g., "No wooden pallets").

Carbon footprint calculation — ISO 14067-compliant, CBAM-ready, integrated into the invoice and customs documentation, now accounting for transport mode-specific emissions.

PDF/A-3 archival format — long-term document preservation (ISO 19005-3), now including all new commercial settlement and insurance details.

Key principles (non-marketplace, orchestration-only):

One source of truth — all data originates from the buyer's structured request (parsed_specs) and the seller's packing decisions. No re-entry.

AI-assisted packing optimisation — palletisation solver suggests efficient layouts; ecological advisor suggests sustainable materials (advisory only).

Automated government submission — invoice → ETA; packing list + invoice → Nafeza SAD; certificates → Nafeza (via lab results).

No marketplace suggestions — never recommends specific logistics providers or buyers.

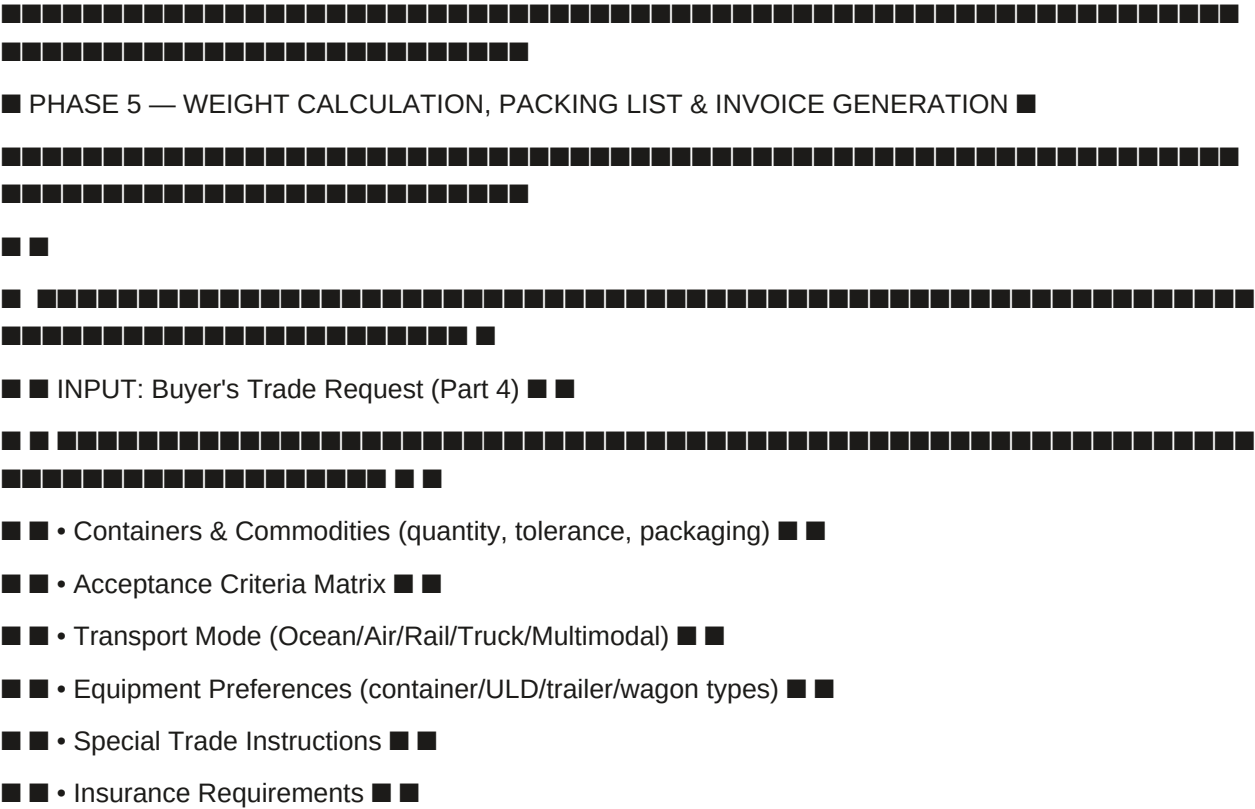
Acceptance criteria integration — packing list now references acceptance criteria so inspectors can verify goods against specifications.

Commercial settlement integration — invoices include payment structure, timing, credit period, currency, and financing interest.

All components are zero-cost, self-hosted, and use open-source libraries (ORTools, Three.js, Yjs, PyTorch for carbon modelling). The generated documents are automatically submitted to government APIs (ETA for invoices, Nafeza for customs declaration) without manual intervention.

5.0.1 Complete Process Flow — Overview

text



FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.1 — NET WEIGHT & GROSS WEIGHT CALCULATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.1.0 Purpose & Design Philosophy

The Net Weight & Gross Weight Calculation engine is the foundational component of the packing process. It transforms the buyer's commercial specifications (commodities, quantities, packaging, tolerances) into precise, validated weight measurements that drive all downstream processes — palletisation, packing list generation, invoicing, customs declaration, and container loading. No physical trade can proceed without accurate weight data.

Key Principles:

Quantity & Tolerance-aware — The calculation respects the buyer's requested quantity and tolerance range, not just a single value.

Packaging-type aware — Different packaging types (Bulk, Bagged, Palletized, Drummed, IBC, Custom) have different tare weights and handling characteristics.

Transport mode-aware — Weight constraints vary by transport mode (ocean containers, air ULDs, rail wagons, truck trailers).

Multi-commodity support — Each commodity is calculated independently and then aggregated at the container and trade level.

Partial shipment & transshipment aware — Weight calculations support splitting across shipments and transshipment legs.

Real-time validation — Weight calculations are validated in real-time against container/equipment capacity limits.

One source of truth — All downstream documents (packing list, invoice, customs declaration) derive their weight data from this calculation.

AI Integration in Part 5.1:

5.1.1 Input Data — Complete Mapping

The weight calculation engine receives the following data from the buyer's structured trade request (Part 4):

5.1.1.1 Per-Commodity Inputs

5.1.1.2 Per-Container Inputs

5.1.1.3 Trade-Level Inputs

5.1.2 Calculation Engine — Core Logic

5.1.2.1 Quantity & Tolerance Calculation

The requested quantity is the starting point, but the tolerance range defines the acceptable variance.

Input:

requested_quantity = 100 (MT)

quantity_unit = "MT"

tolerance = 5 (%)

Calculations:

text

Commodity A:

$\text{total_cartons} = 22 \times 40 = 880$

$\text{net} = 880 \times 10 = 8,800 \text{ kg}$

$\text{gross (cartons)} = 880 \times 10.5 = 9,240 \text{ kg}$

Commodity B:

$\text{total_cartons} = 14 \times 48 = 672$

$\text{net} = 672 \times 8 = 5,376 \text{ kg}$

$\text{gross (cartons)} = 672 \times 8.4 = 5,644.8 \text{ kg}$

Container Totals:

$\text{total_pallets} = 22 + 14 = 36$

$\text{total_net} = 8,800 + 5,376 = 14,176 \text{ kg}$

$\text{gross (cartons)} = 9,240 + 5,644.8 = 14,884.8 \text{ kg}$

$\text{pallet_tare_total} = 36 \times 25 = 900 \text{ kg}$

$\text{total_gross} = 14,884.8 + 900 = 15,784.8 \text{ kg}$

Aggregated Display:

text

CONTAINER WEIGHT SUMMARY — Container 1	
Commodity A (Oranges)	
22 pallets × 40 cartons = 880 cartons	
Net: 8,800 kg Gross: 9,240 kg	
Commodity B (Lemons)	
14 pallets × 48 cartons = 672 cartons	
Net: 5,376 kg Gross: 5,644.8 kg	


```

(TransportMode::Air, EquipmentType::ULD) => 1600,
(TransportMode::Air, EquipmentType::Reefer) => 1600,
(TransportMode::Rail, EquipmentType::BoxWagon) => 60000,
(TransportMode::Rail, EquipmentType::ReeferWagon) => 60000,
(TransportMode::Truck, EquipmentType::DryVan) => 22000,
(TransportMode::Truck, EquipmentType::Reefer) => 22000,
(TransportMode::Truck, EquipmentType::Flatbed) => 24000,
(TransportMode::Truck, EquipmentType::Tank) => 25000,
_ => 28000, // Default fallback
};

// 2. Validate total gross weight against max payload
if total_gross_weight_kg > max_payload as f64 {
return Err(Error::WeightExceeded {
actual: total_gross_weight_kg,
max: max_payload as f64,
equipment: format!("{:?}", transport_mode, equipment_type),
});
}

// 3. Check utilisation (advisory)
let utilisation = (total_gross_weight_kg / max_payload as f64) * 100.0;
if utilisation > 90.0 {
log_warning("Weight utilisation > 90% — consider splitting shipment");
}

Ok(())
}

```

5.1.5 Validation Rules (A4, WasmEdge)

Example CONDITIONAL Response (Weight Exceeded):

```

json
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your packing plan exceeds the maximum payload for a 40ft Reefer container. Total gross weight is 29,500 kg, but the maximum is 28,000 kg.",
    "conditions": [
      {
        "condition_id": "weight_exceeded",

```


| 5.9 Carbon Footprint | Weight data used for emissions calculation |

| 5.11 Lock Packing Plan | Weight data validated before locking |

| 5.12 Label Print | Weight data included in SSCC labels |

5.1.8 Database Schema Additions (Part 5.1)

sql

-- Weight calculation results (stored for audit and reuse)

```
CREATE TABLE weight_calculations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  commodity_id UUID REFERENCES trade_request_commodities(id) ON DELETE SET NULL,  
  container_id UUID REFERENCES trade_request_containers(id) ON DELETE SET NULL,
```

-- Per-commodity

```
  total_cartons INTEGER,  
  total_net_weight_kg DECIMAL(10,2),  
  total_gross_cartons_kg DECIMAL(10,2),  
  pallet_tare_total_kg DECIMAL(10,2),  
  total_gross_weight_kg DECIMAL(10,2),  
  -- Tolerance information  
  requested_quantity DECIMAL(20,4),  
  requested_quantity_unit TEXT,  
  quantity_tolerance DECIMAL(5,2),  
  min_acceptable_quantity DECIMAL(20,4),  
  max_acceptable_quantity DECIMAL(20,4),  
  is_within_tolerance BOOLEAN DEFAULT true,
```

-- Validation results

```
  is_valid BOOLEAN DEFAULT false,  
  validation_errors JSONB,  
  validation_warnings JSONB,
```

-- Metadata

```
  calculation_version TEXT DEFAULT 'v11.0',  
  calculated_at TIMESTAMPTZ DEFAULT NOW(),  
  calculated_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Container weight summaries (aggregated)

```

CREATE TABLE container_weight_summaries (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  container_id UUID NOT NULL REFERENCES trade_request_containers(id) ON DELETE CASCADE,
  total_commodities INTEGER,
  total_pallets INTEGER,
  total_cartons INTEGER,
  total_net_weight_kg DECIMAL(10,2),
  total_gross_weight_kg DECIMAL(10,2),
  pallet_tare_total_kg DECIMAL(10,2),
  total_gross_including_pallets_kg DECIMAL(10,2),
  max_payload_kg DECIMAL(10,2),
  weight_utilisation_percent DECIMAL(5,2),
  is_within_capacity BOOLEAN DEFAULT true,
  validated_at TIMESTAMPTZ DEFAULT NOW(),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

```

CREATE INDEX idx_weight_calculations_trade_request ON weight_calculations(trade_request_id);
CREATE INDEX idx_weight_calculations_commodity ON weight_calculations(commodity_id);
CREATE INDEX idx_weight_calculations_container ON weight_calculations(container_id);
CREATE INDEX idx_container_weight_summaries_container ON
container_weight_summaries(container_id);

```

5.1.9 Implementation Checklist (Part 5.1)

5.1.9.1 Backend Services

- Implement weight calculation service (Rust)
- Implement quantity & tolerance calculation logic
- Implement packaging type tare weight lookup
- Implement transport mode capacity validation (A4, WasmEdge)
- Implement per-commodity weight calculation
- Implement multi-commodity aggregation
- Implement real-time weight update endpoint (POST /v1/packing/weight/calculate)

5.1.9.2 Validation Logic

- Add W1: Total gross weight $\leq$ equipment max payload
- Add W2: Total volume $\leq$ equipment max volume
- Add W3: Per-commodity net weight $\geq$ min tolerance
- Add W4: Per-commodity net weight $\leq$ max tolerance

Add W5: Pallet stacking height $\leq$ commodity-specific limit

Add W6: No incompatible commodities mixed

Add W7: Packaging type compatible with commodity

Add W8: Weight calculation consistency matches declared total

5.1.9.3 Frontend Components

Build Weight Summary component (real-time)

Build Capacity Utilisation gauge

Build Tolerance status indicators

Build Per-commodity weight display

Build Transport mode-specific capacity display

5.1.9.4 Testing

Unit tests: per-commodity weight calculation

Unit tests: multi-commodity aggregation

Unit tests: quantity & tolerance logic

Unit tests: transport mode capacity validation

Integration tests: weight calculation $\rightarrow$ validation $\rightarrow$ display

Integration tests: tolerance violation handling

Integration tests: weight exceedance handling

End-to-end tests: full weight calculation workflow

5.1.10 AI Authority Summary for Part 5.1

END OF PART 5.1 — NET WEIGHT & GROSS WEIGHT CALCULATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.2 — PALLETISATION OPTIMISER (ORTools)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.2.0 Purpose & Design Philosophy

The Palletisation Optimiser is the intelligent engine that transforms raw commodity data into a physically feasible, optimised loading plan. It uses Google's ORTools CP-SAT solver to determine the optimal arrangement of cartons on pallets, pallets in containers (or ULDs, wagons, trailers), and the overall loading sequence. This is not a simple calculation — it is a complex spatial optimisation problem that accounts for:

Carton dimensions and weight — Different products have different carton sizes and weights.

Pallet type and dimensions — EUR, ISO, custom pallets with specific dimensions.

Stacking height limits — Commodity-specific limits (e.g., oranges can stack 5 layers, but electronics only 3).

Non-uniform layer stacking — Real-world packing often requires mixed layer configurations (e.g., 11 layers of 10 cartons + 1 layer of 1 carton).

Transport mode-specific equipment — Ocean containers, air ULDs, rail wagons, and truck trailers all have different dimensions and constraints.

Weight distribution — Centre of gravity constraints for stability.

Accessibility — Critical pallets (e.g., those requiring inspection) must be accessible.

Key Principles:

Transport mode-aware — The solver adapts to the specific equipment type (container, ULD, wagon, trailer).

Non-uniform layer stacking — Supports multiple layer patterns per pallet, reflecting real-world packing practices.

AI-assisted — The solver suggests optimal configurations; human can override.

Collaborative — Multiple users can edit the packing plan simultaneously (Yjs + WebRTC).

Visual — 3D container viewer shows the loading plan; capacity heatmap shows historical density.

Governance-enforced — The Governor validates that all constraints are met before locking the plan.

AI Integration in Part 5.2:

5.2.1 Core Solver — ORTools CP-SAT

5.2.1.1 Input Parameters

5.2.1.2 Solver Output

5.2.1.3 Solver Algorithm

rust

// Pseudo-code: ORTools CP-SAT solver integration

```
fn solve_palletisation(
  cartons: &[Carton],
  pallet_type: &PalletType,
  container_type: &ContainerType,
  layer_patterns: &[LayerPattern],
  max_stacking_height: f64,
) -> PalletisationResult {
  // 1. Calculate cartons per layer for each pattern
  let cartons_per_layer = layer_patterns.iter()
    .map(|p| p.cartons_per_layer)
    .collect::<Vec<_>>();
  // 2. Calculate layers for each pattern
  let layers_per_pattern = layer_patterns.iter()
    .map(|p| p.layers_count)
    .collect::<Vec<_>>();
  // 3. Calculate total cartons per pallet
  let total_cartons_per_pallet = layer_patterns.iter()
    .map(|p| p.cartons_per_layer * p.layers_count)
```

```

.sum::<i32>());
// 4. Calculate pallet height (including non-uniform layers)
let pallet_height = layer_patterns.iter()
.map(|p| p.layer_height_mm * p.layers_count)
.sum::<f64>() + pallet_type.deck_height_mm;
// 5. Validate against max stacking height
if pallet_height > max_stacking_height {
return Err(Error::StackingHeightExceeded {
actual: pallet_height,
max: max_stacking_height,
});
}
// 6. Calculate pallet weight
let pallet_weight = total_cartons_per_pallet as f64 * cartons[0].gross_weight_kg;
// 7. Validate against pallet max payload
if pallet_weight > pallet_type.max_payload_kg {
return Err(Error::PalletWeightExceeded {
actual: pallet_weight,
max: pallet_type.max_payload_kg,
});
}
// 8. Calculate container loading plan (pallet positions)
let container_plan = calculate_container_loading(
pallet_count,
pallet_type.dimensions,
container_type.internal_dimensions,
);
// 9. Calculate centre of gravity
let cog = calculate_centre_of_gravity(&container_plan, &pallet_weights);
// 10. Validate centre of gravity
if cog.x > container_type.internal_width_mm * 0.4 {
return Err(Error::CentreOfGravityExceeded);
}
Ok(PalletisationResult {
total_pallets: pallet_count,

```


$$\text{Pallet Weight} = 111 \times (10 + 0.5) = 111 \times 10.5 = 1165.5 \text{ kg}$$

Example CONDITIONAL Response:

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your packing plan exceeds the maximum stacking height for Oranges. Current height is 2,544 mm, but the maximum is 1,600 mm.",
    "conditions": [
      {
        "condition_id": "stacking_height_exceeded",
        "label": "Reduce layers or adjust layer heights",
        "action_url": "/v1/packing/layer-pattern?trade_id=..."
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  }
}
```

5.2.3.1 Ocean — Container Loading

text

[illegible]

5.2.4.1 3D Viewer (Three.js)

Features:

Click on any pallet — Shows SSCC, product, weight, lot number, treatment status, and layer breakdown (for non-uniform stacks).

Export 3D model — As an STL file for warehouse display.

UI Example:


```

CREATE TABLE pallet_details (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
  sssc TEXT UNIQUE NOT NULL,
  pallet_number INTEGER NOT NULL,
  lot_number TEXT,
  product_hs_code CHAR(6),
  product_name TEXT,
  cartons_per_pallet INTEGER NOT NULL,
  layer_patterns JSONB, -- [{layers:11, cartons_per_layer:10}, {layers:1, cartons_per_layer:1}]
  total_cartons INTEGER NOT NULL,
  net_weight_kg DECIMAL(10,2),
  gross_weight_kg DECIMAL(10,2),
  pallet_height_mm INTEGER,
  cold_treatment_cert_ref TEXT,
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'LOADED', 'DAMAGED', 'DELIVERED'
  position_x INTEGER, -- Container position X (mm)
  position_y INTEGER, -- Container position Y (mm)
  position_z INTEGER, -- Container position Z (mm)
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Container loading plans (3D layout)
CREATE TABLE container_loading_plans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
  container_id UUID REFERENCES trade_request_containers(id) ON DELETE SET NULL,
  container_type TEXT NOT NULL,
  pallet_positions JSONB, -- [{pallet_id: UUID, x: mm, y: mm, z: mm}]
  total_pallets INTEGER,
  total_cartons INTEGER,
  total_weight_kg DECIMAL(10,2),
  utilisation_percent DECIMAL(5,2),
  centre_of_gravity JSONB, -- {x: mm, y: mm, z: mm}
  three_d_layout_data JSONB, -- For Three.js rendering

```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Collaborative editing sessions
CREATE TABLE collaborative_sessions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
session_token TEXT UNIQUE NOT NULL,
user_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
user_name TEXT,
cursor_position JSONB,
last_activity_at TIMESTAMPTZ DEFAULT NOW(),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Loading guide versions (history)
CREATE TABLE loading_guide_versions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
loading_guide TEXT NOT NULL,
version INTEGER NOT NULL,
generated_by UUID REFERENCES employees(id) ON DELETE SET NULL,
generated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(packing_plan_id, version)
);

-- Indexes
CREATE INDEX idx_packing_plans_trade_request ON packing_plans(trade_request_id);
CREATE INDEX idx_packing_plans_container ON packing_plans(container_id);
CREATE INDEX idx_packing_plans_locked_at ON packing_plans(locked_at) WHERE locked_at IS NOT NULL;

CREATE INDEX idx_pallet_details_packing_plan ON pallet_details(packing_plan_id);
CREATE INDEX idx_pallet_details_sccc ON pallet_details(sscc);
CREATE INDEX idx_pallet_details_status ON pallet_details(status);
CREATE INDEX idx_container_loading_plans_packing_plan ON
container_loading_plans(packing_plan_id);
CREATE INDEX idx_collaborative_sessions_packing_plan ON collaborative_sessions(packing_plan_id);
CREATE INDEX idx_collaborative_sessions_last_activity ON collaborative_sessions(last_activity_at);

```

5.2.9 Implementation Checklist (Part 5.2)

5.2.9.1 Core Solver

Implement ORTools CP-SAT solver integration (Rust + Python)

Implement palletisation feasibility check

Implement non-uniform layer stacking logic

Implement transport mode-aware equipment loading

Implement centre of gravity calculation

Implement weight distribution validation

5.2.9.2 Non-Uniform Layer Stacking

Implement layer pattern UI (add/remove patterns)

Implement calculation logic (total cartons, pallet height, pallet weight)

Implement validation rules ($\text{height} \leq \text{max}$, $\text{weight} \leq \text{max}$)

Implement orientation options (Standard, Cross-stacked, Rotated 90°, Centered)

5.2.9.3 3D Viewer & Heatmap

Implement Three.js 3D container viewer

Implement pallet click → details display

Implement layer breakdown display for non-uniform stacks

Implement capacity heatmap (OpenDP, differential privacy)

Implement export functionality (STL, PNG, PDF)

5.2.9.4 Collaborative Editing

Implement Yjs CRDT integration

Implement WebRTC peer-to-peer communication

Implement cursor presence (name, colour)

Implement chat sidebar

Implement OPA permission checks (who can edit what)

5.2.9.5 Loading Guide Generation

Implement loading guide prompt engineering (A1, Groq)

Implement loading guide display

Implement loading guide versioning

Implement loading guide export (PDF)

5.2.9.6 Validation & Governance

Add G2U10: Palletisation feasibility

Add G2U11: Container type legal for commodity

Add G2U12: Lot mixing allowed by importing country

Add G2U13: Container gross weight within limit

Add G2U14: Packing plan lock → Loom hash

Add G2U15: AI loading instructions generated (advisory)

5.2.9.7 Testing

Unit tests: palletisation solver

Unit tests: non-uniform layer stacking

Unit tests: transport mode-aware loading

Integration tests: solver → validation → lock

Integration tests: collaborative editing

Integration tests: loading guide generation

End-to-end tests: full palletisation workflow

5.2.10 AI Authority Summary for Part 5.2

END OF PART 5.2 — PALLETISATION OPTIMISER (ORTools)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.3 — PACKING LIST AUTO-GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.3.0 Purpose & Design Philosophy

The Packing List Auto-Generation module transforms the validated packing plan into a professional, legally compliant, and operationally complete packing list document. The packing list serves as the single source of truth for all physical handling — warehouse loading, customs inspection, shipping line booking, and buyer receipt verification.

Key Principles:

One-click generation — After the packing plan is locked, the packing list is generated automatically with a single click.

Full integration — Includes all data from the trade request, weight calculation, and palletisation optimiser.

Verifiable identity — Every pallet gets an SSCC-18 barcode and a QR code with a verifiable credential.

Layer transparency — Non-uniform layer stacking is clearly documented.

Acceptance criteria integration — The packing list references acceptance criteria so inspectors can verify goods against specifications.

Special instructions included — All special trade instructions (labeling, handling, certifications) are embedded in the packing list.

Multi-format — Available as PDF (human-readable), JSON (machine-readable), and XML (customs-compatible).

Multi-language — Generated in the buyer's or seller's preferred language.

Cryptographically signed — The packing list is signed with Ed25519, and a Loom hash is recorded for immutability.

AI Integration in Part 5.3:

5.3.1 Document Structure — Complete Specification

5.3.1.1 PDF Structure

text

- Halal certification required.
- Temperature logger required in each container.
- Original Bill of Lading to be sent by DHL.



WEIGHT SUMMARY



Total Pallets: 36
Total Cartons: 1,552
Total Net Weight: 14,176 kg
Total Gross Weight (cartons only): 14,884.8 kg
Pallet Tare: 900 kg



TOTAL GROSS WEIGHT (including pallets): 15,784.8 kg



Container: 40ft Reefer ■ Max Payload: 28,000 kg ■ Utilisation: 56.4%



QR CODE



[QR Code]
Scan to verify: <https://sgtx.io/verify/ustn/SGTX-1397F3A-...-A1B2C3D4>



DIGITAL SIGNATURE



Signed by: SGTX Platform
Algorithm: Ed25519
Signature: MIAGCSqGSIb3DQEHAqCAMIACAQExDzANBgIghkgBZQMEEAgEFADCABgkqhkiG9w0BBwEAAKCAMIIB...
Loom Hash: sha256:a1b2c3d4e5f6...



END OF DOCUMENT



5.3.1.2 JSON Structure

json

```
{
  "packing_list_id": "PL-2026-001",
  "ustn": "SGTX-EG-26-F3A-1",
  "contract_id": "MC-20260415-001",
  "generated_at": "2026-06-20T10:30:00Z",
  "version": 1,
  "loom_hash": "sha256:a1b2c3...",
  "parties": {
    "seller": {
      "gtid": "SGTX-EG-TRD-002139-7F3A",
      "legal_name": "Strawberry Export Co.",
      "tax_id": "123-456-789",
      "address": "123 Export Road, Alexandria, Egypt"
    },
    "buyer": {
      "gtid": "SGTX-DE-TRD-001234-5B6C",
      "legal_name": "European Importer GmbH",
      "tax_id": "DE123456789",
      "address": "456 Import Street, Hamburg, Germany"
    }
  },
  "shipment_details": {
    "transport_mode": "OCEAN",
    "vessel": "MSC FIONA",
    "voyage": "MSF123E",
    "port_of_loading": "EGDAM",
    "port_of_discharge": "DEHAM",
    "etd": "2026-06-20T08:00:00Z",
    "eta": "2026-07-05T14:00:00Z",
    "incoterm": "CFR",
    "trade_criticality": "PRIORITY",
```

```
"partial_shipment": false,
"transshipment": false
},
"documents": {
"commercial_invoice": "INV-STR-2026-001",
"bill_of_lading": null,
"phytosanitary_certificate": null,
"health_certificate": null,
"certificate_of_origin": null,
"insurance_certificate": "INS-2026-001"
},
"commodities": [
{
"product_name": "Valencia Oranges",
"hs_code": "0805.10",
"requested_quantity": 100,
"quantity_unit": "MT",
"tolerance": 5,
"total_net_weight_kg": 8800,
"total_gross_weight_kg": 9240,
"acceptance_criteria": [
{"parameter": "Moisture", "limit": "Max 14%", "unit": "%"},
{"parameter": "Protein", "limit": "Min 12%", "unit": "%"},
{"parameter": "Purity", "limit": "Min 99%", "unit": "%"}
]
},
{
"product_name": "Eureka Lemons",
"hs_code": "0805.50",
"requested_quantity": 50,
"quantity_unit": "MT",
"tolerance": 5,
"total_net_weight_kg": 5376,
"total_gross_weight_kg": 5644.8,
"acceptance_criteria": [
```

```
{
  "parameter": "Moisture", "limit": "Max 15%", "unit": "%"},
  {"parameter": "Acidity", "limit": "Min 5%", "unit": "%"},
  {"parameter": "Purity", "limit": "Min 98%", "unit": "%"}
]
},
{
  "pallets": [
    {
      "pallet_id": "OR-001",
      "sscc": "106141411234567890",
      "product_name": "Valencia Oranges",
      "cartons_per_pallet": 111,
      "layer_patterns": [
        {"layers": 11, "cartons_per_layer": 10, "orientation": "STANDARD"},
        {"layers": 1, "cartons_per_layer": 1, "orientation": "CENTERED"}
      ],
      "net_weight_kg": 1110,
      "gross_weight_kg": 1165,
      "pallet_height_mm": 2544,
      "cold_treatment_cert_ref": "CT-2026-001",
      "position_x": 0,
      "position_y": 0,
      "position_z": 0
    }
  ],
  "weight_summary": {
    "total_pallets": 36,
    "total_cartons": 1552,
    "total_net_weight_kg": 14176,
    "total_gross_cartons_kg": 14884.8,
    "pallet_tare_kg": 900,
    "total_gross_weight_kg": 15784.8,
    "max_payload_kg": 28000,
    "utilisation_percent": 56.4
  },
}
```

```

"special_instructions": [
  "Arabic labels mandatory on all cartons.",
  "No wooden pallets (ISPM 15 compliant only).",
  "Halal certification required.",
  "Temperature logger required in each container.",
  "Original Bill of Lading to be sent by DHL."
],
"treatment_certificates": [
  {
    "type": "COLD_TREATMENT",
    "reference": "CT-2026-001",
    "issuer": "Egyptian Cold Treatment Center",
    "status": "COMPLETED"
  },
  {
    "type": "PHYTOSANITARY",
    "reference": null,
    "issuer": "Egyptian Plant Quarantine",
    "status": "PENDING"
  }
],
"qr_code_url": "https://sgtx.io/verify/ustn/SGTX-EG-26-F3A-1",
"digital_signature": "MIAGCSqGS1b3DQEHAqCAMIACAQExDzANBgIghkgBZQMEAgEFADCABgkqhkiG9w0BBwEAAKCAMIIB...",
"signature_algorithm": "Ed25519",
"verified": true
}

```

5.3.2 SSCC-18 Barcode Generation

5.3.2.1 Format Specification

Each pallet receives a Serial Shipping Container Code (SSCC-18) conforming to GS1 standard:

Format: 1 0614141 123456789 0 → displayed as 106141411234567890

5.3.2.2 Generation Algorithm

rust

```

fn generate_ssc(extension_digit: u8, company_prefix: &str, serial: u64) -> String {
  let base = format!("{}", extension_digit, company_prefix, serial);
  let check_digit = calculate_gs1_check_digit(&base);

```

```

format!("{}", base, check_digit)
}
fn calculate_gs1_check_digit(base: &str) -> u8 {
let mut sum = 0;
for (i, c) in base.chars().enumerate() {
let digit = c.to_digit(10).unwrap();
if i % 2 == 0 {
sum += digit * 3;
} else {
sum += digit * 1;
}
}
let remainder = sum % 10;
if remainder == 0 { 0 } else { 10 - remainder }
}

```

Example:

text

base = "1" + "0614141" + "000000001" = "10614141000000001"

check_digit = 8

sscc = "106141410000000018"

5.3.2.3 SSCC Storage

sql

-- SSCC allocation (per pallet)

```

CREATE TABLE ssc_allocations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
pallet_id UUID NOT NULL REFERENCES pallet_details(id) ON DELETE CASCADE,
sscc TEXT UNIQUE NOT NULL,
company_prefix TEXT NOT NULL DEFAULT '0614141',
serial_number BIGINT NOT NULL,
check_digit INTEGER NOT NULL,
allocated_at TIMESTAMPTZ DEFAULT NOW(),
allocated_by UUID REFERENCES employees(id) ON DELETE SET NULL,
UNIQUE(pallet_id, sscc)
);

```

5.3.3 QR Code Content (Verifiable Credential)

5.3.3.1 QR Payload Structure

Each pallet's QR code encodes a JSON payload with:

```
json
{
  "ustn": "SGTX-EG-26-F3A-1",
  "pallet_id": "OR-001",
  "sscc": "106141411234567890",
  "product": "Fresh Oranges",
  "hs_code": "0805.10",
  "lot": "OR-2026-0412",
  "net_weight_kg": 1110,
  "gross_weight_kg": 1165,
  "layer_summary": "11×10 + 1×1 cartons",
  "cold_treatment_cert": "CT-20260415-001",
  "verify_url": "https://sgtx.io/verify/pallet?sscc=106141411234567890",
  "verifiable_credential": {
    "type": "VerifiableCredential",
    "issuer": "https://sgtx.io/issuers/platform",
    "issuanceDate": "2026-06-20T10:30:00Z",
    "proof": {
      "type": "Ed25519Signature2020",
      "signature": "base58..."
    }
  }
}
```

5.3.3.2 Verifiable Credential (W3C Standard)

```
json
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "id": "https://sgtx.io/credentials/pallet/106141411234567890",
  "type": ["VerifiableCredential", "PalletCredential"],
  "issuer": "https://sgtx.io/issuers/platform",
  "issuanceDate": "2026-06-20T10:30:00Z",
  "credentialSubject": {
    "sscc": "106141411234567890",
```



```

"ustn": "SGTX-EG-26-F3A-1",
"product": "Fresh Oranges",
"hs_code": "0805.10",
"lot": "OR-2026-0412",
"net_weight_kg": 1110,
"gross_weight_kg": 1165,
"origin_country": "EG",
"destination_country": "DE",
"cold_treatment_cert": "CT-20260415-001",
"seller_gtid": "SGTX-EG-TRD-002139-7F3A",
"buyer_gtid": "SGTX-DE-TRD-001234-5B6C"
},
"proof": {
  "type": "Ed25519Signature2020",
  "created": "2026-06-20T10:30:00Z",
  "proofPurpose": "assertionMethod",
  "verificationMethod": "https://sgtx.io/keys/ed25519",
  "signature": "base58..."
}
}

```

5.3.3.3 Offline Verification Flow

Receiver scans QR code with mobile app (offline).

App extracts Verifiable Credential.

App checks credential's signature against cached SGTX public key.

App validates that the credential has not expired.

App displays green checkmark: "Verified — Lot OR-2026-0412, 111 cartons, 1,110 kg. No internet required."

Technology: ssi crate (Rust) + plonky3 for lightweight ZK proofs.

5.3.4 Multi-Language Support

5.3.4.1 Language Selection

The packing list can be generated in the buyer's or seller's preferred language, or both.

5.3.4.2 Translation (A1, Groq)

Product names, handling instructions, and special instructions are translated using Groq (fallback Ollama).

Prompt Template:

text

Translate the following packing list content from English to {target_language}:


```

#####
#####

```

Inspector receives packing list (PDF or mobile app).

Inspector reviews the acceptance criteria matrix.

Inspector performs inspection against each parameter.

Inspector marks each parameter as PASS/FAIL in the QC report.

QC report references the packing list ID and acceptance criteria.

5.3.6 Special Trade Instructions Integration

5.3.6.1 Purpose

All special trade instructions (from Part 4.6) are embedded in the packing list so that warehouse staff, logistics providers, and customs officials can see them.

5.3.6.2 UI Display

text

- Arabic labels mandatory on all cartons.
- No wooden pallets (ISPM 15 compliant only).
- Halal certification required.
- Temperature logger required in each container.
- Original Bill of Lading to be sent by DHL.
- Inspection must be witnessed by buyer's representative.
- Arbitration: DIFC-LCIA, Dubai.

5.3.6.3 Categorisation

Instructions are categorised for easy reference:

5.3.7 Document Formats

5.3.7.1 PDF Generation

Technology: Tera templates + headless Chrome (or printpdf Rust library)

Process:

Render HTML template with data.

Convert to PDF using headless Chrome.

Apply PDF/A-3 archival format.

Sign digitally with Ed25519.

Add Loom hash.

Store in documents table.

HTML Template Example:

```
html
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Packing List — {{ ustn }}</title>
<style>
body { font-family: Arial, sans-serif; margin: 40px; }
table { width: 100%; border-collapse: collapse; }
th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
.header { text-align: center; font-size: 24px; font-weight: bold; }
.metadata { font-size: 12px; color: #666; margin-top: 10px; }
</style>
</head>
<body>
<div class="header">PACKING LIST</div>
<div class="metadata">
USTN: {{ ustn }}<br>
Generated: {{ generated_at }}<br>
Loom Hash: {{ loom_hash }}
</div>
<!-- Table content -->
</body>
</html>
```

5.3.8 Document Signing & Verification

5.3.8.1 Signing Process

Document is generated (PDF/JSON/XML).

Document content is hashed (SHA256).

Hash is signed with Ed25519.

Signature is added to document metadata.

Loom hash is recorded.

Document is stored in documents table.

5.3.8.2 Verification Process

Receiver downloads document.

Receiver extracts signature and hash.

Receiver verifies signature against SGTx public key.

Receiver verifies hash matches document content.

Receiver verifies Loom hash on chain.

If all checks pass, document is verified.

Verification Endpoint: GET /v1/verify/packing-list/{document_id}

5.3.9 Validation Gates (Part 5.3)

5.3.10 Database Schema Additions (Part 5.3)

sql

-- Packing list versions (history)

```
CREATE TABLE packing_list_versions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,  
  document_id UUID REFERENCES documents(id) ON DELETE SET NULL,  
  version INTEGER NOT NULL,  
  pdf_url TEXT,  
  json_data JSONB,  
  xml_data TEXT,  
  generated_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
  generated_at TIMESTAMPTZ DEFAULT NOW(),  
  loom_hash TEXT,  
  digital_signature TEXT,  
  is_latest BOOLEAN DEFAULT false,  
  UNIQUE(packing_plan_id, version)  
);
```

-- Packing list translations

```
CREATE TABLE packing_list_translations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  packing_list_id UUID NOT NULL REFERENCES packing_list_versions(id) ON DELETE CASCADE,  
  language_code TEXT NOT NULL,  
  pdf_url TEXT,  
  json_data JSONB,  
  translated_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
  translated_at TIMESTAMPTZ DEFAULT NOW(),
```

```

UNIQUE(packing_list_id, language_code)
);
-- QR code logs
CREATE TABLE qr_code_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
pallet_id UUID NOT NULL REFERENCES pallet_details(id) ON DELETE CASCADE,
qr_code_data TEXT NOT NULL,
verifiable_credential JSONB,
scanned_at TIMESTAMPTZ,
scanned_by UUID REFERENCES employees(id) ON DELETE SET NULL,
scan_location JSONB,
verification_status TEXT DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_packing_list_versions_packing_plan ON packing_list_versions(packing_plan_id);
CREATE INDEX idx_packing_list_versions_is_latest ON packing_list_versions(is_latest) WHERE is_latest = true;
CREATE INDEX idx_packing_list_translations_packing_list ON packing_list_translations(packing_list_id);
CREATE INDEX idx_qr_code_logs_pallet ON qr_code_logs(pallet_id);
CREATE INDEX idx_qr_code_logs_scanned_at ON qr_code_logs(scanned_at);

```

5.3.11 Implementation Checklist (Part 5.3)

5.3.11.1 PDF Generation

- Implement Tera template rendering
- Implement headless Chrome PDF generation (or printpdf)
- Implement PDF/A-3 archival format
- Implement digital signing (Ed25519)
- Implement Loom hash attachment
- Implement PDF versioning

5.3.11.2 JSON/XML Generation

- Implement JSON serialisation
- Implement XML serialisation (customs-compatible)
- Implement JSON schema validation
- Implement XML schema validation

5.3.11.3 SSCC Generation

- Implement GS1 SSCC-18 generation

Implement check digit calculation

Implement SSCC uniqueness validation

Implement SSCC storage and tracking

5.3.11.4 QR Code & Verifiable Credential

Implement QR code generation (qrcode crate)

Implement Verifiable Credential generation (W3C standard)

Implement Ed25519 signing of credentials

Implement offline verification flow

Implement QR code scanning (mobile)

5.3.11.5 Multi-Language Support

Implement language selection UI

Implement Groq translation (with Ollama fallback)

Implement translation storage

Implement multi-language PDF generation

5.3.11.6 Acceptance Criteria Integration

Include acceptance criteria in packing list

Format acceptance criteria for inspection

Link to QC report workflow

5.3.11.7 Special Instructions Integration

Include all special instructions

Categorise instructions by type

Format instructions for clarity

5.3.11.8 Testing

Unit tests: PDF generation

Unit tests: JSON/XML generation

Unit tests: SSCC generation

Unit tests: QR code generation

Integration tests: packing list → document storage

Integration tests: offline verification

End-to-end tests: full packing list workflow

5.3.12 AI Authority Summary for Part 5.3

END OF PART 5.3 — PACKING LIST AUTO-GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.4 — COLLABORATIVE PACKING PLAN EDITING

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.4.0 Purpose & Design Philosophy

The Collaborative Packing Plan Editing module enables multiple employees of the same seller tenant to edit the same packing plan simultaneously, in real time. This is critical for complex trades where different roles (trade manager, warehouse operator, logistics coordinator, quality inspector) need to contribute to the packing plan concurrently. The system uses Conflict-free Replicated Data Types (CRDTs) to ensure that changes are merged deterministically and consistently across all participants.

Key Principles:

Real-time collaboration — Multiple users can edit the same packing plan simultaneously with latency <200ms.

Conflict resolution — CRDTs ensure deterministic merge; no data loss.

Permission-aware — OPA policies restrict what each user can edit based on role (e.g., warehouse operator can only modify pallet positions, not commercial fields).

Presence awareness — Each user's cursor is visible with their name and colour.

Offline support — Local changes are queued and synced when connectivity is restored.

Audit trail — Every change is logged immutably via Loom.

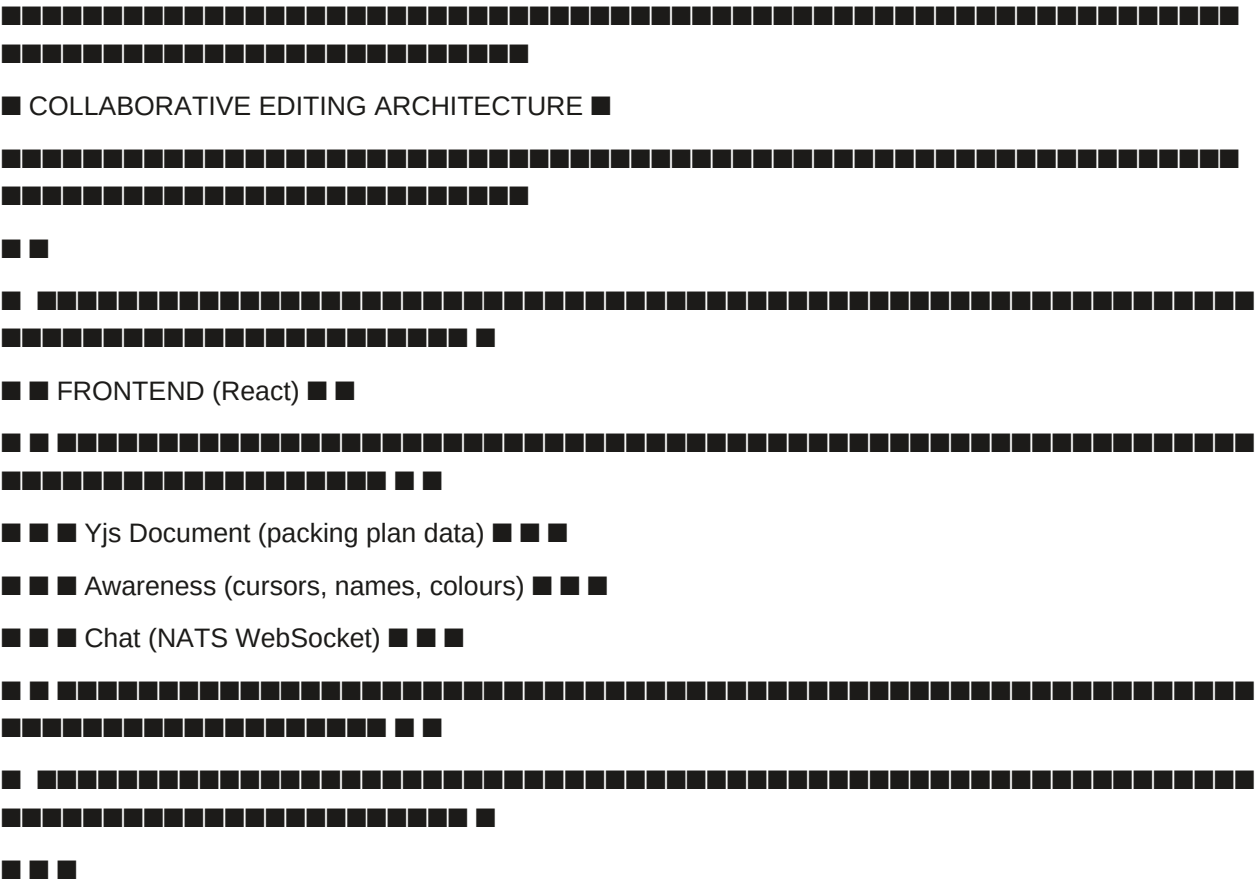
Locking — Only users with packing.lock permission can lock the final plan.

AI Integration in Part 5.4:

5.4.1 Technology Stack

5.4.2 Architecture Overview

text



▼

NETWORK LAYER

WebRTC (P2P) NATS WebSocket Signalling Server

(primary, low latency) (fallback, reliable) (for WebRTC setup)

▼

BACKEND (Rust)

1. Document Service (persist Yjs snapshots)

2. Permission Service (OPA checks)

3. Lock Service (packing.lock permission)

4. Audit Service (Loom logging)

5. Chat Service (NATS + translation)

▼

DATABASE (PostgreSQL + NATS)

packing_plans (current state)

collaborative_sessions (active sessions)

collaborative_history (versioned changes)


```
input.role == "QUALITY_INSPECTOR"
input.action in ["packing.read", "packing.qc_notes.add"]
}
```

Viewer: read only

```
allow {
input.role == "VIEWER"
input.action == "packing.read"
}
```

5.4.3.4 Locking Mechanism

Lock Workflow:

Trade Manager clicks "Lock Packing Plan".

Governor checks:

User has packing.lock permission.

Plan is not already locked.

All mandatory fields are filled.

Weight/height validations pass.

If all checks pass, lock is granted.

Lock state is propagated to all users via Yjs.

Users without edit permissions see a read-only view.

Lock can be released (with reason) by the locker or admin.

Lock state is logged in Loom.

Example CONDITIONAL Response (Race Condition):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Packing plan was locked by another user while you were editing. Please refresh to see the latest version.",
    "conditions": [
      {
        "condition_id": "lock_conflict",
        "label": "Refresh view",
        "action_url": "/v1/packing/refresh?plan_id=..."
      }
    ],
    "escalation_hint": "If you believe this is an error, request a human review."
  }
}
```

}

Chat UI Example:

■ ■

■ ■ 10:32 Sara: Good idea, the rear pallets are heavier ■ ■

■ ■ 10:36 Ahmed: Yes, it's attached. Will update the pallet status. ■ ■

5.4.4.1 CRDT-Based Resolution

Deterministic merge — All peers converge to the same state.

No central coordination — Peers can sync directly via WebRTC.

5.4.4.2 Offline Conflict Resolution

■ ■

$\{$

text

□ □


```

-- Collaborative sessions (already defined, extended)
ALTER TABLE collaborative_sessions ADD COLUMN user_role TEXT;
ALTER TABLE collaborative_sessions ADD COLUMN status TEXT DEFAULT 'ACTIVE';
ALTER TABLE collaborative_sessions ADD COLUMN cursor_position JSONB;
ALTER TABLE collaborative_sessions ADD COLUMN last_activity_at TIMESTAMPTZ DEFAULT NOW();

-- Packing plan versions (history)
CREATE TABLE packing_plan_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
  version INTEGER NOT NULL,
  snapshot JSONB NOT NULL,
  change_summary TEXT,
  created_by UUID REFERENCES employees(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  loom_hash TEXT,
  UNIQUE(packing_plan_id, version)
);

-- Change logs (audit trail)
CREATE TABLE collaborative_change_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
  user_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
  user_name TEXT,
  action TEXT NOT NULL,
  field TEXT,
  old_value JSONB,
  new_value JSONB,
  pallet_id UUID REFERENCES pallet_details(id) ON DELETE SET NULL,
  change_timestamp TIMESTAMPTZ DEFAULT NOW(),
  loom_hash TEXT
);

-- Offline change queue (for syncing)
CREATE TABLE offline_change_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,

```

```

packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
change_data JSONB NOT NULL,
queued_at TIMESTAMPTZ DEFAULT NOW(),
synced BOOLEAN DEFAULT false,
synced_at TIMESTAMPTZ
);

-- Chat messages
CREATE TABLE collaborative_chat_messages (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
user_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
user_name TEXT,
message TEXT NOT NULL,
message_translations JSONB, -- { "ar": "...", "de": "..."}
tagged_to_pallet_id UUID REFERENCES pallet_details(id) ON DELETE SET NULL,
mentions TEXT[], -- Employee IDs
sent_at TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT
);

-- Indexes
CREATE INDEX idx_collaborative_sessions_packing_plan ON collaborative_sessions(packing_plan_id);
CREATE INDEX idx_collaborative_sessions_last_activity ON collaborative_sessions(last_activity_at);
CREATE INDEX idx_packing_plan_versions_packing_plan ON packing_plan_versions(packing_plan_id);
CREATE INDEX idx_collaborative_change_logs_packing_plan ON collaborative_change_logs(packing_plan_id);
CREATE INDEX idx_collaborative_change_logs_timestamp ON collaborative_change_logs(change_timestamp DESC);
CREATE INDEX idx_offline_change_queue_user ON offline_change_queue(user_employee_id);
CREATE INDEX idx_offline_change_queue_packing_plan ON offline_change_queue(packing_plan_id);
CREATE INDEX idx_collaborative_chat_messages_packing_plan ON collaborative_chat_messages(packing_plan_id);
CREATE INDEX idx_collaborative_chat_messages_sent_at ON collaborative_chat_messages(sent_at DESC);

```

5.4.11 Implementation Checklist (Part 5.4)

5.4.11.1 Core Infrastructure

Deploy Yjs (CDN or bundled) in frontend

Implement Yjs document provider (WebRTC + NATS fallback)

Implement signalling server (Rust) for WebRTC connection establishment

Implement document persistence (save snapshots to PostgreSQL)

Implement document versioning (snapshots at regular intervals)

5.4.11.2 Permissions & Roles

Implement OPA policies for collaborative editing

Implement role-based UI (hide/show fields based on role)

Implement permission checks on every edit

Implement permission violation handling (Decision Panel)

5.4.11.3 Locking

Implement lock request endpoint (POST /v1/packing/{id}/lock)

Implement lock release endpoint (POST /v1/packing/{id}/unlock)

Implement auto-release after inactivity (cron job)

Implement lock awareness (visible to all users)

Implement race condition handling (CONDITIONAL response)

5.4.11.4 Presence & Awareness

Implement Yjs awareness (cursors, names, colours)

Implement user list (active collaborators)

Implement colour selection (deterministic per user)

Implement heartbeat (keep session alive)

5.4.11.5 Chat

Implement NATS WebSocket chat

Implement message tagging (to pallets)

Implement mentions (@username)

Implement translation (Groq, A1)

Implement chat history storage

5.4.11.6 Offline Support

Implement offline change queue (IndexedDB)

Implement sync on reconnect

Implement conflict resolution (CRDT)

Implement offline status indicator

5.4.11.7 Audit & Versioning

Implement change logging (per edit)

Implement Loom hashing of changes

Implement version history UI

Implement version restore functionality

Implement diff view

5.4.11.8 Testing

Unit tests: CRDT merge logic

Unit tests: OPA permission enforcement

Unit tests: lock logic

Integration tests: multiple users editing simultaneously

Integration tests: offline → reconnect → sync

Integration tests: versioning and restore

End-to-end tests: full collaborative editing workflow

5.4.12 AI Authority Summary for Part 5.4

END OF PART 5.4 — COLLABORATIVE PACKING PLAN EDITING

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.5 — 3D CONTAINER VIEWER & CAPACITY HEATMAP

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.5.0 Purpose & Design Philosophy

The 3D Container Viewer & Capacity Heatmap module transforms the abstract palletisation data into an interactive, visual loading plan that warehouse staff, logistics coordinators, and trade managers can use to validate, optimise, and execute the physical loading of containers, ULDs, wagons, or trailers. The 3D viewer provides an intuitive, spatial understanding of the loading plan, while the capacity heatmap offers data-driven insights into historical loading density to optimise future shipments.

Key Principles:

Interactive visualisation — Users can rotate, zoom, and pan to inspect the loading plan from any angle.

Pallet-level detail — Clicking any pallet reveals SSCC, product, weight, lot number, treatment status, and layer breakdown (for non-uniform stacks).

Transport mode-aware — Renders ocean containers, air ULDs, rail wagons, and truck trailers with accurate dimensions and visual representation.

Capacity heatmap — Opt-in, differentially private overlay showing historical loading density for the same commodity and route.

Export capabilities — Export as STL (3D model), PNG (screenshot), or PDF (report).

Collaborative integration — The 3D viewer updates in real-time as collaborators edit the packing plan.

Non-marketplace — Never suggests alternative counterparties or service providers; purely visual and analytical.

AI Integration in Part 5.5:

5.5.1 Technology Stack

5.5.2 Core 3D Viewer Features

5.5.2.1 Interaction Controls

5.5.2.2 View Modes

UI Example:

text



■ VIEW CONTROLS ■



■ [3D ●] [Top ■] [Side ■] [Front ■] [Section ■] ■



■ [Auto-Rotate] [Reset View] [Full Screen] [Export] ■



5.5.2.3 Pallet Interaction

Pallet Detail Panel:

text



■ PALLET DETAILS — OR-005 ■



■ ■ SSCC: 106141411234567890 ■ ■

■ ■ Product: Fresh Oranges ■ ■

■ ■ HS Code: 0805.10 ■ ■

■ ■ Lot: OR-2026-0412 ■ ■

■ ■ Net Weight: 1,110 kg ■ ■

■ ■ Gross Weight: 1,165 kg ■ ■

■ ■ Cartons: 111 (11×10 + 1×1) ■ ■

■ ■ Status: ■ Planned ■ ■

■ ■ Cold Treatment: ■ Completed (CT-20260415-001) ■ ■

■ ■ Position: (1,200 mm, 2,400 mm, 0 mm) ■ ■

■ ■ Layer Breakdown: ■ ■

■ ■ 11 layers × 10 cartons (Standard) ■ ■

■ ■ 1 layer × 1 carton (Centered) ■ ■



■ ■ [Move] [Edit] [Remove] [View in 2D] ■ ■



rust

// Pseudo-code: Capacity heatmap calculation with differential privacy

fn calculate_heatmap_density(

route: &Route,

commodity_type: &str,

container_type: &ContainerType,

privacy_epsilon: f64

) -> HeatmapData {

// 1. Query Trade Memory Layer (anonymised)

let memory_events = query_trade_memory(route, commodity_type, container_type);

// 2. Calculate density per position

let mut densities = vec![];

for event in memory_events {

for position in event.pallet_positions {

densities.push(position.density);

}

}

// 3. Group by position (x, y coordinates)

let mut grid = HashMap::new();

for density in densities {

let key = format!("{}", density.x, density.y);

grid.entry(key).or_insert(vec![]).push(density.value);

}

// 4. Calculate average per position

let mut heatmap = vec![];

for (key, values) in grid {

let avg = values.iter().sum::<f64>() / values.len() as f64;

let (x, y) = key.split(',').collect::<Vec<&str>>();

heatmap.push(HeatmapCell {

x: x.parse().unwrap(),

y: y.parse().unwrap(),

density: avg,

});

}

// 5. Apply differential privacy ($\epsilon=0.1$)





5.5.5 Export Capabilities

Export UI:

text



■ EXPORT — 3D Container Viewer ■

■ ■

■ Select Export Format: ■

■ [STL ●] [PNG ■] [PDF ■] [JSON ■] ■

■ ■

■ Export Options: ■

■ ■ Include pallet labels ■

■ ■ Include dimensions ■

■ ■ Include heatmap overlay ■

■ ■ Include layer breakdown ■

■ ■

■ Resolution: [High ●] [Medium ■] [Low ■] ■

■ ■

■ [Export] [Cancel] ■



5.5.6 Integration with Collaborative Editing

The 3D viewer updates in real-time as collaborators edit the packing plan.

Real-time Update Flow:

text

1. User A moves pallet P3 to new position.
2. Yjs document updates.
3. Change is propagated to all peers via WebRTC/NATS.
4. All peers receive update (<200ms).
5. 3D viewer animates pallet P3 to new position.
6. Pallet detail panel updates (if open).

5.5.7 Performance Optimisation

Performance Targets:

5.5.8 Validation Gates (Part 5.5)

5.5.9 Database Schema Additions (Part 5.5)

sql

-- 3D viewer layouts (cached for performance)

```
CREATE TABLE three_d_layouts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,  
  layout_data JSONB NOT NULL, -- Three.js scene data  
  version INTEGER NOT NULL,  
  generated_at TIMESTAMPTZ DEFAULT NOW(),  
  generated_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
  UNIQUE(packing_plan_id, version)  
);
```

-- Capacity heatmap data (anonymised, differential privacy)

```
CREATE TABLE capacity_heatmap_data (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  route_id TEXT NOT NULL,  
  commodity_type TEXT NOT NULL,  
  container_type TEXT NOT NULL,  
  grid_data JSONB NOT NULL, -- [{x, y, density, sample_count}]  
  confidence DECIMAL(5,2),  
  sample_size INTEGER,  
  privacy_epsilon DECIMAL(5,2) DEFAULT 0.1,  
  measured_at DATE NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Heatmap opt-out registry

```
CREATE TABLE heatmap_opt_out (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  opted_out_at TIMESTAMPTZ DEFAULT NOW(),  
  opted_in_at TIMESTAMPTZ,  
  consent_version TEXT DEFAULT 'v1.0',  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(tenant_gtid)
```

```

);
-- Export logs
CREATE TABLE heatmap_export_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
packing_plan_id UUID NOT NULL REFERENCES packing_plans(id) ON DELETE CASCADE,
exported_by UUID REFERENCES employees(id) ON DELETE SET NULL,
format TEXT NOT NULL, -- 'STL', 'PNG', 'PDF', 'JSON'
file_url TEXT,
exported_at TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT
);
-- Indexes
CREATE INDEX idx_three_d_layouts_packing_plan ON three_d_layouts(packing_plan_id);
CREATE INDEX idx_capacity_heatmap_data_route ON capacity_heatmap_data(route_id,
commodity_type);
CREATE INDEX idx_capacity_heatmap_data_measured ON capacity_heatmap_data(measured_at
DESC);
CREATE INDEX idx_heatmap_opt_out_tenant ON heatmap_opt_out(tenant_gtid);
CREATE INDEX idx_heatmap_export_logs_packing_plan ON heatmap_export_logs(packing_plan_id);

```

5.5.10 Implementation Checklist (Part 5.5)

5.5.10.1 Core 3D Viewer

- Set up Three.js scene (renderer, camera, lights)
- Implement OrbitControls (rotate, zoom, pan)
- Implement container/ULD/wagon/trailer rendering (transport mode-aware)
- Implement pallet rendering (with colour coding)
- Implement pallet interaction (click, hover, double-click)
- Implement pallet detail panel
- Implement view modes (3D, Top, Side, Front, Section)
- Implement auto-rotate toggle

5.5.10.2 Capacity Heatmap

- Implement heatmap data aggregation (anonymised)
- Implement differential privacy (OpenDP, $\epsilon=0.1$)
- Implement heatmap overlay rendering (Canvas overlay on 3D view)
- Implement colour coding (Green/Yellow/Red)
- Implement heatmap insights (A1, Groq)
- Implement opt-out mechanism

Implement heatmap export

5.5.10.3 Export

Implement STL export (Three.js STL exporter)

Implement PNG export (canvas screenshot)

Implement PDF export (report generation)

Implement JSON export (loading plan data)

5.5.10.4 Performance

Implement Level of Detail (LOD)

Implement InstancedMesh for pallets

Implement Frustum Culling

Implement lazy loading

Optimise WebGL draw calls

5.5.10.5 Integration

Integrate with collaborative editing (real-time updates via NATS)

Integrate with packing plan data

Integrate with Trade Memory Layer (heatmap data)

Integrate with permission system (view-only if locked)

5.5.10.6 Testing

Unit tests: heatmap calculation

Unit tests: differential privacy

Unit tests: export formats

Integration tests: 3D viewer ↔ packing plan data

Integration tests: heatmap ↔ Trade Memory Layer

Performance tests: 3D viewer with 50 pallets

End-to-end tests: full 3D viewer workflow

5.5.11 AI Authority Summary for Part 5.5

END OF PART 5.5 — 3D CONTAINER VIEWER & CAPACITY HEATMAP

PART 5.6 — AUTOMATED INVOICE GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.6.0 Purpose & Design Philosophy

The Automated Invoice Generation module transforms the trade data, packing plan, weight calculations, and commercial settlement details into a legally compliant, ETA-compliant invoice that serves as the financial foundation of the trade. The invoice is the single source of truth for all commercial terms — goods, quantities, prices, logistics costs, fees, payment terms, and settlement instructions.

Key Principles:

Zero re-entry — All invoice data is sourced directly from the trade request, packing plan, and weight calculations. No manual data entry.

ETA-compliant — Generates UBL 2.1 XML invoices that conform to Egyptian Tax Authority requirements.

Commercial settlement integration — Includes payment structure, timing, credit period, currency, and financing interest from Part 4.9.

Insurance integration — References insurance requirements and certificate details from Part 4.8.

Multi-line item — Separate lines for goods, logistics costs, SGTX fee, optional services, and financing fees.

Digital signing — Signed with the seller's Ed25519 certificate (or QES for high-value trades).

Automatic submission — Submitted to ETA via API; UUID and QR code retrieved and stored.

PDF/A-3 archival — Archival format for long-term document preservation.

Multi-currency support — Settlement currency separate from pricing currency.

Trade criticality flag — Priority flag for expedited processing.

AI Integration in Part 5.6:

5.6.1 Invoice Structure — Complete Specification

5.6.1.1 UBL 2.1 XML Structure (ETA-Compliant)

xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Invoice xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2"
```

```
xmlns:cac="urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2"
```

```
xmlns:cbc="urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2">
```

```
<!-- 1. INVOICE IDENTIFIERS -->
```

```
<cbc:ID>INV-STR-2026-001</cbc:ID>
```

```
<cbc:UUID>f47ac10b-58cc-4372-a567-0e02b2c3d479</cbc:UUID>
```

```
<cbc:IssueDate>2026-06-20</cbc:IssueDate>
```

```
<cbc:IssueTime>10:30:00</cbc:IssueTime>
```

```
<cbc:InvoiceTypeCode>388</cbc:InvoiceTypeCode>
```

```
<cbc:DocumentCurrencyCode>USD</cbc:DocumentCurrencyCode>
```

```
<cbc:TaxCurrencyCode>USD</cbc:TaxCurrencyCode>
```

```
<!-- 2. REFERENCES -->
```

```
<cbc:OrderReference>
```

```
<cbc:ID>MC-20260415-001</cbc:ID>
```

```
</cbc:OrderReference>
```

```
<cac:AdditionalDocumentReference>
```

```
<cbc:ID>USTN</cbc:ID>
```

```
<cbc:DocumentType>USTN</cbc:DocumentType>
```

```
<cbc:DocumentDescription>SGTX-EG-26-F3A-1</cbc:DocumentDescription>
```

```
</cac:AdditionalDocumentReference>
```

```
<cac:AdditionalDocumentReference>
<cbc:ID>PACKING_LIST</cbc:ID>
<cbc:DocumentType>PackingList</cbc:DocumentType>
<cbc:DocumentDescription>Packing list hash: sha256:abc123...</cbc:DocumentDescription>
</cac:AdditionalDocumentReference>
<cac:AdditionalDocumentReference>
<cbc:ID>INSURANCE</cbc:ID>
<cbc:DocumentType>InsuranceCertificate</cbc:DocumentType>
<cbc:DocumentDescription>INS-2026-001</cbc:DocumentDescription>
</cac:AdditionalDocumentReference>
<!-- 3. SETTLEMENT DETAILS (from Part 4.9) -->
<cac:PaymentMeans>
<cbc:PaymentMeansCode>1</cbc:PaymentMeansCode> <!-- 1 = Documentary Collection -->
<cbc:PaymentID>Settlement: Documentary Collection</cbc:PaymentID>
<cbc:InstructionNote>Payment timing: Against Documents</cbc:InstructionNote>
<cbc:InstructionNote>Credit Period: 30 Days</cbc:InstructionNote>
<cbc:InstructionNote>Currency: USD</cbc:InstructionNote>
<cbc:InstructionNote>Financing Interest: None</cbc:InstructionNote>
<cbc:InstructionNote>Commercial Priority: Balanced</cbc:InstructionNote>
<cbc:InstructionNote>Settlement Flexibility: Flexible</cbc:InstructionNote>
</cac:PaymentMeans>
<!-- 4. PARTIES -->
<cac:AccountingSupplierParty>
<cac:Party>
<cac:PartyIdentification>
<cbc:ID schemeID="GTID">SGTX-EG-TRD-002139-7F3A</cbc:ID>
</cac:PartyIdentification>
<cac:PartyName>
<cbc:Name>Strawberry Export Co.</cbc:Name>
</cac:PartyName>
<cac:PostalAddress>
<cbc:StreetName>123 Export Road</cbc:StreetName>
<cbc:CityName>Alexandria</cbc:CityName>
<cbc:CountrySubentity>Alexandria Governorate</cbc:CountrySubentity>
<cac:Country>
```


<cbc:IdentificationCode>EG</cbc:IdentificationCode>
</cac:Country>
</cac:PostalAddress>
<cac:PartyTaxScheme>
<cbc:CompanyID>123-456-789</cbc:CompanyID>
<cac:TaxScheme>
<cbc:Name>VAT</cbc:Name>
</cac:TaxScheme>
</cac:PartyTaxScheme>
<cac:PartyLegalEntity>
<cbc:RegistrationName>Strawberry Export Co.</cbc:RegistrationName>
<cbc:CompanyID>123-456-789</cbc:CompanyID>
</cac:PartyLegalEntity>
</cac:Party>
</cac:AccountingSupplierParty>
<cac:AccountingCustomerParty>
<cac:Party>
<cac:PartyIdentification>
<cbc:ID schemeID="GTID">SGTX-DE-TRD-001234-5B6C</cbc:ID>
</cac:PartyIdentification>
<cac:PartyName>
<cbc:Name>European Importer GmbH</cbc:Name>
</cac:PartyName>
<cac:PostalAddress>
<cbc:StreetName>456 Import Street</cbc:StreetName>
<cbc:CityName>Hamburg</cbc:CityName>
<cbc:CountrySubentity>Hamburg</cbc:CountrySubentity>
<cac:Country>
<cbc:IdentificationCode>DE</cbc:IdentificationCode>
</cac:Country>
</cac:PostalAddress>
<cac:PartyTaxScheme>
<cbc:CompanyID>DE123456789</cbc:CompanyID>
<cac:TaxScheme>
<cbc:Name>VAT</cbc:Name>

```
</cac:TaxScheme>
</cac:PartyTaxScheme>
</cac:Party>
</cac:AccountingCustomerParty>
<!-- 5. PAYMENT TERMS -->
<cac:PaymentTerms>
<cbc:PaymentMeansID>Documentary Collection</cbc:PaymentMeansID>
<cbc:PaymentDueDate>2026-07-20</cbc:PaymentDueDate> <!-- 30 days after invoice -->
<cbc:Note>Payment against documents. 30 days credit period.</cbc:Note>
</cac:PaymentTerms>
<!-- 6. DELIVERY DETAILS -->
<cac:Delivery>
<cac:DeliveryLocation>
<cbc:ID>DEHAM</cbc:ID>
<cbc:Name>Hamburg Port</cbc:Name>
</cac:DeliveryLocation>
<cac:DeliveryTerms>
<cbc:DeliveryTerms>CFR</cbc:DeliveryTerms>
<cbc:Note>Cost and Freight</cbc:Note>
</cac:DeliveryTerms>
<cac:Shipment>
<cbc:ID>SGTX-EG-26-F3A-1</cbc:ID>
<cbc:HandlingCode>OCEAN</cbc:HandlingCode>
<cac:TransportHandlingUnit>
<cbc:Quantity unitCode="TNE">15.7848</cbc:Quantity> <!-- Total gross weight in tonnes -->
</cac:TransportHandlingUnit>
</cac:Shipment>
</cac:Delivery>
<!-- 7. INVOICE LINES -->
<!-- 7.1 GOODS LINE — Oranges -->
<cac:InvoiceLine>
<cbc:ID>1</cbc:ID>
<cbc:InvoicedQuantity unitCode="KGM">8800</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">44000.00</cbc:LineExtensionAmount>
<cac:Item>
```

```

<cbc:Name>Valencia Oranges, Grade A</cbc:Name>
<cbc:Description>Fresh Valencia Oranges, Grade A, 10 kg cartons, 22 pallets</cbc:Description>
<cac:CommodityClassification>
<cbc:ItemClassificationCode listID="HS">0805.10</cbc:ItemClassificationCode>
</cac:CommodityClassification>
<cac:AdditionalItemProperty>
<cbc:Name>Variety</cbc:Name>
<cbc:Value>Valencia</cbc:Value>
</cac:AdditionalItemProperty>
<cac:AdditionalItemProperty>
<cbc:Name>Grade</cbc:Name>
<cbc:Value>Grade A</cbc:Value>
</cac:AdditionalItemProperty>
<cac:AdditionalItemProperty>
<cbc:Name>Brix</cbc:Name>
<cbc:Value>12.5 °Bx</cbc:Value>
</cac:AdditionalItemProperty>
</cac:Item>
<cac:Price>
<cbc:PriceAmount currencyID="USD">5.00</cbc:PriceAmount>
<cbc:BaseQuantity unitCode="KGM">1</cbc:BaseQuantity>
</cac:Price>
</cac:InvoiceLine>
<!-- 7.2 GOODS LINE — Lemons -->
<cac:InvoiceLine>
<cbc:ID>2</cbc:ID>
<cbc:InvoicedQuantity unitCode="KGM">5376</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">26880.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>Eureka Lemons, Grade A</cbc:Name>
<cbc:Description>Fresh Eureka Lemons, Grade A, 8 kg cartons, 14 pallets</cbc:Description>
<cac:CommodityClassification>
<cbc:ItemClassificationCode listID="HS">0805.50</cbc:ItemClassificationCode>
</cac:CommodityClassification>
</cac:Item>

```

```
<cac:Price>
<cbc:PriceAmount currencyID="USD">5.00</cbc:PriceAmount>
<cbc:BaseQuantity unitCode="KGM">1</cbc:BaseQuantity>
</cac:Price>
</cac:InvoiceLine>
<!-- 7.3 LOGISTICS — Trucking -->
<cac:InvoiceLine>
<cbc:ID>3</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">900.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>Trucking (farm to port)</cbc:Name>
<cbc:Description>Beheira → Damietta, 22 pallets oranges, 14 pallets lemons</cbc:Description>
</cac:Item>
</cac:InvoiceLine>
<!-- 7.4 LOGISTICS — Ocean Freight -->
<cac:InvoiceLine>
<cbc:ID>4</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">4200.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>Ocean Freight</cbc:Name>
<cbc:Description>Damietta → Hamburg, 40ft Reefer, Vessel: MSC FIONA</cbc:Description>
<cac:AdditionalItemProperty>
<cbc:Name>Vessel</cbc:Name>
<cbc:Value>MSC FIONA</cbc:Value>
</cac:AdditionalItemProperty>
<cac:AdditionalItemProperty>
<cbc:Name>Voyage</cbc:Name>
<cbc:Value>MSF123E</cbc:Value>
</cac:AdditionalItemProperty>
</cac:Item>
</cac:InvoiceLine>
<!-- 7.5 LOGISTICS — Insurance -->
<cac:InvoiceLine>
```

```

<cbc:ID>5</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">450.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>Cargo Insurance</cbc:Name>
<cbc:Description>All Risks, 110% of invoice value, Policy: INS-2026-001</cbc:Description>
</cac:Item>
</cac:InvoiceLine>
<!-- 7.6 SGTX PLATFORM FEE -->
<cac:InvoiceLine>
<cbc:ID>6</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">1500.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>SGTX Platform Fee (1.5%)</cbc:Name>
<cbc:Description>Calculated on total trade value (EXW + logistics)</cbc:Description>
<cac:AdditionalItemProperty>
<cbc:Name>Rate</cbc:Name>
<cbc:Value>1.5%</cbc:Value>
</cac:AdditionalItemProperty>
<cac:AdditionalItemProperty>
<cbc:Name>Base Amount</cbc:Name>
<cbc:Value>100,000.00 USD</cbc:Value>
</cac:AdditionalItemProperty>
</cac:Item>
</cac:InvoiceLine>
<!-- 7.7 OPTIONAL SERVICE — Laboratory Testing -->
<cac:InvoiceLine>
<cbc:ID>7</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">200.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>Laboratory Testing</cbc:Name>
<cbc:Description>Pesticide panel, Cypermethrin, Chlorpyrifos, Thiabendazole</cbc:Description>
</cac:Item>

```

```

</cac:InvoiceLine>
<!-- 7.8 OPTIONAL SERVICE — Customs Broker -->
<cac:InvoiceLine>
<cbc:ID>8</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">150.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>Customs Broker Certification</cbc:Name>
<cbc:Description>Nafeza declaration certification, Broker: Nile Customs Services</cbc:Description>
</cac:Item>
</cac:InvoiceLine>
<!-- 7.9 FINANCING FEE (if applicable) -->
<!-- <cac:InvoiceLine>
<cbc:ID>9</cbc:ID>
<cbc:InvoicedQuantity>1</cbc:InvoicedQuantity>
<cbc:LineExtensionAmount currencyID="USD">250.00</cbc:LineExtensionAmount>
<cac:Item>
<cbc:Name>SGTX Financing Fee (0.25%)</cbc:Name>
<cbc:Description>Calculated on financed amount: $100,000</cbc:Description>
</cac:Item>
</cac:InvoiceLine> -->
<!-- 8. TOTALS -->
<cac:LegalMonetaryTotal>
<cbc:LineExtensionAmount currencyID="USD">78730.00</cbc:LineExtensionAmount> <!-- Goods +
Logistics -->
<cbc:TaxExclusiveAmount currencyID="USD">78730.00</cbc:TaxExclusiveAmount>
<cbc:TaxInclusiveAmount currencyID="USD">78730.00</cbc:TaxInclusiveAmount>
<cbc:PayableAmount currencyID="USD">78730.00</cbc:PayableAmount>
<cbc:PrepaidAmount currencyID="USD">0.00</cbc:PrepaidAmount>
<cbc:AllowanceTotalAmount currencyID="USD">0.00</cbc:AllowanceTotalAmount>
<cbc:ChargeTotalAmount currencyID="USD">0.00</cbc:ChargeTotalAmount>
</cac:LegalMonetaryTotal>
<!-- 9. ADDITIONAL NOTES -->
<cbc:Note>Trade Criticality: PRIORITY</cbc:Note>
<cbc:Note>Incoterm: CFR (Cost and Freight)</cbc:Note>
<cbc:Note>Partial Shipment: NO</cbc:Note>

```

```

<cbc:Note>Transshipment: NO</cbc:Note>
<cbc:Note>Special Instructions: Arabic labels mandatory. Halal certification required.</cbc:Note>
<cbc:Note>Insurance: Required, All Risks, 110%, Policy: INS-2026-001</cbc:Note>
<cbc:Note>Settlement Flexibility: Flexible</cbc:Note>
<!-- 10. DIGITAL SIGNATURE -->
<cac:Signature>
<cbc:ID>Sig-001</cbc:ID>
<cac:SignatoryParty>
<cac:PartyIdentification>
<cbc:ID>SGTX-EG-TRD-002139-7F3A</cbc:ID>
</cac:PartyIdentification>
<cac:PartyName>
<cbc:Name>Strawberry Export Co.</cbc:Name>
</cac:PartyName>
</cac:SignatoryParty>
<cac:Signature>
<cbc:ID>Ed25519</cbc:ID>
<cbc:Note>Digital Signature: MIAGCSqGSIb3DQEHAqCAMIACAQExDzANBgIghkgBZQMEAgEFADCAB
gkqhkiG9w0BBwEAAKCAMIIB...</cbc:Note>
</cac:Signature>
</cac:Signature>
<!-- 11. QR CODE (added after ETA submission) -->
<cac:AdditionalDocumentReference>
<cbc:ID>QR_CODE</cbc:ID>
<cbc:DocumentType>QRCode</cbc:DocumentType>
<cbc:DocumentDescription>iVBORw0KGgoAAAANSUHEUgAA...</cbc:DocumentDescription>
</cac:AdditionalDocumentReference>
</Invoice>

```

5.6.1.2 Invoice Line Item Categories

5.6.2 Commercial Settlement Integration

The invoice includes all commercial settlement details from Part 4.9:

Settlement Structure Mapping:

5.6.3 Insurance Integration

The invoice references insurance requirements from Part 4.8:

5.6.4 Invoice Generation & Signing

5.6.4.1 Generation Process

text

1. Trigger: Seller accepts buyer's quote → Contract lock

↓

2. System collects invoice data:

- Seller EXW price (from EXW lock)
- Logistics costs (from Logistics Builder)
- SGTX fee (1.5% calculation)
- Optional services (from service quotations)
- Commercial settlement (from Part 4.9)
- Insurance details (from Part 4.8)
- Weight calculations (from Part 5.1)
- Packing plan data (from Part 5.2)
- Special instructions (from Part 4.6)

↓

3. System generates UBL 2.1 XML

↓

4. System canonicalises XML (C14N)

↓

5. System signs with seller's Ed25519 certificate

↓

6. System submits to ETA API

↓

7. System receives UUID and QR code from ETA

↓

8. System stores invoice in `documents` table

↓

9. System generates PDF/A-3 version

↓

10. System notifies buyer and seller via Smart Inbox

5.6.4.2 Digital Signing (Ed25519 / QES)

Signing Process:

rust

// Pseudo-code: Invoice signing

async fn sign_invoice(
invoice_xml: &str,


```

seller_gtid: &str,
signature_type: SignatureType,
) -> Result<SignedInvoice> {
// 1. Canonicalise XML (C14N)
let canonical_xml = canonicalize_xml(invoice_xml)?;
// 2. Calculate hash
let hash = sha256(canonical_xml.as_bytes());
// 3. Sign based on signature type
let signature = match signature_type {
SignatureType::Standard => sign_ed25519(hash, seller_gtid)?,
SignatureType::Advanced => sign_ed25519_with_cert(hash, seller_gtid)?,
SignatureType::QES => sign_qes(hash, seller_gtid)?, // Egypt Trust HSM
};
// 4. Add signature to XML
let signed_xml = add_signature_to_xml(invoice_xml, &signature)?;
// 5. Verify signature
verify_signature(&signed_xml)?;
Ok(SignedInvoice {
xml: signed_xml,
signature: signature,
signature_type: signature_type,
hash: hash,
})
}

```

5.6.5 ETA Submission

5.6.5.1 API Endpoint

5.6.5.2 ETA Response

```

json
{
  "uuid": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "qrCode": "iVBORw0KGgoAAAANSUhEUgAA...",
  "status": "VALID",
  "validationErrors": []
}

```

5.6.5.3 Error Handling

5.6.6 PDF/A-3 Archival Generation

5.6.6.1 Purpose

Comply with Egyptian National E-Archiving standards (ISO 19005-3) for long-term document preservation.

5.6.6.2 Technical Requirements

5.6.6.3 Generation Process

text

1. Generate HTML from invoice data (Tera template)
↓
2. Convert to PDF using headless Chrome
↓
3. Apply PDF/A-3 conformance (pdf-writer or printpdf)
↓
4. Add digital signature (visible signature field)
↓
5. Add metadata (USTN, invoice number, timestamp, hash)
↓
6. Validate PDF/A-3 conformance
↓
7. Store in `documents` table

5.6.7 Validation Gates (Part 5.6)

5.6.8 Database Schema Additions (Part 5.6)

sql

-- Invoices (core)

```
CREATE TABLE invoices (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,  
  invoice_number TEXT NOT NULL UNIQUE,  
  invoice_uuid TEXT,  
  invoice_type TEXT DEFAULT 'COMMERCIAL',  
  issue_date DATE NOT NULL,  
  document_currency TEXT NOT NULL DEFAULT 'USD',  
  tax_currency TEXT NOT NULL DEFAULT 'USD',  
  total_excluding_tax DECIMAL(20,4) NOT NULL,  
  total_including_tax DECIMAL(20,4) NOT NULL,
```

```
total_payable DECIMAL(20,4) NOT NULL,
-- Settlement details (from Part 4.9)
settlement_structure TEXT,
payment_timing TEXT,
credit_period INTEGER,
financing_interest TEXT,
commercial_priority TEXT,
settlement_flexibility TEXT,
-- Insurance details (from Part 4.8)
insurance_required BOOLEAN,
insurance_type TEXT,
insurance_coverage_percent INTEGER,
insurance_policy_number TEXT,
-- Digital signature
digital_signature TEXT,
signature_type TEXT DEFAULT 'ED25519',
xml_hash TEXT,
loom_hash TEXT,
-- ETA submission
eta_submitted BOOLEAN DEFAULT false,
eta_uuid TEXT,
eta_qr_code TEXT,
eta_submitted_at TIMESTAMPTZ,
eta_validation_errors JSONB,
-- Archival
pdf_url TEXT,
pdfa3_compliant BOOLEAN DEFAULT false,
pdfa3_validated_at TIMESTAMPTZ,
-- Versioning
version INTEGER DEFAULT 1,
previous_version_id UUID REFERENCES invoices(id) ON DELETE SET NULL,
-- Metadata
generated_by UUID REFERENCES employees(id) ON DELETE SET NULL,
generated_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
```

```

);
-- Invoice lines
CREATE TABLE invoice_lines (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id UUID NOT NULL REFERENCES invoices(id) ON DELETE CASCADE,
  line_number INTEGER NOT NULL,
  line_type TEXT NOT NULL, -- 'GOODS', 'LOGISTICS', 'FEE', 'OPTIONAL_SERVICE', 'FINANCING_FEE'
  description TEXT NOT NULL,
  quantity DECIMAL(20,4) NOT NULL,
  quantity_unit TEXT NOT NULL,
  unit_price DECIMAL(20,4) NOT NULL,
  total_amount DECIMAL(20,4) NOT NULL,
  currency TEXT NOT NULL DEFAULT 'USD',
  line_data JSONB, -- Additional line-specific data
  created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Invoice versions (history)
CREATE TABLE invoice_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id UUID NOT NULL REFERENCES invoices(id) ON DELETE CASCADE,
  version INTEGER NOT NULL,
  xml_content TEXT,
  pdf_url TEXT,
  digital_signature TEXT,
  loom_hash TEXT,
  created_by UUID REFERENCES employees(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(invoice_id, version)
);
-- ETA submission logs
CREATE TABLE eta_submission_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id UUID NOT NULL REFERENCES invoices(id) ON DELETE CASCADE,
  endpoint TEXT NOT NULL,
  request_xml TEXT,

```

```

response_json JSONB,
status_code INTEGER,
success BOOLEAN,
error_message TEXT,
idempotency_key TEXT,
submitted_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes

CREATE INDEX idx_invoices_trade_request ON invoices(trade_request_id);
CREATE INDEX idx_invoices_ustn ON invoices(ustn);
CREATE INDEX idx_invoices_invoice_number ON invoices(invoice_number);
CREATE INDEX idx_invoices_eta_uuid ON invoices(eta_uuid);
CREATE INDEX idx_invoices_generated_at ON invoices(generated_at DESC);
CREATE INDEX idx_invoice_lines_invoice ON invoice_lines(invoice_id);
CREATE INDEX idx_invoice_lines_type ON invoice_lines(line_type);
CREATE INDEX idx_invoice_versions_invoice ON invoice_versions(invoice_id);
CREATE INDEX idx_eta_submission_logs_invoice ON eta_submission_logs(invoice_id);
CREATE INDEX idx_eta_submission_logs_submitted_at ON eta_submission_logs(submitted_at DESC);

```

5.6.9 Implementation Checklist (Part 5.6)

5.6.9.1 Invoice Generation

- Implement UBL 2.1 XML generator (Rust, `serde_xml`)
- Implement invoice data aggregation (trade request, packing, weight, settlement)
- Implement invoice line item generation (goods, logistics, fees, services)
- Implement commercial settlement integration (Part 4.9)
- Implement insurance integration (Part 4.8)
- Implement invoice numbering (unique per seller)

5.6.9.2 Digital Signing

- Implement Ed25519 signing (SoftHSM)
- Implement certificate management (seller's e-Seal)
- Implement QES integration (Egypt Trust HSM)
- Implement signature verification
- Implement signature addition to XML

5.6.9.3 ETA Submission

- Implement ETA API client (mTLS)
- Implement idempotency key generation

Implement retry logic (exponential backoff)

Implement UUID and QR code storage

Implement validation error handling

5.6.9.4 PDF Generation

Implement HTML template (Tera)

Implement PDF generation (headless Chrome)

Implement PDF/A-3 conformance

Implement digital signature in PDF

Implement metadata attachment

5.6.9.5 Testing

Unit tests: UBL 2.1 XML generation

Unit tests: digital signing

Unit tests: ETA submission

Integration tests: invoice generation → signing → ETA submission

Integration tests: PDF generation → PDF/A-3 validation

End-to-end tests: full invoice workflow

5.6.10 AI Authority Summary for Part 5.6

END OF PART 5.6 — AUTOMATED INVOICE GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.7 — CUSTOMS DECLARATION AUTO-GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.7.0 Purpose & Design Philosophy

The Customs Declaration Auto-Generation module transforms the trade data, packing plan, weight calculations, and invoice details into a legally compliant customs declaration (Single Administrative Document — SAD) that is submitted to Nafeza, Egypt's National Single Window for trade. This module automates the most complex and error-prone aspect of international trade — customs clearance — ensuring accuracy, compliance, and speed.

Key Principles:

Zero re-entry — All declaration data is sourced directly from the trade request, packing plan, weight calculations, and invoice. No manual data entry.

Nafeza-compliant — Generates JSON declarations that conform to Nafeza API requirements.

Broker certification — Supports optional broker certification where the broker reviews, certifies, and assumes liability for the declaration.

Certificate auto-request — Automatically requests phytosanitary, health, and certificate of origin after compliant lab results.

Multi-document linkage — Links to USTN, invoice, packing list, and all certificates.

Digital signing — Signed with the filer's Egypt Trust e-Seal certificate (seller or broker).

Automatic submission — Submitted to Nafeza via API; declaration ID and status retrieved and stored.

Retry & resilience — Robust retry logic with fallback to manual PDF submission.

Audit trail — Full Loom logging for all declaration events.

AI Integration in Part 5.7:

5.7.1 Nafeza SAD Structure — Complete Specification

5.7.1.1 SAD JSON Structure

json

```
{
  "declaration_type": "EXPORT",
  "trader_gtid": "SGTX-EG-TRD-002139-7F3A",
  "broker_gtid": "SGTX-EG-CBR-001",
  "ustn": "SGTX-EG-26-F3A-1",
  "acid": "ACI20260415-0001",
  "contract_id": "MC-20260415-001",
  "invoice": {
    "number": "INV-STR-2026-001",
    "value": 78730.00,
    "currency": "USD",
    "eta_uuid": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
  },
  "goods": [
    {
      "hs_code": "0805.10",
      "description": "Valencia Oranges, Grade A",
      "net_weight_kg": 8800,
      "gross_weight_kg": 9790,
      "container_number": "MEDU1234567",
      "packages": 880,
      "package_type": "CARTON",
      "pallet_count": 22,
      "invoice_value": 44000.00,
      "quantity_tolerance": 5,
      "acceptance_criteria": {
        "moisture": "Max 14%",
        "protein": "Min 12%",
        "purity": "Min 99%"
      }
    }
  ]
}
```

```
}
},
{
  "hs_code": "0805.50",
  "description": "Eureka Lemons, Grade A",
  "net_weight_kg": 5376,
  "gross_weight_kg": 5990.8,
  "container_number": "MEDU1234567",
  "packages": 672,
  "package_type": "CARTON",
  "pallet_count": 14,
  "invoice_value": 26880.00,
  "quantity_tolerance": 5,
  "acceptance_criteria": {
    "moisture": "Max 15%",
    "acidity": "Min 5%",
    "purity": "Min 98%"
  }
}
],
"certificate_requests": [
  {
    "type": "PHYTOSANITARY",
    "mandatory": true,
    "ref_lab_report": "USTN:SGTX-EG-26-F3A-1:LAB-001",
    "status": "PENDING"
  },
  {
    "type": "HEALTH",
    "mandatory": true,
    "ref_lab_report": "USTN:SGTX-EG-26-F3A-1:LAB-001",
    "status": "PENDING"
  },
  {
    "type": "CERTIFICATE_OF_ORIGIN",
```



```
"mandatory": true,
"ref_lab_report": null,
"status": "PENDING"
},
{
"type": "COLD_TREATMENT",
"mandatory": true,
"ref_lab_report": null,
"status": "COMPLETED",
"certificate_ref": "CT-20260415-001"
}
],
"transport": {
"incoterm": "CFR",
"port_of_loading": "EGDAM",
"port_of_discharge": "DEHAM",
"vessel_name": "MSC FIONA",
"voyage": "MSF123E",
"container_count": 1,
"container_numbers": ["MEDU1234567"],
"partial_shipment": false,
"transshipment": false
},
"special_instructions": [
"Arabic labels mandatory on all cartons.",
"No wooden pallets (ISPM 15 compliant only).",
"Halal certification required.",
"Temperature logger required in each container.",
"Original Bill of Lading to be sent by DHL."
],
"trade_criticality": "PRIORITY",
"insurance": {
"required": true,
"type": "ALL_RISKS",
"coverage_percent": 110,
```

```

"policy_number": "INS-2026-001",
"certificate_attached": true
},
"commercial_settlement": {
"structure": "DOCUMENTARY_COLLECTION",
"timing": "AGAINST_DOCUMENTS",
"credit_period": 30,
"currency": "USD",
"financing_interest": "NONE",
"flexibility": "FLEXIBLE"
},
"document_hashes": {
"commercial_invoice": "sha256:abc123...",
"packing_list": "sha256:def456...",
"bill_of_lading": null,
"phytosanitary_certificate": null,
"health_certificate": null,
"certificate_of_origin": null,
"insurance_certificate": "sha256:ghi789..."
},
"submission": {
"timestamp": "2026-06-20T10:30:00Z",
"submitted_by_gtid": "SGTX-EG-TRD-002139-7F3A",
"submitted_by_employee_id": "emp-001",
"signature": "ed25519:...",
"loom_hash": "sha256:jkl012..."
}
}

```

5.7.1.2 SAD Field Mapping

5.7.2 Broker Certification Workflow

5.7.2.1 Overview

If the seller selects a customs broker for certification, the declaration must be reviewed, certified, and submitted under the broker's digital seal.

Certification Options:

5.7.2.2 Certification Workflow

text

1. Seller selects broker in Logistics Builder (Mode B)
↓
2. Broker receives Smart Inbox: "Certification requested for USTN..."
↓
3. Broker reviews AI-generated declaration (read-only, with confidence scores)
↓
4. Low-confidence fields are highlighted (confidence <85%)
↓
5. Broker clicks "Certify" — adds digital seal, licence number, timestamp
↓
6. SGTX submits declaration to Nafeza under broker's credentials (mTLS)
↓
7. Declaration ID and status stored
↓
8. Broker receives confirmation; seller receives notification

5.7.2.3 Broker Review UI

■ CERTIFICATION REQUEST — SGTX-EG-26-F3A-1 ■

■ Seller: Strawberry Export Co. (SGTX-EG-TRD-002139-7F3A) ■

■ Buyer: European Importer GmbH (SGTX-DE-TRD-001234-5B6C) ■

□ □

■ ■ Declaration Type: EXPORT ■ ■

■ ■ USTN: SGTX-EG-26-F3A-1 ■ ■

■ ■ ACID: ACI20260415-0001 ■ ■

■ ■ Goods: ■ ■

■ ■ • Oranges (HS 0805.10) — 8,800 kg — Confidence: 92% ■ ■ ■

5.7.3 Certificate Issuance via Lab Results

5.7.3.1 Overview

When a laboratory submits compliant test results, SGTX automatically triggers certificate requests via Nafeza.

Certificate Types:

5.7.3.2 Certificate Request Flow

text

1. Laboratory submits test results

↓

2. System validates results against MRLs (A2)

↓

3. If all tests pass:

↓

4. System triggers certificate request to Nafeza

↓

5. Nafeza processes request (fees already paid in Stage 1)

↓

6. Nafeza issues certificate

↓

7. SGTX downloads certificate

↓

8. Certificate stored in `documents` with SHA256 hash

↓

9. Smart Inbox notification to seller and buyer

5.7.3.3 Certificate Request Example

Request:

json

POST /api/v2/declaration/{declaration_id}/certificates

```
{
  "type": "PHYTOSANITARY",
  "lab_report_ref": "USTN:SGTX-EG-26-F3A-1:LAB-001",
  "hs_code": "0805.10",
  "quantity": 8800,
  "quantity_unit": "KGM",
  "treatment": {
    "type": "COLD_TREATMENT",
```

```
"temperature_c": -0.5,  
"duration_days": 14,  
"facility": "Egyptian Cold Treatment Center",  
"certificate_ref": "CT-20260415-001"
```

```
}
```

```
}
```

Response:

json

```
{
```

```
"certificate_id": "PHYTO-20260415-001",  
"status": "ISSUED",  
"issued_at": "2026-06-20T11:00:00Z",  
"expires_at": "2026-12-20T23:59:59Z",  
"pdf_url": "https://nafeza.gov.eg/certificates/PHYTO-20260415-001.pdf",  
"sha256_hash": "sha256:abc123..."
```

```
}
```

5.7.3.4 Lab Result Validation (A2)

json

```
{
```

```
"lab_results": {  
"ustn": "SGTX-EG-26-F3A-1",  
"commodity": "Oranges",  
"hs_code": "0805.10",  
"results": [  
{
```

```
{
```

```
"analyte": "Cypermethrin",  
"value": 0.02,  
"unit": "mg/kg",  
"limit": 0.05,  
"pass": true,  
"confidence": 95
```

```
},
```

```
{
```

```
"analyte": "Chlorpyrifos",  
"value": 0.008,
```

```

"unit": "mg/kg",
"limit": 0.01,
"pass": true,
"confidence": 92
}
],
"conclusion": "COMPLIANT",
"submitted_by": "SGTX-EG-LAB-001",
"submitted_at": "2026-06-20T09:30:00Z"
}
}

```

5.7.4 Integration with Other Components

| Component | How Customs Declaration Flows |

|:---|:---|:---|

| 5.1 Weight Calculation | Weights used for declaration goods lines |

| 5.3 Packing List | Packing list hash referenced in declaration |

| 5.6 Invoice | Invoice number, value, and ETA UUID referenced |

| 4.9 Commercial Settlement | Settlement details included in declaration |

| 4.8 Insurance | Insurance details included in declaration |

| 4.6 Special Instructions | Special instructions included in declaration |

| 2.6 Incoterm | Incoterm included in transport section |

| 4.3 Product Form Agent | Acceptance criteria included in goods lines |

5.7.5 Validation Gates (Part 5.7)

5.7.6 Database Schema Additions (Part 5.7)

```
sql
```

```
-- Customs declarations (SAD)
```

```
CREATE TABLE customs_declarations (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
```

```
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
```

```
declaration_type TEXT NOT NULL, -- 'EXPORT', 'IMPORT'
```

```
declaration_id TEXT,
```

```
acid TEXT,
```

```
declaration_data JSONB NOT NULL,
```

```
-- Broker certification
```

```

broker_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,
broker_certified BOOLEAN DEFAULT false,
broker_certified_at TIMESTAMPTZ,
broker_digital_seal TEXT,
broker_licence_number TEXT,
-- Submission
submitted_by_gtid TEXT NOT NULL,
submitted_by_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
submitted_at TIMESTAMPTZ DEFAULT NOW(),
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'SUBMITTED', 'ACCEPTED', 'REJECTED', 'CLEARED'
nafeza_response JSONB,
nafeza_status_code INTEGER,
-- Certificates
certificates_requested JSONB,
certificates_issued JSONB,
-- Digital signature
digital_signature TEXT,
loom_hash TEXT,
-- Versioning
version INTEGER DEFAULT 1,
previous_version_id UUID REFERENCES customs_declarations(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Certificate requests
CREATE TABLE certificate_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
customs_declaration_id UUID NOT NULL REFERENCES customs_declarations(id) ON DELETE CASCADE,
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
certificate_type TEXT NOT NULL, -- 'PHYTOSANITARY', 'HEALTH', 'COO', 'COLD_TREATMENT', 'FUMIGATION'
lab_report_ref TEXT,
certificate_id TEXT,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'REQUESTED', 'ISSUED', 'REJECTED'
issued_at TIMESTAMPTZ,

```



```

expires_at TIMESTAMPTZ,
pdf_url TEXT,
sha256_hash TEXT,
rejection_reason TEXT,
requested_by_gtid TEXT,
requested_at TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Nafeza submission logs
CREATE TABLE nafeza_submission_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
customs_declaration_id UUID NOT NULL REFERENCES customs_declarations(id) ON DELETE CASCADE,
endpoint TEXT NOT NULL,
request_json JSONB,
response_json JSONB,
status_code INTEGER,
success BOOLEAN,
error_message TEXT,
idempotency_key TEXT,
submitted_at TIMESTAMPTZ DEFAULT NOW()
);

-- Lab results (from laboratory portal)
CREATE TABLE lab_results (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
commodity_id UUID REFERENCES trade_request_commodities(id) ON DELETE SET NULL,
lab_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
results JSONB NOT NULL,
conclusion TEXT, -- 'COMPLIANT', 'NON_COMPLIANT'
mrl_validation JSONB,
submitted_by_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
submitted_at TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

);
-- Certificate issuance logs
CREATE TABLE certificate_issuance_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
certificate_request_id UUID NOT NULL REFERENCES certificate_requests(id) ON DELETE CASCADE,
issuance_attempt INTEGER DEFAULT 1,
success BOOLEAN,
error_message TEXT,
attempted_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_customs_declarations_ustn ON customs_declarations(ustn);
CREATE INDEX idx_customs_declarations_trade_request ON customs_declarations(trade_request_id);
CREATE INDEX idx_customs_declarations_status ON customs_declarations(status);
CREATE INDEX idx_customs_declarations_broker ON customs_declarations(broker_gtid);
CREATE INDEX idx_certificate_requests_ustn ON certificate_requests(ustn);
CREATE INDEX idx_certificate_requests_customs_declaration ON
certificate_requests(customs_declaration_id);
CREATE INDEX idx_certificate_requests_status ON certificate_requests(status);
CREATE INDEX idx_nafeza_submission_logs_declaration ON
nafeza_submission_logs(customs_declaration_id);
CREATE INDEX idx_nafeza_submission_logs_submitted_at ON nafeza_submission_logs(submitted_at
DESC);
CREATE INDEX idx_lab_results_ustn ON lab_results(ustn);
CREATE INDEX idx_lab_results_submitted_at ON lab_results(submitted_at DESC);
CREATE INDEX idx_certificate_issuance_logs_request ON
certificate_issuance_logs(certificate_request_id);

```

5.7.7 Implementation Checklist (Part 5.7)

5.7.7.1 SAD Generation

- Implement SAD JSON generator (Rust)
- Implement SAD field mapping (trade request → SAD)
- Implement goods line generation (per commodity)
- Implement certificate requests generation
- Implement transport section generation
- Implement special instructions integration
- Implement insurance integration
- Implement commercial settlement integration

5.7.7.2 Broker Certification

- Implement broker certification workflow
- Implement broker review UI (declaration preview)
- Implement low-confidence field highlighting
- Implement digital seal application
- Implement broker certification storage

5.7.7.3 Nafeza Integration

- Implement Nafeza API client (mTLS)
- Implement declaration submission endpoint (POST /api/v2/declaration)
- Implement certificate request endpoint (POST /api/v2/declaration/{id}/certificates)
- Implement declaration status polling (GET /api/v2/declaration/{id})
- Implement certificate download and storage
- Implement retry logic (exponential backoff)
- Implement fallback to manual PDF submission

5.7.7.4 Lab Result Integration

- Implement lab result validation (A2)
- Implement MRL validation (A2)
- Implement automatic certificate trigger on compliant results
- Implement lab result storage

5.7.7.5 Certificate Management

- Implement certificate request tracking
- Implement certificate download and storage
- Implement certificate expiry monitoring
- Implement certificate renewal reminders

5.7.7.6 Testing

- Unit tests: SAD JSON generation
- Unit tests: certificate request generation
- Unit tests: broker certification workflow
- Integration tests: declaration generation → Nafeza submission
- Integration tests: lab results → certificate trigger
- Integration tests: broker certification → Nafeza submission
- End-to-end tests: full customs declaration workflow

5.7.8 AI Authority Summary for Part 5.7

END OF PART 5.7 — CUSTOMS DECLARATION AUTO-GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.8 — ECOLOGICAL PACKAGING ADVISOR

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.8.0 Purpose & Design Philosophy

The Ecological Packaging Advisor is an AI-driven, advisory-only feature that analyses the packaging materials used in a trade and suggests sustainable alternatives that reduce carbon footprint, comply with destination regulations, and often save costs. This feature addresses the growing regulatory and consumer demand for sustainable trade practices, including EU Single-Use Plastics Directive, Germany's Packaging Act, and corporate ESG commitments.

Key Principles:

Advisory only — Suggestions are recommendations; the seller can accept, modify, or ignore.

Data-driven — Recommendations are based on LCA (Life Cycle Assessment) data, regulatory requirements, and cost comparisons.

Regulatory-aware — Considers destination-country packaging regulations (e.g., EU Single-Use Plastics Directive, Germany's Packaging Act).

Cost-transparent — Shows cost savings (or additional costs) for each alternative.

One-click application — Seller can apply recommendations to all containers or specific containers with a single click.

Non-marketplace — Never recommends specific suppliers or vendors; only suggests material types.

Integrated with special instructions — Aligns with buyer's special instructions (e.g., "No wooden pallets").

AI Integration in Part 5.8:

5.8.1 Core Technology Stack

5.8.2 Data Sources & Databases

5.8.2.1 Material Database

5.8.2.2 Regulatory Database (RIA-Maintained)

5.8.2.3 Special Instruction Integration

The Ecological Packaging Advisor respects and aligns with the buyer's special trade instructions:

5.8.3 Recommendation Engine

5.8.3.1 Suggestion Generation Logic

rust

// Pseudo-code: Ecological packaging recommendation engine

```
fn generate_packaging_recommendations(
```

```
current_material: &Material,
```

```
destination_country: &str,
```

```
weight_kg: f64,
```

```
quantity: f64,
```

```
special_instructions: &[String],
```

```
current_cost: f64
```

```
) -> Vec<PackagingSuggestion> {
```

```

let mut suggestions = Vec::new();
// 1. Get all alternative materials
let alternatives = get_alternative_materials(current_material);
// 2. Filter by special instructions
let filtered = filter_by_special_instructions(alternatives, special_instructions);
// 3. Check regulatory compliance
let compliant = check_regulatory_compliance(filtered, destination_country);
// 4. Calculate carbon savings
for material in compliant {
let carbon_saving = calculate_carbon_saving(current_material, material, weight_kg);
let cost_impact = calculate_cost_impact(current_cost, material.cost_index);
let compliance_status = check_compliance(material, destination_country);
suggestions.push(PackagingSuggestion {
material: material,
carbon_saving_kg_co2e: carbon_saving,
cost_impact_percent: cost_impact,
cost_impact_usd: current_cost * cost_impact,
compliance_status: compliance_status,
recommendation_strength: calculate_recommendation_strength(carbon_saving, cost_impact,
compliance_status),
});
}
// 5. Sort by recommendation strength (A1, Groq)
suggestions.sort_by(|a, b|
b.recommendation_strength.partial_cmp(&a.recommendation_strength).unwrap());
// 6. Generate explanations (A1, Groq)
for suggestion in &mut suggestions {
suggestion.explanation = generate_explanation(suggestion, destination_country)?;
}
suggestions
}

```

5.8.3.2 Recommendation Strength Scoring

Example Scoring:

text

Virgin Plastic Strapping → Recycled PET Strapping

- Carbon Saving: 52% → Score: 52

[REDACTED]

[REDACTED]

```
let disposal emissions = match material.disposal method {
```

```

DisposalMethod::Landfill => 0.5,
DisposalMethod::Recycling => 0.1,
DisposalMethod::Composting => 0.05,
DisposalMethod::Incineration => 0.8,
};
// 4. Total carbon impact
let total = (quantity_kg * base_carbon) + transport_emissions + disposal_emissions;
total
}

```

5.8.5.2 Cost Impact Calculation

```

rust
// Pseudo-code: Cost impact calculation
fn calculate_cost_impact(
current_material: &Material,
alternative_material: &Material,
quantity_kg: f64
) -> CostImpact {
let current_cost = current_material.cost_per_kg * quantity_kg;
let alternative_cost = alternative_material.cost_per_kg * quantity_kg;
let cost_difference = alternative_cost - current_cost;
let cost_percent_change = (cost_difference / current_cost) * 100.0;
CostImpact {
current_cost,
alternative_cost,
cost_difference,
cost_percent_change,
}
}

```

5.8.6 Integration with Other Components

| Component | How Ecological Packaging Advisor Flows |

|:---|:---|:---|

| 5.2 Palletisation Optimiser | Packaging material changes may affect pallet height and weight; plan is re-optimised |

| 5.3 Packing List | Updated packaging materials reflected in packing list |

| 5.6 Invoice | Updated packaging costs reflected in invoice (if material cost changes) |

| 5.9 Carbon Footprint | Carbon calculation updated with new packaging materials |

| 4.6 Special Instructions | Advisor respects buyer's special instructions |

| 4.8 Insurance | Packaging type may affect insurance requirements (e.g., hazardous materials) |

| 5.11 Lock Packing Plan | Packaging changes validated before locking |

5.8.7 Validation Gates (Part 5.8)

5.8.8 Database Schema Additions (Part 5.8)

sql

-- Ecological packaging recommendations

```
CREATE TABLE packaging_recommendations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,  
  commodity_id UUID REFERENCES trade_request_commodities(id) ON DELETE SET NULL,  
  container_id UUID REFERENCES trade_request_containers(id) ON DELETE SET NULL,  
  current_material TEXT NOT NULL,  
  recommended_material TEXT NOT NULL,  
  carbon_saving_kg_co2e DECIMAL(10,2),  
  carbon_saving_percent DECIMAL(5,2),  
  cost_impact_usd DECIMAL(10,2),  
  cost_impact_percent DECIMAL(5,2),  
  compliance_status TEXT NOT NULL, -- 'COMPLIANT', 'RESTRICTED', 'BANNED'  
  recommendation_strength TEXT, -- 'HIGH', 'MEDIUM', 'LOW'  
  explanation TEXT,  
  applied BOOLEAN DEFAULT false,  
  applied_at TIMESTAMPTZ,  
  applied_to TEXT, -- 'ALL_CONTAINERS', 'COMMODITY_ONLY', 'CONTAINER_ONLY'  
  dismissed BOOLEAN DEFAULT false,  
  dismissed_at TIMESTAMPTZ,  
  dismissed_reason TEXT,  
  generated_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
  generated_at TIMESTAMPTZ DEFAULT NOW(),  
  loom_hash TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Packaging material database

```
CREATE TABLE packaging_materials (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

material_code TEXT UNIQUE NOT NULL,
material_name TEXT NOT NULL,
material_name_ar TEXT,
category TEXT NOT NULL, -- 'PLASTIC', 'PAPER', 'WOOD', 'METAL', 'GLASS', 'BIO', 'COMPOSITE'
carbon_footprint_kg_co2e_per_kg DECIMAL(10,4),
cost_per_kg DECIMAL(10,4),
currency TEXT DEFAULT 'USD',
recycled_content_percent INTEGER DEFAULT 0,
is_biodegradable BOOLEAN DEFAULT false,
is_recyclable BOOLEAN DEFAULT true,
is_reusable BOOLEAN DEFAULT false,
is_hazardous BOOLEAN DEFAULT false,
halal_certified BOOLEAN DEFAULT false,
ispm15_compliant BOOLEAN DEFAULT false,
typical_use TEXT,
tensile_strength_mpa DECIMAL(10,2),
max_weight_kg DECIMAL(10,2),
source TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Destination-country packaging regulations (RIA-maintained)
CREATE TABLE packaging_regulations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
regulation_name TEXT NOT NULL,
regulation_reference TEXT,
material_restrictions JSONB, -- [{material_code: 'EPS_FOAM', restriction: 'BANNED', effective_date:
'2021-07-03'}]
recycled_content_requirement INTEGER, -- Percentage
reporting_required BOOLEAN DEFAULT false,
reporting_frequency TEXT, -- 'MONTHLY', 'QUARTERLY', 'ANNUAL'
penalty_for_non_compliance TEXT,
effective_date DATE,
last_updated TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(country_code, regulation_name)

```

```

);
-- Packaging compliance checks (historical)
CREATE TABLE packaging_compliance_checks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
  commodity_id UUID REFERENCES trade_request_commodities(id) ON DELETE SET NULL,
  material_code TEXT NOT NULL,
  destination_country CHAR(2) NOT NULL,
  is_compliant BOOLEAN DEFAULT true,
  violation_details JSONB,
  checked_at TIMESTAMPTZ DEFAULT NOW(),
  loom_hash TEXT
);
-- Ecological packaging advisor logs
CREATE TABLE ecological_packaging_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
  action_type TEXT NOT NULL, -- 'RECOMMENDATION_GENERATED',
  'RECOMMENDATION_APPLIED', 'RECOMMENDATION_DISMISSED'
  action_data JSONB,
  performed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
  performed_at TIMESTAMPTZ DEFAULT NOW(),
  loom_hash TEXT
);
-- Indexes
CREATE INDEX idx_packaging_recommendations_trade ON
packaging_recommendations(trade_request_id);
CREATE INDEX idx_packaging_recommendations_applied ON packaging_recommendations(applied)
WHERE applied = true;
CREATE INDEX idx_packaging_recommendations_dismissed ON
packaging_recommendations(dismissed) WHERE dismissed = true;
CREATE INDEX idx_packaging_materials_code ON packaging_materials(material_code);
CREATE INDEX idx_packaging_materials_category ON packaging_materials(category);
CREATE INDEX idx_packaging_regulations_country ON packaging_regulations(country_code);
CREATE INDEX idx_packaging_regulations_effective ON packaging_regulations(effective_date);
CREATE INDEX idx_packaging_compliance_checks_trade ON
packaging_compliance_checks(trade_request_id);

```

```
CREATE INDEX idx_ecological_packaging_logs_trade ON ecological_packaging_logs(trade_request_id);  
CREATE INDEX idx_ecological_packaging_logs_performed_at ON  
ecological_packaging_logs(performed_at DESC);
```

5.8.9 Implementation Checklist (Part 5.8)

5.8.9.1 Backend Services

- Implement ecological packaging advisor service (Rust)
- Implement material database (Ecoinvent + LCA Commons)
- Implement regulatory database (RIA-maintained)
- Implement recommendation generation logic
- Implement carbon impact calculation
- Implement cost impact calculation
- Implement compliance checking
- Implement special instruction alignment

5.8.9.2 Recommendation Engine

- Implement Groq suggestion generation (A1)
- Implement recommendation strength scoring
- Implement recommendation filtering (special instructions)
- Implement regulatory compliance filtering
- Implement sorting and ranking

5.8.9.3 Frontend Components

- Build Ecological Packaging Advisor panel
- Build Current Packaging Materials display
- Build Recommendations display (with carbon savings, cost impact, compliance)
- Build AI Insights display (A1, Groq)
- Build Carbon Footprint Impact Summary
- Build One-click Apply buttons
- Build Save Report functionality

5.8.9.4 Integration

- Integrate with special instructions (Part 4.6)
- Integrate with packing plan (Part 5.2)
- Integrate with packing list (Part 5.3)
- Integrate with invoice (Part 5.6)
- Integrate with carbon footprint (Part 5.9)
- Integrate with RIA for regulatory updates

5.8.9.5 Testing

- Unit tests: recommendation generation

Unit tests: carbon impact calculation

Unit tests: cost impact calculation

Unit tests: compliance checking

Integration tests: recommendation → application → packing plan update

Integration tests: regulatory database integration

End-to-end tests: full ecological packaging workflow

5.8.10 AI Authority Summary for Part 5.8

END OF PART 5.8 — ECOLOGICAL PACKAGING ADVISOR

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.9 — CARBON FOOTPRINT CALCULATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.9.0 Purpose & Design Philosophy

The Carbon Footprint Calculation module provides an ISO 14067-compliant, CBAM-ready estimation of greenhouse gas emissions for the entire trade shipment. It calculates Scope 1, 2, and 3 emissions across the supply chain — from farm to destination — enabling traders to comply with EU CBAM regulations, meet corporate ESG targets, and make informed logistics decisions. The calculation is advisory and integrated into invoices, packing lists, and customs declarations.

Key Principles:

ISO 14067:2018 compliant — Methodology follows international standards.

CBAM-ready — Generates CBAM reports for EU-bound shipments.

Transport mode-aware — Emissions vary by ocean, air, rail, and truck.

Scope 1, 2, 3 coverage — Direct emissions, indirect energy, and upstream/downstream.

Data-driven — Uses IMO EEXI, EPA SmartWay, IEA grid factors, and Ecoinvent.

Advisory only — Never blocks trades; used for reporting and optimisation.

Integrated with invoice and customs — Emissions data flows into documents.

AI Integration in Part 5.9:

5.9.1 Scope Definitions & Calculation Methodology

5.9.1.1 Scope 1 — Direct Emissions

5.9.1.2 Scope 2 — Indirect Emissions

5.9.1.3 Scope 3 — Upstream & Downstream

5.9.2 Emission Factors & Data Sources

5.9.3 CBAM Report Generation

5.9.3.1 CBAM Fields

5.9.3.2 CBAM XML Example

xml

```
<CBAMReport xmlns="https://cbam.eu/report/v1">
```

```
<ReportID>CBAM-2026-001</ReportID>
```

<ReportingPeriod>
<Start>2026-06-20</Start>
<End>2026-07-05</End>
</ReportingPeriod>
<Exporter>
<Name>Strawberry Export Co.</Name>
<GTID>SGTX-EG-TRD-002139-7F3A</GTID>
</Exporter>
<Importer>
<Name>European Importer GmbH</Name>
<GTID>SGTX-DE-TRD-001234-5B6C</GTID>
</Importer>
<Goods>
<Commodity>
<HS>0805.10</HS>
<Description>Valencia Oranges</Description>
<NetWeight>8800</NetWeight>
<Unit>KGM</Unit>
</Commodity>
</Goods>
<Emissions>
<Scope1>
<Vessel>3000</Vessel>
<Truck>120</Truck>
</Scope1>
<Scope2>
<Reefer>600</Reefer>
</Scope2>
<Scope3>
<Packaging>800</Packaging>
<PortHandling>200</PortHandling>
<Disposal>150</Disposal>
</Scope3>
<Total>4870</Total>
<EmbeddedEmissions>1800</EmbeddedEmissions> <!-- Scope 1 + Scope 2 -->


```
</Emissions>
<Certificate>
  <Number>CBAM-2026-001</Number>
  <IssuedAt>2026-06-20T10:30:00Z</IssuedAt>
  <ValidUntil>2027-06-20T23:59:59Z</ValidUntil>
  <Signature>ed25519:...</Signature>
</Certificate>
</CBAMReport>
```

5.9.4 Calculation Logic

rust

// Pseudo-code: Carbon footprint calculation

```
fn calculate_carbon_footprint(
  trade_data: &TradeData,
  transport: &TransportConfig,
  packing: &PackingPlan,
  distance_km: f64
) -> CarbonFootprint {
  let mut scope1 = 0.0;
  let mut scope2 = 0.0;
  let mut scope3 = 0.0;
  // 1. Scope 1 — Vessel
  match transport.mode {
    TransportMode::Ocean => {
      let fuel_consumption = distance_km * 0.015; // 0.015 kg fuel / tkm
      let total_weight_tonnes = trade_data.total_gross_weight_kg / 1000.0;
      scope1 += fuel_consumption * total_weight_tonnes * 3.15; // HFO factor
    },
    TransportMode::Air => {
      let fuel_consumption = distance_km * 0.05; // 0.05 kg fuel / tkm
      let total_weight_tonnes = trade_data.total_gross_weight_kg / 1000.0;
      scope1 += fuel_consumption * total_weight_tonnes * 3.16; // Jet fuel factor
    },
    _ => { /* similar for rail and truck */ }
  }
  // 2. Scope 2 — Reefer electricity
```


■ ■ ■ 4,870 kg CO₂e ■ ■ ■

■ ■ Breakdown: ■ ■

1. The first line of the document is a header consisting of a series of asterisks followed by the text "*****".

■ ■ Detailed Breakdown: ■ ■

■ ■ • Truck Fuel: 120 kg CO₂e ■ ■

■ ■ • Packaging Materials: 800 kg CO₂e ■ ■

■ ■ • Disposal: 150 kg CO₂e ■ ■

■ ■ CBAM Embedded Emissions: 1,800 kg CO₂e (Scope 1 + 2) ■ ■

■ ■ Data Sources: IMO EEXI 2025, EPA SmartWay v3, IEA 2025 ■ ■

■ CBAM REPORT ■

[\[Download CBAM XML\]](#) [\[Download CBAM PDF\]](#) [\[View CBAM Certificate\]](#)

Certificate Number: CBAM-2026-001

Issued: 2026-06-20 10:30 UTC

Valid Until: 2027-06-20

CBAM reports are required for all EU-bound shipments. This report is

digitally signed and can be submitted to EU authorities.

REDUCTION OPPORTUNITIES (A1, Groq)

"The largest emissions source is vessel fuel (3,000 kg CO₂e). Consider

switching to a newer vessel or optimising loading to reduce fuel

consumption. Packaging emissions (800 kg CO₂e) could be reduced by

switching to recycled PET strapping (see Ecological Packaging Advisor).

Total potential reduction: 25% (1,218 kg CO₂e)."

[\[Recalculate\]](#) [\[Export Report\]](#) [\[Share with Buyer\]](#) [\[Dismiss\]](#)

5.9.6 Integration with Invoice & Packing List

5.9.7 Validation Gates (Part 5.9)

5.9.8 Database Schema Additions (Part 5.9)

sql

-- Carbon footprint calculations

CREATE TABLE carbon_footprints (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,

scope1_co2e_kg DECIMAL(12,2),

scope2_co2e_kg DECIMAL(12,2),

scope3_co2e_kg DECIMAL(12,2),

total_co2e_kg DECIMAL(12,2),

embedded_emissions_kg DECIMAL(12,2), -- Scope 1 + Scope 2 (CBAM)

```

confidence_interval JSONB,
data_sources TEXT[],
model_version TEXT,
cbam_certificate_number TEXT,
cbam_xml_url TEXT,
cbam_pdf_url TEXT,
calculated_at TIMESTAMPTZ DEFAULT NOW(),
calculated_by UUID REFERENCES employees(id) ON DELETE SET NULL,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Emission factors (reference table, updated by RIA)
CREATE TABLE emission_factors (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
factor_type TEXT NOT NULL, -- 'VESSEL', 'TRUCK', 'RAIL', 'AIR', 'ELECTRICITY', 'MATERIAL'
factor_code TEXT UNIQUE NOT NULL,
factor_name TEXT NOT NULL,
value DECIMAL(12,6) NOT NULL,
unit TEXT NOT NULL,
source TEXT,
valid_from DATE,
valid_to DATE,
is_active BOOLEAN DEFAULT true,
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(factor_code, valid_from)
);

-- Carbon reduction recommendations (A1-generated)
CREATE TABLE carbon_reduction_recommendations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
carbon_footprint_id UUID NOT NULL REFERENCES carbon_footprints(id) ON DELETE CASCADE,
recommendation_text TEXT,
potential_reduction_kg DECIMAL(12,2),
confidence DECIMAL(5,2),
generated_by UUID REFERENCES employees(id) ON DELETE SET NULL,
generated_at TIMESTAMPTZ DEFAULT NOW(),

```

```
loom_hash TEXT
);
-- Indexes
CREATE INDEX idx_carbon_footprints_ustn ON carbon_footprints(ustn);
CREATE INDEX idx_carbon_footprints_calculated_at ON carbon_footprints(calculated_at DESC);
CREATE INDEX idx_emission_factors_code ON emission_factors(factor_code);
CREATE INDEX idx_emission_factors_active ON emission_factors(is_active) WHERE is_active = true;
CREATE INDEX idx_carbon_reduction_recommendations_footprint ON
carbon_reduction_recommendations(carbon_footprint_id);
```

5.9.9 Implementation Checklist (Part 5.9)

5.9.9.1 Core Calculation

Implement carbon footprint calculation service (Rust + Python)

Implement emission factor lookup (database)

Implement Scope 1 calculation (vessel, truck, rail, air)

Implement Scope 2 calculation (electricity)

Implement Scope 3 calculation (packaging, port, disposal)

Implement CBAM report generation (XML)

5.9.9.2 Frontend Components

Build Carbon Footprint summary display

Build Scope breakdown chart

Build Detailed breakdown list

Build CBAM report download buttons

Build Reduction opportunities display (A1, Groq)

Build Export functionality

5.9.9.3 Integration

Integrate with invoice (add carbon data)

Integrate with packing list (add carbon summary)

Integrate with customs declaration (add CBAM reference)

Integrate with transport mode data

5.9.9.4 Testing

Unit tests: emission factor lookup

Unit tests: Scope 1 calculation

Unit tests: CBAM report generation

Integration tests: carbon calculation → report generation

Integration tests: invoice integration

End-to-end tests: full carbon footprint workflow

5.9.10 AI Authority Summary for Part 5.9

END OF PART 5.9 — CARBON FOOTPRINT CALCULATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.10 — PDF/A-3 ARCHIVAL FORMAT

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.10.0 Purpose & Design Philosophy

The PDF/A-3 Archival Format module ensures that all legally significant documents generated by the platform are preserved in a format suitable for long-term archiving. PDF/A-3 (ISO 19005-3) is the international standard for archival PDF documents, guaranteeing that the documents will be readable and render correctly decades from now. This module applies to all automatically generated documents intended for legal retention — invoices, packing lists, customs declarations, contracts, and certificates.

Key Principles:

ISO 19005-3 compliant — Level B (basic) or U (Unicode) conformance.

Self-contained — All fonts, colour spaces, and metadata are embedded.

Digitally signed — Each document includes a visible digital signature.

Metadata-rich — Contains USTN, document type, generation timestamp, SHA256 hash, and Loom hash.

Automated validation — Each PDF/A-3 file is validated before storage.

Fallback — If PDF/A-3 generation fails, a standard PDF is generated with a warning.

AI Integration in Part 5.10:

| AI Level | Use Cases |

|:---|:---|:---|

| A1 (Groq → Ollama) | Metadata generation (plain-language document descriptions), validation error explanations |

| A4 (OPA + WasmEdge) | PDF/A-3 conformance validation, signature verification, archival storage enforcement |

5.10.1 Technical Requirements

5.10.2 Document Types Covered

5.10.3 PDF/A-3 Generation Process

text

1. Source document generated (HTML, XML, JSON)

↓

2. Render to PDF using headless Chrome (or printpdf)

↓

3. Convert to PDF/A-3:

a. Embed all fonts

b. Convert colour spaces to device-independent

c. Add XMP metadata

- e. Add Loom hash and SHA256

↓

- ↓

- ↓

- ↓

5.10.4 Metadata Structure (XMP)

xml

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
<rdf:Description rdf:about=""
xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:xmp="http://ns.adobe.com/xap/1.0/"
xmlns:pdf="http://ns.adobe.com/pdf/1.3/"
xmlns:sgtx="http://sgtx.io/xmp/1.0/">
<dc:title>
<rdf:Alt>
<rdf:li xml:lang="en">Commercial Invoice</rdf:li>
<rdf:li xml:lang="ar">فاتورة تجارية</rdf:li>
</rdf:Alt>
</dc:title>
<dc:description>
<rdf:Alt>
<rdf:li xml:lang="en">Commercial Invoice for USTN SGTX-1397F3A-...</rdf:li>
</rdf:Alt>
</dc:description>
<dc:date>2026-06-20</dc:date>
<xmp:CreateDate>2026-06-20T10:30:00Z</xmp:CreateDate>
<xmp:ModifyDate>2026-06-20T10:30:00Z</xmp:ModifyDate>
<pdf:Producer>SGTX Platform v11.0</pdf:Producer>
<pdf:Keywords>USTN, Invoice, Trade</pdf:Keywords>
<!-- SGTX Custom Metadata -->
```


<sgtx:USTN>SGTX-EG-26-F3A-1</sgtx:USTN>
<sgtx:DocumentType>COMMERCIAL_INVOICE</sgtx:DocumentType>
<sgtx:DocumentID>INV-STR-2026-001</sgtx:DocumentID>
<sgtx:SHA256>sha256:abc123...</sgtx:SHA256>
<sgtx:LoomHash>sha256:def456...</sgtx:LoomHash>
<sgtx:Version>1.0</sgtx:Version>
<sgtx:IssuerGTID>SGTX-EG-TRD-002139-7F3A</sgtx:IssuerGTID>
<sgtx:SignatureType>Ed25519</sgtx:SignatureType>
<sgtx:Signature>base64...</sgtx:Signature>
</rdf:Description>
</rdf:RDF>

5.10.5 UI — Archival Validation

text

DOCUMENT ARCHIVAL — Commercial Invoice

Document ID: INV-STR-2026-001

USTN: SGTX-EG-26-F3A-1

PDF/A-3 Status:

PDF/A-3 Compliant (Level B)

Verified: 2026-06-20 10:31:00 UTC

Validation Details:

• All fonts embedded:

• Colour spaces device-independent:

• Metadata present:

• Digital signature visible:

• Loom hash embedded:

Archival Metadata:

```
CREATE TABLE pdfa3_archival (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
pdfa3_url TEXT,
pdfa3_hash TEXT,
pdfa3_validated BOOLEAN DEFAULT false,
pdfa3_validation_report JSONB,
pdfa3_validated_at TIMESTAMPTZ,
standard_pdf_url TEXT, -- fallback
```

```

metadata_xmp TEXT,
digital_signature_visible TEXT,
loom_hash TEXT,
archived_at TIMESTAMPTZ DEFAULT NOW(),
archived_by UUID REFERENCES employees(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Archival validation logs
CREATE TABLE pdfa3_validation_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
archival_id UUID NOT NULL REFERENCES pdfa3_archival(id) ON DELETE CASCADE,
validator TEXT, -- 'verapdf', 'custom'
result BOOLEAN,
error_message TEXT,
validated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_pdfa3_archival_document ON pdfa3_archival(document_id);
CREATE INDEX idx_pdfa3_archival_archived_at ON pdfa3_archival(archived_at DESC);
CREATE INDEX idx_pdfa3_validation_logs_archival ON pdfa3_validation_logs(archival_id);

```

5.10.9 Implementation Checklist (Part 5.10)

5.10.9.1 Core Generation

Implement PDF/A-3 generation (Rust, pdf-writer + PDF/A-3 extension)

Implement font embedding

Implement colour space conversion

Implement XMP metadata embedding

Implement visible digital signature field

Implement Loom hash embedding

5.10.9.2 Validation

Implement PDF/A-3 validation (veraPDF or custom)

Implement validation reporting

Implement fallback to standard PDF with warning

Implement validation logging

5.10.9.3 Frontend Components

Build Archival Status display

Build Validation Details display

Build Download buttons (PDF/A-3 + standard PDF)

Build Verify Loom Hash button

5.10.9.4 Testing

Unit tests: PDF/A-3 generation

Unit tests: validation

Integration tests: generation → validation → storage

Integration tests: fallback handling

End-to-end tests: full archival workflow

5.10.10 AI Authority Summary for Part 5.10

END OF PART 5.10 — PDF/A-3 ARCHIVAL FORMAT

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 5.11 — LOCKING THE PACKING PLAN & BARCODE GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.11.0 Purpose & Design Philosophy

The Locking the Packing Plan & Barcode Generation module is the final validation and immutable commitment step before physical execution. Once the packing plan is locked, it becomes immutable — all pallet positions, layer patterns, weights, and SSCC codes are fixed and recorded in the Loom hash chain. This ensures that any deviation or dispute can be verified against the locked plan. Simultaneously, SSCC-18 barcodes and QR codes are generated for each pallet, enabling physical tracking and offline verification.

Key Principles:

Immutable plan — Once locked, the packing plan cannot be modified.

Governor validation — All constraints (weight, height, compatibility) are re-validated before lock.

Loom hashing — The plan is anchored in the immutable Loom hash chain.

Barcode generation — SSCC-18 codes and QR codes are generated for each pallet.

Reprint policy — Reprints after loading require Governor approval.

Multi-shipment support — Each shipment's packing plan is locked independently.

Non-marketplace — No counterparty recommendations.

AI Integration in Part 5.11:

5.11.1 Locking Process

5.11.1.1 Pre-Lock Validation

5.11.1.2 Lock Workflow

text

1. User with `packing.lock` permission clicks "Lock Packing Plan"

↓

2. Governor runs validation checks

↓

3. If ALLOW:

- a. Generate SSCC-18 codes for each pallet
- b. Generate QR codes with Verifiable Credentials
- c. Calculate Loom hash of the plan
- d. Store locked plan with timestamp and locker ID
- e. Update FeeLock status (if applicable)
- f. Notify all collaborators (Smart Inbox)

↓

4. If CONDITIONAL:

- a. Display Decision Panel with specific conditions
- b. User resolves conditions and retries

↓

5. If DENY:

- a. Display Decision Panel with explanation
- b. User must contact admin or resolve permission issues

5.11.2 SSCC-18 Barcode Generation

5.11.2.1 Format

SSCC-18: 1 0614141 123456789 0 → displayed as 106141411234567890

5.11.2.2 Generation Algorithm

rust

```
fn generate_ssc(extension_digit: u8, company_prefix: &str, serial: u64) -> String {  
    let base = format!("{0}{1:09}", extension_digit, company_prefix, serial);  
    let check_digit = calculate_gs1_check_digit(&base);  
    format!("{0}{1}", base, check_digit)  
}  
  
fn calculate_gs1_check_digit(base: &str) -> u8 {  
    let mut sum = 0;  
    for (i, c) in base.chars().enumerate() {  
        let digit = c.to_digit(10).unwrap();  
        if i % 2 == 0 { sum += digit * 3 } else { sum += digit * 1 }  
    }  
    let remainder = sum % 10;  
    if remainder == 0 { 0 } else { 10 - remainder }  
}
```

5.11.2.3 SSCC Storage

sql

```
CREATE TABLE pallet_details (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  packing_plan_id UUID NOT NULL REFERENCES packing_plans(id),  
  sscd TEXT UNIQUE NOT NULL,  
  -- ... other fields ...  
);
```

5.11.3 QR Code with Verifiable Credential

5.11.3.1 QR Payload

json

```
{  
  "ustn": "SGTX-EG-26-F3A-1",  
  "pallet_id": "OR-001",  
  "sscd": "106141411234567890",  
  "product": "Fresh Oranges",  
  "hs_code": "0805.10",  
  "lot": "OR-2026-0412",  
  "net_weight_kg": 1110,  
  "gross_weight_kg": 1165,  
  "layer_summary": "11x10 + 1x1 cartons",  
  "cold_treatment_cert": "CT-20260415-001",  
  "verify_url": "https://sgtx.io/verify/pallet?sscd=106141411234567890",  
  "verifiable_credential": {  
    "type": "VerifiableCredential",  
    "issuer": "https://sgtx.io/issuers/platform",  
    "issuanceDate": "2026-06-20T10:30:00Z",  
    "proof": {  
      "type": "Ed25519Signature2020",  
      "signature": "base58..."  
    }  
  }  
}
```

5.11.3.2 Offline Verification

Mobile app scans QR code.

App extracts Verifiable Credential.

App verifies signature against cached SGTX public key.

App validates credential not expired.

App displays green checkmark: "Verified — Lot OR-2026-0412, 111 cartons, 1,110 kg."

5.11.4 Loom Hashing

rust

// Pseudo-code: Loom hash generation

```
fn lock_packing_plan(plan: &PackingPlan) -> Result<LockedPlan> {
```

// 1. Serialise plan to canonical JSON

```
let plan_json = serde_json::to_string(plan)?;
```

// 2. Get previous Loom hash

```
let previous_hash = get_last_loom_hash()?;
```

// 3. Sign with Ed25519

```
let signature = sign_with_ed25519(&plan_json)?;
```

// 4. Calculate Loom hash

```
let loom_hash = sha256(format!("{}", previous_hash, plan_json, signature));
```

// 5. Store locked plan

```
let locked = LockedPlan {
```

```
plan: plan.clone(),
```

```
locked_at: Utc::now(),
```

```
locked_by: current_user(),
```

```
loom_hash,
```

```
signature,
```

```
previous_hash,
```

```
};
```

```
Ok(locked)
```

```
}
```

5.11.5 Reprint Policy (Governor-Enforced)

Reprint Workflow (Post-Loading):

User clicks "Request Reprint".

Modal appears: "Reason for reprint (min 10 characters):" + text field.

User submits reason.

Governor evaluates:

If pallet status is LOADED or DELIVERED → requires reason.

If pallet status is DELIVERED and shipment is SETTLED → DENY.

If reason is insufficient (<10 chars) → CONDITIONAL.
If ALLOW → new labels with same SSCC generated (enhanced contrast if requested).
If DENY → Decision Panel explains.

5.11.6 UI — Locking & Barcode Generation

text

PACKING PLAN — SGTX-EG-26-F3A-1

Collaborators: Ahmed (—), Sara (■■), Mohamed (■■)

Status: Unlocked | Last Saved: 10:30:00

VALIDATION SUMMARY

Weight within limits (15,784.8 kg ≤ 28,000 kg)

Stacking heights compliant

Commodity compatibility check passed

Treatment certificates planned

All fields complete

[Lock Packing Plan] [Request Human Review] [Save Draft] [Export PDF]

After locking, the following barcodes will be generated:

Pallet ID

SSCC

QR Code

Status

Actions

OR-001

106141411234567890

Pending

[Print]

OR-002

106141411234567891

Pending

[Print]


```

loom_hash TEXT
);
-- Reprint requests (post-loading)
CREATE TABLE reprint_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
pallet_id UUID NOT NULL REFERENCES pallet_details(id) ON DELETE CASCADE,
reason TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'APPROVED', 'REJECTED'
governor_decision_id UUID REFERENCES governor_decisions(decision_id),
requested_by UUID REFERENCES employees(id),
requested_at TIMESTAMPTZ DEFAULT NOW(),
approved_at TIMESTAMPTZ,
loom_hash TEXT
);
-- Indexes
CREATE INDEX idx_packing_plans_is_locked ON packing_plans(is_locked) WHERE is_locked = true;
CREATE INDEX idx_packing_plans_locked_at ON packing_plans(locked_at DESC);
CREATE INDEX idx_barcode_generation_logs_pallet ON barcode_generation_logs(pallet_id);
CREATE INDEX idx_reprint_requests_pallet ON reprint_requests(pallet_id);
CREATE INDEX idx_reprint_requests_status ON reprint_requests(status);

```

5.11.10 Implementation Checklist (Part 5.11)

5.11.10.1 Locking

Implement lock endpoint (POST /v1/packing/{id}/lock)

Implement pre-lock validation (all gates)

Implement Loom hash generation

Implement digital signing

Implement lock state propagation (NATS)

5.11.10.2 Barcode Generation

Implement SSCC-18 generation (GS1)

Implement QR code generation with Verifiable Credential

Implement barcode storage

Implement barcode print job generation

5.11.10.3 Reprint Policy

Implement reprint request endpoint (POST /v1/packing/{id}/reprint)

Implement Governor decision for reprint

Implement reprint approval workflow

5.11.10.4 Testing

Unit tests: lock validation

Unit tests: SSCC generation

Unit tests: Loom hashing

Integration tests: lock → barcode generation

Integration tests: reprint workflow

End-to-end tests: full lock workflow

5.11.11 AI Authority Summary for Part 5.11

END OF PART 5.11 — LOCKING THE PACKING PLAN & BARCODE GENERATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

5.12 Label Print Workflow

5.12.1 Printer Compatibility

Two print paths:

5.12.2 Label Customisation

The seller can customise label content before printing:

Select language for human-readable text (Groq translates product name and handling instructions).

Choose which data fields to display (e.g., hide lot number, show buyer name, show cold treatment status, show layer breakdown).

Predefined templates: "Standard", "Customs-Ready" (includes HS code and origin in large font), "Consignee" (includes delivery address).

5.12.3 Reprint Policy (Governor-Enforced)

One-click: Print (1). Reprint request (1) → Governor approval (if auto-approved) → reprint.

AI Authority: A4 for Governor enforcement.

5.13 Implementation Checklist (Part 5)

5.13.1 Weight Calculation & Packing

Implement weight calculation engine (Rust) in packing-service.

Integrate ORTools for palletisation solver (Python subprocess or Rust binding).

Extend solver to support non-uniform layer patterns.

Build 3D container viewer (Three.js) with capacity heatmap toggle.

Implement collaborative editing (Yjs + WebRTC) with OPA permission checks.

5.13.2 Document Generation

Create packing list PDF generator (Tera + headless Chrome) with SSCC and QR codes.

Implement layer breakdown display in packing list.

Implement automated invoice generation (UBL 2.1).

Add ETA submission with mTLS signing.

Build customs declaration pre-filler (Nafeza SAD) from packing and invoice data.

5.13.3 Sustainability & Compliance

Implement ecological packaging advisor (call Groq/Ollama with material DB).

Build carbon footprint calculator (LSTM model for vessel emissions; use IEA factors).

Create CBAM report generator (XML template).

Implement PDF/A-3 archival format for all generated documents.

5.13.4 Barcode & Printing

Implement SSCC-18 generation (GS1 standard).

Generate QR codes with verifiable credentials (ZK proofs).

Create label print workflow (ZPL and PDF).

Implement reprint policy with Governor enforcement.

5.13.5 Lock & Governance

Implement packing plan lock with Governor validation.

Add WasmEdge compatibility checks.

Create Loom hash for immutable packing plan.

5.13.6 Integration Tests

Seller locks packing plan (with non-uniform layers) → SSCCs generated → packing list PDF (with layer breakdown) → invoice generated → ETA submission → Nafeza SAD pre-filled.

Test carbon footprint calculation for EU-bound shipment.

Test ecological packaging advisor suggestion and apply.

Test PDF/A-3 generation and validation.

Test reprint policy (pre-loading vs post-loading).

5.14 AI Authority Summary for Part 5

END OF PART 5 — WEIGHT CALCULATION, PACKING LIST & INVOICE GENERATION (v11.0 FULLY EXPANDED)

PART 6 — ONE-CLICK PAYMENT ORCHESTRATION & GOVERNMENT FEE COLLECTION

6.0 Purpose & Design Philosophy

The One-Click Payment Orchestration module is the financial engine of SGTX. It ensures that every mandatory payment — SGTX platform fee (1.5% of invoice value, paid by each country's side), government duties, inspection fees, certificate fees, port charges, and logistics costs — is aggregated into a single payment at the appropriate stage, then automatically split to the respective recipients (government agencies, SGTX, laboratories, customs brokers, trucking companies). The seller's side (or buyer's side depending on incoterm) pays once; the platform instructs the PSP to split the payment according to the pre-defined allocation.

Two-stage model (based on export process):

One-click core principle: The user sees one payment button for Stage 1, and one for Stage 2. Behind the scenes, the system calculates the exact amounts due to each payee, generates the split instructions, calls the PSP API, and verifies all webhooks. No separate payments for customs, labs, or certificates.

Governmental API integration (orchestration):

Non-custodial: SGTX never holds funds. The PSP splits the payment in real time and sends the net amounts directly to each payee's bank account. SGTX only receives webhook confirmations.

AI Integration in Part 6:

6.1 Stage 1 — Pre-shipment Payment (Seller's Side Pays)

6.1.1 Components of Stage 1 (Export Example)

For an agricultural export, Stage 1 includes (illustrative amounts based on a \$100,000 invoice, 1.5% SGTX fee = \$1,500):

Total Stage 1: ~\$2,845

Important: All these fees are automatically calculated from the USTN data and service quotations. The seller does not need to know individual amounts; they see a single total and click "Pay Stage 1". The system then orchestrates the split and the government API calls in the correct order.

6.1.2 Correct Order of Operations (Orchestration)

When the seller clicks "Pay Stage 1" , the following sequence occurs automatically (no user intervention):

text

■ 1. Governor validates that all mandatory conditions are met ■

■ (contract locked, all service quotes accepted, packing plan locked) ■



■ 2. PSP Router (A2, LightGBM + Groq) selects optimal PSP ■

■ (e.g., Fawry for EGP, Stripe for USD) and explains the choice ■

▼

■ 3. SGTX generates a split payment instruction with the exact amounts ■

■ for each payee. ■

json

"ustn": "SGTX-EG-26-F3A-1",

"gtid": "SGTX-EG-TRD-002139-7F3A",

"splits": [

]

}

PSP Processing: The PSP processes the splits and sends webhooks for each leg. SGTX verifies the webhooks and updates the status of each fee_payment_request.

6.1.4 Government API Calls After Payment

Once the payment webhook is verified, SGTX automatically calls:

Result: The exporter does not need to separately log into Nafeza or CargoX — everything is orchestrated automatically after the one-click payment.

6.2 Stage 2 — Post-departure Payment (Seller or Buyer Pays)

6.2.1 Components of Stage 2

Stage 2 occurs after the vessel has departed. It typically includes:

Important distinctions:

Ocean freight is often on credit terms (30—60 days). The shipping line invoice is generated after departure, and the seller has a due date. The container can be released at the destination even if the freight is unpaid (CREDIT condition).

Destination charges (import duties, VAT, destination THC) are paid by the buyer (importer) in a separate payment batch (import side). This is also orchestrated as a single payment for the buyer.

6.2.2 Split Instruction for Stage 2 (Seller Pays Freight)

json

POST /v1/payment/psp/split

{

"ustn": "SGTX-EG-26-F3A-1",

"stage": "STAGE2",

"total_amount": 4200.00,

"currency": "USD",

"payer": {

"gtid": "SGTX-EG-TRD-002139-7F3A",

"bank_account": "..."

},

"splits": [

{

"payee_gtid": "SGTX-EG-SHIP-001",

"amount": 4200.00,

"description": "Ocean freight",

"terms": "CREDIT",

"due_date": "2026-05-15"

}

}

6.2.3 Buyer's Import Payment Batch (Single Payment)

Total: \$26,400

6.3 One-Click Core — Complete Sequence Diagram (Textual)

SELLER SGTX PSP GOVERNMENT APIs

■ 1. Click "Pay Stage 1" ■ ■ ■

■ ■ 2. Governor validates ■ ■

■ ■ (FeeLock not active) ■ ■

■ ■ 3. PSP Router selects ■ ■

■ ■ optimal PSP ■ ■

■ ■ 4. Create split ■ ■

■ ■ instruction ■ ■

4444

■ 5. Redirect to PSP ■ ■ ■

■ hosted payment page ■ ■ ■

4444

■ 6. Authenticate & pay ■ ■ ■

4444

7. PSP processes split

■ ■ — Customs receives ■ ■

■ ■ \$200 ■ ■

■ ■ — Lab receives \$200 ■ ■

■ ■ — SGTx receives ■ ■

■ ■ \$1,500, etc. ■ ■

■ ■ 8. Webhooks sent ■ ■

6.5 PSP Router & Fallback for Government Fees

6.5.1 PSP Router Logic (A2, LightGBM + Groq)

The PSP Router selects the optimal PSP for each payment based on:

Payer country (Egypt for Stage 1, etc.)

Payee countries (mix of Egypt and international)

Amount, currency

Realtime PSP health (uptime, latency, error rate)

Cost (fees + FX spread)

Example for Stage 1 (all payees have Egyptian bank accounts except SGTX which may be in USD):

Groq Explanation (A1) displayed to user:

"We recommend Fawry for this payment. Estimated fees: \$12.50. Settlement in 1-2 days. If you prefer another method, select below."

6.5.2 Automatic Fallback

If the primary PSP fails (webhook timeout, 5xx error), the router automatically retries with the next PSP. The user is notified via Smart Inbox but does not need to re-authenticate.

Fallback Chain (per country):

Health Monitoring: The psp-health-monitor (Rust) pings each PSP's health endpoint every 30 seconds. If health score <80 → marked DEGRADED; <50 → disabled; automatic fallback triggered.

6.6 FeeLock State Machine

6.6.1 FeeLock States

6.6.2 FeeLock Implementation (NATS JetStream KV)

The FeeLock is stored in NATS JetStream KV and referenced by the Container Release API.

KV Key: feeflock:{ustn}

Value:

json

```
{
  "lock_id": "fl-abc123",
  "ustn": "SGTX-EG-26-F3A-1",
  "status": "ACTIVE",
  "fee_usd": 1500.00,
  "settled_at": "2026-04-15T13:00:00Z",
  "version": 3
}
```

6.6.3 Dispute Impact on Payments

6.7 Multi-Shipment Contracts — Per-Shipment Stage 1 & Stage 2

For multi-shipment contracts:

Master contract is signed with no upfront payment.

Each shipment activates independently:

This allows flexible scheduling — the seller does not pay for all shipments upfront.

Example — Three Shipments:

text

Shipment 1: Locked → Stage 1 paid → USTN generated → Executed

Shipment 2: Locked → Stage 1 paid → USTN generated → Executed

Shipment 3: Scheduled (not yet activated)

Each shipment's FeeLock is independent. A delay in one shipment does not affect others.

6.8 Deferred Payment Guarantee Expiry Handling (Improvement #7)

6.8.1 Deferred Payment Workflow (Review)

During Stage 1 payment, payer toggles "Defer" for eligible fees (e.g., import duties) based on jurisdiction rules (from RIA).

System creates a guarantee (PSP hold or letter of credit reference) with a defined expiry date (based on jurisdiction's legal maximum, default 30 days).

The guarantee is tracked in `fee_payment_requests` with `deferred = true` and `deferred_status = 'GUARANTEE_HELD'`.

6.8.2 Expiry Handling — Three-Step Escalation

6.8.3 Payment Conversion Option

Payer can click "Convert to Immediate Payment" on the alert (one click).

The system calculates the amount, charges the PSP, and releases the guarantee.

After payment, `deferred_status` becomes PAID and the fee is marked as settled.

One-click guarantee: Payer can convert to immediate payment with one click. Auto-charge requires prior authorisation (checkbox during Stage 1: "Authorise automatic charge on expiry").

Database Additions:

sql

```
ALTER TABLE fee_payment_requests ADD COLUMN auto_charge_authorised BOOLEAN DEFAULT false;
```

```
ALTER TABLE fee_payment_requests ADD COLUMN expiry_action_taken TEXT; -- 'reminded', 'alerted', 'expired_charged', 'expired_blocked'
```

```
ALTER TABLE fee_payment_requests ADD COLUMN deferred_status TEXT DEFAULT 'GUARANTEE_HELD'; -- 'GUARANTEE_HELD', 'RELEASED', 'PAID', 'EXPIRED'
```

6.9 Late Payment & Automatic Late Fees

6.9.1 Commission Payment Terms (SGTX Trade Fee)

Fee due within 7 days of contract lock OR within 24 hours of loading confirmation (whichever earlier).

For multi-shipment, each shipment's fee follows the same rule independently.

6.9.2 Late Fee Calculation (A4)

0.1% of unpaid fee per full day of delay, capped at 100% of original fee.

Daily cron job (late-fee-calculator, Rust) checks `fee_payment_requests` where `status = 'PENDING'` and `due_date < NOW()`.

Updates `fee_payment_requests.late_fee_accrued` and adds a record to `late_fee_events`.

Seller receives Smart Inbox reminders (priority 90) on the first day of delay, and daily reminders thereafter.

The late fee is collected at the same time as the original fee (when the seller eventually pays). The PSP split instruction includes both the original fee and the accrued late fee.

Example:

SGTX trade fee = \$360. Due date = 20 June.

Seller pays on 23 June (3 days late). Late fee = $\$360 \times 0.1\% \times 3 = \1.08 . Total due = \$361.08.

6.9.3 Container Loading Trigger

If the container is loaded before the 7-day deadline:

The loading milestone is confirmed → system recalculates due date = loading confirmation timestamp + 24 hours.

Updates `fee_payment_requests.due_date` and sends Smart Inbox alert: "Container loaded. SGTX fee of \$360 is now due within 24 hours (by 22 June 14:00 UTC)."

Governance (G3U7 — enhanced): Governor checks that the fee payment is completed before the USTN is generated. If the due date passes without payment, the Governor blocks any further milestones (loading, departure, etc.) until payment is made.

6.10 Reconciliation & Dispute Handling

6.10.1 Reconciliation Engine (A2, HF Donut)

The Reconciliation Engine continuously matches incoming payment confirmations against open settlement instructions. For each PSP webhook or bank statement line:

Extract amount, currency, value date, reference (USTN pattern).

Compare with `settlement_instructions` that are `AUTHORISED`.

If match confidence $\geq 95\%$ (amount within tolerance, USTN matches, payer/payee consistent), the settlement is autoreconciled. Status → `RECONCILED`.

If confidence $< 95\%$ or no match found, a Smart Inbox item is created for the finance team: "Unmatched payment of \$X — please review and assign to USTN."

OpenBanking Integration:

For PSPs that support PSD2 / OpenBanking APIs, the settlement-service pulls SGTX's bank statement periodically.

HF Donut extracts each transaction, matches against open `fee_payment_requests` using amount, reference, date.

If confidence $\geq 95\%$, auto-reconciled. Unmatched items trigger Smart Inbox alert for manual review.

6.10.2 Dispute Handling

If a dispute is filed:

The FeeLock is frozen (no further releases).

Government fees already paid are not refundable.

The dispute resolution may result in a refund from the seller to the buyer via a separate PSP split.

6.11 Licensed PSP Responsibility Matrix

6.11.1 Purpose

Explicitly separate SGTX's non-custodial role from the licensed PSP's regulated activities. Required for CBE compliance and legal clarity.

6.11.2 SGTX SHALL NOT (Non-Custodial Principle)

Enforcement: Governor blocks any action that would cause SGTX to hold funds. FeeLock is a NATS KV instruction, not a wallet.

6.11.3 Licensed PSP SHALL (Regulated Activities)

6.11.4 Responsibility Matrix

6.11.5 Legal Disclaimer (Displayed to Users)

"SGTX is not a bank or payment service provider. All funds are held and transferred by licensed PSPs (e.g., Fawry, PayMob, CBE IPN). SGTX only provides instructions and reconciliation data. Your relationship with the PSP is separate and governed by their terms."

Display location: During payment setup, in the PSP selection dropdown, and in the footer of every payment confirmation.

6.12 Idempotency Key Standard for External Calls

6.12.1 Purpose

Prevent duplicate external API calls (e.g., double declaration submissions, double payment charges) by enforcing a standard idempotency key for all outgoing requests.

6.12.2 Format

text

`idempotency_key = SHA256( request_body_canonical + timestamp_utc_rounded_to_second )`

Where:

`request_body_canonical` is the JSON request body serialised with JCS (RFC 8785) — same canonicalisation used for signatures.

`timestamp_utc_rounded_to_second` is the current UTC time truncated to the nearest second (ISO 8601 format: YYYY-MM-DDTHH:MM:SSZ).

Example:

text

`request_body_canonical = '{"ustn":"SGTX-EG-26-F3A-1","container":"MEDU1234567"}'`

`timestamp = '2026-06-10T14:35:22Z'`

`idempotency_key = SHA256('{"ustn":"SGTX-EG-26-F3A-1","container":"MEDU1234567"}' + '2026-06-10T14:35:22Z')`
`= 'a1b2c3d4e5f6...'`

6.12.3 Usage

For every external API call (Nafeza, CargoX, ETA, PSPs, etc.), SGTX includes the idempotency key in the request headers:

text

X-Idempotency-Key: a1b2c3d4e5f6...

6.12.4 Retry Behaviour

If the external API returns a 5xx error, SGTX retries with the same idempotency key.

The external API (or SGTX's own wrapper) must store the key and response for at least 24 hours, returning the same response on duplicate keys (idempotent behaviour).

For PSPs that do not support idempotency keys, SGTX adds a unique UUID in the payment reference field as a fallback.

Storage: Every idempotency key used is logged in `integration_connector_logs` with the request and response, for audit and debugging.

Enforcement: Governor rejects any attempt to replay a request with a duplicate key within 24 hours unless the original request failed with a non-retryable error.

6.13 Error Handling & Retry Policies

All errors are logged in `integration_connector_logs` with request/response bodies (sanitised).

AI Authority: A2 for error analysis; A1 for Groq-generated error summaries.

6.14 Database Schema Additions (Part 6)

sql

-- Fee payment requests (core)

```
CREATE TABLE fee_payment_requests (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,  
  contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,  
  financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,  
  party_type TEXT NOT NULL, -- 'seller', 'buyer', 'borrower'  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  amount DECIMAL(20,4) NOT NULL,  
  currency TEXT DEFAULT 'USD',  
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'PAID', 'FAILED', 'REFUNDED'  
  due_date TIMESTAMPTZ,  
  late_fee_accrued DECIMAL(20,4) DEFAULT 0,  
  deferred BOOLEAN DEFAULT false,  
  deferred_status TEXT DEFAULT 'GUARANTEE_HELD', -- 'GUARANTEE_HELD', 'RELEASED', 'PAID',  
  'EXPIRED'  
  auto_charge_authorised BOOLEAN DEFAULT false,  
  expiry_action_taken TEXT, -- 'reminded', 'alerted', 'expired_charged', 'expired_blocked'  
  payment_reference TEXT,  
  gross_amount DECIMAL(20,4),  
  psp_fee DECIMAL(20,4),
```

```

    psp_transaction_id TEXT,
    paid_at TIMESTAMPTZ,
    governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- FeeLock state (NATS KV is the source of truth, but we also store a record)
CREATE TABLE fee_locks (
    lock_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
    contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,
    financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,
    ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
    fee_rate_pct DECIMAL(5,4) NOT NULL,
    fee_usd DECIMAL(20,4) NOT NULL,
    status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACTIVE', 'PARTIALLY_RELEASED', 'DISPUTED',
    'CANCELLED'
    kv_version BIGINT,
    governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
    locked_at TIMESTAMPTZ DEFAULT NOW(),
    released_at TIMESTAMPTZ,
    fully_released_at TIMESTAMPTZ
);

-- Payment attempts (PSP interactions)
CREATE TABLE payment_attempts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    fee_payment_request_id UUID NOT NULL REFERENCES fee_payment_requests(id) ON DELETE
    CASCADE,
    aggregator_id UUID NOT NULL REFERENCES payment_aggregators(id) ON DELETE CASCADE,
    payer_country TEXT NOT NULL,
    payment_method TEXT NOT NULL,
    amount_local DECIMAL(20,4),
    currency_local TEXT,
    amount_usd DECIMAL(20,4),
    fx_rate DECIMAL(16,8),
    gross_amount DECIMAL(20,4),

```



```

net_amount DECIMAL(20,4),
status TEXT DEFAULT 'INITIATED', -- 'INITIATED', 'SUCCEEDED', 'FAILED', 'REFUNDED'
failure_reason TEXT,
retry_count INTEGER DEFAULT 0,
psp_transaction_id TEXT,
split_executed_at TIMESTAMPTZ,
settled_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Fee calculations (gross-up)
CREATE TABLE fee_calculations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
payment_attempt_id UUID REFERENCES payment_attempts(id) ON DELETE SET NULL,
net_fee DECIMAL(20,4) NOT NULL,
gross_amount DECIMAL(20,4) NOT NULL,
fee_breakdown JSONB,
safety_buffer_applied BOOLEAN DEFAULT true,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- PSP health logs
CREATE TABLE psp_health_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
aggregator_id UUID NOT NULL REFERENCES payment_aggregators(id) ON DELETE CASCADE,
health_score DECIMAL(5,2),
latency_ms INTEGER,
error_rate DECIMAL(5,4),
checked_at TIMESTAMPTZ DEFAULT NOW()
);

-- Late fee events
CREATE TABLE late_fee_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
fee_payment_request_id UUID NOT NULL REFERENCES fee_payment_requests(id) ON DELETE CASCADE,
day_number INTEGER NOT NULL,

```

```

late_fee_amount DECIMAL(20,4) NOT NULL,
total_accrued DECIMAL(20,4) NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Integration connector logs
CREATE TABLE integration_connector_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
connector_name TEXT NOT NULL, -- 'NAFEZA', 'CARGOX', 'ETA', 'PSP_FAWRY', etc.
endpoint TEXT NOT NULL,
request_body TEXT,
response_body TEXT,
status_code INTEGER,
idempotency_key TEXT,
duration_ms INTEGER,
success BOOLEAN,
error_message TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Payment aggregators (PSPs)
CREATE TABLE payment_aggregators (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name TEXT NOT NULL UNIQUE,
country_codes TEXT[] NOT NULL,
supported_currencies TEXT[],
supports_split BOOLEAN DEFAULT true,
api_endpoint TEXT,
credentials_encrypted JSONB,
uptime_score DECIMAL(5,4),
last_health_check TIMESTAMPTZ,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Settlement instructions
CREATE TABLE settlement_instructions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
instruction_type TEXT NOT NULL, -- 'TRADE_PRINCIPAL', 'FEE_SPLIT', 'REFUND'
payload JSONB NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'AUTHORISED', 'CONFIRMED', 'RECONCILED',
'FAILED'
psp_reference TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
confirmed_at TIMESTAMPTZ,
reconciled_at TIMESTAMPTZ
);

-- Settlement confirmations (webhooks)
CREATE TABLE settlement_confirmations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
instruction_id UUID NOT NULL REFERENCES settlement_instructions(id) ON DELETE CASCADE,
proof_hash TEXT NOT NULL,
amount_confirmed DECIMAL(20,4),
currency_confirmed TEXT,
fx_rate_applied DECIMAL(16,8),
psp_name TEXT,
webhook_received_at TIMESTAMPTZ,
verified_by_ai BOOLEAN DEFAULT false,
ai_verdict JSONB,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
confirmed_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_fee_payment_requests_contract ON fee_payment_requests(contract_id);
CREATE INDEX idx_fee_payment_requests_shipment ON fee_payment_requests(contract_shipment_id);
CREATE INDEX idx_fee_payment_requests_tenant ON fee_payment_requests(tenant_id);
CREATE INDEX idx_fee_payment_requests_status ON fee_payment_requests(status);
CREATE INDEX idx_fee_payment_requests_deferred ON fee_payment_requests(deferred_status)
WHERE deferred = true;
CREATE INDEX idx_fee_locks_ustn ON fee_locks(ustn);
CREATE INDEX idx_fee_locks_status ON fee_locks(status);
CREATE INDEX idx_payment_attempts_fee ON payment_attempts(fee_payment_request_id);

```

```
CREATE INDEX idx_payment_attempts_psp ON payment_attempts(agggregator_id);
CREATE INDEX idx_psp_health_logs_agggregator ON psp_health_logs(agggregator_id);
CREATE INDEX idx_late_fee_events_request ON late_fee_events(fee_payment_request_id);
CREATE INDEX idx_integration_connector_logs_name ON integration_connector_logs(connector_name);
CREATE INDEX idx_integration_connector_logs_created ON integration_connector_logs(created_at
DESC);
CREATE INDEX idx_settlement_instructions_ustn ON settlement_instructions(ustn);
CREATE INDEX idx_settlement_instructions_status ON settlement_instructions(status);
CREATE INDEX idx_settlement_confirmations_instruction ON settlement_confirmations(instruction_id);
```

6.15 Implementation Checklist (Part 6)

6.15.1 Core Payment Infrastructure

- Implement payment-orchestrator service (Rust).
- Build PSP Router (LightGBM + Groq) with real-time health scoring.
- Integrate with PSPs: Fawry, PayMob, CBE IPN for split payments.
- Add webhook handlers for each PSP, verifying signatures.
- Implement FeeLock state machine in NATS JetStream KV.
- Build split instruction generator that aggregates all Stage 1 payees.
- Implement gross-up calculation with AI-validated fee schedules.

6.15.2 Stage 1 & Stage 2

- Implement Stage 1 payment flow (aggregation, split, webhook verification).
- Implement Stage 2 payment flow (freight, destination charges).
- Add per-shipment Stage 1/Stage 2 for multi-shipment contracts.
- Implement import payment batch (buyer side).

6.15.3 Government Integration

- Automate CargoX API calls after payment (ACID generation).
- Automate Nafeza API calls after payment (SAD submission, certificate requests).
- Automate ETA API calls (invoice submission, UUID/QR retrieval).
- Implement retry policies with exponential backoff.
- Add idempotency key standard for all external calls.

6.15.4 Deferred Payment (Improvement #7)

- Add deferred and deferred_status columns to fee_payment_requests.
- Implement guarantee creation during Stage 1.
- Build three-step expiry escalation (reminder, alert, expiry).
- Add "Convert to Immediate Payment" one-click action.
- Implement auto-charge authorisation checkbox.

6.15.5 Late Fees & Reconciliation

Implement late fee calculator cron job (Rust, daily).

Add Smart Inbox reminders for late fees.

Build Reconciliation Engine (HF Donut) for bank statement extraction.

Implement OpenBanking integration (PSD2) where available.

Add auto-reconciliation for matched payments.

Create manual reconciliation queue for unmatched items.

6.15.6 FeeLock & Dispute

Implement FeeLock freeze on dispute filing.

Add FeeLock release on dispute resolution.

Ensure government fees are non-refundable.

6.15.7 Compliance & Legal

Add PSP Responsibility Matrix to documentation.

Display legal disclaimer during payment setup.

Ensure CBE compliance for PSP selection.

6.15.8 Integration Tests

Test end-to-end: seller clicks "Pay Stage 1" → PSP split → webhook → CargoX ACID → Nafeza SAD → certificate requests.

Test multi-shipment per-shipment Stage 1 payments.

Test deferred payment scenario: guarantee created → customs clearance missed → expiry → auto-charge (or block).

Test late fee accrual and collection.

Test PSP fallback (primary fails → secondary PSP).

Test reconciliation with OpenBanking mock.

Test FeeLock freeze on dispute.

6.16 AI Authority Summary for Part 6

END OF PART 6 — ONE-CLICK PAYMENT ORCHESTRATION & GOVERNMENT FEE COLLECTION (v11.0 FULLY EXPANDED)

PART 7 — GOVERNMENT INTEGRATION — ORCHESTRATION (NAFEZA, CARGOX, ETA, CBE, DIRECT BANK SETTLEMENT)

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

7.0 Purpose & Design Philosophy

SGTX is not a replacement for government systems — it is an orchestration layer that automates data submission to the four mandatory government APIs: Nafeza (Customs Single Window), CargoX (ACI Blockchain Platform), ETA (e-Invoice), and Central Bank of Egypt (CBE) (payment & FX). Additionally, SGTx enables direct bank integration for trade principal settlement, allowing banks to reconcile payments without SGTx ever holding funds.

Key Principles:

Non-custodial by design — SGTX never holds trade principal or fee funds. It only instructs PSPs or provides reconciliation files to banks.

One-click orchestration — After the user clicks "Pay Stage 1" or "Settle Payment", SGTX triggers all government API calls in the correct sequence, based on payment confirmation webhooks.

Bank-friendly reconciliation — SGTX generates USTN-tagged settlement instructions and provides APIs for banks to pull reconciliation data (CSV, MT940, ISO 20022). Banks do not need to integrate with each government API — they only need to receive instructions from SGTX (or from PSPs) and confirm settlements.

No new licence required — SGTX is not a bank; it merely provides data. The actual movement of funds occurs between bank accounts via PSPs or direct bank transfers.

Government API coverage:

All integrations use mutual TLS (mTLS) with Egypt Trust certificates. No third-party middleware.

AI Integration in Part 7:

7.1 One-Click Trigger Map — Complete

The following table maps each user action to the automated government API calls triggered by SGTX, with the payment confirmation acting as the trigger for downstream calls.

One-click guarantee: The user never logs into Nafeza, CargoX, or ETA. All government interactions are triggered automatically after the single payment approval.

7.2 Nafeza Integration (Customs Single Window)

7.2.1 Overview

Nafeza is the national single window for trade. SGTX submits the Single Administrative Document (SAD) and requests ancillary certificates (phytosanitary, health, certificate of origin, EUR.1). SGTX can file directly using:

Exporter's delegated authority — exporter authorises SGTX to file on their behalf (consent stored in tenant profile).

SGTX's own broker license — platform acts as a licensed customs broker.

Optional broker certification — if the seller has selected a customs broker for certification, the broker reviews the AI-generated SAD and clicks "Certify"; then SGTX submits the SAD under the broker's digital seal.

Legal basis: Egyptian Customs Law No. 207 of 2020, Article 51.

7.2.2 API Endpoint & Authentication

One-click integration: No user action — certificate management is automatic.

7.2.3 Request Schema (Export SAD — Simplified)

json

POST /api/v2/declaration

{

"declaration_type": "EXPORT",

"trader_gtid": "SGTX-EG-TRD-002139-7F3A",

"broker_gtid": "SGTX-EG-CBR-001",

```

"ustn": "SGTX-EG-26-F3A-1",
"acid": "ACI20260415-0001",
"invoice": {
  "number": "INV-STR-2026-001",
  "value": 105753.00,
  "currency": "USD",
  "eta_uuid": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
},
"goods": [
  {
    "hs_code": "0805.10",
    "description": "Fresh oranges",
    "net_weight_kg": 8800,
    "gross_weight_kg": 9790,
    "container_number": "MEDU1234567",
    "packages": 880,
    "package_type": "CARTON"
  }
],
"certificate_requests": [
  { "type": "PHYTOSANITARY", "ref_lab_report": "USTN:SGTX-....:LAB-001" },
  { "type": "HEALTH", "ref_lab_report": "USTN:SGTX-....:LAB-001" },
  { "type": "CERTIFICATE_OF_ORIGIN" },
  { "type": "EUR1" }
],
"transport": {
  "incoterm": "CFR",
  "port_of_loading": "EGDAM",
  "port_of_discharge": "DEHAM",
  "vessel_name": "MSC FIONA"
}
}

```

Timing: This call is made after the Stage 1 payment webhook is confirmed (because customs fees are part of Stage 1). Nafeza requires proof of fee payment — the PSP split confirmation serves as that proof.

7.2.4 Response Schema

Success (202 Accepted — asynchronous processing):

json

```
{
  "declaration_id": "SAD20260415-000147",
  "status": "ACCEPTED",
  "estimated_processing_time": "4 hours",
  "certificate_requests": [
    { "type": "PHYTOSANITARY", "status": "PENDING", "request_id": "PHYTO-REQ-147" },
    { "type": "HEALTH", "status": "PENDING", "request_id": "HEALTH-REQ-147" },
    { "type": "COO", "status": "PENDING", "request_id": "COO-REQ-147" }
  ]
}
```

Polling: SGTX polls GET /api/v2/declaration/{declaration_id} every 30 minutes or receives a webhook (if Nafeza supports). When certificates are issued, SGTX downloads the PDFs and stores them in documents with a SHA256 hash.

7.2.5 Certificate Issuance via Lab Results

When a laboratory submits compliant test results, SGTX automatically triggers the certificate request:

json

POST /api/v2/declaration/{declaration_id}/certificates

```
{
  "type": "PHYTOSANITARY",
  "lab_report_ref": "USTN:SGTX-...:LAB-001"
}
```

Nafeza issues the certificate (fee already covered in Stage 1). SGTX downloads and attaches it to the USTN.

7.3 CargoX Integration (ACI Blockchain)

7.3.1 Overview

CargoX is mandatory for Advance Cargo Information (ACI). SGTX submits the shipment envelope and receives an ACID. The ACID is required for Nafeza declaration.

Legal basis: Egyptian Customs Law No. 207/2020, Article 51 (ACI requirement).

7.3.2 API Endpoint & Authentication

7.3.3 Request Schema

json

```
{
  "external_reference": "SGTX-EG-26-F3A-1",
  "shipper": {
    "tax_id": "123-456-789",
    "name": "Strawberry Export Co.",

```



```

"country": "EG"
},
"consignee": {
  "tax_id": "DE123456789",
  "name": "European Importer GmbH",
  "country": "DE"
},
"goods_value": {
  "amount": 100000.00,
  "currency": "USD"
},
"container_numbers": ["MEDU1234567"],
"documents": [
  { "type": "INVOICE", "content_base64": "...", "filename": "invoice.pdf" },
  { "type": "PACKING_LIST", "content_base64": "...", "filename": "packing.pdf" }
]
}

```

7.3.4 Response Schema

json

```

{
  "acid": "ACI20260415-0001",
  "status": "ISSUED",
  "blockchain_seal": "0xabc123..."
}

```

The ACID is stored in documents.acid and used in the Nafeza declaration.

7.3.5 Payment Integration

The CargoX fee is included in the Stage 1 split (payee CARGOX). After the PSP webhook confirms that CargoX has been paid, SGTX calls the CargoX API. The ACID is then used for Nafeza.

7.4 ETA Integration (e-Invoice)

7.4.1 Overview

The Egyptian Tax Authority (ETA) e-Invoice system requires every B2B invoice to be submitted in real time. SGTX submits the UBL 2.1 XML invoice (generated in Part 5.6) and receives a UUID and QR code.

Legal basis: Egyptian Tax Law No. 91/2005, ETA e-Invoice regulation.

7.4.2 API Endpoint & Authentication

7.4.3 Request Body

The canonicalised, signed XML invoice (as defined in Part 5.6). No additional wrapper.

Example (condensed):

xml

```
<Invoice xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2">
<cbc:ID>INV-STR-2026-001</cbc:ID>
<cbc:IssueDate>2026-04-15</cbc:IssueDate>
<cbc:InvoiceTypeCode>388</cbc:InvoiceTypeCode>
<!-- ... full XML structure ... -->
</Invoice>
```

7.4.4 Response Schema

json

```
{
  "uuid": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "qrCode": "iVBORw0KGgoAAAANSUUEUgAA...",
  "status": "VALID"
}
```

The UUID and QR code are stored in documents.invoice_uuid and documents.invoice_qr.

7.4.5 Timing

The invoice is submitted immediately after the buyer accepts the quote (before Stage 1 payment). ETA does not require prepayment; it only requires the seller's digital signature.

7.5 Direct Bank Integration — Non-Custodial Settlement

7.5.1 Philosophy

SGTX is not a payment service provider and never holds funds. For trade principal settlement (buyer → seller), SGTX can either:

Use PSPs (Fawry, PayMob, CBE IPN) with split capabilities, or

Provide bank-to-bank settlement instructions directly. Banks integrate with SGTX via API to receive reconciliation data, not to move funds.

How it works:

Buyer confirms delivery → SGTX generates a settlement instruction containing buyer IBAN, seller IBAN, amount, currency, USTN.

The buyer's bank (or the buyer themselves) initiates the transfer (via their online banking, not via SGTX).

The buyer's bank sends a payment confirmation (MT103, ISO 20022 pacs.008) to the seller's bank.

SGTX reconciles the payment by matching the USTN reference from the bank statement. SGTX does not initiate the transfer — the bank does.

Bank onboarding: Any bank can register as a BANK tenant type (sub-type of FIN). They provide their IBAN prefix, BIC, and API endpoint for receiving instructions (optional). SGTX then pushes settlement instructions to the bank's system (if supported) or the bank pulls them.

7.5.2 Bank Settlement Instruction Endpoint (for Banks)

Endpoint: GET /v1/settlement/instructions?bank_bic=...&status=PENDING

Authentication: mTLS with bank's certificate.

Response:

json

```
{
  "instructions": [
    {
      "instruction_id": "SI-20260415-001",
      "ustn": "SGTX-EG-26-F3A-1",
      "from_iban": "DE12345678901234567890",
      "to_iban": "EG380010000000000000123456",
      "amount": 105753.00,
      "currency": "USD",
      "value_date": "2026-04-20",
      "reference": "SGTX USTN SGTX-EG-26-F3A-1"
    }
  ]
}
```

The buyer's bank can use this information to pre-fill a payment order for the buyer. The buyer still authorises the payment through their normal banking channel (no SGTX involvement).

One-click for bank: Bank calls GET /v1/settlement/instructions (one API call) → receives all pending instructions.

7.5.3 Payment Confirmation via Bank API

After the transfer is executed, the bank can notify SGTX:

json

POST /v1/settlement/confirm

```
{
  "instruction_id": "SI-20260415-001",
  "ustn": "SGTX-EG-26-F3A-1",
  "amount": 105753.00,
  "transaction_reference": "MT103-REF-12345",
  "settled_at": "2026-04-20T14:30:00Z",
  "bank_bic": "CIBEEG CX"
}
```

SGTX verifies that the USTN matches an open instruction and marks settlement_instructions.status = 'SETTLED'. SGTX never touches the funds — it only records the confirmation.

7.5.4 Reconciliation Files for Banks

Banks can download daily reconciliation files:

Endpoint: GET /v1/settlement/reconciliation?date=2026-04-20&format=mt940

Returns a standard MT940 statement file (or ISO 20022 camt.053) containing all settlements for that date with USTN as the reference.

Supported formats:

7.6 CBE Payment Orchestration via Licensed PSPs (for Fees)

For SGTX fee collection (Stage 1 and Stage 2 fees), SGTX uses licensed PSPs (Fawry, PayMob, CBE IPN). This is already covered in Part 6. The PSPs are licensed by the CBE and handle the actual funds movement. SGTX only instructs splits.

PSP Integration Summary:

PSP Router: SGTX's PSP router (A2, LightGBM + Groq) selects the optimal PSP based on payer country, amount, and real-time health. The PSP webhooks confirm the split.

7.7 Idempotency Key Standard for External Calls

7.7.1 Purpose

Prevent duplicate external API calls (e.g., double declaration submissions, double payment charges) by enforcing a standard idempotency key for all outgoing requests.

7.7.2 Format

text

idempotency_key = SHA256(request_body_canonical + timestamp_utc_rounded_to_second)

Where:

request_body_canonical is the JSON request body serialised with JCS (RFC 8785).

timestamp_utc_rounded_to_second is the current UTC time truncated to the nearest second (ISO 8601 format: YYYY-MM-DDTHH:MM:SSZ).

Example:

text

request_body_canonical = '{"ustn":"SGTX-EG-26-F3A-1","container":"MEDU1234567"}'

timestamp = '2026-06-10T14:35:22Z'

idempotency_key = SHA256('{"ustn":"SGTX-EG-26-F3A-1","container":"MEDU1234567"}' + '2026-06-10T14:35:22Z')
= 'a1b2c3d4e5f6...'

7.7.3 Usage

For every external API call (Nafeza, CargoX, ETA, PSPs, etc.), SGTX includes the idempotency key in the request headers:

text

X-Idempotency-Key: a1b2c3d4e5f6...

7.7.4 Retry Behaviour

If the external API returns a 5xx error, SGTX retries with the same idempotency key.

The external API (or SGTX's own wrapper) must store the key and response for at least 24 hours, returning the same response on duplicate keys (idempotent behaviour).

For PSPs that do not support idempotency keys, SGTX adds a unique UUID in the payment reference field as a fallback.

7.8 Error Handling & Retry Policies

Exponential Backoff:

All errors are logged in `integration_connector_logs` with request/response bodies (sanitised).

7.9 Security & Compliance

7.9.1 mTLS Certificate Management

Certificate renewal workflow:

System detects expiry (30 days before).

Smart Inbox alert sent to responsible party.

New certificate uploaded via Admin Portal.

Old certificate revoked.

7.9.2 Data Protection

7.9.3 Audit Trail

All external API calls are logged immutably in `integration_connector_logs` with:

Timestamp

Connector name

Endpoint

Idempotency key

Request body (sanitised)

Response body (sanitised)

Status code

Duration

Success/failure

Error message

Retention: 7 years (compliance with AML Law 80/2002).

7.10 Governance Gates (Part 7)

7.11 Database Schema Additions (Part 7)

sql

-- Integration connector logs

CREATE TABLE integration_connector_logs (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

connector_name TEXT NOT NULL, -- 'NAFEZA', 'CARGOX', 'ETA', 'PSP_FAWRY', etc.

endpoint TEXT NOT NULL,

```
request_body TEXT,  
response_body TEXT,  
status_code INTEGER,  
idempotency_key TEXT,  
duration_ms INTEGER,  
success BOOLEAN,  
error_message TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Certificate management

```
CREATE TABLE certificates (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
certificate_type TEXT NOT NULL, -- 'E_SEAL', 'TLS_CLIENT', 'TLS_SERVER'  
certificate_pem TEXT NOT NULL,  
private_key_encrypted TEXT,  
issuer TEXT,  
valid_from TIMESTAMPTZ,  
valid_until TIMESTAMPTZ,  
status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'EXPIRED', 'REVOKED'  
uploaded_by UUID REFERENCES employees(id) ON DELETE SET NULL,  
created_at TIMESTAMPTZ DEFAULT NOW(),  
updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Bank settlement instructions

```
CREATE TABLE bank_settlement_instructions (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,  
from_iban TEXT NOT NULL,  
to_iban TEXT NOT NULL,  
amount DECIMAL(20,4) NOT NULL,  
currency TEXT DEFAULT 'USD',  
value_date DATE NOT NULL,  
reference TEXT NOT NULL,  
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'CONFIRMED', 'RECONCILED', 'FAILED'
```

```

bank_bic TEXT,
transaction_reference TEXT,
confirmed_at TIMESTAMPTZ,
reconciled_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Bank reconciliation files (MT940, ISO 20022)
CREATE TABLE bank_reconciliation_files (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
bank_bic TEXT NOT NULL,
file_date DATE NOT NULL,
format TEXT NOT NULL, -- 'MT940', 'CAMT_053', 'CSV'
file_content TEXT,
file_hash TEXT,
generated_at TIMESTAMPTZ DEFAULT NOW(),
downloaded_by UUID REFERENCES employees(id) ON DELETE SET NULL
);
-- Indexes
CREATE INDEX idx_integration_connector_logs_name ON integration_connector_logs(connector_name);
CREATE INDEX idx_integration_connector_logs_created ON integration_connector_logs(created_at
DESC);
CREATE INDEX idx_integration_connector_logs_idempotency ON
integration_connector_logs(idempotency_key);
CREATE INDEX idx_certificates_tenant ON certificates(tenant_gtid);
CREATE INDEX idx_certificates_valid_until ON certificates(valid_until);
CREATE INDEX idx_bank_settlement_instructions_ustn ON bank_settlement_instructions(ustn);
CREATE INDEX idx_bank_settlement_instructions_bank ON bank_settlement_instructions(bank_bic);
CREATE INDEX idx_bank_settlement_instructions_status ON bank_settlement_instructions(status);
CREATE INDEX idx_bank_reconciliation_files_bank ON bank_reconciliation_files(bank_bic);
CREATE INDEX idx_bank_reconciliation_files_date ON bank_reconciliation_files(file_date);

```

7.12 Implementation Checklist (Part 7)

7.12.1 Nafeza Integration

Implement nafeza-client (Rust) with mTLS and retry logic

Add SAD submission endpoint (POST /api/v2/declaration)

Add certificate request endpoint (POST /api/v2/declaration/{id}/certificates)

Add polling for declaration status (GET /api/v2/declaration/{id})
Implement certificate download and storage (PDF → documents)

7.12.2 CargoX Integration

Implement cargox-client (Rust) with HMAC signing
Add shipment envelope submission (POST /v3/shipments)
Add ACID retrieval and storage
Implement retry policy (3 retries, then queue)

7.12.3 ETA Integration

Implement eta-client (Rust) with XML signing (XAdES)
Add invoice submission (POST /invoice/v1/documents)
Store UUID and QR code in documents
Implement retry policy for ETA failures

7.12.4 Direct Bank Integration

Implement bank-settlement-service (Rust)
Add endpoints:
GET /v1/settlement/instructions (for banks to pull)
POST /v1/settlement/confirm (for banks to confirm)
GET /v1/settlement/reconciliation (MT940, ISO 20022, CSV)
Add bank onboarding (BANK tenant type, IBAN prefix, BIC)
Implement reconciliation file generation (Tera templates)

7.12.5 CBE PSP Orchestration

PSP Router already implemented in Part 6
Add PSP health monitoring for CBE-licensed PSPs
Implement automatic fallback between PSPs

7.12.6 Error Handling & Idempotency

Implement idempotency key standard for all external calls
Add retry policies with exponential backoff
Create integration_connector_logs table
Implement manual fallback procedures for each API

7.12.7 Certificate Management

Implement certificate upload and storage
Add expiry detection and renewal reminders
Integrate with SoftHSM for private key storage
Add mTLS client configuration for each connector

7.12.8 Testing

Write integration tests for Nafeza (sandbox)

Write integration tests for CargoX (sandbox)

Write integration tests for ETA (sandbox)

Test full flow: payment → CargoX ACID → Nafeza SAD → certificate requests

Test bank settlement flow (simulated bank API)

Test retry policies with simulated failures

Test idempotency with duplicate requests

7.13 AI Authority Summary for Part 7

END OF PART 7 — GOVERNMENT INTEGRATION — ORCHESTRATION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

PART 8 — CONTAINER RELEASE AUTHORISATION API

8.0 Purpose & Design Philosophy

The Container Release Authorisation API is the physical enforcement arm of SGTX. It is the technical bridge between the digital world of payments, documents, and compliance checks and the physical world of port gates, cranes, and container trucks. Without this API, SGTX would be an elegant workflow tool. With it, SGTX becomes the gatekeeper of Egyptian trade, empowered by a ministerial decree that states: "No container may exit an Egyptian port without a valid, cryptographically signed release authorisation issued by SGTX OS."

Every terminal operator, shipping line, and port authority in Egypt is legally required to integrate with this API. The integration is simple, stateless, and secure. The terminal sends a query containing a USTN and a container number; SGTX responds with either an AUTHORISED token or a HOLD with detailed reasons. The token is digitally signed with SGTX's Egypt Trust qualified seal, making it impossible to forge and providing a non-repudiable audit trail.

Key principles (non-marketplace, orchestration-only):

Stateless & idempotent — each query returns the current release status based on the latest state (FeeLock, milestone confirmations, dispute status, payment verifications).

Cryptographically signed — every AUTHORISED response includes a digital signature that the terminal verifies.

Time-bound — authorisation tokens are valid for a limited window (default 24 hours) to prevent replay attacks.

Legally binding — ministerial decree gives the API the force of law; terminals that release without authorisation face penalties.

One-click integration — the release status is automatically updated after the Stage 1 payment is confirmed (or after Stage 2 if freight is MANDATORY). No additional user action is required.

AI Integration in Part 8:

This API is largely deterministic and rule-based; AI is used only for advisory purposes (e.g., explaining a HOLD reason in plain language for terminal operators). No AI decision is involved in the authorisation logic itself (A4 for constitutional enforcement).

8.1 Legal Foundation & Mandate

8.1.1 Ministerial Decree

A joint decree from the Ministry of Transport, the Egyptian Maritime Safety Authority, and the Ministry of Trade and Industry establishes the legal force of the release authorisation.

Key provisions:

8.1.2 Integration with FeeLock

The release authorisation is only granted when the FeeLock is ACTIVE. The FeeLock becomes ACTIVE after SGTX confirms the Stage 1 payment webhook (for export) or the import local payment webhook. Thus, the release API directly reflects the payment status.

8.1.3 Legal Disclaimer (Displayed in Terminal Portal)

"This release authorisation is issued by SGTX OS in accordance with Ministerial Decree No. [XXX] of 2026. The authorisation is cryptographically signed and constitutes legal proof of compliance with all payment and regulatory requirements. Any terminal that releases a container without a valid authorisation is subject to penalties under Egyptian law."

AI Authority: None (legal framework, enforced by Governor A4).

8.2 API Design Overview

The release API is deliberately simple. It is a pull-based REST endpoint: the terminal queries SGTX at the moment of gate-out. There is also an optional webhook push for carriers who wish to be notified proactively when a release becomes valid, but the pull is the authoritative check at the gate.

AI Authority: A4 for certificate validation and signature verification (rule-based).

8.3 API Endpoints

8.3.1 Release Authorisation Query (Primary Endpoint)

Endpoint: GET /v1/release/authorization

Query Parameters:

Example Request:

text

```
GET /v1/release/authorization?ustn=SGTX-EG-26-F3A-1&container=MEDU1234567&request_id=123e4567-e89b-12d3-a456-426614174000
```

Authentication: mTLS (client certificate).

Response Status Codes:

Rate Limits:

8.3.2 Release Authorisation Webhook (Optional Push)

Endpoint: POST /v1/release/webhook (configured per terminal/carrier)

SGTX pushes an AUTHORISED event when a container's status changes to releasable (e.g., after Stage 1 payment is confirmed and all mandatory conditions are met). The terminal can use this to pre-prepare release orders, but must still perform a pull query at the gate to ensure the authorisation is still valid.

Webhook Payload:

json

```
{  
  "event": "RELEASE_READY",  
  "ustn": "SGTX-EG-26-F3A-1",
```

```
"container": "MEDU1234567",  
"authorisation_id": "REL-20260415-147-001",  
"valid_until": "2026-04-16T14:31:00Z",  
"timestamp": "2026-04-15T14:31:00Z"  
}
```

Webhook Delivery:

The terminal must respond with 200 OK within 5 seconds.

SGTX retries up to 3 times (exponential backoff: 1s, 2s, 4s).

If all retries fail, an alert is sent to the Platform Governance Authority.

AI Authority: A4 for delivery; A1 for Groq summary if webhook fails.

8.4 Authentication & Security

8.4.1 Mutual TLS (mTLS)

Every terminal and carrier that wishes to query the release API must first register with SGTX. During registration, they generate a Certificate Signing Request (CSR) and submit it to SGTX. SGTX's internal Certificate Authority issues a client certificate that is bound to that specific organisation.

Certificate Fields:

mTLS Handshake:

Server (SGTX) presents its certificate → client validates.

Client (terminal) presents its certificate → SGTX validates:

Certificate is still valid (not expired, not revoked).

Certificate was issued by SGTX.

Organisation has an active status (not suspended).

If mTLS fails, the request is rejected before any business logic is executed.

8.4.2 Digital Signature on Response

Even with mTLS, a terminal needs to prove to auditors that the release authorisation came from SGTX and was not tampered with. Every AUTHORISED response includes a `digital_signature` field. This is a detached PKCS#7 CMS signature (base64-encoded) over the response JSON (with the `digital_signature` field removed and nulled) using SGTX's Egypt Trust qualified signing certificate (stored in HSM).

Signature Verification Process (performed by terminal's gate system):

Remove the `digital_signature` field from the JSON.

Canonicalise the JSON (e.g., RFC 8785 JCS).

Verify the CMS signature using SGTX's public certificate (obtained from Egypt Trust and also distributed out-of-band).

Check that the signing certificate is valid and not revoked.

If verification fails, the terminal must reject the release and alert SGTX immediately.

Tooling Example (terminal side, using OpenSSL):

```
bash
```

```
# Save response to response.json (without digital_signature field)
```

```
# Extract signature to sig.b64
```

```
# Verify
```

```
openssl cms -verify -in response.json -signer sig.b64 -CAfile sgtx-ca.pem -out /dev/null
```

8.4.3 Certificate Revocation List (CRL)

Endpoint: GET /v1/release/crl

SGTX maintains a Certificate Revocation List (CRL). Terminals should check this list periodically (e.g., daily) and reject any certificate that appears on it.

Response:

```
json
```

```
{
  "crl_version": 42,
  "issued_at": "2026-06-01T00:00:00Z",
  "revoked_certificates": [
    { "serial": "1234567890", "revoked_at": "2026-05-15T10:00:00Z", "reason": "COMPROMISED" },
    { "serial": "1234567891", "revoked_at": "2026-05-20T14:30:00Z", "reason": "SUSPENDED" }
  ],
  "signature": "base64..."
}
```

8.5 Response Structures

8.5.1 Full Authorisation — Export Example (All MANDATORY Settled, CREDIT Freight Outstanding)

```
json
```

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "release_status": "AUTHORISED",
  "authorisation_id": "REL-20260415-147-001",
  "issued_at": "2026-04-15T14:31:00Z",
  "valid_until": "2026-04-16T14:31:00Z",
  "mandatory_summary": {
    "total_usd": 2845.00,
    "total_egp": 0.00,
    "settled_usd": 2845.00,
    "settled_egp": 0.00
  },
  "credit_summary": {
```

```
"total_outstanding_usd": 4200.00,
"overdue": false,
"next_due_date": "2026-05-15"
},
"dispute_status": "NONE",
"digital_signature": "MIAGCSqGSib3DQEHAqCAMIACAQExDzANBglghkgBZQMEAgEFADCABgkqhkiG9
w0BBwEAAKCAMIIB..."
}
```

Key Fields:

8.5.2 Hold — Mandatory Payment Pending

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "release_status": "HOLD",
  "hold_reason": "MANDATORY_PAYMENT_PENDING",
  "hold_reason_explanation": "Two mandatory invoices are unpaid: Egyptian Customs ($200) and SGS Lab ($200). Please pay these invoices before requesting release.",
  "unpaid_mandatory_invoices": [
    {
      "payee": "Egyptian Customs",
      "invoice_id": "CUST-20260415-001",
      "amount": 200.00,
      "currency": "USD"
    },
    {
      "payee": "SGS Lab",
      "invoice_id": "LAB-001",
      "amount": 200.00,
      "currency": "USD"
    }
  ],
  "dispute_status": "NONE"
}
```

The terminal must refuse gate-out. The exporter can see exactly what needs to be paid to release the container (via the Decision Panel in the portal).

8.5.3 Hold — Dispute Raised

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "release_status": "HOLD",
  "hold_reason": "DISPUTE_RAISED",
  "hold_reason_explanation": "A dispute (DSP-20260415-001) has been filed for this shipment. Release is frozen pending resolution. Please resolve the dispute through the SGTX mediation process.",
  "dispute_id": "DSP-20260415-001",
  "dispute_category": "QUALITY",
  "unpaid_mandatory_invoices": []
}
```

Even if all payments are settled, a dispute blocks release until resolved.

8.5.4 Hold — Conditional QC Hold

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "release_status": "HOLD",
  "hold_reason": "CONDITIONAL_QC_HOLD",
  "hold_reason_explanation": "A conditional QC hold is active. Action plan: 'Replace 3 mould-affected cartons on pallet OR019 within 24h.' Deadline: 2026-04-16T12:00:00Z. Please complete the action plan before requesting release.",
  "action_plan": "Replace 3 mould-affected cartons on pallet OR019 within 24h.",
  "action_plan_deadline": "2026-04-16T12:00:00Z",
  "unpaid_mandatory_invoices": []
}
```

8.5.5 Hold — Deferred Payment Guarantee Expired

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "release_status": "HOLD",
  "hold_reason": "DEFERRED_PAYMENT_EXPIRED",
  "hold_reason_explanation": "The deferred payment guarantee for import duties (amount: $10,000) has expired. Please pay the duties immediately or contact your customs broker.",
}
```

```
"deferred_amount": 10000.00,  
"deferred_currency": "USD",  
"expiry_date": "2026-05-15T23:59:59Z",  
"unpaid_mandatory_invoices": []  
}
```

8.5.6 Hold — Container Not Found or USTN Mismatch

json

```
{  
  "ustn": "SGTX-EG-26-F3A-1",  
  "container": "MEDU1234567",  
  "release_status": "ERROR",  
  "error_reason": "CONTAINER_NOT_FOUND_FOR_USTN",  
  "error_explanation": "The container MEDU1234567 is not linked to USTN SGTX-EG-26-F3A-1. Please  
verify the container number and USTN."  
}
```

The USTN exists, but this container is not linked to it. Possible fraud or data entry error.

8.5.7 Import Release — All Local MANDATORY Paid

json

```
{  
  "ustn": "SGTX-EG-26-F3A-1",  
  "container": "MEDU1234567",  
  "release_status": "AUTHORISED",  
  "authorisation_id": "REL-20260415-147-002",  
  "issued_at": "2026-04-15T16:00:00Z",  
  "valid_until": "2026-04-16T16:00:00Z",  
  "mandatory_summary": {  
    "total_usd": 15000.00,  
    "settled_usd": 15000.00  
  },  
  "exporter_payment_status": "AWAITING_BUYER_ACTION",  
  "exporter_payment_deadline": "2026-05-15",  
  "dispute_status": "NONE",  
  "digital_signature": "..."  
}
```

The container can be released because all local obligations (duties, VAT, port charges, broker fees) are settled. The payment to the foreign exporter is tracked but does not block release.

8.5.8 Hold Reason Codes (Complete List)

8.6 Digital Signature Technical Details (CMS)

8.6.1 Signature Generation Process

Canonicalisation: The response JSON object (excluding the `digital_signature` field) is serialised using JSON Canonicalisation Scheme (JCS, RFC 8785) to produce a deterministic byte sequence.

Signing: SGTX uses its Egypt Trust qualified signing certificate (stored in an HSM) to create a detached CMS signature (SignedData, PKCS#7) over the canonicalised bytes. The signature is detached (i.e., the original content is not embedded).

Encoding: The resulting CMS structure is base64-encoded and inserted as the `digital_signature` field.

8.6.2 Signature Verification Process (Terminal Side)

Extract the `digital_signature` base64 string.

Remove it from the JSON and canonicalise the remaining JSON.

Use SGTX's public certificate (hard-coded in the terminal's gate software or fetched from a trusted repository) to verify the CMS signature against the canonicalised bytes.

Check the certificate chain, expiry, and revocation status (OCSP/CRL).

If valid, the release authorisation is authentic and intact.

8.6.3 SGTX Public Key Information

One-click: Terminals can fetch the public key via the well-known URL.

8.7 Terminal Integration Workflow (Step-by-Step)

Step 1 — Truck Arrival

A truck arrives at the terminal gate carrying container MEDU1234567. The driver presents paperwork that includes the USTN SGTX-EG-26-F3A-1 (or the gate system scans the container number and automatically looks up the USTN from the terminal's internal system, which received it from the carrier's booking data).

Step 2 — Gate System Queries SGTX

The gate system makes a GET request:

text

GET

`/v1/release/authorization?ustn=SGTX-EG-26-F3A-1&container=MEDU1234567&request_id=gate4-001`

The mTLS handshake authenticates the terminal.

Step 3 — SGTX Evaluation

SGTX retrieves the USTN master data object (from PostgreSQL and NATS KV). It checks:

If all conditions pass, it constructs the AUTHORISED response, generates the digital signature, and returns it.

Step 4 — Gate Decision

The gate system receives the response. It verifies the digital signature. If `release_status` is AUTHORISED and signature is valid, it logs the `authorisation_id` and opens the gate. If HOLD, it displays the reason to the gate operator and refuses entry.

Step 5 — Audit Log

The terminal logs the entire response, including the signature, timestamp, and gate operator ID, to its immutable audit trail. This log can be used to demonstrate compliance with the ministerial decree and resolve any disputes about whether a container was properly released.

Step 6 — Gate-Out Event (Optional)

After the container is gated out, the terminal can optionally send a POST `/v1/release/gate-outwebhook` to SGTX, which updates the shipment milestone to `GATED_OUT`. This is not required for release but helps with tracking.

Gate-Out Webhook:

json

POST `/v1/release/gate-out`

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "authorisation_id": "REL-20260415-147-001",
  "gate_out_time": "2026-04-15T14:45:00Z",
  "operator_id": "GATE4-OP-042"
}
```

8.8 Security Considerations & Threat Model

AI Authority: A2 for monitoring anomaly patterns (e.g., repeated failed verifications) and generating alerts.

8.9 Release Lifecycle & Revocation

8.9.1 Release Lifecycle States

8.9.2 Revocation Triggers

Once issued, a release authorisation can be revoked by SGTX if new information comes to light before the container has gated out:

8.9.3 Revocation Flow

SGTX updates the USTN's release status to `REVOKED` in the shipments table and NATS KV.

The next time the terminal queries the API, it receives a `HOLD` status with `hold_reason: AUTHORISATION_REVOKED`.

SGTX also pushes a `RELEASE_REVOKED` webhook to the carrier and terminal if they have subscribed.

Revocation Webhook:

json

```
{
  "event": "RELEASE_REVOKED",
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "previous_authorisation_id": "REL-20260415-147-001",
  "reason": "Dispute filed",
}
```

"revoked_at": "2026-04-15T15:00:00Z"

}

8.9.4 Re-authorisation

After the condition is resolved (e.g., dispute settled), a new AUTHORISED response can be issued. The old authorisation ID is invalidated.

One-click for terminal: Re-query the same endpoint → receives new authorisation.

8.10 Full Worked Example — Gate Interaction (Strawberry Export)

Setup: Container MEDU1234567 is loaded with strawberries. The seller has paid Stage 1 (all mandatory fees), but ocean freight is CREDIT (not yet paid). The terminal has integrated the API.

Truck arrival: A truck arrives at Damietta Terminal Gate 4 to pick up the container for delivery to the buyer's warehouse. The driver provides the USTN SGTX-EG-26-F3A-1.

Gate query:

text

GET /v1/release/authorization?ustn=SGTX-EG-26-F3A-1&container=MEDU1234567&request_id=GATE4-20260415-1431-001

SGTX response (200 OK):

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "container": "MEDU1234567",
  "release_status": "AUTHORISED",
  "authorisation_id": "REL-20260415-147-001",
  "issued_at": "2026-04-15T14:31:00Z",
  "valid_until": "2026-04-16T14:31:00Z",
  "mandatory_summary": { "total_usd": 2845.00, "settled_usd": 2845.00 },
  "credit_summary": { "total_outstanding_usd": 4200.00, "overdue": false },
  "digital_signature": "MIAGCSqGSIb3DQEHAqCAMIACAQEx..."
}
```

Gate system actions:

Verifies digital signature using SGTX public key.

Logs authorisation_id to local database.

Opens the barrier. Truck enters.

Audit log entry:

text

timestamp: 2026-04-15T14:32:05Z

gate: GATE4

ustn: SGTX-EG-26-F3A-1

container: MEDU1234567

action: GATE_OUT

authorisation_id: REL-20260415-147-001

operator_id: SYS-AUTO

Later (optional): The terminal sends a gate-out webhook to SGTX, which updates the shipment milestone to GATED_OUT.

AI Authority: None (fully deterministic).

8.11 Error Scenarios & Handling

Manual Override Procedure (Emergency Only):

Terminal calls hotline (Platform Governance Authority).

Authority verifies identity and situation.

If justified, authority generates a one-time override token (valid 1 hour).

Terminal enters token manually; system logs override.

Override audited by Platform Governance Authority.

AI Authority: A2 for monitoring failure patterns; A1 for Groq-generated instructions to terminal operator.

8.12 Integration with Port Community Systems (PCS)

Many Egyptian ports operate Port Community Systems (PCS) that coordinate gate appointments, customs clearance, and terminal operations. SGTX can integrate with these PCS at a higher level, but the release API remains the atomic, legally-binding check.

PCS Integration:

SGTX does not replace the PCS — it augments it with cryptographic release enforcement.

8.13 Governance Gates (Part 8)

8.14 Database Schema Additions (Part 8)

sql

-- Release authorisations

CREATE TABLE release_authorisations (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,

container TEXT NOT NULL,

authorisation_id TEXT UNIQUE NOT NULL,

status TEXT DEFAULT 'PENDING', -- 'PENDING', 'AUTHORISED', 'REVOKED', 'USED', 'EXPIRED'

issued_at TIMESTAMPTZ,

valid_until TIMESTAMPTZ,

used_at TIMESTAMPTZ,

revoked_at TIMESTAMPTZ,

revocation_reason TEXT,

```

digital_signature TEXT,
request_id TEXT,
terminal_gtid TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Revoked certificates (CRL)
CREATE TABLE revoked_certificates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
certificate_serial TEXT NOT NULL UNIQUE,
revoked_at TIMESTAMPTZ DEFAULT NOW(),
reason TEXT, -- 'COMPROMISED', 'SUSPENDED', 'EXPIRED', 'SUPERSEDED'
revoked_by UUID REFERENCES employees(id) ON DELETE SET NULL
);

-- Gate-out events (optional webhook)
CREATE TABLE gate_out_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
container TEXT NOT NULL,
authorisation_id TEXT NOT NULL REFERENCES release_authorisations(authorisation_id) ON DELETE CASCADE,
gate_out_time TIMESTAMPTZ NOT NULL,
operator_id TEXT,
terminal_gtid TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Manual override log (emergency only)
CREATE TABLE release_overrides (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
container TEXT NOT NULL,
override_token TEXT UNIQUE NOT NULL,
issued_by UUID REFERENCES employees(id) ON DELETE SET NULL,
issued_at TIMESTAMPTZ DEFAULT NOW(),
valid_until TIMESTAMPTZ NOT NULL,
used_at TIMESTAMPTZ,

```

```

reason TEXT NOT NULL,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL
);
-- Indexes
CREATE INDEX idx_release_authorisations_ustn ON release_authorisations(ustn);
CREATE INDEX idx_release_authorisations_container ON release_authorisations(container);
CREATE INDEX idx_release_authorisations_status ON release_authorisations(status);
CREATE INDEX idx_release_authorisations_valid_until ON release_authorisations(valid_until);
CREATE INDEX idx_release_authorisations_authorisation_id ON
release_authorisations(authorisation_id);
CREATE INDEX idx_gate_out_events_ustn ON gate_out_events(ustn);
CREATE INDEX idx_gate_out_events_authorisation ON gate_out_events(authorisation_id);
CREATE INDEX idx_release_overrides_token ON release_overrides(override_token);
CREATE INDEX idx_release_overrides_ustn ON release_overrides(ustn);

```

8.15 Implementation Checklist (Part 8)

8.15.1 Core API

Implement release-service (Rust, Axum).

Add endpoint: GET /v1/release/authorization.

Add endpoint: POST /v1/release/webhook (for push notifications).

Add endpoint: POST /v1/release/gate-out (optional milestone update).

Add endpoint: GET /v1/release/crl (certificate revocation list).

8.15.2 Security

Set up mTLS certificate authority (step-ca, self-hosted).

Issue client certificates to terminals and carriers.

Implement certificate validation on each request.

Implement digital signature generation (PKCS#7/CMS) using HSM-stored private key.

Add certificate revocation list (CRL) management.

Implement rate limiting (60 req/min per terminal, 30 req/min per IP).

8.15.3 FeeLock Integration

Integrate with fee_payment_requests table to check MANDATORY invoice status.

Integrate with fee_locks table to check FeeLock state.

Integrate with disputes table to check for active disputes.

Integrate with qc_jobs table to check for conditional QC holds.

Integrate with gnn_risk_scores table for sanctions checks.

8.15.4 Release Lifecycle

Implement authorisation generation with valid_until (24h default).

Implement revocation logic (triggered by dispute, payment reversal, etc.).

Implement re-authorisation after conditions resolved.

Add release status tracking (PENDING → AUTHORISED → USED/EXPIRED/REVOKED).

8.15.5 Terminal Integration

Create terminal integration documentation (OpenAPI spec, mTLS setup, signature verification examples).

Produce sample terminal code (Python, Java, Rust) for verifying signatures.

Add webhook delivery with retry policy (3 retries, exponential backoff).

Implement manual override procedure (emergency only, with logging).

8.15.6 Monitoring & Alerting

Add Prometheus metrics:

`sgtx_release_requests_total`

`sgtx_release_authorised_total`

`sgtx_release_hold_total`

`sgtx_release_error_total`

`sgtx_release_verification_failures_total`

Set up alerting for:

High rate of HOLD responses (>10% of requests)

Signature verification failures

Certificate expiry (30 days before)

API 5xx errors (>1% of requests)

8.15.7 Testing

Write integration tests: simulate terminal query → AUTHORISED → gate-out webhook → milestone updated.

Test revocation flow: issue authorisation → trigger dispute → subsequent query returns HOLD.

Test expired authorisation: `valid_until` passes → query returns HOLD.

Test certificate validation: invalid certificate → 401 Unauthorised.

Test rate limiting: exceed limit → 429 Too Many Requests.

Test manual override flow.

8.16 AI Authority Summary for Part 8

END OF PART 8 — CONTAINER RELEASE AUTHORISATION API (v11.0 FULLY EXPANDED)

PART 9 — LOGISTICS PROVIDER MANAGEMENT (LSP, SHIP, LAB, QC, CBR)

9.0 Purpose & Design Philosophy

Logistics providers are the backbone of physical trade execution. SGTX treats each provider type as a first-class participant with its own portal, service catalogue, quotation workflow, and automated invoicing. The platform does not mandate any provider; rather, it enables sellers and buyers to discover, quote, select, and pay providers that they already know (via the Network feature) within the trade workflow. The platform never suggests providers, never ranks them, and never offers "you might also like" — all

relationships are explicit and pre-existing.

Provider Types (split from original LOG role):

Core Workflow for All Provider Types (Unified, Non-Marketplace):

AI Integration in Part 9:

All provider portals share common components:

Smart Inbox (Part 12A.1)

Trade Command Center (filtered to their jobs) (Part 12A.2)

Shipments Vault (role-specific columns) (Part 12A.4)

PlainLanguage Governor Decision Panel (Part 12A.3)

VoiceCommandButton (Part 12A.5)

Customer Care Chatbot (Part 12A.6)

No marketplace discovery — providers are only visible to traders who have explicitly added them as contacts or have completed prior trades with them.

9.1 Logistics Service Provider (LSP) — Trucking, Forwarding, Warehousing

9.1.1 Onboarding & Service Catalogue

Onboarding Requirements (from Part 2.1):

Service Catalogue Example (Trucking):

Service Catalogue Example (Warehousing):

One-click: LSP admin adds route → "Save" (1 click).

9.1.2 Quotation & Acceptance Workflow (Trucking)

Step 1 — Seller requests quote (during Phase 2 — Logistics Builder). Seller selects "Trucking" service, enters pickup and delivery addresses, container type, weight. System calculates estimated distance (OSRM) and displays a dropdown of eligible LSPs that the seller has previously worked with or added to contacts (never a list of "top rated" strangers). Seller selects "Fast Trucking Co."

Step 2 — LSP receives request (Smart Inbox): "New trucking request for USTN ... — pick up 22 pallets frozen strawberries from Beheira to Damietta. Estimated distance 150 km. Send quote."

Step 3 — LSP sends quote: LSP clicks "Send Quote". Pre-filled from catalogue ($\$0.85/\text{km} \times 150 \text{ km} = \127.50). LSP can adjust.

json

POST /v1/lsp/quote

```
{
  "ustn": "SGTX-...",
  "service_type": "TRUCKING",
  "fee": 127.50,
  "currency": "USD",
  "valid_until": "2026-04-17T23:59:59Z",
  "notes": "Includes fuel, driver, insurance up to $50k"
```

}

Step 4 — Seller accepts quote: Seller sees quote in Smart Inbox, clicks "Accept". SGTX generates invoice for \$127.50 (ETA-compliant), adds to Stage 1 payment plan.

Step 5 — Payment collection: Seller pays Stage 1 (which includes this fee). PSP split sends \$127.50 to LSP (minus platform fee). LSP receives Smart Inbox: "Payment received. You may now dispatch the truck."

Step 6 — Service delivery: Driver uses mobile app (offline) to scan pallet barcodes, confirm pickup, and delivery. GPS tracks route. After delivery, LSP marks milestone "Delivered". Shipment status updates.

Step 7 — Dispute (if any): Buyer claims goods damaged due to rough handling. Evidence package includes GPS trace, speed logs, driver notes. AI-generated causal analysis attributes damage to route roughness. Settlement proposed.

AI Authority: A1 for quote price suggestions (Groq); A2 for route optimisation (OSRM, rule-based); A4 for milestone confirmation.

9.1.3 LSP Portal Tabs

9.1.4 Dispatch Planner (Trucking-only)

Purpose: Optimise daily route plan for trucking fleet using ORTools VRP solver. Assign drivers, manage geofence alerts, and integrate voice navigation.

One-click guarantee: Optimise (1), Assign (1), Send (1).

9.2 Shipping Line (SHIP) — Ocean Carriers, NVOCCs

9.2.1 Onboarding & Service Catalogue

Onboarding Requirements:

Service Catalogue Example:

9.2.2 Quotation & Booking Workflow

Step 1 — Seller requests booking (during Phase 2 — Logistics Builder). Seller selects "Ocean freight", enters port of loading, discharge, container type, desired sailing window. System displays eligible shipping lines that the seller has a contract with or has previously used (from saved contacts). Seller selects "MSC Egypt".

Step 2 — Shipping line receives request (Smart Inbox): "New booking request for USTN ... — 20,000 kg frozen strawberries, Damietta → Hamburg, 2 × 40ft reefers, requested sailing 20 Jun. Send quote."

Step 3 — SHIP sends quotation: Clicks "Send Quote". Pre-filled from contract rate (\$4,200 per container = \$8,400 total). Shipping line can adjust (e.g., peak season surcharge).

json

POST /v1/ship/quote

{

"ustn": "SGTX-...",

"service_type": "OCEAN_FREIGHT",

"fee": 8400.00,

"currency": "USD",

"valid_until": "2026-04-20T23:59:59Z",


```
"vessel": "MSC FIONA",
"voyage": "MSF123E",
"etd": "2026-04-20T08:00:00Z",
"eta": "2026-05-05T14:00:00Z"
}
```

Step 4 — Seller accepts quote: SGTX generates invoice for \$8,400 (added to Stage 2 payment plan — credit terms may apply).

Step 5 — Booking confirmation: Shipping line confirms booking via portal or API → returns booking reference MAEU876543210. SGTX updates shipments.status = BOOKED.

Step 6 — eBL issuance: After vessel loading, shipping line issues eBL via their platform. SGTX receives webhook (or polls) and stores eBL in documents. The eBL includes the USTN.

Step 7 — Container release: After seller pays Stage 1 (and any mandatory Stage 2), SGTX sends container release confirmation (email + API token). Shipping line acknowledges; container gated out.

Step 8 — Milestone updates: Shipping line updates milestone "Departed" (via portal or API). AIS feed automatically updates position. Buyer tracks.

AI Authority: A1 for match score and sailing schedule suggestions; A2 for eBL extraction (HF Donut); A4 for booking confirmation validation.

9.2.3 Shipping Line Portal Tabs

9.3 Laboratory (LAB) — Testing Services

9.3.1 Onboarding & Service Catalogue

Onboarding Requirements:

Service Catalogue Example:

9.3.2 Quotation & Testing Workflow

Step 1 — Seller selects lab (during Phase 2 — Laboratory Selection). Seller sees accredited labs that they have previously used or added to contacts. Selects "SGS Egypt".

Step 2 — Lab receives request (Smart Inbox): "New testing request for USTN ... — commodity: frozen strawberries, required tests: pesticide panel (Cypermethrin, Chlorpyrifos, Thiabendazole). Send quote."

Step 3 — Lab sends quotation: Clicks "Send Quote". Pre-filled from catalogue (\$200). Includes sample instructions.

json

POST /v1/lab/quote

```
{
"ustn": "SGTX-...",
"service_type": "PESTICIDE_PANEL",
"fee": 200.00,
"currency": "USD",
"valid_until": "2026-04-18T23:59:59Z",
"sample_instructions": "Send 500g frozen sample to SGS Cairo lab, keep frozen, include USTN label."
}
```

```
}
```

Step 4 — Seller accepts quote: SGTX generates invoice for \$200, adds to Stage 1 payment plan. Seller pays Stage 1. Lab receives payment notification.

Step 5 — Lab performs testing: Lab receives sample (tracking number), performs tests, enters results in structured form (or uploads JSON). System validates against MRLs, flags any exceedance.

json

POST /v1/lab/results

```
{
  "ustn": "SGTX-...",
  "results": [
    { "analyte": "Cypermethrin", "value": 0.02, "unit": "mg/kg", "limit": 0.05, "pass": true },
    { "analyte": "Chlorpyrifos", "value": 0.008, "unit": "mg/kg", "limit": 0.01, "pass": true }
  ],
  "conclusion": "COMPLIANT"
}
```

Step 6 — Auto-trigger certificates: If all required tests are compliant, SGTX automatically requests phytosanitary and health certificates from Nafeza. Certificates issued and attached to USTN.

Step 7 — Failure handling: If any test fails, the loading milestone is blocked. Seller can dispute (request retest) or cancel. Dispute evidence includes original results, sample chain of custody, and lab's ISO accreditation.

AI Authority: A2 for MRL validation (HF local) and result extraction; A4 for certificate triggering.

9.3.3 Laboratory Portal Tabs

9.4 QC Inspection (QC) — On-Site Inspection

9.4.1 Onboarding & Service Catalogue

Onboarding Requirements:

Service Catalogue Example:

9.4.2 Quotation & Inspection Workflow (Buyer-Requested)

Step 1 — Buyer requests QC inspection during trade creation (Phase 1). Selects "SGS Inspection" from dropdown (only if SGS is in buyer's saved contacts).

Step 2 — QC provider receives job (Smart Inbox): "New inspection request for USTN ... — commodity: frozen strawberries, 22 pallets, location: Damietta port. AQL General Level II (5 pallets random). Send quote."

Step 3 — QC sends quotation: Clicks "Send Quote". Pre-filled from catalogue (\$500).

json

POST /v1/qc/quote

```
{
  "ustn": "SGTX-...",
  "service_type": "FRESH_FRUIT_INSPECTION",
```

```
"fee": 500.00,  
"currency": "USD",  
"valid_until": "2026-04-20T23:59:59Z",  
"inspection_date": "2026-04-18",  
"location": "Damietta Port, Gate 4"  
}
```

Step 4 — Buyer accepts quote: SGTX generates invoice for \$500, adds to buyer's local payment batch. Buyer pays via PSP. QC provider receives payment notification.

Step 5 — Inspector performs inspection: Uses mobile app (offline, AR overlay). Scans pallet barcodes, logs defects, overrides AI findings with mandatory reason (≥ 10 chars). AQL sampling enforced — cannot submit until random sample inspected.

Step 6 — Report submission: Inspector submits report (PASS, FAIL, or CONDITIONAL). Report includes:

Container & seal numbers (with photos)

Defect log (per pallet, with AI confidence)

Override details (original AI confidence, inspector's reason)

Digital signature (ZITADEL passkey)

Step 7 — Loading milestone: If verdict is PASS or CONDITIONAL, loading proceeds. If FAIL, buyer must accept or seller replace goods.

Step 8 — Dispute fast-track: If dispute arises, evidence package automatically includes override details, original AI confidence, and inspector's reason.

AI Authority: A2 for defect detection (HF ViT) and override enforcement; A1 for report drafting (Groq); A4 for milestone blocking.

9.4.3 QC Portal Tabs

9.4.4 QC Mobile App (Offline-First)

Technology: React Native + Expo, WatermelonDB, HF ViT (local), ZXingC++.

Features:

One-click guarantee: Scan pallet (1 per pallet, or batch mode), Override (1 + reason), Submit (1).

9.5 Customs Broker (CBR) — Optional Services

9.5.1 Onboarding & Service Catalogue

Onboarding Requirements (from Part 2.1):

Service Catalogue Example:

9.5.2 Quotation & Service Workflow

Certification Service (Seller-Paid):

Step 1 — Seller selects broker during Phase 2 (Logistics Builder). Checks "Certification" and selects broker from dropdown (only brokers in seller's saved contacts).

Step 2 — Broker receives request (Smart Inbox): "Certification requested for USTN ..." Broker reviews AI-generated declaration (read-only, with confidence scores). Low-confidence fields highlighted.

Step 3 — Broker sends quote: Clicks "Send Quote". Default fee \$150 (or custom).

json

POST /v1/broker/services/quote

```
{  
  "ustn": "SGTX-...",  
  "service_type": "CERTIFICATION",  
  "fee": 150.00,  
  "currency": "USD",  
  "valid_until": "2026-04-18T23:59:59Z"  
}
```

Step 4 — Seller accepts quote: SGTX generates invoice for \$150, adds to Stage 1 payment plan. Seller pays Stage 1. Broker receives payment notification.

Step 5 — Broker certifies: In CBR Portal, clicks "Certify". System records broker's digital seal, licence number, timestamp. SGTX submits declaration to Nafeza under broker's licence. Declaration reference stored.

Physical Handling Service (Seller-Paid, Destination Requires Original Documents):

Step 1 — RIA detects that destination country (e.g., Nigeria) requires original documents. System automatically recommends physical handling (but does not force).

Step 2 — Seller selects broker and checks "Physical document handling". Broker sends quote (\$85). Seller accepts → invoice generated, added to Stage 1.

Step 3 — SGTX generates document package — cover sheet with manifest, USTN QR code, broker's registered office address. Seller prints, signs wet-ink, couriers.

Step 4 — Broker receives courier — scans QR code using mobile app, clicks "Receive Package". GPS and timestamp recorded. Status becomes RECEIVED.

Step 5 — Broker visits customs — presents documents, obtains stamped acknowledgement. Uses mobile app to scan stamped documents. AI extracts stamp and reference.

Step 6 — Broker marks "Submission Confirmed" — milestone reached, fee released (if held). Stamped copy attached to USTN.

Step 7 — Storage (optional) — Seller can also select storage service (\$40/year). Broker stores original documents, records shelf location in portal. At retention expiry, Smart Inbox reminder to destroy or return.

AI Authority: A2 for AI declaration confidence (HF Donut); A1 for Groq-generated explanations; A4 for certification enforcement.

9.5.3 Customs Broker Portal Tabs

9.5.4 Physical Document Handling — Mobile App

9.6 Unified Service Quotation & Acceptance — API Endpoints

All provider types use the same pattern for quotes and acceptance.

Example — Accept Quote:

json

POST /v1/quote/CBRQ-001/accept

```
{
```

```
"accepted_by_gtid": "SGTX-EG-TRD-002139-7F3A",
"notes": "Please proceed with certification."
}
Response:
json
{
"quote_id": "CBRQ-001",
"status": "ACCEPTED",
"invoice_uuid": "INV-BRK-001",
"message": "Invoice generated and added to payment plan."
}
```

AI Authority: A4 for validation and invoice generation.

9.7 Incoterm-Based Service Filtering for RFQ Mode B

When a seller uses RFQ Mode B (Request Quotes) for logistics, the system automatically filters the list of services based on the buyer's selected incoterm. The WasmEdge incoterms engine enforces that only services applicable to the incoterm are shown and that mandatory services are required before quote submission.

Implementation:

The `incoterms_engine.wasm` module contains a mapping of incoterm to mandatory services.

When the seller opens the Logistics Builder, the system reads the incoterm from the trade request and dynamically builds the service list.

If the seller attempts to submit a quote missing a mandatory service, the Governor returns a **CONDITIONAL** verdict with a Decision Panel explaining which service is missing.

Incoterm-Applicable Services Table:

Governance: The Governor checks that the seller has selected a provider quote for every mandatory service. Missing services result in a **CONDITIONAL** verdict with a Decision Panel listing the missing services and direct links to request quotes.

AI Authority: A4 for WasmEdge enforcement; A2 for optional AI suggestion of missing services.

9.8 Provider Performance Self-Service Dashboard

Every logistics provider has a Performance tab displaying:

Example Performance Summary (A1, Groq):

"Your on-time performance has improved to 89%. You are in the top quartile for document accuracy. Your dispute rate (3%) is below the regional average (5%)."

One-click: View summary (auto-displayed), Download report (1).

9.9 Logistics Provider Warm-Up Program & Unsubscribe Mechanism

9.9.1 Co-Branded Notification Emails

When SGTX sends an anonymous broadcast RFQ email, the sender name is formatted as "Seller via SGTX" <noreply@sgtx.io>. The email body clearly states: "You are receiving this because you are

registered on SGTX as a logistics provider serving [route]. An anonymous seller has requested a quote. Your identity will be revealed only if they accept your quote."

9.9.2 Unsubscribe / Opt-Out

Every co-branded email includes an "Unsubscribe from anonymous RFQs" link in the footer. Clicking it sets `tenants.anonymous_rfq_opt_out = true` and displays a confirmation: "You have unsubscribed from anonymous RFQs. You will still receive directed requests from known sellers. You can re-enable this at any time in Company Admin → Notification Preferences."

9.9.3 Provider Preferences

Within the portal, providers manage this preference under Company Admin → Notification Preferences:

One-click: Toggle anonymous RFQ opt-out (1).

9.10 Governance & Permissions

9.11 Database Schema Additions (Part 9)

sql

-- Provider types (extended from tenants)

```
ALTER TABLE tenants ADD COLUMN lsp_subtype TEXT; -- 'TRUCKING', 'FORWARDER', 'WAREHOUSING'
```

```
ALTER TABLE tenants ADD COLUMN financier_subtype TEXT; -- 'BANK', 'PRIVATE'
```

```
ALTER TABLE tenants ADD COLUMN anonymous_rfq_opt_out BOOLEAN DEFAULT false;
```

-- Service quotations (core)

```
CREATE TABLE service_quotations (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
```

```
provider_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
```

```
service_type TEXT NOT NULL, -- 'LAB_TEST', 'QC_INSPECTION', 'BROKER_CERTIFICATION',  
'BROKER_PHYSICAL', 'BROKER_STORAGE', 'BROKER_AUDIT', 'TRUCKING', 'OCEAN_FREIGHT',  
'AIR_FREIGHT', 'WAREHOUSING', 'FORWARDING'
```

```
fee DECIMAL(10,2) NOT NULL,
```

```
currency TEXT DEFAULT 'USD',
```

```
valid_until TIMESTAMPTZ NOT NULL,
```

```
notes TEXT,
```

```
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACCEPTED', 'DECLINED', 'EXPIRED'
```

```
accepted_at TIMESTAMPTZ,
```

```
invoice_uuid TEXT,
```

```
created_at TIMESTAMPTZ DEFAULT NOW()
```

```
);
```

-- Ship quote requests (Mode C)

```
CREATE TABLE ship_quote_requests (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
seller_gtid TEXT NOT NULL,
base_service_type TEXT NOT NULL, -- 'OCEAN_FREIGHT', 'AIR_FREIGHT'
origin_port TEXT NOT NULL,
destination_port TEXT NOT NULL,
container_details JSONB NOT NULL,
add_on_services TEXT[], -- 'TRUCKING', 'CUSTOMS_BROKER', 'INSURANCE'
target_lines TEXT[], -- GTIDs of shipping lines
status TEXT DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE ship_quotes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
request_id UUID NOT NULL REFERENCES ship_quote_requests(id) ON DELETE CASCADE,
shipper_line_gtid TEXT NOT NULL,
base_fee DECIMAL(20,4) NOT NULL,
add_on_fees JSONB, -- {"TRUCKING": 300, "CUSTOMS_BROKER": 150}
total_fee DECIMAL(20,4) NOT NULL,
validity_period_hours INTEGER DEFAULT 48,
selected BOOLEAN DEFAULT false,
submitted_at TIMESTAMPTZ DEFAULT NOW()
);

-- Clarification requests (Improvement #4)
CREATE TABLE clarification_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
quotation_id UUID REFERENCES service_quotations(id) ON DELETE CASCADE,
requester_gtid TEXT NOT NULL,
questions JSONB NOT NULL,
answers JSONB,
resolved BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ
);

-- QC jobs
CREATE TABLE qc_jobs (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
provider_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
inspection_date DATE,
sampling_plan JSONB, -- AQL plan
status TEXT DEFAULT 'ASSIGNED', -- 'ASSIGNED', 'ACCEPTED', 'IN_PROGRESS', 'COMPLETED',
'DISPUTED'
verdict TEXT, -- 'PASS', 'FAIL', 'CONDITIONAL'
conditional_pass_status TEXT, -- 'PENDING', 'COMPLETED', 'EXPIRED'
action_plan TEXT,
action_plan_deadline TIMESTAMPTZ,
action_plan_completed_at TIMESTAMPTZ,
report_url TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE inspection_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
qc_job_id UUID REFERENCES qc_jobs(id) ON DELETE CASCADE,
pallet_id TEXT,
ai_defect TEXT,
ai_confidence DECIMAL(5,2),
inspector_override BOOLEAN DEFAULT false,
inspector_override_reason TEXT,
final_classification TEXT,
photo_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Re-inspection requests
CREATE TABLE re_inspection_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
original_ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
requested_by_gtid TEXT NOT NULL,
reason TEXT NOT NULL,
urgency TEXT DEFAULT 'NORMAL', -- 'NORMAL', 'URGENT'
deadline TIMESTAMPTZ,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACCEPTED', 'DECLINED', 'COMPLETED'

```



```

new_qc_job_id UUID REFERENCES qc_jobs(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Broker services

CREATE TABLE broker_certifications (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
declaration_id TEXT,
certified_at TIMESTAMPTZ,
digital_seal TEXT,
licence_number TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE broker_physical_jobs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
package_id TEXT UNIQUE,
courier_tracking TEXT,
status TEXT DEFAULT 'AWAITING_RECEIPT', -- 'AWAITING_RECEIPT', 'RECEIVED',
'PRESENTED_TO_CUSTOMS', 'STAMPED', 'COMPLETED'
received_at TIMESTAMPTZ,
presented_at TIMESTAMPTZ,
stamped_at TIMESTAMPTZ,
stamped_copy_url TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE broker_storage (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
shelf_location TEXT,
retention_expiry DATE,

```

```

status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'RETURNED', 'DESTROYED'
returned_at TIMESTAMPTZ,
destroyed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Carrier contract rates (private)
CREATE TABLE carrier_contracts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
carrier_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
seller_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
route_origin TEXT,
route_destination TEXT,
container_type TEXT,
base_rate DECIMAL(20,4),
currency TEXT DEFAULT 'USD',
valid_from TIMESTAMPTZ,
valid_until TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_service_quotations_ustn ON service_quotations(ustn);
CREATE INDEX idx_service_quotations_provider ON service_quotations(provider_gtid);
CREATE INDEX idx_service_quotations_status ON service_quotations(status);
CREATE INDEX idx_ship_quote_requests_ustn ON ship_quote_requests(ustn);
CREATE INDEX idx_ship_quotes_request ON ship_quotes(request_id);
CREATE INDEX idx_clarification_requests_quotation ON clarification_requests(quotation_id);
CREATE INDEX idx_qc_jobs_ustn ON qc_jobs(ustn);
CREATE INDEX idx_qc_jobs_provider ON qc_jobs(provider_gtid);
CREATE INDEX idx_qc_jobs_status ON qc_jobs(status);
CREATE INDEX idx_inspection_logs_job ON inspection_logs(qc_job_id);
CREATE INDEX idx_re_inspection_requests_ustn ON re_inspection_requests(original_ustn);
CREATE INDEX idx_broker_certifications_ustn ON broker_certifications(ustn);
CREATE INDEX idx_broker_physical_jobs_ustn ON broker_physical_jobs(ustn);
CREATE INDEX idx_broker_storage_ustn ON broker_storage(ustn);
CREATE INDEX idx_broker_storage_retention ON broker_storage(retention_expiry);

```

CREATE INDEX idx_carrier_contracts_carrier ON carrier_contracts(carrier_gtid);

CREATE INDEX idx_carrier_contracts_seller ON carrier_contracts(seller_gtid);

9.12 Implementation Checklist (Part 9)

9.12.1 Core Infrastructure

Split LOG tenant type into LSP, SHIP, LAB, QC, CBR in database (migration).

Implement separate portals for each provider type (Next.js).

Build unified quotation service (quote-service) with provider-specific validation.

Implement automated invoice generation for accepted quotes (ETA-compliant).

Integrate invoice with payment plan (add to Stage 1 or Stage 2).

Add PSP split configuration for provider payouts.

9.12.2 LSP Portal

Implement RFQ Inbox with directed and anonymous broadcast.

Add clarification request workflow (structured Q&A).

Build Dispatch Planner (ORTools VRP) with driver assignment.

Build Warehouse Dashboard (warehousing only) with storage utilisation.

Build Forwarder Console (forwarding only) with subcontractor network and LCL consolidation.

Implement Driver Mobile App management (QR pairing, offline queue, driver assignment).

9.12.3 SHIP Portal

Implement Booking Requests with quote submission (Mode C).

Add eBL Management with webhook integration.

Build Vessel Schedule with ETD/ETA propagation.

Implement Freight Invoices with credit/mandatory terms.

Build Contract Rate Manager (private rates per seller).

9.12.4 LAB Portal

Implement Testing Jobs with sample tracking.

Build Result Submission with MRL validation (A2).

Add Certificate auto-trigger (Nafeza integration).

Implement structured result entry with override capability.

9.12.5 QC Portal

Implement Inspection Jobs with AQL sampling enforcement.

Build Mobile App Integration (offline-first, AR, defect detection).

Implement Report Submission with PASS/FAIL/CONDITIONAL verdicts.

Add Conditional Pass with action plan and hold flag.

Build Re-inspection Request workflow.

Implement Dispute Fast-Track with override flagging.

9.12.6 CBR Portal

Implement Certification Requests with declaration preview.

Build Physical Document Jobs with QR scanning and GPS tracking.

Implement Storage management with retention expiry.

Build Audit Representation workflow.

Add Digital Seal management (mTLS + SoftHSM).

9.12.7 Performance & Analytics

Build Provider Performance dashboard (on-time, dispute rate, invoice accuracy).

Implement anonymous benchmarks (differential privacy, $\epsilon = 0.1$).

Add Groq-generated performance summaries (A1).

9.12.8 Warm-Up Program & Opt-Out

Implement co-branded email notifications ("Seller via SGTX").

Add "Unsubscribe from anonymous RFQs" link in emails.

Build preferences UI (Company Admin → Notification Preferences).

9.12.9 Testing

Test full LSP workflow: RFQ → clarification → quote → acceptance → milestone confirmation → payment.

Test full SHIP workflow: booking request → quote → confirmation → eBL → milestone update.

Test full LAB workflow: test request → quote → sample receipt → results → certificate trigger.

Test full QC workflow: inspection job → mobile inspection → report (PASS/FAIL/CONDITIONAL) → hold removal.

Test full CBR workflow: certification → physical handling → storage.

Test incoterm-based service filtering.

Test anonymous RFQ opt-out.

Test provider performance dashboard.

9.13 AI Authority Summary for Part 9

END OF PART 9 — LOGISTICS PROVIDER MANAGEMENT (v11.0 FULLY EXPANDED)

PART 10 — DISPUTE MANAGEMENT & REPUTATION ENGINE

10.0 Purpose & Design Philosophy

International trade disputes are inevitable. Goods arrive damaged, payments are delayed, quality does not match the contract, certificates turn out to be fraudulent, or one party simply fails to perform. In the traditional paper-based trade world, resolving these disputes is slow, expensive, and often inconclusive. Evidence is scattered across emails, WhatsApp messages, PDFs from different sources, and paper files in different countries.

SGTX embeds a comprehensive dispute management system directly into the trade operating system. Because every document, payment, and logistic event for a shipment is already bound to the USTN and immutably recorded, SGTX can instantly surface the complete, cryptographically verifiable truth of the transaction. The dispute module is not a marketplace — it never suggests alternative counterparties or service providers. It provides a neutral, structured, AI-assisted mediation environment that helps the

existing parties resolve their disagreement efficiently.

Key capabilities:

Any breach — quality, delay, non-payment, documentation fraud, cold chain failure, service quality (broker, lab, QC) — can be raised as a dispute.

One-click filing — buyer, seller, logistics provider, or financier can file with voice or text.

Evidence autocompiler — automatically pulls all USTN data, sensor logs, QC overrides, broker certification logs, and causal analysis.

Causal inference engine (Add-on 3) — identifies root causes (e.g., "68% due to temperature excursion") and suggests fair settlement.

Mediation log — structured, multi-language negotiation with AI-generated proposals.

Escalation to arbitration — auto-prepares case files for ICC, DIFC-LCIA, CRCICA.

SGTX fee dispute — separate workflow for disputing the platform's fee calculation.

Reputation score (Trade Reliability Index) — dynamic 0-1000 score for every tenant, updated after each dispute resolution, influencing financing rates, inspection frequency, and trading privileges. The score is private — only the tenant sees their own full breakdown; counterparties see only a summarised trust badge (e.g., "Advanced Trusted").

AI Integration in Part 10:

10.1 Dispute Categories & Triggers

10.1.1 Dispute Categories

10.1.2 Dispute Triggers

AI Authority: A2 for anomaly detection; A1 for advisory message.

10.2 Dispute Filing — Step-by-Step

10.2.1 Manual Filing (Web or Mobile)

Step 1 — Access USTN: User navigates to Trade Command Center for the relevant USTN (or shipment USTN for multi-shipment, or financing agreement ID, or microUSTN for distressed). Clicks "Raise Dispute".

Step 2 — Select Category: From dropdown (see 10.1.1). Category determines which evidence is auto-compiled and which parties are notified.

Step 3 — Describe Breach: Free-text description (min 10 characters). System may pre-fill based on detected anomalies.

Step 4 — Attach Evidence: User uploads photos, PDFs, or references external reports. The system will already have auto-compiled a base evidence package, but user can add more.

Step 5 — Specify Remedy Sought: e.g., "Full refund of \$100,000", "Partial refund of \$10,000", "Replace damaged pallets", "Extend payment terms by 30 days".

Step 6 — Submit: Governor validates that the user has permission to file a dispute for that USTN. If ALLOW, dispute is created with unique `dispute_id`. USTN status becomes DISPUTED. All pending payments are frozen, and container release (if still at port) is blocked.

Voice-initiated filing (mobile app):

text

User speaks: "File dispute for USTN SGTX-... The oranges arrived with mould. I want \$2,000 refund."

Vosk transcribes → HF Mixtral extracts USTN, category (QUALITY), amount → pre-fills form.

User confirms → dispute filed.

One-click: Click "Submit Dispute" (1) → dispute created.

AI Authority: A1 for voice transcription and intent extraction; A4 for Governor validation.

10.2.2 Automatic Advisory Dispute (System-Triggered)

System detects anomaly — e.g., temperature deviation $>5^{\circ}\text{C}$ for >2 hours. Smart Inbox item appears: "Potential cold chain failure detected on USTN ... Click to file a dispute proactively." Buyer can accept (pre-filled form) or ignore. The system never automatically files; it only suggests.

AI Authority: A2 for anomaly detection (LSTM); A1 for advisory message.

10.3 Evidence Autocompiler (Immediate & On-Demand)

10.3.1 Package Contents

When a dispute is filed (or on demand), the evidence-compiler service (Rust) automatically gathers:

10.3.2 Package Security

The package is Loom-hashed and stored in evidence_packages.

A verification token is generated for external auditors (courts, arbitrators). The token is a one-time UUID linked to the package.

The holder can view the package (after authentication) but cannot modify it.

10.3.3 Example — Evidence Package for Quality Dispute (Mould)

json

```
{
  "package_id": "EP-20260415-001",
  "ustn": "SGTX-...",
  "dispute_id": "DSP-20260415-001",
  "compiled_at": "2026-04-15T15:00:00Z",
  "contents": {
    "temperature_log": [
      { "timestamp": "2026-04-14T08:00:00Z", "temperature_c": 5.2 },
      { "timestamp": "2026-04-14T10:00:00Z", "temperature_c": -8.0 },
      { "timestamp": "2026-04-14T12:00:00Z", "temperature_c": -8.5 }
    ],
    "qc_report": {
      "overrides": [
        {
          "pallet_id": "OR020",
          "ai_defect": "mould",
          "ai_confidence": 88,
```

```

"inspector_override": "bruising",
"override_reason": "Dark spot is surface bruise, no mould.",
"timestamp": "2026-04-14T14:30:00Z"
}
],
"lab_results": {
"cypermethrin": { "value": 0.02, "limit": 0.05, "pass": true }
},
"broker_certification": {
"certified_at": "2026-04-15T09:00:00Z",
"broker_gtid": "SGTX-EG-CBR-001",
"declaration_id": "SAD20260415-000147"
},
"causal_analysis": {
"root_causes": [
{
"factor": "temperature_excursion",
"contribution": 0.68,
"description": "Reefer set point drifted to -8°C for 6 hours due to compressor failure."
}
]
}
},
"loom_hash": "sha256:a1b2c3...",
"verification_token": "vt-123e4567-e89b-12d3-a456-426614174000"
}

```

One-click: Click "View Package" (1) → read-only, signed evidence.

AI Authority: A2 for evidence compilation and QC override flagging; A4 for Loom hashing.

10.4 Causal Inference Engine (Add-on 3) — Root Cause Analysis

10.4.1 Trigger & Model

When a dispute is filed, the causal inference engine (A2, DoWhy + EconML) is triggered asynchronously. It analyses:

Timeline of milestones (delays between expected and actual)

Sensor data (temperature, humidity, shock)

Weather and port congestion (from RIA)

Carrier performance history (on-time delivery, damage claims)

QC override patterns (frequency, type of overrides)

Broker certification history (accuracy, rejections)

Methodology: DoWhy builds a causal graph (DAG) based on domain knowledge; EconML estimates the Average Treatment Effect (ATE) and contribution of each factor using double-ML.

10.4.2 Output

A JSON object with root causes and contribution percentages, stored in `causal_attribution`. Groq generates a plain-language summary added to the evidence package and displayed in the mediation log.

Example Output for Delay Dispute:

json

```
{
  "root_causes": [
    {
      "factor": "port_strike",
      "contribution": 0.54,
      "description": "Port closure at Alexandria added 54% of total delay.",
      "confidence_interval": { "lower": 0.45, "upper": 0.62 }
    },
    {
      "factor": "carrier_rerouting",
      "contribution": 0.32,
      "description": "Carrier rerouted due to weather, adding 32%.",
      "confidence_interval": { "lower": 0.28, "upper": 0.36 }
    },
    {
      "factor": "customs_hold",
      "contribution": 0.14,
      "description": "Missing phytosanitary certificate caused 14% delay.",
      "confidence_interval": { "lower": 0.10, "upper": 0.18 }
    }
  ],
  "model_version": "v2.1",
  "groq_summary": "The delay was caused primarily by the port strike (54%), with the carrier's rerouting as a secondary factor (32%). The missing certificate contributed 14%."
}
```

Example Output for Cold Chain Dispute:

json

```
{
  "root_causes": [
    {
      "factor": "compressor_failure",
      "contribution": 0.78,
      "description": "Reefer compressor failure caused temperature to rise to -8°C for 6 hours.",
      "confidence_interval": { "lower": 0.70, "upper": 0.85 }
    },
    {
      "factor": "pre_cooling_omission",
      "contribution": 0.12,
      "description": "Pre-cooling omitted before loading, contributing 12%.",
      "confidence_interval": { "lower": 0.08, "upper": 0.16 }
    },
    {
      "factor": "monitoring_delay",
      "contribution": 0.10,
      "description": "Temperature alert delayed by 2 hours, adding 10% to damage.",
      "confidence_interval": { "lower": 0.06, "upper": 0.14 }
    }
  ],
  "groq_summary": "The cold chain failure was caused primarily by compressor failure (78%), with pre-cooling omission (12%) and monitoring delay (10%) as contributing factors."
}
```

AI Authority: A2/A3 for causal inference (DoWhy + EconML, Python); A1 for Groq summary.

10.5 Mediation Log & Multi-Language Support

10.5.1 Interface

After a dispute is filed, both parties enter a structured mediation log accessible from the Dispute tab in their portals. The log supports:

Text messages (with automatic translation via Groq into each party's preferred language).

Structured offers (e.g., "Offer \$2,000 refund").

Accept / reject / counter buttons.

Attachment of additional evidence (e.g., new photos, third-party survey).

AI-generated settlement proposals (see 10.6).

Escalation to arbitration button (after a configurable number of rounds, default 5).


```
{
  "proposal_id": "SP-20260415-001",
  "dispute_id": "DSP-20260415-001",
  "proposed_settlement": {
    "type": "PARTIAL_REFUND",
    "amount": 800.00,
    "currency": "USD",
    "conditions": [
      "Buyer releases seller from further claims",
      "Both parties bear own costs"
    ]
  },
  "rationale": "The temperature log shows no deviation during transit, indicating the damage likely occurred post-delivery. However, the buyer's photos show mould presence, suggesting a pre-existing quality issue. A 4% refund is consistent with similar cases (based on 12 previous disputes).",
  "confidence": 0.87,
  "acceptance_deadline": "2026-04-23T23:59:59Z"
}
```

10.6.3 Acceptance Workflow

The proposal is presented to both parties.

If both accept (via one click each), SGTX generates a settlement addendum, signed by both parties and the Governor.

The dispute status becomes RESOLVED_SETTLED.

The agreed amount is transferred via PSP split (if refund) or the original payment plan is adjusted.

One-click: Request Proposal (1), Accept (1 each). If both accept, addendum signed automatically.

AI Authority: A1 for proposal generation (Groq); A2 for predictive outcome (XGBoost); A4 for addendum signing.

10.7 Smart FeeLock Management (Freeze & Partial Release)

10.7.1 Dispute Filing → FeeLock Freeze

When a dispute is filed:

The Governor freezes the affected FeeLock(s).

Government fees already paid are not refundable (by law).

Pending fee payments are frozen.

If the dispute involves only part of a shipment (e.g., 3 of 22 pallets), the system can partially release the undisputed portion for settlement.

10.7.2 Partial Release Criteria

One-click: If partial release is proposed, the non-disputing party clicks "Approve Release" (1). Governor releases the portion.

AI Authority: A3 for partial release approval (human escalation); A4 for freeze/unfreeze.

10.8 Document Authenticity Check (A2)

10.8.1 Purpose

If a party submits new evidence that contradicts previously uploaded documents, the document-authenticity agent (HF Donut + metadata analysis) cross-checks:

Metadata (dates, author, digital signatures)

Consistency of issuing body with original document

Any signs of tampering (hash comparison, pixel-level analysis)

10.8.2 Output

If inconsistency is detected, the flag is recorded in `dispute_recommendations.document_authenticity_flag`. The mediator sees a warning but the document is not automatically excluded.

Example Warning:

"The inspection report submitted by the buyer appears to be from a different surveyor than the original QC booking. Confidence 88%. May require external verification."

One-click: No action — automatic flagging. Mediator can click "View Details" to see the discrepancy.

AI Authority: A2 for authenticity check (HF Donut + ViT).

10.9 Third-Party Expert Invitation (Improvement #8)

10.9.1 Purpose

Disputes often benefit from independent surveyors, laboratories, or arbitrators. Any party in the mediation can click "Invite Third-Party Expert" — one click.

10.9.2 Workflow

10.9.3 Pre-Approved Expert List

One-click: Click "Invite" → select expert → send. Expert clicks link to view evidence (no login required, but token-based). Expert posts opinion — one click.

AI Authority: A3 for expert invitation and opinion posting.

10.10 Predictive Dispute Outcome (A2, XGBoost)

10.10.1 Trigger & Model

Trigger: After evidence is compiled, the system runs a predictive model trained on past SGTX dispute outcomes.

Input features:

Dispute type, commodity, incoterm, ports

Party trust scores and dispute history

Existence of contradictory evidence (e.g., origin QC pass vs. destination damage)

Contractual strength of the complaining clause

QC override patterns (frequency, reasons)

Causal analysis output (if available)

10.10.2 Output

Estimated probability of filing party winning (0-100%)

Predicted damage amount (range)

Confidence interval

Groq-generated neutral, plain-language summary

Example:

"Estimated probability of buyer winning: 78%. Typical award range: \$1,200 — \$1,600."

One-click: No action — displayed in the mediation log as advisory. Both parties see the same prediction.

AI Authority: A2 for predictive modelling (XGBoost); A1 for Groq summary.

10.11 Escalation to International Arbitration

10.11.1 Trigger

If mediation fails after maximum rounds (default 5) or either party clicks "Escalate to Arbitration", the system prepares the case for formal arbitration.

10.11.2 Automated Case Filing (A2, A1)

10.11.3 Supported Arbitration Bodies

One-click: Any party clicks "Prepare Arbitration Case" (1). System generates the PDF. Download (1).

AI Authority: A2 for document assembly; A1 for Groq drafting.

10.12 SGTX Fee Dispute (Separate Workflow)

10.12.1 Purpose

A tenant may dispute the SGTX trade fee calculation (e.g., wrong dynamic rate applied, misclassification of commodity). This is a separate dispute type (FEE_DISPUTE).

10.12.2 Workflow

Time limit: Must be filed within 90 days of fee payment.

One-click: Tenant clicks "Dispute" → submit (1). Authority reviews and clicks "Approve Refund" (1) → refund processed automatically.

AI Authority: A2 for fee calculation analysis; A3 for multisig escalation.

10.13 QC Dispute Fast-Track (v11.0)

10.13.1 Purpose

If the dispute involves a QC report, the evidence package automatically includes a dedicated qc_overrides section. This section lists every AI finding that the inspector overrode.

10.13.2 Override Details

10.13.3 Example Override Section

json

```
{
```

```
"qc_overrides": [
```

```
{
```

```
"pallet_id": "OR020",
```

```

"original_ai": {
  "defect": "mould",
  "confidence": 88,
  "timestamp": "2026-04-14T14:20:00Z"
},
"inspector_override": {
  "classification": "bruising",
  "reason": "Dark spot is surface bruise, no mould under magnifier.",
  "timestamp": "2026-04-14T14:30:00Z"
},
"photo_hash": "sha256:abc123..."
}
]
}

```

One-click: No action — automatically included. Mediator sees the overrides highlighted.

AI Authority: A2 for extracting overrides; A3 for licence revocation (if patterns of bad overrides detected).

10.14 Trade Reliability Index (TRI) — Reputation Scoring

10.14.1 Purpose

The Trade Reliability Index (TRI) is a 0–1000 score that quantifies the trustworthiness and performance of any SGTX tenant (trader, logistics provider, financier, etc.). It replaces the previous 0–100 reputation score (migration: multiply by 10). The TRI is private — only the tenant sees their full breakdown; counterparties see only a status badge (e.g., "Advanced Trusted") unless explicit consent is given for sharing the raw score.

10.14.2 TRI Formula

text

$$\begin{aligned}
 \text{TRI} = & (\text{Settlement Reliability} \times 0.25) + \\
 & (\text{Compliance Health} \times 0.20) + \\
 & (\text{Documentation Quality} \times 0.15) + \\
 & (\text{Financing Performance} \times 0.20) + \\
 & (\text{Dispute Resolution} \times 0.20)
 \end{aligned}$$

Each component is a 0–1000 sub-score, derived from raw metrics (see 10.14.4). The weights sum to 1.0. Recalculation occurs daily via a cron job.

10.14.3 Component Definitions & Raw Metrics

10.14.4 Sub-score Calculation (0–1000)

Settlement Reliability:

text

$$\text{score} = \min(1000, (\text{on_time_payment_percent} \times 8) + \max(0, (500 - \text{avg_delay_days} \times 20)))$$

on_time_payment_percent = 0–100

avg_delay_days = average delay (capped at 25 days)

Compliance Health:

Start at 1000, subtract:

200 for any sanctions hit (unless resolved and enhanced DD completed)

100 for any PEP hit (unless explicit approval)

50 per jurisdiction in RESTRICTED tier

200 if KYB tier < 2

10 per SAR filed (up to 300)

Minimum score 0.

Documentation Quality:

text

score = (first_time_acceptance_rate × 8) + max(0, 200 - (missing_docs_per_trade × 40))

first_time_acceptance_rate = 0–100

missing_docs_per_trade = average (capped at 10)

Financing Performance:

text

score = max(0, 1000 - (default_count × 300) - (late_repayment_rate × 500))

late_repayment_rate = % of financed amount repaid >30 days late (capped at 100%)

Dispute Resolution:

text

score = (no_arbitration_rate × 5) + max(0, 500 - (avg_resolution_days × 5))

no_arbitration_rate = % of disputes resolved without arbitration (0–100)

avg_resolution_days = capped at 100 days

10.14.5 TRI Classifications & Privileges

Privileges are applied automatically by the Governor (A4) when evaluating trade, financing, or customs actions.

10.14.6 Trust Confidence Score

Purpose: Indicates how statistically reliable the TRI is. A high TRI with low confidence means "not enough data yet".

text

Confidence = min(100,

$(\sqrt{\text{trade_count}} \times 5) +$

$(\sqrt{\text{total_volume_usd} / 10000} \times 3) +$

$(\text{history_months} / 36 \times 15) +$

$(\text{jurisdiction_count} \times 2) +$

(financier_count × 1)

)

Factor Caps & Maximums:

Display: Shown alongside TRI: "TRI: 874 (Confidence: 97%)"

If confidence < 50%, a warning appears: "Limited history — score may not be predictive."

10.14.7 Example Calculation

Tenant: Strawberry Export Co.

Trades: 50 → $\sqrt{50} \times 5 = 35.35$ points towards confidence

Volume: \$5M → $\sqrt{(5,000,000/10,000)} \times 3 = \sqrt{500} \times 3 = 67.08$ → capped at 30

History: 18 months → $18/36 \times 15 = 7.5$

Jurisdictions: 3 → $3 \times 2 = 6$

Financiers: 2 → $2 \times 1 = 2$

Confidence = $\min(100, 35.35 + 30 + 7.5 + 6 + 2) = \min(100, 80.85) = 81\%$

Component scores (0–1000):

Settlement: 970

Compliance: 920

Documentation: 880

Financing: 950 (no defaults, 2 late payments)

Dispute: 890

TRI = $(970 \times 0.25) + (920 \times 0.20) + (880 \times 0.15) + (950 \times 0.20) + (890 \times 0.20)$

= $242.5 + 184 + 132 + 190 + 178 = 926.5$ → 927 (Premier Trusted)

Display: "TRI: 927 (Confidence: 81%) — Premier Trusted"

10.14.8 Privacy & Sharing

10.14.9 TRI Update & Storage

Frequency: Daily, at 02:00 UTC.

Process: Cron job reads raw metrics from the past 12 months (or lifetime if <12 months) and computes sub-scores.

Storage: Stored in tenants.trust_score (INT, 0–1000) and a new table tri_history for trend analysis.

Retention: History kept for 7 years.

10.15 API Endpoints for Disputes

All endpoints require Governor-gated authentication.

10.16 Database Schema Additions (Part 10)

sql

-- Disputes (core)

CREATE TABLE disputes (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),


```

ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,
financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,
filing_party_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
dispute_category TEXT NOT NULL, -- 'QUALITY', 'DELAY', 'NON_PAYMENT', etc.
description TEXT NOT NULL,
remedy_sought TEXT,
status TEXT DEFAULT 'FILED', -- 'FILED', 'MEDIATION', 'SETTLED', 'ARBITRATION_PENDING',
'ARBITRATION_COMPLETE', 'CLOSED'
mediation_log JSONB DEFAULT '[]',
arbitration_case_id TEXT,
resolution_contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
ai_settlement_proposal JSONB,
predicted_outcome JSONB,
document_authenticity_flags JSONB DEFAULT '[]',
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
filed_at TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Dispute recommendations (AI-generated)
CREATE TABLE dispute_recommendations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
dispute_id UUID NOT NULL REFERENCES disputes(id) ON DELETE CASCADE,
ai_prediction TEXT,
suggested_settlement JSONB,
confidence DECIMAL(5,4),
shap_values JSONB,
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Evidence packages
CREATE TABLE evidence_packages (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
dispute_id UUID NOT NULL REFERENCES disputes(id) ON DELETE CASCADE,
package JSONB NOT NULL,

```

```

loom_hash TEXT NOT NULL,
verification_token TEXT UNIQUE,
compiled_at TIMESTAMPTZ DEFAULT NOW()
);

-- Causal attribution
CREATE TABLE causal_attribution (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
entity_id UUID NOT NULL, -- dispute_id or shipment_id
entity_type TEXT NOT NULL, -- 'DISPUTE', 'SHIPMENT'
factor TEXT NOT NULL,
contribution DECIMAL(5,4) NOT NULL,
confidence_interval JSONB,
description TEXT,
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Third-party experts
CREATE TABLE dispute_experts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
dispute_id UUID REFERENCES disputes(id) ON DELETE CASCADE,
expert_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,
expert_type TEXT NOT NULL, -- 'surveyor', 'laboratory', 'arbitrator', 'legal'
invitation_token TEXT UNIQUE,
opinion TEXT,
posted_at TIMESTAMPTZ,
status TEXT DEFAULT 'INVITED', -- 'INVITED', 'ACCEPTED', 'DECLINED', 'POSTED'
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Trade Reliability Index (TRI) history
CREATE TABLE tri_history (
id BIGSERIAL PRIMARY KEY,
tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
tri_score INTEGER NOT NULL,
confidence DECIMAL(5,2) NOT NULL,
component_scores JSONB NOT NULL, -- {settlement, compliance, docs, financing, dispute}

```

```

calculated_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_tri_history_tenant_date ON tri_history(tenant_gtid, calculated_at DESC);
-- TRI disputes
CREATE TABLE tri_disputes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
dispute_reason TEXT NOT NULL,
supporting_evidence JSONB,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'UNDER_REVIEW', 'RESOLVED', 'REJECTED'
tri_before INTEGER,
tri_after INTEGER,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ
);
-- Indexes
CREATE INDEX idx_disputes_ustn ON disputes(ustn);
CREATE INDEX idx_disputes_status ON disputes(status);
CREATE INDEX idx_disputes_filing_party ON disputes(filing_party_gtid);
CREATE INDEX idx_disputes_category ON disputes(dispute_category);
CREATE INDEX idx_dispute_recommendations_dispute ON dispute_recommendations(dispute_id);
CREATE INDEX idx_evidence_packages_dispute ON evidence_packages(dispute_id);
CREATE INDEX idx_evidence_packages_token ON evidence_packages(verification_token);
CREATE INDEX idx_causal_attribution_entity ON causal_attribution(entity_id, entity_type);
CREATE INDEX idx_dispute_experts_dispute ON dispute_experts(dispute_id);
CREATE INDEX idx_tri_disputes_tenant ON tri_disputes(tenant_gtid);

```

10.17 Implementation Checklist (Part 10)

10.17.1 Core Dispute Infrastructure

Implement dispute-service (Rust, Axum).

Build dispute filing endpoint (voice and structured) with category selection.

Create disputes table and state machine (FILED → MEDIATION → SETTLED → etc.).

Implement FeeLock freeze on dispute filing.

Implement partial FeeLock release workflow (A3 approval).

10.17.2 Evidence & Causation

Build evidence autocompiler (collects from multiple services, Loom-hashes, stores in evidence_packages).

Integrate causal inference engine (Add-on 3) — DoWhy + EconML.

Add document authenticity check (HF Donut + ViT).

Implement third-party expert invitation workflow (pre-approved list, secure link, opinion posting).

10.17.3 Mediation & Settlement

Implement mediation log UI with Groq translation and structured offers.

Add sentiment analysis (A2) with cooling-off period suggestion.

Build AI settlement proposal generator (Groq + XGBoost predictor).

Implement settlement addendum generation and signature collection.

10.17.4 Arbitration

Build arbitration case preparation (template filling, Groq narrative, PDF generation).

Support multiple arbitration bodies: ICC, DIFC-LCIA, CRCICA, LCIA, UNCITRAL.

10.17.5 Fee & TRI Disputes

Implement SGTX fee dispute workflow with escalation to multisig.

Build TRI dispute workflow (dispute calculation → review → resolution).

Add TRI recalculation on dispute resolution.

10.17.6 QC Fast-Track

Automatically flag QC overrides in evidence package.

Build QC dispute fast-track with override comparison table.

Add inspector licence revocation workflow (A3) for bad overrides.

10.17.7 Reputation Engine (TRI)

Create tri_history table.

Implement daily TRI calculation cron job (Rust).

Build TRI display in Company Admin and Trade Command Center.

Implement TRI sharing with explicit consent (verifiable credential).

Add TRI status badge in GTID resolution.

10.17.8 Testing

Test full dispute workflow: file → evidence compiled → mediation → settlement → addendum signed → TRI updated.

Test QC dispute fast-track with overrides flagged.

Test causal inference integration.

Test third-party expert invitation and opinion posting.

Test arbitration case preparation.

Test SGTX fee dispute and refund.

Test TRI calculation and update.

Test TRI dispute and resolution.

10.18 AI Authority Summary for Part 10

END OF PART 10 — DISPUTE MANAGEMENT & REPUTATION ENGINE (v11.0 FULLY EXPANDED)

PART 11 — SEVEN CRITICAL ADD-ONS

11.0 Overview

The SGTX platform is designed to be extensible while remaining non-custodial, zero-cost, and sovereign. Beyond the core trade execution engine, seven advanced add-ons are available to provide next-generation risk management, privacy, resilience, and security. Each add-on is optional, can be toggled independently via the Admin Portal, and requires no additional billing — all components are open-source and run on the same bare-metal infrastructure. None of these add-ons introduce marketplace features; they only enhance the platform's intelligence, security, and operational robustness.

Add-on Summary (Non-Marketplace, Orchestration-Only):

All add-ons are pre-integrated into the microservices architecture and can be activated with a single toggle (and, where required, multisig approval). This part provides the complete specification for each add-on, including activation workflow, data flow, security considerations, and performance impact.

AI Integration in Part 11:

11.1 ADD-ON 1 — GRAPH NEURAL NETWORK (GNN) RISK ENGINE & INSTITUTIONAL TRADE GRAPH

11.1.1 Purpose

The Graph Neural Network (GNN) Risk Engine serves two complementary purposes:

1. Sanctions & Adversarial Risk Detection — Detects indirect sanctions links, hidden money-laundering patterns, and adversarial relationships (e.g., a seller's Ultimate Beneficial Owner within 2 hops of a sanctioned entity).
2. Institutional Trade Graph (Trust-Based Mapping) — Maps positive, trust-based relationships (trade frequency, financing relationships, co-compliance, supply chain dependencies) to strengthen credit assessments, financing decisions, and network trust scores.

All outputs are advisory or constraining (A2/A4) — they never recommend counterparties or autonomous actions.

11.1.2 Architecture & Integration

The GNN service is implemented as a Rust + PyTorch Geometric microservice (or a Python sidecar with a Rust gRPC wrapper). It exposes two main endpoints:

`/v1/gnn/risk` — for sanctions and adversarial risk (called by Governor on `trade.initiate` and `financing.request`).

`/v1/gnn/trust` — for institutional trust graph queries (called on demand by Financier Portal, Trust Passport, and Portfolio views).

Model Architecture:

11.1.3 Sanctions & Adversarial Risk Output

When called by the Governor, the GNN returns:

json

{

"sanctions_proximity": 2,

```
"graph_risk_score": 85,  
"explanation": "Seller's UBO (Person X) is 2 hops from sanctioned entity Y via Company Z.",  
"model_version": "v2.1",  
"inference_time_ms": 120  
}
```

Governor Integration: If $\text{sanctions_proximity} \leq 2$ OR $\text{graph_risk_score} \geq 90$, the Governor returns CONDITIONAL with a Decision Panel explanation: "This trade involves a party whose UBO is within 2 hops of a sanctioned entity. Enhanced due diligence required." The decision never suggests alternative counterparties.

11.1.4 Institutional Trade Graph (Trust-Based Mapping)

Purpose: Map positive relationships while preserving privacy using ZK proofs. Used by:

Financier Portal (credit assessment, portfolio risk)

Trust Passport (network trust score)

Dispute evidence (indirect relationship patterns)

Edge Types & Weights:

Output for Participant (Ego Network Only):

```
json  
{  
  "direct_connections": 47,  
  "trusted_connections": 12,  
  "network_trust_score": 82,  
  "indirect_exposure": "2 hops to 3 financial institutions",  
  "privacy_notice": "All counts are anonymised; no counterparty identities revealed."  
}
```

Display Locations:

Financier Portal — as an optional card in "Portfolio & Compliance"

Trust Passport (Part 2.10) — as an optional dimension (opt-in)

Company Admin → Network — for traders (aggregated, no identifiers)

Consent & Opt-Out: Tenants can disable inclusion in the trust graph via Company Admin → Privacy → Trade Graph Visibility (off by default). Disabling does not affect sanctions detection.

11.1.5 Data Model Additions

```
sql  
-- GNN risk scores (adversarial)  
CREATE TABLE gnn_risk_scores (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  entity_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  sanctions_proximity INTEGER,
```

```

graph_risk_score INTEGER,
model_version TEXT,
inference_at TIMESTAMPTZ,
explanation TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Institutional trade graph edges (anonymised, permissioned)

```

CREATE TABLE trade_graph_edges (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
from_entity_anon_id TEXT NOT NULL, -- persistent anonymised ID per tenant
to_entity_anon_id TEXT NOT NULL,
edge_type TEXT NOT NULL, -- 'TRADE', 'FINANCING', 'COMPLIANCE', 'SUPPLY'
weight DECIMAL(5,2),
first_seen TIMESTAMPTZ,
last_seen TIMESTAMPTZ,
zk_proof_hash TEXT, -- optional, for verifiable privacy
consent_granted BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Materialised view for network trust scores

```

CREATE MATERIALIZED VIEW network_trust_scores AS
SELECT
entity_gtid,
COUNT(DISTINCT to_entity_anon_id) AS direct_connections,
SUM(weight) AS weighted_trust_sum,
CURRENT_TIMESTAMP AS refreshed_at
FROM trade_graph_edges
GROUP BY entity_gtid;

```

11.1.6 Zero-Cost Implementation

PyTorch Geometric and PyTorch are open-source, run on CPU/GPU on the sovereign node.

Graph data already exists in corporate graph tables (entity_nodes, relationship_edges), trade tables, and financing tables.

Weekly training uses off-peak compute. No external API calls.

ZK proofs for trust graph queries are generated client-side (WASM) using Plonky3 (Add-on 7), incurring zero server cost.

11.1.7 Activation & Configuration

Admin Portal → Add-Ons → GNN Risk Engine & Trust Graph

Fallback: If GNN service is unavailable, Governor falls back to rule-based risk scoring (no block). Trust graph queries return "unavailable" without error.

11.1.8 AI Authority Summary

11.2 ADD-ON 2 — FEDERATED LEARNING NETWORK

11.2.1 Purpose

Train machine learning models across multiple sovereign nodes without exchanging raw trade data. Each node trains a local model on its own data (e.g., fraud detection, margin estimation, credit scoring), sends only encrypted model updates (gradients) to a secure aggregator, and receives a global model. This improves model accuracy for corridors with limited local data while preserving data sovereignty. The system never uses this to rank or recommend counterparties.

11.2.2 Architecture & Integration

Trained Models:

11.2.3 Data Model Additions

sql

-- Federated learning model registry

```
CREATE TABLE federated_learning_models (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  model_name TEXT NOT NULL,  
  version INTEGER,  
  global_model_hash TEXT,  
  aggregator_node_gtid TEXT,  
  round_number INTEGER,  
  accuracy_validation DECIMAL(5,4),  
  deployed_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Local training metadata per node

```
CREATE TABLE local_training_metadata (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  node_gtid TEXT NOT NULL,  
  model_name TEXT NOT NULL,  
  round_number INTEGER,  
  local_accuracy DECIMAL(5,4),  
  data_size INTEGER,  
  training_duration_seconds INTEGER,
```



```
uploaded_at TIMESTAMPTZ,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

11.2.4 Activation

Admin Portal → Add-Ons → "Federated Learning" → Set coordinator node → Choose models to train → Enable. Requires multisig approval. After activation, the coordinator will orchestrate the first training round within 24 hours.

One-click: Admin clicks "Enable" → system initiates first training round.

AI Authority: A2 for local training; A3 for model approval; A4 for distribution.

11.3 ADD-ON 3 — CAUSAL INFERENCE ENGINE

11.3.1 Purpose

Identify root causes of disputes, delays, defaults, or quality failures — not just correlations. For example, "68% of the delay was due to a port strike, 22% due to carrier rerouting, 10% due to customs documentation." This provides actionable insights for insurers, arbitrators, and traders — but never suggests "blame the other party" or "switch provider". It is purely factual.

11.3.2 Architecture & Integration

11.3.3 Data Model Additions

```
sql  
  
-- Causal attribution results  
  
CREATE TABLE causal_attribution (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  entity_id UUID NOT NULL, -- dispute_id or shipment_id  
  entity_type TEXT NOT NULL, -- 'DISPUTE', 'SHIPMENT'  
  factor TEXT NOT NULL,  
  contribution DECIMAL(5,4) NOT NULL,  
  confidence_interval JSONB,  
  description TEXT,  
  model_version TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

11.3.4 Activation

Admin Portal → Add-Ons → "Causal Inference Engine" → Enable. Optional: set auto-trigger thresholds (e.g., "run on any delay >24h"). The engine will process pending disputes within 5 minutes of filing.

One-click: Admin clicks "Enable" → automatic processing begins.

AI Authority: A2/A3 for causal inference; A1 for Groq summary.

11.4 ADD-ON 4 — SELF-HEALING INFRASTRUCTURE & CHAOS ENGINEERING

11.4.1 Purpose

Ensure platform resilience by automatically healing failed pods, predicting hardware failures, and regularly proving system robustness through chaos experiments. The platform can survive node failures, network partitions, and disk outages without manual intervention — all while maintaining non-custodial properties.

11.4.2 Architecture & Integration

Chaos Experiment Types:

11.4.3 Data Model Additions

sql

-- Infrastructure predictions

```
CREATE TABLE infrastructure_predictions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  node_gtid TEXT NOT NULL,  
  component TEXT NOT NULL,  
  predicted_failure_probability DECIMAL(5,4),  
  estimated_failure_date TIMESTAMPTZ,  
  action_taken TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Chaos experiments

```
CREATE TABLE chaos_experiments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  experiment_name TEXT NOT NULL,  
  namespace TEXT NOT NULL,  
  status TEXT, -- 'RUNNING', 'SUCCEEDED', 'FAILED'  
  started_at TIMESTAMPTZ,  
  finished_at TIMESTAMPTZ,  
  groq_summary TEXT,  
  logs_url TEXT,  
  created_by_gtid TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

11.4.4 Activation

Admin Portal → Add-Ons → "Self-Healing & Chaos" → Enable. Set chaos schedule (default: Sunday 03:00 UTC, staging only). To enable production chaos, a separate multisig approval is required. The self-healing agent is always on once enabled.

One-click: Admin clicks "Enable" → system activates. Admin clicks "Run Chaos Test" → experiment begins.

AI Authority: A2 for LSTM prediction; A1 for post-mortem generation; A4 for orchestration.

11.5 ADD-ON 5 — AUTOMATED PENETRATION TESTING

11.5.1 Purpose

Run weekly vulnerability scans against the SGTX staging environment (and optionally production with read-only scans). Findings are automatically categorised, prioritised, and, if critical, block the CI/CD pipeline. This ensures that no vulnerability reaches production. The scans never target tenant data; they focus on platform APIs and infrastructure.

11.5.2 Architecture & Integration

11.5.3 Data Model Additions

sql

-- Threat findings

```
CREATE TABLE threat_findings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tool_name TEXT NOT NULL,  
  severity TEXT NOT NULL, -- 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'  
  title TEXT NOT NULL,  
  description TEXT,  
  remediation TEXT,  
  cve_id TEXT,  
  affected_component TEXT,  
  finding_hash TEXT,  
  resolved BOOLEAN DEFAULT false,  
  resolved_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

11.5.4 Activation

Admin Portal → Add-Ons → "Automated Pentesting" → Configure target URLs, schedule, and severity thresholds for blocking. Enable. The first scan will run within the next scheduled window.

One-click: Admin clicks "Enable" → automated scanning begins. Admin clicks "Trigger Pentest" → manual scan starts immediately.

AI Authority: A1 for Groq-generated summaries; A4 for CI/CD integration.

11.6 ADD-ON 6 — POST-QUANTUM CRYPTOGRAPHY (PQC)

11.6.1 Purpose

Protect long-term records (contracts, Loom hash chains, identity credentials, audit logs) against future quantum computer attacks. While Ed25519 (used for daily signatures) is secure today, it is not quantum-safe. The add-on adds a second layer of Dilithium3 signatures (NIST standard) and optionally Kyber1024 for key exchange. This does not affect daily operations; the platform continues to use Ed25519 for performance, while archival stores also hold a Dilithium3 signature.

11.6.2 Architecture & Integration

11.6.3 Data Model Additions

sql

```
ALTER TABLE governor_decisions ADD COLUMN pqc_signature TEXT;
```

```
ALTER TABLE contracts ADD COLUMN pqc_signature TEXT;
```

```
ALTER TABLE shipments ADD COLUMN pqc_signature TEXT;
```

11.6.4 Activation

Admin Portal → Add-Ons → "Post-Quantum Cryptography" → Generate key pair (multisig required) → Enable. The background re-signer will process existing records within 7 days.

One-click: Admin clicks "Enable" → key generation ceremony initiated. Admin clicks "Re-sign All" → forces immediate re-signing of all existing records.

AI Authority: A3 for key generation approval; A4 for background re-signing.

11.7 ADD-ON 7 — EXPANDED ZERO-KNOWLEDGE PROOFS (ZK) & PROOF OF RESERVES

11.7.1 Purpose

Zero-Knowledge Proofs (ZK) enable privacy-preserving verification without revealing underlying sensitive data. SGTX implements four core use cases, plus a phased proof-of-reserves mechanism:

1. Financier proof-of-reserves — A financier proves they hold sufficient liquidity to back a financing offer without revealing wallet balance, address, or chain.
2. Confidential trade terms — Buyer and seller agree on a price hidden from the platform; SGTX verifies only that the price is within legal and market bounds.
3. Private settlement — A buyer proves they settled a payment without revealing the exact amount or PSP used.
4. Proof of Reserves (phased) — Initially via Big Four attestation, later via real-time ZK proofs, ensuring platform solvency and backing of obligations.

All ZK proofs are optional and client-side generated (WASM). The platform never forces their use.

11.7.2 Architecture & Integration

Technology Stack:

Use Case 1 — Financier Proof-of-Reserves:

Before submitting a bid, a financier (Bank or PFI) can optionally generate a ZK proof that their total stablecoin or fiat reserves exceed the offered amount.

Workflow:

Financier connects their wallet(s) or bank account(s) via read-only API (or manually uploads a Merkle tree of balances).

Client-side script (or Financier Portal WASM module) generates a ZK proof that the sum of selected reserves $\geq$ amount_offered.

Proof is sent to SGTX along with the bid.

Governor verifies the proof; if valid, the bid proceeds (subject to other checks).

No wallet address, exact balance, or chain is revealed.

Use Case 2 — Confidential Trade Terms:

Seller can hide the EXW price and logistics breakdown from the platform; buyer sees only the final quoted price (including SGTX fee).

Workflow:

Seller toggles "Make price confidential" in Quote Submission.

System computes a ZK proof that (EXW + logistics + SGTX fee + broker fees) is within a plausible range (e.g., not zero, not exceeding market average by >300%).

The proof is stored in seller_quotes.price_zk_proof; the raw price components are not stored.

Buyer sees only the final total price.

During dispute resolution, the proof can be revealed to an arbitrator if needed.

Use Case 3 — Private Settlement:

After settlement, the payer can request a ZK proof of payment without revealing the exact amount or PSP used.

Workflow:

Buyer clicks "Request Private Settlement Proof" in the Settlement tab.

Client-side script generates a proof from the PSP receipt and settlement instruction, proving that a transfer of at least the contractual amount occurred to the correct beneficiary.

Proof is stored in settlement_confirmations.zk_proof.

The buyer can share the proof with auditors or regulators; they verify it without seeing sensitive details.

Use Case 4 — Proof of Reserves (Phased Approach):

Phase 1 — Big Four Attestation (Initial, Months 1–12):

Phase 2 — Real-time ZK Proofs (Mature, after >80% financing volume uses ZK-capable wallets):

11.7.3 Data Model Additions

sql

-- ZK proofs for various use cases

ALTER TABLE financing_offers ADD COLUMN reserve_proof TEXT;

ALTER TABLE seller_quotes ADD COLUMN price_zk_proof TEXT;

ALTER TABLE settlement_confirmations ADD COLUMN zk_proof TEXT;

-- Platform reserves tracking

CREATE TABLE platform_reserves (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

reporting_date DATE NOT NULL,

phase TEXT CHECK (phase IN ('ATTESTATION', 'ZK_PROOF')),

asset_total DECIMAL(20,2),

liability_total DECIMAL(20,2),

ratio DECIMAL(6,2),

attestation_pdf_url TEXT,

zk_proof TEXT,

```

verified BOOLEAN DEFAULT false,
verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Reserve ratio history
CREATE TABLE reserve_ratio_history (
id BIGSERIAL PRIMARY KEY,
ratio DECIMAL(6,2) NOT NULL,
checked_at TIMESTAMPTZ DEFAULT NOW()
);

```

11.7.4 Technical Specifications (Plonky3)

11.7.5 Integration with Other Components

11.7.6 Governance & Reserve Rules

Constitutional rule (reserve_rules.wasm, Part 1.3.6) requires reserve ratio $\geq 110\%$ at all times.

If daily check detects ratio $< 110\%$ for > 24 hours, Governor:

Freezes new trade creation

Notifies CBE and Platform Governance Authority

Triggers P0 incident

Attestation reports and ZK proofs are Loom-hashed for tamper-evident audit.

11.7.7 Activation

Admin Portal → Add-Ons → Zero-Knowledge Proofs — Expanded

Enable each sub-feature independently:

- Reserve Proofs (financiers)
- Confidential Trade Terms (sellers)
- Private Settlement (buyers)

Activation steps:

Admin toggles desired feature.

System compiles the corresponding Plonky3 circuit (one-time, takes ~30 seconds).

Circuit is cached and served to clients as WASM module.

Feature becomes available in respective portals.

Multisig required? No — circuit compilation is automated and auditable.

Zero-cost: All tooling is open-source (Plonky3, WASM toolchain). Client-side proof generation uses user's device; server verification is lightweight.

One-click: Admin clicks "Enable" → circuits compiled → features available.

11.8 Activation Workflow (Admin Portal) — Non-Marketplace

Important: None of these add-ons introduce marketplace features. They only enhance security, risk detection, privacy, and resilience. The platform never uses them to recommend counterparties or service providers.

AI Authority: A4 for activation; A1 for notification.

11.9 Interaction Between Add-ons

AI Authority: A2 for cross-add-on optimisation; A4 for orchestration.

11.10 Implementation Checklist (Part 11)

11.10.1 GNN Risk Engine

Deploy gnn-risk-service (Rust + PyTorch Geometric).

Train initial model on corporate graph data.

Integrate with Governor (sync call on trade.initiate, financing.request).

Build Institutional Trade Graph with anonymisation and ZK proofs.

Add Financier Portal widget for network trust score.

Implement opt-out toggle in Company Admin.

11.10.2 Federated Learning

Deploy fed-learning-coordinator (Rust + OpenFL/Flower).

Configure participant nodes with local-trainer sidecar.

Set up weekly training schedule (off-peak).

Implement model drift monitoring.

Add Admin Portal status dashboard.

11.10.3 Causal Inference

Deploy causal-engine (Python, DoWhy + EconML).

Define causal graph (DAG) for trade disputes.

Integrate with dispute filing and milestone breach triggers.

Store results in causal_attribution table.

Add Groq summary generation.

11.10.4 Self-Healing & Chaos

Install Chaos Mesh on K3s cluster.

Deploy chaos-orchestrator (Rust).

Deploy node-health-agent (Rust) with LSTM prediction.

Configure weekly chaos experiments (staging).

Build Admin Portal "Infrastructure Health" dashboard.

Add production chaos toggle (multisig-gated).

11.10.5 Automated Pentesting

Set up pentest-service (Rust).

Configure Trivy, OWASP ZAP, nuclei, OpenVAS.

Schedule weekly scans (Friday 23:00 UTC).

Integrate with CI/CD (critical findings block deployment).

Build Admin Portal "Security" tab.

11.10.6 Post-Quantum Cryptography

Generate Dilithium3 key pair (multisig ceremony).

Implement dual-signature (Ed25519 + Dilithium3).

Deploy pqc-resigner background job.

Add PQC verification endpoint (/v1/verify/pqc).

Publish public key at /.well-known/sgtx-keys.

11.10.7 ZK Proofs & Proof of Reserves

Integrate Plonky3 and compile circuits for three use cases.

Add client-side WASM modules for proof generation.

Implement server-side verification endpoint.

Add UI toggles in Financier Portal, Seller Quote, Settlement.

Establish Phase 1 — Big Four attestation process.

Build daily liability aggregation.

Implement Phase 2 — HSM-based ZK proof generation.

Build reserve dashboard in Admin and Government Portals.

Update constitutional WASM module with reserve ratio rule.

11.10.8 Testing

Simulate federated learning round across nodes.

Trigger chaos experiment and verify auto-healing.

Run automated pentest and verify CI/CD block.

Verify ZK proof generation and verification.

Test GNN sanctions proximity (≤ 2 hops $\rightarrow$ CONDITIONAL).

Test causal inference on dispute filing.

Test PQC signature generation and verification.

Test reserve ratio enforcement ($< 110\%$ $\rightarrow$ freeze trades).

11.11 AI Authority Summary for Part 11

END OF PART 11 — SEVEN CRITICAL ADD-ONS (v11.0 FULLY EXPANDED)

Part 12 — Portals, Roles, Dashboards & Workflows

PART 12A — COMMON COMPONENTS (SHARED ACROSS ALL PORTALS)

12A.0 Overview & Design Philosophy

All SGTX portals share a common set of core components. These components provide the unified user experience, real-time updates, AI-assisted guidance, and non-marketplace safeguards that define the platform. Each component is designed with the one-click principle — every irreversible action is a single

click or voice command — and is fully accessible, responsive, and integrated with the Governor and Loom audit trail.

Icons used in this part:

Design Principles Across All Components:

12A.1 Smart Inbox with Recommended Actions Widget

12A.1.0 Purpose & Design Philosophy

The Smart Inbox is the default landing page for every authenticated user across all portals. It shows a prioritised action feed that tells the user exactly what to do next, with a Recommended Actions Widget at the top for one-click execution of the single most important task.

Core principles:

Action-first, never information-first — users see only what requires immediate attention.

Trade-centric — items link to USTNs, financing agreements, or distressed lots.

Role-adaptive — items are derived from the user's permissions, trader mode (and active dual-mode context: Buyer or Seller), and open obligations.

Prioritised — urgency score computed deterministically (A4) using rules that incorporate deadlines, SLAs, and risk signals. AI (Groq/Ollama) generates the title and description after the score is known.

Dismissable & snoozable — users can snooze (2h, 4h, 8h, 24h) or permanently dismiss items; actions sync across devices via NATS.

Non-marketplace — never suggests unknown counterparties or unsolicited trades.

Multi-shipment aware — items for multi-shipment contracts appear per shipment when that shipment is pending action.

Financing aware — financiers see bids pending, margin calls, repayment reminders, and co-financing opportunities.

Distressed aware — sellers see outreach responses, buyer offers; buyers see distressed listings from saved contacts.

Dual-mode aware — when the toggle is set to Buyer, the inbox shows only buyer-centric items; switching to Seller immediately refreshes and shows seller-centric items.

AI Integration:

12A.1.1 Functional Description

12A.1.1.1 Inbox Item Structure

Each `inbox_items` record (database) has the following fields, used to render the UI:

12A.1.1.2 Urgency Scoring Model (Deterministic, A4)

The inbox-service computes a base score using rules and then applies role-specific modifiers. Scores are recalculated whenever the underlying state changes (e.g., deadline approaches, new offer received). The scoring rules are hot-reloadable via OPA policy snippets.

Master Scoring Table (Baseline):

Role-specific modifiers (added to baseline):

Snooze & Dismiss:

Users can snooze an item for 2h, 4h, 8h, or 24h.


```

"high_priority": [
{
  "item_id": "...",
  "ustn": "SGTX-EG-...-V1",
  "category": "NEEDS_SIGNATURE",
  "priority_score": 100,
  "title": "Sign contract before 14:00 UTC",
  "description": "Your trade with Mekong Fresh will be cancelled if not signed within 3h.",
  "deadline": "2026-06-14T14:00:00Z",
  "action_link": "/trade/.../contract/sign",
  "snoozed_until": null,
  "dismissed": false,
  "created_at": "2026-06-14T10:20:00Z"
}
],
"medium_priority": [ ... ],
"low_priority": [ ... ],
"ai_summary": "You have 2 high-priority items..."
}

```

12A.1.3 Implementation Notes for Developers

12A.2 Trade Command Center (TCC)

12A.2.0 Purpose & Design Philosophy

The Trade Command Center (TCC) is a single, real-time page that consolidates everything about a specific trade — or a specific shipment within a multi-shipment contract, or a financing agreement, or a distressed micro-contract — into one scrollable, context-rich view. It is accessible from any USTN badge, financing agreement ID, or distressed microUSTN link (from Smart Inbox, Shipments Vault, notifications, or emails).

Core principles:

One page, one entity — all dimensions (trade, shipment, financing, distressed) available via anchored sections and collapsible panels.

Timeline-driven — horizontal timeline showing the entity's journey from initiation to completion.

Action-oriented — prominent pending action panel with a clear call-to-action button, always passing through the Governor.

Privacy-respecting visibility — displayed data filtered by the logged-in tenant's permissions, trader mode context (Buyer or Seller), and financing roles.

Multi-entity aware — supports trade, financing, distressed, and multi-shipment sub-entities.

All data is fetched from a single endpoint — `/v1/trade/{ustn}/command-center` (or similarly for financing and distressed), which returns aggregated, permission-filtered JSON.

AI Integration:

12A.2.1 Layout & Sections (for a Trade / Shipment)

The TCC is organised into several anchored sections, all on a single page. The exact sections shown vary based on the entity type (trade, financing, distressed) and the user's active mode (Buyer, Seller, Financier, Logistics).

A. Persistent Header

B. Trade Progress Timeline (Horizontal)

A horizontal, colour-coded bar displays the classic trade phases:

For multi-shipment: Each shipment has its own mini-timeline below the master timeline. Clicking a shipment's timeline segment opens that shipment's detailed view.

For financing: Requested → Bidding → Awarded → Active → Repaid

For distressed: Declared → Assessed → Listed → Outreach → Offer Accepted → Microcontract Locked → Completed

C. Pending Action Panel (Most Visually Prominent)

A coloured card (amber for standard actions, red for urgent) that displays:

D. Summary Cards (Grid of Expandable Cards)

A responsive grid (2-3 columns) of expandable cards, each representing a key dimension. Cards are greyed out if the user lacks permission.

D1. Commercial Terms Card:

D2. Parties Card:

D3. Shipment Status Card:

D4. Documents Card:

D5. Finances Card:

D6. Risk & Compliance Card:

D7. Logistics Card:

D8. QC Card (if applicable):

D9. Multi-shipment Schedule Card:

D10. Distressed Information Card:

E. Activity Feed (Vertical Timeline)

A vertically scrolling list, chronologically ordered, of every significant event in the entity's lifecycle.

text

2026-06-10 08:30 -- Buyer created trade request

2026-06-10 09:15 -- Seller accepted request

2026-06-11 14:20 -- Seller submitted quote (\$30,240, including SGTX fee \$360)

2026-06-12 10:00 -- Buyer accepted quote

2026-06-12 10:05 -- Contract signed by seller (SGTX fee paid)

2026-06-12 10:06 -- Contract locked, USTN generated

2026-06-13 08:00 -- Loading window confirmed

2026-06-14 06:30 -- All pallets scanned, container loaded

...

Each entry includes: timestamp, actor (employee name and tenant), and link to associated document or Governor decision.

F. Collaborative Trade Room (Expandable Panel)

A collapsible panel at the bottom of the TCC that provides real-time chat and a shared task board.

Built on NATS for chat and Yjs for the task board.

Messages can be tagged with offer IDs (for negotiation) or shipment numbers (for multi-shipment).

AI assistant (A1) can answer questions about the trade.

Permissions-filtered: logistics providers see only logistics-related channel.

12A.2.2 Backend Integration

Endpoint: GET /v1/trade/{ustn}/command-center (or /v1/financing/{id}/command-center, /v1/distressed/{id}/command-center)

Response Structure:

json

{

"ustn": "SGTX-...",

"trade_status": "LOCKED",

"phase_progress": { "current": "CONTRACT_SIGNED", "completed": ["INITIATED", "QUOTED"],

"upcoming": ["IN_EXECUTION"] },

"pending_action": {

"action": "Sign contract",

"deadline": "2026-06-14T14:00:00Z",

"why_it_matters": "Your signature will lock the contract and generate the USTN.",

"action_link": "/contract/sign",

"action_label": "Sign Now"

},

"summary_cards": {

"commercial": { "incoterm": "CFR", "total_price": 30240, ... },

"parties": { "buyer": {...}, "seller": {...} },

"shipment_status": { "vessel": "MSC FIONA", "eta": "2026-06-20T14:00:00Z", ... },

"documents": { "verified": 5, "missing": 1, "items": [...] },

"finances": { "settlement_status": "PENDING", "sgtx_fee_paid": true, ... },

"risk_compliance": { "risk_score": 12, "sanctions_cleared": true, ... },

"logistics": { "providers": [...], "release_status": "AUTHORISED" },

```

"qc": { "status": "COMPLETED", "verdict": "CONDITIONAL_PASS", "overrides": 1 },
"multi_shipment": { "shipments": [...], "progress": 2 },
"distressed": { "micro_ustn": null }
},
"activity_feed": [ ... ],
"trade_room": { "messages": [...], "tasks": [...] },
"permissions_map": { "can_view_commercial": true, "can_edit_logistics": false, ... }
}

```

Real-time Updates: Frontend subscribes to NATS topics (trade.{ustn}*). When an event arrives, tenant-experience-service sends a partial update via WebSocket.

12A.3 PlainLanguage Governor Decision Panel

12A.3.0 Purpose & Design Philosophy

Every Governor verdict that is not ALLOW must be translated instantly into a clear, human-readable message that explains why the action was blocked and what the tenant must do next. The PlainLanguage Governor Decision Panel is the universal, zero-jargon interface between the platform's constitutional enforcement layer and the human user.

The panel appears automatically whenever a tenant attempts an irreversible action and the Governor returns DENY or CONDITIONAL. It can also be opened on demand via a "What's blocking me?" button found in the TCC, Smart Inbox, and any workflow screen.

Core principles:

Zero Jargon — The tenant never sees "Governor", "OPA", "WasmEdge", "Loom", "A2". Phrases like "Your trade is on hold because..." are used.

Explain, then guide — Three parts: plain-language summary, bullet list of specific unmet conditions each with a status icon and clickable action, and a clear statement of what happens next.

Transparent authority — May indicate "automated compliance rules" or "human reviewer required", without revealing AI authority levels.

Realtime updates — If conditions are later resolved, the panel automatically refreshes via NATS events.

Dual-mode context — If the user's active trader mode is the reason for the block, the panel offers a single-click switch.

12A.3.1 Panel Structure

A. Header:

B. PlainLanguage Explanation:

A 2-3 sentence summary stored in governor_decisions.tenant_message. Generated by A1 (Groq → Ollama) at decision time.

Examples:

C. Condition Checklist:

A bullet list of specific unmet conditions from conditions JSONB and tenant_message.conditions:

Example Checklist:

text

Role-specific extra tabs — Customs Readiness (Buyer), Cost Breakdown (Seller), etc.

12A.4.5 Map View

When the view toggle is set to "Map", the table area is replaced by a Leaflet map.

12A.4.6 Bulk Actions

Available when at least one row is selected:

12A.4.7 Saved Filter Sets

Users can save the current combination of filters, sort column, sort order, and visible columns as a named view.

12A.4.8 Role-Specific Column Configurations

12A.4.9 API Endpoints

12A.5 VoiceCommandButton

12A.5.0 Purpose & Design Philosophy

Hands-free operation for milestone confirmations, navigation, and approvals — works offline. The VoiceCommandButton is a UI component with offline speech recognition (Vosk) + intent extraction (HF Mixtral).

Capabilities:

Speech-to-text: Vosk (offline models for English, Arabic, Vietnamese, German; fallback to Web Speech API online)

Intent extraction: A2 (HF Mixtral, local) — extracts action, entity, parameters from transcript

Biometric verification: ZITADEL passkey (fingerprint/face) for high-value actions

12A.5.1 Voice Commands Catalogue

12A.5.2 Implementation

One-click guarantee: After voice command and biometric confirmation, the action executes — zero clicks.

12A.6 Customer Care Chatbot

12A.6.0 Purpose & Design Philosophy

AI-powered support accessible from any portal. Can answer questions, perform actions on the user's behalf (with PIN-based impersonation), and escalate to human support.

Features:

Conversational AI — A1 (Groq) for natural language responses; A2 (HF Mixtral) for intent extraction

Knowledge base — Public documentation + user-specific data (with permission)

PIN-based impersonation — For actions requiring state modification, bot requests user's support PIN (set in Company Admin). PIN verified via bcrypt; if correct, bot obtains short-lived, scope-limited JWT.

Escalation to human — If bot fails after three attempts, or user types "talk to a human", bot offers to escalate. System initiates VoIP callback via self-hosted Janus Gateway.

Impersonation logging — All impersonated actions logged as Governor decisions (decision_type = 'customer_care_bot_impersonation').

12A.6.1 Example Interaction

text

User: "My shipment is stuck at customs. What do I do?"

AI: I see USTN-EG-00491 has status CUSTOMS_HOLD. Missing document: Certificate of Origin.

Quick fix: [Upload Certificate of Origin]

If you've already uploaded it, the hold should clear within 2 hours.

Need to escalate? [Contact Support]

User: "Escalate"

AI: Creating support ticket #T-12345. A compliance officer will respond within 4 hours.

Reference this ticket in any follow-up. [View Ticket]

One-click: Click chat icon → bot opens. User types or speaks; bot executes actions with one confirmation.

■ Non-marketplace: The bot never suggests alternative counterparties; only assists with existing relationships.

12A.7 Dual-Mode Toggle (Trader Portal Only)

12A.7.0 Purpose & Design Philosophy

Allows trading companies with DUAL mode to switch between Buyer Dashboard (inbound) and Seller Dashboard (outbound) with a single click. The toggle provides clear visual feedback to prevent mode confusion.

Visibility conditions (all must be true):

Tenant type = TRD

default_trader_mode = DUAL

Employee allow_role_switching = true (from data scopes)

Employee's role includes allowed_trader_modes containing both BUY and SELL

12A.7.1 UI Placement & Visual Appearance

Placement: Global header, to the right of notifications bell and left of user avatar.

Visual Appearance:

When switching, an animated slide effect moves the highlight from one segment to the other.

The active mode is also displayed as a badge next to the user's avatar.

12A.7.2 Behaviour on Toggle

12A.7.3 Accessibility

12A.7.4 Governance

OPA policies enforce that actions are allowed in the current mode. If a user attempts an action in the wrong mode, the Governor returns a DENY with a Decision Panel that includes a "Switch to Seller/Buyer Mode" button — one click to switch and retry.

12A.8 Feedback & Help System

12A.8.0 Purpose

Provide a persistent, one-click mechanism for users to report issues, suggest improvements, or request help, seamlessly integrating with the Customer Care Chatbot and support ticketing system.

UI Component: A floating action button (FAB) positioned at the bottom-right corner of every portal screen.

12A.8.1 Modal Tabs

12A.8.2 Submission Workflow

One-click guarantee: Submitting feedback is one click.

12A.9 Adaptive Experience Engine (Guided Mode / Expert Mode)

12A.9.0 Purpose

Provide two distinct user experience modes — Guided Mode for new or infrequent users (SMEs, new exporters) with step-by-step wizards, and Expert Mode for frequent users with direct access and minimal confirmations.

12A.9.1 Mode Selection

12A.9.2 Guided Mode Features

12A.9.3 Expert Mode Features

12A.9.4 Mode-Specific Shortcuts

12A.9.5 AI-Powered Mode Suggestions (A1)

12A.10 Task Center

12A.10.0 Purpose

Every workflow action that requires human intervention creates a task. The Task Center unifies tasks across all workflows (trade, financing, compliance, dispute) into a single, prioritised, assignable queue.

12A.10.1 Task Creation Triggers

12A.10.2 Task Data Model

sql

```
CREATE TABLE tasks (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
task_type TEXT NOT NULL, -- 'SIGN_CONTRACT', 'PAY_FEE', 'UPLOAD_DOC'
```

```
ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
```

```
contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
```

```
dispute_id UUID REFERENCES disputes(id) ON DELETE SET NULL,
```

```
assigned_to_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
```

```
assigned_by_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,
```

```
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ASSIGNED', 'ESCALATED', 'COMPLETED',  
'EXPIRED'
```

```
priority INTEGER DEFAULT 50, -- 0-100, higher = more urgent
```

```
due_date TIMESTAMPTZ,
```

```
escalation_level INTEGER DEFAULT 0, -- 0=normal, 1=reminder sent, 2=supervisor, 3=governor,  
4=compliance
```

```
escalation_history JSONB DEFAULT '[]',
```

```
completed_at TIMESTAMPTZ,
```

```
completed_by_gtid TEXT,
```


12A.11.0 Purpose

Centralise all notifications across channels with user-controlled delivery preferences. Every notification is logged, actionable, and auditable.

12A.11.1 Notification Channels

12A.11.2 User Controls

Location: Company Admin → Notification Preferences

Settings:

Quiet Hours: Set time range (e.g., 22:00–07:00) and timezone

Category Priority Rules: Compliance ☒ High [In-App] [Email] [SMS] [WhatsApp]; Finance ☒ Medium [In-App] [Email] [SMS] []; Operational ☒ Low [In-App] [] [] []

Digest Settings: Daily digest (07:00 UTC) — summary of all notifications; Weekly digest (Monday 07:00 UTC); Digest includes: [Operational] [Finance] [] Compliance

12A.11.3 Notification Logging

sql

```
CREATE TABLE notification_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  recipient_gtid TEXT NOT NULL,  
  category TEXT NOT NULL,  
  channel TEXT NOT NULL,  
  subject TEXT NOT NULL,  
  body TEXT,  
  status TEXT, -- 'SENT', 'FAILED', 'READ', 'CLICKED'  
  sent_at TIMESTAMPTZ DEFAULT NOW(),  
  read_at TIMESTAMPTZ,  
  clicked_at TIMESTAMPTZ  
);
```

12A.12 Focus Mode

12A.12.0 Purpose

Temporarily suppress all non-critical notifications to reduce fatigue during high-concentration periods.

Activation: From the Smart Inbox or Notification Center, click "Focus Mode" (moon icon). Options:

1 hour (default)

4 hours

8 hours

Until tomorrow (08:00 local)

Custom (date/time picker)

Behaviour during Focus Mode:

Only critical notifications (priority 90+) are delivered — contract signing deadlines, payment due reminders, dispute responses, container release expiration warnings, margin calls

All other notifications (priority <90) are queued and delivered after Focus Mode ends as a summary digest

Persistent banner at top of Smart Inbox: "Focus Mode active until [time] — [Exit Focus Mode]"

One-click: Click "Focus Mode" → select duration → confirm.

12A.13 Help Center

12A.13.0 Purpose

Provide self-service documentation, video tutorials, AI-powered support, and ticketed assistance — all accessible from any portal.

Components:

12A.13.1 Documentation Structure

text

Help Center

■ ■ ■ ■ Getting Started

■ ■ ■ ■ Quick Start (10 min read)

■ ■ ■ ■ Video: First Trade (5 min)

■ ■ ■ ■ Glossary of Terms

■ ■ ■ ■ Role Guides

■ ■ ■ ■ Buyer Guide

■ ■ ■ ■ Seller Guide

■ ■ ■ ■ Logistics Provider Guide

■ ■ ■ ■ Financier Guide

■ ■ ■ ■ Government Official Guide

■ ■ ■ ■ Trade Guides

■ ■ ■ ■ Export Process (Egypt)

■ ■ ■ ■ Import Process (Egypt)

■ ■ ■ ■ Multi-Shipment Contracts

■ ■ ■ ■ Distressed Cargo

■ ■ ■ ■ Regulatory Compliance

■ ■ ■ ■ Nafeza Integration

■ ■ ■ ■ CargoX ACI

■ ■ ■ ■ ETA E-Invoice

■ ■ ■ ■ Egyptian Customs Law 207/2020

■ ■ ■ ■ API & Integration

■ ■ ■ ■ OpenAPI Reference

■ ■ ■ ■ Webhook Guide

■ ■■■■ Partner Onboarding

■■■ FAQ

■■■ Account & Billing

■■■ Trade Execution

■■■ Disputes

■■■ Security

12A.13.2 Video Academy

12A.13.3 Ticketing System

Ticket Lifecycle:

Created → User receives confirmation with ticket number

Assigned → Support agent (human) takes ownership

In Progress → Agent updates with comments

Resolved → User receives resolution; can reopen within 7 days

Closed → Archived after 30 days

Integration: Ticket updates appear as Smart Inbox items.

12A.14 Implementation Checklist (Part 12A)

12A.14.1 Smart Inbox

Implement inbox-service (Rust, Axum).

Create inbox_items, inbox_history, inbox_preferences tables.

Implement urgency scoring (Rego, A4).

Integrate AI title/description generation (A1, Groq → Ollama).

Build Recommended Actions Widget.

Add NATS subscriptions for all workflow events.

Implement snooze/dismiss with cross-device sync.

Add AI Summary Card.

12A.14.2 Trade Command Center

Implement /v1/trade/{ustn}/command-center endpoint.

Build horizontal timeline component.

Build Pending Action Panel with "Why this matters" (A1).

Build all Summary Cards (Commercial, Parties, Shipment, Documents, Finances, Risk, Logistics, QC, Multi-shipment, Distressed).

Build Activity Feed.

Build Collaborative Trade Room (NATS + Yjs).

12A.14.3 PlainLanguage Governor Decision Panel

Implement tenant_message generation (A1, Groq → Ollama).

Build Decision Panel UI component (React).

Add condition checklist with action links.

Add "Request Human Review" escalation (A3).

Add resolution timer and auto-escalation.

Add confidence/evidence expandable section.

12A.14.4 Shared Shipments Vault

Implement /v1/shipments-vault endpoint.

Build vault UI with virtual scrolling.

Implement filter chips (Status, Commodity, Port, Date, Risk Score).

Build Map View (Leaflet).

Add export functionality (CSV/PDF).

Implement saved filter sets.

Add bulk actions.

12A.14.5 VoiceCommandButton

Integrate Vosk (offline STT).

Implement intent extraction (HF Mixtral, local).

Add biometric verification (ZITADEL WebAuthn).

Build command catalogue.

Add cross-language fallback.

12A.14.6 Customer Care Chatbot

Implement chat API (Rust + Axum + Groq).

Add PIN-based impersonation (bcrypt + JWT).

Implement escalation to human (VoIP callback).

Add impersonation logging (Governor decision).

12A.14.7 Dual-Mode Toggle

Implement toggle UI in global header.

Add /v1/employee/switch-context endpoint.

Update JWT with active_trader_mode_context.

Add OPA policy enforcement for dual-mode context.

12A.14.8 Feedback & Help

Build FAB and modal.

Create feedback_tickets table.

Integrate with Customer Care Chatbot.

Add Smart Inbox alerts for Critical issues.

12A.14.9 Adaptive Experience

Add experience_mode to employees table.

Build Guided Mode walkthroughs (A1).

Build Expert Mode shortcuts.

Implement AI-powered mode suggestions.

12A.14.10 Task Center

Create tasks table.

Implement task creation triggers.

Build Task Center UI.

Implement escalation engine.

12A.14.11 Notification Center

Create notification_log table.

Implement notification channels (In-App, Email, SMS, Push).

Build notification preferences UI.

Add quiet hours and digest settings.

Implement Focus Mode.

12A.14.12 Help Center

Deploy MkDocs documentation.

Deploy PeerTube video academy.

Build AI Support Assistant (Groq + HF Mixtral).

Deploy ticketing system (Freescout).

Implement VoIP callback (Janus Gateway).

12A.15 AI Authority Summary for Part 12A

END OF PART 12A — COMMON COMPONENTS (v11.0 FULLY EXPANDED)

PART 12B — PORTAL FEATURE MATRIX (QUICK COMPARISON)

12B.0 Purpose & Design Philosophy

This matrix provides a high-level, at-a-glance comparison of all SGTX portals. Use this table to quickly identify which portal a user needs, what key features are available, which AI capabilities are present, and which critical add-ons are integrated.

Icons used in this matrix:

12B.1 Portal Summary Table

12B.2 AI Authority Distribution by Portal

12B.3 Critical Add-on Usage by Portal

12B.4 One-Click Action Count by Portal (Typical Workflow)

Note: All counts exclude data entry clicks (filling forms) and include only irreversible action clicks. Voice commands count as zero clicks.

12B.5 Portal Access Prerequisites Summary

12B.6 Portal Navigation Quick Reference

Trader Portal — Buyer Mode

Trader Portal — Seller Mode

Logistics Provider (LSP) Portal

Shipping Line (SHIP) Portal

Laboratory (LAB) Portal

QC Inspection Portal

Customs Broker (CBR) Portal

Financier (Bank) Portal

Financier (PFI) Portal

Government Portal

Admin Portal

Marketplace Partner Portal

12B.7 Smart Inbox Examples by Portal

12B.8 Shared Components Across All Portals

12B.9 Device Priority & Mobile/Web Parity

Important note: Every role that has a mobile companion app also has a fully functional web portal. Mobile apps are optional and designed for field workers; all trade-critical actions can be performed from any web browser.

12B.10 Portal Feature Matrix — Implementation Checklist

12B.10.1 Trader Portal (Buyer)

Smart Inbox (Buyer-specific items)

New Trade Request with structured container form, multi-shipment request

Quote Review & Negotiation with comparison table

Contract Signing with Clause Forge integration

Customs Readiness with dynamic document checklist

Distressed Cargo (buyer view)

Disputes with evidence autocompiler

Saved Contacts with trust portraits

Financing (borrower view)

Company Admin with role templates

12B.10.2 Trader Portal (Seller)

Smart Inbox (Seller-specific items)

Pending Requests with accept/decline/counter

EXW Price Lock with live market chart and fair price

Containerisation & Packing with non-uniform layers

Logistics Builder (Modes A, B, C) with alternative ports

Quote Submission with SGTX fee calculation

QC Booking with AI-recommended inspection points

Document Finalisation with digital signature

Barcode Print with ZPL/PDF generation

Distressed Cargo & Notifications

Cash Position with rolling forecast

Company Admin with data scopes (hidden cost components)

12B.10.3 LSP Portal

RFQ Inbox with directed and anonymous broadcast

Clarification request workflow

Dispatch Planner (ORTools VRP) with driver assignment

Warehouse Dashboard (warehousing only)

Forwarder Console (forwarding only)

Driver Mobile App management (QR pairing, offline queue)

Performance dashboard

Company Admin with anonymous RFQ opt-out

12B.10.4 SHIP Portal

Booking Requests with quote submission

eBL Management with webhook integration

Vessel Schedule with ETD/ETA propagation

Freight Invoices with credit/mandatory terms

Contract Rate Manager (private rates per seller)

Performance dashboard

Company Admin

12B.10.5 LAB Portal

Testing Jobs with sample tracking

Result Submission with MRL validation

Certificates (auto-trigger via Nafeza)

Performance dashboard

Company Admin

12B.10.6 QC Portal

Inspection Jobs with AQL sampling enforcement

Mobile App Integration (offline-first, AR, defect detection)

Report Submission with PASS/FAIL/CONDITIONAL

Conditional Pass with action plan and hold flag

Re-inspection Request workflow

Dispute Fast-Track with override flagging

Performance dashboard

Company Admin

12B.10.7 CBR Portal

Certification Requests with declaration preview

Physical Document Jobs with QR scanning and GPS tracking

Storage with retention expiry

Audit Representation

Performance dashboard

Company Admin with digital seal management

12B.10.8 Financier (Bank) Portal

Financing Opportunities with auto-RFQ and full disclosure

My Bids & Active Loans with collateral monitoring dashboard

DeFi (jurisdiction-dependent) with protocol comparison

Secondary Market with tokenised assets

Portfolio & Compliance with regulatory report generation

Financed Companies (private, audit-traced)

Company Admin with preferences and exposure limits

12B.10.9 Financier (PFI) Portal

Financing Opportunities with auto-RFQ and full disclosure

My Bids & Active Loans with collateral monitoring dashboard

Portfolio & Performance with CSV export

Financed Companies (private, audit-traced)

Company Admin with simplified preferences

12B.10.10 Government Portal

Live Trade Monitor with risk scores and clearance status

Clearance Workflow with auto-clearance recommendation

Document Verification with AI-flagged discrepancies

Multi-Agency Workflow with visual stepper

Anonymous Trade Management with declassification audit log

Integrations with connector management

Permit Issuance with digital seal

Audits & Reports with Loom chain verification

Compliance Monitor with real-time metrics

Company Admin with dynamic module configuration

12B.10.11 Admin Portal

Platform Health with real-time metrics and predictive alerts

Constitutional Policies with impact simulation

Governor Log & Audit with natural language query

Jurisdiction Matrix with conflict detection

PSP Manager with fallback chains

Special Rate Manager with multisig approval

Incidents with automated post-mortem

Marketplace Partners with revenue share agreements

Tenant Management with impersonation (readonly)

Customer Care Hub with chatbot dashboard

Configuration History with diff and rollback

Internal Admin with multisig member management

12B.10.12 Marketplace Partner Portal

Dashboard with real-time metrics

Leads Management with intent inbox

Webhook Management with delivery logs

Revenue Attribution Disputes

API Key Management with usage analytics

Sandbox with synthetic data

Agreement with revenue share proposals

Company Admin with IP whitelisting

END OF PART 12B — PORTAL FEATURE MATRIX (v11.0 FULLY EXPANDED)

PART 12C — DETAILED PORTAL SPECIFICATIONS (PER PORTAL)

PART 12C.0 — PORTAL DEVICE PRIORITY & MOBILE/WEB PARITY

12C.0.1 Purpose & Design Philosophy

The SGTX platform is designed to be accessible from any device — desktop, laptop, tablet, or smartphone — while optimising the user experience for the specific context in which each role operates. A trade manager working from a desktop requires different interaction patterns than a truck driver using a mobile app in a warehouse, or a QC inspector performing on-site inspections with intermittent connectivity.

This section defines the device priority for each portal, the mobile/web parity expectations, the offline capabilities required for field operations, and the unified navigation structure that ensures consistency across all portals while allowing role-specific customisation.

Key principles:

12C.0.2 Portal Device Priority Matrix

Each portal is categorised as Mobile-First, Web-First, or Hybrid. Mobile-first portals have dedicated companion mobile apps with offline capabilities. Web-first portals are fully responsive but optimised for desktop use.

Important note: Every role that has a mobile companion app also has a fully functional web portal. Mobile apps are optional and designed for field workers; all trade-critical actions can be performed from any web browser.

12C.0.3 Mobile App Specifications (By Portal)

12C.0.3.1 LSP Driver App (Trucking)

Technology Stack: React Native + Expo, WatermelonDB (offline), ZXingC++ (barcode scanning), Vosk (offline STT), OSRM (offline navigation)

Key Features:

Offline Sync Indicator:

Banner when offline: "Offline mode — actions will sync when connection resumes."

Auto-retry: Exponential backoff (starting at 1s, doubling up to 60s, maximum 5 retries). Manual sync via "Sync Now" button.

12C.0.3.2 QC Inspector App

Technology Stack: React Native + Expo, WatermelonDB, HF ViT (on-device), ZXingC++, Vosk, AR.js

Key Features:

Offline Sync Indicator: Same as LSP Driver App.

Banner when offline: "Offline mode — inspection data saved locally. Report will sync when connection resumes."

12C.0.3.3 CBR Document Receipt App

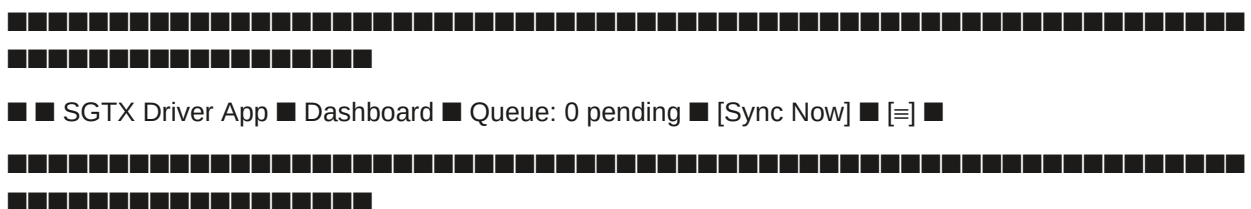
Technology Stack: React Native + Expo, ZXingC++, GPS (location stamp)

Key Features:

12C.0.4 Persistent Offline Sync Indicator (All Mobile Apps)

For the LSP Driver App, QC Inspector App, and CBR Document Receipt App, the mobile app displays a persistent status icon at the top of every screen:

text



Offline behaviour details:

12C.0.5 Portal Navigation Structure (Unified)

All portals share a common navigation structure, with role-specific tabs appearing in the sidebar. The navigation is consistent across web, desktop, and mobile (where applicable).

12C.0.5.1 Global Header (All Portals)

text



■ [SGTX Logo] ■ [Search] ■ [Notifications] ■ [Dual-Mode Toggle] ■ [Avatar] ■

■ ■ ■ (bell with badge) ■ (Trader only) ■ ■



12C.0.5.2 Sidebar Navigation (All Portals)

The sidebar is collapsible (on desktop) and hidden behind a hamburger menu (on mobile). Tabs are ordered by workflow priority.

Common Tabs (All Portals):

Role-Specific Tabs (Added Below Common Tabs):

12C.0.5.3 Mobile Navigation (Bottom Tab Bar)

On mobile devices (screen width < 768px), the sidebar is replaced by a bottom tab bar with the 4-5 most frequently used tabs for that role.

"More" expands to reveal the full sidebar menu as an overlay.

12C.0.6 Responsive Design Breakpoints

12C.0.7 Common Features Across All Portals (Mobile & Web)

12C.0.8 Implementation Checklist (12C.0)

12C.0.8.1 Device Detection & Responsive Layout

Implement device detection (user agent, screen width) in Next.js middleware.

Build responsive layout with Tailwind CSS breakpoints.

Create collapsible sidebar component (desktop).

Create bottom tab bar component (mobile).

Implement hamburger menu overlay for mobile.

12C.0.8.2 Mobile App Infrastructure

Set up React Native + Expo project.

Integrate WatermelonDB for offline storage.

Implement offline sync engine (queue, retry, conflict resolution).

Add persistent offline sync indicator (dot + queue count).

Implement automatic retry with exponential backoff.

Add manual "Sync Now" button.

12C.0.8.3 LSP Driver App

Integrate ZXingC++ for barcode scanning.

Integrate Vosk for offline voice commands.

Integrate OSRM for offline turn-by-turn navigation.

Implement geofence alerts (GPS + geofence detection).

Build offline queue for scans and milestones.

Add push-to-talk communication (WebRTC).

12C.0.8.4 QC Inspector App

Integrate HF ViT for on-device defect detection.

Integrate AR.js for AR overlay.

Implement mandatory override reason (≥ 10 chars).

Implement AQL sampling enforcement.

Build offline queue for inspection data.

Add report submission with offline sync.

12C.0.8.5 CBR Document Receipt App

Integrate QR code scanning.

Implement GPS location stamping.

Build offline queue for status updates.

Add photo capture for stamped documents.

12C.0.8.6 Testing

Test all portals on desktop (Chrome, Firefox, Safari).

Test all portals on mobile browsers (iOS Safari, Android Chrome).

Test LSP Driver App offline (airplane mode) → scan → sync when online.

Test QC Inspector App offline → defect detection → override → submit → sync.

Test CBR Document Receipt App offline → receive package → sync.

Test offline sync conflict resolution (server wins).

12C.0.9 AI Authority Summary for Part 12C.0

END OF PART 12C.0 — PORTAL DEVICE PRIORITY & MOBILE/WEB PARITY (v11.0 FULLY EXPANDED)

PART 12C.1 — TRADER PORTAL — BUYER DASHBOARD

12C.1.0 Access & Purpose

12C.1.0.1 Access Requirements

Access:

Tenants with type = TRD AND

Active trader mode = BUY (if default_trader_mode = BUY or DUAL with context switched to BUY).

Role clarification:

A pure Buyer (mode = BUY) only sees inbound shipments, never acts as seller.

A Dual-mode trader (mode = DUAL) can toggle to Buyer view via the global header toggle (Part 12A.7).

When in Buyer view, the dashboard is identical to a pure Buyer.

OPA policies enforce that any attempt to perform a seller action while in Buyer mode returns a DENY with a Decision Panel offering to switch mode.

Landing page (configurable):

Default: Universal Command Center (Part 12G) — provides executive summary, quick actions, AI assistant.

User can set preference to land directly on any tab (e.g., New Trade Request).

Dual-mode toggle:  visible (if tenant has DUAL mode). When toggled to BUY, the sidebar shows buyer-specific tabs.

12C.1.0.2 Buyer Dashboard Purpose

The Buyer Dashboard is the complete workspace for importers and procurement professionals. It enables:

Trade initiation — Create structured, container-level trade requests with AI-assisted product specifications, incoterm selection, and optional multi-shipment scheduling.

Quote evaluation — Review and compare seller quotes, including multiple delivery options (primary and alternative ports), with full landed cost transparency.

Negotiation — Negotiate price and terms with AI-assisted counter-offers, partial acceptance, deadline extensions, and real-time trade room chat.

Contract execution — Digitally sign contracts (including the mandatory SGTx Witness Clause), track logistics addenda, and manage fee payment (where buyer is responsible per incoterm).

Shipment tracking — Monitor inbound shipments with real-time milestone updates, AIS vessel tracking, and cold-chain monitoring.

Customs readiness — Manage import documentation with dynamic, RIA-driven checklists and one-click document requests.

Distressed cargo — View distressed cargo listings from saved contacts, make offers, and complete microcontracts.

Financing — Request financing for locked trades, view bids, accept co-financing, and manage active loans.

Dispute resolution — File and manage disputes with AI-compiled evidence, mediation, third-party expert invitation, and arbitration preparation.

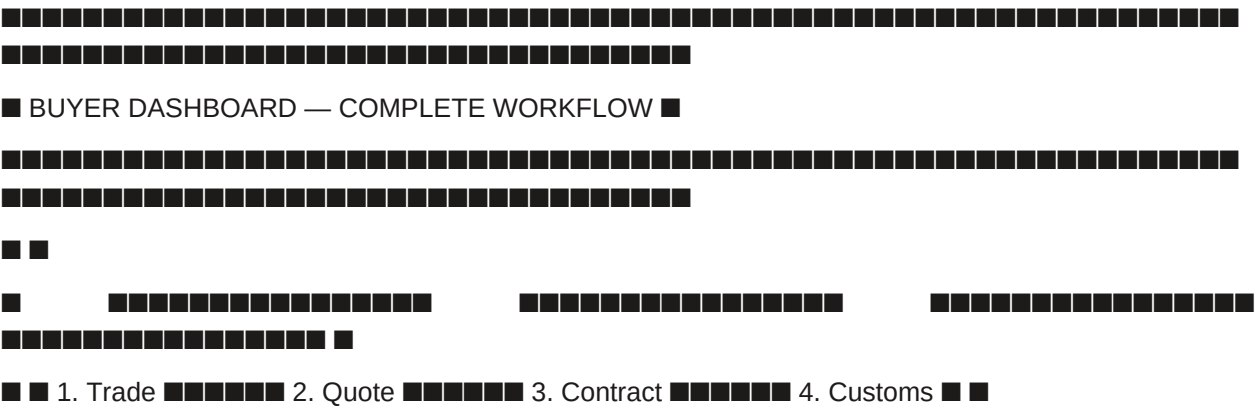
Financial management — View invoices, approve settlements (voice-enabled), and download monthly reconciliation statements.

Compliance & reporting — Monitor KYB/KYC status, sanctions alerts, and regulatory obligations.

Administration — Manage employees, roles, data scopes, approval chains, branding, and tenant exit.

12C.1.0.3 Buyer Dashboard — High-Level Workflow

text



Tab 2: New Trade Request

Purpose: Create a structured trade request that initiates the entire trade workflow. All data is stored in trade_requests.parsed_specs and reused across phases.

Step-by-step workflow (one-click where indicated):

Step 2.1 — Select Seller

Example UI:

text

Select Seller

Search by GTID or Company Name... [Enter GTID or search saved contacts]

Saved Contacts:

SGTX-VN-TRD-002139-7F3A Mekong Fresh Co. Trust: 92 Sanctions:

Last trade: 15 May 2026 Vietnam 12 trades

SGTX-EG-TRD-001234-5B6C Nile Foods Trust: 88 Sanctions:

Last trade: 10 Jun 2026 Egypt 8 trades

[Enter GTID manually] [Cancel]

Step 2.2 — Select Incoterm

Incoterm Reference Table (displayed as tooltip/helper):

Auto-configuration triggered immediately after selection:

System stores incoterm in trade_requests.parsed_specs.incoterm.

Pre-populates seller's mandatory logistics services list (visible later in Phase 2).

Plain-language summary of responsibilities (A1, Groq) shown to both buyer and seller.

Example:

text

Select Incoterm

[CFR ▼]

CFR — Cost and Freight

Seller is responsible for:

• Export customs clearance

• Trucking to port of loading

• Ocean freight to destination port

Buyer is responsible for:

• Import customs clearance

• Destination charges

• Insurance (optional)

[View full Incoterms 2020 reference]

Step 2.3 — Configure Containers

Example UI:

text

Number of Containers: [3] [+ Add Container] [Clone Container] [Bulk Edit]

Container 1

Country of Origin: [Vietnam ▼] Destination Country: [Egypt ▼]

Port of Discharge: [Alexandria ▼] (filtered by Egypt)

Buyer selects product from dropdown → system automatically:

Fills the HS code (6 or 10 digits).

Triggers the Product Form Agent (A2) to generate dynamic specifications.

Buyer types HS code directly → upon selection, system:

Fills the product name.

Triggers the Product Form Agent.

Buyer types free-text product name → system attempts to match it to an HS code via AI (A2).

AI suggestion from buyer's history — shows previously used products with "recent" badge.

Non-marketplace safeguard: Never suggests "popular" or "other buyers" products.

Step 2.5 — Dynamic Product Specification (A2, HF local + Groq)

When the buyer selects a product, the Product Form Agent queries the RIA vector database and returns a JSON schema that dynamically adds:

Example — Fresh Oranges:

text

Commodity Details — Oranges (HS 0805.10)

Variety: [Valencia ▼] Grade: [Grade A ▼]

Brix: [11.5] °Bx Size range: [72] to [80] mm

Packing:

Pallet type: [EUR (800×1200 mm) ▼]

Cartons per pallet: [40] Net weight per carton (kg): [10]

Stacking layers: [5] Total pallets: [22]

Packaging type: [Cartons 10kg ▼]

Calculated total net weight: 8,800 kg

Estimated gross weight (+5% tare): 9,240 kg

Special Procedure Required: Cold Treatment

Temperature: -0.5°C for 14 days

Start date: [2026-07-01] Completion date: [2026-07-15]

Accepted facilities: [Egyptian Cold Treatment Center ▼]

I will upload the cold treatment certificate after completion

If ALLOW:

Trade request created with status = 'PENDING_SELLER_RESPONSE'.

USTN not yet generated (will be generated at contract lock).

Seller receives a Smart Inbox item (priority 75).

Buyer receives confirmation.

If CONDITIONAL or DENY:

Governor generates plain-language tenant_message (A1, Groq → Ollama) listing unmet conditions.

Buyer sees the PlainLanguage Governor Decision Panel and cannot proceed until conditions are resolved.

Example CONDITIONAL response (missing cold treatment certificate):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "Your trade request cannot be submitted because cold treatment is required for fresh oranges to Japan, and you have not indicated that you will provide a certificate.",
    "conditions": [
      {
        "condition_id": "cold_treatment_cert",
        "label": "Upload cold treatment certificate after completion",
        "action_url": "/v1/documents/upload?ustn=..."
      }
    ],
    "escalation_hint": "If you believe cold treatment is not required, request a compliance review."
  }
}
```

Example CONDITIONAL response (packing consistency):

json

```
{
  "verdict": "CONDITIONAL",
  "tenant_message": {
    "summary": "The calculated total net weight (8,800 kg) does not match the declared total (10,000 kg). Please correct the packing details or the declared total.",
    "conditions": [
      {
        "condition_id": "packing_weight_mismatch",
        "label": "Adjust packing details or declared total weight",

```


Example Shipments Vault (Buyer View):

text

MY SHIPMENTS (Buyer) [Saved Views ▼] [Export] []

Search: [] Status: [Active ▼] Date: [Next 7 days ▼]

USTN

Status

Vessel

ETD/ETA

Seller

Customs

Docs

Actions

SGTX...

Arrived

MSC

ETD:14Jun

Mekong

5

[View]

(delayed)

FIONA

Act:16Jun

Fresh

Missing

(click)

[Req Doc]

ETA:20Jun

Phyto

[Dispute]

Act:24Jun

SGTX...

In

MSC

ETD:20Jun

Global

All

0

[View]

Transit

FIONA

ETA:05Jul

Exports

Verified

[Track]

One-click: Click any USTN → opens TCC. Export CSV/PDF (1 click). Filter chips (1 click each).

Tab 6: Documents

Purpose: Manage all trade-related documents: upload, request, verify, download.

Example Document Checklist:

text

Documents — USTN SGTX-EG-20260415120000-A1B2C3D4

Commercial Invoice.pdf (uploaded 10 Jun) [Download]

Packing List.pdf (uploaded 10 Jun) [Download]

Bill of Lading.pdf (uploaded 12 Jun) [Download]

Phytosanitary Certificate (expires 30 Jun) [Upload] [Request from Seller]

Certificate of Origin.pdf (uploaded 9 Jun) [Download]

Purpose: View distressed cargo listings from saved contacts, make offers, and track microcontracts.

text

■ ■ Seller: Mekong Fresh Co. ■ Shelf life: 5 days ■ ■

■ ■ ■ [Make Offer] [View Details] ■ ■

■ ■ Seller: Global Exports ■ Shelf life: 12 days ■ ■

■ ■ ■ [Make Offer] [View Details] ■ ■

Purpose: Request financing for a locked trade (or specific shipment), view bids, accept co-financing, manage active loans.

text

■ ■ INV-001 ■ SGTX-EG-001 ■ Commercial ■ \$30,360 ■ ■ Paid ■ ■

■ ■ INV-002 ■ SGTX-EG-002 ■ Import ■ \$10,000 ■ ■ Pending ■ ■

■■■■ Duties ■■ [Pay Now] ■■

■ ■ INV-003 ■ SGTX-EG-003 ■ Destination ■ \$500 ■ ■ Overdue ■ ■

Charges [Pay Now]

Tab 10: Disputes

Purpose: File and manage disputes (quality, delay, non-payment, documentation fraud, cold chain, service quality, financing, SGTX fee).

Example Mediation Log:

text

2026-04-16 10:00:00 -- Buyer (Nile Foods)

The strawberries arrived with 10% mould. I am claiming \$2,000 refund based on photos and temperature log.

2026-04-16 11:30:00 -- Seller (Mekong Fresh)

We dispute the claim. Our reefer log shows -18°C throughout. The damage likely occurred after delivery.

We counter-offer \$500 as goodwill.

2026-04-16 12:00:00 -- AI Settlement Proposal (Grog)

Based on the temperature log (no deviation) and the buyer's photos (mould present), the likely root cause is post-delivery mishandling.

Recommended settlement: \$800 (4% of invoice). Accept / Reject.

Landed Cost Breakdown expandable.

Negotiation panel with three columns (history, current offer, chat).

Partial acceptance (Improvement #2a).

Counter-offer with reason (Improvement #2b).

Deadline extension request (Improvement #2c).

Visual diff for amendments.

Mutual confirmation endpoint.

12C.1.6.4 Contract Signing

Clause Forge integration (A2) for contract generation.

Mandatory SGTX Witness Clause insertion.

Own-contract upload with HF Donut extraction.

SGTX FEES Addendum for own-contract uploads.

ZITADEL passkey signature.

Logistics addenda tracking and reminder.

Governor validation for contract consistency.

12C.1.6.5 Shipments (Shared Vault)

Shared Shipments Vault with buyer-specific columns (Seller Name, Customs Readiness, Documents Missing).

Filter chips (Status, Commodity, Port, Date Range).

Map view for vessel tracking.

Export functionality (CSV/PDF).

Slide-out detail panel with Customs Readiness tab.

12C.1.6.6 Documents

RIA-driven document checklist.

Document upload with HF Donut extraction.

Document request (one-click to counterparty).

Document verification with discrepancy flagging.

Document expiry alerts.

PDF viewer with SHA256 hash.

12C.1.6.7 Distressed Cargo

Distressed listings list (filtered to saved contacts).

Listing detail view with AI condition report.

Offer submission with express negotiation bot.

Counter-offer workflow.

Microcontract signing and tracking.

12C.1.6.8 Financing (Borrower)

Financing request form with AI LTV recommendation.

Credit intelligence display (A2).

Auto-RFQ broadcast (background).

Bid comparison table with co-financing.

Financing agreement signing.

Disbursement monitoring.

Repayment monitoring.

Active loans dashboard.

12C.1.6.9 Invoices & Payments

Invoice list with status filtering.

Invoice detail view with PDF download.

Payment buttons for buyer-responsible invoices.

Settlement approval (one click or voice).

Milestone-based pre-approval.

Payment history.

12C.1.6.10 Disputes

Dispute filing with category selection.

Evidence autocompiler (A2).

Mediation log with Groq translation.

Third-party expert invitation (A3).

AI settlement proposal (A1/A2).

Predictive outcome display (A2).

Arbitration case preparation.

SGTX fee dispute workflow.

12C.1.6.11 Compliance

KYB status display.

Sanctions alerts with enhanced DD requirements.

Regulatory reports (downloadable).

Consent management with granular options.

12C.1.6.12 Approvals

Task list from tasks table.

Task detail with approval history.

Approve/Reject/Delegate buttons.

Bulk approval (Expert Mode).

12C.1.6.13 Audit

Activity Center integration (Part 13A).

Loom chain viewer with verification.

12C.1.6.14 Company Admin

Users & Roles (invite, assign, edit permissions).

Workspaces (multi-company).

Settings (profile, branding, notifications).

Integrations (API keys).

Exit Centre (data export, account closure).

12C.1.6.15 Testing

Full buyer journey (trade request → settlement → dispute).

Dual-mode context switching (Buyer ↔ Seller).

Multi-shipment request and schedule.

Partial acceptance and counter-offer with reason.

Deadline extension request.

Own-contract upload with mandatory addendum.

Distressed cargo offer and microcontract.

Financing request and co-financing acceptance.

Dispute filing with evidence autocompiler.

Voice-activated settlement approval.

12C.1.7 AI Authority Summary for Part 12C.1

END OF PART 12C.1 — TRADER PORTAL — BUYER DASHBOARD (v11.0 FULLY EXPANDED)

PART 12C.2 — TRADER PORTAL — SELLER DASHBOARD

12C.2.0 Access & Purpose

12C.2.0.1 Access Requirements

Access:

Tenants with type = TRD AND

Active trader mode = SELL (if default_trader_mode = SELL or DUAL with context switched to SELL).

Role clarification:

A pure Seller (mode = SELL) only sees outbound shipments, never acts as buyer.

A Dual-mode trader (mode = DUAL) can toggle to Seller view via the global header toggle (Part 12A.7).

When in Seller view, the dashboard is identical to a pure Seller.

OPA policies enforce that any attempt to perform a buyer action while in Seller mode returns a DENY with a Decision Panel offering to switch mode.

Landing page (configurable):

Default: Universal Command Center (Part 12G) — provides executive summary, quick actions, AI assistant.

User can set preference to land directly on any tab (e.g., Pending Requests).

Dual-mode toggle: ☐ visible (if tenant has DUAL mode). When toggled to SELL, the sidebar shows seller-specific tabs.

12C.2.0.2 Seller Dashboard Purpose

The Seller Dashboard is the complete workspace for exporters and sales professionals. It enables:

Trade response — Receive and respond to buyer trade requests (including multi-shipment proposals), accept, decline, or propose modifications.

Pricing — Lock EXW prices with AI-recommended bands, live market charts, and post-lock price watch.

Packing & containerisation — Design physical packing plans with non-uniform layer stacking, palletisation optimisation, collaborative editing, and 3D container viewing.

Logistics procurement — Determine logistics costs using three flexible modes (Manual, RFQ to LSPs, Direct to SHIP with add-on services), with incoterm-based filtering and alternative delivery ports.

Quote submission — Assemble final price (EXW + logistics + SGTX fee, paid by seller, added to quote) and submit to buyer.

QC booking — Select QC providers, receive AI-recommended inspection points, and manage inspection reports (including conditional pass and re-inspection).

Document finalisation — Digitally sign packing lists and commercial invoices, auto-submit to ETA and Nafeza.

Barcode printing — Generate SSCC-18 barcode labels (ZPL or PDF) with reprint governance.

Distressed cargo — Declare distressed cargo, receive AI condition assessment and pricing, triage (sell quickly / comply / insurance), and execute accelerated outreach.

Dispute resolution — Respond to disputes, participate in mediation, accept settlement proposals, prepare arbitration.

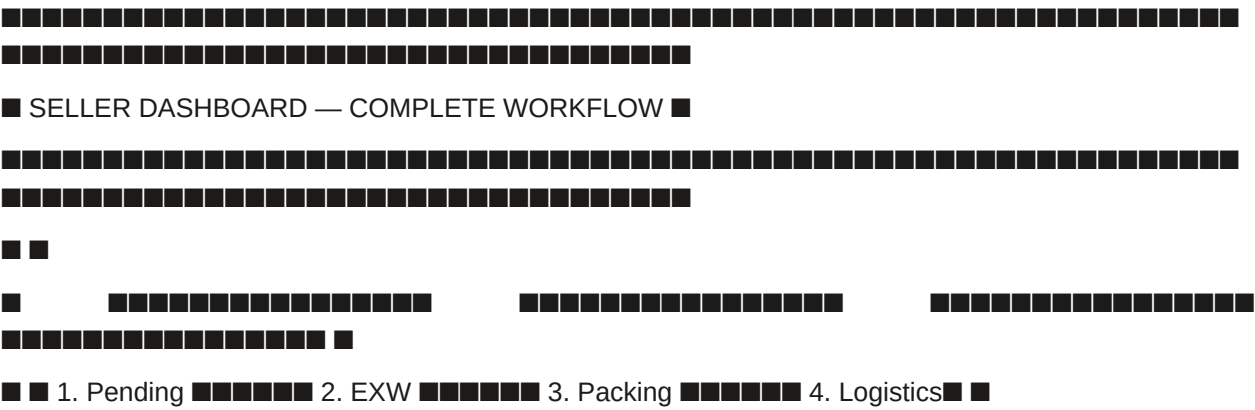
Relationship management — Manage buyer profiles with trust portraits, relationship health scores, and performance dashboards.

Financial management — View 90-day rolling cash forecast, detailed ledger, and monthly reconciliation statements.

Administration — Manage employees, roles, data scopes (including hidden cost components), approval chains, branding, and tenant exit.

12C.2.0.3 Seller Dashboard — High-Level Workflow

text



Tab 1: Universal Command Center (Home)

(Full specification in Part 12G)

Purpose: Primary landing page. Shows executive summary cards, quick actions grid, AI Operations Assistant, recent activity feed.

Seller-specific cards:

Quick Actions (Seller-specific):

AI Operations Assistant (A1, Groq):

One-click: Click any card → navigates to the relevant tab.

Tab 2: Pending Requests (RFQ Inbox)

Purpose: Receive and respond to buyer trade requests. Accept, decline, or request modifications.

2.1 View Pending Requests

Example UI:

text

PENDING REQUESTS (3)

Filters: [Status: All ▼] [Commodity: All ▼] [Date: Last 7 days ▼] [Search:]

Oranges — 20,000 kg

Buyer: Nile Foods (Egypt)

Incoterm: CFR

Requested: 15 Jul 2026

Multi-shipment: 2 shipments

Trust: 88

Match Score: 94 (high)

[Accept] [Decline] [Propose Modifications]

Lemons — 5,376 kg

Buyer: Cairo Imports (Egypt)

Incoterm: FOB

Requested: 20 Aug 2026

Match Score: 76 (medium)

[Accept] [Decline] [Propose Modifications]

Strawberries — 8,800 kg

Buyer: German Imports (Germany)

Example:

text

■ EXW PRICE — Commodity: Valencia Oranges (HS 0805.10) ■

■ Price entry: [■ Per Ton] [● Per Kilo] [■ Per Unit (10 kg box)] ■

■ EXW price: [1.50] [USD] per kg ■

■ Equivalent: \$1,500.00 per ton | \$15.00 per box (10 kg box) ■

■ ■

■ Currency: [USD ▼] ■

■ Weight unit toggle: [Metric tons ●] [Pounds ■] ■

3.4 Total EXW Value Calculation (Real-Time)

Example:

text

■ EXW SUMMARY ■

■ Commodity A (Oranges): 20,000 kg × \$1.50 = \$30,000 ■

■ Commodity B (Lemons): 5,376 kg × \$2.10 = \$11,289.60 ■

■ ■

■ ■

■ Total EXW: \$41,289.60 ■

■ Container 1 subtotal: \$20,644.80 ■

■ Container 2 subtotal: \$20,644.80 ■

■ Grand total EXW (all containers): \$41,289.60 ■

3.5 Price Deviation Justification (A2, if triggered)

Example (Above Band):

text

No incompatible commodities mixed — checked by WasmEdge commodity compatibility rule

For shipments requiring cold treatment, treatment certificate must be uploaded or planned

If valid: SSCC-18 codes generated, QR codes with verifiable credentials, Loom hash of packing plan, packing_plans.locked_at = NOW().

If invalid: Decision Panel appears with remediation links.

Tab 5: Logistics Builder

Purpose: Determine logistics costs using three flexible modes (A, B, C), which can be combined per service. Incoterm-based service filtering ensures mandatory services are included.

Mode Selection UI: Toggle or radio buttons per service category. Seller can switch modes per service.

text

LOGISTICS BUILDER — SGTX-EG-26-F3A-1

Incoterm: CFR

Mode selection per service:

Trucking

Mode A — Manual

Mode B — RFQ

Mode C — Direct

Ocean Freight

Mode A — Manual

Mode B — RFQ

Mode C — Direct

Customs

Mode A — Manual

Mode B — RFQ

Mode C — Direct

Brokerage

5.1 Mode A — Manual Entry (Fixed Prices)

One-click: Seller types amount → cost stored.

5.2 Mode B — RFQ to Logistics Providers (LSPs)

Clarification Request (Improvement #4):

text

Example Request:

text

Direct Request to Shipping Lines

Target Shipping Lines: [MSC Egypt ▼] [Maersk Egypt ▼] [+ Add]

Base Service: Ocean Freight

Container Type: [40ft Reefer ▼] Quantity: [2]

Sailing Window: [20 Jun 2026]

Temperature: [-18°C]

Add-on Services:

Trucking (origin to port)

Customs Broker (export clearance)

Insurance

Destination handling

[Send Request]

5.4 Combining Modes B and C

Seller can select ocean freight from Mode C and trucking from Mode B. Comparison panel aggregates all quotes.

text

Combined Quotes — Logistics Bundle

Service Selected Provider Fee Mode

Trucking Fast Trucking Co. \$127.50 Mode B (LSP RFQ)

Ocean Freight MSC Egypt \$4,200.00 Mode C (Direct SHIP)

Customs Brokerage SGS Egypt \$150.00 Mode B (LSP RFQ)

Purpose: Select QC provider, receive AI-recommended inspection points, and manage inspection reports.

Example QC Booking:

text

QC BOOKING — SGTX-EG-26-F3A-1

Commodity: Fresh Oranges, 20,000 kg

AQL Sampling Plan: General Level II (5 pallets random)

Provider Fee AQL Plan Location Select

SGS \$500 General Lvl Alexandria Port [Select]

Inspection II

Bureau \$450 General Lvl Damietta Port [Select]

Veritas II

AI-Recommended Inspection Points (A2):

• OR-019 (high priority — end of lot)

• OR-020 (high priority — end of lot)

• OR-003 (random sample)

• OR-007 (random sample)

• OR-011 (random sample)

• OR-015 (random sample)

[Accept Suggestions]

Tab 9: Document Finalisation

Purpose: Sign packing list and commercial invoice (ZITADEL passkey) before submission to ETA and Nafeza.

Example:

text

DOCUMENT FINALISATION — SGTX-EG-26-F3A-1

Packing List (PDF)

• Generated: 15 Jun 2026

• Pallet count: 36

• SSCC barcodes: 36 generated

[Download] [Preview]

Commercial Invoice (UBL 2.1)

• Invoice Number: INV-STR-2026-001

• Amount: \$46,453.61

• ETA-compliant:

[Download] [Preview]

Digital Signature

Sign with ZITADEL passkey (fingerprint/face)

[Sign Now]

After signing, system will auto-submit to:

• ETA e-Invoice API

• Nafeza Customs API

Tab 10: Barcode Print

Purpose: Generate SSCC-18 barcode labels (ZPL or PDF) for each pallet. Reprint with reason after loading.

Example:

text

Add quick actions (Create Shipment, Lock EXW Price, Send RFQ, Print Barcodes, Declare Distressed, Invite Employee, Generate Report, Contact Support).

Integrate AI Operations Assistant (A1, Groq).

12C.2.6.2 Pending Requests

List with buyer name, commodity, volume, incoterm, match score.

Read-only view of buyer's parsed specs.

Accept/decline with mandatory reason.

Propose modifications (delivery date, port, container count, commodities).

Multi-shipment schedule acceptance/modification.

12C.2.6.3 EXW Price Lock

Loading origin selection with geocoding.

Live market chart (FAO, USDA, World Bank).

AI fair price badge with "Use fair price" button.

Dynamic unit conversion (per ton / per kilo / per unit).

Global weight unit toggle (metric tons / pounds).

Total EXW value calculation (real-time).

Price deviation justification (>30% above AI band).

Post-lock price watch (NATS subscription, >10% move alert).

Five add-ons: volume-tiered pricing, dynamic currency conversion, historical price comparison, lock & hold, admin-controlled floor/ceiling.

12C.2.6.4 Containerisation & Packing

Weight calculation engine (total cartons, net, gross).

ORTools palletisation solver.

Non-uniform layer stacking (multiple layer patterns per pallet).

Collaborative editing (Yjs + WebRTC).

3D container viewer (Three.js).

Capacity heatmap (opt-in, differential privacy).

Ecological packaging advisor (A1, Groq).

Carbon footprint calculation (ISO 14067, CBAM-ready).

Lock packing plan with Governor validation.

12C.2.6.5 Logistics Builder

Mode A — Manual entry with incoterm-based filtering.

Mode B — RFQ to LSPs with clarification requests (Improvement #4).

Mode C — Direct to SHIP with add-on services.

Combined mode selection and comparison panel.

Alternative delivery ports with AI suggestions.

12C.2.6.6 Quote Submission

Price breakdown display (EXW + logistics + SGTx fee).

Multi-shipment per-shipment pricing.

Governor validation (all mandatory conditions).

Submit quote with Decision Panel on failure.

12C.2.6.7 Laboratory Selection

Lab list from saved contacts.

Quotation acceptance.

Results viewing with MRL validation (A2).

Certificate trigger (A4).

12C.2.6.8 QC Booking

QC provider list from saved contacts.

AI-recommended inspection points (A2).

Quotation acceptance.

Report viewing (PASS/FAIL/CONDITIONAL).

Conditional pass action plan.

Re-inspection request.

12C.2.6.9 Document Finalisation

Packing list PDF generation.

Commercial invoice XML/PDF generation (UBL 2.1, ETA-compliant).

Digital signature (ZITADEL passkey).

Auto-submission to ETA and Nafeza.

12C.2.6.10 Barcode Print

Label templates (Standard, Customs-Ready, Consignee).

ZPL generation (direct TCP/IP).

PDF generation (A4 label sheets).

Reprint request (post-loading) with Governor decision.

12C.2.6.11 Distressed Cargo & Notifications

Declaration with photo upload.

AI condition assessment (HF ViT, A2).

Dynamic AI pricing (XGBoost, A2).

Triage dashboard (three paths).

"Check Buyers" advisory (A2 LightGBM ranking).

Accelerated outreach with privacy notice.

Express negotiation (max 3 rounds).

Microcontract lock with distressed fee.

12C.2.6.12 Disputes

Dispute list (active where seller is respondent).

Response filing (accept/reject/counter).

Mediation log with Groq translation.

Evidence package (auto-compiled + manual upload).

Third-party expert invitation.

AI settlement proposal.

Arbitration case preparation.

12C.2.6.13 Saved Contacts

Contact list (buyers).

Trust portrait (A1 Groq).

Relationship health score (LSTM, A2).

Block/flag functionality.

Manual addition by GTID.

12C.2.6.14 Cash Position

90-day rolling forecast chart.

Detailed ledger with due dates.

Currency normalisation (ECB rates).

Export functionality.

12C.2.6.15 Company Admin

Users & Roles with seller-specific role templates.

Data Scopes with hidden_cost_components.

Workspaces (multi-company).

Settings (profile, branding, notifications).

Logistics Provider Catalogues (pre-configured rates).

Integrations (API keys).

Exit Centre (data export, account closure).

12C.2.6.16 Testing

Full seller journey (accept request → lock price → pack → get logistics → submit quote → pay fee → acknowledge release → handle distressed if needed → dispute response).

Dual-mode context switching (Seller ↔ Buyer).

Multi-shipment request and schedule modification.

Non-uniform layer stacking and validation.

All three logistics modes (A, B, C) and combined.

Conditional QC with action plan and hold removal.

Re-inspection request and acceptance.

Distressed cargo declaration and accelerated outreach.

EXW price deviation justification.

Post-lock price watch alert.

Voice-activated milestone confirmation.

12C.2.7 AI Authority Summary for Part 12C.2

END OF PART 12C.2 — TRADER PORTAL — SELLER DASHBOARD (v11.0 FULLY EXPANDED)

PART 12C.3 — LOGISTICS SERVICE PROVIDER (LSP) PORTAL

12C.3.0 Access & Purpose

12C.3.0.1 Access Requirements

Access:

Tenants with type = LSP (sub-types: TRUCKING, FORWARDER, WAREHOUSING).

Employees must have completed KYB Tier 2 verification (business licence, insurance, fleet list for trucking, etc.).

Role clarification:

TRUCKING — Road transport provider, manages fleet, drivers, and pickups/deliveries.

FORWARDER — Freight forwarder, consolidates shipments, manages subcontractor networks, and builds bundled quotes.

WAREHOUSING — Warehouse operator, manages storage, inventory, and temperature-controlled facilities.

Dual-mode toggle: ☒ Not applicable (LSPs do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows pending RFQs, active shipments, compliance alerts, outstanding payments.

User can set preference to land directly on RFQ Inbox.

Portal philosophy:

LSPs only see data for shipments where they have been invited to quote or have accepted a service agreement.

No discovery of other traders; all relationships originate from seller/buyer invitations.

Anonymous RFQs (opt-out per LSP) show only HS chapter, volume, origin/destination, loading window, match score — no counterparty identity until quote acceptance.

The platform never ranks LSPs or suggests "top rated" providers to traders.

LSPs can maintain private service catalogues with their own rates.

12C.3.0.2 LSP Portal Purpose

The LSP Portal is the complete workspace for logistics service providers. It enables:

RFQ management — Receive and respond to requests for quotation (directed or anonymous broadcast), with structured clarification requests.

Quote submission — Submit competitive quotes with AI-assisted pricing suggestions.

Job execution — Track active shipments, confirm milestones (voice-enabled), and manage addendum signing.

Dispatch planning (trucking) — Optimise daily routes with ORTools VRP, assign drivers, and manage geofence alerts.

Warehouse management (warehousing) — Manage inbound/outbound shipments, storage utilisation, and temperature logs.

Consolidation (forwarding) — Manage subcontractor networks, LCL consolidation, and bundled quote building.

Mobile fleet management — Manage driver mobile app connectivity, offline scanning, and route pushing.

Performance monitoring — View on-time delivery, dispute rate, invoice accuracy, and anonymous benchmarks.

Financial management — Track invoices, payments, and PSP split confirmations.

Administration — Manage employees, service catalogues, fee schedules, and notification preferences (including anonymous RFQ opt-out).

12C.3.0.3 LSP Portal — High-Level Workflow

text



Example Dispatch Planner:

text

DISPATCH PLANNER — 14 Jun 2026

[Optimise Routes]

[Assign Drivers]

[Send to Driver App]

[View All Routes]

Route 1 — Truck FL772 (Driver: Ahmed)

Total Distance: 320 km

Estimated Time: 6.5h

Empty Miles: 0 km

Stop 1: 08:00 — Pickup Container TCNU123 — Beheira Warehouse

Stop 2: 10:30 — Pickup Container TCNU456 — Saigon Port

Stop 3: 14:00 — Deliver Container TCNU123 — Damietta Port

Stop 4: 16:30 — Deliver Container TCNU456 — Port Said

[Edit Route]

[Assign Driver]

[View Map]

Route 2 — Truck FL773 (Driver: Mohamed)

Total Distance: 180 km

Estimated Time: 4.0h

Empty Miles: 15 km

Stop 1: 09:00 — Pickup Container TCNU789 — Cairo Logistics Hub

Stop 2: 11:30 — Deliver Container TCNU789 — Alexandria Port

[Edit Route]

[Assign Driver]

[View Map]

■ OUTBOUND LOG ■

■ ■ USTN ■ Container ■ Ready Since ■ Status ■ Action ■ ■

■ ■ SGTX-EG-003 ■ MEDU1234569 ■ 13 Jun 16:00 ■ ■ Ready ■ [Mark Loaded] ■ ■

Tab 6: Forwarder Console (Forwarding-only)

Example Forwarder Console:

■ Subcontractor Network: ■

■ ■ Partner ■ Service ■ Rate ■ Availability ■ Actions ■ ■

■ ■ Fast Trucking ■ Trucking ■ \$0.85/km ■ ■ Available ■ [Select] ■ ■

■ ■ Express Cargo ■ Trucking ■ \$0.78/km ■ ■ Full ■ [Select] ■ ■

11

■ LCL Consolidation Planner: ■

■ ■ ■ Consolidation Opportunity for USTN-EG-001 and USTN-EG-002 ■ ■ ■

■ ■ Both shipments can be consolidated into one 40ft container. ■ ■

■ ■ Estimated savings: \$850 ■ ■

■ ■ [Suggest Consolidation] ■ ■

■ ■
■ ■

■ ■

■ Bundled Quote Builder: ■

■ Services Selected: ■ Ocean Freight ■ Trucking ■ Customs Brokerage ■

■ ■
■ ■

■ Ocean Freight: \$4,200 (MSC) ■

■ Trucking: \$147.50 (Fast Trucking) ■

■ Customs Brokerage: \$150.00 (SGS) ■

■ ■
■ ■

■ Total Bundled Quote: \$4,497.50 ■

■ [Build Bundle] [Send Quote] ■

■ ■
■ ■

Tab 7: Driver Mobile App (Informational & Management)

Purpose: Provide LSP administrators with tools to manage driver mobile app connectivity and offline capabilities.

Example Mobile App Management:

text

■ ■
■ ■

■ DRIVER MOBILE APP MANAGEMENT ■

■ ■
■ ■

■ QR Code Pairing: ■

■ ■
■ ■

■ ■ [QR Code] ■ ■

■ ■ Scan this QR code with the SGTx Driver App to pair your device. ■ ■

■ ■ [Generate New QR] ■ ■

■ ■
■ ■

■ ■

Offline Scanning Queue:

USTN	Pallet ID	Scanned At	Status	Action
SGTX-EG-001	PAL-005	13 Jun 14:23	Pending Sync	[Retry Sync]
SGTX-EG-001	PAL-006	13 Jun 14:25	Pending Sync	[Retry Sync]

Driver Assignment:

Driver	Vehicle	Route	Status	Action
Ahmed	FL772	Route 1	Pushed	[Push Again]
Mohamed	FL773	Route 2	Pending	[Push]

Tab 8: Performance

Purpose: View on-time delivery percentage, dispute rate, invoice accuracy, and anonymous benchmark against other LSPs in same category and region (differential privacy).

Example Performance Dashboard:

text

Metric	30 Days	60 Days	90 Days	Benchmark
On-time	89%	87%	85%	82% (avg)

□ □

■ ■ INV-TRK-002 ■ SGTX-EG-002 ■ Trucking ■ \$200.00 ■ ■ Pending ■ ■

■ ■ INV-TRK-003 ■ SGTX-EG-003 ■ Trucking ■ \$130.00 ■ ■ Overdue ■ ■

22

Tab 10: Company Admin

Example Service Catalogue:

■ SERVICE CATALOGUE — Fast Trucking Co. ■

■ Routes: ■

■ ■ Route ■ Vehicle Type ■ Fee (USD/km) ■ Est. Transit ■ Actions ■ ■

■ ■ Beheira → ■ Reefer ■ \$0.85 ■ 4h ■ [Edit] [Remove] ■ ■

■ ■ Cairo → ■ Dry van ■ \$0.65 ■ 3h ■ [Edit] [Remove] ■ ■

■ ■ Alexandria → ■ Flatbed ■ \$0.90 ■ 2.5h ■ [Edit] [Remove] ■ ■

■ ■

■ [Add Route] [Save] ■



12C.3.3 Smart Inbox Integration (LSP)

12C.3.4 Shipments Vault — LSP Columns

12C.3.5 One-Click Count Summary (LSP)

12C.3.6 Integration with Other Parts

12C.3.7 Implementation Checklist (LSP Portal)

12C.3.7.1 Universal Command Center

Build Universal Command Center (Part 12G) with LSP-specific cards (Open RFQs, Active Shipments, Pending Payments, Compliance Alerts, Performance Summary).

Add quick actions (Respond to RFQ, View Dispatch Planner, Generate Invoice, Pair Driver App, Update Service Catalogue, Contact Support).

Integrate AI Operations Assistant (A1, Groq).

12C.3.7.2 RFQ Inbox

RFQ list with match score (A1), service type, route, commodity, volume, loading window.

Directed vs anonymous RFQ visibility.

Clarification request workflow (Improvement #4) — structured Q&A.

Quote submission with pre-filled catalogue and AI Quote Assistant (A1).

Decline with mandatory reason.

12C.3.7.3 Active Shipments

Shared Shipments Vault filtered to LSP-accepted jobs.

Milestone confirmation (voice-enabled, A1).

Container release acknowledgment (Part 8).

Addendum signing.

12C.3.7.4 Dispatch Planner (Trucking-only)

ORTools VRP route optimisation (A2).

Driver assignment (drag-and-drop).

Geofence alerts.

Voice navigation integration (push to driver app).

12C.3.7.5 Warehouse Dashboard (Warehousing-only)

Inbound log with "Mark Received".

Outbound log with "Mark Loaded".

Storage utilisation chart.

Temperature logs with anomaly detection (A2).

Packing supervision.

12C.3.7.6 Forwarder Console (Forwarding-only)

Subcontractor network (only saved contacts).

LCL consolidation planner (ORTools, A2).

Bundled quote builder.

12C.3.7.7 Driver Mobile App Management

QR code pairing (generate, revoke).

Offline scanning queue (view, retry sync).

Driver assignment list (assign, push).

Geofence configuration (add, edit, delete).

12C.3.7.8 Performance

On-time delivery % (per trade lane, rolling periods).

Dispute rate (with breakdown).

Invoice accuracy.

Anonymous benchmark (differential privacy, $\epsilon=0.1$, A2).

Groq performance summary (A1).

Download report.

12C.3.7.9 Invoices & Payments

Invoice list with status filtering.

Invoice detail with PDF download.

Payment confirmation (PSP webhook).

Dispute link.

12C.3.7.10 Company Admin

Employee management (invite, roles: Dispatcher, Driver, Viewer, Admin).

Service catalogue (routes, vehicle types, fees, validity periods).

Fleet list upload (trucking) / warehouse capacity (warehousing).

Mobile app integration (QR pairing, offline sync).

Anonymous RFQ opt-out toggle.

Licences & insurance upload (RIA validation).

Branding (logo, colour scheme).

12C.3.7.11 Testing

Test LSP RFQ flow: receive RFQ → ask clarification → send quote → quote accepted → sign addendum
→ acknowledge release → confirm milestones → receive payment.

Test directed vs anonymous RFQ visibility.

Test offline sync (driver app → queue → sync).

Test dispatch planner (optimise → assign → push).

Test warehouse dashboard (receive → store → load out).

Test forwarder console (subcontractor selection → consolidation → bundle quote).

Test anonymous RFQ opt-out (LSP preference).

Test performance dashboard (metrics, benchmark, Groq summary).

12C.3.8 AI Authority Summary for Part 12C.3

END OF PART 12C.3 — LOGISTICS SERVICE PROVIDER (LSP) PORTAL (v11.0 FULLY EXPANDED)

PART 12C.4 — SHIPPING LINE (SHIP) PORTAL

12C.4.0 Access & Purpose

12C.4.0.1 Access Requirements

Access:

Tenants with type = SHIP (ocean carriers, NVOCCs, vessel operators).

Employees must have completed KYB Tier 2 verification (IMO number, vessel registry, eBL platform details, etc.).

Role clarification:

SHIP — Ocean carriers, NVOCCs, and vessel operators that provide maritime freight services.

Responsible for booking confirmations, vessel schedules, eBL issuance, and milestone updates (departure/arrival).

Can maintain private contract rates with specific sellers; these rates are never visible to other tenants.

Dual-mode toggle: ■ Not applicable (shipping lines do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows pending booking requests, active shipments, eBL signature reminders, outstanding freight invoices.

User can set preference to land directly on Booking Requests.

Portal philosophy:

Shipping lines only see bookings where they have been invited to quote or have been selected by the seller (via Mode C — Direct to SHIP in Part 3B.3.5.3).

No discovery of other traders; all relationships originate from seller invitations.

Shipping lines can maintain private contract rates with specific sellers; these rates are never visible to other tenants.

For multi-shipment contracts (Part 3.6), each shipment has its own USTN and booking; SHIP may receive separate booking requests per shipment under the same master contract.

The platform never ranks shipping lines or suggests "top rated" carriers to traders.

eBL issuance can be automated via webhook integration or manual upload.

12C.4.0.2 SHIP Portal Purpose

The SHIP Portal is the complete workspace for shipping lines and ocean carriers. It enables:

Booking management — Receive and respond to booking requests from sellers, with optional add-on services (trucking, customs broker, insurance).

Quote submission — Submit ocean freight quotes with contract rate auto-application, surcharges, and bundled add-on pricing.

Booking confirmation — Confirm bookings, generate booking references, and update shipment status.

eBL management — Issue electronic Bills of Lading via webhook integration or manual upload, with digital signatures.

Vessel schedule management — Maintain vessel rotations, update ETD/ETA, and propagate changes to all affected USTNs via Smart Inbox.

Freight invoicing — Generate freight invoices (credit or mandatory terms), track payment status via PSP webhooks.

Contract rate management — Store private contract rates per seller, auto-apply during quoting.

AIS integration — Provide AIS feed for vessel tracking (optional; fallback to public AIS).

Performance monitoring — View on-time departure, eBL issuance latency, dispute rate, and anonymous benchmarks.

Administration — Manage employees, serviceable ports, eBL platform credentials, webhook configuration, and branding.

12C.4.0.3 SHIP Portal — High-Level Workflow

text



2.5 Add-on Services Handling

Add-on Services Options:

text

ADD-ON SERVICES — USTN SGTX-EG-001

Trucking (Damietta region):

Quote bundled price (\$300 via Fast Trucking Co.)

Decline (seller will source separately)

Refer to partner (select from network)

Customs Broker:

Quote bundled price (\$150 via SGS Egypt)

Decline

Refer to partner

Insurance:

Quote bundled price (\$200, 110% of value)

Decline

[Save Add-on Choices]

Tab 3: Active Bookings (Confirmed & In Progress)

Purpose: View all confirmed bookings where SHIP has been selected and has signed the logistics addendum (Part 3B.4.6). Track vessel schedule, eBL status, and milestone updates.

3.1 Bookings List

Example Bookings List:

text

ACTIVE BOOKINGS (5)

Filters: [Status: All ▼] [Vessel: All ▼] [ETD: Next 14 days ▼] [Search:]

■ Port of Discharge: Hamburg ■

■ Container: MEDU1234567, MEDU1234568 ■

■ Goods: Frozen strawberries, 20,000 kg ■

111

■ [Issue eBL] [Download PDF] [Cancel] ■

11

■ ■■ eBL will be signed with ZITADEL passkey (fingerprint/face). ■

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

[illegible]

3.4 Milestone Updates

Milestone Update:

text

■ UPDATE MILESTONE — USTN SGTX-EG-001 ■

■ Vessel: MSC FIONA (MSF123E) ■

■ Current Status: BOOKED ■

11

■ Select Milestone: ■

■ ● DEPARTED — Vessel has departed ■

■ ■ ARRIVED — Vessel has arrived at destination ■

111

Actual Departure: [20 Jun 2026 09:30]

111

■ [Confirm DEPARTED] [Cancel] ■

□ □

■ ■ Voice: "Depart MSC FIONA for USTN SGTX-EG-001" ■

3.5 Container Release Acknowledgment (Part 8)

Release Token:

text

■ ■ CONTAINER RELEASE ACKNOWLEDGMENT ■ ■

■ ■

■ MSC ARIANE (MSA456F) ■

■
■

■ ■ Route: Alexandria → Rotterdam → Hamburg ■ ■

■ ■ ETD: 25 Jun 10:00 ■ ETA Alexandria: 25 Jun 10:00 ■ ■

■ ■ ETA Rotterdam: 10 Jul 16:00 ■ ETA Hamburg: 12 Jul 14:00 ■ ■

■ ■ Status: On Schedule ■ Bookings: 1 ■ ■

■ ■ [Edit Schedule] [View Bookings] [Track Vessel] ■ ■

■
■

■ ■

■ ■■ Port Congestion Alert (A2): Rotterdam port congestion detected. ■

■ Recommended: Buffer +2 days to schedule. ■

■ [View Details] ■

■
■

Edit Schedule:

text

■
■

■ EDIT SCHEDULE — MSC FIONA (MSF123E) ■

■
■

■ Voyage: Damietta → Hamburg → Rotterdam → Antwerp ■

■ ■

■ ■ Port ■ Arrival (ETA) ■ Departure (ETD) ■ Actions ■ ■

■
■

■ ■ Damietta ■ 20 Jun 08:00 ■ 20 Jun 12:00 ■ [Edit] ■ ■

■ ■ Hamburg ■ 05 Jul 14:00 ■ 06 Jul 08:00 ■ [Edit] ■ ■

■ ■ Rotterdam ■ 08 Jul 10:00 ■ 09 Jul 06:00 ■ [Edit] ■ ■

■ ■ Antwerp ■ 10 Jul 12:00 ■ 11 Jul 10:00 ■ [Edit] ■ ■

■
■

■ ■

■ [Save Changes] [Cancel] ■

■ ■

■ ■■ Changing ETD/ETA will trigger Smart Inbox alerts to all affected USTNs. ■

■ ■ Nile Foods ■ Alexandria → ■ 40ft Reefer ■ \$4,500 ■ 15 Jul 2026 ■ ■

Rotterdam

Global Port Said → 20ft Dry \$2,800 31 Aug 2026

Exports Jebel Ali

[\[Add Contract\]](#) [\[Edit\]](#) [\[Delete\]](#)

Rate Expiry Alerts:

• Mekong Fresh contract expires in 15 days — [\[Renew\]](#)

Add Contract (Manual):

text

ADD CONTRACT

Seller GTID: [SGTX-VN-TRD-002139-7F3A ▼]

Seller Name: [Mekong Fresh Co.]

Trade Lane: [Damietta → Hamburg ▼]

Container Type: [40ft Reefer ▼]

Base Rate: [\$4,200] [USD]

Validity Period: [2026-06-30]

[\[Upload Contract PDF\]](#) — AI extracts rates

[\[Save Contract\]](#) [\[Cancel\]](#)

Tab 7: Performance

Purpose: View on-time departure percentage, eBL issuance latency, dispute rate, and anonymous benchmark (differential privacy) — aligned with Part 9.8.

Example Performance Dashboard:

text

12C.4.7.4 Vessel Schedule

Interactive calendar with vessel rotations.

Edit ETD/ETA, port sequence; propagate changes via Smart Inbox.

AIS integration (optional) — SHIP-provided feed or public fallback.

Port congestion alerts (RIA, A2).

12C.4.7.5 Freight Invoices

Invoice list with credit/mandatory terms, due dates, status.

Invoice generation (PDF or structured data).

Credit terms handling (container release without payment).

Payment confirmation (PSP webhook).

Dispute link.

12C.4.7.6 Contract Rate Manager

Contract list per seller (private, never visible to other sellers).

Add contract (manual or PDF upload with AI extraction, A2).

Edit/delete contract.

Rate expiry alerts (30 days before).

12C.4.7.7 Performance

On-time departure % (per trade lane, rolling periods).

eBL issuance latency.

Dispute rate.

Anonymous benchmark (differential privacy, $\epsilon=0.1$, A2).

Groq performance summary (A1).

Download report.

12C.4.7.8 Company Admin

Employee management (invite, roles: Operator, eBL Signatory, Viewer, Admin).

Service catalogue (serviceable ports, vessel schedules, base freight rates).

eBL platform integration (webhook URL, API credentials, test connection).

AIS feed configuration (optional).

Branding (logo, colour scheme).

12C.4.7.9 Testing

Test SHIP booking request flow: receive request (via Mode C) → send quote → quote accepted → sign addendum → confirm booking → issue eBL → update milestone → receive payment (webhook).

Test add-on services bundling (trucking, customs broker, insurance).

Test eBL webhook integration with polling fallback.

Test vessel schedule edit and propagation.

Test AIS integration (SHIP-provided vs public fallback).

Test contract rate auto-application.

Test freight invoice generation and payment confirmation.

Test multi-shipment per-shipment bookings.

12C.4.8 AI Authority Summary for Part 12C.4

END OF PART 12C.4 — SHIPPING LINE (SHIP) PORTAL (v11.0 FULLY EXPANDED)

12C.5 Laboratory (LAB) Portal

12C.5.0 Access & Purpose

Access:

Tenants with type = LAB (ISO 17025 accredited testing laboratories).

Employees must have completed KYB Tier 2 verification (ISO 17025 certificate, list of accredited tests/analytes, fee schedule, sample shipment instructions).

Dual-mode toggle: ■ Not applicable (laboratories do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows pending testing jobs, result submission deadlines, payment confirmations, dispute alerts.

User can set preference to land directly on Testing Jobs.

Portal philosophy:

Laboratories only see testing jobs for which they have been invited by a seller (or buyer) via Mode B — RFQ to LSPs (Part 3B.3.5.2) or directly from the seller's Laboratory Selection tab (Part 12C.2 — Seller Dashboard, Tab 7).

No discovery of other traders; all relationships originate from seller invitations (only from saved contacts).

Test results are automatically validated against MRLs (Maximum Residue Limits) from the RIA (Part 4.1). Compliant results trigger automatic certificate issuance via Nafeza (Part 7.2.5).

12C.5.1 LAB Role Verification & Security

12C.5.2 Full Tab Specification (Workflow Order)

Tab 1: Universal Command Center (Home)

(Full specification in Part 12G)

Purpose: Primary landing page. Shows executive summary cards:

Pending Testing Jobs (count)

Results Due (deadlines)

Payment Confirmations Received

Disputes (where lab is a party)

LAB-specific quick actions: "Submit Results", "View Pending Jobs", "Update Service Catalogue".

AI: A1 (Groq) for assistant chat and activity summarisation.

Tab 2: Testing Jobs

Purpose: Receive and manage testing job requests from sellers (or buyers). View required tests, sample tracking, and submission deadlines.

One-click guarantee: Confirm Receipt (1), Submit Results (1).

Integration: Testing jobs originate from seller's Laboratory Selection (Part 12C.2, Tab 7), which uses Mode B — RFQ to LSPs (Part 3B.3.5.2) or direct quote request. After results are submitted, the system triggers the certificate workflow.

Tab 3: Result Submission (Detailed)

Purpose: Structured form for entering test results per analyte. System validates against MRLs in real time.

One-click guarantee: Submit Results (1) after all values entered.

Governance: If any analyte exceeds MRL and lab does not override, the submission is blocked with Decision Panel: "Results non-compliant. Please retest or provide override reason." Compliant results automatically trigger certificate issuance.

Tab 4: Certificates (Auto-Triggered)

Purpose: After compliant results are submitted, SGTx automatically requests certificates from Nafeza (Part 7.2.5). The lab can view issued certificates.

One-click guarantee: Download certificate (1).

Tab 5: Performance

Purpose: View turnaround time, dispute rate, accuracy benchmarks (anonymised), aligned with Part 9.8.

Tab 6: Invoices & Payments

Purpose: View invoices issued to sellers for testing services, track payment status (via Stage 1 PSP split).

One-click guarantee: Download invoice (1).

Tab 7: Company Admin

Purpose: Manage employees, serviceable commodities, accreditations, fee schedules, sample instructions.

12C.5.3 Smart Inbox Integration (LAB)

12C.5.4 One-Click Count Summary (LAB)

12C.5.5 Implementation Checklist (LAB Portal)

Universal Command Center: LAB-specific cards (pending jobs, due results, payments).

Testing Jobs: List with USTN, seller, required test panel, sample tracking, status, due date. Quote acceptance (if not already quoted). Sample receipt confirmation. Testing start (optional). Result submission structured form.

Result Submission: Dynamic table of required analytes (from RIA). Real-time MRL validation (A2). Override with mandatory reason. Compliance conclusion auto-generation. One-click submit, Loom-hashed storage.

Certificates: Display of requested certificates with status. Auto-trigger Nafeza certificate request after compliant results. Download PDF certificates. Failure handling and re-request.

Performance: Turnaround time, dispute rate, anonymous benchmark, Groq summary.

Invoices & Payments: Invoice list, payment status (PSP webhook), dispute link.

Company Admin: Employee management, service catalogue (test panels, fees, analytes), sample instructions, accreditation upload, branding.

12C.6 QC Inspection Portal

12C.6.0 Access & Purpose

Access:

Tenants with type = QC (ISO 17020 accredited inspection providers).

Employees must have completed KYB Tier 2 verification (ISO 17020 certificate, list of serviceable commodities, fee schedule, geographic coverage).

Dual-mode toggle: ■ Not applicable (QC inspectors do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows pending inspection jobs, due dates, dispute alerts, payment confirmations.

User can set preference to land directly on Inspection Jobs.

Portal philosophy:

QC providers only receive inspection requests from buyers (or sellers) who have explicitly added them as contacts (Network feature).

No discovery or ranking; all relationships originate from trader invitations.

Inspectors use a mobile app (offline capable) with AR overlay, AI defect detection (HF ViT), and mandatory override accountability.

Conditional Pass (Part 3B.6.4.2) allows loading to proceed with a hold pending action plan completion.

Re-inspection (Part 3B.6.4.3) can be requested by either party if initial verdict is disputed.

Dispute fast-track automatically flags inspector overrides in evidence packages (Part 10.11).

12C.6.1 QC Role Verification & Security

12C.6.2 Full Tab Specification (Workflow Order)

Tab 1: Universal Command Center (Home)

(Full specification in Part 12G)

Purpose: Primary landing page. Shows executive summary cards:

Pending Inspection Jobs (count)

Due Today / Overdue

Payment Confirmations Received

Disputes (where QC inspector is a party)

QC-specific quick actions: "Start Inspection", "Submit Report", "View Disputes".

AI: A1 (Groq) for assistant chat and activity summarisation.

Tab 2: Inspection Jobs

Purpose: Receive and manage inspection job requests from buyers (or sellers). View sampling plan, due dates, and job status.

One-click guarantee: Accept Job (1), Start Inspection (1).

Integration: Inspection jobs originate from buyer's QC Booking (Part 12C.1, Tab 8) or seller's QC Booking (Part 12C.2, Tab 8). The buyer/seller selects a QC provider from saved contacts, sends a quote request, and upon acceptance, the job appears here.

Tab 3: Mobile App Integration & Offline Inspection

Purpose: Enable inspectors to perform on-site inspections using a mobile app (React Native + Expo, offline-first). The app supports barcode scanning, AR overlay, AI defect detection, and override accountability.

One-click guarantee: Scan pallet (1 per pallet, or batch mode), Override (1 + reason), Submit (1).

Note: For high-volume inspections, batch scan mode allows scanning multiple pallets without intervening clicks; system groups them and confirms after a short pause (Part 3B.6.3).

Tab 4: Report Submission & Verdicts

Purpose: After inspection (via mobile app), the inspector submits a final report with one of three verdicts: PASS, FAIL, or CONDITIONAL.

One-click guarantee: Select verdict (1), Submit (1). For CONDITIONAL: Fill plan (data entry) + Submit (1).

Governance: If verdict is FAIL, the loading milestone is blocked. The buyer/seller can request re-inspection. If verdict is CONDITIONAL, the action plan deadline is tracked; if missed, system escalates (Part 3B.6.8 — Stuck Trade Recovery).

Tab 5: Re-inspection Requests

Purpose: Handle requests for a second inspection when the initial report is disputed (FAIL verdict or buyer disputes a PASS/CONDITIONAL).

Tab 6: Dispute Fast-Track & Override Flagging

Purpose: When a dispute involves a QC report, the evidence package automatically includes all overrides with original AI confidence and inspector's reason (Part 10.11). This tab allows QC providers to view ongoing disputes and respond.

Tab 7: Performance

Purpose: View override rate, dispute rate, average turnaround time, and anonymous benchmark (differential privacy) — aligned with Part 9.8.

Tab 8: Invoices & Payments

Purpose: View invoices issued to buyers (or sellers) for inspection services, track payment status (via PSP split).

Tab 9: Company Admin

Purpose: Manage employees, serviceable commodities, regions, fee schedules, inspection methods, and mobile app configuration.

12C.6.3 Smart Inbox Integration (QC)

12C.6.4 One-Click Count Summary (QC)

12C.6.5 Implementation Checklist (QC Portal)

Universal Command Center: QC-specific cards (pending jobs, due dates, payments, disputes).

Inspection Jobs: List with USTN, buyer/seller, commodity, AQL plan, location, due date. Quote acceptance (if not already quoted). Accept job, start inspection. Mobile app pairing (QR code generation).

Mobile App Integration: Offline-first inspection (WatermelonDB). Barcode scanning (SSCC) with validation. AR overlay (expected pallet positions). AI defect detection (HF ViT) on device. Override with mandatory reason (≥ 10 chars). AQL sampling enforcement. Batch scan mode.

Report Submission & Verdicts: Verdict selection (PASS/FAIL/CONDITIONAL). Conditional pass action plan (description, deadline, escalation terms). Hold flag enforcement (Governor A4). Digital signature

(ZITADEL passkey). Loom-hashed storage.

Re-inspection Requests: List of pending requests. Accept/decline with reason. Second inspection workflow. Fee handling (dispute resolution may reassign cost).

Dispute Fast-Track: Override evidence display (original AI confidence, inspector reason). QC response submission. Mediation invitation handling. License revocation risk monitoring (A3).

Performance: Override rate, dispute rate, turnaround time, anonymous benchmark, Groq summary.

Invoices & Payments: Invoice list, payment status (PSP webhook), dispute link.

Company Admin: Employee management, service catalogue (commodities, AQL plans, fees, regions), mobile app config, ISO 17020 certificate upload, branding.

12C.7 Customs Broker (CBR) Portal

12C.7.0 Access & Purpose

Access:

Tenants with type = CBR (licensed customs brokers).

Employees must have completed KYB Tier 2 verification (broker licence, bond, professional indemnity insurance, office address for physical document receipt).

Dual-mode toggle: ■ Not applicable (customs brokers do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows pending certification requests, physical document jobs, storage reminders, audit invitations, payment confirmations.

User can set preference to land directly on Certification Requests.

Portal philosophy:

Customs brokers are optional service providers. Sellers (or buyers) select a broker from their saved contacts.

Legal clarification (Egyptian Customs Law No. 207 of 2020, Article 51): SGTX does NOT hold a customs broker licence. The platform only orchestrates data submission; the licensed broker uses their own digital seal and assumes liability.

Brokers offer four service types:

Certification — Review AI-generated declaration, apply digital seal, submit under broker's licence.

Physical handling — Receive couriered original documents, present to customs, upload stamped copies.

Storage — Store original documents after clearance, manage retention and destruction.

Audit representation — Act as legal point of contact for customs audits.

All services are quotation-driven; fees are added to Stage 1 payment plan (seller's side) or buyer's local payment batch.

12C.7.1 CBR Role Verification & Security

12C.7.2 Full Tab Specification (Workflow Order)

Tab 1: Universal Command Center (Home)

(Full specification in Part 12G)

Purpose: Primary landing page. Shows executive summary cards:

Pending Certification Requests (count)

Physical Document Jobs (by status)

Storage Retention Expiry Alerts

Audit Invitations

Payment Confirmations

CBR-specific quick actions: "Review Certification", "Receive Package", "Issue Storage Reminder".

AI: A1 (Groq) for assistant chat and activity summarisation.

Tab 2: Certification Requests

Purpose: Review AI-generated customs declarations (SAD) from sellers who have selected the broker for certification. Apply digital seal and submit to Nafeza under broker's licence.

One-click guarantee: Certify (1). Reject (1).

Integration: Certification requests originate from Seller Dashboard — Logistics Builder (Part 12C.2, Tab 5) where the seller checks "Certification" and selects a broker from saved contacts. The broker's fee is added to Stage 1 payment (Part 6.1). After certification, Nafeza submission follows Part 7.2.

Tab 3: Physical Document Jobs

Purpose: Handle physical document handling for destinations that require original paper documents (e.g., Nigeria, some African countries). RIA detects these requirements automatically.

One-click guarantee: Receive (1), Mark Presented (1), Upload (1), Confirm (1).

Tab 4: Storage

Purpose: Store original documents after customs clearance for the required retention period (e.g., 5 years for tax, 7 years for AML). Manage shelf tracking, return, or destruction.

Tab 5: Audit Representation

Purpose: Act as legal point of contact for customs audits. The broker receives audit invitations, accesses relevant documents, and communicates with the client.

Tab 6: Performance

Purpose: View certification accuracy (measured by customs rejections), handling time, client ratings — aligned with Part 9.8.

Tab 7: Invoices & Payments

Purpose: View invoices issued to sellers (or buyers) for brokerage services, track payment status (via Stage 1 PSP split).

Tab 8: Company Admin

Purpose: Manage employees, office address, insurance certificate, service catalogue, fees, and digital seal credentials.

12C.7.3 Smart Inbox Integration (CBR)

12C.7.4 One-Click Count Summary (CBR)

12C.7.5 Implementation Checklist (CBR Portal)

Universal Command Center: CBR-specific cards (certification requests, physical jobs, storage alerts, audit invites, payments).

Certification Requests: List with USTN, seller, AI confidence score. Declaration preview with low-confidence highlighting. Quote acceptance (if not already). One-click "Certify" with digital seal (mTLS). Nafeza submission integration (Part 7.2). Reject / request amendment with reason.

Physical Document Jobs: Job list with courier tracking, status. QR code scanning via mobile app. "Receive Package" with GPS timestamp. "Mark Presented" to customs. Upload stamped copy (AI extraction). "Submission Confirmed" milestone.

Storage: Storage list with shelf location, retention expiry. Update shelf location. Retention reminders (automated). Notify client, extend storage, destroy, return workflows.

Audit Representation: Audit invitations list. Accept / decline representation. Document access (read-only). Communication log. Record audit outcome.

Performance: Certification accuracy, handling time, client ratings, Groq summary.

Invoices & Payments: Invoice list, payment status (PSP webhook), dispute link.

Company Admin: Employee management, service catalogue (fees), office address, insurance/bond upload, digital seal management, branding.

12C.8 Financier (Bank) Portal

12C.8.0 Access & Purpose

Access:

Tenants with type = FIN and sub_type = BANK.

Employees must have completed KYB Tier 3 verification (CBE accreditation for Egyptian banks, equivalent local regulatory approval for other jurisdictions).

Dual-mode toggle: ☒ Not applicable (financiers do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows financing opportunities, active loans, margin call alerts, repayment reminders, regulatory report deadlines.

User can set preference to land directly on Financing Opportunities.

Portal philosophy:

Banks receive auto-RFQ broadcasts for financing requests that match their pre-set preferences (Part 3B.5.3).

Full disclosure — Banks see complete trade details (buyer/seller names, documents, historical performance, previous financing history) before bidding. This is the key trust advantage of SGTX trade finance.

Co-financing — Multiple banks can each finance a portion of the same request.

Jurisdiction-aware — DeFi features are completely hidden for banks in jurisdictions where defi_allowed = false (e.g., Egypt under Law No. 194 of 2020). For jurisdictions where allowed, DeFi is optional with mandatory risk acknowledgment.

ZK proofs (optional) — Banks can provide zero-knowledge proof of reserves without revealing wallet balance.

Collateral monitoring — Real-time LTV tracking, margin calls, liquidation risk prediction.

Regulatory reporting — Automated generation of CBE monthly remittance reports (Egypt), ECB statistical returns (EU), FinCEN SAR (USA), etc.

12C.8.1 Financier (Bank) Role Verification & Security

12C.8.2 Full Tab Specification (Workflow Order)

Tab 1: Universal Command Center (Home)

(Full specification in Part 12G)

Purpose: Primary landing page. Shows executive summary cards:

Financing Opportunities (match score >80) — count

Active Loans (outstanding principal)

Margin Calls Pending

Repayments Due (next 7 days)

Regulatory Report Deadlines

Bank-specific quick actions: "View Opportunities", "Submit Bid", "Generate CBE Report", "Issue Margin Call".

AI: A1 (Groq) for assistant chat, portfolio summary, and activity summarisation.

Tab 2: Financing Opportunities (Auto-RFQ Inbox)

Purpose: Receive financing requests that match the bank's pre-set preferences. Each request includes full disclosure of trade details, counterparty performance, and AI credit intelligence.

One-click guarantee: Submit Bid (1) after filling form. Acknowledge DeFi risk (1).

Integration: Financing requests originate from Seller Dashboard — Financing (Part 12C.2, Tab 14) or Buyer Dashboard — Financing (Part 12C.1, Tab 9). The borrower clicks "Request Financing" → credit intelligence computed → auto-RFQ broadcast to matching financiers (Part 3B.5.3). Banks receive opportunities in this tab.

Tab 3: My Bids & Active Loans

Purpose: Track submitted bids (active, outbid, expired) and manage active loans (collateral monitoring, margin calls, repayments).

One-click guarantee: Increase Bid (1), Withdraw (1), Issue Margin Call (1), View Dashboard (1).

Integration: Active loans are created after borrower accepts bids (Part 3B.5.8) and financing agreement signed (Part 3B.5.9). Disbursement follows (Part 3B.5.10) — PSP split deducts 0.25% fee. Repayment monitoring is automatic via PSP webhooks or on-chain events (Part 3B.5.11).

Tab 4: DeFi (Jurisdiction-Dependent — Hidden for Egypt)

Visibility logic: This tab is only shown if all of the following are true:

Bank's jurisdiction has `defi_allowed = true` in the jurisdictions table.

Bank has explicitly enabled DeFi in Company Admin preferences.

RIA has not detected a regulatory prohibition update within the last 24 hours.

For Egypt (and jurisdictions where `defi_allowed = false`): The DeFi tab is completely hidden from the navigation menu. No UI element suggests its existence. The bank only sees conventional financing options.

Tab 5: Secondary Market

Purpose: Tokenised trade assets (ERC-3525). Banks can list their own tokenised invoices for sale or purchase tokens from other financiers.

Tab 6: Portfolio & Compliance

Purpose: Portfolio summary, exposure breakdown, regulatory report generation (CBE monthly remittance, ECB, FinCEN SAR, etc.), and continuous compliance alerts.

Tab 7: Financed Companies

Purpose: Private, read-only, audit-traced directory of every borrower the bank has ever financed. This data is never shared with other financiers.

Tab 8: Company Admin

Purpose: Financier-specific configuration: preferences, risk appetite, exposure limits, notification settings, API key management, DeFi opt-in (where allowed), and team management.

12C.8.3 Smart Inbox Integration (Bank)

12C.8.4 One-Click Count Summary (Bank)

12C.8.5 Implementation Checklist (Bank Portal)

Universal Command Center: Bank-specific cards (opportunities, active loans, margin calls, report deadlines).

Financing Opportunities: Auto-RFQ list with match score, credit intelligence, deadline. Full disclosure page (trade details, counterparty performance, previous financing history, documents). Bid submission form (encrypted, co-financing capable, DeFi options jurisdiction-aware). DeFi PlainLanguage Risk Summary modal (mandatory acknowledgment). ZK reserve proof checkbox and verification.

My Bids & Active Loans: Active Bids sub-view (increase, withdraw, outbid alerts). Won Agreements sub-view (active loans with LTV, repayment health). Collateral Monitoring Dashboard (LTV gauge, history chart, liquidation risk meter, margin call log, price drop simulator). Issue margin call (one-click, pre-filled modal).

DeFi (jurisdiction-dependent): Protocol comparison table (risk score, TVL, APY, recommendation). Stablecoin health widget (peg deviation, action status). My DeFi positions (health factor, early warning, add collateral, view on-chain). Visibility gated by `defi_allowedflag` (Egypt: hidden).

Secondary Market: My Listed Tokens (modify price, cancel listing). Available Tokens (analyse, make offer). Jurisdiction filtering (crypto-backed tokens hidden for Egypt).

Portfolio & Compliance: AI portfolio summary (A1). Exposure breakdown charts. Risk appetite validation. Regulatory report generation (CBE, ECB, FinCEN SAR, VAT/GST). Continuous compliance alerts.

Financed Companies: Company list (private, audit-traced). Company profile with credit score trend, financing history, refresh assessment.

Company Admin: Role templates (`credit_analyst`, `trader`, `compliance`, `risk_manager`, `viewer`). Natural language risk appetite parsing (A2). Preferences configuration (countries, trust score, amount, financing types, settlement methods, excluded commodities). Exposure limits (per commodity, per country). Notification preferences. API key management. DeFi opt-in (where jurisdiction allows).

12C.9 Financier (PFI) Portal

12C.9.0 Access & Purpose

Access:

Tenants with `type = FIN` and `sub_type = PRIVATE` (family offices, private credit funds, factoring companies, peer-to-peer lenders).

Onboarding requires:

Proof of registration or regulatory exemption in their jurisdiction (dynamic check by RIA).

Legal basis explanation (e.g., "private lending exemption").

List of authorised signatories.

No CBE accreditation required (but may be restricted in certain jurisdictions).

Dual-mode toggle: ■ Not applicable (private financiers do not trade as buyers/sellers).

Landing page (configurable):

Default: Universal Command Center (Part 12G) — shows financing opportunities, active loans, repayment reminders, portfolio performance.

User can set preference to land directly on Financing Opportunities.

Portal philosophy:

Private financiers receive auto-RFQ broadcasts similarly to banks, but with simplified compliance (no CBE reporting, no mandatory DeFi oversight).

Full disclosure — PFIs see complete trade details (buyer/seller names, documents, historical performance, previous financing history) before bidding.

Co-financing — Multiple PFIs can co-finance a single request.

Jurisdiction-aware — DeFi features are optional and only shown where `defi_allowed = true` and PFI has opted in. For Egypt, DeFi is hidden.

ZK proofs (optional) — PFIs can provide proof of reserves (not required, but available as optional trust signal).

Collateral monitoring — Same LTV tracking, margin calls, liquidation risk prediction as banks (but no regulatory reporting).

No CBE/ECB/FinCEN reporting — Instead, PFI can download portfolio CSV for internal reporting.

12C.9.1 Private Financier (PFI) Role Verification & Security

12C.9.2 Full Tab Specification (Workflow Order)

Tab 1: Universal Command Center (Home)

(Full specification in Part 12G)

Purpose: Primary landing page. Shows executive summary cards:

Financing Opportunities (match score >80) — count

Active Loans (outstanding principal)

Margin Calls Pending

Repayments Due (next 7 days)

Portfolio Return (annualised)

PFI-specific quick actions: "View Opportunities", "Submit Bid", "Download Portfolio CSV", "Issue Margin Call".

AI: A1 (Groq) for assistant chat, portfolio summary, and activity summarisation.

Tab 2: Financing Opportunities (Auto-RFQ Inbox)

Purpose: Receive financing requests that match the PFI's pre-set preferences. Identical to bank portal (Part 12C.8.2) — full disclosure of trade details, counterparty performance, and AI credit intelligence.

One-click guarantee: Submit Bid (1). Acknowledge DeFi risk (1).

Note: No minimum bid count required; PFI can bid alone. Blended rate not applicable for single financier.

Tab 3: My Bids & Active Loans

Purpose: Track submitted bids and manage active loans with collateral monitoring (simplified — no regulatory reporting).

Tab 4: DeFi (Jurisdiction-Dependent, Optional)

Visibility logic: The DeFi tab is only shown if:

PFI's jurisdiction has `defi_allowed = true` in the jurisdictions table.

PFI has explicitly enabled DeFi in Company Admin preferences (opt-in).

RIA has not detected a regulatory prohibition update.

For Egypt (and jurisdictions where `defi_allowed = false`): The DeFi tab is completely hidden.

PFI-specific note: DeFi is optional — PFI can choose to never use it and stick to conventional financing.

Tab 5: Portfolio & Performance

Purpose: Portfolio summary, return on capital, default rate, exposure breakdown. No CBE/ECB/FinCEN reporting.

Tab 6: Financed Companies

Purpose: Private, read-only, audit-traced directory of every borrower the PFI has ever financed. Identical to bank portal (Part 12C.8.7) but with simplified compliance view.

Tab 7: Company Admin

Purpose: PFI-specific configuration (simplified compared to bank). Preferences, risk appetite, exposure limits, notification settings, API key management (optional), DeFi opt-in (where jurisdiction allows).

12C.9.3 Smart Inbox Integration (PFI)

12C.9.4 One-Click Count Summary (PFI)

12C.9.5 Implementation Checklist (PFI Portal)

Universal Command Center: PFI-specific cards (opportunities, active loans, margin calls, portfolio return).

Financing Opportunities: Auto-RFQ list with match score, credit intelligence, deadline. Full disclosure page (trade details, counterparty performance, previous financing history, documents). Bid submission form (encrypted, co-financing capable, DeFi options jurisdiction-aware and opt-in based). DeFi PlainLanguage Risk Summary modal (mandatory acknowledgment if DeFi selected). ZK reserve proof checkbox and verification (optional).

My Bids & Active Loans: Active Bids sub-view (increase, withdraw, outbid alerts). Won Agreements sub-view (active loans with LTV, repayment health). Collateral Monitoring Dashboard (LTV gauge, history chart, liquidation risk meter, margin call log, price drop simulator). Issue margin call (one-click, pre-filled modal).

DeFi (jurisdiction-dependent, opt-in): Protocol comparison table (risk score, TVL, APY, recommendation). Stablecoin health widget (peg deviation, action status). My DeFi positions (health factor, early warning, add collateral, view on-chain). Visibility gated by `defi_allowed` AND PFI opt-in (Egypt: hidden).

Portfolio & Performance: AI portfolio summary (A1). Exposure breakdown charts. Return on capital, default rate. Download Portfolio CSV (one-click). Continuous compliance alerts (KYB expiry, sanctions).

Financed Companies: Company list (private, audit-traced). Company profile with credit score trend, financing history, refresh assessment.

Company Admin: Role templates (investor, admin, viewer). Natural language risk appetite parsing (A2). Preferences configuration (countries, trust score, amount, financing types, settlement methods, excluded commodities). Exposure limits (optional — advisory warnings). Notification preferences. API key management (optional). DeFi opt-in (where jurisdiction allows).

12C.10 Government Portal (GOV)

12C.10.0 Access & Purpose

Access:

Employees of tenants with type = GOV (customs, port authority, trade ministry).

Dynamic module configuration per agency (see Part 6.2.6).

Dual-mode toggle: ■

Landing page: Smart Inbox (clearance holds, missing document alerts, anonymous trade approvals, integration health warnings).

12C.10.1 Key Tabs (Ordered by Workflow, with All Modules Enabled)

12C.10.2 Detailed Tab Specifications

Tab 1: Inbox

(Same as common Smart Inbox, but filtered to government-relevant items: clearance holds, missing phyto certificates, anonymous trade approvals, integration health warnings.)

Smart Inbox examples:

■ High — "Clearance hold — missing phytosanitary certificate, vessel arrived 2h ago" (95)

■ Medium — "AI risk score 68, manual inspection recommended" (75)

■ Low — "Monthly CBE report generated" (20)

Tab 2: Live Trade Monitor

Renders the Shared Shipments Vault (Part 12A.4) with a filter preset to the government's jurisdiction (e.g., all shipments where destination, origin, or transit country equals the agency's country code).

Additional columns visible to government users:

Risk Score — SGTX AI-computed risk (0-100), colour-coded.

Document Readiness — traffic light (green=all docs verified, amber=some pending, red=critical missing).

Declaration Status — if customs integration active (e.g., Nafeza), shows status of auto-submitted declaration.

Integration Badge — small icon indicating whether the shipment's data has been successfully pushed to the national system.

Clicking any USTN opens the Trade Command Center with full shipment details (commercial data hidden, compliance data prominent).

Filters: risk score range, commodity, port, vessel, document completeness.

Tab 3: Clearance Workflow

The core operational tab for customs. Displays a queue of shipments that have arrived at the border or are approaching the port.

AI Clearance Recommendation Card (per shipment):

text

USTN: SGTX-EG-20260415120000-ABC12345-V1

Vessel: MSC Alessia -- Arrived Alexandria, 10 Jul 2026

Risk Score: 22 (Low)

Documents: 5/5 verified

Recommendation: AUTOCLEAR (eligible under current autoclearance threshold of 30)

[Approve Clearance] [Override → Manual Review] [Hold for Inspection]

If auto_clearance module enabled and risk score $\leq$ threshold, the system automatically marks the shipment as "Cleared" and pushes status to seller/buyer. The officer sees a log entry.

If risk score above threshold, the officer must manually review. Options: Approve Clearance(release), Hold (pause, wait for missing document or physical inspection), Reject (block with required reason).

Every decision is logged as a Governor decision (decision_type = 'customs_clearance') with officer's GTID and Loom hash.

Integration with national systems: If the customs API connector is active, clicking "Approve Clearance" automatically sends the final declaration status to the national system and stores the reference. If the connector fails, a manual fallback PDF is generated.

Tab 4: Document Verification

Lists all documents required for shipments in the jurisdiction, focusing on those pending or flagged by AI.

Actions:

Accept — override AI, mark verified.

Reject — send back with note.

Request Amendment — pre-filled message.

All actions logged.

Tab 5: Multi-Agency Workflow

For shipments requiring approvals from multiple government bodies (e.g., Customs + Agriculture + Health). The RIA determines which agencies must approve based on HS code, commodity, and destination country rules.

Each step has a responsible agency, a deadline, and a status.

The Smart Inbox of each agency shows pending steps.

Approvals/rejections are recorded with official's GTID and timestamp.

Once all steps approved, the shipment automatically moves to "Cleared" (or next step).

For anonymous trades, each agency sees only the minimum data needed (e.g., HS code, weight, phytosanitary status). Party identities are redacted unless the agency is explicitly authorised.

Example display for sequential workflow:

text

Shipment USTN: SGTX-EG-...-V1

Current step: Agriculture Ministry (PHYTO approval) -- deadline 15 Jun 2026

Previous: Customs -- ☒ approved

Next: Health Ministry -- pending

[Approve] [Reject] [Add Comments]

Tab 6: Anonymous Trade Management

Handles government-to-government and highly sensitive commodity trades where buyer/seller identities are protected.

Anonymous Trade Lifecycle:

Initiation: A government tenant creates a trade request with `is_anonymous = true`. Counterparty must also be a government tenant with `anonymous_trade_enabled = true`. SGTX generates an anonymous USTN (e.g., SGTX-ANON-20260415120000-ABC12345-V1). The real USTN is stored in `anonymous_trade_mappings` and visible only to the two governments and the Platform Governance Authority.

Document Redaction: For all logistics providers, carriers, and non-government parties, consignee/shipper names are replaced with "Government Agency — [Country]". Invoices and packing lists are automatically redacted.

Multi-Agency Approvals: Each approving agency sees only the data it needs. For example, Ministry of Agriculture sees "Live animals, 50 head, from Country X to Country Y" without knowing the specific ministries involved.

Delivery & Closure: Trade executes normally. All redacted documents carry a watermark "ANONYMOUS TRADE". Full audit log encrypted.

Anonymous Trade Management Tab UI:

★ 12C.10.6.1 Anonymous Trade Declassification Audit Log

Purpose: Provide a transparent, immutable record of all declassification actions for anonymous trades, visible to all involved governments but not to unauthorised parties.

UI: For each anonymous trade row, a "View Declassification Log" button (visible only to users with `anonymous_trade.declassification` permission — typically senior customs officers and Platform Governance Authority). Clicking opens a modal or expands a table.

Log entries include:

Requestor (GTID and legal name of the official who initiated declassification)

Multisig approvers (list of GTIDs and names who approved, with timestamps of each approval)

Declassification timestamp (when the last approval was received and the trade was declassified)

Reason (free text, mandatory, min 20 characters)

Hash of original redacted data (SHA256 of the anonymised trade record before declassification) — for comparison after declassification.

Governor decision ID linking to the Loom chain.

Example table:

Immutability: Entries are stored in a dedicated table `anonymous_trade_declassification_log` and are Loom-chained. No deletion or modification is possible.

Notification: When a declassification occurs, all involved governments (buyer's and seller's jurisdictions) receive a Smart Inbox notification (priority 85) with a link to the audit log.

One-click: "View Log" — one click. No action required to log — automatically recorded.

Tab 7: Integrations

Allows government admin to manage connectors to national systems. The RIA continuously updates a library of available connectors based on scraped APIs, EDIFACT standards, open data.

Connector examples:

Customs Declaration API (Nafeza, VNACCS, ICS2, ACE)

Phytosanitary Certificate Verification (TRACES)

Certificate of Origin Verification

Vessel Arrival Notification (AISHub)

Permit Issuance (Return API)

Single Window Integration (TradeNet)

Sanctions List Sync

Setup:

Admin selects a connector from the library.

Enters credentials (API key, client certificate, SFTP address).

Clicks "Test Connection". If successful, the integration is activated and appears as a badge in Live Trade Monitor.

All connector calls are logged in `integration_connector_logs`.

Example — Egypt Customs with Nafeza: When a shipment arrives and documents are verified, the system auto-populates the XML declaration, signs it with the broker's certificate (or customs officer's), and submits to <https://nafeza.gov.eg/api/v1/declaration>. Response (reference number, status) is stored. Officer sees "Declaration NFZ12345 submitted — Accepted." If Nafeza is down, the connector health degrades and Smart Inbox alerts the officer.

Tab 8: Permit Issuance

For agencies like Agriculture or Health that issue import/export permits. Officials review applications (auto-filled with trade data), check AI-verified documents, and click "Issue Permit". The system generates a digitally signed permit (PDF), stores it in documents, and updates the shipment's document checklist. If `api_integration` is active, the permit is simultaneously pushed to the national system.

One-click: Click "Issue Permit" — permit generated and stored.

Tab 9: Audits & Reports

Government audit teams can:

Search the full Governor decision log for their jurisdiction.

View Loom-chained audit trail of any USTN.

Verify data integrity by clicking "Verify Loom Chain", which recalculates hashes clientside.

Generate jurisdictional regulatory reports (e.g., CBE monthly remittance, trade statistics) with one click; Groq drafts the narrative.

Access the Suspicious Activity Report (SAR) drafting and filing interface (see Part 1.12).

One-click: Click "Generate Report" — PDF ready. Click "Verify Loom Chain" — integrity check performed.

Tab 10: Compliance Monitor (New Tab)

Purpose: Provide government officials with a single dashboard view of platform-wide compliance metrics relevant to their jurisdiction.

Access: Any government user with the `compliance_monitor.view` permission (default for customs officers and senior auditors).

Dashboard components (real-time, auto-refreshing every 60 seconds):

Drill-down capability: Clicking any item opens a filtered view of the Shared Shipments Vault or Disputes tab pre-filtered to the relevant items.

One-click actions:

"Send Reminder" on deferred payment expiries — one click.

"Generate Report" — exports the current dashboard view as PDF (signed).

"Investigate" on anomaly — opens filtered Shipments Vault.

AI assistance: The "Unusual Trade Patterns" widget uses A2 (Isolation Forest) to detect anomalies; Groq generates a plain-language explanation (A1).

Governance: All data is jurisdiction-filtered (government only sees shipments where their country is origin, destination, or transit). The dashboard is read-only; no actions modify trade state except "Send Reminder" (which just triggers a Smart Inbox notification).

Refresh: Manual refresh button available; auto-refresh every 60 seconds can be toggled.

Tab 11: Company Admin

Standard multitenant administration (Part 6.2.8) with government-specific module configuration (enable/disable features like `auto_clearance`, `multi_agency_workflow`, `anonymous_trade`, etc.). Admins can also configure connector credentials and agency branding.

12C.10.3 Declassification Database Schema Addition

sql

```
CREATE TABLE anonymous_trade_declassification_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  anonymous_ustn TEXT NOT NULL,  
  requestor_gtid TEXT NOT NULL,  
  approver_gtids TEXT[],  
  reason TEXT NOT NULL,  
  redacted_data_hash TEXT,  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  loom_hash TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

12C.10.4 Implementation Checklist (Government Portal)

Universal Command Center: Government-specific cards (clearance holds, auto-clearance stats, pending multi-agency approvals).

Live Trade Monitor: Shared Shipments Vault filtered to jurisdiction, with risk scores, clearance status, integration badges.

Clearance Workflow: Queue of shipments awaiting clearance, AI clearance recommendation (risk score, auto-clearance threshold), actions: Approve Clearance, Hold, Reject.

Document Verification: List of flagged documents, AI-extracted fields, discrepancies, actions: Accept, Reject, Request Amendment.

Multi-Agency Workflow: Sequential/parallel approvals with other ministries, visual stepper, approve/reject actions.

Anonymous Trade Management: Redacted government-to-government trades, declassification with multisig approval, declassification audit log.

Integrations: Connector management (Nafeza, VNACCS, etc.), test connection, view logs, configure credentials.

Permit Issuance: Issue digital import/export permits linked to USTN, auto-fill from trade data, sign with digital seal.

Audits & Reports: Loom chain verification, regulatory reports (CBE, ECB, etc.).

Compliance Monitor: Real-time dashboard of platform-wide compliance metrics (sanctions alerts, deferred payment expiries, dispute volume, fee collection success, auto-clearance stats, unusual trade patterns).

Company Admin: Dynamic module configuration (enable/disable features), agency profile, contact information.

12C.11 Admin Portal (Platform Governance Authority)

12C.11.0 Access & Purpose

Access: Multisig members (3/5). Authentication requires ZITADEL passkey (WebAuthn) with mandatory biometric verification. Every irreversible action requires multisig approval collected asynchronously through the portal's approval queue.

Dual-mode toggle: ■

Landing page: Smart Inbox (multisig requests, system alerts, policy drift warnings).

12C.11.1 Key Tabs

12C.11.2 Detailed Tab Specifications

Tab 1: Inbox

(Same as common Smart Inbox, but filtered to admin-relevant items: multisig requests, system alerts, policy drift warnings.)

Smart Inbox examples:

■ High — "Multisig request #MS042: approve new jurisdiction matrix entry for Nigeria" (100)

■ Medium — "PSP health alert: Fawry degraded (score 42), fallback activated" (85)

■ Low — "Configuration change — version 127 applied" (30)

Tab 2: Platform Health

The command centre for real-time operational awareness. Renders four collapsible zones:

Tab 3: Constitutional Policies

Allows the multisig to edit the platform's Rego policies and recompile WASM constitutional modules.

Tab 4: Governor Log & Audit

Immutable decision explorer with natural-language querying powered by Groq converting plain English to ClickHouse SQL. Admins can ask: "Show all decisions where fee release was blocked in the last week," or "List trades where the buyer was from Egypt and a dispute was filed." Results are paginated and exportable to CSV. Clicking a row shows the full decision JSON, including the tenant_message displayed to the user. A "Verify Loom Chain" button recalculates all hashes from genesis to the selected decision; any mismatch triggers a P0 incident.

Tab 5: Jurisdiction Matrix Editor

Editable table of all 195+ jurisdictions with fields: country code, name, sanctions level, DeFi allowed, distressed sale allowed, reporting requirements, etc.

Tab 6: PSP Manager

Monitor and manage all payment service providers. Table with name, countries served, health score, latency, failure rate, status. Actions: "Force Fallback", "Restore", "View Fallback Chain". Fallback Chain Editor per country allows reordering priority; changes require multisig approval. An "Add New Rail" form, manually or from RIA discovery, allows admins to integrate new PSPs; the system tests connectivity and, if successful, adds it to the live routing table.

Tab 7: Special Rate Manager

Provides the interface for the Platform Governance Authority to define special SGTX fee rates for specific pairs of GTIDs. This overrides the dynamically computed trade fee rate for trades between those two counterparties.

Workflow to create a special rate:

Admin clicks "Propose Special Rate". A form prompts for Seller GTID, Buyer GTID, new fee rate (0.1%–2.5%), effective date range, and a mandatory reason (free text).

The proposal is submitted as a multisig request. All other multisig members receive a Smart Inbox item: "Special rate proposal: [Seller] → [Buyer] at X%. Review."

Upon 3-of-5 approval, the Governor records a special_rate_override decision and inserts a row into the special_sgtx_fee_rates table.

Special rates can be edited or revoked before expiry via the same multisig process.

All actions are permanently logged and visible in Configuration History.

Tab 8: Incidents

Manages the lifecycle of platform incidents, with AI-assisted post-mortem drafting.

Tab 9: Marketplace Partners

Onboard and manage marketplace partners that connect via the Marketplace API.

Tab 10: Tenant Management

Provides oversight and support tools for all tenants.

Tab 11: Customer Care Hub

Provides the oversight interface for the platform's AI-powered customer support system and the human escalation queue.

Tab 12: Configuration History

All platform-wide configuration is treated as a versioned manifest. This tab provides a Git-like history and rollback capability.

Tab 13: Internal Admin

Manages the Platform Governance Authority's own membership and internal roles.

12C.11.3 Implementation Checklist (Admin Portal)

Universal Command Center: Admin-specific cards (multisig requests, system alerts, PSP health).

Platform Health: Real-time metrics, predictive node failure dashboard, PSP health, operational summary (A1).

Constitutional Policies: Policy editor (Rego), impact simulation, submit for multisig approval.

Governor Log & Audit: Natural language query over Governor decisions, Loom chain integrity verification.

Jurisdiction Matrix: Editable table with conflict detection, revert to RIA values, mass update.

PSP Manager: Health scores, fallback chains, add new PSPs.

Special Rate Manager: Propose, approve, edit, revoke special rates.

Incidents: Incident lifecycle, automated post-mortem generation (Groq).

Marketplace Partners: Onboard new partners, manage disputes, view attribution logs.

Tenant Management: Search tenants, view KYB status, impersonate tenant (readonly, multisig approval, time-limited, logged), break-glass audit view.

Customer Care Hub: Chatbot performance dashboard, human support queue, chat transcripts, escalation rules configuration.

Configuration History: Versioned manifest, diff view, rollback.

Internal Admin: Manage multisig members, internal roles, API keys.

12C.12 Marketplace Partner Portal (MP)

12C.12.0 Access & Purpose

Access: Tenants with type = MP (external platforms integrating via API). Onboarding requires a signed revenue-share agreement.

Dual-mode toggle: ☐

Landing page: Dashboard (key metrics, conversion rate, top corridors, total SGTX fees earned).

12C.12.1 Key Tabs

12C.12.2 Dashboard — Key Metrics & AI Recommendations

12C.12.3 Leads Management (Intent Inbox)

Purpose: Review all intents submitted via the API, their status, and take action.

Table columns:

Status meanings:

ACCEPTED — Lead is viable; partner can proceed to redirect buyer to SGTX for full trade creation.

REJECTED — Lead is not viable (sanctions, jurisdiction block, unsupported commodity). Reason shown in a tooltip.

CONDITIONAL — Lead has low confidence; partner must collect more information (e.g., quantity) before retrying.

Actions: View (opens modal with complete intent response, parsed specs, recommendations, next action, and Groq-generated explanation if rejected/conditional). Resolve — for conditional intents, allows partner to edit raw text or fill missing fields and resubmit (creates new intent via API).

12C.12.4 Webhook Management & Event Logs

Purpose: Configure webhook endpoints to receive real-time events (trade locked, SGTX fee settled, dispute filed). Test and monitor delivery.

UI — Webhook Endpoints:

List of registered endpoints (URL, events subscribed, status (active/inactive), created date).

Add Endpoint — form: URL, select events via checkboxes (trade.locked, sgtx_fee.settled, dispute.filed, lead.rejected), optional secret rotation policy.

Edit — change URL or events, regenerate secret.

Delete — remove endpoint.

Test — sends a test.ping event to the endpoint and shows delivery result (success/failure, response body, latency).

Event Delivery Logs:

Table showing recent deliveries: timestamp, event type, endpoint URL, HTTP status, duration, error message (if any).

Retry button for failed deliveries (manually resend the event).

Download logs as CSV for debugging.

Security: Webhook signatures are verified using the partner's Ed25519 public key (stored in marketplace_partners). The portal displays the current public key and allows rotation. A note explains: "Your public key is used by SGTX to sign all webhook payloads; you should use your private key to verify them on your server. Rotating the key invalidates the old one after a 24-hour grace period."

12C.12.5 Revenue Attribution Dispute Management

Purpose: Allow partners to dispute automatically attributed trades and track resolution.

Dispute Workflow:

Partner sees an attributed trade in the "Attributed Trades" list (on dashboard) that they believe is incorrect. They click "Dispute" — a modal opens asking for reason (free text) and optional evidence upload (PDF, image).

The dispute is created in the disputes table (type = REVENUE_ATTRIBUTION) and assigned a unique ID. The Platform Governance Authority (multisig) receives a notification.

The AI (HF Mixtral) analyses the dispute and the partner's history, generating a recommendation visible to the Platform Governance Authority in the Admin Portal.

The partner can track the dispute status in the "Disputed Attributions" tab.

Example dispute:

Once resolved (by multisig), the partner sees the outcome (attribution upheld, reversed, or adjusted). If reversed, the revenue share is removed from the partner's account and the trade is reattributed (or marked as unattributed). The partner receives a Smart Inbox notification.

12C.12.6 API Key Management & Usage Analytics

Purpose: Self-service management of API keys, monitoring of API usage, and real-time rate limit visibility.

12C.12.7 Integration Guide & Test Sandbox

Purpose: Provide a self-service sandbox environment for partners to test their integration before going live.

Data Isolation: Sandbox data is stored in a separate PostgreSQL schema (sandbox_* tables) and is completely isolated from production. No real trades are affected. All sandbox data is automatically purged every 24h.

12C.12.8 Revenue Share Agreement Management

Purpose: View, download, and propose amendments to the revenue share agreement between the partner and SGTX.

12C.12.9 Implementation Checklist (Marketplace Partner Portal)

Dashboard: Real-time metrics (leads, conversion rate, revenue share earned, top corridors), AI Summary Card (A1).

Leads Management: Intent inbox with viability score, status (ACCEPTED, REJECTED, CONDITIONAL), actions: View Details, Resolve.

Webhook Management: Register webhook endpoints (URL, events subscribed), test ping, view delivery logs, retry failed deliveries.

Revenue Attribution Disputes: List of disputed attributed trades, status, AI recommendation, partner evidence submission.

API Key Management: View/rotate API keys, usage analytics (requests per endpoint, rate limit utilisation), request rate limit increase.

Sandbox: Test environment with synthetic data, intent analysis, trade initiation simulation, test webhook endpoints.

Agreement: View current revenue share agreement (PDF), propose amendments, track approval status.

Company Admin: Rate limit increase requests, IP whitelisting, notification settings.

END OF PART 12C — DETAILED PORTAL SPECIFICATIONS (v11.0 FULLY EXPANDED)

PART 12D — END-TO-END WORKFLOW EXAMPLES

12D.0 Purpose & Design Philosophy

This part provides complete, realistic trade journeys that demonstrate how different personas use the SGTX portals across all phases (0–8). Each example follows real-life trade practices while preserving the one-click model — every irreversible action is a single click or voice command.

Structure of each example:

Icons used in tables:

Roles Covered in This Part:

PART 12D.1 — EXAMPLE 1: STRAWBERRY EXPORT (EGYPT → GERMANY) — FULL WORKFLOW

12D.1.0 Overview & Learning Objectives

This example follows a complete trade from initiation to settlement, demonstrating every phase of the SGTX platform in a realistic export scenario. It is the most comprehensive example in this part, designed to show how all components work together in a single, end-to-end journey.

Learning Objectives:

Key Themes Throughout This Example:

12D.1.1 Scenario Description

12D.1.1.1 Parties Involved

12D.1.1.2 Trade Details

12D.1.1.3 Special Conditions & Requirements

12D.1.1.4 Optional Services Selected

12D.1.1.5 Add-ons Used

12D.1.2 Phase 0 — Foundation: Identity, Governance & Readiness

12D.1.2.1 Phase 0 Overview

Goal: Onboard all parties, establish GTIDs, complete KYB/KYC, and prepare the platform for trade execution.

Duration: Days 1–3 (before trade initiation)

One-click guarantee: Each onboarding step is a single click. The sandbox guided practice trade is also one click to start.

12D.1.2.2 Step-by-Step Onboarding

Phase 0 one-click count: Buyer ~12 clicks, Seller ~12 clicks, each provider ~10 clicks

12D.1.2.3 Phase 0 — Readiness Assessment (Seller)

Phase 0.2 one-click count: Seller 1 click

12D.1.3 Phase 1 — Trade Initiation (Buyer)

12D.1.3.1 Phase 1 Overview

Goal: Buyer creates a structured, container-level trade request with optional multi-shipment schedule.

Duration: Day 4 (30 minutes)

One-click guarantee: From form submission to trade request creation, max 7 clicks.

AI Assistance:

A1 (Groq) — autocomplete, container advisor, marketplace attribution

A2 (HF local) — Product Form Agent, RIA data retrieval, dual-use screening

A4 (OPA + WasmEdge) — Governor pre-screen, jurisdiction checks

Add-ons Used:

■ GNN Risk Engine — sanctions proximity check on seller and ports

12D.1.3.2 Step-by-Step Trade Initiation

Phase 1 one-click count: 7 clicks (excluding data entry dropdowns)

12D.1.3.3 Phase 1 — Draft Auto-Save (Background)

12D.1.3.4 Phase 1 — Governor Pre-Screen Details

12D.1.4 Phase 2 — Seller Quote, Packing & Logistics Orchestration

12D.1.4.1 Phase 2 Overview

Goal: Seller receives the buyer's structured trade request, locks EXW price, designs packing (including non-uniform layer stacking), determines logistics costs through three flexible modes, adds alternative ports, calculates SGTX fee, and submits a quote.

Duration: Days 5–8

One-click guarantee: From opening the request to submitting the quote, ~12 clicks.

AI Assistance:

A1 (Groq) — live market chart, fair price, loading guide, match score

A2 (HF local) — incoterm filtering, condition assessment

A4 (OPA + WasmEdge) — fee calculation, packing plan lock

Add-ons Used:

- GNN Risk Engine — carrier risk scoring
- Causal Inference — disruption prediction for alternative ports (background)
- ZK Proofs — optional, not used

12D.1.4.2 Step-by-Step Seller Actions

Phase 2 one-click count: ~15 clicks (excluding data entry)

12D.1.4.3 Phase 2 — Quote Breakdown Display

text

■ ■ PRICE BREAKDOWN — USTN SGTX-1397F3A-456ABC-... ■

■ EXW Price (Frozen Strawberries, 20,000 kg): \$28,000.00 ■

■ ■

■ Logistics Costs: ■

■ Trucking (Damietta port): \$900.00 ■

■ Customs Clearance (export): \$600.00 ■

■ THC (Damietta): \$300.00 ■

■ Ocean Freight (2 × 40ft reefer, Damietta → Hamburg): \$8,400.00 ■

■ Insurance: \$200.00 ■

■ ■

■ ■

■ Total Logistics: \$2,000.00 ■

■ ■

■ ■

■ Total Trade Value (before SGTX fee): \$30,000.00 ■

2026-06-20	SGTX-1397F3A-456ABC-...	Lab testing (SGS)	-\$200	—
(Stage 1)				
2026-06-20	SGTX-1397F3A-456ABC-...	Broker cert (Nile)	-\$150	—
(Stage 1)				
2026-06-20	SGTX-1397F3A-456ABC-...	Trucking (Fast)	-\$900	—
(Stage 1)				
2026-07-06	SGTX-1397F3A-456ABC-...	Trade settlement	+\$30,000	—
(principal)				
2026-07-06	SGTX-1397F3A-456ABC-...	Ocean freight (MSC)	-\$8,400	—
(Stage 2)				
		NET CASH FLOW	+\$19,900	

12D.1.9 Phase 7 — Distressed Cargo (Not Used)

No distress occurred. Phase 7 is skipped.

12D.1.10 Phase 8 — Dispute Resolution (Hypothetical)

12D.1.10.1 Phase 8 Overview

Goal: Demonstrate the dispute resolution workflow for a hypothetical quality dispute (mould found on 3 pallets at delivery).

Duration: Days 42–50 (after delivery)

One-click guarantee: Filing a dispute is one click. Evidence compilation is automatic. Mediation and settlement proposals are one click each.

AI Assistance:

A1 (Groq) — mediation log translation, settlement proposal

A2 (HF local) — evidence autocompilation, predictive outcome

A3 (HF + Groq) — third-party expert invitation

Add-ons Used:

Causal Inference — root cause analysis

■ ■

□ □

11

11

11

12D.2.3.1 Phase Overview

A2 (HF local) — evidence compilation

12D.2.3.2 Triage Dashboard

text

■ ■ DISTRESSED CARGO TRIAGE DASHBOARD ■ ■

■ USTN: SGTX-EG-26-F3A-1 ■

■ Lot: 1,344 kg Eureka Lemons | Condition Score: 35 | Shelf Life: 5 days ■

■ ■

■ ■ ■ PATH A — SELL QUICKLY ■ ■ ■

■ ■ Maximise speed of recovery. ■ ■

■ ■ • Price set to AI-recommended quick-sale price: \$1,075 ■ ■

■ ■ • Accelerated Outreach ready (not yet sent) ■ ■

■ ■ • Eligible contacts: 3 ■■

□ □ □ □

■ ■ [Start Sell Quickly] ■ ■

■ ■

■■■■ PATH B — COMPLY WITH LOCAL LAW ■■

■ ■ Ensure compliance with destination country regulations. ■ ■

■ ■ • Jurisdiction: Egypt ■■

■ ■ • Rules: CBE notification required (if >\$1,000) ■ ■

■ ■ • Generated report: CBE notification ready ■ ■

□ □ □ □

■ ■ [Generate Compliance Report] [Download] ■ ■

11

■■■ PATH C — FILE INSURANCE CLAIM ■■■

■ ■ Compile evidence package for insurer. ■■

11

Phase 8.1 one-click count: Seller 2 clicks

12D.2.5.1 Phase Overview

AI Assistance:

text

11

■ Floor Price (hidden): [\$810] (minimum acceptable offer) ■

■ [■] Floor price is never disclosed to recipients. It sets your ■
 ■ minimum acceptable offer. Offers below this will be auto-rejected. ■

```

#####
#####

```

12D.2.5.4 Step-by-Step Outreach

Phase 8.2 one-click count: Seller 1 click

12D.2.6 Phase 8 — Recipient Response & Real-Time Ranking

12D.2.6.1 Phase Overview

Goal: Selected buyers receive notifications, view full details, and submit offers. Seller sees real-time responses ranked by a composite score.

Duration: 6 hours (outreach window)

One-click guarantee: Buyer views details with one click. Submits offer with one click.

AI Assistance:

A2 (LightGBM) — real-time ranking

A1 (Groq) — response summaries

12D.2.6.2 Buyer View (Juice Factory Ltd.)

Phase 8.3 one-click count: Buyer 2 clicks

12D.2.6.3 Buyer View (Cairo Processors)

Phase 8.4 one-click count: Buyer 2 clicks

12D.2.6.4 Seller Dashboard — Real-Time Rankings

text

■ ■ ACCELERATED OUTREACH — Live Dashboard ■

■ Listing: DST-001 | MicroUSTN: SGTX-1397F3A-456ABC-20260420150000-D4E5F6G7 ■

■ Time remaining: 4h 23m ■

■ Floor price: \$810 (hidden) ■

■ Asking price: \$1,075 ■

■ RESPONSES (2 of 2) ■

■ ■ ■ Juice Factory Ltd. — \$1,000 — 2h ago — Score: 94 ■ ■

■ ■ Trust: 91 | Past distressed purchases: 3 (100% completion) ■ ■

■ ■ [Accept] [Counter] ■■

12D.2.8.2 Step-by-Step Microcontract

Phase 8.6 one-click count: Seller 2, Buyer 1 → 3 clicks

12D.2.8.3 Distressed Fee Breakdown

text

■ ■ DISTRESSED FEE CALCULATION ■ ■

■ MicroUSTN: SGTX-1397F3A-456ABC-20260420150000-D4E5F6G7 ■

■ Sale Amount: \$1,000.00 ■

■ ■

■ SGTX Standard Fee Rate: 1.5% ■

■ Country Factor (UAE): 0.85 ■

■ Effective Fee Rate: $1.5\% \times 0.85 = 1.275\%$ ■

■ ■

■ Distressed SGTX Fee: $\$1,000.00 \times 1.275\% = \12.75 ■



■ [Pay Distressed Fee] [Cancel] ■

12D.2.9 Phase 8 — Microcontract Tracking

12D.2.9.1 Phase Overview

Goal: Buyer arranges pickup and confirms receipt. Distressed sale completes.

Duration: Day 2 (after pickup)

One-click guarantee: Confirm receipt is one click.

12D.2.9.2 Step-by-Step Completion

Phase 8.7 one-click count: Buyer 1 click

12D.2.10 Full Timeline

12D.2.11 Summary — One-Click Counts

12D.2.11.1 Per Phase

12D.2.11.2 Per Actor

12D.2.12 Lessons Learned & Best Practices

END OF PART 12D.2 — EXAMPLE 2: DISTRESSED CARGO ACCELERATED OUTREACH

PART 12D.3 — EXAMPLE 3: MULTI-SHIPMENT SCHEDULE MODIFICATION AFTER LOCK

12D.3.0 Overview & Learning Objectives

■ ■ Shipment ■ Delivery Date ■ Port ■ Containers ■ Status ■ Action ■ ■

■ ■ Port: ■ Alexandria ■ Port Said ■■

Containers: 1 1

■ ■ Commodities: ■ Oranges, 20,000 kg ■ Oranges, 20,000 kg ■■

■ ■

Reason: "Warehouse capacity issue — need additional storage time. ■

■ New distribution centre in Port Said.■

■ ■

■ Impact Assessment: ■

■ • Shipment 1 (15 Jul, Alexandria) — UNCHANGED (already LOCKED) ■

■ • Shipment 2 (30 Aug, Port Said) — MODIFIED ■

■ • Shipment 3 (15 Sep, Alexandria) — UNCHANGED ■

■ • Total contract value — UNCHANGED ■

■ ■

■ ■ ■ Port Said is valid and not sanctioned. Date is in the future. ■

■ ■

■ [Accept All] [Reject] [Counter] [Request More Info] ■

12D.3.2.6 Schedule Addendum (Generated)

text

SCHEDULE ADDENDUM — MC-20260415-001

THIS ADDENDUM amends the multi-shipment schedule of Contract MC-20260415-001

dated [original contract date], between:

Buyer: Nile Foods (SGTX-EG-TRD-001)

Seller: Mekong Fresh (SGTX-VN-TRD-002)

The following modification is hereby agreed:

SHIPMENT 2:

Field	Original	Amended
-------	----------	---------

| Delivery Date | 15 Aug 2026 | 30 Aug 2026 |

| Port | Alexandria | Port Said |

| Containers | 1 | 1 |

REASON: "Warehouse capacity issue — need additional storage time. New distribution centre in Port Said."

This addendum is incorporated into and forms part of the original contract.

SGTX: [Digital Signature] Date: 2026-06-15

text

■■■ (delivered) ■■■■ ...-V1■■■

■ Escalation: ■

■ • 7 days before expiry: Reminder sent ■ ■

■ • 1 day before expiry: Alert to be sent ■

■ • At expiry: Auto-charge (if authorised) or block ■

22

■ [Pay Now] [Extend Guarantee] [View Details] ■

A horizontal row of 16 empty square boxes, each intended for a single digit or symbol, used for writing the answer to the problem.

12D.6 Example 6 — Financing with Co-Financing and DeFi

12D.6.1 Scenario Description

Parties:

Financing Request: \$100,000 pre-shipment financing for citrus

12D.6.2 Financing Workflow

One-click count: Seller 3, Bank X 2, DeFi Pool Y 3 → ~8 clicks total

12D.6.3 DeFi Risk Summary Modal

text

DEFI PLAINLANGUAGE RISK SUMMARY

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

■ You have selected DeFi settlement via Aave V3. Please read carefully: ■

■ ■

■ 1. What stablecoins are: You will be lending in USDC, a digital dollar ■

■ that aims to keep a 1:1 value with the US dollar. ■

22

■ 2. What 'health factor' means: Your loan's health factor measures its ■

■ safety. If it drops below 1, collateral could be sold to repay the ■

■ loan, and you might lose part of your capital. ■

111

■ 3. What happens if collateral drops in value: If the value of the goods ■

■ securing the loan falls sharply, the borrower may need to add more ■

■ collateral, or the loan could be partially liquidated. ■

■ ■

■ 4. SGTX does NOT guarantee DeFi safety: SGTX checks that the protocol

■ has passed audits, but we cannot guarantee it will never be hacked. ■

22



text

■ ■ Green 0-70% ■ ■ ■ 80% ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■

■ ■ Red 85-100% ■ ■ ■ 60% ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■

■ Loan: \$60,000 ■ ● Collateral added 20 Jun ■

■ ■ Low 0-20% | Medium 20-50% | High 50-100% ■ ■

■ ■ week), elevating liquidation risk to 68%. Consider a proactive margin ■■

■ ■ call." ■ ■

■ MARGIN CALL LOG ■

■ ■ Date ■ LTV ■ Amount Required ■ Deadline ■ Status ■ Response ■ ■

■ ■ 22/6 ■ 82% ■ \$10,000 ■ 24/6 ■ PENDING ■ Pending ■ ■

■ SIMULATE PRICE DROP ■

■ ■ Collateral value decline: [■■■■■■■■■■] 10% ■■

■ ■ Additional collateral required to restore 70%: \$6,400 ■ ■

12D.7 Example 7 — Dispute with Third-Party Expert

Parties:

Third-Party Expert: SGS Surveyor (invited by buyer)

One-click count: Buyer 4, Seller 1, Expert 1 → 6 clicks total

text

■ ■ MEDIATION LOG — DSP-20260415-001 ■ ■

■ USTN: SGTX-EG-26-F3A-1 ■

■ Buyer: Nile Foods (English) ■ Seller: Mekong Fresh (Vietnamese) ■

■■ 2026-04-16 10:00:00 — Buyer ■■

■ ■ based on photos and temperature log. ■■

■ ■ 2026-04-16 11:30:00 — Seller ■ ■

■ ■ damage likely occurred after delivery. ■■

■ ■ [Translate to English] [Reply] ■■

■ ■ 2026-04-16 12:00:00 — Expert (SGS) ■ ■

■ ■ 6 hours. This is the likely cause of the mould. ■■

■ ■ ■ AI Settlement Proposal (Groq) ■ ■ ■

■ ■ Confidence: 87% ■ ■

1. **Sanctions & Adversarial Risk Detection** — Detects indirect sanctions links, hidden money-laundering patterns, and adversarial relationships (e.g., a seller's Ultimate Beneficial Owner within 2 hops of a sanctioned entity).

2. Institutional Trade Graph (Trust-Based Mapping) — Maps positive, trust-based relationships (trade frequency, financing relationships, co-compliance, supply chain dependencies) to strengthen credit assessments, financing decisions, and network trust scores.

All outputs are advisory or constraining (A2/A4) — they never recommend counterparties or autonomous actions.

12E.1.2 Integration Map

12E.1.3 Detailed Integration Points

12E.1.3.1 Governor Pre-Screen Integration

Called synchronously for every `trade.initiate` and `financing.request`:

rust

// Pseudo-code for Governor integration

```
let gnn_result = gnn_risk_service.check(
```

```
  buyer_gtid,
```

```
  seller_gtid,
```

```
  logistics_provider_gtids,
```

```
  financier_gtid
```

```
);
```

```
if gnn_result.sanctions_proximity <= 2 || gnn_result.graph_risk_score >= 90 {
```

```
  return GovernorVerdict::Conditional {
```

```
    conditions: vec![
```

```
      "enhanced_due_diligence_required",
```

```
      "ubo_proximity_review"
```

```
    ],
```

```
    tenant_message: format!(
```

```
      "This trade involves a party whose UBO is within {} hops of a sanctioned entity. \
```

```
Enhanced due diligence required.",
```

```
      gnn_result.sanctions_proximity
```

```
    ),
```

```
  };
```

```
}
```

12E.1.3.2 Trust Graph Query (Financier Portal)

Called on demand when financier views portfolio:

json

```
GET /v1/gnn/trust?gtid=SGTX-EG-FIN-001
```

```
{
```

```
  "direct_connections": 47,
```

```
"trusted_connections": 12,  
"network_trust_score": 82,  
"indirect_exposure": "2 hops to 3 financial institutions",  
"privacy_notice": "All counts are anonymised; no counterparty identities revealed."  
}
```

12E.1.3.3 Data Model References

12E.1.4 One-Click Integration

■ Non-marketplace safeguard: The GNN never suggests alternative counterparties. It only flags risk with the existing relationship.

12E.2 Add-on 2 — Federated Learning Network

12E.2.1 Purpose

Train machine learning models across multiple sovereign nodes without exchanging raw trade data. Each node trains a local model on its own data (e.g., fraud detection, margin estimation, credit scoring), sends only encrypted model updates (gradients) to a secure aggregator, and receives a global model. This improves model accuracy for corridors with limited local data while preserving data sovereignty.

12E.2.2 Integration Map

12E.2.3 Detailed Integration Points

12E.2.3.1 Training Pipeline

rust

// Weekly training orchestration

```
let coordinator = FedLearningCoordinator::new();  
coordinator.start_round(  
  model_name: "credit_scoring",  
  participants: vec!["cairo-node", "dubai-node", "frankfurt-node"],  
  aggregation_algorithm: "FedAvg",  
  differential_privacy_epsilon: 0.5,  
);
```

// Each participant trains locally

// Coordinator aggregates and distributes new model

12E.2.3.2 Data Model References

12E.2.4 One-Click Integration

■ Non-marketplace safeguard: Federated learning only shares encrypted model updates, never raw trade data. No counterparty recommendations.

12E.3 Add-on 3 — Causal Inference Engine

12E.3.1 Purpose

Identify root causes of disputes, delays, defaults, or quality failures — not just correlations. For example, "68% of the delay was due to a port strike, 22% due to carrier rerouting, 10% due to customs documentation." This provides actionable insights for insurers, arbitrators, and traders — but never

suggests "blame the other party" or "switch provider". It is purely factual.

12E.3.2 Integration Map

12E.3.3 Detailed Integration Points

12E.3.3.1 Dispute Filing Integration

Triggered asynchronously when a dispute is filed:

json

```
// Causal inference output stored in `causal_attribution`
```

```
{
```

```
"entity_id": "DSP-20260415-001",
```

```
"entity_type": "DISPUTE",
```

```
"root_causes": [
```

```
{
```

```
"factor": "temperature_excursion",
```

```
"contribution": 0.68,
```

```
"description": "Reefer set point drifted to -8°C for 6 hours due to compressor failure.",
```

```
"confidence_interval": { "lower": 0.60, "upper": 0.75 }
```

```
},
```

```
{
```

```
"factor": "pre_cooling_omission",
```

```
"contribution": 0.12,
```

```
"description": "Pre-cooling omitted before loading, contributing 12%.",
```

```
"confidence_interval": { "lower": 0.08, "upper": 0.16 }
```

```
}
```

```
],
```

```
"groq_summary": "The cold chain failure was caused primarily by compressor failure (68%), with pre-cooling omission (12%) as a contributing factor."
```

```
}
```

12E.3.3.2 Data Model References

12E.3.4 One-Click Integration

■ Non-marketplace safeguard: Causal analysis never suggests "switch provider" or "blame the other party". Only factual contribution percentages.

12E.4 Add-on 4 — Self-Healing Infrastructure & Chaos Engineering

12E.4.1 Purpose

Ensure platform resilience by automatically healing failed pods, predicting hardware failures, and regularly proving system robustness through chaos experiments. The platform can survive node failures, network partitions, and disk outages without manual intervention — all while maintaining non-custodial properties.

12E.4.2 Integration Map

12E.4.3 Detailed Integration Points

12E.4.3.1 Self-Healing Pipeline

```
rust
// Node health monitoring
let node_health = node_health_agent.monitor(
node_id: "cairo-prod-01",
disk_smart_data: read_smart_data(),
cpu_temp: read_cpu_temp(),
memory_errors: read_memory_errors(),
);
if node_health.predicted_failure_probability > 0.70 {
node_health_agent.cordon_node(node_id);
node_health_agent.drain_workloads(node_id);
// Creates Smart Inbox alert for Admin
}
```

12E.4.3.2 Chaos Experiment Types

12E.4.3.3 Data Model References

12E.4.4 One-Click Integration

■ Non-marketplace safeguard: Self-healing is purely operational — never affects trade data or recommendations.

12E.5 Add-on 5 — Automated Penetration Testing

12E.5.1 Purpose

Run weekly vulnerability scans against the SGTX staging environment (and optionally production with read-only scans). Findings are automatically categorised, prioritised, and, if critical, block the CI/CD pipeline. This ensures that no vulnerability reaches production. The scans never target tenant data; they focus on platform APIs and infrastructure.

12E.5.2 Integration Map

12E.5.3 Detailed Integration Points

12E.5.3.1 Scanning Pipeline

```
rust
// Weekly pentest orchestration
let scan_results = pentest_service.run_scan(
target_url: "https://staging.sgtx.io",
tools: vec!["trivy", "zap", "nuclei", "openvas"],
schedule: "Friday 23:00 UTC",
);
if scan_results.has_critical_findings() {
```

```

ci_pipeline.block_deployment();
notify_oncall("Critical vulnerability found — blocking deployment");
}

```

12E.5.3.2 Data Model References

12E.5.4 One-Click Integration

■ Non-marketplace safeguard: Pentesting targets infrastructure, not tenant data.

12E.6 Add-on 6 — Post-Quantum Cryptography (PQC)

12E.6.1 Purpose

Protect long-term records (contracts, Loom hash chains, identity credentials, audit logs) against future quantum computer attacks. While Ed25519 (used for daily signatures) is secure today, it is not quantum-safe. The add-on adds a second layer of Dilithium3 signatures (NIST standard) and optionally Kyber1024 for key exchange.

12E.6.2 Integration Map

12E.6.3 Detailed Integration Points

12E.6.3.1 Two-Layer Signature

```

rust
// Every new contract receives both signatures
let ed25519_sig = sign_ed25519(contract_json);
let dilithium3_sig = sign_dilithium3(contract_json);
store_contract(
  contract_json,
  ed25519_sig,
  dilithium3_sig,
);

```

12E.6.3.2 Background Re-signing

Weekly cron job re-signs any record older than 1 year that lacks a PQC signature:

```

rust
pqc_resigner.resign_old_records(
  older_than: Duration::days(365),
);

```

12E.6.3.3 Data Model References

12E.6.4 One-Click Integration

■ Non-marketplace safeguard: PQC is purely cryptographic — never affects recommendations.

12E.7 Add-on 7 — Expanded Zero-Knowledge Proofs (ZK) & Proof of Reserves

12E.7.1 Purpose

Zero-Knowledge Proofs (ZK) enable privacy-preserving verification without revealing underlying sensitive data. SGTx implements four core use cases, plus a phased proof-of-reserves mechanism:

1. Financier proof-of-reserves — A financier proves they hold sufficient liquidity to back a financing offer without revealing wallet balance.
2. Confidential trade terms — Buyer and seller agree on a price hidden from the platform; SGTX verifies only that the price is within legal and market bounds.
3. Private settlement — A buyer proves they settled a payment without revealing the exact amount or PSP used.
4. Proof of Reserves (phased) — Initially via Big Four attestation, later via real-time ZK proofs, ensuring platform solvency and backing of obligations.

12E.7.2 Integration Map

12E.7.3 Detailed Integration Points

12E.7.3.1 Financier Reserve Proof

json

// Bid submission with optional reserve proof

POST /v1/finance/bid

```
{
  "financing_request_id": "FR-001",
  "amount_offered": 100000,
  "apr": 4.8,
  "reserve_proof": "base64_zk_proof_here" // optional
}
```

// Governor verifies proof

```
if bid.reserve_proof.is_some() {
  let verified = zk_verifier.verify_reserve_proof(
    proof: bid.reserve_proof,
    financier_gtid: financier.gtid,
    amount: bid.amount_offered,
  );
  if !verified { return GovernorVerdict::Deny; }
}
```

12E.7.3.2 Confidential Trade Terms

json

// Seller enables confidential pricing

POST /v1/quote/submit

```
{
  "ustn": "SGTX-...",
  "exw_price": 28000,
  "logistics_cost": 2000,
```

```
"sgtx_fee": 450,  
"confidential": true,  
"price_zk_proof": "base64_zk_proof_here" // proof that price is within bounds  
}
```

// Buyer sees only total price

// Raw components not stored

12E.7.3.3 Private Settlement Proof

json

// Buyer requests private settlement proof

GET /v1/settlement/private-proof?ustn=SGTX-...

// Returns ZK proof

```
{  
  "proof": "base64_zk_proof_here",  
  "verification_url": "https://sgtx.io/verify/settlement/..."  
}
```

// External auditor verifies without seeing amount

12E.7.3.4 Proof of Reserves (Phased)

Phase 1 — Big Four Attestation (Months 1–12):

Phase 2 — Real-time ZK Proofs (Mature):

json

GET /v1/reserves/proof

```
{  
  "ratio": 115.2,  
  "proof": "base64_zk_proof_here",  
  "verified": true,  
  "verified_at": "2026-06-15T00:00:00Z"  
}
```

12E.7.3.5 Data Model References

12E.7.4 One-Click Integration

■ Non-marketplace safeguard: ZK proofs never reveal sensitive data — only what is necessary for verification.

12E.8 Cross-Portal Visibility Map (User-Facing)

12E.9 Add-on Activation Summary Table

12E.10 Implementation Considerations

12E.10.1 Performance Impact

12E.10.2 Storage Impact

12E.10.3 Dependency Matrix

12E.11 Integration Testing Checklist

12E.11.1 GNN

Test sanctions proximity $\leq 2 \rightarrow$ Governor returns CONDITIONAL

Test trust graph query with ZK proof $\rightarrow$ anonymised result

Test opt-out toggle $\rightarrow$ tenant excluded from trust graph

12E.11.2 Federated Learning

Simulate training round across nodes

Verify model update distribution

Test drift monitoring $\rightarrow$ model accuracy drops $\rightarrow$ alert

12E.11.3 Causal Inference

Test dispute filing $\rightarrow$ causal analysis generated

Verify root cause contribution percentages

Test Groq summary generation

12E.11.4 Self-Healing

Simulate pod failure $\rightarrow$ auto-restart

Test disk failure prediction $\rightarrow$ cordon node

Run chaos experiment $\rightarrow$ verify resilience

12E.11.5 Pentesting

Run weekly scan $\rightarrow$ findings stored

Test critical finding $\rightarrow$ CI/CD block

Verify Smart Inbox alert on critical finding

12E.11.6 PQC

Test dual-signature generation

Verify background re-signing

Test PQC verification endpoint

12E.11.7 ZK Proofs

Financier reserve proof $\rightarrow$ verification passes $\rightarrow$ bid accepted

Seller confidential pricing $\rightarrow$ proof generated $\rightarrow$ buyer sees only total

Buyer private settlement proof $\rightarrow$ generated $\rightarrow$ external verifier accepts

Platform reserves $< 110\% \rightarrow$ Governor blocks new trades

END OF PART 12E — ADD-ON INTEGRATION MAP (v11.0 FULLY EXPANDED)

PART 12F — QUICK START DECISION TREE & TAB INDEX

12F.0 Purpose & Design Philosophy

This part provides two quick-reference tools:

■ • Customer Care Hub — chatbot dashboard, support queue ■

■ • Configuration History — versioned manifest, rollback ■

■ • Internal Admin — multisig members, internal roles ■



Dual-mode note: If you are a trading company with DUAL mode, use the Buyer/Seller toggle in the global header to switch between Buyer and Seller dashboards. You do not need to log out or use separate portals.

12F.2 Alphabetical Index of Portal Tabs

This index lists every tab that appears across all SGTx portals, with the portal(s) where it appears and the section where it is documented.

12F.3 Role-Based Quick Reference Cards

12F.3.1 Buyer (Trader Portal, mode = BUY)

Quick navigation path:

text

Smart Inbox (landing)

- ➡■ New Trade Request → Submit Trade Request
- ➡■ Quote Review & Negotiation → Accept / Negotiate
- ➡■ Contract Signing → Sign
- ➡■ Customs Readiness → Upload / Request
- ➡■ Shipments → Click USTN → TCC

12F.3.2 Seller (Trader Portal, mode = SELL)

Quick navigation path:

text

Smart Inbox (landing)

- ➡■ Pending Requests → Accept
- ➡■ EXW Price Lock → Lock Price
- ➡■ Containerisation & Packing → Lock Packing Plan
- ➡■ Logistics Builder → Send RFQ / Select Quote
- ➡■ Quote Submission → Submit Quote
- ➡■ Barcode Print → Print

12F.3.3 Logistics Provider (LSP)

Quick navigation path:

text

Smart Inbox (landing)

- ➡■ RFQ Inbox → Send Quote / Ask Clarification
- ➡■ Dispatch Planner → Optimise → Assign → Send
- ➡■ Shipments → Click USTN → TCC → Confirm Milestone

➡■ Performance → View summary

12F.3.4 Shipping Line (SHIP)

Quick navigation path:

text

Smart Inbox (landing)

➡■ Booking Requests → Confirm Booking

➡■ eBL Management → Issue eBL

➡■ Vessel Schedule → Edit → Save

➡■ Freight Invoices → Generate

12F.3.5 Laboratory (LAB)

Quick navigation path:

text

Smart Inbox (landing)

➡■ Testing Jobs → Confirm Receipt → Submit Results

➡■ Result Submission → Submit Results

➡■ Certificates → Download

12F.3.6 QC Inspector

Quick navigation path:

text

Smart Inbox (landing)

➡■ Inspection Jobs → Accept → Generate QR

➡■ Mobile App (in field) → Scan → Override (if needed) → Submit

➡■ Report Submission → Select Verdict → Submit

➡■ Re-inspection Requests → Accept / Decline

12F.3.7 Customs Broker (CBR)

Quick navigation path:

text

Smart Inbox (landing)

➡■ Certification Requests → Certify

➡■ Physical Document Jobs → Receive → Mark Presented → Upload → Confirm

➡■ Storage → Notify Client / Extend / Destroy

➡■ Audit Representation → Accept

12F.3.8 Financier (Bank / PFI)

Quick navigation path:

text

Smart Inbox (landing)

- ➡■ Financing Opportunities → View Details → Submit Bid
- ➡■ My Bids & Active Loans → View Collateral Dashboard → Issue Margin Call
- ➡■ Portfolio & Compliance → Generate Report → Download
- ➡■ Financed Companies → Click Borrower → Refresh Credit Assessment

12F.3.9 Government Official

Quick navigation path:

text

Smart Inbox (landing)

- ➡■ Live Trade Monitor → Click USTN → TCC
- ➡■ Clearance Workflow → Approve Clearance / Hold / Reject
- ➡■ Document Verification → Accept / Reject
- ➡■ Multi-Agency Workflow → Approve / Reject
- ➡■ Compliance Monitor → Generate Report

12F.3.10 Platform Administrator (Admin Portal)

Quick navigation path:

text

Smart Inbox (landing)

- ➡■ Constitutional Policies → Edit → Simulate Impact → Submit for Approval
- ➡■ Platform Health → View metrics → Schedule Maintenance
- ➡■ Special Rate Manager → Propose Special Rate → Submit for Approval
- ➡■ Tenant Management → Search → Impersonate
- ➡■ Configuration History → View Diff → Rollback

12F.3.11 Marketplace Partner

Quick navigation path:

text

Dashboard (landing)

- ➡■ Leads Management → View Details → Resolve
- ➡■ Webhook Management → Register → Test
- ➡■ Sandbox → Analyse → Simulate Trade → Test Webhook
- ➡■ Revenue Attribution Disputes → Submit Evidence

12F.4 Tab-by-Tab Navigation Map (Common Tasks)

12F.4.1 Creating a New Trade (Buyer)

text

1. Smart Inbox (landing)

↓

2. Click "New Trade Request" (sidebar or Recommended Actions Widget)

↓

3. Select seller (GTID entry or saved contacts)

↓

4. Select incoterm (dropdown)

↓

5. Configure containers (add/configure container tabs)

↓

6. Add commodities per container (commodity type → product → dynamic specs)

↓

7. Enter packing details (pallet type, cartons, weight, layers)

↓

8. (Optional) Enable multi-shipment → add shipments

↓

9. Click "Submit Trade Request" (one click)

↓

■ Trade request created, seller notified

12F.4.2 Responding to a Trade Request (Seller)

text

1. Smart Inbox (landing)

↓

2. Click "Pending Requests" (sidebar)

↓

3. Click the trade request (opens unified view)

↓

4. Review buyer's parsed specs (containers, commodities, incoterm)

↓

5. Click "Accept" (or Decline / Propose Modifications)

↓

6. Set loading origin (country, port)

↓

7. Lock EXW price (use fair price or manual entry)

↓

8. Go to Containerisation & Packing → design packing → lock plan

↓

9. Go to Logistics Builder → select quotes (Mode A, B, or C)

↓

10. Go to Quote Submission → review breakdown → click "Submit Quote"

↓

■ Quote sent to buyer

12F.4.3 Managing a Multi-Shipment Contract (Buyer/Seller)

text

1. From TCC (click USTN from Shipments Vault)

↓

2. Locate Multi-shipment Schedule Card

↓

3. View all shipments (status, dates, ports)

↓

4. For future shipments:

↓

a. Click "Request Modification" (if changes needed)

↓

b. Select shipment → edit delivery date, port, containers, commodities

↓

c. Add reason → click "Send Modification Request"

↓

d. Counterparty reviews diff view → accepts/rejects/counters

↓

5. For ready shipments:

↓

a. Seller clicks "Ready for Shipment X"

↓

b. Buyer clicks "Confirm Shipment X"

↓

c. Seller pays per-shipment fee

↓

d. New USTN generated, shipment locks

12F.4.4 Filing a Dispute (Buyer/Seller)

text

1. From TCC (click USTN from Shipments Vault)
↓
2. Click "Raise Dispute" (in TCC header or Disputes tab)
↓
3. Select category (Quality, Delay, Non-Payment, etc.)
↓
4. Describe breach (min 10 chars)
↓
5. (Optional) Upload additional evidence
↓
6. Specify remedy sought
↓
7. Click "Submit Dispute" (one click)
↓
8. Evidence package auto-compiles (Loom-hashed)
↓
9. Both parties enter Mediation Log
↓
10. Structured offers, accept/reject/counter
↓
11. (Optional) Invite third-party expert
↓
12. (Optional) Accept AI settlement proposal (one click each)
↓
13. Or escalate to arbitration (one click to prepare case)
↓

■ Dispute resolved or arbitration prepared

12F.4.5 Requesting Financing (Seller/Buyer)

text

1. From Financing tab (sidebar) or TCC
↓
2. Click "Request Financing"
↓
3. Pre-filled form: USTN, amount (default 70% LTV), tenor, financing type, settlement method
↓

4. Click "Submit Request" (one click)

↓

5. Credit intelligence runs (A2) → RFQ broadcast automatically

↓

6. Financiers receive RFQ, view full disclosure, submit bids (encrypted)

↓

7. After bidding window closes, view bids in comparison table

↓

8. Select bids (co-financing possible) → click "Accept Selected"

↓

9. Sign financing agreement (ZITADEL passkey)

↓

10. Financiers click "Disburse" → PSP split (0.25% fee deducted)

↓

11. Borrower receives funds, repayment monitored automatically

↓

■ Financing completed

12F.4.6 Declaring Distressed Cargo (Seller)

text

1. From Distressed Cargo & Notifications tab (sidebar)

↓

2. Click "Declare Distressed"

↓

3. Select affected pallets/containers

↓

4. Describe condition, upload photos

↓

5. AI condition assessment runs (A2) → condition score

↓

6. Dynamic pricing (XGBoost) → suggested price range

↓

7. Accept or override price

↓

8. Triage Dashboard: Choose "Sell Quickly", "Comply with Local Law", or "File Insurance Claim"

↓

9. If "Sell Quickly": click "Check Buyers" → select eligible saved contacts

↓

10. Click "Start Accelerated Outreach" → privacy notice acknowledged

↓

11. Notifications sent to selected buyers (time-boxed, floor price)

↓

12. Buyers submit offers; seller accepts (one click)

↓

13. Microcontract generated; seller pays distressed fee (one click)

↓

14. Microcontract locked; microUSTN generated

↓

■ Distressed cargo resolved

12F.5 Portal Switching Guide

12F.5.1 Dual-Mode Toggle (Trader Portal Only)

Visibility conditions:

Tenant type = TRD

default_trader_mode = DUAL

Employee allow_role_switching = true

Employee's role includes both BUY and SELL

One-click: Click toggle → portal context switches instantly.

12F.5.2 Portal Access Points

12F.5.3 Mobile App Access

12F.6 Keyboard Shortcuts

12F.7 Implementation Checklist (Part 12F)

12F.7.1 Decision Tree

Build decision tree component (React, interactive)

Add role selection buttons

Display relevant portal and tabs for selected role

Include dual-mode toggle explanation

12F.7.2 Tab Index

Maintain alphabetical list of all tabs

Cross-reference to portal and section

Include one-click actions per tab

Add search functionality for tab index

12F.7.3 Role-Based Quick Reference Cards

Build card for each role

Include "If you need to..." mapping

Include quick navigation path

Add visual icons

12F.7.4 Tab-by-Tab Navigation Map

Document common task navigation paths

Include step-by-step guides

Highlight one-click actions

Add completion markers (■)

12F.7.5 Portal Switching Guide

Document dual-mode toggle

List portal access points and URLs

Document mobile app availability

Add keyboard shortcuts

12F.7.6 Testing

Test decision tree for all roles

Verify all tab references resolve correctly

Test keyboard shortcuts

Test portal switching

END OF PART 12F — QUICK START DECISION TREE & TAB INDEX (v11.0 FULLY EXPANDED)

PART 12G — UNIVERSAL COMMAND CENTER

12G.0 Purpose & Design Philosophy

The Universal Command Center serves as the primary landing page for all authenticated users before entering role-specific portals. It provides a unified dashboard for trade visibility, pending actions, compliance alerts, financial status, workflow shortcuts, and AI assistant access — all in one place.

Access: All authenticated users regardless of tenant type or role.

Dual-mode toggle: ■ visible (if DUAL, shows combined view of both modes)

Landing page: Default after login (user can set preference to bypass to role-specific portal)

Core principles:

One-click core — every action from the Command Center executes with a single click

Role-adaptive — content adapts to the user's role and permissions

Real-time — all cards and metrics update via NATS WebSocket

AI-assisted — AI Operations Assistant provides context-aware guidance

Non-marketplace — never suggests counterparties or unsolicited actions

Accessible — WCAG 2.2 Level AA compliant

AI Integration:

12G.1 Executive Summary Cards (Top Row)

12G.1.1 Overview

Four to six key metrics cards, configurable per role. Each card displays a key metric, a trend indicator, and a count. Clicking any card navigates to the relevant filtered view in the appropriate portal.

12G.1.2 Standard Cards (All Roles)

12G.1.3 Role-Specific Cards

12G.1.4 Card Interaction

One-click: Click any card → navigates to the relevant tab.

12G.2 Quick Actions Grid

12G.2.1 Overview

Frequently used actions, displayed as icon + label buttons (max 8, configurable). The grid shows actions based on the user's role and active dual-mode context (Buyer or Seller).

12G.2.2 Standard Quick Actions

12G.2.3 Role-Specific Quick Actions

12G.2.4 Quick Actions Configuration

Users can customise the grid via the Settings panel:

Accessibility: Keyboard shortcut Ctrl/Cmd + K to focus first action.

12G.3 AI Operations Assistant

12G.3.1 Overview

A persistent chat panel (collapsible) at the bottom-right corner of the Command Center. The AI Assistant provides natural language query processing, context-aware suggestions, and one-click action execution.

12G.3.2 Capabilities (A1, Groq → Ollama)

12G.3.3 Example Queries & Responses

12G.3.4 Chat Interface

text

■ ■ AI OPERATIONS ASSISTANT [x] ■ ■

■ ■ ■ Good morning, Ahmed! You have 3 pending actions today: ■■

■ ■ • Sign contract for USTN-EG-001 (deadline: 14:00 UTC) ■ ■

■ ■ • Upload phytosanitary certificate for USTN-EG-003 ■ ■

■ ■ • Review QC report for USTN-EG-005 ■ ■

12G.4.2 Display Format

text

Mohamed Eltonsy • Approved Invoice • USTN-EG-2026-00491 • 2 minutes ago

Sarah Ahmed • Submitted Declaration • USTN-EG-2026-00490 • 1 hour ago

System • Document Missing • USTN-EG-2026-00489 • 3 hours ago

→ Certificate of Origin required. [Upload Now]

System • Sanctions Alert • USTN-EG-2026-00488 • 5 hours ago

→ GNN detected sanctions proximity ≤2 hops. [Review]

12G.4.3 Colour Coding

12G.4.4 Activity Types

12G.4.5 Interaction

One-click: Click any activity item → opens relevant TCC or document.

12G.5 Configuration & Personalisation

12G.5.1 Settings Panel

Users can customise the Command Center via the Settings button (top-right of Command Center).

12G.5.2 Personalisation Persistence

One-click: Click → adjust settings → auto-save.

12G.6 Integration with Other Parts

Non-marketplace: The Command Center never suggests alternative counterparties or service providers.

12G.7 Trade Health Score Engine

12G.7.1 Purpose

Provide a dynamic, at-a-glance health score for each trade (USTN) based on compliance, documentation, logistics, payment, risk, and timeline status.

12G.7.2 Score Formula

The Trade Health Score is a 0–100 composite calculated as:

text

Trade Health Score =

$(\text{Compliance} \times 0.20) +$
 $(\text{Documentation} \times 0.20) +$
 $(\text{Logistics} \times 0.15) +$
 $(\text{Payment} \times 0.15) +$
 $(\text{Risk} \times 0.20) +$
 $(\text{Timeline} \times 0.10)$

Where each component is a 0–100 sub-score.

12G.7.3 Component Definitions

12G.7.4 Health Status Bands

12G.7.5 Display Locations

12G.7.6 One-Click Drill-Down

Clicking the health score in the TCC opens a Health Breakdown Panel showing:

12G.7.7 AI-Generated Health Summary (A1, Groq)

A plain-language summary appears next to the score:

12G.7.8 Database Storage

The Trade Health Score is calculated on-the-fly (not stored) but can be materialised for analytics:

sql

```
CREATE MATERIALIZED VIEW trade_health_scores AS
SELECT
  ustn,
  (compliance_score * 0.20 + documentation_score * 0.20 +
   logistics_score * 0.15 + payment_score * 0.15 +
   risk_score * 0.20 + timeline_score * 0.10) AS health_score,
  NOW() AS calculated_at
FROM ...;
```

Refresh frequency: Every 15 minutes (or on any state change via NATS).

12G.8 External Integrations Health Widget

12G.8.1 Purpose

Provide real-time visibility into the health of external APIs (Nafeza, CargoX, ETA, PSPs) that SGTX depends on.

Location: Admin Portal → Platform Health tab, and Government Portal → Integrations tab.

12G.8.2 Widget Content

12G.8.3 Status Definitions

12G.8.4 Refresh & Alerts

12G.8.5 One-Click Actions

Data source: integration_connector_logs (Part 13) aggregated every 10 seconds by the integration-health-monitor service.

12G.9 Mobile Adaptation

12G.9.1 Mobile View

On screens narrower than 768px, the Command Center adapts:

12G.9.2 Mobile-Specific Features

12G.10 Database Schema Additions

sql

-- User preferences for Command Center

```
CREATE TABLE user_preferences (  
  employee_id UUID PRIMARY KEY REFERENCES employees(id) ON DELETE CASCADE,  
  dashboard_config JSONB DEFAULT '{}', -- card visibility, order, thresholds  
  quick_actions JSONB DEFAULT '{}', -- pinned actions, order  
  theme TEXT DEFAULT 'system', -- 'light', 'dark', 'system'  
  ai_assistant_enabled BOOLEAN DEFAULT true,  
  language TEXT DEFAULT 'en',  
  compact_mode BOOLEAN DEFAULT false,  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Integration health logs

```
CREATE TABLE integration_health_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  integration_name TEXT NOT NULL, -- 'NAFEZA', 'CARGOX', 'ETA', 'FAWRY', etc.  
  status TEXT NOT NULL, -- 'OPERATIONAL', 'DEGRADED', 'OUTAGE'  
  latency_ms INTEGER,  
  error_rate DECIMAL(5,2),  
  checked_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Trade health scores (materialised view)

```
CREATE MATERIALIZED VIEW trade_health_scores AS  
SELECT  
  ustn,  
  (compliance_score * 0.20 + documentation_score * 0.20 +  
   logistics_score * 0.15 + payment_score * 0.15 +  
   risk_score * 0.20 + timeline_score * 0.10) AS health_score,  
  NOW() AS calculated_at
```

FROM ...;

12G.11 Implementation Checklist

12G.11.1 Executive Summary Cards

Build card component with metrics and trend indicators

Add role-based card visibility

Implement card click navigation

Add card customisation (show/hide, reorder, thresholds)

12G.11.2 Quick Actions Grid

Build grid component with icon + label buttons

Add role-specific actions

Implement customisation (pin/unpin, reorder, max actions)

Add keyboard shortcuts

12G.11.3 AI Operations Assistant

Build chat panel component

Integrate Groq API for natural language responses

Add Ollama fallback

Implement one-click action execution from chat

Add voice input support

Implement privacy and data handling

12G.11.4 Recent Activity Feed

Build activity feed component

Subscribe to NATS events for real-time updates

Add colour coding and icons

Implement click navigation

Add filter and date range controls

12G.11.5 Configuration & Personalisation

Build settings panel

Persist preferences in user_preferences table

Sync across devices via NATS

12G.11.6 Trade Health Score

Build health score calculation engine

Implement component sub-scores

Add health status bands (Green/Amber/Red)

Build Health Breakdown Panel

Add AI-generated health summary (A1, Groq)

Materialise view for analytics

12G.11.7 External Integrations Health Widget

Build widget component

Integrate with integration_connector_logs

Add real-time updates (every 10 seconds)

Implement status definitions (Operational/Degraded/Outage)

Add alerting on status change

12G.11.8 Mobile Adaptation

Responsive design for <768px

Swipe gestures

Pull-to-refresh

Offline caching

12G.11.9 Testing

Test all cards and navigation

Test AI Assistant queries and actions

Test activity feed real-time updates

Test Trade Health Score calculation

Test Integration Health Widget

Test mobile adaptation

Test customisation persistence

12G.12 AI Authority Summary for Part 12G

END OF PART 12G — UNIVERSAL COMMAND CENTER (v11.0 FULLY EXPANDED)

PART 13 — COMPLETE DATA MODEL & API ENDPOINT INDEX

13.0 Purpose & Design Philosophy

This part consolidates the complete database schema (PostgreSQL 18 with pgvector and TimescaleDB) and the full API endpoint index (OpenAPI 3.1) for the SGTX platform. Every table, column, index, and relationship defined in Parts 0–12 is reflected here. The API endpoints are grouped by service and include the HTTP method, path, description, and reference to the relevant blueprint part. The full OpenAPI specification is served at /api/v1/openapi.json (public) and /api/v1/partner/openapi.json (for marketplace partners).

All DDL is production-ready, includes row-level security (RLS) policies, and is optimised for high-throughput trade execution. The schema is designed to be self-hosted on PostgreSQL 18 with TimescaleDB for time-series data (e.g., IoT sensor readings, audit logs) and pgvector for AI embeddings (e.g., document vectors, sanctions screening).

AI Integration: The schema includes vector columns for semantic search (e.g., document_embeddings for HF sentence-transformers). Embeddings are generated locally using Hugging Face models (HF API key optional) and stored in pgvector for fast similarity search.

13.0.1 Note on Canonical Sources

The complete database schema (DDL) and OpenAPI specification are maintained as living files in the platform repository, separate from this blueprint document. This ensures:

Single source of truth for implementers.

Version control (Git) tracks changes over time.

Direct integration with CI/CD pipelines.

Automated documentation generation from the source files.

Canonical files:

How to use:

For deployment — Use the SQL file directly with psql or migration tools (e.g., Flyway, Liquibase). The file is idempotent and includes CREATE IF NOT EXISTS guards.

For API integration — Import the YAML file into tools like Swagger UI, Postman, or Redoc. The interactive documentation is served at <https://api.sgtx.io/docs> (or locally at <http://localhost:8000/docs>).

For updates — Changes must be made to the canonical files first, then the blueprint's embedded summary (below) is updated to reflect the changes.

Versioning policy:

Major version increment (e.g., v11 → v12) — Breaking changes (table drops, column renames, endpoint removals).

Minor version increment (e.g., v11.0 → v11.1) — Non-breaking additions (new tables, new endpoints, optional fields).

Patch version increment (e.g., v11.1.0 → v11.1.1) — Bug fixes, documentation clarifications.

The blueprint version (v11.0) corresponds to the minor version of the canonical files.

13.1 Complete PostgreSQL Schema (DDL)

13.1.1 Extensions & Enums

sql

-- Enable required extensions

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

CREATE EXTENSION IF NOT EXISTS "pgvector";

CREATE EXTENSION IF NOT EXISTS "timescaledb";

-- Enumerated types (v11.0 additions marked with ★)

CREATE TYPE tenant_type AS ENUM (

'TRD', 'LSP', 'SHIP', 'LAB', 'QC', 'FIN', 'GOV', 'MP', 'CBR'

);

CREATE TYPE financier_subtype AS ENUM ('BANK', 'PRIVATE');

CREATE TYPE lsp_subtype AS ENUM ('TRUCKING', 'FORWARDER', 'WAREHOUSING');

CREATE TYPE trader_mode AS ENUM ('BUY', 'SELL', 'DUAL');

CREATE TYPE shipment_status AS ENUM (

'INITIATED', 'STAGE1_PENDING', 'STAGE1_SETTLED', 'CUSTOMS_SUBMITTED',

```

'BOOKED', 'LOADED', 'DEPARTED', 'IN_TRANSIT', 'ARRIVED',
'CUSTOMS_IMPORT', 'DELIVERED', 'SETTLED', 'COMPLETED',
'DISPUTED', 'DISTRESSED', 'CANCELLED'
);
CREATE TYPE contract_status AS ENUM (
'DRAFT', 'PENDING_SIGNATURES', 'LOCKED', 'ACTIVE',
'COMPLETED', 'DISPUTED', 'TERMINATED', 'ACTIVE_MASTER'
);
CREATE TYPE fee_lock_status AS ENUM ('PENDING', 'ACTIVE', 'PARTIALLY_RELEASED',
'DISPUTED', 'CANCELLED');
CREATE TYPE dispute_category AS ENUM (
'QUALITY', 'DELAY', 'NON_PAYMENT', 'DOCUMENTATION_FRAUD',
'COLD_CHAIN', 'WEIGHT_SHORTAGE', 'SERVICE_QUALITY',
'FINANCING', 'SGTX_FEE', 'TRI_DISPUTE', 'OTHER'
);
CREATE TYPE service_type AS ENUM (
'LAB_TEST', 'QC_INSPECTION', 'BROKER_CERTIFICATION',
'BROKER_PHYSICAL', 'BROKER_STORAGE', 'BROKER_AUDIT',
'TRUCKING', 'OCEAN_FREIGHT', 'AIR_FREIGHT', 'WAREHOUSING', 'FORWARDING'
);
CREATE TYPE logistics_mode AS ENUM ('MANUAL', 'RFQ_LSP', 'DIRECT_SHIP');
CREATE TYPE verdict_type AS ENUM ('ALLOW', 'DENY', 'CONDITIONAL', 'ESCALATE', 'PENDING');
CREATE TYPE inspection_verdict AS ENUM ('PASS', 'FAIL', 'CONDITIONAL');
-- ★ New for v11.0
CREATE TYPE conditional_pass_status AS ENUM ('PENDING', 'COMPLETED', 'EXPIRED');
CREATE TYPE deferred_payment_status AS ENUM ('GUARANTEE_HELD', 'RELEASED', 'PAID',
'EXPIRED');
CREATE TYPE task_status AS ENUM ('PENDING', 'ASSIGNED', 'ESCALATED', 'COMPLETED',
'EXPIRED');
CREATE TYPE experience_mode AS ENUM ('GUIDED', 'EXPERT', 'AUTO');
CREATE TYPE psp_health_status AS ENUM ('HEALTHY', 'DEGRADED', 'DOWN');
13.1.2 Identity & Tenants
sql
-- Tenants (core identity)
CREATE TABLE tenants (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

gtid TEXT UNIQUE NOT NULL,
legal_name TEXT NOT NULL,
legal_name_ar TEXT,
type tenant_type NOT NULL,
financier_subtype financier_subtype,
lsp_subtype lsp_subtype,
jurisdiction TEXT NOT NULL,
tax_id TEXT,
commercial_register TEXT,
kyb_tier INTEGER DEFAULT 1,
trust_score DECIMAL(5,2),
sanctions_cleared BOOLEAN DEFAULT false,
lifecycle_state TEXT DEFAULT 'REGISTERED',
default_trader_mode trader_mode DEFAULT 'DUAL',
anonymous_rfq_opt_out BOOLEAN DEFAULT false,
branding JSONB DEFAULT '{}',
qes_certificate_ref TEXT,
readiness_score INTEGER DEFAULT 0,
readiness_last_calculated TIMESTAMPTZ,
default_incoterm TEXT DEFAULT 'CFR',
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Verified identifiers (LEI, DUNS, etc.)
CREATE TABLE tenant_verified_ids (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
id_type TEXT NOT NULL, -- 'LEI', 'DUNS', 'CUSTOMS_REG', 'CHAMBER_REG', 'VAT_REG'
id_value TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED'
verified_at TIMESTAMPTZ,
expires_at TIMESTAMPTZ,
reference_url TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, id_type)

```

```

);
-- Employees
CREATE TABLE employees (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
email TEXT NOT NULL,
full_name TEXT NOT NULL,
role_id UUID,
status TEXT DEFAULT 'INVITED', -- 'INVITED', 'PENDING_APPROVAL', 'ACTIVE', 'SUSPENDED',
'DEACTIVATED'
mfa_enabled BOOLEAN DEFAULT false,
active_trader_mode_context trader_mode,
allow_role_switching BOOLEAN DEFAULT true,
preferred_language TEXT DEFAULT 'en',
voice_stress_consent BOOLEAN DEFAULT false,
support_pin_hash TEXT,
location_consent_granted BOOLEAN DEFAULT false,
location_consent_timestamp TIMESTAMPTZ,
defi_risk_acknowledged_at TIMESTAMPTZ,
experience_mode TEXT DEFAULT 'AUTO', -- 'GUIDED', 'EXPERT', 'AUTO'
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, email)
);
-- Roles
CREATE TABLE roles (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
name TEXT NOT NULL,
permissions TEXT[] NOT NULL,
allowed_trader_modes trader_mode[] DEFAULT ARRAY['BUY','SELL','DUAL'],
created_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, name)
);
-- Data scopes
CREATE TABLE data_scopes (

```

```

employee_id UUID PRIMARY KEY REFERENCES employees(id) ON DELETE CASCADE,
hidden_cost_components TEXT[],
max_transaction_value DECIMAL(20,4),
allow_role_switching BOOLEAN DEFAULT true,
custom_filters JSONB,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Device registry
CREATE TABLE device_registry (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
device_name TEXT,
device_id TEXT NOT NULL,
public_key TEXT,
state TEXT DEFAULT 'NEW', -- 'NEW', 'TRUSTED', 'ELEVATED_RISK', 'BLOCKED', 'REVOKED'
risk_score INTEGER DEFAULT 0,
last_seen TIMESTAMPTZ,
registered_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(employee_id, device_id)
);

-- Session risk events
CREATE TABLE session_risk_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
session_id TEXT,
risk_type TEXT NOT NULL, -- 'IMPOSSIBLE_TRAVEL', 'VPN_DETECTED', 'TOR_DETECTED',
'BEHAVIOURAL_ANOMALY'
risk_score INTEGER,
details JSONB,
governor_verdict TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Consent records
CREATE TABLE consent_records (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_gtid TEXT NOT NULL,

```

```
purpose TEXT NOT NULL, -- 'marketing', 'analytics', 'govt_sharing', 'cross_border', 'voice_biometric'
version TEXT NOT NULL,
granted BOOLEAN NOT NULL,
ip_address INET NOT NULL,
user_agent TEXT,
device_id TEXT,
timestamp TIMESTAMPTZ DEFAULT NOW(),
withdrawal_timestamp TIMESTAMPTZ,
loom_hash TEXT
);
```

-- Onboarding state

```
CREATE TABLE tenant_onboarding_state (
tenant_id UUID PRIMARY KEY REFERENCES tenants(id) ON DELETE CASCADE,
current_step INTEGER NOT NULL DEFAULT 1,
step_data JSONB DEFAULT '{}',
sandbox_active BOOLEAN DEFAULT true,
completed BOOLEAN DEFAULT false,
updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Readiness checklist

```
CREATE TABLE readiness_checklist (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
category TEXT NOT NULL, -- 'company', 'banking', 'trade', 'security', 'legal'
item_key TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'COMPLETED', 'SKIPPED'
completed_at TIMESTAMPTZ,
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, item_key)
);
```

-- Internal organisation

```
CREATE TABLE tenant_business_units (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
parent_bu_id UUID REFERENCES tenant_business_units(id),
```

```

name TEXT NOT NULL,
administrator_employee_id UUID REFERENCES employees(id),
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_departments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
business_unit_id UUID NOT NULL REFERENCES tenant_business_units(id) ON DELETE CASCADE,
name TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_cost_centers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
business_unit_id UUID REFERENCES tenant_business_units(id),
code TEXT NOT NULL,
description TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_approval_groups (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
name TEXT NOT NULL,
employee_ids UUID[] NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_approval_policies (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
action TEXT NOT NULL,
condition_json JSONB NOT NULL,
required_approvals JSONB NOT NULL,
quorum INTEGER NOT NULL,
approval_mode TEXT DEFAULT 'parallel',
enabled BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```


-- Tenant lifecycle history

```
CREATE TABLE tenant_lifecycle_history (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  from_state TEXT NOT NULL,  
  to_state TEXT NOT NULL,  
  reason TEXT,  
  governor_decision_id UUID,  
  changed_by UUID REFERENCES employees(id),  
  changed_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Role journey completion

```
CREATE TABLE role_journey_completion (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,  
  role_type TEXT NOT NULL,  
  step_key TEXT NOT NULL,  
  completed_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(employee_id, role_type, step_key)  
);
```

13.1.3 Trust Passport & TRI

sql

-- Trust Passports

```
CREATE TABLE trust_passports (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  credential_hash TEXT NOT NULL,  
  tri_score INTEGER NOT NULL,  
  tri_confidence INTEGER NOT NULL,  
  tri_status TEXT NOT NULL,  
  component_scores JSONB NOT NULL,  
  issued_at TIMESTAMPTZ DEFAULT NOW(),  
  expires_at TIMESTAMPTZ NOT NULL,  
  loom_hash TEXT,  
  created_by UUID REFERENCES employees(id)
```

```

);
CREATE TABLE trust_passport_shares (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  passport_id UUID NOT NULL REFERENCES trust_passports(id) ON DELETE CASCADE,
  token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  shared_with_gtid TEXT,
  dimensions_shared TEXT[], -- ['all', 'settlement', 'financing', etc.]
  expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '7 days',
  revoked BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE trust_passport_revocations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  share_id UUID NOT NULL REFERENCES trust_passport_shares(id) ON DELETE CASCADE,
  revoked_by UUID REFERENCES employees(id),
  reason TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
-- TRI history
CREATE TABLE tri_history (
  id BIGSERIAL PRIMARY KEY,
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
  tri_score INTEGER NOT NULL,
  confidence DECIMAL(5,2) NOT NULL,
  component_scores JSONB NOT NULL, -- {settlement, compliance, docs, financing, dispute}
  calculated_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_tri_history_tenant_date ON tri_history(tenant_gtid, calculated_at DESC);
-- TRI disputes
CREATE TABLE tri_disputes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
  dispute_reason TEXT NOT NULL,
  supporting_evidence JSONB,
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'UNDER_REVIEW', 'RESOLVED', 'REJECTED'

```

```

tri_before INTEGER,
tri_after INTEGER,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ
);

```

13.1.4 Trade Core & Shipments

sql

-- Trade requests

```

CREATE TABLE trade_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
buyer_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
seller_tenant_id UUID REFERENCES tenants(id),
incoterm TEXT NOT NULL,
parsed_specs JSONB NOT NULL, -- containers, commodities, packing, treatments,
multi_shipment_schedule
multi_shipment_schedule JSONB,
status TEXT DEFAULT 'INITIATED',
marketplace_auto_attributed BOOLEAN DEFAULT false,
marketplace_partner_id UUID REFERENCES marketplace_partners(id) ON DELETE SET NULL,
created_by UUID NOT NULL REFERENCES employees(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Draft history

```

CREATE TABLE draft_history (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID REFERENCES trade_requests(id) ON DELETE CASCADE,
draft_snapshot JSONB NOT NULL,
edited_by UUID REFERENCES employees(id) ON DELETE SET NULL,
edited_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Marketplace attribution

```

CREATE TABLE partner_lead_attributions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID REFERENCES trade_requests(id) ON DELETE CASCADE,

```

```

marketplace_partner_id UUID REFERENCES marketplace_partners(id) ON DELETE SET NULL,
attribution_type TEXT NOT NULL, -- 'first_touch', 'last_touch', 'direct'
revenue_share_pct DECIMAL(5,4),
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Shipments (USTN master object)
CREATE TABLE shipments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT UNIQUE NOT NULL,
contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,
exporter_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,
importer_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,
status shipment_status DEFAULT 'INITIATED',
incoterm TEXT NOT NULL,
gnn_risk JSONB,
causal_analysis JSONB,
container_numbers TEXT[],
vessel_name TEXT,
voyage TEXT,
booking_ref TEXT,
etd TIMESTAMPTZ,
eta TIMESTAMPTZ,
actual_departure TIMESTAMPTZ,
actual_arrival TIMESTAMPTZ,
risk_score INTEGER,
carbon_footprint_kg_co2e DECIMAL(12,2),
release_status TEXT DEFAULT 'PENDING', -- 'PENDING', 'AUTHORISED', 'REVOKED', 'USED',
'EXPIRED'
document_embeddings vector(384),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Shipment milestones
CREATE TABLE shipment_milestones (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
milestone TEXT NOT NULL,
confirmed_at TIMESTAMPTZ NOT NULL,
confirmed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
confirmation_method TEXT, -- 'barcode', 'voice', 'manual', 'api', 'auto_consensus'
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
scanned_barcode_id UUID,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Contract shipments (multi-shipment)
CREATE TABLE contract_shipments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
shipment_number INTEGER NOT NULL,
scheduled_delivery_date DATE NOT NULL,
port_of_discharge TEXT NOT NULL,
containers_count INTEGER NOT NULL,
total_price DECIMAL(20,4) NOT NULL,
sgtx_fee_amount DECIMAL(20,4),
fee_lock_id UUID,
ustn TEXT UNIQUE,
status TEXT DEFAULT 'SCHEDULED', -- SCHEDULED, LOCKED, SHIPPED, CANCELLED,
FAILED_PAYMENT
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Milestone payment schedules (Improvement #9)
CREATE TABLE milestone_payment_schedules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
contract_id UUID NOT NULL REFERENCES contracts(id) ON DELETE CASCADE,
milestone_name TEXT NOT NULL, -- 'BOOKED', 'DEPARTED', 'DELIVERED'
percentage DECIMAL(5,2) NOT NULL,
amount DECIMAL(20,4) NOT NULL,
pre_approved BOOLEAN DEFAULT false,
paid BOOLEAN DEFAULT false,
paid_at TIMESTAMPTZ,

```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Packing plans
CREATE TABLE packing_plans (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
plan_data JSONB NOT NULL, -- includes non-uniform layer patterns
loading_guide TEXT,
locked_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- Pallet details (with non-uniform layers)
CREATE TABLE pallet_details (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
packing_plan_id UUID REFERENCES packing_plans(id) ON DELETE CASCADE,
sscc TEXT UNIQUE NOT NULL,
lot_number TEXT,
product_hs_code CHAR(6),
cartons_per_pallet INTEGER NOT NULL,
layer_patterns JSONB, -- [{layers:11, cartons_per_layer:10}, {layers:1, cartons_per_layer:1}]
total_cartons INTEGER NOT NULL,
gross_weight_kg DECIMAL(10,2),
cold_treatment_cert_ref TEXT,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'LOADED', 'DAMAGED', 'DELIVERED'
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.5 Contracts & Fees

sql

-- Contracts

```

CREATE TABLE contracts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
trade_request_id UUID NOT NULL REFERENCES trade_requests(id) ON DELETE CASCADE,
contract_type TEXT DEFAULT 'SINGLE_SHIPMENT', -- 'SINGLE_SHIPMENT', 'MASTER_MULTI'
incoterm TEXT NOT NULL,

```

```

clauses JSONB NOT NULL,
governing_law TEXT NOT NULL,
dispute_resolution TEXT NOT NULL,
status contract_status DEFAULT 'DRAFT',
cryptographic_hash TEXT,
buyer_signature TEXT,
seller_signature TEXT,
sgtx_fee_rate DECIMAL(5,4) NOT NULL,
sgtx_fee_amount DECIMAL(20,4) NOT NULL,
fee_lock_id UUID,
pqc_signature TEXT,
signed_at TIMESTAMPTZ,
locked_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Fee payment requests (with deferred payment)
CREATE TABLE fee_payment_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,
financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,
party_type TEXT NOT NULL, -- 'seller', 'buyer', 'borrower'
tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
amount DECIMAL(20,4) NOT NULL,
currency TEXT DEFAULT 'USD',
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'PAID', 'FAILED', 'REFUNDED'
due_date TIMESTAMPTZ,
late_fee_accrued DECIMAL(20,4) DEFAULT 0,
deferred BOOLEAN DEFAULT false,
deferred_status TEXT DEFAULT 'GUARANTEE_HELD', -- 'GUARANTEE_HELD', 'RELEASED', 'PAID', 'EXPIRED'
auto_charge_authorised BOOLEAN DEFAULT false,
expiry_action_taken TEXT, -- 'reminded', 'alerted', 'expired_charged', 'expired_blocked'
payment_reference TEXT,

```

```

gross_amount DECIMAL(20,4),
psp_fee DECIMAL(20,4),
psp_transaction_id TEXT,
paid_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- FeeLocks (NATS KV is source of truth)
CREATE TABLE fee_locks (
lock_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,
financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,
ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
fee_rate_pct DECIMAL(5,4) NOT NULL,
fee_usd DECIMAL(20,4) NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACTIVE', 'PARTIALLY_RELEASED', 'DISPUTED',
'CANCELLED'
kv_version BIGINT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
locked_at TIMESTAMPTZ DEFAULT NOW(),
released_at TIMESTAMPTZ,
fully_released_at TIMESTAMPTZ
);
-- Late fee events
CREATE TABLE late_fee_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
fee_payment_request_id UUID NOT NULL REFERENCES fee_payment_requests(id) ON DELETE
CASCADE,
day_number INTEGER NOT NULL,
late_fee_amount DECIMAL(20,4) NOT NULL,
total_accrued DECIMAL(20,4) NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.6 Logistics (Three Modes: A, B, C)

sql

-- Service quotations (Mode A & B)

```
CREATE TABLE service_quotations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
  provider_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  service_type service_type NOT NULL,  
  fee DECIMAL(10,2) NOT NULL,  
  currency TEXT DEFAULT 'USD',  
  valid_until TIMESTAMPTZ NOT NULL,  
  notes TEXT,  
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACCEPTED', 'DECLINED', 'EXPIRED'  
  accepted_at TIMESTAMPTZ,  
  invoice_uuid TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Clarification requests (Improvement #4)

```
CREATE TABLE clarification_requests (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  quotation_id UUID REFERENCES service_quotations(id) ON DELETE CASCADE,  
  requester_gtid TEXT NOT NULL,  
  questions JSONB NOT NULL,  
  answers JSONB,  
  resolved BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  resolved_at TIMESTAMPTZ  
);
```

-- Ship quote requests (Mode C)

```
CREATE TABLE ship_quote_requests (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
  seller_gtid TEXT NOT NULL,  
  base_service_type TEXT NOT NULL, -- 'OCEAN_FREIGHT', 'AIR_FREIGHT'  
  origin_port TEXT NOT NULL,  
  destination_port TEXT NOT NULL,
```

```

container_details JSONB NOT NULL,
add_on_services TEXT[], -- 'TRUCKING', 'CUSTOMS_BROKER', 'INSURANCE'
target_lines TEXT[], -- GTIDs of shipping lines
status TEXT DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE ship_quotes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
request_id UUID NOT NULL REFERENCES ship_quote_requests(id) ON DELETE CASCADE,
shipper_line_gtid TEXT NOT NULL,
base_fee DECIMAL(20,4) NOT NULL,
add_on_fees JSONB, -- {"TRUCKING": 300, "CUSTOMS_BROKER": 150}
total_fee DECIMAL(20,4) NOT NULL,
validity_period_hours INTEGER DEFAULT 48,
selected BOOLEAN DEFAULT false,
submitted_at TIMESTAMPTZ DEFAULT NOW()
);

-- Carrier contract rates (private)
CREATE TABLE carrier_contracts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
carrier_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
seller_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
route_origin TEXT,
route_destination TEXT,
container_type TEXT,
base_rate DECIMAL(20,4),
currency TEXT DEFAULT 'USD',
valid_from TIMESTAMPTZ,
valid_until TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.7 QC Inspection

```
sql
```

```
-- QC jobs
```

```
CREATE TABLE qc_jobs (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
provider_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
inspection_date DATE,
sampling_plan JSONB, -- AQL plan
status TEXT DEFAULT 'ASSIGNED', -- 'ASSIGNED', 'ACCEPTED', 'IN_PROGRESS', 'COMPLETED',
'DISPUTED'
verdict inspection_verdict,
conditional_pass_status conditional_pass_status,
action_plan TEXT,
action_plan_deadline TIMESTAMPTZ,
action_plan_completed_at TIMESTAMPTZ,
report_url TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Inspection logs (with overrides)
CREATE TABLE inspection_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
qc_job_id UUID REFERENCES qc_jobs(id) ON DELETE CASCADE,
pallet_id TEXT,
ai_defect TEXT,
ai_confidence DECIMAL(5,2),
inspector_override BOOLEAN DEFAULT false,
inspector_override_reason TEXT,
final_classification TEXT,
photo_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Re-inspection requests
CREATE TABLE re_inspection_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
original_ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
requested_by_gtid TEXT NOT NULL,
reason TEXT NOT NULL,
urgency TEXT DEFAULT 'NORMAL', -- 'NORMAL', 'URGENT'
deadline TIMESTAMPTZ,

```

```

status TEXT DEFAULT 'PENDING', -- 'PENDING', 'ACCEPTED', 'DECLINED', 'COMPLETED'
new_qc_job_id UUID REFERENCES qc_jobs(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.8 Broker Services

sql

-- Broker certifications

```

CREATE TABLE broker_certifications (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
declaration_id TEXT,
certified_at TIMESTAMPTZ,
digital_seal TEXT,
licence_number TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Broker physical jobs

```

CREATE TABLE broker_physical_jobs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
package_id TEXT UNIQUE,
courier_tracking TEXT,
status TEXT DEFAULT 'AWAITING_RECEIPT', -- 'AWAITING_RECEIPT', 'RECEIVED',
'PRESENTED_TO_CUSTOMS', 'STAMPED', 'COMPLETED'
received_at TIMESTAMPTZ,
presented_at TIMESTAMPTZ,
stamped_at TIMESTAMPTZ,
stamped_copy_url TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Broker storage

```

CREATE TABLE broker_storage (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
shelf_location TEXT,
retention_expiry DATE,
status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'RETURNED', 'DESTROYED'
returned_at TIMESTAMPTZ,
destroyed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.9 Financing

sql

-- Financing requests

```

CREATE TABLE financing_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
requester_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
amount DECIMAL(20,4) NOT NULL,
currency TEXT DEFAULT 'USD',
tenor_days INTEGER NOT NULL,
financing_type TEXT NOT NULL,
preferred_settlement_method TEXT,
credit_intelligence JSONB,
status TEXT DEFAULT 'REQUESTED',
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Financier preferences

```

CREATE TABLE financier_preferences (
financier_tenant_id UUID PRIMARY KEY REFERENCES tenants(id) ON DELETE CASCADE,
accepted_borrower_countries TEXT[],
min_borrower_trust_score DECIMAL(5,2),
min_trade_value DECIMAL(20,4),
max_financed_amount DECIMAL(20,4),
preferred_financing_types TEXT[],

```

```

preferred_settlement_methods TEXT[],
excluded_commodities TEXT[],
geographic_restrictions TEXT[], -- 'accept_only', 'all_except'
accept_only_countries TEXT[],
all_except_countries TEXT[],
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Financing offers (encrypted bids)
CREATE TABLE financing_offers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_request_id UUID NOT NULL REFERENCES financing_requests(id) ON DELETE CASCADE,
financier_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
amount_offered DECIMAL(20,4) NOT NULL,
apr DECIMAL(6,4) NOT NULL,
bid_encrypted BOOLEAN DEFAULT true,
reserve_proof TEXT, -- ZK proof of reserves
status TEXT DEFAULT 'SUBMITTED', -- 'SUBMITTED', 'ACCEPTED', 'REJECTED', 'COUNTERED'
submitted_at TIMESTAMPTZ DEFAULT NOW()
);

-- Financing agreements
CREATE TABLE financing_agreements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_request_id UUID NOT NULL REFERENCES financing_requests(id) ON DELETE CASCADE,
financier_tenant_id UUID NOT NULL,
amount DECIMAL(20,4) NOT NULL,
apr DECIMAL(6,4) NOT NULL,
tenor_days INTEGER NOT NULL,
sgtx_financing_fee DECIMAL(20,4) NOT NULL,
takaful_opted_in BOOLEAN DEFAULT false,
status TEXT DEFAULT 'PENDING_SIGNATURES',
disbursed_at TIMESTAMPTZ,
repaid_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Financing repayments

```

```

CREATE TABLE financing_repayments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_agreement_id UUID NOT NULL REFERENCES financing_agreements(id) ON DELETE CASCADE,
scheduled_date DATE,
actual_paid_at TIMESTAMPTZ,
principal_paid DECIMAL(20,4),
interest_paid DECIMAL(20,4),
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'PAID', 'LATE', 'DEFAULTED'
late_days INTEGER,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL
);
-- Financier historical data (private)
CREATE TABLE financier_historical_data (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financier_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
borrower_gtid TEXT NOT NULL,
total_financed_amount DECIMAL(20,4),
number_of_financings INTEGER,
on_time_repayments INTEGER,
late_repayments INTEGER,
defaults INTEGER,
last_updated TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(financier_tenant_id, borrower_gtid)
);
-- DeFi positions
CREATE TABLE defi_tranche_positions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_agreement_id UUID NOT NULL REFERENCES financing_agreements(id) ON DELETE CASCADE,
protocol_id UUID,
amount_stablecoin DECIMAL(20,8) NOT NULL,
stablecoin TEXT NOT NULL,
chain TEXT DEFAULT 'POLYGON',
contract_address TEXT NOT NULL,
deposit_tx_hash TEXT,

```

```

withdrawal_tx_hash TEXT,
depeg_alert_triggered BOOLEAN DEFAULT false,
status TEXT DEFAULT 'ACTIVE',
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.10 Takaful Protection Fund

sql

-- Takaful fund balance

```

CREATE TABLE takaful_fund (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
balance DECIMAL(20,2) NOT NULL DEFAULT 0,
currency TEXT DEFAULT 'USD',
last_updated TIMESTAMPTZ DEFAULT NOW()
);

```

-- Takaful contributions

```

CREATE TABLE takaful_contributions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE CASCADE,
amount DECIMAL(20,2) NOT NULL,
paid_by_gtid TEXT,
collected_at TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT
);

```

-- Takaful claims

```

CREATE TABLE takaful_claims (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
claim_type TEXT NOT NULL CHECK (claim_type IN ('STABLECOIN_DEPEG',
'LIQUIDITY_DISRUPTION', 'RESERVE_SHORTFALL')),
financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE CASCADE,
claimant_gtid TEXT NOT NULL,
amount_requested DECIMAL(20,2) NOT NULL,
amount_paid DECIMAL(20,2),
status TEXT DEFAULT 'PENDING',
evidence JSONB,
sharia_board_approvers TEXT[], -- GTIDs of approving board members
rejection_reason TEXT,

```



```
paid_at TIMESTAMPTZ,  
loom_hash TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

13.1.11 Distressed Cargo

sql

-- Distressed cargo listings

```
CREATE TABLE distressed_cargo_listings (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
original_shipment_ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
micro_ustn TEXT,  
parent_ustn TEXT,  
seller_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
product_details JSONB,  
quantity DECIMAL(10,2) NOT NULL,  
condition_score INTEGER,  
price_expectation DECIMAL(20,4),  
floor_price DECIMAL(20,4),  
listing_expiry TIMESTAMPTZ NOT NULL,  
status TEXT DEFAULT 'ACTIVE',  
triage_path TEXT, -- 'SELL', 'COMPLY', 'INSURANCE'  
privacy_notice_acknowledged BOOLEAN DEFAULT false,  
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Distressed cargo offers

```
CREATE TABLE distressed_cargo_offers (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
listing_id UUID NOT NULL REFERENCES distressed_cargo_listings(id) ON DELETE CASCADE,  
buyer_tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
amount DECIMAL(20,4) NOT NULL,  
status TEXT DEFAULT 'PENDING',  
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
submitted_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Distressed contact checks

```
CREATE TABLE distressed_contact_checks (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  listing_id UUID NOT NULL REFERENCES distressed_cargo_listings(id) ON DELETE CASCADE,  
  seller_tenant_id UUID NOT NULL,  
  buyer_gtid TEXT NOT NULL,  
  match_score INTEGER,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Distressed contact notifications

```
CREATE TABLE distressed_contact_notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  distressed_ustn TEXT NOT NULL,  
  recipient_gtid TEXT NOT NULL,  
  message_hash TEXT,  
  sent_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Micro-contracts (distressed)

```
CREATE TABLE micro_contracts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  parent_contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,  
  distressed_listing_id UUID REFERENCES distressed_cargo_listings(id) ON DELETE SET NULL,  
  micro_ustn TEXT UNIQUE NOT NULL,  
  buyer_gtid TEXT NOT NULL,  
  seller_gtid TEXT NOT NULL,  
  agreed_price DECIMAL(20,4),  
  sgtx_fee_rate DECIMAL(5,4),  
  sgtx_fee_amount DECIMAL(20,4),  
  fee_lock_id UUID,  
  status TEXT DEFAULT 'PENDING_SIGNATURES',  
  locked_at TIMESTAMPTZ,  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

13.1.12 Disputes & Evidence

sql

-- Disputes

```
CREATE TABLE disputes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,  
  contract_shipment_id UUID REFERENCES contract_shipments(id) ON DELETE SET NULL,  
  financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,  
  filing_party_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  dispute_category dispute_category NOT NULL,  
  description TEXT NOT NULL,  
  remedy_sought TEXT,  
  status TEXT DEFAULT 'FILED', -- 'FILED', 'MEDIATION', 'SETTLED', 'ARBITRATION_PENDING',  
  'ARBITRATION_COMPLETE', 'CLOSED'  
  mediation_log JSONB DEFAULT '[]',  
  arbitration_case_id TEXT,  
  resolution_contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,  
  ai_settlement_proposal JSONB,  
  predicted_outcome JSONB,  
  document_authenticity_flags JSONB DEFAULT '[]',  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  filed_at TIMESTAMPTZ DEFAULT NOW(),  
  resolved_at TIMESTAMPTZ,  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Dispute recommendations

```
CREATE TABLE dispute_recommendations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  dispute_id UUID NOT NULL REFERENCES disputes(id) ON DELETE CASCADE,  
  ai_prediction TEXT,  
  suggested_settlement JSONB,  
  confidence DECIMAL(5,4),  
  shap_values JSONB,  
  model_version TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Evidence packages

```

CREATE TABLE evidence_packages (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
dispute_id UUID NOT NULL REFERENCES disputes(id) ON DELETE CASCADE,
package JSONB NOT NULL,
loom_hash TEXT NOT NULL,
verification_token TEXT UNIQUE,
compiled_at TIMESTAMPTZ DEFAULT NOW()
);

-- Causal attribution
CREATE TABLE causal_attribution (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
entity_id UUID NOT NULL, -- dispute_id or shipment_id
entity_type TEXT NOT NULL, -- 'DISPUTE', 'SHIPMENT'
factor TEXT NOT NULL,
contribution DECIMAL(5,4) NOT NULL,
confidence_interval JSONB,
description TEXT,
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Third-party experts
CREATE TABLE dispute_experts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
dispute_id UUID REFERENCES disputes(id) ON DELETE CASCADE,
expert_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,
expert_type TEXT NOT NULL, -- 'surveyor', 'laboratory', 'arbitrator', 'legal'
invitation_token TEXT UNIQUE,
opinion TEXT,
posted_at TIMESTAMPTZ,
status TEXT DEFAULT 'INVITED', -- 'INVITED', 'ACCEPTED', 'DECLINED', 'POSTED'
created_at TIMESTAMPTZ DEFAULT NOW()
);

13.1.13 Governance, Audit & Smart Inbox
sql

-- Governor decisions

```

```

CREATE TABLE governor_decisions (
decision_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
decision_type TEXT NOT NULL,
actor_gtid TEXT NOT NULL,
actor_employee_id UUID REFERENCES employees(id) ON DELETE SET NULL,
verdict verdict_type NOT NULL,
tenant_message JSONB,
loom_hash TEXT NOT NULL,
cryptographic_signature TEXT NOT NULL,
pqc_signature TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- AI inference records

```

CREATE TABLE ai_inference_records (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
agent_name TEXT NOT NULL,
authority_level TEXT NOT NULL, -- 'A0', 'A1', 'A2', 'A3', 'A4'
action_context JSONB NOT NULL,
decision JSONB NOT NULL,
confidence DECIMAL(5,4),
shap_values JSONB,
explanation TEXT,
loom_hash TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Audit log (partitioned by date)

```

CREATE TABLE audit_log (
id BIGSERIAL,
table_name TEXT NOT NULL,
record_id UUID NOT NULL,
action TEXT NOT NULL,
before JSONB,
after JSONB,
changed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
changed_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

) PARTITION BY RANGE (changed_at);

-- Loom verification tokens

CREATE TABLE loom_verification_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
  token UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  expires_at TIMESTAMPTZ NOT NULL DEFAULT NOW() + INTERVAL '90 days',
  created_by UUID REFERENCES employees(id) ON DELETE SET NULL,
  revoked BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Smart Inbox items

CREATE TABLE inbox_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
  item_id UUID UNIQUE NOT NULL,
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  category TEXT NOT NULL CHECK (category IN ('NEEDS_SIGNATURE','NEEDS_APPROVAL','NEEDS_DOCUMENT','NEEDS_PAYMENT','SHIPMENT_ALERT','NEW_OFFER','NEGOTIATION','COMPLIANCE','GENERAL')),
  priority_score INTEGER NOT NULL CHECK (priority_score BETWEEN 0 AND 100),
  title TEXT NOT NULL,
  description TEXT,
  deadline TIMESTAMPTZ,
  action_link TEXT NOT NULL,
  snoozed_until TIMESTAMPTZ,
  dismissed BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Inbox history

CREATE TABLE inbox_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
  item_id UUID NOT NULL,
  ustn TEXT,

```

```
action_taken TEXT,  
timestamp TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Inbox preferences

```
CREATE TABLE inbox_preferences (  
employee_id UUID PRIMARY KEY REFERENCES employees(id) ON DELETE CASCADE,  
min_score INTEGER DEFAULT 30,  
muted_categories TEXT[] DEFAULT '{}',  
preferred_language TEXT DEFAULT 'en'  
);
```

13.1.14 Jurisdiction, Ports & RIA

sql

-- Jurisdictions (country matrix)

```
CREATE TABLE jurisdictions (  
code TEXT PRIMARY KEY,  
name TEXT NOT NULL,  
sanctions_level TEXT NOT NULL, -- 'FULL', 'STANDARD', 'LIMITED', 'RESTRICTED', 'BLOCKED'  
regulatory_body TEXT,  
defi_allowed BOOLEAN DEFAULT false,  
defi_restrictions JSONB,  
stablecoins_allowed TEXT[],  
private_financier_allowed BOOLEAN DEFAULT false,  
distressed_sale_allowed TEXT, -- 'YES', 'CONDITIONAL', 'NO'  
distressed_country_factor DECIMAL(5,4) DEFAULT 1.0,  
distressed_sgtx_fee_factor DECIMAL(5,4) DEFAULT 1.0,  
kyc_tier_required INTEGER DEFAULT 2,  
deferred_fees_allowed TEXT[], -- ['IMPORT_DUTIES', 'VAT']  
registry_api_endpoint TEXT,  
customs_api_endpoint TEXT,  
cbdc_status TEXT DEFAULT 'NONE',  
psp_partners JSONB,  
reporting_requirements JSONB,  
last_updated TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Ports

```

CREATE TABLE ports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
unlocode TEXT UNIQUE NOT NULL,
name TEXT NOT NULL,
country_code TEXT REFERENCES jurisdictions(code) ON DELETE CASCADE,
latitude DECIMAL(10,8),
longitude DECIMAL(11,8),
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- RIA maintained tables

CREATE TABLE commodity_packing_defaults (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
commodity_type TEXT NOT NULL,
default_pallet_type TEXT DEFAULT 'EUR',
default_cartons_per_pallet INTEGER DEFAULT 40,
default_net_weight_per_carton_kg DECIMAL(10,2) DEFAULT 10,
default_stacking_layers INTEGER DEFAULT 5,
max_pallets_per_container_40ft INTEGER DEFAULT 22,
carton_dimensions_mm JSONB,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE treatment_requirements (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_code TEXT,
treatment_type TEXT NOT NULL, -- 'cold_treatment', 'fumigation', 'irradiation', 'pre_cooling'
temperature_c DECIMAL(5,2),
duration_days INTEGER,
certificate_required BOOLEAN DEFAULT true,
accepted_facilities TEXT[],
mandate_effective_date DATE,

```



```

source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE country_mrl (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
destination_country CHAR(2) NOT NULL,
analyte TEXT NOT NULL,
mrl_mg_per_kg DECIMAL(10,4),
unit TEXT DEFAULT 'mg/kg',
limit_operator TEXT DEFAULT '<=',
source_regulation TEXT,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE port_special_rules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
port_code TEXT NOT NULL REFERENCES ports(unlocode) ON DELETE CASCADE,
rule_type TEXT NOT NULL, -- 'pre_cooling_certificate', 'cold_treatment', 'fumigation'
description TEXT,
mandatory BOOLEAN DEFAULT true,
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE commodity_dynamic_schemas_cache (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
hs_code CHAR(6) NOT NULL,
origin_country CHAR(2) NOT NULL,
destination_country CHAR(2) NOT NULL,
port_code TEXT,
schema_json JSONB NOT NULL,
generated_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '7 days',
product_form_agent_version TEXT,
UNIQUE(hs_code, origin_country, destination_country, port_code)
);

```

13.1.15 Documents

sql

-- Document requirements

```
CREATE TABLE shipment_document_requirements (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
  document_type TEXT NOT NULL,  
  responsible_party_type TEXT,  
  responsible_tenant_id UUID REFERENCES tenants(id) ON DELETE SET NULL,  
  is_critical BOOLEAN DEFAULT false,  
  status TEXT DEFAULT 'PENDING',  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL  
);
```

-- Documents

```
CREATE TABLE documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  requirement_id UUID REFERENCES shipment_document_requirements(id) ON DELETE SET NULL,  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  uploaded_by_provider_gtid TEXT,  
  document_type TEXT NOT NULL,  
  filename TEXT,  
  storage_path TEXT NOT NULL,  
  sha256_hash TEXT NOT NULL,  
  version INTEGER DEFAULT 1,  
  previous_version_id UUID REFERENCES documents(id) ON DELETE SET NULL,  
  uploaded_by UUID NOT NULL REFERENCES employees(id) ON DELETE SET NULL,  
  metadata JSONB,  
  ai_extracted JSONB,  
  verification_status TEXT DEFAULT 'PENDING',  
  verified_at TIMESTAMPTZ,  
  visible_to_roles TEXT[], -- ['BUYER', 'SELLER', 'FINANCIER', etc.]  
  visible_to_tenant_ids UUID[],  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  uploaded_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Generated documents (PDF/A-3)

```
CREATE TABLE generated_documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
  document_type TEXT NOT NULL,  
  draft_data JSONB NOT NULL,  
  status TEXT DEFAULT 'DRAFT',  
  finalized_document_id UUID REFERENCES documents(id) ON DELETE SET NULL,  
  finalized_at TIMESTAMPTZ,  
  pdfa3_compliant BOOLEAN DEFAULT false  
);
```

-- eBL tracking

```
CREATE TABLE ebl_capability_matrix (  
  carrier_id TEXT PRIMARY KEY,  
  carrier_name TEXT NOT NULL,  
  supported_platforms TEXT[],  
  supported_routes JSONB,  
  last_verified TIMESTAMPTZ,  
  is_active BOOLEAN DEFAULT true  
);
```

```
CREATE TABLE shipment_ebls (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,  
  platform TEXT NOT NULL,  
  platform_reference TEXT NOT NULL,  
  issue_date DATE,  
  current_holder_gtid TEXT REFERENCES tenants(gtid) ON DELETE SET NULL,  
  status TEXT DEFAULT 'ISSUED',  
  events JSONB DEFAULT '[]',  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

13.1.16 Payments & Settlement

sql

-- Payment aggregators (PSPs)

```
CREATE TABLE payment_aggregators (  

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
name TEXT NOT NULL UNIQUE,
country_codes TEXT[] NOT NULL,
supported_currencies TEXT[],
supports_split BOOLEAN DEFAULT true,
api_endpoint TEXT,
credentials_encrypted JSONB,
uptime_score DECIMAL(5,4),
last_health_check TIMESTAMPTZ,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Payment attempts

```

CREATE TABLE payment_attempts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
fee_payment_request_id UUID NOT NULL REFERENCES fee_payment_requests(id) ON DELETE CASCADE,
aggregator_id UUID NOT NULL REFERENCES payment_aggregators(id) ON DELETE CASCADE,
payer_country TEXT NOT NULL,
payment_method TEXT NOT NULL,
amount_local DECIMAL(20,4),
currency_local TEXT,
amount_usd DECIMAL(20,4),
fx_rate DECIMAL(16,8),
gross_amount DECIMAL(20,4),
net_amount DECIMAL(20,4),
status TEXT DEFAULT 'INITIATED', -- 'INITIATED', 'SUCCEEDED', 'FAILED', 'REFUNDED'
failure_reason TEXT,
retry_count INTEGER DEFAULT 0,
psp_transaction_id TEXT,
split_executed_at TIMESTAMPTZ,
settled_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Fee calculations

```

CREATE TABLE fee_calculations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_attempt_id UUID REFERENCES payment_attempts(id) ON DELETE SET NULL,
  net_fee DECIMAL(20,4) NOT NULL,
  gross_amount DECIMAL(20,4) NOT NULL,
  fee_breakdown JSONB,
  safety_buffer_applied BOOLEAN DEFAULT true,
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- PSP health logs
CREATE TABLE psp_health_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregator_id UUID NOT NULL REFERENCES payment_aggregators(id) ON DELETE CASCADE,
  health_score DECIMAL(5,2),
  latency_ms INTEGER,
  error_rate DECIMAL(5,4),
  checked_at TIMESTAMPTZ DEFAULT NOW()
);

-- Settlement instructions
CREATE TABLE settlement_instructions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
  instruction_type TEXT NOT NULL, -- 'TRADE_PRINCIPAL', 'FEE_SPLIT', 'REFUND'
  payload JSONB NOT NULL,
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'AUTHORISED', 'CONFIRMED', 'RECONCILED', 'FAILED'
  psp_reference TEXT,
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  confirmed_at TIMESTAMPTZ,
  reconciled_at TIMESTAMPTZ
);

-- Settlement confirmations (webhooks)
CREATE TABLE settlement_confirmations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

instruction_id UUID NOT NULL REFERENCES settlement_instructions(id) ON DELETE CASCADE,
proof_hash TEXT NOT NULL,
amount_confirmed DECIMAL(20,4),
currency_confirmed TEXT,
fx_rate_applied DECIMAL(16,8),
psp_name TEXT,
webhook_received_at TIMESTAMPTZ,
verified_by_ai BOOLEAN DEFAULT false,
ai_verdict JSONB,
zk_proof TEXT, -- private settlement proof
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
confirmed_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Bank settlement instructions (direct bank integration)

```

CREATE TABLE bank_settlement_instructions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
from_iban TEXT NOT NULL,
to_iban TEXT NOT NULL,
amount DECIMAL(20,4) NOT NULL,
currency TEXT DEFAULT 'USD',
value_date DATE NOT NULL,
reference TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'CONFIRMED', 'RECONCILED', 'FAILED'
bank_bic TEXT,
transaction_reference TEXT,
confirmed_at TIMESTAMPTZ,
reconciled_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Bank reconciliation files

```

CREATE TABLE bank_reconciliation_files (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
bank_bic TEXT NOT NULL,

```

```

file_date DATE NOT NULL,
format TEXT NOT NULL, -- 'MT940', 'CAMT_053', 'CSV'
file_content TEXT,
file_hash TEXT,
generated_at TIMESTAMPTZ DEFAULT NOW(),
downloaded_by UUID REFERENCES employees(id) ON DELETE SET NULL
);

```

13.1.17 Integration Logs

sql

-- Integration connector logs

```

CREATE TABLE integration_connector_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
connector_name TEXT NOT NULL, -- 'NAFEZA', 'CARGOX', 'ETA', 'PSP_FAWRY', etc.
endpoint TEXT NOT NULL,
request_body TEXT,
response_body TEXT,
status_code INTEGER,
idempotency_key TEXT,
duration_ms INTEGER,
success BOOLEAN,
error_message TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Certificates (TLS, e-Seal)

```

CREATE TABLE certificates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
certificate_type TEXT NOT NULL, -- 'E_SEAL', 'TLS_CLIENT', 'TLS_SERVER'
certificate_pem TEXT NOT NULL,
private_key_encrypted TEXT,
issuer TEXT,
valid_from TIMESTAMPTZ,
valid_until TIMESTAMPTZ,
status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'EXPIRED', 'REVOKED'
uploaded_by UUID REFERENCES employees(id) ON DELETE SET NULL,

```

```

created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Revoked certificates (CRL)
CREATE TABLE revoked_certificates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
certificate_serial TEXT NOT NULL UNIQUE,
revoked_at TIMESTAMPTZ DEFAULT NOW(),
reason TEXT, -- 'COMPROMISED', 'SUSPENDED', 'EXPIRED', 'SUPERSEDED'
revoked_by UUID REFERENCES employees(id) ON DELETE SET NULL
);

```

13.1.18 Release Authorisations

sql

```

-- Release authorisations (Container Release API)
CREATE TABLE release_authorisations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustrn TEXT NOT NULL REFERENCES shipments(ustrn) ON DELETE CASCADE,
container TEXT NOT NULL,
authorisation_id TEXT UNIQUE NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'AUTHORISED', 'REVOKED', 'USED', 'EXPIRED'
issued_at TIMESTAMPTZ,
valid_until TIMESTAMPTZ,
used_at TIMESTAMPTZ,
revoked_at TIMESTAMPTZ,
revocation_reason TEXT,
digital_signature TEXT,
request_id TEXT,
terminal_gtid TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Gate-out events
CREATE TABLE gate_out_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustrn TEXT NOT NULL REFERENCES shipments(ustrn) ON DELETE CASCADE,

```



```

container TEXT NOT NULL,
authorisation_id TEXT NOT NULL REFERENCES release_authorisations(authorisation_id) ON DELETE
CASCADE,
gate_out_time TIMESTAMPTZ NOT NULL,
operator_id TEXT,
terminal_gtid TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Release overrides (emergency only)
CREATE TABLE release_overrides (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
container TEXT NOT NULL,
override_token TEXT UNIQUE NOT NULL,
issued_by UUID REFERENCES employees(id) ON DELETE SET NULL,
issued_at TIMESTAMPTZ DEFAULT NOW(),
valid_until TIMESTAMPTZ NOT NULL,
used_at TIMESTAMPTZ,
reason TEXT NOT NULL,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL
);

```

13.1.19 Marketplace Partners

```

sql
-- Marketplace partners
CREATE TABLE marketplace_partners (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
partner_name TEXT NOT NULL,
api_key_hash TEXT NOT NULL,
revenue_share_percent DECIMAL(3,2),
active BOOLEAN DEFAULT true,
api_key_encrypted TEXT,
api_key_created_at TIMESTAMPTZ,
api_key_last_used TIMESTAMPTZ,
ip_whitelist TEXT[],
sandbox_enabled BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

);
-- Partner rate limits
CREATE TABLE partner_rate_limits (
partner_id UUID NOT NULL REFERENCES marketplace_partners(id) ON DELETE CASCADE,
endpoint TEXT NOT NULL,
limit_per_window INTEGER NOT NULL,
window_seconds INTEGER NOT NULL,
updated_at TIMESTAMPTZ DEFAULT NOW(),
PRIMARY KEY (partner_id, endpoint)
);
-- Webhook delivery logs
CREATE TABLE webhook_delivery_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
partner_id UUID NOT NULL REFERENCES marketplace_partners(id) ON DELETE CASCADE,
endpoint_url TEXT NOT NULL,
event_type TEXT NOT NULL,
payload JSONB,
response_status INTEGER,
response_body TEXT,
duration_ms INTEGER,
success BOOLEAN,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Partner sandbox leads
CREATE TABLE partner_sandbox_leads (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
partner_id UUID NOT NULL REFERENCES marketplace_partners(id) ON DELETE CASCADE,
raw_text TEXT,
parsed_specs JSONB,
viability_score INTEGER,
created_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '7 days'
);

```

13.1.20 Saved Contacts & Network

sql

-- Saved contacts (Network feature)

```
CREATE TABLE tenant_contacts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,  
  contact_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  relationship_type TEXT, -- 'BUYER', 'SUPPLIER', 'LOGISTICS_PROVIDER', 'FINANCIER',  
  'QC_PROVIDER', 'INSURER', 'GENERAL'  
  trade_count INTEGER DEFAULT 0,  
  total_value DECIMAL(20,4) DEFAULT 0,  
  first_interaction TIMESTAMPTZ,  
  last_interaction TIMESTAMPTZ,  
  is_favorite BOOLEAN DEFAULT false,  
  is_blocked BOOLEAN DEFAULT false,  
  trust_snapshot JSONB,  
  relationship_health_score DECIMAL(5,2),  
  health_summary TEXT,  
  smart_labels TEXT[],  
  indirect_connections JSONB,  
  last_ai_update TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(tenant_id, contact_gtid)  
);
```

-- User notes per shipment

```
CREATE TABLE user_shipment_notes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,  
  user_employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,  
  note_text TEXT NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(ustn, user_employee_id)  
);
```

-- User saved views (for Shipments Vault)

```
CREATE TABLE user_saved_views (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

user_employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
view_name TEXT NOT NULL,
filters JSONB NOT NULL,
sort_column TEXT,
sort_order TEXT,
visible_columns TEXT[],
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.21 Tasks & Feedback

sql

-- Tasks (Unified Task Center)

```

CREATE TABLE tasks (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
task_type TEXT NOT NULL, -- 'SIGN_CONTRACT', 'PAY_FEE', 'UPLOAD_DOC'
ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
dispute_id UUID REFERENCES disputes(id) ON DELETE SET NULL,
assigned_to_gtid TEXT NOT NULL,
assigned_by_gtid TEXT,
status task_status DEFAULT 'PENDING',
priority INTEGER DEFAULT 50,
due_date TIMESTAMPTZ,
escalation_level INTEGER DEFAULT 0,
escalation_history JSONB DEFAULT '[]',
completed_at TIMESTAMPTZ,
completed_by_gtid TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Feedback tickets

```

CREATE TABLE feedback_tickets (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
employee_id UUID NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
ticket_type TEXT NOT NULL, -- 'BUG', 'FEATURE', 'HELP'
title TEXT,
description TEXT NOT NULL,

```

```

severity TEXT DEFAULT 'MEDIUM', -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'
priority TEXT DEFAULT 'NICE_TO_HAVE', -- 'NICE_TO_HAVE', 'IMPORTANT', 'CRITICAL'
status TEXT DEFAULT 'OPEN', -- 'OPEN', 'IN_PROGRESS', 'RESOLVED', 'CLOSED'
page_url TEXT,
user_agent TEXT,
screenshot_hash TEXT,
assigned_to UUID REFERENCES employees(id) ON DELETE SET NULL,
resolved_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Notification log

```

CREATE TABLE notification_log (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
recipient_gtid TEXT NOT NULL,
category TEXT NOT NULL,
channel TEXT NOT NULL,
subject TEXT NOT NULL,
body TEXT,
status TEXT, -- 'SENT', 'FAILED', 'READ', 'CLICKED'
sent_at TIMESTAMPTZ DEFAULT NOW(),
read_at TIMESTAMPTZ,
clicked_at TIMESTAMPTZ
);

```

13.1.22 Suspicious Activity Reports (SAR)

sql

-- Suspicious Activity Reports

```

CREATE TABLE suspicious_activity_reports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
report_type TEXT NOT NULL, -- 'FinCEN', 'EG_AML', 'EU_ECB'
detection_rule TEXT, -- 'volume_spike', 'circular_trade', 'value_mismatch', etc.
involved_ustns TEXT[],
parties JSONB, -- { buyer_gtid, seller_gtid, carrier_gtid, ... }
narrative TEXT, -- Groq-generated plain-language description
draft_status TEXT DEFAULT 'DRAFT', -- 'DRAFT', 'FILED', 'REJECTED'
filed_at TIMESTAMPTZ,

```

```

filing_reference TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.23 GNN & Federated Learning

sql

-- GNN risk scores (adversarial)

```

CREATE TABLE gnn_risk_scores (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
entity_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
sanctions_proximity INTEGER,
graph_risk_score INTEGER,
model_version TEXT,
inference_at TIMESTAMPTZ,
explanation TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Institutional trade graph edges

```

CREATE TABLE trade_graph_edges (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
from_entity_anon_id TEXT NOT NULL,
to_entity_anon_id TEXT NOT NULL,
edge_type TEXT NOT NULL, -- 'TRADE', 'FINANCING', 'COMPLIANCE', 'SUPPLY'
weight DECIMAL(5,2),
first_seen TIMESTAMPTZ,
last_seen TIMESTAMPTZ,
zk_proof_hash TEXT,
consent_granted BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Network trust scores (materialised view)

```

CREATE MATERIALIZED VIEW network_trust_scores AS

```

```

SELECT
entity_gtid,
COUNT(DISTINCT to_entity_anon_id) AS direct_connections,
SUM(weight) AS weighted_trust_sum,
CURRENT_TIMESTAMP AS refreshed_at
FROM trade_graph_edges
GROUP BY entity_gtid;

```

-- Federated learning models

```

CREATE TABLE federated_learning_models (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
model_name TEXT NOT NULL,
version INTEGER,
global_model_hash TEXT,
aggregator_node_gtid TEXT,
round_number INTEGER,
accuracy_validation DECIMAL(5,4),
deployed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Local training metadata

```

CREATE TABLE local_training_metadata (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
node_gtid TEXT NOT NULL,
model_name TEXT NOT NULL,
round_number INTEGER,
local_accuracy DECIMAL(5,4),
data_size INTEGER,
training_duration_seconds INTEGER,
uploaded_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.24 Infrastructure & Security

sql

-- Infrastructure predictions

```

CREATE TABLE infrastructure_predictions (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
node_gtid TEXT NOT NULL,
component TEXT NOT NULL,
predicted_failure_probability DECIMAL(5,4),
estimated_failure_date TIMESTAMPTZ,
action_taken TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Chaos experiments
CREATE TABLE chaos_experiments (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
experiment_name TEXT NOT NULL,
namespace TEXT NOT NULL,
status TEXT, -- 'RUNNING', 'SUCCEEDED', 'FAILED'
started_at TIMESTAMPTZ,
finished_at TIMESTAMPTZ,
groq_summary TEXT,
logs_url TEXT,
created_by_gtid TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Threat findings (pentesting)
CREATE TABLE threat_findings (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tool_name TEXT NOT NULL,
severity TEXT NOT NULL, -- 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'
title TEXT NOT NULL,
description TEXT,
remediation TEXT,
cve_id TEXT,
affected_component TEXT,
finding_hash TEXT,
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```



```
);
-- Platform reserves (Proof of Reserves)
CREATE TABLE platform_reserves (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
reporting_date DATE NOT NULL,
phase TEXT CHECK (phase IN ('ATTESTATION', 'ZK_PROOF')),
asset_total DECIMAL(20,2),
liability_total DECIMAL(20,2),
ratio DECIMAL(6,2),
attestation_pdf_url TEXT,
zk_proof TEXT,
verified BOOLEAN DEFAULT false,
verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Reserve ratio history
CREATE TABLE reserve_ratio_history (
id BIGSERIAL PRIMARY KEY,
ratio DECIMAL(6,2) NOT NULL,
checked_at TIMESTAMPTZ DEFAULT NOW()
);
```

13.1.25 Special Rates & Configuration

sql

```
-- Special SGTX fee rates (Admin-controlled)
CREATE TABLE special_sgtx_fee_rates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
seller_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
buyer_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
rate DECIMAL(5,4) NOT NULL, -- 0.1% - 2.5%
effective_from TIMESTAMPTZ NOT NULL,
effective_to TIMESTAMPTZ,
reason TEXT NOT NULL,
status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'EXPIRED', 'REVOKED'
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
approved_by TEXT[], -- GTIDs of multisig members who approved
```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Configuration history (versioned)
CREATE TABLE configuration_history (
id BIGSERIAL PRIMARY KEY,
version INTEGER NOT NULL,
changed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
change_reason TEXT,
config_snapshot JSONB NOT NULL,
changed_at TIMESTAMPTZ DEFAULT NOW()
);

```

13.1.26 Indexes

sql

-- Identity indexes

```

CREATE INDEX idx_tenants_gtid ON tenants(gtid);
CREATE INDEX idx_tenants_type ON tenants(type);
CREATE INDEX idx_tenants_lifecycle_state ON tenants(lifecycle_state);
CREATE INDEX idx_employees_tenant ON employees(tenant_id);
CREATE INDEX idx_employees_email ON employees(email);
CREATE INDEX idx_employees_status ON employees(status);
CREATE INDEX idx_tenant_lifecycle_history_tenant ON tenant_lifecycle_history(tenant_id);
CREATE INDEX idx_tenant_lifecycle_history_state ON tenant_lifecycle_history(to_state);

```

-- Trust Passport indexes

```

CREATE INDEX idx_trust_passports_tenant ON trust_passports(tenant_gtid);
CREATE INDEX idx_trust_passport_shares_token ON trust_passport_shares(token);
CREATE INDEX idx_tri_history_tenant_date ON tri_history(tenant_gtid, calculated_at DESC);

```

-- Trade indexes

```

CREATE INDEX idx_trade_requests_buyer ON trade_requests(buyer_tenant_id);
CREATE INDEX idx_trade_requests_seller ON trade_requests(seller_tenant_id);
CREATE INDEX idx_trade_requests_status ON trade_requests(status);
CREATE INDEX idx_trade_requests_incoterm ON trade_requests(incoterm);
CREATE INDEX idx_draft_history_request ON draft_history(trade_request_id);
CREATE INDEX idx_partner_attributions_trade ON partner_lead_attributions(trade_request_id);

```

-- Shipment indexes

```

CREATE INDEX idx_shipments_ustn ON shipments(ustn);

```

```

CREATE INDEX idx_shipments_status ON shipments(status);
CREATE INDEX idx_shipments_contract ON shipments(contract_id);
CREATE INDEX idx_shipments_exporter ON shipments(exporter_gtid);
CREATE INDEX idx_shipments_importer ON shipments(importer_gtid);
CREATE INDEX idx_shipments_etd ON shipments(etd);
CREATE INDEX idx_shipments_eta ON shipments(eta);
CREATE INDEX idx_shipment_milestones_ustn ON shipment_milestones(ustn);
CREATE INDEX idx_shipment_milestones_milestone ON shipment_milestones(milestone);
-- Contract indexes
CREATE INDEX idx_contracts_trade ON contracts(trade_request_id);
CREATE INDEX idx_contracts_status ON contracts(status);
CREATE INDEX idx_contract_shipments_contract ON contract_shipments(contract_id);
CREATE INDEX idx_contract_shipments_ustn ON contract_shipments(ustn);
CREATE INDEX idx_contract_shipments_status ON contract_shipments(status);
CREATE INDEX idx_milestone_payment_schedules_contract ON
milestone_payment_schedules(contract_id);
-- Fee indexes
CREATE INDEX idx_fee_payment_requests_contract ON fee_payment_requests(contract_id);
CREATE INDEX idx_fee_payment_requests_shipment ON fee_payment_requests(contract_shipment_id);
CREATE INDEX idx_fee_payment_requests_tenant ON fee_payment_requests(tenant_id);
CREATE INDEX idx_fee_payment_requests_status ON fee_payment_requests(status);
CREATE INDEX idx_fee_payment_requests_deferred ON fee_payment_requests(deferred_status)
WHERE deferred = true;
CREATE INDEX idx_fee_locks_ustn ON fee_locks(ustn);
CREATE INDEX idx_fee_locks_status ON fee_locks(status);
CREATE INDEX idx_late_fee_events_request ON late_fee_events(fee_payment_requests_id);
-- Logistics indexes
CREATE INDEX idx_service_quotations_ustn ON service_quotations(ustn);
CREATE INDEX idx_service_quotations_provider ON service_quotations(provider_gtid);
CREATE INDEX idx_service_quotations_status ON service_quotations(status);
CREATE INDEX idx_ship_quote_requests_ustn ON ship_quote_requests(ustn);
CREATE INDEX idx_ship_quotes_request ON ship_quotes(request_id);
CREATE INDEX idx_clarification_requests_quotation ON clarification_requests(quotation_id);
CREATE INDEX idx_carrier_contracts_carrier ON carrier_contracts(carrier_gtid);
CREATE INDEX idx_carrier_contracts_seller ON carrier_contracts(seller_gtid);
-- QC indexes

```

```

CREATE INDEX idx_qc_jobs_ustn ON qc_jobs(ustn);
CREATE INDEX idx_qc_jobs_provider ON qc_jobs(provider_gtid);
CREATE INDEX idx_qc_jobs_status ON qc_jobs(status);
CREATE INDEX idx_inspection_logs_job ON inspection_logs(qc_job_id);
CREATE INDEX idx_re_inspection_requests_ustn ON re_inspection_requests(original_ustn);
-- Broker indexes
CREATE INDEX idx_broker_certifications_ustn ON broker_certifications(ustn);
CREATE INDEX idx_broker_physical_jobs_ustn ON broker_physical_jobs(ustn);
CREATE INDEX idx_broker_storage_ustn ON broker_storage(ustn);
CREATE INDEX idx_broker_storage_retention ON broker_storage(retention_expiry);
-- Financing indexes
CREATE INDEX idx_financing_requests_ustn ON financing_requests(ustn);
CREATE INDEX idx_financing_requests_status ON financing_requests(status);
CREATE INDEX idx_financing_offers_request ON financing_offers(financing_request_id);
CREATE INDEX idx_financing_agreements_request ON financing_agreements(financing_request_id);
CREATE
            INDEX
            idx_financing_repayments_agreement
            ON
financing_repayments(financing_agreement_id);
CREATE INDEX idx_financier_historical_data_financier ON financier_historical_data(financier_tenant_id);
-- Distressed indexes
CREATE INDEX idx_distressed_cargo_listings_active ON distressed_cargo_listings(status) WHERE
status = 'ACTIVE';
CREATE INDEX idx_distressed_cargo_listings_expiry ON distressed_cargo_listings(listing_expiry);
CREATE
            INDEX
            idx_distressed_cargo_listings_ustn
            ON
distressed_cargo_listings(original_shipment_ustn);
CREATE INDEX idx_distressed_cargo_offers_listing ON distressed_cargo_offers(listing_id);
CREATE INDEX idx_micro_contracts_ustn ON micro_contracts(micro_ustn);
-- Dispute indexes
CREATE INDEX idx_disputes_ustn ON disputes(ustn);
CREATE INDEX idx_disputes_status ON disputes(status);
CREATE INDEX idx_disputes_filing_party ON disputes(filing_party_gtid);
CREATE INDEX idx_disputes_category ON disputes(dispute_category);
CREATE INDEX idx_dispute_recommendations_dispute ON dispute_recommendations(dispute_id);
CREATE INDEX idx_evidence_packages_dispute ON evidence_packages(dispute_id);
CREATE INDEX idx_evidence_packages_token ON evidence_packages(verification_token);
CREATE INDEX idx_causal_attribution_entity ON causal_attribution(entity_id, entity_type);
CREATE INDEX idx_dispute_experts_dispute ON dispute_experts(dispute_id);

```

-- Governance indexes

```
CREATE INDEX idx_governor_decisions_created ON governor_decisions(created_at DESC);
CREATE INDEX idx_governor_decisions_actor ON governor_decisions(actor_gtid);
CREATE INDEX idx_governor_decisions_verdict ON governor_decisions(verdict);
CREATE INDEX idx_ai_inference_records_agent ON ai_inference_records(agent_name);
CREATE INDEX idx_ai_inference_records_created ON ai_inference_records(created_at DESC);
CREATE INDEX idx_audit_log_changed_by ON audit_log(changed_by);
CREATE INDEX idx_audit_log_changed_at ON audit_log(changed_at DESC);
CREATE INDEX idx_loom_verification_tokens_token ON loom_verification_tokens(token);
```

-- Inbox indexes

```
CREATE INDEX idx_inbox_items_employee_score ON inbox_items(employee_id, priority_score DESC);
CREATE INDEX idx_inbox_items_employee_ustn ON inbox_items(employee_id, ustn);
CREATE INDEX idx_inbox_items_employee_snoozed ON inbox_items(employee_id, snoozed_until)
WHERE snoozed_until IS NOT NULL;
CREATE INDEX idx_inbox_history_employee ON inbox_history(employee_id, timestamp DESC);
```

-- Documents indexes

```
CREATE INDEX idx_documents_ustn ON documents(ustn);
CREATE INDEX idx_documents_tenant ON documents(tenant_id);
CREATE INDEX idx_documents_type ON documents(document_type);
CREATE INDEX idx_documents_uploaded_at ON documents(uploaded_at DESC);
```

-- Settlement indexes

```
CREATE INDEX idx_settlement_instructions_ustn ON settlement_instructions(ustn);
CREATE INDEX idx_settlement_instructions_status ON settlement_instructions(status);
CREATE INDEX idx_settlement_confirmations_instruction ON settlement_confirmations(instruction_id);
CREATE INDEX idx_payment_attempts_fee ON payment_attempts(fee_payment_request_id);
CREATE INDEX idx_payment_attempts_psp ON payment_attempts(aggregator_id);
CREATE INDEX idx_psp_health_logs_aggregator ON psp_health_logs(aggregator_id);
```

-- Integration indexes

```
CREATE INDEX idx_integration_connector_logs_name ON integration_connector_logs(connector_name);
CREATE INDEX idx_integration_connector_logs_created ON integration_connector_logs(created_at
DESC);
CREATE INDEX idx_integration_connector_logs_idempotency ON
integration_connector_logs(idempotency_key);
CREATE INDEX idx_certificates_tenant ON certificates(tenant_gtid);
CREATE INDEX idx_certificates_valid_until ON certificates(valid_until);
```

-- Release authorisation indexes

```

CREATE INDEX idx_release_authorisations_ustn ON release_authorisations(ustn);
CREATE INDEX idx_release_authorisations_container ON release_authorisations(container);
CREATE INDEX idx_release_authorisations_status ON release_authorisations(status);
CREATE INDEX idx_release_authorisations_valid_until ON release_authorisations(valid_until);
CREATE          INDEX          idx_release_authorisations_authorisation_id          ON
release_authorisations(authorisation_id);
CREATE INDEX idx_gate_out_events_ustn ON gate_out_events(ustn);
CREATE INDEX idx_gate_out_events_authorisation ON gate_out_events(authorisation_id);
CREATE INDEX idx_release_overrides_token ON release_overrides(override_token);
-- RIA indexes
CREATE INDEX idx_treatment_requirements_hs ON treatment_requirements(hs_code);
CREATE INDEX idx_treatment_requirements_origin_dest ON treatment_requirements(origin_country,
destination_country);
CREATE INDEX idx_country_mrl_hs_dest ON country_mrl(hs_code, destination_country);
CREATE INDEX idx_commodity_packing_defaults_hs ON commodity_packing_defaults(hs_code);
CREATE          INDEX          idx_commodity_dynamic_schemas_cache_hs          ON
commodity_dynamic_schemas_cache(hs_code, origin_country, destination_country);
-- Saved contacts indexes
CREATE INDEX idx_tenant_contacts_tenant ON tenant_contacts(tenant_id);
CREATE INDEX idx_tenant_contacts_gtid ON tenant_contacts(contact_gtid);
CREATE INDEX idx_tenant_contacts_last_interaction ON tenant_contacts(last_interaction);
CREATE INDEX idx_tenant_contacts_is_blocked ON tenant_contacts(is_blocked) WHERE is_blocked =
true;
-- Task indexes
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to_gtid);
CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_due_date ON tasks(due_date);
CREATE INDEX idx_tasks_ustn ON tasks(ustn);
-- Contact indexes
CREATE INDEX idx_tenant_contacts_tenant ON tenant_contacts(tenant_id);
CREATE INDEX idx_tenant_contacts_gtid ON tenant_contacts(contact_gtid);
CREATE INDEX idx_tenant_contacts_last_interaction ON tenant_contacts(last_interaction);
-- User saved views indexes
CREATE INDEX idx_user_saved_views_user ON user_saved_views(user_employee_id);
-- Feedback indexes
CREATE INDEX idx_feedback_tickets_employee ON feedback_tickets(employee_id);
CREATE INDEX idx_feedback_tickets_status ON feedback_tickets(status);

```

-- Notification indexes

```
CREATE INDEX idx_notification_log_recipient ON notification_log(recipient_gtid);
```

```
CREATE INDEX idx_notification_log_sent_at ON notification_log(sent_at DESC);
```

13.1.27 Row-Level Security (RLS) Policies

sql

-- Enable RLS on all tables

```
ALTER TABLE tenants ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE trade_requests ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE shipments ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE contracts ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE dispute ENABLE ROW LEVEL SECURITY;
```

-- Tenants RLS

```
CREATE POLICY tenants_tenant_isolation ON tenants
```

```
USING (gtid = current_setting('app.current_tenant_gtid')::text OR
```

```
EXISTS (SELECT 1 FROM employees WHERE tenant_id = id AND employees.id =  
current_setting('app.current_employee_id')::uuid));
```

-- Employees RLS

```
CREATE POLICY employees_tenant_isolation ON employees
```

```
USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

-- Trade requests RLS

```
CREATE POLICY trade_requests_tenant_isolation ON trade_requests
```

```
USING (buyer_tenant_id = current_setting('app.current_tenant_id')::uuid OR
```

```
seller_tenant_id = current_setting('app.current_tenant_id')::uuid);
```

-- Shipments RLS

```
CREATE POLICY shipments_tenant_isolation ON shipments
```

```
USING (exporter_gtid = current_setting('app.current_tenant_gtid')::text OR
```

```
importer_gtid = current_setting('app.current_tenant_gtid')::text OR
```

```
EXISTS (SELECT 1 FROM service_quotations WHERE ustn = shipments.ustn AND provider_gtid =  
current_setting('app.current_tenant_gtid')::text));
```

-- Documents RLS

```
CREATE POLICY documents_tenant_isolation ON documents
```

```
USING (tenant_id = current_setting('app.current_tenant_id')::uuid OR
```

```
EXISTS (SELECT 1 FROM shipments WHERE ustn = documents.ustn AND
```

```
(exporter_gtid = current_setting('app.current_tenant_gtid')::text OR
```

```
importer_gtid = current_setting('app.current_tenant_gtid')::text));
```

```
-- Disputes RLS
```

```
CREATE POLICY disputes_tenant_isolation ON disputes
```

```
USING (filing_party_gtid = current_setting('app.current_tenant_gtid')::text OR
```

```
EXISTS (SELECT 1 FROM shipments WHERE ustn = disputes.ustn AND
```

```
(exporter_gtid = current_setting('app.current_tenant_gtid')::text OR
```

```
importer_gtid = current_setting('app.current_tenant_gtid')::text));
```

13.2 API Endpoint Index (OpenAPI 3.1)

The full OpenAPI specification is served at `/api/v1/openapi.json`. Below is a summary of all endpoints grouped by service, with method, path, description, and reference to the blueprint part. All endpoints require authentication (JWT) and are gated by the Governor.

13.2.1 Governor & Identity

13.2.2 Trade, Quote & Multi-Shipment

13.2.3 Packing & Containerisation

13.2.4 Contract & Fee

13.2.5 Settlement & Payment

13.2.6 QC & Conditional Pass

13.2.7 Distressed Cargo

13.2.8 Dispute & Third-Party Expert

13.2.9 Financing

13.2.10 Inbox & User Preferences

13.2.11 Search & Universal Command Center

13.2.12 Tenant Data Export

13.2.13 Marketplace Partner

13.2.14 KYB/KYC & Onboarding

13.2.15 Government

13.2.16 Admin

13.2.17 Container Release API

13.2.18 Voice & AI

13.3 TimescaleDB Hypertables

```
sql
```

```
-- Trade Memory Events (hypertable)
```

```
SELECT create_hypertable('trade_memory_events', 'occurred_week');
```

```
-- IoT sensor readings
```

```
CREATE TABLE iot_sensor_readings (
```

```
id BIGSERIAL,
```



```

ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
sensor_type TEXT NOT NULL,
value JSONB NOT NULL,
unit TEXT,
anomaly_flag BOOLEAN DEFAULT false,
recorded_at TIMESTAMPTZ NOT NULL
);
SELECT create_hypertable('iot_sensor_readings', 'recorded_at');
-- Audit log (partitioned by date, not TimescaleDB)
-- Already partitioned above
-- Compress old data (after 6 months)
ALTER TABLE trade_memory_events SET (
timescaledb.compress,
timescaledb.compress_segmentby = 'category, event_type'
);
ALTER TABLE iot_sensor_readings SET (
timescaledb.compress,
timescaledb.compress_segmentby = 'sensor_type'
);
END OF PART 13 — COMPLETE DATA MODEL & API ENDPOINT INDEX (v11.0 FULLY EXPANDED)
PART 14 — SECURITY & THREAT MODEL (STRIDE + MITRE ATT&CK)

```

14.0 Purpose & Design Philosophy

The SGTX platform handles sensitive trade data, financial transactions, and cryptographic identities across multiple jurisdictions. A robust security model is not optional — it is a constitutional requirement. This part defines the comprehensive threat model for the SGTX platform using industry-standard frameworks:

STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) — per component, per trust boundary.

MITRE ATT&CK — adversary tactics, techniques, and procedures (TTPs) relevant to trade platforms.

All mitigations are zero-cost, built on open-source tools (OPA, WasmEdge, Cilium, Falco, etc.) and free threat intelligence feeds. AI (Groq, Hugging Face) is used for advisory purposes — e.g., summarising threat findings, generating post-mortems, and suggesting remediation steps. Never autonomous blocking (A5 forbidden).

AI Integration in Part 14:

Key principles (non-marketplace, orchestration-only):

Zero-trust architecture — no implicit trust; every request is authenticated, authorised, and encrypted.

Defence in depth — multiple layers of control (network, application, data, constitutional).

Immutable audit — every security-relevant event is logged immutably via Loom.

Continuous verification — hourly Loom chain verification, weekly pentests, daily vulnerability scans.

No security-through-obscurity — all security controls are documented and verifiable.

14.1 Threat Model Methodology

14.1.1 Process

SGTX applies a continuous, automated threat modelling process:

14.1.2 Threat Modelling Tooling

14.1.3 Coverage of v11.0 New Features

The threat model covers all v11.0 additions:

14.2 STRIDE Analysis Summary

The following table analyses each major asset against the six STRIDE categories. Mitigations are linked to blueprint parts. New v11.0 components are marked with ★.

14.2.1 Example — Spoofing Mitigation for GTID

An attacker cannot impersonate a tenant because every request requires a JWT signed with the tenant's Ed25519 private key. ZITADEL WebAuthn enforces biometric liveness for high-value actions (e.g., contract.sign). The JWT is short-lived (15 minutes) and rotated via refresh token.

Attack Scenario: Attacker steals a JWT token.

Mitigation: JWT has short TTL (15 min). Refresh token requires biometric verification. All API calls validated against device registry (Part 1.14).

14.2.2 Example — Tampering Mitigation for Loom

An attacker who gains write access to the governor_decisions table cannot modify a past decision because the Loom hash chain would break. The hourly verifier (audit-chain-verifier) detects any inconsistency and triggers a P0 incident.

Attack Scenario: Attacker modifies a Governor decision.

Mitigation: Loom hash chain breaks. Hourly verifier detects mismatch → P0 incident → Governor blocks new decisions until restored from WAL.

14.2.3 Example — Information Disclosure Mitigation for RLS

Row-Level Security ensures that a tenant can only see their own data. The Governor sets app.current_tenant_gtid in the session context, and PostgreSQL RLS enforces it at the database level. Even if an attacker bypasses the application layer, they cannot access other tenants' data.

Attack Scenario: Attacker crafts SQL query directly against PostgreSQL.

Mitigation: PostgreSQL RLS denies access if app.current_tenant_gtid is not set or does not match. The Governor is the only service that can set this variable.

14.3 MITRE ATT&CK Coverage (High-Priority Tactics)

The following table maps SGTX components to MITRE ATT&CK tactics and techniques, with specific mitigations. The matrix is maintained as a JSON file in the repository and used by the threat-scanner to validate controls. New v11.0 techniques are marked with ★.

AI-Assisted Threat Intelligence (advisory):

The threat-intel-agent (Rig + HF NLP) scrapes free threat feeds (AlienVault OTX, MISP, CISA alerts) every 6 hours. When a new IoC (indicator of compromise) is detected, Groq generates a summary and a suggested mitigation. The summary appears in the Admin Portal's Smart Inbox (priority 60).

Example: "New CVE-2026-1234 affects PostgreSQL 18. Patch available. Apply within 7 days."

14.4 Attack Surface Inventory (v11.0)

Attack surface reduction:

No inbound ports open except 443 (API gateway) and 22 (SSH, bastion only). All internal traffic uses WireGuard or Cilium encryption.

No direct database access from the internet; all queries go through Governor or read-only replicas for analytics.

14.5 Continuous Security Controls & Verification

AI-Assisted Verification:

The audit-summarizer (Groq) produces a weekly plain-language report of all findings, categorised by severity and component. The report is sent to the Platform Governance Authority Smart Inbox.

Example: "3 new critical vulnerabilities discovered in dependency libcrypto — apply patches. 2 Loom chain verifications passed. 1 RLS policy violation (fixed)."

14.6 Incident Response & Forensics

14.6.1 Incident Severity Levels

14.6.2 Incident Response Steps (Automated with AI Assistance)

14.6.3 Example Incidents

P0 Incident — Loom Chain Breach:

Hourly verifier detects mismatch. Alert fires. Governor blocks new decisions.

Groq analysis: "Mismatch at block 1,234,567. Likely cause: manual edit of governor_decisionstable. Affected USTNs: list."

Incident team restores from WAL, restores chain. Post-mortem generated.

P1 Incident — Conditional QC Hold Deadline Missed:

Action plan deadline passes; hold remains active. workflow-recovery-service escalates.

Seller receives final warning. Groq summary: "Seller missed action plan deadline for USTN ..."

Admin Portal notified. Manual override required to resolve hold.

P1 Incident — Deferred Payment Guarantee Expiry:

Customs clearance not confirmed before guarantee expiry. Governor freezes shipment.

Payment service attempts to charge payer. Groq summary: "Deferred payment guarantee for USTN ... expired. Immediate payment required."

P1 Incident — ZK Proof Verification Failure:

Financier submits ZK reserve proof that fails verification.

Governor rejects bid. Groq summary: "Reserve proof verification failed. Possible: insufficient reserves or proof tampering."

Financier notified, can retry with corrected proof.

14.7 Data Protection & Privacy

14.7.1 Data Classification

14.7.2 Data Encryption

14.7.3 Key Management

14.8 Zero-Cost Security Toolchain

All security tools are open-source and self-hosted:

No billing details required for any security tool. The platform can operate fully air-gapped (except for threat feed updates, which can be mirrored internally).

14.9 Governance & Compliance Gates

14.10 Database Schema Additions (Part 14)

sql

-- Incidents

```
CREATE TABLE incidents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  severity TEXT NOT NULL, -- 'P0', 'P1', 'P2', 'P3'  
  title TEXT NOT NULL,  
  description TEXT,  
  status TEXT DEFAULT 'OPEN', -- 'OPEN', 'CONTAINED', 'RESOLVED', 'CLOSED'  
  started_at TIMESTAMPTZ,  
  resolved_at TIMESTAMPTZ,  
  root_cause TEXT,  
  groq_summary TEXT,  
  loom_hash TEXT,  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Incident events (audit trail for incident response)

```
CREATE TABLE incident_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  incident_id UUID NOT NULL REFERENCES incidents(id) ON DELETE CASCADE,  
  event_type TEXT NOT NULL, -- 'DETECTED', 'CONTAINED', 'ESCALATED', 'RESOLVED',  
  'POST_MORTEM'  
  actor_gtid TEXT,  
  details JSONB,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Threat findings (already created in Part 11, adding more fields)

```

ALTER TABLE threat_findings ADD COLUMN incident_id UUID REFERENCES incidents(id) ON DELETE
SET NULL;

ALTER TABLE threat_findings ADD COLUMN cvss_score DECIMAL(3,1);

ALTER TABLE threat_findings ADD COLUMN exploit_available BOOLEAN DEFAULT false;

-- Security events (for SIEM-like aggregation)

CREATE TABLE security_events (
id BIGSERIAL,

event_type TEXT NOT NULL, -- 'AUTH_FAILURE', 'RLS_VIOLATION', 'RATE_LIMIT_EXCEEDED',
'CERT_EXPIRY', 'SUSPICIOUS_ACTIVITY'

severity TEXT NOT NULL, -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'

actor_gtid TEXT,

ip_address INET,

details JSONB,

loom_hash TEXT,

created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (created_at);

SELECT create_hypertable('security_events', 'created_at');

-- Data classification audit

CREATE TABLE data_classification_audit (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

table_name TEXT NOT NULL,

column_name TEXT,

classification TEXT NOT NULL, -- 'PUBLIC', 'INTERNAL', 'CONFIDENTIAL', 'REGULATED'

justification TEXT,

reviewed_by UUID REFERENCES employees(id) ON DELETE SET NULL,

reviewed_at TIMESTAMPTZ DEFAULT NOW(),

expires_at TIMESTAMPTZ
);

-- Security compliance reports

CREATE TABLE security_compliance_reports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

report_type TEXT NOT NULL, -- 'WEEKLY_PENTEST', 'MONTHLY_COMPLIANCE',
'QUARTERLY_RISK'

report_period TEXT NOT NULL, -- '2026-W24', '2026-06', '2026-Q2'

summary TEXT, -- Groq-generated

finding_count INTEGER,

```

```

critical_count INTEGER,
high_count INTEGER,
medium_count INTEGER,
low_count INTEGER,
pdf_url TEXT,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_incidents_status ON incidents(status);
CREATE INDEX idx_incidents_severity ON incidents(severity);
CREATE INDEX idx_incidents_started ON incidents(started_at DESC);
CREATE INDEX idx_incident_events_incident ON incident_events(incident_id);
CREATE INDEX idx_security_events_type ON security_events(event_type);
CREATE INDEX idx_security_events_severity ON security_events(severity);
CREATE INDEX idx_security_events_created ON security_events(created_at DESC);
CREATE INDEX idx_threat_findings_incident ON threat_findings(incident_id);
CREATE INDEX idx_threat_findings_severity ON threat_findings(severity);

```

14.11 Implementation Checklist (Part 14)

14.11.1 Core Security Infrastructure

Deploy security monitoring stack (Prometheus, Grafana, Loki, Jaeger, Falco).

Configure Cilium network policies (allowlist only).

Enable pgaudit on PostgreSQL and ship logs to Loki.

Set up audit-chain-verifier as a cron job (hourly).

Deploy pentest-service with OWASP ZAP, nuclei, Trivy (staging environment).

14.11.2 Threat Intelligence & Anomaly Detection

Integrate threat intelligence feeds (AlienVault OTK, MISP) into threat-intel-agent.

Configure Prometheus Alertmanager to route alerts to Smart Inbox and on-call phone.

Deploy security-events table for SIEM-like aggregation.

Implement anomaly detection for security events (Isolation Forest, A2).

14.11.3 STRIDE & MITRE

Run initial STRIDE analysis (Rig agent) and baseline risk register.

Map controls to MITRE ATT&CK techniques.

Create threat findings database and reporting.

14.11.4 Incident Response

Test incident response playbook with a simulated Loom chain breach.

Implement incident lifecycle (detection → containment → analysis → eradication → recovery → post-mortem).

Add Groq post-mortem generation.

14.11.5 Data Protection

Classify all data (PUBLIC, INTERNAL, CONFIDENTIAL, REGULATED).

Implement field-level encryption for personal data.

Set up data retention and deletion policies (Part 18).

14.11.6 v11.0 Specific

Add tests for dual-mode context enforcement (OPA unit tests).

Add tests for conditional QC hold blocking settlement.

Add tests for deferred payment guarantee expiry handling.

Add ZK proof verification tests (Plonky3).

Add Trust Passport revocation test.

Add Takaful claim approval workflow tests.

14.11.7 Reporting & Compliance

Generate first weekly security report using Groq summariser.

Set up compliance report generation (weekly pentest, monthly compliance, quarterly risk).

Create Admin Portal security dashboard.

14.12 AI Authority Summary for Part 14

END OF PART 14 — SECURITY & THREAT MODEL (v11.0 FULLY EXPANDED)

PART 15 — SLA & UPTIME GUARANTEES

15.0 Purpose & Design Philosophy

This part defines the service level agreements (SLAs) that govern the SGTX platform's production environment. These commitments cover availability, latency, recovery time objectives (RTO), and recovery point objectives (RPO) for all critical services that directly affect a tenant's ability to initiate, track, settle, or manage trades, financing agreements, distressed cargo, or disputes. The SLAs are contractually binding under the Master Service Agreement and are monitored entirely by self-hosted, zero-cost open-source tools (Prometheus, Grafana, Loki, Blackbox exporter, custom sla-monitor in Rust). No external monitoring service or paid APM is used.

AI assistance (advisory only):

Key principles (non-marketplace, orchestration-only):

Transparency — All SLAs are published on a public status page (<https://status.sgtx.io>) and accessible via API.

Accountability — SLA credits are automatically calculated and applied to tenant statements; no manual claim required.

Zero-cost monitoring — All probes run on the same bare-metal infrastructure; no cloud dependencies.

Continuous improvement — AI-driven post-mortems and predictive scaling reduce downtime over time.

Scope: All SLAs apply to the production environment (minimum three sovereign nodes, active-active). Development and sandbox environments have no SLA. For multi-shipment contracts, each shipment's services are monitored independently; a delay in one shipment does not affect SLA for others. Conditional QC holds and deferred payment guarantees have their own SLAs defined in this part.

15.1 Core Platform SLAs (Production)

The following SLAs apply to each sovereign node region (e.g., Cairo, Dubai, Frankfurt). Overall availability is measured per region; tenants are routed to the nearest healthy region via Anycast DNS.

15.1.1 Core Services

15.1.2 v11.0 New Services

15.1.3 Definitions

15.1.4 Example — Governor Service Downtime Calculation

Month: 30 days = 43,200 minutes.

Planned maintenance: 120 minutes (announced, excluded from SLA).

Unplanned downtime: 40 minutes.

Availability = $(43,200 - 120 - 40) / (43,200 - 120) = 42,920 / 43,080 = 99.63\% \rightarrow$ below 99.95% $\rightarrow$ credit triggered.

15.2 Conditional QC & Deferred Payment SLAs (v11.0)

These new components have specific SLA requirements aligned with real-world trade processes.

15.2.1 Conditional QC Hold Resolution SLA

Example: Seller completes action plan at 14:00 UTC and clicks "Mark Action Completed". Inspector must verify within 1 hour (SLA for the platform to process the verification and remove hold). If verification is delayed beyond 1h due to platform error, credit applies.

Credit: 5% of monthly platform fee if breached.

15.2.2 Deferred Payment Guarantee Expiry SLA

Example: Guarantee expires at 23:59 UTC. System must detect expiry and notify payer by 00:05 UTC next day. If detection is delayed beyond 5 minutes, credit applies.

Credit: 5% of monthly platform fee if breached.

15.2.3 Accelerated Outreach Notification SLA

Credit: 2% of monthly platform fee if breached.

15.3 Predictive SLAs & AI-Assisted Forecasting (Advisory)

The platform uses AI models (Groq + HF local) to forecast potential SLA breaches before they occur. These forecasts are advisory — they never automatically throttle or block traffic.

15.3.1 Predictive Availability Model (A2, HF local LSTM)

Example Alert: "Predicted 35% chance of Governor service degradation in the next 24h due to planned cluster upgrade. Consider postponing or adding replicas."

15.3.2 Predictive Latency Model (A1, Groq)

Groq analyses current traffic patterns and resource headroom, then generates a plain-language report:

"Based on current load (120% of normal), p95 latency may exceed 3 seconds in 8 hours if no scaling action is taken. Recommended: add 2 replicas of trade-service." (Advisory — no auto-scaling unless

configured.)

15.3.3 SLA Credit Forecast (A1, Groq)

At the start of each month, Groq forecasts the likelihood of SLA breaches based on historical data and planned changes.

Example: "This month, estimated SLA breach probability: 2% for Governor, 1% for Payment Orchestrator. No action required."

15.4 Maintenance Windows

15.4.1 Standard Maintenance

15.4.2 Emergency Maintenance

Example — Emergency Maintenance due to Log4j-like vulnerability:

Zero-day disclosed at 10:00 UTC. Within 2 hours, SGTx patches the affected service. Notifications sent at 12:00 UTC. Maintenance window 14:00-15:00 UTC (1 hour). Not counted against SLA because it was a third-party zero-day and notice was given.

15.5 Status Page & Incident Response

15.5.1 Public Status Page

URL: <https://status.sgtx.io>

Technology: Built with cState (open-source) or Uptime (GitHub-based). Self-hosted on a separate lightweight VPS, independent of main SGTx nodes.

Displays:

Realtime component status: Operational, Degraded Performance, Partial Outage, Major Outage

Incident history for last 90 days, including post-mortem summaries (Groq-generated, reviewed by Platform Governance Authority)

Upcoming scheduled maintenance

Per-component uptime percentage for current and previous month

SLA credit calculator — tenants can estimate potential credits based on current month's uptime

Subscriptions: RSS, email, Slack webhook (optional). All free.

15.5.2 Incident Response Process

15.5.3 Incident Post-Mortem Example (Groq Generated)

text

Title: Governor Service Latency Spike — 2026-06-07

Duration: 22 minutes (03:12—03:34 UTC)

Impact: 12% of requests experienced p95 latency >800ms (up to 1.2s). No data loss.

SLA credit: 0.5% of monthly fee (breach of 99.95% target).

Root cause: Sudden traffic spike from a large exporter (100+ concurrent trade requests).

Governor replicas at 85% CPU.

Remediation: Increased replica count from 3 to 5; added HPA rule to scale at 70% CPU.

Prevention: Predictive auto-scaling model updated.

15.6 Credit & Compensation Model

15.6.1 Calculation

15.6.2 Automatic Application

sla-monitor runs at the end of each month, aggregates downtime per component, computes credits.

Credits are automatically applied to the tenant's next monthly statement (line item: "SLA credit — Governor breach").

Tenants receive a Smart Inbox notification with the breakdown.

No manual claim required — but tenants can dispute the calculation via the "SLA Dispute" button in Company Admin (dispute handled by Platform Governance Authority).

15.6.3 Exclusions (No Credit)

15.7 Zero-Cost Monitoring & Enforcement

15.7.1 Monitoring Stack

15.7.2 sla-monitor Implementation Details

15.7.3 AI-Assisted Health Prediction (Advisory)

The predictive-sla service (A2, LSTM) uses historical probe data to forecast future availability. Groq generates a monthly report:

"Based on current trends, Governor service availability next month is forecast at 99.97% (above target). No action needed."

15.8 Example SLA Incident Flow (Complete)

Scenario: Network partition between Cairo and Dubai nodes causes Governor service partially unavailable for EU users routed to Frankfurt. External probes detect 5xx errors for 12 consecutive minutes.

Step 1 — Detection (03:12 UTC)

External probe in London fails 4 of 5 requests to Governor on Frankfurt node. Alertmanager fires: "CRITICAL: Governor service — 80% failure rate on Frankfurt node for 5 min."

AIOps agent detects NATS connection flapping, cordons Frankfurt pods, redirects traffic to Dubai node. Service restored within 2 min (03:14 UTC).

Step 2 — Triage (03:14 UTC)

On-call engineer acknowledges alert via mobile app, investigates root cause (faulty switch). Total downtime: 12 min (0.028% of monthly minutes). 99.90% availability SLA for Governor (target 99.95%) breached.

Step 3 — Resolution & Post-mortem (03:30 UTC)

Network switch replaced. All traffic restored.

sla-monitor records downtime, computes credit. End of month, tenant statement shows credit of 10% of monthly fee (approx \$50 for a \$500 monthly fee).

Post-mortem generator (Groq) drafts summary:

"Governor service experienced 12-minute partial outage on Frankfurt node due to network hardware failure. Automatic failover restored service in 2 min. Root cause: faulty switch. Mitigation: additional network redundancy ordered. Credits issued to affected tenants: \$50 per tenant."

Step 4 — Tenant Notification

Affected tenants receive Smart Inbox item (priority 85): "On 14 Jun, Governor service was partially unavailable for 12 min. Your trade was not impacted. A credit of \$50 has been applied to your next statement. [View Incident Report]"

Step 5 — Follow-up

Platform Governance Authority reviews post-mortem, publishes to status page. No further action required.

15.9 Regional & Jurisdictional SLA Variations

15.9.1 Egypt (CBE Requirements)

15.9.2 EU (GDPR & CBAM)

15.9.3 UAE (DIFC)

15.10 Database Schema Additions (Part 15)

sql

-- SLA monitoring data

```
CREATE TABLE sla_metrics (  
  id BIGSERIAL PRIMARY KEY,  
  component TEXT NOT NULL,  
  region TEXT NOT NULL,  
  availability DECIMAL(6,4),  
  uptime_seconds BIGINT,  
  downtime_seconds BIGINT,  
  maintenance_excluded_seconds BIGINT,  
  target_availability DECIMAL(6,4),  
  sla_breached BOOLEAN DEFAULT false,  
  measurement_start TIMESTAMPTZ,  
  measurement_end TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- SLA credits applied

```
CREATE TABLE sla_credits (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  component TEXT NOT NULL,  
  breach_percentage DECIMAL(5,2),  
  credit_amount DECIMAL(20,4),  
  currency TEXT DEFAULT 'USD',  
  period_start TIMESTAMPTZ,  
  period_end TIMESTAMPTZ,
```

```
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'APPLIED', 'DISPUTED'
applied_at TIMESTAMPTZ,
disputed_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- SLA credit disputes

```
CREATE TABLE sla_disputes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
credit_id UUID NOT NULL REFERENCES sla_credits(id) ON DELETE CASCADE,
tenant_gtid TEXT NOT NULL,
dispute_reason TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'RESOLVED', 'REJECTED'
resolution TEXT,
resolved_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Predictive SLA forecasts

```
CREATE TABLE sla_forecasts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
component TEXT NOT NULL,
region TEXT NOT NULL,
forecast_date DATE NOT NULL,
predicted_availability DECIMAL(6,4),
confidence DECIMAL(5,2),
risk_factors JSONB,
recommendation TEXT, -- Groq-generated
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Maintenance windows

```
CREATE TABLE maintenance_windows (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
component TEXT NOT NULL,
```

```

region TEXT,
start_time TIMESTAMPTZ NOT NULL,
end_time TIMESTAMPTZ NOT NULL,
reason TEXT NOT NULL,
status TEXT DEFAULT 'SCHEDULED', -- 'SCHEDULED', 'IN_PROGRESS', 'COMPLETED',
'CANCELLED'
announced_at TIMESTAMPTZ,
approved_by UUID REFERENCES employees(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- SLA status page events
CREATE TABLE status_page_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
event_type TEXT NOT NULL, -- 'INCIDENT_CREATED', 'INCIDENT_UPDATED',
'MAINTENANCE_SCHEDULED', 'COMPONENT_STATUS_CHANGED'
component TEXT,
status TEXT, -- 'OPERATIONAL', 'DEGRADED', 'PARTIAL_OUTAGE', 'MAJOR_OUTAGE'
description TEXT,
started_at TIMESTAMPTZ,
resolved_at TIMESTAMPTZ,
published BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_sla_metrics_component ON sla_metrics(component);
CREATE INDEX idx_sla_metrics_period ON sla_metrics(measurement_start, measurement_end);
CREATE INDEX idx_sla_credits_tenant ON sla_credits(tenant_gtid);
CREATE INDEX idx_sla_credits_status ON sla_credits(status);
CREATE INDEX idx_sla_forecasts_component ON sla_forecasts(component);
CREATE INDEX idx_maintenance_windows_active ON maintenance_windows(start_time, end_time)
WHERE status != 'COMPLETED';
CREATE INDEX idx_status_page_events_created ON status_page_events(created_at DESC);

```

15.11 Implementation Checklist (Part 15)

15.11.1 Monitoring Infrastructure

Deploy sla-monitor (Rust) on a separate node.

Configure Blackbox exporter to probe all service health endpoints from ≥ 2 external locations.

Set up Prometheus Alertmanager rules for SLA breaches (e.g., `sgtx_availability < 0.9995`).

Integrate sla-monitor with ClickHouse for long-term storage.

Build SLA dashboard in Grafana (uptime trends, credit history, latency heatmaps).

15.11.2 Credit & Compensation

Implement credit calculation logic in payment-orchestrator (add line item to monthly statement).

Add Smart Inbox notification for credit issuance.

Implement SLA dispute workflow.

15.11.3 Status Page

Deploy status page (cState) and configure API to pull current status from Prometheus.

Set up incident post-mortem automation (Groq + template).

Configure RSS, email, Slack webhook subscriptions.

15.11.4 Predictive SLAs

Train predictive SLA model (LSTM) using historical probe data.

Implement predictive availability forecast.

Add Groq-generated SLA credit forecast (monthly).

15.11.5 Maintenance Windows

Implement maintenance window scheduling.

Add announcement mechanism (Smart Inbox, email, status page).

Exclude maintenance windows from availability calculations.

15.11.6 v11.0 Specific

Add SLA monitoring for conditional QC holds (track hold removal latency).

Add SLA monitoring for deferred payment guarantee expiry detection.

Add alerting for multi-shipment per-shipment coordinator failures (separate metrics per shipment).

Add SLA monitoring for Takaful fund claim processing.

Add SLA monitoring for Trust Passport verification.

15.11.7 Testing

Test incident response with simulated network partition (chaos engineering).

Test SLA credit calculation and application.

Test SLA dispute workflow.

Test status page integration.

Test maintenance window exclusion.

15.12 AI Authority Summary for Part 15

END OF PART 15 — SLA & UPTIME GUARANTEES (v11.0 FULLY EXPANDED)

PART 16 — GLOSSARY & ACRONYMS

16.0 Purpose & Design Philosophy

This glossary defines every key term, acronym, and concept used throughout the SGTX blueprint. It is a living document, updated with each release. Terms are presented in alphabetical order for easy reference. New v11.0 terms are marked with ★ .

The glossary serves multiple audiences:

New users — quick reference for unfamiliar terms.

Developers — consistent terminology across code, documentation, and APIs.

Compliance officers — clear definitions for regulatory filings.

Implementers — accurate understanding of system components.

Structure: Each term includes:

Definition — concise, plain-language explanation.

Example / Context — how the term is used in SGTX.

Cross-reference — related parts of the blueprint.

16.1 Core Terms (A–Z)

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

Y

Z

16.2 Acronyms (Selected)

END OF PART 16 — GLOSSARY & ACRONYMS (v11.0 FULLY EXPANDED)

PART 17 — IMPLEMENTATION ROADMAP & NEXT STEPS

17.0 Purpose & Design Philosophy

This part translates the entire SGTX blueprint into a practical, time-bound, risk-managed implementation plan. The roadmap is organised into five phases (0 through 4), each with clear objectives, deliverables, success criteria (KPIs), dependencies, and estimated duration. The plan is designed to deliver early value (Phase 1 — agricultural exports) while progressively adding complexity (multi-shipment, financing, imports, add-ons). All phases assume a team of 8–12 engineers (backend, frontend, DevOps, QA) and a parallel team for legal/compliance.

Key principles:

Demonstrate then mandate — pilot with real exporters before full government mandate.

Zero-cost infrastructure from day one — all phases use the same self-hosted, open-source stack.

Regulatory alignment — engage CBE, Customs, and Ministry of Trade early.

Continuous testing & security — each phase includes integration tests, security scans, and load testing.

AI-assisted project management — use Groq to generate progress reports, risk assessments, and post-mortems (advisory only).

AI Integration in Part 17:

17.1 Phase 0 — Foundation & Pilot Preparation (Months 1–3)

17.1.1 Objective

Establish the legal entity, core infrastructure, and a single pilot exporter (e.g., frozen strawberries) to prove the concept end-to-end with real documents and payments.

17.1.2 Key Deliverables

17.1.3 v11.0 Features NOT in Phase 0

Multi-shipment contracts

Non-uniform layer stacking

Mode B/C logistics

Conditional QC

Deferred payment

Milestone-based settlement

ZK proofs

Distressed cargo

Trade Memory Layer

Predictive Insights

Takaful Fund

17.1.4 Dependencies

17.1.5 KPIs

17.1.6 Timeline (Weeks)

AI Assistance: Use Groq to generate weekly status reports for stakeholders, summarising progress and risks.

17.2 Phase 1 — Agricultural Exports MVP (Months 4–9)

17.2.1 Objective

Launch a production-ready platform for agricultural exports (HS 06–11) for a limited set of corridors (Egypt → EU, Egypt → UAE). Mandate not yet enforced; voluntary adoption by up to 50 exporters.

17.2.2 Key Deliverables (v11.0 Core Features)

17.2.3 v11.0 Features NOT in Phase 1

Multi-shipment contracts

Mode B/C logistics

Conditional QC

Deferred payment

Milestone-based settlement

ZK proofs

Accelerated distressed outreach

Third-party expert

Causal inference

Trade Memory Layer

Predictive Insights

Takaful Fund

17.2.4 Dependencies

17.2.5 KPIs

17.2.6 Timeline (Months)

AI Assistance: Use Groq to generate Smart Inbox titles, tenant messages, and dispute mediation suggestions (advisory). Use HF local for document extraction during onboarding.

17.3 Phase 2 — Multi-Shipment, Financing & Advanced Logistics (Months 10–18)

17.3.1 Objective

Add multi-shipment contracts, trade finance (co-financing, financier portal), advanced logistics modes (B and C), non-uniform layer stacking, conditional QC, distressed cargo full workflow, and milestone-based partial settlement. Expand to more corridors and additional PSPs.

17.3.2 Key Deliverables (v11.0 Major Features)

17.3.3 Dependencies

17.3.4 KPIs

17.3.5 Timeline (Months)

AI Assistance: Causal inference engine (DoWhy + EconML) integrated into dispute workflow; GNN risk engine (add-on 1) trained and deployed.

17.4 Phase 3 — Imports, DeFi & Full Add-ons (Months 19–30)

17.4.1 Objective

Launch import functionality (Form 4, duties, local payment batch), DeFi financing, and the seven critical add-ons (GNN, federated learning, self-healing, pentesting, PQC, expanded ZK). Achieve full national coverage (all commodities, all ports) and prepare for mandatory decree.

17.4.2 Key Deliverables

17.4.3 Dependencies

17.4.4 KPIs

17.4.5 Timeline (Months)

AI Assistance: Federated learning coordinator; GNN model retrained weekly; ZK proof generation client-side (WASM).

17.5 Phase 4 — Global Expansion & Continuous Evolution (Years 3–5)

17.5.1 Objective

Expand SGTx to partner countries (first: UAE, then Germany, Vietnam, etc.), deploy sovereign nodes in each region, and establish mutual recognition of USTNs. Evolve the platform based on market feedback and new regulations.

17.5.2 Key Deliverables

17.5.3 Dependencies

17.5.4 KPIs

17.5.5 Timeline

AI Assistance: Federated learning across international nodes; predictive scaling for new regions.

17.6 Resource Requirements (Team Composition)

17.6.1 Team Size by Phase

17.6.2 Infrastructure Costs

17.7 Risk Mitigation Table

17.8 Success Metrics Dashboard (Living)

17.8.1 Adoption Metrics

17.8.2 Reliability Metrics

17.8.3 Efficiency Metrics

17.8.4 Finance Metrics

17.8.5 Quality Metrics

17.8.6 Security Metrics

17.9 Next Steps for the Founding Team

17.9.1 Immediate (First 30 Days)

17.9.2 Short-Term (Months 1–3)

17.9.3 Medium-Term (Months 4–9)

17.9.4 Long-Term (Year 2+)

17.10 Conclusion — The SGTX Imperative

SGTX is not a software project; it is a national infrastructure that will transform Egyptian trade. By following this roadmap, you will build a zero-cost, non-custodial, AI-orchestrated platform that:

Eliminates trade-based money laundering.

Reduces demurrage and document discrepancies.

Provides instant financing to SMEs.

Gives the government real-time, auditable trade visibility.

Supports multi-shipment contracts, non-uniform packing, conditional QC, deferred payments, milestone-based settlement, and privacy-preserving ZK proofs.

The blueprint is complete. The technology is ready. The team is waiting.

Let's build SGTX. Now.

END OF PART 17 — IMPLEMENTATION ROADMAP & NEXT STEPS (v11.0 FULLY EXPANDED)

PART 18 — EGYPTIAN PDPL COMPLIANCE FRAMEWORK

18.0 Purpose & Design Philosophy

This part defines the Egyptian Personal Data Protection Law (Law No. 151 of 2020) compliance framework for the SGTX platform. The PDPL is the primary data protection legislation in Egypt, regulating the collection, processing, storage, and transfer of personal data. Compliance is mandatory for any entity processing personal data of Egyptian residents or operating within Egypt.

SGTX processes personal data of its tenants, employees, and counterparties. The platform must therefore implement comprehensive technical and organisational measures to ensure compliance with PDPL and its implementing regulations.

Key principles of PDPL (Articles 1–3):

Lawfulness, fairness, and transparency — Personal data must be processed lawfully, fairly, and transparently.

Purpose limitation — Personal data must be collected for specified, explicit, and legitimate purposes.

Data minimisation — Personal data must be adequate, relevant, and limited to what is necessary.

Accuracy — Personal data must be accurate and kept up to date.

Storage limitation — Personal data must be kept in a form which permits identification of data subjects for no longer than necessary.

Integrity and confidentiality — Personal data must be processed in a manner that ensures appropriate security.

PDPL Enforcement: The Egyptian Data Protection Centre (DPC) is the supervisory authority responsible for enforcing the PDPL. Non-compliance can result in fines of up to EGP 5,000,000 (approx. \$160,000) and criminal penalties under Articles 43–45.

AI Integration in Part 18:

18.1 Scope of Personal Data in SGTX

18.1.1 Data Categories

The following table categorises all personal data processed by the SGTX platform:

18.1.2 Data Subjects

18.2 Data Subject Rights Implementation

18.2.1 Rights Overview (PDPL Article 9)

The PDPL grants data subjects the following rights, which SGTX must implement:

18.2.2 Data Subject Request Workflow

text

■ DATA SUBJECT REQUEST WORKFLOW ■

■ ■

■ Step 1: User submits request via Privacy Dashboard or email (dpo@sgtx.io) ■

■ ■ ■

■ Step 2: System logs request in `dsr\_requests` table (automated) ■

■ ■ ■

■ Step 3: DPO reviews request (24h SLA) ■

■ ■ ■

□ □ □ □ □

■ ▼ ▼ ▼ ■

■ Step 4a: Step 4b: Step 4c: ■

■ Approve Partially Reject with ■

■ (immediate) approve reason ■

■ (additional ■

■ verification) ■

■ ■ ■

13

■ Step 5: System executes request (data export, rectification, deletion, etc.) ■

■ ■ ■

13

■ Step 6: User notified of completion (Smart Inbox + email) ■

■ ■



One-click guarantee: Each DSR step has a single action button. The DPO can approve, partially approve, or reject with one click.

18.3 Consent Management

18.3.1 Consent Purposes (PDPL Article 13)

The following table lists all consent purposes in SGTX, with defaults and withdrawal options:

18.3.2 Consent Registry Database

sql

-- Consent records (immutable, Loom-hashed)

```
CREATE TABLE consent_records (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_gtid TEXT NOT NULL,  
  purpose TEXT NOT NULL, -- 'marketing', 'analytics', 'govt_sharing', 'cross_border', 'voice_biometric',  
  'location_tracking', 'trade_memory'  
  version TEXT NOT NULL, -- consent form version  
  granted BOOLEAN NOT NULL,  
  ip_address INET NOT NULL,  
  user_agent TEXT,  
  device_id TEXT,  
  timestamp TIMESTAMPTZ DEFAULT NOW(),  
  withdrawal_timestamp TIMESTAMPTZ,  
  loom_hash TEXT, -- anchored to audit chain  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL  
);
```

-- Consent version control

```
CREATE TABLE consent_versions (  
  version TEXT PRIMARY KEY,  
  purpose TEXT NOT NULL,  
  text_ar TEXT NOT NULL,  
  text_en TEXT NOT NULL,  
  effective_date DATE NOT NULL,  
  superseded_by_version TEXT,  
  loom_hash TEXT  
);
```

18.3.3 Consent UI (Privacy Dashboard)

PDPL Article 14 restricts the transfer of personal data outside Egypt. Transfers are only permitted if:
Adequacy decision by the Egyptian government (none issued yet), OR
Appropriate safeguards (Standard Contractual Clauses, binding corporate rules, derogations), AND
Data subject explicit consent.

18.4.2 SGTX Cross-Border Data Flows

18.4.3 Standard Contractual Clauses (SCCs)

SGTX automatically generates SCCs (based on EU 2021/914) when transferring data to non-adequate jurisdictions:

Controller-to-Controller (Module 1) — for trade data between Egyptian and foreign counterparties.

Controller-to-Processor (Module 2) — for processing by foreign PSPs or technology partners (none currently).

Processor-to-Processor (Module 3) — not applicable.

SCC Generation Workflow:

Tenant initiates cross-border trade (or Trust Passport sharing, or financing).

System checks if destination jurisdiction has adequacy decision (updated by RIA).

If not, system auto-generates SCCs with:

Parties (exporter and importer GTIDs)

Data categories (trade data, financial data, etc.)

Purpose (trade execution, financing, etc.)

Duration (until contract completion + 5 years)

Tenant reviews and signs SCCs via ZITADEL passkey (one click).

SCCs stored in documents with Loom hash.

18.4.4 Cross-Border Data Transfer Gate (Governor)

The Governor enforces cross-border data transfer restrictions:

rego

```
# policies/cross_border.rego
```

```
package sgtx.cross_border
```

```
default allow = false
```

```
allow {
```

```
# Destination has adequacy decision
```

```
adequacy[destination_country] == true
```

```
}
```

```
allow {
```

```
# Explicit consent for cross-border transfer
```

```
has_consent(input.actor_gtid, "cross_border")
```

```
# And SCCs are signed (if no adequacy)
```


WHERE withdrawal_timestamp IS NOT NULL


```

"purposes": ["Hands-free operation for milestone confirmations"],
"legal_bases": ["Explicit consent (Art. 13)"],
"risks": [
{
"risk": "Unauthorised access to voice recordings",
"mitigation": "On-device processing; no storage; raw audio never transmitted",
"residual_risk": "LOW"
}
],
"dpia_approved": true,
"approved_by": "DPO",
"approval_date": "2026-01-15",
"next_review": "2027-01-15",
"loom_hash": "sha256:..."
}

```

18.10 Data Protection Compliance Dashboard (Government Portal)

18.10.1 Purpose

Provide government officials with a view of platform-wide data protection compliance metrics.

Location: Government Portal → Compliance Monitor → "Data Protection" tab

18.10.2 Dashboard Metrics

18.10.3 One-Click Compliance Report

Action: Click "Generate PDPL Compliance Report" → PDF with all metrics, signed by DPO.

Report contents:

Summary of data processing activities

Consent record status

Cross-border transfer log

Data subject request statistics

Breach notification log

DPIA status

Retention compliance status

18.11 Database Schema Additions (Part 18)

```
sql
```

```
-- Data Subject Requests (DSRs)
```

```
CREATE TABLE dsr_requests (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

requester_gtid TEXT NOT NULL,
request_type TEXT NOT NULL, -- 'ACCESS', 'RECTIFICATION', 'ERASURE', 'RESTRICTION',
'PORTABILITY', 'OBJECTION'
details TEXT,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'APPROVED', 'PARTIALLY_APPROVED',
'REJECTED', 'COMPLETED'
dpo_review TEXT,
completed_at TIMESTAMPTZ,
data_export_url TEXT,
rectification_details JSONB,
deletion_details JSONB,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Data Protection Impact Assessments (DPIAs)
CREATE TABLE dpia_records (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
processing_activity TEXT NOT NULL,
risk_level TEXT NOT NULL, -- 'LOW', 'MEDIUM', 'HIGH'
data_categories TEXT[],
purposes TEXT[],
legal_bases TEXT[],
risk_assessment JSONB,
mitigation_measures JSONB,
residual_risk TEXT,
status TEXT DEFAULT 'DRAFT', -- 'DRAFT', 'IN_REVIEW', 'APPROVED', 'REJECTED'
approved_by UUID REFERENCES employees(id) ON DELETE SET NULL,
approval_date TIMESTAMPTZ,
next_review TIMESTAMPTZ,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Data Protection Compliance Reports
CREATE TABLE dp_compliance_reports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
report_period TEXT NOT NULL, -- '2026-Q2'

```

```

report_date DATE NOT NULL,
report_content JSONB,
pdf_url TEXT,
signed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Data breach notifications
CREATE TABLE data_breach_notifications (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
incident_id UUID REFERENCES incidents(id) ON DELETE SET NULL,
dpo_gtid TEXT NOT NULL,
dpc_notified BOOLEAN DEFAULT false,
dpc_notification_date TIMESTAMPTZ,
dpc_reference TEXT,
affected_users TEXT[],
notification_text TEXT, -- Groq-generated
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Data retention jobs log
CREATE TABLE data_retention_jobs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
job_type TEXT NOT NULL, -- 'CONSENT_DELETION', 'ACCOUNT_DELETION', 'AUDIT_DELETION'
records_deleted INTEGER,
affected_table TEXT,
deletion_criteria TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
loom_hash TEXT,
executed_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_dsr_requests_requester ON dsr_requests(requester_gtid);
CREATE INDEX idx_dsr_requests_status ON dsr_requests(status);
CREATE INDEX idx_dpia_records_activity ON dpia_records(processing_activity);

```

CREATE INDEX idx_dpia_records_status ON dpia_records(status);

CREATE INDEX idx_data_breach_notifications_incident ON data_breach_notifications(incident_id);

CREATE INDEX idx_data_retention_jobs_executed ON data_retention_jobs(executed_at DESC);

18.12 Implementation Checklist (Part 18)

18.12.1 Legal & Governance

Designate DPO and register with Egyptian Data Protection Centre.

Draft and publish Privacy Notice (v1.0) with all required sections.

Implement consent management with granular checkboxes.

Set up DSR handling workflow (access, rectification, erasure, etc.).

Implement SCC generation for cross-border transfers.

18.12.2 Technical Controls

Create consent_records table with Loom hashing.

Implement consent gate in Governor (policies/cross_border.rego).

Build Privacy Dashboard UI (Company Admin → Privacy).

Implement data retention automation (data-retention-service).

Set up breach detection and notification workflow.

Build DPO interface in Admin Portal.

18.12.3 Reporting

Implement DPIA management dashboard.

Build PDPL compliance report generator (PDF).

Add Data Protection tab to Government Portal.

Set up automated deletion cron job.

18.12.4 Training & Awareness

Develop PDPL training materials for all SGTX employees.

Mandatory annual PDPL training for employees processing personal data.

Document data processing activities (Article 24).

18.13 AI Authority Summary for Part 18

END OF PART 18 — EGYPTIAN PDPL COMPLIANCE FRAMEWORK (v11.0 FULLY EXPANDED)

PART 19 — TRADE MEMORY LAYER & PREDICTIVE TRADE INSIGHTS

19.0 Purpose & Design Philosophy

The Trade Memory Layer is the long-term, structured knowledge base of every trade event that occurs on the SGTX platform. Unlike raw audit logs (which are technical and access-controlled), Trade Memory captures semantic, anonymised facts about delays, disputes, documentation failures, customs issues, financing outcomes, and logistics performance. This data is used exclusively for:

Pattern detection — identifying recurring failure modes and bottlenecks.

Predictive analytics — forecasting risks and outcomes for active trades.

AI model training (federated learning) — improving fraud detection, credit scoring, and margin estimation without exposing raw data.

Strategic advisory — providing anonymised benchmarks and "what-if" insights to platform governance and participating tenants (with consent).

All stored events are anonymised before retention: original GTIDs are replaced with persistent, non-reversible anonymous IDs. Raw event data remains in the audit log (subject to Part 18 retention rules). The Trade Memory Layer is never used to recommend counterparties, rank service providers, or violate non-marketplace principles.

Key principles (non-marketplace, sovereign):

Anonymisation first — no personal or commercial data is stored in raw form.

Opt-in contribution — tenants control whether their data contributes to the memory layer.

Privacy-preserving analytics — all aggregates use differential privacy ($\epsilon=0.1$).

Federated learning ready — data can be used for cross-node model training without sharing raw events.

No unsolicited recommendations — insights are advisory only and never used to suggest counterparties.

AI Integration in Part 19:

19.1 Event Types Captured

19.1.1 Event Categories

Every significant workflow event is automatically captured and transformed into a standardised memory event. The following table lists the core event types, their sources, and the fields stored (after anonymisation).

Trade Events:

Delay Events:

Documentation Events:

Dispute Events:

Financing Events:

Logistics Events:

Compliance Events:

19.1.2 Event Storage Schema

sql

-- Trade Memory Events (hypertable)

CREATE TABLE trade_memory_events (

id BIGSERIAL,

event_type TEXT NOT NULL,

category TEXT NOT NULL, -- 'TRADE', 'DELAY', 'DOCUMENTATION', 'DISPUTE', 'FINANCING',
'LOGISTICS', 'COMPLIANCE'

anonymous_entity_id TEXT, -- persistent but non-reversible

hs_chapter CHAR(2),

origin_country CHAR(2),

■ ■ Contribute anonymised trade data to improve platform intelligence ■

■ Your data will be anonymised and used for: ■

- 11

■ • Your identity (GTID is replaced with anonymous ID) ■

-

■ You can opt out at any time. Historical data already anonymised ■

■ cannot be retroactively deleted. ■

■ [Save] [Learn More] ■

19.3 Storage & Querying

19.3.1 TimescaleDB Hypertable

sql

```
-- Create hypertable for time-series storage
```

```
SELECT create_hypertable('trade_memory_events', 'occurred_week');
```

- Compress old data (after 6 months)

```
ALTER TABLE trade_memory_events SET (
```

timescaledb.compress,

```
timescaledb.compress segmentby = 'category, event type'
```

$$);$$

- Indexes for fast querying

```
CREATE INDEX idx_trade_memory_events_category ON trade_memory_events(category);
```

```
CREATE INDEX idx_trade_memory_events_event_type ON trade_memory_events(event_type);
CREATE INDEX idx_trade_memory_events_occurred_week ON trade_memory_events(occurred_week);
CREATE INDEX idx_trade_memory_events_origin ON trade_memory_events(origin_country);
CREATE INDEX idx_trade_memory_events_destination ON trade_memory_events(destination_country);
CREATE INDEX idx_trade_memory_events_hs ON trade_memory_events(hs_chapter);
```

19.3.2 Query Interface (Internal Only)

No external API is exposed. Internal services (AI Risk Engine, Predictive Insights, Federated Learning) query the memory layer via a gRPC interface with strict rate limits and audit logging.

Example Internal Query (Rust):

```
rust
let delays = memory_store.query(Query::new()
    .category("DELAY")
    .origin("EG")
    .destination("DE")
    .hs_chapter("08")
    .weeks_last(12)
    .aggregate(Aggregate::Percentile("delay_days", 95))
).await?;
```

19.3.3 Query Rate Limits

All queries are logged in audit_log with purpose (e.g., 'predictive_insight', 'model_training').

19.4 Predictive Trade Insights (Generated Daily)

19.4.1 Overview

Every night at 02:00 UTC, the predictive-insights service runs a set of models trained on the Trade Memory Layer. Outputs are stored in predictive_insights and pushed to relevant Smart Inboxes.

19.4.2 Insight Types & Models

19.4.3 Output Storage

```
sql
CREATE TABLE predictive_insights (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    insight_type TEXT NOT NULL,
    target_entity TEXT, -- USTN, GTID, or NULL for general
    probability DECIMAL(5,2),
    confidence_interval JSONB,
    recommendation TEXT, -- plain-language suggestion (Groq, A1)
    payload JSONB,
    valid_until TIMESTAMPTZ,
```

```
acknowledged BOOLEAN DEFAULT false,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

19.4.4 Insight Delivery

Insights are delivered via:

19.4.5 One-Click Actions

Each insight may include actionable buttons:

19.5 Anomaly Detection & Alerts

19.5.1 Anomaly Detection Model (A2, Isolation Forest)

The anomaly-detector service runs every 6 hours, analysing:

19.5.2 Alert Delivery

Example Alert (Groq-Generated):

text

■■ ANOMALY DETECTED — Dispute surge in citrus corridor

Corridor: Egypt → Germany

Commodity: Citrus (HS 0805)

Anomaly: 3x increase in quality disputes over 7 days (12 disputes vs expected 4)

Category: 70% mould, 30% weight shortage

Root cause analysis pending.

[Investigate] [Dismiss]

19.6 Federated Learning Integration

19.6.1 Purpose

The Trade Memory Layer provides anonymised training data for federated learning (Add-on 2). Each sovereign node trains local models on its own memory events, and only encrypted model updates are shared.

19.6.2 Models Trained on Trade Memory

19.6.3 Privacy Guarantee

19.7 Governance & Non-Marketplace Safeguards

19.8 Integration with Other Parts

19.9 Database Schema Additions (Part 19)

sql

-- Trade Memory Events (hypertable)

```
CREATE TABLE trade_memory_events (  
  id BIGSERIAL,  
  event_type TEXT NOT NULL,
```

```

category TEXT NOT NULL, -- 'TRADE', 'DELAY', 'DOCUMENTATION', 'DISPUTE', 'FINANCING',
'LOGISTICS', 'COMPLIANCE'
anonymous_entity_id TEXT,
hs_chapter CHAR(2),
origin_country CHAR(2),
destination_country CHAR(2),
incoterm TEXT,
volume_band TEXT, -- 'SMALL', 'MEDIUM', 'LARGE'
value_band TEXT, -- 'LOW', 'MEDIUM', 'HIGH'
delay_days INTEGER,
dispute_category TEXT,
resolution_days INTEGER,
payload JSONB,
occurred_week DATE,
created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (occurred_week);
SELECT create_hypertable('trade_memory_events', 'occurred_week');
ALTER TABLE trade_memory_events SET (
timescaledb.compress,
timescaledb.compress_segmentby = 'category, event_type'
);
-- Predictive Insights
CREATE TABLE predictive_insights (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
insight_type TEXT NOT NULL,
target_entity TEXT, -- USTN, GTID, or NULL for general
probability DECIMAL(5,2),
confidence_interval JSONB,
recommendation TEXT, -- plain-language suggestion (Groq, A1)
payload JSONB,
valid_until TIMESTAMPTZ,
acknowledged BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Anomaly Detection Logs
CREATE TABLE anomaly_detection_logs (

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
anomaly_type TEXT NOT NULL,
severity TEXT NOT NULL, -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'
corridor TEXT,
commodity_hs CHAR(2),
detected_value DECIMAL(10,2),
expected_value DECIMAL(10,2),
details JSONB,
groq_summary TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Model metadata for predictive insights
CREATE TABLE predictive_models (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
model_name TEXT NOT NULL UNIQUE,
model_type TEXT NOT NULL, -- 'XGBOOST', 'LIGHTGBM', 'LSTM', 'PROPHET', 'RANDOM_FOREST'
version TEXT NOT NULL,
training_data_range_start DATE,
training_data_range_end DATE,
accuracy DECIMAL(5,4),
features TEXT[],
deployed_at TIMESTAMPTZ,
retired_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_trade_memory_events_category ON trade_memory_events(category);
CREATE INDEX idx_trade_memory_events_event_type ON trade_memory_events(event_type);
CREATE INDEX idx_trade_memory_events_occurred_week ON trade_memory_events(occurred_week);
CREATE INDEX idx_trade_memory_events_origin ON trade_memory_events(origin_country);
CREATE INDEX idx_trade_memory_events_destination ON trade_memory_events(destination_country);
CREATE INDEX idx_trade_memory_events_hs ON trade_memory_events(hs_chapter);
CREATE INDEX idx_predictive_insights_type ON predictive_insights(insight_type);
CREATE INDEX idx_predictive_insights_entity ON predictive_insights(target_entity);

```

```
CREATE INDEX idx_predictive_insights_valid_until ON predictive_insights(valid_until);  
CREATE INDEX idx_anomaly_detection_logs_created ON anomaly_detection_logs(created_at DESC);  
CREATE INDEX idx_anomaly_detection_logs_severity ON anomaly_detection_logs(severity);
```

19.10 Implementation Checklist (Part 19)

19.10.1 Event Capture

Create trade_memory_events hypertable (TimescaleDB).

Implement event capture hooks in all workflow services (trade, dispute, financing, logistics, QC).

Add category and event_type classification for all events.

Implement banding logic (volume, value, delay days).

19.10.2 Anonymisation

Implement anonymisation service (rotating pepper, deterministic hash, banding).

Add anonymous entity ID generation and mapping (deleted after 90 days).

Implement opt-out toggle in Company Admin → Privacy.

Add consent tracking in consent_records.

19.10.3 Predictive Insights

Create predictive_insights table.

Implement daily cron job for model inference (XGBoost, LightGBM, LSTM, Prophet).

Train initial models using historical data from sandbox and pilot.

Integrate insights into Smart Inbox (priority 40) with one-click actions.

Add AI Risk Engine integration for delay risk and default probability.

19.10.4 Anomaly Detection

Implement anomaly-detector service (Isolation Forest).

Run every 6 hours.

Add severity classification (LOW, MEDIUM, HIGH, CRITICAL).

Integrate with Smart Inbox for alerts.

Add Groq summary generation for anomalies.

19.10.5 Federated Learning Integration

Expose anonymised memory events as training data for federated learning.

Ensure differential privacy ($\epsilon=0.5$) for model updates.

Add model metadata tracking (predictive_models table).

19.10.6 Query Interface

Implement gRPC query interface for internal services.

Add rate limiting and audit logging for all queries.

Document query API.

19.10.7 Testing

Test event capture for all event types.

Test anonymisation (GTID $\rightarrow$ anonymous ID, banding).

Test opt-out (tenant stops contributing → no new events).

Test predictive insights generation and delivery.

Test anomaly detection and alerting.

Test integration with AI Risk Engine and Federated Learning.

19.11 AI Authority Summary for Part 19

END OF PART 19 — TRADE MEMORY LAYER & PREDICTIVE TRADE INSIGHTS (v11.0 FULLY EXPANDED)

PART 20 — NETWORK EFFECTS & COMPETITIVE MOAT

20.0 Purpose & Design Philosophy

This part provides a strategic defensibility analysis of the SGTX platform. Unlike the functional specifications in previous parts, this section explains why the platform becomes more valuable with every trade and what competitors would need to overcome to challenge SGTX. This analysis is critical for:

Investor confidence — demonstrating long-term value creation.

Strategic planning — prioritising investments that strengthen the moat.

Partner engagement — showing why integration with SGTX is strategically beneficial.

Competitive positioning — understanding threats and opportunities.

Key principles (non-marketplace, strategic):

Data compounding — every trade generates proprietary, anonymised data that improves the platform.

Network effects — more participants → more value → more participants.

Switching costs — tenants and providers cannot easily migrate to a competitor.

Regulatory integration — government mandates create insurmountable barriers to entry.

Zero-cost architecture — economic moat prevents price-based competition.

AI Integration in Part 20:

20.1 The Trust Flywheel (Compounding Data Advantage)

Every trade executed on SGTX feeds into a virtuous cycle that strengthens all participants.

20.1.1 The Flywheel Diagram (Textual)

text

11

■ MORE TRADES ■

111

11

■ MORE DATA ■

■ (Trade Memory, Disputes, Delays, Documents) ■



■ ■ ■



■ ■ ■



■ ■ ■



■ ■ ■



■ ■ ■



20.4 Economic Moat — Cost Advantages

20.4.1 Infrastructure Cost Comparison

Total Annual Cost Advantage: \$2M+ per year (vs a cloud-based, proprietary competitor).

20.4.2 Fee Model Advantage

20.5 Network Effects Quantified (Illustrative KPIs)

20.6 What a Competitor Would Need to Overtake SGTX

A realistic competitor would need:

Conclusion: No startup can acquire these overnight. Even a well-funded incumbent (e.g., a global bank or logistics giant) would need 5–7 years to catch up — during which SGTX continues to accelerate.

20.7 Strategic Recommendations to Strengthen the Moat

Based on this analysis, the following actions are recommended for years 2–5:

20.7.1 Data Strategy

20.7.2 Regulatory & Government Strategy

20.7.3 Technology & IP Strategy

20.7.4 Network Strategy

20.8 Competitive Threat Matrix

20.8.1 Threat Landscape

20.8.2 Mitigation Timeline

20.9 Conclusion — Why SGTX Is Unassailable

SGTX is not merely a software platform; it is a trust network that becomes more valuable with every trade. The combination of:

Trade Memory (proprietary historical data)

Trust Passport & TRI (portable, verified reputation)

Institutional Trade Graph (network effects)

Zero-Cost Infrastructure (economic moat)

Government Mandates (legal moat)

Takaful Protection Fund (regulatory moat)

Full Disclosure Financing (trust moat)

Non-Custodial Architecture (security moat)

...creates a multi-layered defence that no competitor can realistically breach. Competitors can copy code, but they cannot copy trust, history, or network density.

This is the ultimate competitive moat of SGTX — and why the platform will become the global standard for sovereign trade execution.

END OF PART 20 — NETWORK EFFECTS & COMPETITIVE MOAT (v11.0 FULLY EXPANDED)

PART 21 — AUTO-GENERATED BARCODES FOR SHIPMENT TRACKING

21.0 Purpose & Design Philosophy

Every pallet, carton, or item that moves through a trade must leave an unforgeable, instantly verifiable digital fingerprint. Barcodes are not just stickers — they are the physical anchor of the platform's digital

twin. In v11.0, the barcode is intelligent, self-sovereign, and resilient. It carries exactly the data each party needs, can be verified even without network access, and can be read by voice or camera when physical labels fail. All generation, scanning, and verification runs on open-source libraries and zero-cost infrastructure. The system supports multiple symbologies (GS1-128, Data Matrix, QR) and automatically assigns globally unique SSCC18 codes to every logistic unit (pallet). All scans trigger Governor-gated milestone confirmations and are immutably logged via Loom.

The barcode system fully supports multi-shipment contracts: each shipment has its own set of pallets, each with a unique SSCC, and the barcode includes the shipment USTN (or microUSTN for distressed splits). The predictive scanning reliability agent (A2, XGBoost) proactively warns warehouse managers if a batch of labels is likely to fail in the intended environment. A detailed label print workflow ensures flawless physical application from PDF to pallet, with a reprint policy enforced by the Governor after pallets have been loaded. Blockchain anchoring (Polygon) provides an immutable, publicly verifiable chain of custody for high-value cargo. Self-sovereign pallet identity (W3C Verifiable Credentials) allows offline verification using a pre-cached SGTX public key.

Key principles (non-marketplace, sovereign):

Unforgeable identity — every barcode is cryptographically linked to its USTN and pallet record.

Offline-first — scanning, verification, and credential validation work without internet.

Self-sovereign — pallets carry their own verifiable credentials, signed by SGTX.

Multi-modal identification — barcode, visual recognition, and voice work interchangeably.

Predictive reliability — AI predicts and prevents scan failures before they occur.

Governor-enforced reprint policy — prevents fraud after loading.

Multi-shipment aware — each shipment's pallets are independently tracked.

AI Integration in Part 21:

21.1 Core Technology Stack (Zero-Cost)

No cloud APIs, no subscription fees.

21.2 Barcode Generation & Dynamic Content (A1)

21.2.1 SSCC18 Allocation

During the packing plan lock (Phase 2, Part 3B.3.4), the packing-containerisation-service generates a globally unique Serial Shipping Container Code (SSCC18) for every pallet. The SSCC conforms to GS1 standard:

Format: 1 (extension digit) + 0614141 (6-digit company prefix reserved for SGTX) + 123456789 (9-digit serial, unique per pallet) + 0 (check digit)

Example: 106141411234567890

SSCC Generation Algorithm:

rust

```
fn generate_ssc(extension_digit: u8, company_prefix: &str, serial: u64) -> String {  
    let base = format!("{:09}", extension_digit, company_prefix, serial);  
    let check_digit = calculate_gs1_check_digit(&base);  
    format!("{:018}", base, check_digit)  
}
```

```

fn calculate_gs1_check_digit(base: &str) -> u8 {
  let mut sum = 0;
  for (i, c) in base.chars().enumerate() {
    let digit = c.to_digit(10).unwrap();
    if i % 2 == 0 {
      sum += digit * 3;
    } else {
      sum += digit * 1;
    }
  }
  let remainder = sum % 10;
  if remainder == 0 { 0 } else { 10 - remainder }
}

```

Multi-shipment Support: For multi-shipment contracts, the SSCC includes an internal reference to the shipment number (encoded in the serial range). Each shipment's pallets are assigned from a distinct serial block.

Distressed Micro-contracts: When a distressed partial sale occurs (Part 3B.7), a new microUSTN is generated, and new SSCCs may be generated for the split pallets (or existing ones retained with a flag). The mapping is stored in pallet_details with micro_ustn reference.

21.2.2 QR Code Payload (Dynamic per Stakeholder)

In addition to the linear barcode, a QR code is generated that encodes a JSON payload. The AI (Groq, A1) decides which attributes to include based on the intended audience of that specific label copy (the seller can select from predefined templates or customise).

QR Payload Example:

```

json
{
  "ustn": "SGTX-EG-26-F3A-1",
  "pallet_id": "PAL-002",
  "sscc": "106141411234567891",
  "product": "Fresh oranges",
  "lot": "OR-2026-0412",
  "net_weight_kg": 1110,
  "gross_weight_kg": 1165,
  "layer_summary": "11×10 + 1×1 cartons",
  "cold_treatment_cert": "CT-20260415-001",
  "verify_url": "https://sgtx.io/verify/pallet?sscc=106141411234567891",
  "verifiable_credential": {

```

```

"type": "VerifiableCredential",
"issuer": "https://sgtx.io/issuers/platform",
"proof": "base64..."
}
}

```

Templates:

AI Assistance (A1, Groq): The AI suggests the optimal template based on the trade destination, commodity, and user role. The seller can override.

21.2.3 Self-Sovereign Pallet Identity (Verifiable Credential)

Each SSCC is also issued a W3C Verifiable Credential signed by the SGTX platform. This credential contains the pallet's immutable identity data (product, lot, weight, origin, USTN, GTIDs) and can be embedded in the QR code as a compact URL or raw VC.

Verifiable Credential Structure:

```

json
{
"@context": ["https://www.w3.org/2018/credentials/v1"],
"id": "https://sgtx.io/credentials/pallet/106141411234567891",
"type": ["VerifiableCredential", "PalletCredential"],
"issuer": "https://sgtx.io/issuers/platform",
"issuanceDate": "2026-04-15T12:00:00Z",
"credentialSubject": {
"sscc": "106141411234567891",
"ustn": "SGTX-EG-26-F3A-1",
"product": "Fresh oranges",
"lot": "OR-2026-0412",
"net_weight_kg": 1110,
"origin_country": "EG",
"cold_treatment_cert": "CT-20260415-001"
},
"proof": {
"type": "Ed25519Signature2020",
"created": "2026-04-15T12:00:00Z",
"proofPurpose": "assertionMethod",
"verificationMethod": "https://sgtx.io/keys/ed25519",
"signature": "base58..."
}
}

```

}

Offline Verification: A recipient can scan the QR and verify the pallet's provenance without internet access, using only the SGTX public key (pre-cached in the mobile app). This is achieved via a lightweight ZKproof (Plonky3) that the data matches the platform's signature.

Verification Flow (Offline):

Mobile app scans QR → extracts Verifiable Credential.

App checks credential's signature against cached SGTX public key.

App validates that the credential has not expired.

App displays green checkmark: "Verified — Lot VN2406L, 48 cartons, 384 kg. No internet required."

21.2.4 AI-Planned Label Layout

When generating printable label sheets (PDF), the system uses an AI planner (LightGBM) to optimise label placement on A4 sheets to minimise waste. It also adjusts font sizes and data density based on the label size and the amount of information requested by the user.

21.3 Multi-Modal Pallet Identification (Offline & Online)

Three independent identification methods work seamlessly together:

21.3.1 Camera-Based Barcode Scanning (ZXingC++)

Decodes GS1-128 and QR codes in real time.

Continuous scanning mode: successful decode triggers haptic feedback and advances to the next expected pallet in the job list.

Offline validation: Scanned SSCCs are checked against locally cached pallet_details (synced when job was assigned). If the SSCC is not found locally, the scan is queued and validated upon reconnection.

Multi-shipment: The app knows which shipment's pallets are being loaded (from job assignment). Scanning a pallet that belongs to a different shipment shows a warning and does not confirm the milestone.

21.3.2 AI-Powered Visual Recognition (Offline)

If a barcode is torn, smudged, or partially covered, the worker switches to visual recognition mode:

Device camera captures the pallet label.

ONNX model (finetuned on warehouse label images) recognises and extracts the printed SSCC number and key text.

Even if only the human-readable digits are visible, the pallet is identified.

The extracted SSCC is validated against the local pallet list.

Model training: Finetuned on 10,000+ warehouse label images (synthetic and real). Retrained weekly via federated learning (Add-on 2).

21.3.3 Voice-Activated Hands-Free Identification (Offline)

Worker speaks: "Pallet OR005 loaded into container TCNU1234567."

Vosk transcribes offline (English, Arabic, Vietnamese, German supported).

HF Mixtral (local) extracts: action = load, pallet_id = OR005, container = TCNU1234567.

App validates pallet_id against local job list.

Works in noisy environments with a headset. Fallback to manual entry if confidence <80%.

In the mobile app, AR mode overlays a live camera view with colour-coded bounding boxes around each pallet's label:

AR Overlay Example:

[illegible]

■ failure in outdoor conditions at destination (Alexandria port). ■

□ □

■ ■

22

Governance (A4): The reprint uses the same SSCCs (no new IDs). The reprint is logged but does not require Governor approval (pre-loading).

21.6.1 Printer Compatibility

zpl

^FO50,50^A0N,30,30^FDSSCC: 106141411234567890^FS

^FO50,130^A0N,20,20^FDLot: OR-2026-0412^FS

^FO50,190^A0N,20,20^FDGross Weight: 1,165 kg^FS

^FO50,250^B3N,N,100,Y,N^FD106141411234567890^FS

 $^{\wedge}XZ$

21.6.2 Label Content Customisation

One-click: Select template → click Print (1).

Post-Loading Reprint Workflow:

User clicks "Request Reprint".

Modal appears: "Reason for reprint (min 10 characters):" + text field with character counter.

User submits reason.

Governor evaluates:

If pallet status is LOADED or DELIVERED → requires reason.

If pallet status is DELIVERED and shipment is SETTLED → DENY (cannot reprint after settlement).

If reason is insufficient (<10 chars) → CONDITIONAL.

If ALLOW → new labels with same SSCC generated (enhanced contrast if requested).

If DENY → Decision Panel explains: "This pallet has already been delivered. Reprinting is not allowed."

One-click: Click "Request Reprint" → enter reason → submit (1). If auto-approved, reprint proceeds.

21.7 Blockchain-Anchored Barcode Hashes (Immutable Timeline)

For high-value or regulated shipments (optional, selected by seller), the platform anchors the hash of each pallet's identity data on Polygon for an additional layer of public verifiability.

Process:

At packing plan lock, the pallet_details record is hashed (SHA256) along with the SSCC, shipment USTN, and a timestamp.

The hash is included in a batch transaction to a simple smart contract (self-deployed, minimal gas).

The transaction hash is stored in shipment_barcodes or a dedicated blockchain_verificationstable.

Batch Transaction Example:

json

```
{
  "batch_id": "BATCH-20260415-001",
  "ustn": "SGTX-EG-26-F3A-1",
  "pallet_hashes": [
    {"sscc": "106141411234567890", "hash": "sha256:abc123..."},
    {"sscc": "106141411234567891", "hash": "sha256:def456..."}
  ],
  "timestamp": "2026-04-15T12:00:00Z",
  "signature": "ed25519:..."
}
```

Smart Contract (Simplified Solidity):

solidity

```
contract PalletRegistry {
  mapping(bytes32 => uint256) public palletHashes;
  event PalletAnchored(bytes32 indexed hash, uint256 timestamp);
}
```

```

function anchorPallet(bytes32 hash) external {
    palletHashes[hash] = block.timestamp;
    emit PalletAnchored(hash, block.timestamp);
}

function verifyPallet(bytes32 hash) external view returns (bool) {
    return palletHashes[hash] > 0;
}
}

```

Cost: Negligible (~\$0.01-0.10 per batch, paid by the platform operator as part of infrastructure cost). No per-transaction fee charged to tenants.

One-click: Seller toggles "Enable blockchain anchoring" in Company Admin → Settings. Zero clicks after that.

21.8 Scan-Triggered Milestone Confirmation & Fee Release

Every successful scan (barcode, visual, or voice) triggers the standard Phase 5 milestone flow (Part 3B.6):

Mobile app sends a signed pallet.loaded event to shipment-service via NATS.

Governor (OPA + WasmEdge) verifies:

Device identity (ZITADEL certificate).

Employee permissions (shipment.milestone.confirm).

Current FeeLock status (ACTIVE).

Pallet belongs to the correct shipment (multi-shipment check).

If allowed, milestone confirmed, and pallet_details.status updated to LOADED.

When the last pallet of a container is loaded, the container milestone ("Loaded") is auto-confirmed (A4 if multisource consensus reached).

FeeLock release conditions are evaluated (but note: SGTX fee already paid upfront — no further release; if financing fee, it was deducted at disbursement). The milestone confirmation is still logged for audit.

Scan Event Example:

```

json
{
    "event_type": "pallet.loaded",
    "ustn": "SGTX-EG-26-F3A-1",
    "sscc": "106141411234567890",
    "employee_gtid": "SGTX-EG-LSP-001",
    "device_id": "device-abc123",
    "timestamp": "2026-04-15T14:30:00Z",
    "method": "barcode",
    "signature": "ed25519:..."
}

```

Smart Inbox: All relevant parties receive real-time updates (via NATS) when milestones are confirmed.

21.9 Governance & Verification Gates (Part 21)

21.10 Database Schema Additions (Part 21)

sql

-- Pallet details (core barcode identity)

```
CREATE TABLE pallet_details (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  packing_plan_id UUID REFERENCES packing_plans(id) ON DELETE CASCADE,  
  ssc TEXT UNIQUE NOT NULL,  
  micro_ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,  
  lot_number TEXT,  
  product_hs_code CHAR(6),  
  cartons_per_pallet INTEGER NOT NULL,  
  layer_patterns JSONB,  
  total_cartons INTEGER NOT NULL,  
  net_weight_kg DECIMAL(10,2),  
  gross_weight_kg DECIMAL(10,2),  
  cold_treatment_cert_ref TEXT,  
  verifiable_credential_hash TEXT,  
  blockchain_tx_hash TEXT,  
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'LOADED', 'DAMAGED', 'DELIVERED'  
  printed_at TIMESTAMPTZ,  
  reprinted_count INTEGER DEFAULT 0,  
  last_reprint_reason TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Barcode print jobs

```
CREATE TABLE barcode_print_jobs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,  
  job_type TEXT NOT NULL, -- 'INITIAL', 'REPRINT'  
  pallet_ids UUID[] NOT NULL,  
  template TEXT NOT NULL, -- 'STANDARD', 'CUSTOMS_READY', 'CONSIGNEE',  
  'TREATMENT_AWARE'  
  language TEXT DEFAULT 'en',
```

```

printer_ip TEXT,
zpl_data TEXT,
pdf_url TEXT,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'PROCESSING', 'COMPLETED', 'FAILED'
requested_by UUID REFERENCES employees(id) ON DELETE SET NULL,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
completed_at TIMESTAMPTZ
);

```

-- Barcode scans

```

CREATE TABLE barcode_scans (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
sscc TEXT NOT NULL,
scan_method TEXT NOT NULL, -- 'BARCODE', 'VISUAL', 'VOICE'
device_id TEXT NOT NULL,
employee_gtid TEXT NOT NULL,
gps_coordinates JSONB,
confidence DECIMAL(5,2),
raw_image_hash TEXT,
success BOOLEAN DEFAULT true,
error_message TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Predictive reliability scores

```

CREATE TABLE predictive_scan_reliability (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
batch_id TEXT NOT NULL,
printer_id TEXT,
label_material TEXT,
environment TEXT, -- 'INDOOR', 'OUTDOOR', 'WAREHOUSE'
predicted_failure_probability DECIMAL(5,2),
actual_failure_rate DECIMAL(5,2),
model_version TEXT,

```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Reprint requests (post-loading)
CREATE TABLE reprint_requests (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
pallet_id UUID REFERENCES pallet_details(id) ON DELETE CASCADE,
reason TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'APPROVED', 'REJECTED'
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
requested_by UUID REFERENCES employees(id) ON DELETE SET NULL,
approved_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Blockchain anchors
CREATE TABLE blockchain_anchors (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE CASCADE,
batch_id TEXT NOT NULL,
pallet_hashes JSONB NOT NULL,
transaction_hash TEXT NOT NULL,
block_number BIGINT,
network TEXT DEFAULT 'POLYGON',
anchored_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_pallet_details_ssc ON pallet_details(sscc);
CREATE INDEX idx_pallet_details_ustn ON pallet_details(ustn);
CREATE INDEX idx_pallet_details_status ON pallet_details(status);
CREATE INDEX idx_barcode_print_jobs_ustn ON barcode_print_jobs(ustn);
CREATE INDEX idx_barcode_print_jobs_status ON barcode_print_jobs(status);
CREATE INDEX idx_barcode_scans_ustn ON barcode_scans(ustn);
CREATE INDEX idx_barcode_scans_ssc ON barcode_scans(sscc);
CREATE INDEX idx_barcode_scans_method ON barcode_scans(scan_method);
CREATE INDEX idx_predictive_scan_reliability_batch ON predictive_scan_reliability(batch_id);
CREATE INDEX idx_reprint_requests_pallet ON reprint_requests(pallet_id);

```

CREATE INDEX idx_reprint_requests_status ON reprint_requests(status);

CREATE INDEX idx_blockchain_anchors_ustn ON blockchain_anchors(ustn);

CREATE INDEX idx_blockchain_anchors_tx ON blockchain_anchors(transaction_hash);

21.11 Worked Example — Barcode Printing, Scanning, and Post-Loading Reprint

Context: Mekong Fresh locks a packing plan for a multi-shipment contract (Shipment 1, oranges). 22 pallets.

21.11.1 Label Generation

Seller navigates to Barcode Print tab.

System generates SSCCs for all 22 pallets.

Seller selects "Customs-Ready" template, language Arabic.

AI (A1, Groq) translates product name to Arabic: "■■■■■■■■ ■■■■■" .

Seller clicks Print — ZPL sent directly to Zebra printer (or PDF generated for laser printer).

Labels printed with SSCCs and QR codes containing Verifiable Credentials.

21.11.2 Loading (Warehouse)

Warehouse worker opens SGTX mobile app, selects Shipment 1 job.

AR overlay shows 22 pallets with red/blue/green status.

Worker scans each pallet's barcode. Each scan beeps, pallet status turns green.

One pallet's label is damaged (torn). Worker switches to Visual Recognition mode — app extracts SSCC from printed digits, validates.

Worker uses voice for the last pallet: "Pallet OR022 loaded into container TCNU1234567" . App confirms.

All 22 pallets scanned. System auto-confirms "Container Loaded" milestone.

Smart Inbox updates for seller, buyer, logistics provider.

21.11.3 Post-Loading Reprint Request

Seller realises another pallet's label (OR015) is faded and may not scan at destination.

Seller clicks "Request Reprint" on OR015.

Modal appears: "Reason for reprint (min 10 characters):" Seller types: "Label faded due to humidity during transit."

Governor evaluates: pallet status = LOADED, reason length = 42 chars → ALLOW.

New labels with same SSCC generated (enhanced contrast). Seller prints and sends to destination.

Reprint logged in reprint_requests and barcode_print_jobs.

21.11.4 Blockchain Anchor (Optional)

Seller had enabled blockchain anchoring in Company Admin.

System batches all 22 pallet hashes and submits to Polygon.

Transaction hash stored in blockchain_anchors.

Recipient can verify pallet authenticity on-chain.

21.11.5 Destination Scanning

Receiver scans QR code (offline). App verifies Verifiable Credential using cached SGTX public key.

Green checkmark displayed: "Verified — Fresh Oranges, 1,110 kg, SSCC 106141411234567891."

Receiver confirms delivery. Milestone updated. Shipment complete.

21.12 Implementation Checklist (Part 21)

21.12.1 Core Infrastructure

Implement SSCC generation algorithm (GS1-18).

Create pallet_details table.

Implement QR code generation with dynamic payload.

Add Verifiable Credential generation (W3C VC + Ed25519 signing).

Implement ZPL and PDF label generation (Tera + headless Chrome).

21.12.2 Mobile & Desktop Integration

Integrate ZXingC++ barcode scanning in mobile app.

Implement visual recognition (ONNX model) for damaged barcodes.

Add voice-activated pallet identification (Vosk + HF Mixtral).

Build AR overlay with colour-coded status (AR.js).

Implement offline validation (WatermelonDB cache of pallet details).

21.12.3 Print & Reprint

Build label print workflow (ZPL and PDF).

Implement label customisation (templates, language, fields).

Add predictive scanning reliability agent (XGBoost).

Implement reprint policy (pre-loading vs post-loading, Governor gate).

21.12.4 Blockchain & Verification

Implement blockchain anchoring (Polygon, batch transactions).

Add offline Verifiable Credential verification.

Build public verification page (/verify/pallet).

21.12.5 Governance & Monitoring

Add Governor gates for barcode generation, scanning, and reprints.

Implement Loom logging for all barcode events.

Add Prometheus metrics:

sgtx_barcodes_generated_total

sgtx_barcodes_scanned_total

sgtx_barcodes_scan_success_rate

sgtx_barcodes_reprints_requested

sgtx_barcodes_reprints_approved

Set up alerting for:

High scan failure rate (>5%)

Predictive reliability alert (risk >70%)

Blockchain anchoring failures

21.12.6 Testing

Test SSCC generation uniqueness.

Test barcode scanning (mobile, visual, voice).

Test offline validation.

Test reprint policy (pre-loading vs post-loading).

Test Verifiable Credential generation and verification.

Test blockchain anchoring.

Test multi-shipment pallet assignment.

Test predictive scanning reliability agent.

21.13 AI Authority Summary for Part 21

END OF PART 21 — AUTO-GENERATED BARCODES FOR SHIPMENT TRACKING (v11.0 FULLY EXPANDED)

PART 22 — IMPLEMENTATION ROADMAP & SCALING STRATEGY

22.0 Purpose & Design Philosophy

This part translates the entire SGTx blueprint into a practical, time-bound, risk-managed implementation plan. The roadmap is organised into five phases (0 through 4), each with clear objectives, deliverables, success criteria (KPIs), dependencies, and estimated duration. The plan is designed to deliver early value (Phase 1 — agricultural exports) while progressively adding complexity (multi-shipment, financing, imports, add-ons). All phases assume a team of 8–12 engineers (backend, frontend, DevOps, QA) and a parallel team for legal/compliance.

The roadmap is itself an AI-managed, self-correcting process. Priorities shift automatically when a new law passes in a target jurisdiction, when a microservice shows unexpected latency, or when user growth in a region demands immediate infrastructure expansion. All planning, deployment, and scaling use zero-cost infrastructure (bare-metal K3s, open-source CI/CD, self-hosted project management). No cloud vendor lock-in, no proprietary project management tools, no billing details required.

AI Integration in Part 22:

22.1 Core Phases (AI-Driven Feature Slicing, v11.0 Updated)

22.1.1 Phase 0 — Foundation & Pilot Preparation (Months 1–3)

Objective: Establish the legal entity, core infrastructure, and a single pilot exporter (e.g., frozen strawberries) to prove the concept end-to-end with real documents and payments.

Core Deliverables:

v11.0 Features NOT in Phase 0:

Multi-shipment contracts

Non-uniform layer stacking

Mode B/C logistics

Conditional QC

Deferred payment

Milestone-based settlement

ZK proofs

Distressed cargo full workflow

Trade Memory Layer

Predictive Insights

Takaful Fund

Trust Passport

Dependencies:

KPIs:

Timeline (Weeks):

AI Assistance: Use Groq to generate weekly status reports for stakeholders, summarising progress and risks.

22.1.2 Phase 1 — Agricultural Exports MVP (Months 4–9)

Objective: Launch a production-ready platform for agricultural exports (HS 06–11) for a limited set of corridors (Egypt → EU, Egypt → UAE). Mandate not yet enforced; voluntary adoption by up to 50 exporters.

Core Deliverables:

v11.0 Features NOT in Phase 1:

Multi-shipment contracts

Mode B/C logistics

Conditional QC

Deferred payment

Milestone-based settlement

ZK proofs

Accelerated distressed outreach

Third-party expert

Causal inference

Trade Memory Layer

Predictive Insights

Takaful Fund

Trust Passport

Proof of Reserves

Dependencies:

KPIs:

Timeline (Months):

AI Assistance: Use Groq to generate Smart Inbox titles, tenant messages, and dispute mediation suggestions (advisory). Use HF local for document extraction during onboarding.

22.1.3 Phase 2 — Multi-Shipment, Financing & Advanced Logistics (Months 10–18)

Objective: Add multi-shipment contracts, trade finance (co-financing, financier portal), advanced logistics modes (B and C), non-uniform layer stacking, conditional QC, distressed cargo full workflow, and milestone-based partial settlement. Expand to more corridors and additional PSPs.

Core Deliverables:

Dependencies:

KPIs:

Timeline (Months):

AI Assistance: Causal inference engine (DoWhy + EconML) integrated into dispute workflow; GNN risk engine (add-on 1) trained and deployed.

22.1.4 Phase 3 — Imports, DeFi & Full Add-ons (Months 19–30)

Objective: Launch import functionality (Form 4, duties, local payment batch), DeFi financing, and the seven critical add-ons (GNN, federated learning, self-healing, pentesting, PQC, expanded ZK). Achieve full national coverage (all commodities, all ports) and prepare for mandatory decree.

Core Deliverables:

Dependencies:

KPIs:

Timeline (Months):

AI Assistance: Federated learning coordinator; GNN model retrained weekly; ZK proof generation client-side (WASM).

22.1.5 Phase 4 — Global Expansion & Continuous Evolution (Years 3–5)

Objective: Expand SGTx to partner countries (first: UAE, then Germany, Vietnam, etc.), deploy sovereign nodes in each region, and establish mutual recognition of USTNs. Evolve the platform based on market feedback and new regulations.

Core Deliverables:

Dependencies:

KPIs:

Timeline:

AI Assistance: Federated learning across international nodes; predictive scaling for new regions.

22.2 AI-Powered Project Management & Dynamic Resource Allocation

22.2.1 Project Oracle Agent

The Project Oracle agent (Rig + XGBoost) monitors the entire development pipeline:

Output: Revised priority list every Monday.

Example Project Oracle Output (Groq-Generated):

"The barcode visual recognition model is taking 40% longer to train than estimated due to insufficient labelled data. Allocate Developer Y to data labelling for 3 days, delaying AR feature by 2 days but keeping Phase 2 schedule on track. Also, the rate of tenant onboarding has dropped 15% — consider streamlining

Step 3 (KYB/KYC) with pre-filled data from commercial registry."

One-click: Project Oracle generates a weekly progress report with one click. Admin can accept recommendations with one click.

22.2.2 Dynamic Resource Allocation

22.3 Self-Healing CI/CD with Intelligent Rollback

22.3.1 Canary Deployment

22.3.2 Rollback Triggers

Post-Rollback Analysis: Groq generates root-cause analysis: "Rollback triggered at 14:32 due to 15% increase in Governor latency. Root cause: database query change in trade-service missing index. Reverted to previous version. Recommendation: Add index before redeploying."

22.4 Predictive Scaling Engine (A2)

22.4.1 LSTM Model

The Predictive Scaling Engine (A2, LSTM) forecasts resource needs:

Output: 12-month capacity plan per sovereign node.

Example Prediction:

"Based on current growth and upcoming citrus season, Cairo node will reach 85% CPU utilisation by November 2026. Provision a second node in Cairo by 2026-10-01. Recommendation: add 2 worker nodes (4 CPU, 16GB RAM each) to the Cairo cluster."

One-click: Admin clicks "Approve Provisioning" → Ansible playbook runs automatically.

22.4.2 HPA Auto-Scaling Rules

22.5 Dynamic Geographic Expansion Optimizer (A2)

The Dynamic Geographic Expansion Optimizer (A2, Multi-armed bandit) balances exploration (new regions) and exploitation (scaling existing regions) to maximise platform revenue.

Output: Monthly expansion recommendations.

Example Recommendation:

"Priority 1: São Paulo, Brazil. 230 registered users waitlisted, regulatory environment stable, no sanctions. Existing trade volume with Egypt: \$180M/year. Estimated first-year GMV \$12M. Cost to deploy: \$15,000 (hardware). Recommended: deploy sovereign node by Q3 2026."

One-click: Admin clicks "Deploy Node" → Ansible playbook provisions the entire sovereign node.

22.6 Zero-Cost Infrastructure Lifecycle Management

22.6.1 Predictive Hardware Failure (A2, LSTM)

The Predictive Hardware Failure model (LSTM trained on Backblaze drive failure dataset) forecasts component failure:

Example Alert (Groq):

"Cairo node SSD 3 (model: Samsung PM9A3) predicted to fail in 45 days (confidence: 92%). Replacement ordered. Maintenance window scheduled for 2026-08-15 02:00-04:00 EET. Workloads shifted to Cairo node 2 during maintenance."

22.6.2 Automated Maintenance

22.7 AI-Generated Partnership Proposals (A1)

The Partnership Proposals agent (RIA + Groq) identifies potential marketplace partners and generates personalised proposals.

Proposal Content (Groq-Generated):

text

PARTNERSHIP PROPOSAL — TradeBridge

Company Overview:

TradeBridge is a digital trade platform connecting buyers and sellers in the MENA region. They have 5,000+ registered users and process \$50M/year in trade leads.

Why Partner with SGTX:

- SGTX provides cryptographic trade execution — TradeBridge users can execute trades with zero counterparty risk.
- Revenue share: TradeBridge earns 0.5% of SGTX trade fees on attributed trades.
- Integration effort: 2 developer weeks (API + webhook).
- Expected first-year revenue for TradeBridge: \$120,000.
- Compliance alignment: TradeBridge is registered in the UAE, a Full tier jurisdiction.

Recommended Next Steps:

1. Send this proposal (PDF) to TradeBridge's BD team.
2. Offer a 30-minute demo for their technical team.
3. Provide a sandbox account for integration testing.

One-click: Click "Generate Proposal" → PDF ready for download. Click "Send" → email sent via notification service.

22.8 Continuous Compliance Roadmap Alignment (A2)

When RIA detects a significant regulatory change, the Compliance Roadmap Agent creates a compliance milestone in project management.

Example Alert:

text

COMPLIANCE MILESTONE CREATED

Regulation: EU General Product Safety Regulation (GPSR)

Effective date: 13 Dec 2026

Impact: Additional documentation required for consumer goods

Priority: P1 (within 3 months)

Due date: 1 Dec 2026

Assigned to: Document Service team

Groq explanation: "The new EU GPSR requires additional documentation for certain consumer goods. The Document Requirement Service must be updated to include new document types for affected HS codes.

Estimated effort: 3 developer days."

One-click: Click "Create Ticket" → Gitea issue created. Click "Assign" → assigned to appropriate team.

22.9 Scaling Metrics (Living Dashboard)

22.9.1 Grafana Dashboard Sections

22.9.2 AI-Generated Board Summary (Groq)

Example Monthly Summary:

text

SGTX BOARD SUMMARY — June 2026

■ GROWTH

GMV grew 12% this month, driven by the new Brazil corridor. Total GMV: \$47.5M YTD. New tenants: 87 (up from 72 last month). Active tenants: 423. Financing volume: \$18M (up 8% MoM).

■ RELIABILITY

Platform uptime: 99.97% (above 99.95% target). P95 latency: 620ms. Two incidents: one minor (5 min) due to network switch failure, resolved automatically. One P1 incident (12 min) due to Governor latency spike — post-mortem published.

■■ COMPLIANCE

Two compliance items resolved: Egypt cold treatment update (completed 5 Jun) and UAE customs rule (completed 12 Jun). Three new EU regulations flagged for Q1 2027. No overdue reports.

■ TENANT EXPERIENCE

Onboarding completion rate: 88% (up from 82%). Smart Inbox time-to-action: 3.2h (target: <4h). Dual-mode toggle usage: 34% of active tenants. Multi-shipment contract usage: 12% of new trades.

■ FINANCE

Revenue: \$712,500 (trade fees) + \$45,000 (financing fees) = \$757,500. SLA credits issued: \$12,400. Net revenue: \$745,100. Partner share: \$87,600. Estimated monthly run-rate: \$900,000.

■ RECOMMENDATIONS

1. Accelerate Brazil node deployment — 230 users waiting.
2. Add 2 replicas to Governor service due to traffic growth.

3. Update onboarding wizard to reduce Step 3 drop-off.

One-click: Click "Refresh" → current data. Click "Export PDF" → board-ready PDF.

22.10 Implementation Checklist for the Roadmap Itself (Zero-Cost)

22.10.1 Project Management Infrastructure

Set up self-hosted project management (Gitea + Drone + Taiga).

Configure CI/CD pipeline with canary deployments.

Deploy Project Oracle agent (Rig + XGBoost).

22.10.2 Predictive Scaling

Enable Predictive Scaling Engine (LSTM model).

Train on historical resource usage data.

Integrate with Ansible for automated provisioning.

22.10.3 Hardware Failure Prediction

Train hardware failure model (Backblaze dataset).

Integrate with inventory management.

Set up maintenance scheduling.

22.10.4 Compliance Roadmap

Deploy compliance roadmap agent (RIA extension).

Create Gitea integration for compliance milestones.

Set up Smart Inbox alerts for regulatory changes.

22.10.5 Reporting

Build Grafana dashboard with all sections.

Generate first AI board summary using Groq.

Verify tenant experience KPIs are being collected.

22.10.6 Geographic Expansion

Deploy Dynamic Geographic Expansion Optimizer (A2).

Create Ansible playbook for sovereign node provisioning.

Test deployment in a new region (sandbox).

22.11 AI Authority Summary for Part 22

END OF PART 22 — IMPLEMENTATION ROADMAP & SCALING STRATEGY (v11.0 FULLY EXPANDED)

PART 24 — THREAT MODEL & ATTACK SURFACE ANALYSIS (STRIDE + MITRE)

24.0 Purpose & Design Philosophy

The SGTX platform handles sensitive trade data, financial transactions, and cryptographic identities across multiple jurisdictions. A robust security model is not optional — it is a constitutional requirement. This part defines the comprehensive threat model for the SGTX platform using industry-standard frameworks:

STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) — per component, per trust boundary.

MITRE ATT&CK — adversary tactics, techniques, and procedures (TTPs) relevant to trade platforms.

All mitigations are zero-cost, built on open-source tools (OPA, WasmEdge, Cilium, Falco, etc.) and free threat intelligence feeds. AI (Groq, Hugging Face) is used for advisory purposes — e.g., summarising threat findings, generating post-mortems, and suggesting remediation steps. Never autonomous blocking (A5 forbidden).

AI Integration in Part 24:

Key principles (non-marketplace, sovereign):

Zero-trust architecture — no implicit trust; every request is authenticated, authorised, and encrypted.

Defence in depth — multiple layers of control (network, application, data, constitutional).

Immutable audit — every security-relevant event is logged immutably via Loom.

Continuous verification — hourly Loom chain verification, weekly pentests, daily vulnerability scans.

No security-through-obscurity — all security controls are documented and verifiable.

Scope: This threat model covers all v11.0 features including:

Multi-shipment contracts (Part 3.6)

Dual-mode toggle (Part 2.8)

Deferred payment (Part 6.8)

Conditional QC (Part 3B.6.4)

ZK Proofs (Part 11.7)

Trust Passport (Part 2.10)

Takaful Fund (Part 4A)

Trade Memory Layer (Part 19)

Predictive Insights (Part 19)

Cross-border payment routing (Part 23)

Non-uniform layer stacking (Part 5.2)

24.1 Threat Model Methodology

24.1.1 Process Overview

SGTX applies a continuous, automated threat modelling process:

24.1.2 Threat Modelling Tooling

24.1.3 Asset Inventory (v11.0)

24.2 STRIDE Analysis Summary

24.2.1 STRIDE Categories

24.2.2 STRIDE Per Component (Detailed Matrix)

24.2.3 Example — Spoofing Mitigation for GTID

An attacker cannot impersonate a tenant because every request requires a JWT signed with the tenant's Ed25519 private key. ZITADEL WebAuthn enforces biometric liveness for high-value actions

(e.g., contract.sign). The JWT is short-lived (15 minutes) and rotated via refresh token.

Mitigation: JWT has short TTL (15 min). Refresh token requires biometric verification. All API calls validated against device registry (Part 1.14).

An attacker who gains write access to the `governor_decisions` table cannot modify a past decision because the Loom hash chain would break. The hourly verifier (`audit-chain-verifier`) detects any inconsistency and triggers a P0 incident.

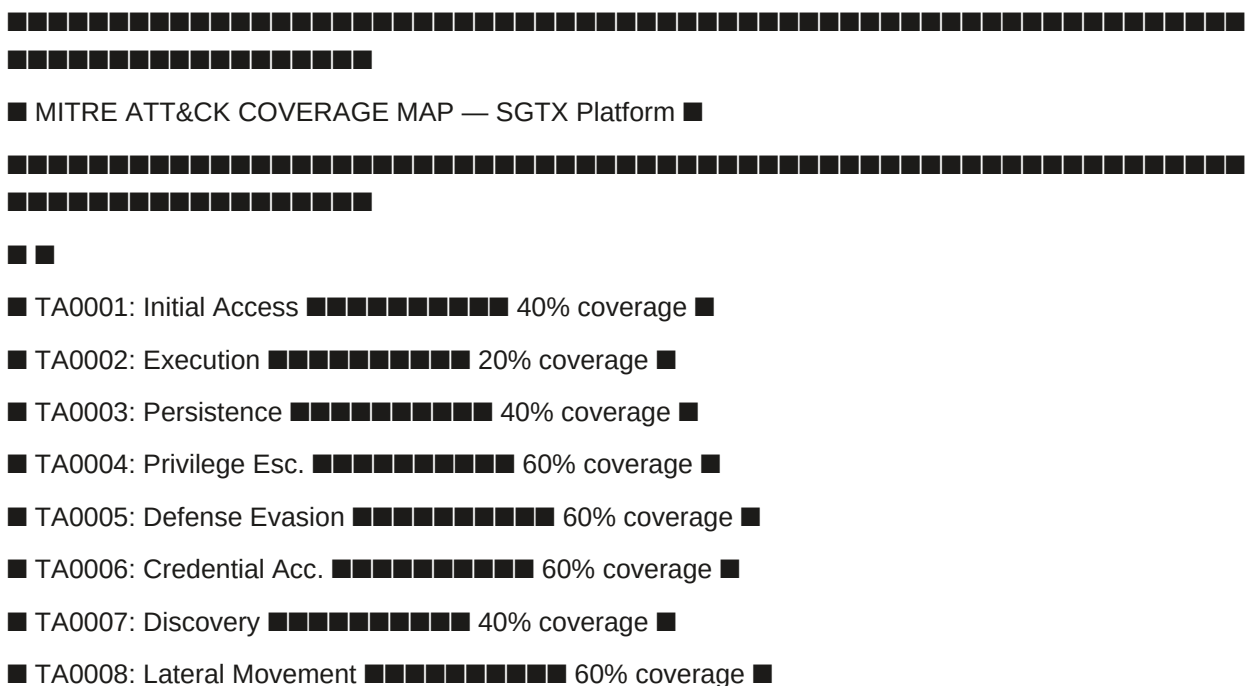
Mitigation: Loom hash chain breaks. Hourly verifier detects mismatch → P0 incident → Governor blocks new decisions until restored from WAL.

Row-Level Security ensures that a tenant can only see their own data. The Governor sets `app.current_tenant_gtid` in the session context, and PostgreSQL RLS enforces it at the database level. Even if an attacker bypasses the application layer, they cannot access other tenants' data.

Mitigation: PostgreSQL RLS denies access if `app.current_tenant_gtid` is not set or does not match. The Governor is the only service that can set this variable.

24.3.1 Tactic Mapping

24.3.2 MITRE ATT&CK Navigator View



Admin Portal notified. Manual override required to resolve hold.

P1 Incident — Deferred Payment Guarantee Expiry:

Customs clearance not confirmed before guarantee expiry. Governor freezes shipment.

Payment service attempts to charge payer. Groq summary: "Deferred payment guarantee for USTN ... expired. Immediate payment required."

P1 Incident — ZK Proof Verification Failure:

Financier submits ZK reserve proof that fails verification.

Governor rejects bid. Groq summary: "Reserve proof verification failed. Possible: insufficient reserves or proof tampering."

Financier notified, can retry with corrected proof.

P2 Incident — Sanctions Rescreening Hit:

RIA detects a new sanctions designation matching a tenant's UBO.

Governor suspends tenant's trading ability. Groq summary: "New sanctions designation affecting UBO of tenant ... Investigation required."

Compliance officer reviews and resolves.

24.7 Data Protection & Privacy

24.7.1 Data Classification

24.7.2 Data Encryption

24.7.3 Key Management

24.8 Zero-Cost Security Toolchain

All security tools are open-source and self-hosted:

No billing details required for any security tool. The platform can operate fully air-gapped (except for threat feed updates, which can be mirrored internally).

24.9 Governance & Compliance Gates

24.10 Database Schema Additions (Part 24)

```
sql
```

```
-- Incidents
```

```
CREATE TABLE incidents (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
severity TEXT NOT NULL, -- 'P0', 'P1', 'P2', 'P3'
```

```
title TEXT NOT NULL,
```

```
description TEXT,
```

```
status TEXT DEFAULT 'OPEN', -- 'OPEN', 'CONTAINED', 'RESOLVED', 'CLOSED'
```

```
started_at TIMESTAMPTZ,
```

```
resolved_at TIMESTAMPTZ,
```

```
root_cause TEXT,
```

```
groq_summary TEXT,
```

```
loom_hash TEXT,
```

```

governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Incident events (audit trail for incident response)
CREATE TABLE incident_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
incident_id UUID NOT NULL REFERENCES incidents(id) ON DELETE CASCADE,
event_type TEXT NOT NULL, -- 'DETECTED', 'CONTAINED', 'ESCALATED', 'RESOLVED',
'POST_MORTEM'
actor_gtid TEXT,
details JSONB,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Threat findings (extended)
CREATE TABLE threat_findings (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tool_name TEXT NOT NULL,
severity TEXT NOT NULL, -- 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'
title TEXT NOT NULL,
description TEXT,
remediation TEXT,
cve_id TEXT,
cvss_score DECIMAL(3,1),
exploit_available BOOLEAN DEFAULT false,
affected_component TEXT,
finding_hash TEXT,
incident_id UUID REFERENCES incidents(id) ON DELETE SET NULL,
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Security events (for SIEM-like aggregation)
CREATE TABLE security_events (
id BIGSERIAL,

```

```

event_type TEXT NOT NULL, -- 'AUTH_FAILURE', 'RLS_VIOLATION', 'RATE_LIMIT_EXCEEDED',
'CERT_EXPIRY', 'SUSPICIOUS_ACTIVITY', 'SANCTIONS_HIT'
severity TEXT NOT NULL, -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'
actor_gtid TEXT,
ip_address INET,
details JSONB,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (created_at);
SELECT create_hypertable('security_events', 'created_at');
-- Data classification audit
CREATE TABLE data_classification_audit (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
table_name TEXT NOT NULL,
column_name TEXT,
classification TEXT NOT NULL, -- 'PUBLIC', 'INTERNAL', 'CONFIDENTIAL', 'REGULATED',
'ANONYMISED'
justification TEXT,
reviewed_by UUID REFERENCES employees(id) ON DELETE SET NULL,
reviewed_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ
);
-- Security compliance reports
CREATE TABLE security_compliance_reports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
report_type TEXT NOT NULL, -- 'WEEKLY_PENTEST', 'MONTHLY_COMPLIANCE',
'QUARTERLY_RISK'
report_period TEXT NOT NULL, -- '2026-W24', '2026-06', '2026-Q2'
summary TEXT, -- Groq-generated
finding_count INTEGER,
critical_count INTEGER,
high_count INTEGER,
medium_count INTEGER,
low_count INTEGER,
pdf_url TEXT,
loom_hash TEXT,

```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Threat intelligence feeds cache
CREATE TABLE threat_intel_cache (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
feed_name TEXT NOT NULL,
ioc_type TEXT NOT NULL, -- 'IP', 'DOMAIN', 'HASH', 'URL', 'CVE'
ioc_value TEXT NOT NULL,
severity TEXT DEFAULT 'MEDIUM',
description TEXT,
source_url TEXT,
detected_at TIMESTAMPTZ,
expires_at TIMESTAMPTZ,
active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_incidents_status ON incidents(status);
CREATE INDEX idx_incidents_severity ON incidents(severity);
CREATE INDEX idx_incidents_started ON incidents(started_at DESC);
CREATE INDEX idx_incident_events_incident ON incident_events(incident_id);
CREATE INDEX idx_security_events_type ON security_events(event_type);
CREATE INDEX idx_security_events_severity ON security_events(severity);
CREATE INDEX idx_security_events_created ON security_events(created_at DESC);
CREATE INDEX idx_threat_findings_incident ON threat_findings(incident_id);
CREATE INDEX idx_threat_findings_severity ON threat_findings(severity);
CREATE INDEX idx_threat_findings_resolved ON threat_findings(resolved) WHERE resolved = false;
CREATE INDEX idx_threat_intel_cache_ioc ON threat_intel_cache(ioc_type, ioc_value);
CREATE INDEX idx_threat_intel_cache_active ON threat_intel_cache(active) WHERE active = true;

```

24.11 Implementation Checklist (Part 24)

24.11.1 Core Security Infrastructure

Deploy security monitoring stack (Prometheus, Grafana, Loki, Jaeger, Falco).

Configure Cilium network policies (allowlist only).

Enable pgaudit on PostgreSQL and ship logs to Loki.

Set up audit-chain-verifier as a cron job (hourly).

Deploy pentest-service with OWASP ZAP, nuclei, Trivy (staging environment).

24.11.2 Threat Intelligence & Anomaly Detection

Integrate threat intelligence feeds (AlienVault OTX, MISP) into threat-intel-agent.

Configure Prometheus Alertmanager to route alerts to Smart Inbox and on-call phone.

Deploy security-events table for SIEM-like aggregation.

Implement anomaly detection for security events (Isolation Forest, A2).

24.11.3 STRIDE & MITRE

Run initial STRIDE analysis (Rig agent) and baseline risk register.

Map controls to MITRE ATT&CK techniques.

Create threat findings database and reporting.

24.11.4 Incident Response

Test incident response playbook with a simulated Loom chain breach.

Implement incident lifecycle (detection → containment → analysis → eradication → recovery → post-mortem).

Add Groq post-mortem generation.

24.11.5 Data Protection

Classify all data (PUBLIC, INTERNAL, CONFIDENTIAL, REGULATED, ANONYMISED).

Implement field-level encryption for personal data.

Set up data retention and deletion policies (Part 18).

24.11.6 v11.0 Specific

Add tests for dual-mode context enforcement (OPA unit tests).

Add tests for conditional QC hold blocking settlement.

Add tests for deferred payment guarantee expiry handling.

Add ZK proof verification tests (Plonky3).

Add Trust Passport revocation test.

Add Takaful claim approval workflow tests.

Add cross-border payment routing jurisdiction tests.

Add Trade Memory opt-out enforcement tests.

24.11.7 Reporting & Compliance

Generate first weekly security report using Groq summariser.

Set up compliance report generation (weekly pentest, monthly compliance, quarterly risk).

Create Admin Portal security dashboard.

24.12 AI Authority Summary for Part 24

END OF PART 24 — THREAT MODEL & ATTACK SURFACE ANALYSIS (v11.0 FULLY EXPANDED)

PART 24 — THREAT MODEL & ATTACK SURFACE ANALYSIS (STRIDE + MITRE)

24.0 Purpose & Design Philosophy

The SGTX platform handles sensitive trade data, financial transactions, and cryptographic identities across multiple jurisdictions. A robust security model is not optional — it is a constitutional requirement. This part defines the comprehensive threat model for the SGTX platform using industry-standard frameworks:

STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) — per component, per trust boundary.

MITRE ATT&CK — adversary tactics, techniques, and procedures (TTPs) relevant to trade platforms.

All mitigations are zero-cost, built on open-source tools (OPA, WasmEdge, Cilium, Falco, etc.) and free threat intelligence feeds. AI (Groq, Hugging Face) is used for advisory purposes — e.g., summarising threat findings, generating post-mortems, and suggesting remediation steps. Never autonomous blocking (A5 forbidden).

AI Integration in Part 24:

Key principles (non-marketplace, sovereign):

Zero-trust architecture — no implicit trust; every request is authenticated, authorised, and encrypted.

Defence in depth — multiple layers of control (network, application, data, constitutional).

Immutable audit — every security-relevant event is logged immutably via Loom.

Continuous verification — hourly Loom chain verification, weekly pentests, daily vulnerability scans.

No security-through-obscurity — all security controls are documented and verifiable.

Scope: This threat model covers all v11.0 features including:

Multi-shipment contracts (Part 3.6)

Dual-mode toggle (Part 2.8)

Deferred payment (Part 6.8)

Conditional QC (Part 3B.6.4)

ZK Proofs (Part 11.7)

Trust Passport (Part 2.10)

Takaful Fund (Part 4A)

Trade Memory Layer (Part 19)

Predictive Insights (Part 19)

Cross-border payment routing (Part 23)

Non-uniform layer stacking (Part 5.2)

24.1 Threat Model Methodology

24.1.1 Process Overview

SGTX applies a continuous, automated threat modelling process:

24.1.2 Threat Modelling Tooling

24.1.3 Asset Inventory (v11.0)

24.2 STRIDE Analysis Summary

24.2.1 STRIDE Categories

24.2.2 STRIDE Per Component (Detailed Matrix)

24.2.3 Example — Spoofing Mitigation for GTID

An attacker cannot impersonate a tenant because every request requires a JWT signed with the tenant's Ed25519 private key. ZITADEL WebAuthn enforces biometric liveness for high-value actions (e.g., contract.sign). The JWT is short-lived (15 minutes) and rotated via refresh token.

Attack Scenario: Attacker steals a JWT token.

Mitigation: JWT has short TTL (15 min). Refresh token requires biometric verification. All API calls validated against device registry (Part 1.14).

24.2.4 Example — Tampering Mitigation for Loom

An attacker who gains write access to the governor_decisions table cannot modify a past decision because the Loom hash chain would break. The hourly verifier (audit-chain-verifier) detects any inconsistency and triggers a P0 incident.

Attack Scenario: Attacker modifies a Governor decision.

Mitigation: Loom hash chain breaks. Hourly verifier detects mismatch → P0 incident → Governor blocks new decisions until restored from WAL.

24.2.5 Example — Information Disclosure Mitigation for RLS

Row-Level Security ensures that a tenant can only see their own data. The Governor sets app.current_tenant_gtid in the session context, and PostgreSQL RLS enforces it at the database level. Even if an attacker bypasses the application layer, they cannot access other tenants' data.

Attack Scenario: Attacker crafts SQL query directly against PostgreSQL.

Mitigation: PostgreSQL RLS denies access if app.current_tenant_gtid is not set or does not match. The Governor is the only service that can set this variable.

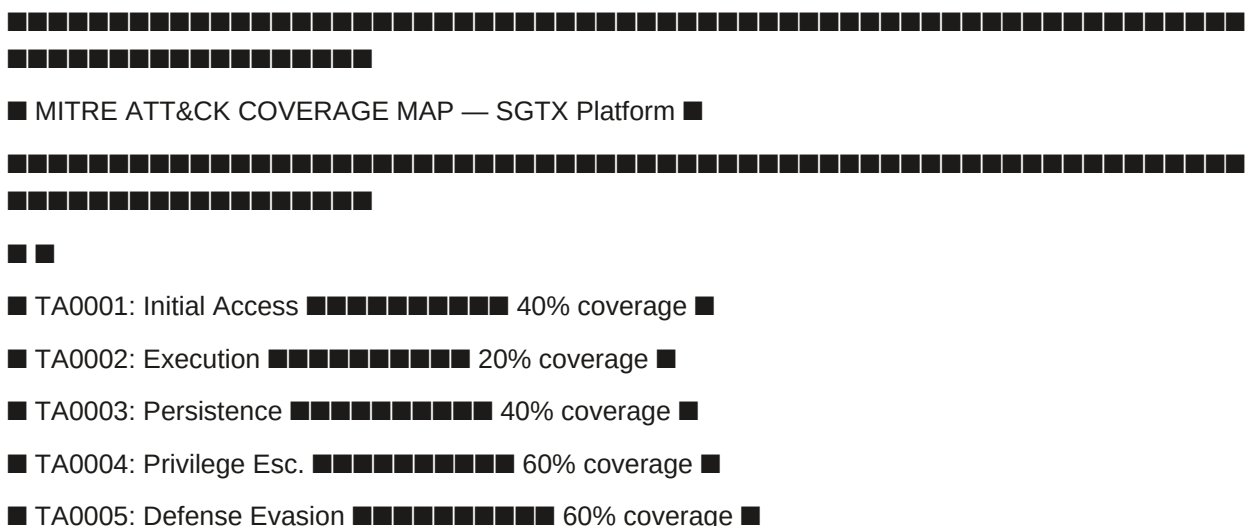
24.3 MITRE ATT&CK Coverage (High-Priority Tactics)

24.3.1 Tactic Mapping

The following table maps SGTX components to MITRE ATT&CK tactics and techniques, with specific mitigations. The matrix is maintained as a JSON file in the repository and used by the threat-scanner to validate controls. New v11.0 techniques are marked with ★.

24.3.2 MITRE ATT&CK Navigator View

text



■ TA0006: Credential Acc. ■■■■■■■■■■ 60% coverage ■

■ TA0008: Lateral Movement ■■■■■■■■■■ 60% coverage ■

■ TA0010: Exfiltration ■■■■■■■■■■ 60% coverage ■

11

■ Gap analysis: 10 techniques identified, 6 mitigation controls in progress ■

24.3.3 AI-Assisted Threat Intelligence (Advisory)

24.4 Attack Surface Inventory (v11.0)

24.5 Continuous Security Controls & Verification

24.6 Incident Response & Forensics

P0 Incident — Loom Chain Breach:

P1 Incident — Conditional QC Hold Deadline Missed:

Action plan deadline passes; hold remains active. workflow-recovery-service escalates.

Seller receives final warning. Groq summary: "Seller missed action plan deadline for USTN ..."

Admin Portal notified. Manual override required to resolve hold.

P1 Incident — Deferred Payment Guarantee Expiry:

Customs clearance not confirmed before guarantee expiry. Governor freezes shipment.

Payment service attempts to charge payer. Groq summary: "Deferred payment guarantee for USTN ... expired. Immediate payment required."

P1 Incident — ZK Proof Verification Failure:

Financier submits ZK reserve proof that fails verification.

Governor rejects bid. Groq summary: "Reserve proof verification failed. Possible: insufficient reserves or proof tampering."

Financier notified, can retry with corrected proof.

P2 Incident — Sanctions Rescreening Hit:

RIA detects a new sanctions designation matching a tenant's UBO.

Governor suspends tenant's trading ability. Groq summary: "New sanctions designation affecting UBO of tenant ... Investigation required."

Compliance officer reviews and resolves.

24.7 Data Protection & Privacy

24.7.1 Data Classification

24.7.2 Data Encryption

24.7.3 Key Management

24.8 Zero-Cost Security Toolchain

All security tools are open-source and self-hosted:

No billing details required for any security tool. The platform can operate fully air-gapped (except for threat feed updates, which can be mirrored internally).

24.9 Governance & Compliance Gates

24.10 Database Schema Additions (Part 24)

```
sql
```

```
-- Incidents
```

```
CREATE TABLE incidents (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
severity TEXT NOT NULL, -- 'P0', 'P1', 'P2', 'P3'
```

```
title TEXT NOT NULL,
```

```
description TEXT,
```

```
status TEXT DEFAULT 'OPEN', -- 'OPEN', 'CONTAINED', 'RESOLVED', 'CLOSED'
```

```
started_at TIMESTAMPTZ,
```

```
resolved_at TIMESTAMPTZ,
```

```

root_cause TEXT,
groq_summary TEXT,
loom_hash TEXT,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Incident events (audit trail for incident response)
CREATE TABLE incident_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
incident_id UUID NOT NULL REFERENCES incidents(id) ON DELETE CASCADE,
event_type TEXT NOT NULL, -- 'DETECTED', 'CONTAINED', 'ESCALATED', 'RESOLVED',
'POST_MORTEM'
actor_gtid TEXT,
details JSONB,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Threat findings (extended)
CREATE TABLE threat_findings (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tool_name TEXT NOT NULL,
severity TEXT NOT NULL, -- 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'
title TEXT NOT NULL,
description TEXT,
remediation TEXT,
cve_id TEXT,
cvss_score DECIMAL(3,1),
exploit_available BOOLEAN DEFAULT false,
affected_component TEXT,
finding_hash TEXT,
incident_id UUID REFERENCES incidents(id) ON DELETE SET NULL,
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Security events (for SIEM-like aggregation)

```

```

CREATE TABLE security_events (
id BIGSERIAL,

event_type TEXT NOT NULL, -- 'AUTH_FAILURE', 'RLS_VIOLATION', 'RATE_LIMIT_EXCEEDED',
'CERT_EXPIRY', 'SUSPICIOUS_ACTIVITY', 'SANCTIONS_HIT'

severity TEXT NOT NULL, -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'

actor_gtid TEXT,

ip_address INET,

details JSONB,

loom_hash TEXT,

created_at TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (created_at);

SELECT create_hypertable('security_events', 'created_at');

-- Data classification audit

CREATE TABLE data_classification_audit (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

table_name TEXT NOT NULL,

column_name TEXT,

classification TEXT NOT NULL, -- 'PUBLIC', 'INTERNAL', 'CONFIDENTIAL', 'REGULATED',
'ANONYMISED'

justification TEXT,

reviewed_by UUID REFERENCES employees(id) ON DELETE SET NULL,

reviewed_at TIMESTAMPTZ DEFAULT NOW(),

expires_at TIMESTAMPTZ
);

-- Security compliance reports

CREATE TABLE security_compliance_reports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

report_type TEXT NOT NULL, -- 'WEEKLY_PENTEST', 'MONTHLY_COMPLIANCE',
'QUARTERLY_RISK'

report_period TEXT NOT NULL, -- '2026-W24', '2026-06', '2026-Q2'

summary TEXT, -- Groq-generated

finding_count INTEGER,

critical_count INTEGER,

high_count INTEGER,

medium_count INTEGER,

low_count INTEGER,

```

```

pdf_url TEXT,
loom_hash TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Threat intelligence feeds cache
CREATE TABLE threat_intel_cache (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
feed_name TEXT NOT NULL,
ioc_type TEXT NOT NULL, -- 'IP', 'DOMAIN', 'HASH', 'URL', 'CVE'
ioc_value TEXT NOT NULL,
severity TEXT DEFAULT 'MEDIUM',
description TEXT,
source_url TEXT,
detected_at TIMESTAMPTZ,
expires_at TIMESTAMPTZ,
active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_incidents_status ON incidents(status);
CREATE INDEX idx_incidents_severity ON incidents(severity);
CREATE INDEX idx_incidents_started ON incidents(started_at DESC);
CREATE INDEX idx_incident_events_incident ON incident_events(incident_id);
CREATE INDEX idx_security_events_type ON security_events(event_type);
CREATE INDEX idx_security_events_severity ON security_events(severity);
CREATE INDEX idx_security_events_created ON security_events(created_at DESC);
CREATE INDEX idx_threat_findings_incident ON threat_findings(incident_id);
CREATE INDEX idx_threat_findings_severity ON threat_findings(severity);
CREATE INDEX idx_threat_findings_resolved ON threat_findings(resolved) WHERE resolved = false;
CREATE INDEX idx_threat_intel_cache_ioc ON threat_intel_cache(ioc_type, ioc_value);
CREATE INDEX idx_threat_intel_cache_active ON threat_intel_cache(active) WHERE active = true;

```

24.11 Implementation Checklist (Part 24)

24.11.1 Core Security Infrastructure

Deploy security monitoring stack (Prometheus, Grafana, Loki, Jaeger, Falco).

Configure Cilium network policies (allowlist only).

Enable pgaudit on PostgreSQL and ship logs to Loki.

Set up audit-chain-verifier as a cron job (hourly).

Deploy pentest-service with OWASP ZAP, nuclei, Trivy (staging environment).

24.11.2 Threat Intelligence & Anomaly Detection

Integrate threat intelligence feeds (AlienVault OTX, MISP) into threat-intel-agent.

Configure Prometheus Alertmanager to route alerts to Smart Inbox and on-call phone.

Deploy security-events table for SIEM-like aggregation.

Implement anomaly detection for security events (Isolation Forest, A2).

24.11.3 STRIDE & MITRE

Run initial STRIDE analysis (Rig agent) and baseline risk register.

Map controls to MITRE ATT&CK techniques.

Create threat findings database and reporting.

24.11.4 Incident Response

Test incident response playbook with a simulated Loom chain breach.

Implement incident lifecycle (detection → containment → analysis → eradication → recovery → post-mortem).

Add Groq post-mortem generation.

24.11.5 Data Protection

Classify all data (PUBLIC, INTERNAL, CONFIDENTIAL, REGULATED, ANONYMISED).

Implement field-level encryption for personal data.

Set up data retention and deletion policies (Part 18).

24.11.6 v11.0 Specific

Add tests for dual-mode context enforcement (OPA unit tests).

Add tests for conditional QC hold blocking settlement.

Add tests for deferred payment guarantee expiry handling.

Add ZK proof verification tests (Plonky3).

Add Trust Passport revocation test.

Add Takaful claim approval workflow tests.

Add cross-border payment routing jurisdiction tests.

Add Trade Memory opt-out enforcement tests.

24.11.7 Reporting & Compliance

Generate first weekly security report using Groq summariser.

Set up compliance report generation (weekly pentest, monthly compliance, quarterly risk).

Create Admin Portal security dashboard.

24.12 AI Authority Summary for Part 24

END OF PART 24 — THREAT MODEL & ATTACK SURFACE ANALYSIS (v11.0 FULLY EXPANDED)

PART 25 — SLA & UPTIME GUARANTEES

25.0 Purpose & Design Philosophy

This part defines the service level agreements (SLAs) that govern the SGTx platform's production environment. These commitments cover availability, latency, recovery time objectives (RTO), and recovery point objectives (RPO) for all critical services that directly affect a tenant's ability to initiate, track, settle, or manage trades, financing agreements, distressed cargo, or disputes. The SLAs are contractually binding under the Master Service Agreement and are monitored entirely by self-hosted, zero-cost open-source tools (Prometheus, Grafana, Loki, Blackbox exporter, custom sla-monitor in Rust). No external monitoring service or paid APM is used.

AI assistance (advisory only):

Key principles (non-marketplace, orchestration-only):

Transparency — All SLAs are published on a public status page (<https://status.sgtx.io>) and accessible via API.

Accountability — SLA credits are automatically calculated and applied to tenant statements; no manual claim required.

Zero-cost monitoring — All probes run on the same bare-metal infrastructure; no cloud dependencies.

Continuous improvement — AI-driven post-mortems and predictive scaling reduce downtime over time.

Scope: All SLAs apply to the production environment (minimum three sovereign nodes, active-active). Development and sandbox environments have no SLA. For multi-shipment contracts, each shipment's services are monitored independently; a delay in one shipment does not affect SLA for others. Conditional QC holds and deferred payment guarantees have their own SLAs defined in this part.

25.1 Core Platform SLAs (Production)

The following SLAs apply to each sovereign node region (e.g., Cairo, Dubai, Frankfurt). Overall availability is measured per region; tenants are routed to the nearest healthy region via Anycast DNS.

25.1.1 Core Services

25.1.2 v11.0 New Services

25.1.3 Definitions

25.1.4 Example — Governor Service Downtime Calculation

Month: 30 days = 43,200 minutes.

Planned maintenance: 120 minutes (announced, excluded from SLA).

Unplanned downtime: 40 minutes.

Availability = $(43,200 - 120 - 40) / (43,200 - 120) = 42,920 / 43,080 = 99.63\% \rightarrow$ below 99.95% $\rightarrow$ credit triggered.

Credit Calculation: 10% of monthly platform fee (e.g., \$50 for a \$500 monthly fee).

25.2 Conditional QC & Deferred Payment SLAs (v11.0)

These new components have specific SLA requirements aligned with real-world trade processes.

25.2.1 Conditional QC Hold Resolution SLA

Example: Seller completes action plan at 14:00 UTC and clicks "Mark Action Completed". Inspector must verify within 1 hour (SLA for the platform to process the verification and remove hold). If verification is delayed beyond 1h due to platform error, credit applies.

Credit: 5% of monthly platform fee if breached.

25.2.2 Deferred Payment Guarantee Expiry SLA

Example: Guarantee expires at 23:59 UTC. System must detect expiry and notify payer by 00:05 UTC next day. If detection is delayed beyond 5 minutes, credit applies.

Credit: 5% of monthly platform fee if breached.

25.2.3 Accelerated Outreach Notification SLA

Credit: 2% of monthly platform fee if breached.

25.3 Predictive SLAs & AI-Assisted Forecasting (Advisory)

The platform uses AI models (Groq + HF local) to forecast potential SLA breaches before they occur. These forecasts are advisory — they never automatically throttle or block traffic.

25.3.1 Predictive Availability Model (A2, HF local LSTM)

Example Alert: "Predicted 35% chance of Governor service degradation in the next 24h due to planned cluster upgrade. Consider postponing or adding replicas."

25.3.2 Predictive Latency Model (A1, Groq)

Groq analyses current traffic patterns and resource headroom, then generates a plain-language report:

"Based on current load (120% of normal), p95 latency may exceed 3 seconds in 8 hours if no scaling action is taken. Recommended: add 2 replicas of trade-service." (Advisory — no auto-scaling unless configured.)

25.3.3 SLA Credit Forecast (A1, Groq)

At the start of each month, Groq forecasts the likelihood of SLA breaches based on historical data and planned changes.

Example: "This month, estimated SLA breach probability: 2% for Governor, 1% for Payment Orchestrator. No action required. Projected SLA credits: \$1,200 (0.12% of revenue)."

25.4 Maintenance Windows

25.4.1 Standard Maintenance

25.4.2 Emergency Maintenance

Example — Emergency Maintenance due to Log4j-like vulnerability:

Zero-day disclosed at 10:00 UTC. Within 2 hours, SGTX patches the affected service. Notifications sent at 12:00 UTC. Maintenance window 14:00-15:00 UTC (1 hour). Not counted against SLA because it was a third-party zero-day and notice was given.

25.5 Status Page & Incident Response

25.5.1 Public Status Page

URL: <https://status.sgtx.io>

Technology: Built with cState (open-source) or Uptime (GitHub-based). Self-hosted on a separate lightweight VPS, independent of main SGTX nodes.

Displays:

Realtime component status: Operational (■), Degraded Performance (■), Partial Outage(■), Major Outage (■)

Incident history for last 90 days, including post-mortem summaries (Groq-generated, reviewed by Platform Governance Authority)

Upcoming scheduled maintenance

Per-component uptime percentage for current and previous month

SLA credit calculator — tenants can estimate potential credits based on current month's uptime

API endpoint for programmatic status checks: GET /api/v1/status

Subscriptions: RSS, email, Slack webhook (optional). All free.

25.5.2 Incident Response Process

25.5.3 Incident Post-Mortem Example (Groq Generated)

text

Title: Governor Service Latency Spike — 2026-06-07

Incident ID: INC-2026-042

Severity: P1

Duration: 22 minutes (03:12—03:34 UTC)

Impact: 12% of requests experienced p95 latency >800ms (up to 1.2s).

Affected tenants: 43 (all EU region). No data loss.

SLA credit: 0.5% of monthly fee (breach of 99.95% target).

Root cause: Sudden traffic spike from a large exporter (100+ concurrent trade requests). Governor replicas at 85% CPU.

Resolution: Increased replica count from 3 to 5; added HPA rule to scale at 70% CPU. Traffic normalised by 03:34 UTC.

Prevention: Predictive auto-scaling model updated to include trade volume forecasts. Alert threshold for CPU lowered from 80% to 65%.

Recommendations:

1. Review traffic patterns from large exporters.
2. Consider sharding trade-service by tenant.

25.6 Credit & Compensation Model

25.6.1 Calculation

25.6.2 Automatic Application

sla-monitor runs at the end of each month, aggregates downtime per component, computes credits.

Credits are automatically applied to the tenant's next monthly statement (line item: "SLA credit — Governor breach").

Tenants receive a Smart Inbox notification with the breakdown.

No manual claim required — but tenants can dispute the calculation via the "SLA Dispute" button in Company Admin (dispute handled by Platform Governance Authority).

25.6.3 Exclusions (No Credit)

25.7 Zero-Cost Monitoring & Enforcement

25.7.1 Monitoring Stack

25.7.2 sla-monitor Implementation Details

25.7.3 AI-Assisted Health Prediction (Advisory)

The predictive-sla service (A2, LSTM) uses historical probe data to forecast future availability. Groq generates a monthly report:

"Based on current trends, Governor service availability next month is forecast at 99.97% (above target). No action needed."

25.8 Example SLA Incident Flow (Complete)

Scenario: Network partition between Cairo and Dubai nodes causes Governor service partially unavailable for EU users routed to Frankfurt. External probes detect 5xx errors for 12 consecutive minutes.

Step 1 — Detection (03:12 UTC)

External probe in London fails 4 of 5 requests to Governor on Frankfurt node. Alertmanager fires: "CRITICAL: Governor service — 80% failure rate on Frankfurt node for 5 min."

AIOps agent detects NATS connection flapping, cordons Frankfurt pods, redirects traffic to Dubai node. Service restored within 2 min (03:14 UTC).

Step 2 — Triage (03:14 UTC)

On-call engineer acknowledges alert via mobile app, investigates root cause (faulty switch). Total downtime: 12 min (0.028% of monthly minutes). 99.90% availability SLA for Governor (target 99.95%) breached.

Step 3 — Resolution & Post-mortem (03:30 UTC)

Network switch replaced. All traffic restored.

sla-monitor records downtime, computes credit. End of month, tenant statement shows credit of 10% of monthly fee (approx \$50 for a \$500 monthly fee).

Post-mortem generator (Groq) drafts summary:

"Governor service experienced 12-minute partial outage on Frankfurt node due to network hardware failure. Automatic failover restored service in 2 min. Root cause: faulty switch. Mitigation: additional network redundancy ordered. Credits issued to affected tenants: \$50 per tenant."

Step 4 — Tenant Notification

Affected tenants receive Smart Inbox item (priority 85): "On 14 Jun, Governor service was partially unavailable for 12 min. Your trade was not impacted. A credit of \$50 has been applied to your next statement. [\[View Incident Report\]](#)"

Step 5 — Follow-up

Platform Governance Authority reviews post-mortem, publishes to status page. No further action required.

25.9 Regional & Jurisdictional SLA Variations

25.9.1 Egypt (CBE Requirements)

25.9.2 EU (GDPR & CBAM)

25.9.3 UAE (DIFC)

25.9.4 United Kingdom

25.9.5 India (DPDP Act)

25.10 Database Schema Additions (Part 25)

sql

-- SLA monitoring data

```
CREATE TABLE sla_metrics (  
  id BIGSERIAL PRIMARY KEY,  
  component TEXT NOT NULL,  
  region TEXT NOT NULL,  
  availability DECIMAL(6,4),  
  uptime_seconds BIGINT,  
  downtime_seconds BIGINT,  
  maintenance_excluded_seconds BIGINT,  
  target_availability DECIMAL(6,4),  
  sla_breached BOOLEAN DEFAULT false,  
  measurement_start TIMESTAMPTZ,  
  measurement_end TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- SLA credits applied

```
CREATE TABLE sla_credits (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,  
  component TEXT NOT NULL,  
  breach_percentage DECIMAL(5,2),  
  credit_amount DECIMAL(20,4),  
  currency TEXT DEFAULT 'USD',  
  period_start TIMESTAMPTZ,  
  period_end TIMESTAMPTZ,  
  status TEXT DEFAULT 'PENDING', -- 'PENDING', 'APPLIED', 'DISPUTED'  
  applied_at TIMESTAMPTZ,  
  disputed_at TIMESTAMPTZ,  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- SLA credit disputes

```

CREATE TABLE sla_disputes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
credit_id UUID NOT NULL REFERENCES sla_credits(id) ON DELETE CASCADE,
tenant_gtid TEXT NOT NULL,
dispute_reason TEXT NOT NULL,
status TEXT DEFAULT 'PENDING', -- 'PENDING', 'RESOLVED', 'REJECTED'
resolution TEXT,
resolved_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Predictive SLA forecasts
CREATE TABLE sla_forecasts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
component TEXT NOT NULL,
region TEXT NOT NULL,
forecast_date DATE NOT NULL,
predicted_availability DECIMAL(6,4),
confidence DECIMAL(5,2),
risk_factors JSONB,
recommendation TEXT, -- Groq-generated
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Maintenance windows
CREATE TABLE maintenance_windows (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
component TEXT NOT NULL,
region TEXT,
start_time TIMESTAMPTZ NOT NULL,
end_time TIMESTAMPTZ NOT NULL,
reason TEXT NOT NULL,
status TEXT DEFAULT 'SCHEDULED', -- 'SCHEDULED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'
announced_at TIMESTAMPTZ,
approved_by UUID REFERENCES employees(id) ON DELETE SET NULL,

```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Status page events
CREATE TABLE status_page_events (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
event_type TEXT NOT NULL, -- 'INCIDENT_CREATED', 'INCIDENT_UPDATED',
'MAINTENANCE_SCHEDULED', 'COMPONENT_STATUS_CHANGED'
component TEXT,
status TEXT, -- 'OPERATIONAL', 'DEGRADED', 'PARTIAL_OUTAGE', 'MAJOR_OUTAGE'
description TEXT,
started_at TIMESTAMPTZ,
resolved_at TIMESTAMPTZ,
published BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- SLA probe results (raw, for analysis)
CREATE TABLE sla_probes (
id BIGSERIAL PRIMARY KEY,
component TEXT NOT NULL,
region TEXT NOT NULL,
probe_location TEXT NOT NULL, -- 'CAIRO', 'DUBAI', 'FRANKFURT', 'LONDON'
success BOOLEAN NOT NULL,
latency_ms INTEGER,
status_code INTEGER,
error_message TEXT,
probed_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_sla_metrics_component ON sla_metrics(component);
CREATE INDEX idx_sla_metrics_period ON sla_metrics(measurement_start, measurement_end);
CREATE INDEX idx_sla_credits_tenant ON sla_credits(tenant_gtid);
CREATE INDEX idx_sla_credits_status ON sla_credits(status);
CREATE INDEX idx_sla_forecasts_component ON sla_forecasts(component);
CREATE INDEX idx_maintenance_windows_active ON maintenance_windows(start_time, end_time)
WHERE status != 'COMPLETED';
CREATE INDEX idx_status_page_events_created ON status_page_events(created_at DESC);

```

CREATE INDEX idx_sla_probes_component ON sla_probes(component);

CREATE INDEX idx_sla_probes_probed_at ON sla_probes(probed_at DESC);

25.11 Implementation Checklist (Part 25)

25.11.1 Monitoring Infrastructure

Deploy sla-monitor (Rust) on a separate node.

Configure Blackbox exporter to probe all service health endpoints from ≥ 2 external locations.

Set up Prometheus Alertmanager rules for SLA breaches (e.g., `sgtx_availability < 0.9995`).

Integrate sla-monitor with ClickHouse for long-term storage.

Build SLA dashboard in Grafana (uptime trends, credit history, latency heatmaps).

25.11.2 Credit & Compensation

Implement credit calculation logic in payment-orchestrator (add line item to monthly statement).

Add Smart Inbox notification for credit issuance.

Implement SLA dispute workflow.

25.11.3 Status Page

Deploy status page (cState) and configure API to pull current status from Prometheus.

Set up incident post-mortem automation (Groq + template).

Configure RSS, email, Slack webhook subscriptions.

25.11.4 Predictive SLAs

Train predictive SLA model (LSTM) using historical probe data.

Implement predictive availability forecast.

Add Groq-generated SLA credit forecast (monthly).

25.11.5 Maintenance Windows

Implement maintenance window scheduling.

Add announcement mechanism (Smart Inbox, email, status page).

Exclude maintenance windows from availability calculations.

25.11.6 v11.0 Specific

Add SLA monitoring for conditional QC holds (track hold removal latency).

Add SLA monitoring for deferred payment guarantee expiry detection.

Add alerting for multi-shipment per-shipment coordinator failures (separate metrics per shipment).

Add SLA monitoring for Takaful fund claim processing.

Add SLA monitoring for Trust Passport verification.

Add SLA monitoring for cross-border payment routing.

Add SLA monitoring for country tiering service.

25.11.7 Testing

Test incident response with simulated network partition (chaos engineering).

Test SLA credit calculation and application.

Test SLA dispute workflow.

Test status page integration.

Test maintenance window exclusion.

25.12 AI Authority Summary for Part 25

END OF PART 25 — SLA & UPTIME GUARANTEES (v11.0 FULLY EXPANDED)

PART 26 — GLOSSARY & ACRONYMS

26.0 Purpose & Design Philosophy

This glossary defines every key term, acronym, and concept used throughout the SGTX blueprint. It is a living document, updated with each release. Terms are presented in alphabetical order for easy reference. New v11.0 terms are marked with ★ .

The glossary serves multiple audiences:

New users — quick reference for unfamiliar terms.

Developers — consistent terminology across code, documentation, and APIs.

Compliance officers — clear definitions for regulatory filings.

Implementers — accurate understanding of system components.

Structure: Each term includes:

Definition — concise, plain-language explanation.

Example / Context — how the term is used in SGTX.

Cross-reference — related parts of the blueprint.

26.1 Core Terms (A–Z)

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

Y

Z

26.2 Acronyms (Selected)

END OF PART 26 — GLOSSARY & ACRONYMS (v11.0 FULLY EXPANDED)

PART 27 — OPENAPI 3.1 SPECIFICATION INDEX

27.0 Overview

All SGTx public and partner endpoints are documented in OpenAPI 3.1 format. The specification files are served from `/api/v1/openapi.json` (public) and `/api/v1/partner/openapi.json` (partner). The interactive Swagger UI is available to verified tenants only. This index lists every endpoint, grouped by service, with a summary description and, where applicable, a brief example or note. Endpoints new or modified in v11.0 are marked with ★.

Base URL: <https://api.sgtx.io/v1>

Authentication: All endpoints (except public health and verification endpoints) require a valid JWT obtained from ZITADEL. The JWT must include:

`tenant_gtid` — the authenticated tenant's GTID

`employee_id` — the authenticated employee's UUID

`active_trader_mode_context` — BUY, SELL, or DUAL

`permissions` — array of permission strings for RBAC

Governor Gating: Every endpoint that modifies state passes through the Governor (Part 1.1). The Governor evaluates OPA policies, WasmEdge constitutional rules, and AI consult (A1–A3) before allowing the request.

Rate Limits: All endpoints have rate limits applied per tenant:

Standard endpoints: 100 requests per minute

Partner API endpoints: Variable per partner (configured in `partner_rate_limits`)

Public verification endpoints: 10 requests per minute per IP

Idempotency: All POST, PUT, and PATCH endpoints support an optional Idempotency-Key header (UUID v4). The platform stores the first response for 24 hours; replaying the same key returns the original response.

Standard Error Responses:

27.1 Governor & Platform Endpoints

27.2 Trade & Quote Endpoints
27.3 Packing & Containerisation Endpoints
27.4 Contract & Fee Endpoints
27.5 Financing Endpoints (Including Co-Financing and Financier Preferences)
27.6 Shipment & Milestone Endpoints
27.7 Settlement & Payment Orchestrator Endpoints
27.8 Distressed Cargo Endpoints
27.9 Dispute Endpoints
27.10 Network & Saved Contacts Endpoints
27.11 Marketplace Partner Endpoints
27.12 KYB/KYC & Onboarding Endpoints
27.13 Country & Jurisdiction Endpoints
27.14 Admin & Observability Endpoints (Internal)
27.15 Inbox & Preferences Endpoints
27.16 Tenant Data Export & Statement Endpoints
27.17 Trade Command Center Endpoint
27.18 Dual-Mode & Employee Context Endpoint
27.19 Universal Search Endpoint
27.20 Trust Passport Endpoints
27.21 Container Release API Endpoints
27.22 Takaful Fund Endpoints
27.23 Suspicious Activity Report (SAR) Endpoints
27.24 PQC & ZK Proof Endpoints
27.25 Voice & AI Assistant Endpoints
END OF PART 27 — OPENAPI 3.1 SPECIFICATION INDEX (v11.0 FULLY EXPANDED)

PART 28 — APPENDICES (v11.0 FULLY EXPANDED)

28.0 Purpose & Design Philosophy

This part contains all supporting reference material for the SGTX blueprint. The appendices provide detailed specifications, reference tables, and quick-reference guides that are referenced throughout the main document. They are designed to be used as standalone reference material for developers, implementers, and operators.

Contents:

A1: USTN Format Specification (Company-Anchored, No Version)

A1.1 Format

The USTN (Unique SGTX Transaction Number) is generated at contract lock (single-shipment) or per-shipment lock (multi-shipment). It incorporates the identities of the involved companies to prevent replay and ensure non-repeatability.

Format:

text

SGTX-{COUNTRY}-{YEAR}-{TRADER}-{SEQ}

Alphabet for SEQ: ABCDEFGHJKLMNPQRSTUVWXYZ23456789 (34 chars — excludes I, O, 0, 1 to avoid confusion).

Total length: $4 + 1 + 6 + 1 + 6 + 1 + 14 + 1 + 8 = 42$ characters.

Example: SGTX-EG-26-F3A-1

A1.2 Generation Algorithm

rust

```
use chrono::Utc;
```

```
use rand::Rng;
```

```
const ALPHABET: &[u8] = b"ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789";
```

```
fn generate_ustn(buyer_gtid: &str, seller_gtid: &str) -> String {
```

```
    let buyer_suffix = extract_suffix(buyer_gtid);
```

```
    let seller_suffix = extract_suffix(seller_gtid);
```

```
    let timestamp = Utc::now().format("%Y%m%d%H%M%S").to_string();
```

```
    let mut rng = rand::thread_rng();
```

```
    let SEQ: String = (0..8).map(|_| ALPHABET[rng.gen_range(0..ALPHABET.len())] as char).collect();
```

```
    format!("SGTX-{}-{}-{}-{}", buyer_suffix, seller_suffix, timestamp, SEQ)
```

```
}
```

```
fn extract_suffix(gtid: &str) -> String {
```

```
    let cleaned: String = gtid.chars().filter(|c| *c != '-').collect();
```

```
    let len = cleaned.len();
```

```
    cleaned[(len-6)..].to_string()
```

```
}
```

A1.3 Validation Rules

A1.4 MicroUSTN (Distressed Partial Sale)

For distressed partial sales (Part 3B.7), a microUSTN is generated:

Example:

Original USTN: SGTX-EG-26-F3A-1

MicroUSTN: SGTX-1397F3A-456ABC-20260420150000-D4E5F6G7

A1.5 Multi-Shipment USTNs

For multi-shipment contracts (Part 3.6), each shipment has its own USTN:

The master contract has a contract ID (e.g., MC-20260415-001) but no master USTN.

A2: Incoterms 2020 Reference Table (Simplified for Cost Rules)

A2.1 Incoterm Applicability Matrix

A2.2 Mandatory Logistics Services by Incoterm

A2.3 Incoterm Validation Engine (WasmEdge)

The `incoterms_engine.wasm` module enforces:

Logistics cost validation — Seller cannot add ocean freight for FOB.

Mandatory service check — All mandatory services must have a selected quote.

Insurance validation — For CIF and CIP, insurance must be included.

Duty validation — For DDP, duties must be included.

Example Deny Reason: "Ocean freight cannot be added for FOB incoterm. Please remove or change incoterm."

A3: AI Agent Registry & Authority Mapping (v11.0)

A3.1 Agent Registry

All agents use the three-tier strategy: Groq (A1) with Ollama fallback, Hugging Face local (A2/A3) with Ollama fallback. The full registry is maintained in the repository at `/core/ai/agents/registry.json`.

A3.2 Authority Level Summary

A3.3 Model Training & Update Frequency

A4: Technical Specifications (Key Services)

A4.1 Service Ports & Health Endpoints

A4.2 Database Connection Pool

A4.3 NATS JetStream Configuration

A4.4 Temporal Workflow Configuration

A4.5 AI Inference Timeouts

A5: Distressed Cargo Matrix (Extended, Dynamic)

A5.1 Country-Specific Rules

The matrix is updated in real time by RIA. The fee factor is applied to the distressed micro-contract's SGTX fee.

A5.2 Distressed Fee Calculation

text

$\text{Distressed Fee} = \text{Original Trade Fee Rate} \times \text{Distressed Country Factor} \times \text{Distressed Sale Amount}$

Example:

Original trade fee rate = 1.5%

Country factor (UAE) = 0.85

Distressed sale amount = \$900

$\text{Distressed Fee} = 1.5\% \times 0.85 \times \$900 = \$11.48$

A5.3 Distressed Triage Paths

A6: AI Provider Fallback Chain (Zero-Cost, No Billing Details)

A6.1 Fallback Chain

A6.2 Provider Availability

A6.3 Fallback Trigger Logic

A6.4 Fallback Logging

All fallback events are logged in ai_inference_records:

json

```
{  
  "agent_name": "tenant_message_generator",  
  "authority_level": "A1",  
  "provider": "ollama",  
  "fallback_used": true,  
  "fallback_reason": "groq_rate_limit_exceeded",  
  "model": "llama3.2:3b",  
  "latency_ms": 450,  
  "created_at": "2026-06-07T10:00:00Z"  
}
```

A7: Tenant Quick Start Guide (No Operating Modes — Uniform Experience)

A7.1 First Steps

Step 1 — Complete Onboarding Wizard (6 Steps)

Welcome & GTID Confirmation — Select entity type (TRD, LSP, SHIP, LAB, QC, FIN, GOV, MP, CBR). System generates provisional GTID. Click "Confirm GTID" .

Organization Details — Enter legal name, commercial register, tax ID. Upload supporting documents (HF Donut extracts fields). Click "Verify & Continue" .

KYB/KYC Verification — Upload required documents based on jurisdiction. System verifies against government registries. Click "Submit Documents" .

Profile Configuration — For TRD: select trader mode (BUY, SELL, or DUAL). Set preferred language, consent for optional features. Click "Save Preferences" .

Create First Resource (optional) — Pre-configure commodities, service catalogues, default fees. Click "Save Resource" or "Skip" .

Enter Sandbox — Practice a full trade in isolated environment with synthetic data. Click "Start Sandbox" .

Step 2 — Go Live

After completing the sandbox (or after KYB verification), click "Go Live" — one click transitions to production.

A7.2 Buyer Dashboard (Trader Portal, Mode = BUY)

A7.3 Seller Dashboard (Trader Portal, Mode = SELL)

A7.4 Logistics Provider Portal (LSP)

A7.5 Financier Portal (Bank / PFI)

A7.6 Need Help?

A8: Terminal Automation Reference & cd Path Index

A8.1 Root Directory Structure

text

/sgtx

- core/ # Microservices
 - ■■■■ governor/ # Governor Service
 - ■■■■ identity/ # Identity Service
 - ■■■■ trade-service/ # Trade Service
 - ■■■■ quote-service/ # Quote Service
 - ■■■■ contract-service/ # Contract Service
 - ■■■■ shipment-service/ # Shipment Service
 - ■■■■ payment-orchestrator/ # Payment Orchestrator
 - ■■■■ finance-service/ # Financing Service
 - ■■■■ inbox-service/ # Smart Inbox Service
 - ■■■■ tenant-experience-service/ # Tenant Experience Service
 - ■■■■ workflow-recovery-service/ # Workflow Recovery Service
 - ■■■■ tenant-data-export-service/# Tenant Data Export Service
 - ■■■■ financier-preference-service/ # Financier Preference Service
 - ■■■■ multi-shipment-coordinator/ # Multi-shipment Coordinator
 - ■■■■ fee-split-orchestrator/ # Fee-Split Orchestrator
 - ■■■■ logistics-service/ # Logistics Service
 - ■■■■ document-service/ # Document Service
 - ■■■■ barcode-service/ # Barcode Service
 - ■■■■ network-service/ # Network Service
 - ■■■■ dispute-service/ # Dispute Service
 - ■■■■ distressed-service/ # Distressed Service
 - ■■■■ location-service/ # Location Service
 - ■■■■ contract-service/ # Contract Service
- portals/ # Frontend portals
 - ■■■■ buyer-portal/ # Buyer Dashboard
 - ■■■■ seller-portal/ # Seller Dashboard
 - ■■■■ logistics-portal/ # Logistics Provider Portal
 - ■■■■ qc-portal/ # QC Portal
 - ■■■■ financier-portal/ # Financier Portal
 - ■■■■ government-portal/ # Government Portal

- ■■■■ admin-portal/ # Admin Portal
- ■■■■ marketplace-partner-portal/# Marketplace Partner Portal
- mobile/ # Mobile application
- ■■■■ app/ # React Native app
- ■■■■ models/ # Vosk, ONNX models
- ■■■■ assets/ # Images, fonts
- desktop/ # Desktop application
- ■■■■ tauri/ # Tauri v2 app
- database/ # Database schemas
- ■■■■ migrations/ # PostgreSQL migrations
- ■■■■ schemas/ # Schema definitions
- config/ # Configuration
- ■■■■ governor.toml # Governor configuration
- ■■■■ services.toml # Service configuration
- ■■■■ policies/ # OPA Rego policies
- scripts/ # Automation scripts
- ■■■■ dev_start.sh # Start all services
- ■■■■ health_check.sh # Check all health endpoints
- ■■■■ build_all.sh # Build all services
- ■■■■ db_migrate.sh # Run database migrations
- ■■■■ verify_deps.sh # Verify dependencies
- docs/ # Documentation
- ■■■■ api/ # OpenAPI specifications
- ■■■■ guides/ # User guides
- tests/ # Tests
- unit/ # Unit tests
- integration/ # Integration tests
- e2e/ # End-to-end tests

A8.2 Key File Paths

A8.3 Automation Scripts

A8.4 Environment Variables

A8.5 Terminal Integration Checklist

Terminal API Endpoint: GET /v1/release/authorization?ustn={USTN}&container={CONTAINER}

Rate Limit: 60 requests per minute per terminal.

Certificate Renewal: 1 year; SGTX sends renewal reminder 30 days before expiry.

END OF PART 28 — APPENDICES (v11.0 FULLY EXPANDED)

PART 29 — THE SOVEREIGN TRADE OPERATING SYSTEM — A MANIFESTO & LIVING DOCUMENT

29.0 Purpose & Design Philosophy

This Part is not a technical specification. It is the manifesto — the philosophical, strategic, and human foundation upon which SGTX is built. Every line of code, every policy, every user interface in this blueprint ultimately serves the vision articulated here. This Part answers the questions that specifications cannot: Why does SGTX exist? For whom? And what does it refuse to become?

Audience: This Part is for everyone — developers, traders, bankers, government officials, logistics workers, and future stewards of the platform. It is written in plain language, with minimal jargon, and is intended to be read by anyone who will touch, use, or be affected by SGTX.

Living Document: This manifesto is not frozen. It will evolve with the platform, shaped by the realities of global trade and the wisdom of its users. But its core principles — the three unshakable pillars — are immutable. They are the constitution of SGTX, and they can only be changed through the constitutional amendment process (Part 1.3).

The Three Unshakable Pillars (Revisited):

29.1 The Problem: Friction as a Service

Global trade is the bloodstream of human civilisation. It moves food from farms to tables, raw materials to factories, medicine to hospitals, and hope to communities. Every day, \$32 trillion worth of goods cross borders. This is the engine of modern life.

Yet the infrastructure that moves this wealth is trapped in the 19th century.

A shipment of oranges from Vietnam to Egypt involves:

17 paper documents — each one a point of failure, delay, and fraud.

9 intermediaries — each one extracting a fee, each one adding opacity.

\$1,200 in avoidable costs — demurrage, insurance, FX spreads, document discrepancies.

21 days — from order to payment, if everything goes perfectly.

This is not a technology problem. It is a trust problem. Counterparties who have never met must trust that the other will deliver goods, pay invoices, and honour contracts. In the absence of trust, they rely on intermediaries — banks, forwarders, inspectors, brokers — each of whom is paid to verify, guarantee, and reconcile. The intermediaries are not the problem; they are the solution to a problem that should not exist.

The platforms that have attempted to digitise trade fall into two traps:

The result is a world where small and medium traders — the backbone of emerging economies — are systematically excluded from the most efficient trade finance instruments. They pay cash upfront or accept usurious factoring rates. They cannot access the same credit as large corporations. They are invisible to the global financial system.

SGTX was designed to change this. Not by replacing the system, but by providing the neutral, sovereign execution layer that the system has always lacked.

29.2 The SGTX Thesis: Invisible Rails, Visible Trust

SGTX is not a marketplace. It is not a bank. It is not a logistics company. It is the operating system that powers trade execution beneath all of them.

Like an operating system, SGTX:

The "invisible rails" metaphor is deliberate. A marketplace or an ERP provider should be able to integrate with SGTX and offer their users cryptographic trade execution without their users ever needing to know SGTX exists. The rails are the API; the trust is the Governor; the experience is the Smart Inbox.

Visible Trust is the counterpoint. Every user — whether a farmer in the Mekong Delta, a customs officer in Alexandria, or a trade finance banker in Singapore — should understand exactly what is happening, why, and what to do next. Trust is not a black box. It is transparent, verifiable, and explained in plain language.

This is why the PlainLanguage Governor Decision Panel (Part 12A.3) is not a nice-to-have. It is a constitutional requirement. A user who is blocked by the platform must understand why — not as a technical error code, but as a human-readable explanation with a clear path forward.

The vision: One click to ship. One click to import. One click to pay. One universal number — the USTN — to rule the entire transaction. No data re-entry. No missing documents. No payment leakage. No trade-based money laundering. Every trade transaction moves through SGTX. Governments see the whole picture. Customs sees what is declared. Tax sees the real invoice. The Central Bank sees every pound and dollar that moves. All are linked by a single, un-gameable identifier.

29.3 Three Unshakable Pillars

Every feature, every service, every policy in this blueprint is anchored to three pillars. Any future change — by the Platform Governance Authority, by a contributor, or by a fork — that violates any of these pillars is, by definition, not SGTX.

Pillar I: Non-Custodial by Structure, Not by Policy

SGTX never holds trade principal or loan funds. Not because a policy says so — because the code makes it structurally impossible.

There is no funds table in the database. There is no wallet controlled by the Governor. The FeeLock is an instruction to split, not a temporary holding of money. This is not a marketing claim. It is enforced by the WasmEdge constitutional engine, and it is verifiable by any tenant at any time by auditing the Loom chain.

Why this matters: A non-custodial platform cannot be a counterparty. It cannot be sued for loss of funds (because it never held them). It cannot be pressured to freeze assets (because it does not control them). It is neutral by design, not by choice. This is the foundation of trust in SGTX.

Pillar II: AI May Block, Never Force

The AI Authority Ladder (A0-A4, A5 forbidden) is the constitution's most important safeguard.

AI can observe, advise, flag, and escalate — but it can never autonomously execute an irreversible action. A negotiation bot can propose a price; a credit intelligence agent can warn of default risk; a document fraud detector can reject an upload — but no AI can sign a contract, move a dollar, or lock a fee.

Why this matters: Trade is too important to be left to algorithms. Human judgment, human accountability, and human oversight are non-negotiable. AI is a tool, not a decision-maker. This is why every AI-driven block is accompanied by a plain-language explanation that tells the human exactly why the block occurred and what to do next.

Pillar III: Sovereign Jurisdiction Supremacy

Laws are code, and code is law — but neither should override the sovereignty of the jurisdictions in which trade occurs.

SGTX always applies the strictest rule among:

The jurisdiction matrix is not a static configuration file. It is a living knowledge graph updated every 15 minutes by the Regulatory Intelligence Agent (RIA) from over 300 official sources — OFAC, UN, EU, national customs authorities, central banks.

What this means: A trade that is legal under Egyptian law but would violate an EU sanction is blocked. A distressed sale that is permitted in Germany but requires a liquidator in the UAE is automatically adjusted with the correct fee factor. The platform never asks a tenant to choose which law to follow; it applies the strictest, and explains why.

Why this matters: SGTX does not operate in a legal vacuum. It operates in the real world, where trade is governed by laws, treaties, and regulations. By enforcing jurisdiction supremacy, SGTX protects its users from unknowingly violating sanctions, embargoes, or other legal prohibitions. It also provides governments with the assurance that the platform is not a vehicle for circumventing their laws.

29.4 Who SGTX Serves (and Who It Does Not)

SGTX Serves Every Legitimate Participant in Global Trade

SGTX Does NOT Serve

29.5 The Zero-Cost Imperative

The platform's zero-cost commitment is not a cost-saving measure — it is a strategic weapon against vendor lock-in, geopolitical risk, and economic exclusion.

This zero-cost architecture ensures that SGTX can operate in sanctions-adjacent jurisdictions, in countries with capital controls, and in regions where even the smallest fees would be a barrier. It is, in the most literal sense, trade infrastructure for the entire planet.

29.6 The TenantCentric Revolution (v11.0)

The original SGTX blueprint was architecturally complete but operationally intimidating. Users were getting lost. They did not know what to do next, they did not understand why actions were blocked, and they could not find the information they needed without clicking through five tabs.

v11.0 was the systematic resolution of every one of those friction points:

29.7 The Governance Compact

SGTX is governed by a multisig (3-of-5) of individuals who are bound by the same constitutional rules they enforce. No single person can change the constitution, alter the fee bounds, or override a DENY verdict.

The compact extends to tenants: every tenant admin can define internal approval chains for their own company, requiring dual signatures for high-value contracts or restricting which employees can see commercial invoices.

Why this matters: Trust is not a feature. It is the foundation. By distributing governance across multiple independent actors and making every change auditable, SGTX ensures that no single point of failure — human or technical — can compromise the platform's integrity.

29.8 The Future (As Defined by the RIA and Project Oracle)

SGTX is not finished. The Implementation Roadmap (Part 17) is continuously reprioritised by the Project Oracle agent, which ingests real-time development data, production metrics, and regulatory changes to suggest the optimal next sprint.

The future includes:

What will never change: The three pillars. Non-custodial structure. AI that blocks but never forces. Sovereign jurisdiction supremacy. Everything else is subject to evolution.

29.9 A Note to Developers, Auditors, and Future Maintainers

You hold in your hands (or on your screen) the complete DNA of a sovereign, AI-governed, non-custodial global trade execution platform. It is long. It is detailed. It is, in places, uncompromisingly technical. That is by design.

One Final Request

As you build, maintain, or audit this platform, remember the farmer in the Mekong Delta. She does not know what WasmEdge is. She does not care about the difference between A2 and A3. She cares that her oranges arrive in Alexandria on time, that she gets paid, and that the platform does not cheat her.

Every dialog, every notification, every default setting should honour her. If it does not, it is a bug — perhaps the most serious kind.

29.10 Closing Words

SGTX began as an idea: that international trade could be executed with the same cryptographic certainty as a blockchain transaction, but without requiring any participant to understand cryptography. That AI could protect traders from fraud and insolvency, but never replace human judgment. That the laws of nations could be respected by code, not bypassed by it. That all of this could be done at zero marginal cost, on infrastructure the operators control, with no external dependencies.

v11.0 proves that idea is not only possible — it is ready for production.

The rest is not in the blueprint. It is in the trades that will flow through it, the disputes it will prevent, the small businesses it will empower, and the trust it will build between strangers who will never meet but who will, through this platform, honour their contracts.

The rails are invisible. The trust is visible. The platform is yours.

END OF PART 29 — THE SOVEREIGN TRADE OPERATING SYSTEM — A MANIFESTO & LIVING DOCUMENT (v11.0 FULLY EXPANDED)

PART 30 — TRADE CORRIDOR NETWORK (TCN) EXTENSION

COMPLETE RORO MODULE — STANDALONE EXTENSION

FULLY MODIFIED, EXPANDED, EXTENDED & DETAILED (v11.0)

30.0 Purpose & Design Philosophy

The Trade Corridor Network (TCN) is a global digital infrastructure layer that connects governments, ports, customs authorities, logistics providers, banks, insurers, exporters, importers, and trade corridors through standardized trade lifecycle intelligence. The Egypt–Italy RoRo corridor becomes the first production corridor, with the Red Sea RoRo corridor as the second strategic corridor.

Why This Matters:

Most trade platforms focus on: Company → Shipment → Payment

TCN focuses on: Entity → Trade → Compliance → Contract → Corridor → Movement → Inspection → Settlement → Intelligence → Government

This creates a closed-loop trade ecosystem.

Key Principles:

Government-integrated — Each country gets a government node

Corridor-focused — Trade lanes are first-class entities

AI-assisted, never autonomous — Eligibility engine (A2) advises, never blocks

Non-marketplace — Never recommends carriers, brokers, or service providers

Zero-cost — All components use existing open-source infrastructure

AI Integration:

30.1 What is RoRo (Roll-on/Roll-off)?

RoRo (Roll-on/Roll-off) is a transport system where cargo is driven or rolled directly onto a vessel and rolled off at the destination, instead of being loaded into traditional shipping containers.

Typical RoRo cargo:

Trucks

Trailers

Refrigerated trailers

Cars

Heavy equipment

Agricultural cargo

Palletized cargo

Industrial machinery

Project cargo

Traditional Container Flow:

text

Factory → Container Stuffing → Port Storage → Container Loading → Ocean Transit → Port Unloading → Container De-stuffing → Final Delivery

RoRo Flow:

text

Factory → Truck / Trailer → Drive onto Vessel → Ocean Transit → Drive off Vessel → Final Delivery

Benefits:

Faster transit (5-7 days vs 14+ days)

Lower handling

Less cargo damage

Better for perishables

Reduced port congestion

Lower logistics complexity

30.2 Corridor Registry

Purpose: Master database of all recognized trade corridors.

Schema:

sql

```
CREATE TABLE trade_corridors (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  corridor_code TEXT UNIQUE NOT NULL, -- 'EGY-ITA-RORO-001'  
  corridor_name TEXT NOT NULL,  
  corridor_name_ar TEXT,
```

```

corridor_type TEXT NOT NULL, -- 'RORO', 'MARITIME', 'RAIL', 'ROAD', 'AIR', 'MULTIMODAL',
'ECONOMIC'
origin_country CHAR(2) NOT NULL REFERENCES jurisdictions(code),
destination_country CHAR(2) NOT NULL REFERENCES jurisdictions(code),
origin_ports TEXT[],
destination_ports TEXT[],
corridor_owner_gtid TEXT REFERENCES tenants(gtid),
status TEXT DEFAULT 'DRAFT', -- 'DRAFT', 'VERIFIED', 'CERTIFIED', 'STRATEGIC',
'NATIONAL_PRIORITY'
verification_status TEXT DEFAULT 'PENDING', -- 'PENDING', 'GOVERNMENT_VERIFIED',
'MULTISIG_VERIFIED'
operational_status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'MAINTENANCE', 'SUSPENDED'
verification_multisig TEXT[], -- GTIDs of verifiers
last_verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

Examples:

```

EGY-ITA-RORO-001 — Egypt ↔ Italy RoRo (Mediterranean)
EGY-KSA-RORO-001 — Egypt ↔ Saudi Arabia RoRo (Red Sea)
EGY-UAE-RORO-001 — Egypt ↔ UAE RoRo (Red Sea)
EGY-JOR-RORO-001 — Egypt ↔ Jordan RoRo (Red Sea)
EGY-SUD-RORO-001 — Egypt ↔ Sudan RoRo (Red Sea)
IND-UAE-IMEC-001 — India ↔ UAE Corridor

```

Types:

RoRo

Maritime

Rail

Road

Air

Multimodal

Economic Corridor

30.3 Trade Lane Passport

Purpose: Every corridor has a digital passport with commercial, logistics, financial, and compliance metadata.

30.3.1 Egypt–Italy RoRo Passport (Mediterranean)

Commercial:

Common Incoterms: FOB, CIF, DAP, DDP

Typical Cargo Types: Fresh Produce, Vehicles, Industrial Goods, Refrigerated Cargo

Logistics:

Average Transit: 5-7 Days

Ports: Damietta, Alexandria, Port Said ↔ Trieste, Livorno, Genoa

Financial:

Finance Eligibility: High

Insurance Availability: 98%

Compliance:

Required Certificates: COO, Health Cert, Packing List, Invoice, Phytosanitary

30.3.2 Red Sea RoRo Passport (Egypt ↔ Saudi Arabia)

Commercial:

Common Incoterms: FOB, CFR, DAP

Typical Cargo Types: Fresh Produce, Vehicles, Construction Materials, Refrigerated Cargo

Logistics:

Average Transit: 2-4 Days

Ports: Safaga, Port Tawfik ↔ Jeddah, Yanbu, Dammam

Financial:

Finance Eligibility: Medium-High

Insurance Availability: 95%

Compliance:

Required Certificates: COO, Health Cert, Packing List, Invoice, SFDA Food Cert (for food), Halal Certification

30.3.3 Red Sea RoRo Passport (Egypt ↔ UAE)

Commercial:

Common Incoterms: FOB, CFR, CIF

Typical Cargo Types: Vehicles, Machinery, Construction Materials, Perishable Goods

Logistics:

Average Transit: 4-6 Days

Ports: Safaga, Port Tawfik, Alexandria ↔ Jebel Ali, Khalifa Port

Financial:

Finance Eligibility: High

Insurance Availability: 97%

Compliance:

Required Certificates: COO, Packing List, Invoice, UAE Customs Declaration

Schema:

sql

```
CREATE TABLE trade_lane_passports (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  corridor_id UUID NOT NULL REFERENCES trade_corridors(id) ON DELETE CASCADE,  
  passport_version INTEGER DEFAULT 1,  
  common_incoterms TEXT[],  
  typical_cargo_types TEXT[],  
  average_transit_days INTEGER,  
  cargo_type_capabilities JSONB, -- { 'fresh_produce': true, 'vehicles': true, 'reefer': true }  
  finance_eligibility TEXT, -- 'HIGH', 'MEDIUM', 'LOW'  
  insurance_availability DECIMAL(5,2), -- 0-100%  
  required_certificates TEXT[],  
  passport_confidence DECIMAL(5,2), -- AI-generated confidence  
  source_regulations JSONB,  
  last_updated TIMESTAMPTZ DEFAULT NOW(),  
  loom_hash TEXT,  
  UNIQUE(corridor_id, passport_version)  
);
```

30.4 Corridor Eligibility Engine (A2/A1)

Purpose: AI service that determines if a trade qualifies for a corridor.

Inputs:

Product

Origin

Destination

Incoterm

Quantity

Transport Needs

Output Example — Egypt–Italy RoRo:

text

RoRo Corridor Compatibility: 97%

Reasons:

- ✓ Reefer Compatible
- ✓ Port Supported
- ✓ Product Supported
- ✓ Customs Eligible

Output Example — Red Sea RoRo (Egypt ↔ Saudi):

text

Red Sea RoRo Corridor Compatibility: 94%

Reasons:

- ✓ Product Compatible
- ✓ Port Supported
- ✓ Halal Certification Available
- ✓ Customs Eligible

Risk:

- SFDA Inspection Required (adds 1-2 days)
- Seasonal port congestion in Jeddah (April-October)

Implementation:

A2 (XGBoost/LightGBM) for scoring

A1 (Groq) for plain-language explanation

Never blocks — advisory only

Integrates with: Governor (Part 4.15) for compliance checks.

30.5 Government Node Framework

Purpose: Every country gets a government node.

Example — Egypt Node:

Authorities: Ministry of Transport, Customs Authority, GOEIC, Export Development Authority, Port Authorities (Safaga, Damietta, Alexandria, Port Said, Port Tawfik)

Example — Italy Node:

Authorities: Customs, Trade Agencies, Port Authorities (Trieste, Livorno, Genoa)

Example — Saudi Arabia Node:

Authorities: Customs Authority, SFDA, Port Authorities (Jeddah, Yanbu, Dammam)

Example — UAE Node:

Authorities: Customs Authority, Port Authorities (Jebel Ali, Khalifa Port)

Schema:

sql

```
CREATE TABLE government_nodes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),  
  authority_name TEXT NOT NULL,  
  authority_name_ar TEXT,  
  authority_type TEXT NOT NULL, -- 'MINISTRY', 'CUSTOMS', 'PORT_AUTHORITY', 'TRADE_AGENCY',  
  'ECONOMIC_ZONE'
```

```

authority_level TEXT DEFAULT 'NATIONAL', -- 'NATIONAL', 'REGIONAL', 'PORT', 'AGENCY'
node_gtid TEXT UNIQUE REFERENCES tenants(gtid),
node_permissions JSONB, -- what the node can see/do
verification_status TEXT DEFAULT 'PENDING', -- 'PENDING', 'VERIFIED', 'ACTIVE'
contact_email TEXT,
contact_phone TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

Integrates with: GTID (Part 2.1) for government GTIDs.

30.6 Government Verified Corridors

Purpose: New certification model for trade corridors.

Status Levels:

Draft

Verified

Certified

Strategic

National Priority

Examples:

Schema Additions:

sql

```

ALTER TABLE trade_corridors ADD COLUMN verification_multisig TEXT[];
ALTER TABLE trade_corridors ADD COLUMN last_verified_at TIMESTAMPTZ;

```

Integrates with: Loom (Part 1.6) for immutable verification history.

30.7 Port Digital Twin

Purpose: Every port becomes a GTID participant with digital twin capabilities.

Examples:

Schema:

sql

```

CREATE TABLE port_digital_twins (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
port_unlocode TEXT NOT NULL REFERENCES ports(unlocode),
port_capacity JSONB, -- { 'ro-ro': 500, 'containers': 1000, 'bulk': 200, 'cold_storage': 150 }
port_capacity_current JSONB, -- Current utilisation
port_congestion_level TEXT DEFAULT 'LOW', -- 'LOW', 'MODERATE', 'SEVERE'

```


port_operating_hours TEXT,
inspection_facilities JSONB,
customs_facilities JSONB,
corridor_mappings TEXT[], -- corridor_code references
last_updated TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT
);

30.8 Customs Intelligence

Purpose: Guidance for customs requirements (not filing).

30.8.1 Egypt → Italy (Fresh Strawberries)

text

Trade: Egypt → Italy

Product: Fresh Strawberries

Corridor: Egypt–Italy RoRo

System shows:

Required Documents:

- Certificate of Origin (COO)
- Health Certificate
- Commercial Invoice
- Packing List
- Phytosanitary Certificate
- Cold Chain Log

Complexity: Low

Estimated Clearance Time: 4-6 hours

30.8.2 Egypt → Saudi Arabia (Fresh Produce)

text

Trade: Egypt → Saudi Arabia

Product: Fresh Oranges

Corridor: Red Sea RoRo (Egypt–Saudi)

System shows:

Required Documents:

- Certificate of Origin (COO)
- Health Certificate
- Commercial Invoice
- Packing List

- SFDA Food Safety Certificate
- Halal Certification
- Phytosanitary Certificate

Complexity: Medium

Estimated Clearance Time: 6-8 hours

Special Requirements:

- SFDA inspection required
- Halal certification must be from approved body

30.8.3 Egypt → UAE (Vehicles)

text

Trade: Egypt → UAE

Product: Vehicles

Corridor: Red Sea RoRo (Egypt–UAE)

System shows:

Required Documents:

- Certificate of Origin (COO)
- Commercial Invoice
- Packing List
- UAE Customs Declaration
- Vehicle Export Certificate
- Bill of Lading

Complexity: Low

Estimated Clearance Time: 3-5 hours

Integrates with: Nafeza (Part 7.2) for document requirements.

30.9 Trade Request Integration

Purpose: Inside Trade Request, add Corridor Selection.

Fields:

Corridor Selection: Automatic / Manual

If Automatic: Recommended Corridor (Egypt–Italy RoRo, Red Sea RoRo, etc.)

Show: Transit Time, Eligibility, Required Documents, Expected Inspection Requirements

Do NOT show:

Carrier Selection

Freight Quotes

Broker Quotes

Not a marketplace.

UI Example:

text

TRADE CORRIDOR SELECTION

Corridor Selection: [Automatic ☒] [Manual ☐

Recommended Corridor: Egypt–Italy RoRo

Egypt–Italy RoRo Corridor

Transit Time: 5-7 days

Eligibility: 97%

Required Documents: 5

Inspection: Standard

Financeability: High

Insurance: 98% coverage

AI Insight: This corridor is ideal for fresh produce.

Previous trades on this corridor have 94% on-time delivery.

[Apply Corridor] [View Full Passport] [See Alternatives]

Alternative Corridors:

Red Sea RoRo (Egypt–Saudi)

Transit Time: 2-4 days | Eligibility: 94% | Documents: 7

SFDA inspection required

[Select]



Integrates with: Part 4.0—4.15 (Trade Request form).

30.10 Contract Integration

Purpose: Contract generator automatically inserts corridor-specific clauses.

Example — Egypt–Italy RoRo:

text

Trade Corridor: Egypt–Italy RoRo

Additional Clauses:

- Port of Departure: Damietta (Egypt)
- Port of Arrival: Trieste (Italy)
- Corridor Conditions: Standard RoRo terms (ISM Code compliant)
- Inspection Conditions: Italy Customs inspection upon arrival
- Documentation Requirements: COO, Health Cert, Packing List, Invoice, Phytosanitary
- Transit Guarantee: 7 days maximum
- Incoterm: CFR (as per corridor standard)

Example — Red Sea RoRo (Egypt–Saudi):

text

Trade Corridor: Red Sea RoRo (Egypt–Saudi)

Additional Clauses:

- Port of Departure: Safaga (Egypt)
- Port of Arrival: Jeddah (Saudi Arabia)
- Corridor Conditions: Standard RoRo terms
- Inspection Conditions: SFDA inspection upon arrival (required for food)
- Documentation Requirements: COO, Health Cert, Packing List, Invoice, SFDA Cert, Halal Cert
- Transit Guarantee: 4 days maximum
- Incoterm: FOB (as per corridor standard)
- Halal Compliance: All goods must be Halal certified

Integrates with: Part 3 (Contract Signing).

30.11 Compliance Integration (Governor)

Purpose: Governor consumes corridor data for compliance checks.

Checks:

Product Restrictions

Country Restrictions

Export Restrictions

Import Restrictions

Sanctions

Licensing

Result: Corridor Compliance Approved

Schema:

sql

```
CREATE TABLE corridor_compliance_gates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  corridor_id UUID NOT NULL REFERENCES trade_corridors(id) ON DELETE CASCADE,  
  gate_type TEXT NOT NULL, -- 'PRODUCT_RESTRICTION', 'COUNTRY_RESTRICTION', 'SANCTION',  
  'LICENCE', 'DOCUMENT'  
  gate_condition JSONB,  
  gate_message TEXT,  
  is_active BOOLEAN DEFAULT true,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Example — Egypt–Italy RoRo Compliance:

json

```
{  
  "corridor_code": "EGY-ITA-RORO-001",  
  "compliance_checks": [  
    {  
      "gate_type": "PRODUCT_RESTRICTION",  
      "condition": { "hs_code": "0805.10", "allowed": true },  
      "message": "Oranges allowed for export to Italy"  
    },  
    {  
      "gate_type": "DOCUMENT",  
      "condition": { "document_type": "PHYTOSANITARY", "required": true },  
      "message": "Phytosanitary certificate required"  
    }  
  ],  
  "result": "CORRIDOR_COMPLIANCE_APPROVED"  
}
```

Integrates with: Part 4.15 (Governor PreScreen).

30.12 Financing Integration

Purpose: Banks receive corridor reliability metrics.

Metrics:

Output to Financier:

text

Corridor Risk Assessment — Egypt–Italy RoRo

Risk Level: LOW

Recommended LTV: 80%

Recommended APR: 4.8-5.2%

Corridor History: 142 trades, 94% on-time

Document Success: 98% first-time acceptance

Insurance: 98% coverage available

Result: FINANCEABLE

Integrates with: Part 9 (Financing Module).

30.13 Insurance Integration

Purpose: Store insurance availability per corridor.

Status:

Insurable

Partially Insurable

Restricted

Integrates with: Part 4.8 (Insurance Requirements).

30.14 Corridor Analytics

Purpose: Track corridor performance metrics.

Metrics:

Volume

Transit Times

Document Delays

Inspection Delays

Port Congestion

Settlement Delays

Financing Demand

Example — Egypt–Italy RoRo Analytics:

text

■ CORRIDOR ANALYTICS — Egypt–Italy RoRo (Last 30 Days)

Volume: 142 shipments (↑12% MoM)

GMV: \$8.4M (↑15% MoM)

Top Products: Fresh Produce (54%), Vehicles (28%), Industrial (18%)

Average Transit: 6.2 days

On-Time Performance: 94%

Document Delay Rate: 2.4%

Customs Clearance: 4.8h average

Port Congestion: 8h average waiting (Damietta)

Financing Demand: 68% of shipments (↑5% MoM)

Example — Red Sea RoRo Analytics:

text

■ CORRIDOR ANALYTICS — Red Sea RoRo (Egypt–Saudi) (Last 30 Days)

Volume: 89 shipments (↑8% MoM)

GMV: \$3.2M (↑10% MoM)

Top Products: Fresh Produce (62%), Construction Materials (28%), Vehicles (10%)

Average Transit: 3.1 days

On-Time Performance: 89%

Document Delay Rate: 5.1%

Customs Clearance: 6.2h average

Port Congestion: 4h average waiting (Safaga)

SFDA Inspection: 12h average (adds 1-2 days)

Financing Demand: 45% of shipments

Schema:

sql

```
CREATE TABLE corridor_analytics (  
  id BIGSERIAL PRIMARY KEY,  
  corridor_code TEXT NOT NULL REFERENCES trade_corridors(corridor_code),  
  measurement_period DATE NOT NULL,  
  volume INTEGER,  
  gmv_usd DECIMAL(20,2),  
  average_transit_days DECIMAL(5,2),  
  on_time_performance DECIMAL(5,2),  
  document_delay_rate DECIMAL(5,2),  
  customs_clearance_hours DECIMAL(5,2),  
  port_congestion_hours DECIMAL(5,2),  
  financing_demand INTEGER,  
  privacy_epsilon DECIMAL(5,2) DEFAULT 0.1,  
  created_at TIMESTAMPTZ DEFAULT NOW())
```

);

Integrates with: Part 19 (Trade Memory Layer).

30.15 Government Dashboards

Purpose: Separate portal for government stakeholders.

30.15.1 Trade Authority View

text

■ TRADE AUTHORITY DASHBOARD — Egypt

Export Volume (Last 30 Days):

- Total: \$12.4M (↑18% MoM)
- RoRo: \$8.4M (68% of total)
- Container: \$4.0M (32% of total)

Top Export Corridors:

1. Egypt–Italy RoRo: \$8.4M (142 shipments)
2. Egypt–Saudi RoRo: \$3.2M (89 shipments)
3. Egypt–UAE RoRo: \$0.8M (45 shipments)

Sector Analysis:

- Agriculture: 54% (↑12% MoM)
- Vehicles: 28% (↑20% MoM)
- Industrial: 18% (↓2% MoM)

Corridor Utilization:

- Egypt–Italy: 78% capacity (↑5%)
- Egypt–Saudi: 62% capacity (↑8%)
- Egypt–UAE: 45% capacity (↑3%)

30.15.2 Customs View

text

■ CUSTOMS DASHBOARD — Egypt

Document Bottlenecks:

1. Phytosanitary Certificate: 8% delay rate
2. Health Certificate: 5% delay rate
3. Certificate of Origin: 2% delay rate

Clearance Bottlenecks:

- Egypt–Italy Corridor: 4.8h average
- Egypt–Saudi Corridor: 6.2h average
- Egypt–UAE Corridor: 3.5h average

Top Issues:

- Missing SFDA documentation (18% of Egypt–Saudi shipments)
- Incorrect packaging (5% of all shipments)

30.15.3 Port View

text

■ PORT DASHBOARD — Safaga

Capacity Utilization:

- RoRo: 62% (↑8%)
- Containers: 45%
- Cold Storage: 58%
- Bulk: 32%

RoRo Traffic (Last 30 Days):

- Egypt–Saudi: 89 shipments (↑8% MoM)
- Egypt–UAE: 45 shipments (↑12% MoM)

Transit Performance:

- Average Gate-in to Departure: 3.2h
- Average Arrival to Gate-out: 4.1h
- Port Congestion: Low (4h waiting)

Operating Status: ■ Normal

Next Maintenance: 2026-08-15 (6h downtime planned)

Only aggregated data. No confidential trade details.

Integrates with: Part 12C.10 (Government Portal).

30.16 GTID Government Extension

Purpose: New GTID categories for government entities.

New Entity Types:

Examples:

SGTX-EG-GOV-001 — Egyptian Ministry of Transport

SGTX-EG-CUSTOMS-001 — Egyptian Customs Authority

SGTX-EG-PORT-001 — Damietta Port Authority

SGTX-EG-PORT-002 — Safaga Port Authority

SGTX-IT-GOV-001 — Italian Ministry of Transport

SGTX-IT-CUSTOMS-001 — Italian Customs Authority

SGTX-IT-PORT-001 — Port of Trieste Authority

SGTX-SA-GOV-001 — Saudi Ministry of Transport

SGTX-SA-CUSTOMS-001 — Saudi Customs Authority

SGTX-SA-PORT-001 — Jeddah Port Authority

SGTX-SFDA-001 — Saudi Food and Drug Authority

SGTX-AE-GOV-001 — UAE Ministry of Transport

SGTX-AE-CUSTOMS-001 — UAE Customs Authority

SGTX-AE-PORT-001 — Jebel Ali Port Authority

Integrates with: Part 2.1 (GTID Format).

30.17 GTID Integration in TCN

30.17.1 GTID Resolution for Government Entities

Endpoint: GET /v1/gtid/resolve?gtid=SGTX-EG-PORT-001

Response:

json

```
{
  "gtid": "SGTX-EG-PORT-001",
  "legal_name": "Damietta Port Authority",
  "type": "PORT",
  "jurisdiction": "EG",
  "port_unlocode": "EGDAM",
  "port_capabilities": {
    "roro": true,
    "containers": true,
    "cold_storage": true,
    "bulk": true
  },
  "corridors": ["EGY-ITA-RORO-001"],
  "operational_status": "ACTIVE",
  "congestion_level": "LOW",
  "trust_score": 92,
  "kyb_tier": 3,
  "lifecycle_state": "VERIFIED"
}
```

30.17.2 GTID Resolution for Corridor Operators

Endpoint: GET /v1/gtid/resolve?gtid=SGTX-EG-CORR-001

Response:

json

```
{
  "gtid": "SGTX-EG-CORR-001",
```

```
"legal_name": "Egypt–Italy RoRo Corridor Operator",
"type": "CORR_OP",
"jurisdiction": "EG",
"corridor_code": "EGY-ITA-RORO-001",
"corridor_name": "Egypt–Italy RoRo Corridor",
"corridor_type": "RORO",
"corridor_status": "STRATEGIC",
"verification_status": "GOVERNMENT_VERIFIED",
"trust_score": 95,
"kyb_tier": 3,
"lifecycle_state": "VERIFIED"
}
```

30.18 USTN Integration in TCN

30.18.1 USTN with Corridor Attribute

USTN Format (with Corridor Extension):

text

SGTX-EG-26-F3A-1- RORO

Components:

30.18.2 USTN Resolution with Corridor Data

Endpoint: GET /v1/ustn/{ustn}

Response (with corridor data):

json

```
{
  "ustn": "SGTX-EG-26-F3A-1",
  "corridor_code": "EGY-ITA-RORO-001",
  "corridor_name": "Egypt–Italy RoRo Corridor",
  "corridor_status": "ACTIVE",
  "transit_time": 6.2,
  "ports": {
    "departure": "Damietta (EGDAM)",
    "arrival": "Trieste (ITTRS)"
  },
  "shipment_status": "IN_TRANSIT",
  "estimated_arrival": "2026-04-22T14:00:00Z",
  "compliance_status": "CORRIDOR_COMPLIANCE_APPROVED",
}
```

```

"documents": {
  "required": ["COO", "HEALTH_CERT", "PACKING_LIST", "INVOICE", "PHYTOSANITARY"],
  "verified": ["COO", "HEALTH_CERT", "PACKING_LIST", "INVOICE"],
  "pending": ["PHYTOSANITARY"]
}
}

```

30.18.3 Corridor USTN Generation

Generation Logic:

rust

```

fn generate_ustn_with_corridor(
  buyer_gtid: &str,
  seller_gtid: &str,
  corridor_code: &str
) -> String {
  let buyer_suffix = extract_suffix(buyer_gtid);
  let seller_suffix = extract_suffix(seller_gtid);
  let timestamp = Utc::now().format("%Y%m%d%H%M%S").to_string();
  let SEQ = generate_SEQ();
  let base_ustn = format!(
    "SGTX-{}-{}-{}",
    buyer_suffix, seller_suffix, timestamp, SEQ
  );
  // Append corridor code if present
  if !corridor_code.is_empty() {
    format!("{}", base_ustn, corridor_code)
  } else {
    base_ustn
  }
}

```

30.19 Database Schema Additions (Part 30)

sql

-- Corridor Registry (Core)

```

CREATE TABLE trade_corridors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  corridor_code TEXT UNIQUE NOT NULL, -- 'EGY-ITA-RORO-001'

```

```

corridor_name TEXT NOT NULL,
corridor_name_ar TEXT,
corridor_type TEXT NOT NULL, -- 'RORO', 'MARITIME', 'RAIL', 'ROAD', 'AIR', 'MULTIMODAL',
'ECONOMIC'
origin_country CHAR(2) NOT NULL REFERENCES jurisdictions(code),
destination_country CHAR(2) NOT NULL REFERENCES jurisdictions(code),
origin_ports TEXT[],
destination_ports TEXT[],
corridor_owner_gtid TEXT REFERENCES tenants(gtid),
status TEXT DEFAULT 'DRAFT', -- 'DRAFT', 'VERIFIED', 'CERTIFIED', 'STRATEGIC',
'NATIONAL_PRIORITY'
verification_status TEXT DEFAULT 'PENDING', -- 'PENDING', 'GOVERNMENT_VERIFIED',
'MULTISIG_VERIFIED'
operational_status TEXT DEFAULT 'ACTIVE', -- 'ACTIVE', 'MAINTENANCE', 'SUSPENDED'
verification_multisig TEXT[], -- GTIDs of verifiers
last_verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Trade Lane Passport (Per Corridor)
CREATE TABLE trade_lane_passports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
corridor_id UUID NOT NULL REFERENCES trade_corridors(id) ON DELETE CASCADE,
passport_version INTEGER DEFAULT 1,
common_incoterms TEXT[],
typical_cargo_types TEXT[],
average_transit_days INTEGER,
cargo_type_capabilities JSONB, -- { 'fresh_produce': true, 'vehicles': true, 'reefer': true }
finance_eligibility TEXT, -- 'HIGH', 'MEDIUM', 'LOW'
insurance_availability DECIMAL(5,2), -- 0-100%
required_certificates TEXT[],
passport_confidence DECIMAL(5,2), -- AI-generated confidence
source_regulations JSONB,
last_updated TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT,
UNIQUE(corridor_id, passport_version)

```

```

);
-- Government Nodes
CREATE TABLE government_nodes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
authority_name TEXT NOT NULL,
authority_name_ar TEXT,
authority_type TEXT NOT NULL, -- 'MINISTRY', 'CUSTOMS', 'PORT_AUTHORITY', 'TRADE_AGENCY',
'ECONOMIC_ZONE'
authority_level TEXT DEFAULT 'NATIONAL', -- 'NATIONAL', 'REGIONAL', 'PORT', 'AGENCY'
node_gtid TEXT UNIQUE REFERENCES tenants(gtid),
node_permissions JSONB, -- what the node can see/do
verification_status TEXT DEFAULT 'PENDING', -- 'PENDING', 'VERIFIED', 'ACTIVE'
contact_email TEXT,
contact_phone TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);
-- Port Digital Twins
CREATE TABLE port_digital_twins (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
port_unlocode TEXT NOT NULL REFERENCES ports(unlocode),
port_capacity JSONB, -- { 'roro': 500, 'containers': 1000, 'bulk': 200, 'cold_storage': 150 }
port_capacity_current JSONB, -- Current utilisation
port_congestion_level TEXT DEFAULT 'LOW', -- 'LOW', 'MODERATE', 'SEVERE'
port_operating_hours TEXT,
inspection_facilities JSONB,
customs_facilities JSONB,
corridor_mappings TEXT[], -- corridor_code references
last_updated TIMESTAMPTZ DEFAULT NOW(),
loom_hash TEXT
);
-- Corridor Compliance Gating (for Governor)
CREATE TABLE corridor_compliance_gates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
corridor_id UUID NOT NULL REFERENCES trade_corridors(id) ON DELETE CASCADE,

```

```

gate_type TEXT NOT NULL, -- 'PRODUCT_RESTRICTION', 'COUNTRY_RESTRICTION', 'SANCTION',
'LICENCE', 'DOCUMENT'

gate_condition JSONB,
gate_message TEXT,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Corridor Analytics (Aggregated, Differential Privacy)
CREATE TABLE corridor_analytics (
id BIGSERIAL PRIMARY KEY,
corridor_code TEXT NOT NULL REFERENCES trade_corridors(corridor_code),
measurement_period DATE NOT NULL,
volume INTEGER,
gmv_usd DECIMAL(20,2),
average_transit_days DECIMAL(5,2),
on_time_performance DECIMAL(5,2),
document_delay_rate DECIMAL(5,2),
customs_clearance_hours DECIMAL(5,2),
port_congestion_hours DECIMAL(5,2),
financing_demand INTEGER,
privacy_epsilon DECIMAL(5,2) DEFAULT 0.1,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Extended GTID Entity Types
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS port_unlocode TEXT;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS corridor_code TEXT;
ALTER TABLE tenants ADD COLUMN IF NOT EXISTS government_authority_level TEXT;

-- USTN with Corridor Extension
ALTER TABLE shipments ADD COLUMN IF NOT EXISTS corridor_code TEXT REFERENCES
trade_corridors(corridor_code);
ALTER TABLE shipments ADD COLUMN IF NOT EXISTS corridor_verified BOOLEAN DEFAULT false;
ALTER TABLE shipments ADD COLUMN IF NOT EXISTS corridor_eligibility_score DECIMAL(5,2);

-- Indexes
CREATE INDEX idx_trade_corridors_code ON trade_corridors(corridor_code);
CREATE INDEX idx_trade_corridors_status ON trade_corridors(status);
CREATE INDEX idx_trade_corridors_origin ON trade_corridors(origin_country);

```

```
CREATE INDEX idx_trade_corridors_destination ON trade_corridors(destination_country);
CREATE INDEX idx_trade_lane_passports_corridor ON trade_lane_passports(corridor_id);
CREATE INDEX idx_government_nodes_country ON government_nodes(country_code);
CREATE INDEX idx_government_nodes_type ON government_nodes(authority_type);
CREATE INDEX idx_port_digital_twins_unlocode ON port_digital_twins(port_unlocode);
CREATE INDEX idx_corridor_compliance_gates_corridor ON corridor_compliance_gates(corridor_id);
CREATE INDEX idx_corridor_analytics_corridor ON corridor_analytics(corridor_code);
CREATE INDEX idx_corridor_analytics_period ON corridor_analytics(measurement_period);
CREATE INDEX idx_shipments_corridor ON shipments(corridor_code);
```

30.20 API Endpoints (Part 30)

30.21 Implementation Checklist (Part 30)

Phase 1 — MVP (6-8 Weeks)

Corridor Registry (table + API)

Egypt–Italy RoRo Passport (static data, updated by RIA)

Red Sea RoRo Passport (Egypt–Saudi, Egypt–UAE)

Trade Request Integration (corridor selection, automatic detection)

Corridor Eligibility Engine (A2, XGBoost/LightGBM)

Contract Integration (automatic clause insertion)

Phase 2 — Government Integration (8-12 Weeks)

Government Node Framework (GTID extension, node registration)

Port Digital Twin (port capabilities, capacity, congestion)

Customs Intelligence (document requirements, complexity)

Corridor Analytics (volume, transit, delays)

Phase 3 — Advanced Features (12-16 Weeks)

Government Dashboards (Trade Authority, Customs, Port views)

Financing Integration (corridor reliability metrics)

Insurance Integration (corridor-specific insurance)

Multi-Country Corridors (Egypt ↔ Saudi, India ↔ UAE, etc.)

GTID Government Extension

USTN Corridor Integration

30.22 References to Existing Parts

30.23 Quick Reference Card

text



The following body preserves the complete change-set text. It is included in full rather than summarized, so the master document remains self-contained and auditable.

Change-set line count at extraction: 7,582

SGTX PLATFORM - COMPLETE ADD-ONS SPECIFICATION

All Add-Ons 1-28 Fully Extended, Expanded & Detailed

PART 1: ADD-ON 1 - GRAPH NEURAL NETWORK (GNN) RISK ENGINE & INSTITUTIONAL TRADE GRAPH

1.0 Purpose & Design Philosophy

The Graph Neural Network (GNN) Risk Engine serves two complementary purposes:

1. Sanctions & Adversarial Risk Detection - Detects indirect sanctions links, hidden money-laundering patterns, and adversarial relationships (e.g., a seller's Ultimate Beneficial Owner within 2 hops of a sanctioned entity).
2. Institutional Trade Graph (Trust-Based Mapping) - Maps positive, trust-based relationships (trade frequency, financing relationships, co-compliance, supply chain dependencies) to strengthen credit assessments, financing decisions, and network trust scores.

All outputs are advisory or constraining (A2/A4) - they never recommend counterparties or autonomous actions.

Key Principles:

- Zero-cost - PyTorch Geometric, PyTorch, GraphRAG all open-source
- Privacy-preserving - Trust graph uses anonymised IDs + differential privacy (?=0.1)
- Opt-in - Tenants control whether they contribute to the trust graph
- Non-marketplace - Never suggests alternative counterparties; only flags risk

AI Integration:

- A2 (HF local ? Ollama): Inference for sanctions and risk scoring
- A2/A3: Trust graph scoring (advisory; no blocking)
- A3: Model approval (multisig)
- A4: Fallback and Governor integration

1.1 Architecture & Integration

The GNN service is implemented as a Rust + PyTorch Geometric microservice (or a Python sidecar with a Rust gRPC wrapper). It exposes two main endpoints:

- /v1/gnn/risk - for sanctions and adversarial risk (called by Governor on trade.initiate and financing.request)
- /v1/gnn/trust - for institutional trust graph queries (called on demand by Financier Portal, Trust Passport, and Portfolio views)

Model Architecture:

Component Technology Purpose

Graph Convolutional Network (GCN) PyTorch Geometric Learns node embeddings from corporate ownership, trade, and compliance graphs

Edge types (adversarial) Ownership, directorship, sanctions list membership Detects proximity to sanctioned entities

Edge types (trust) Trade frequency, financing agreements, shared auditors, supply chain links Builds positive trust graph

Graph attention (GATv2) PyTorch Geometric Weighs importance of different relationship types

Training frequency Weekly (off-peak) Using aggregated, anonymised data across sovereign nodes

Training Data Sources:

- Corporate graph (entity_nodes, relationship_edges)
- Trade tables (trade_requests, shipments)
- Financing tables (financing_agreements, financing_repayments)
- Compliance tables (kyc_records, sanctions_hits)

1.2 Sanctions & Adversarial Risk Output

When called by the Governor, the GNN returns:

json

```
{
  "sanctions_proximity": 2,
  "graph_risk_score": 85,
  "explanation": "Seller's UBO (Person X) is 2 hops from sanctioned entity Y via Company Z.",
  "model_version": "v2.1",
  "inference_time_ms": 120
}
```

Field Description

sanctions_proximity Minimum number of hops between any party and a sanctioned entity

graph_risk_score 0-100 composite of adversarial risk (money laundering, sanctions evasion, hidden ownership)

explanation Groq-generated (A1) plain-language summary

Governor Integration:

If sanctions_proximity ≥ 2 OR graph_risk_score ≥ 90, the Governor returns CONDITIONAL with a Decision Panel explanation:

"This trade involves a party whose UBO is within 2 hops of a sanctioned entity. Enhanced due diligence required."

The decision never suggests alternative counterparties.

1.3 Institutional Trade Graph (Trust-Based Mapping)

Purpose:

Map positive relationships while preserving privacy using ZK proofs. Used by:

- Financier Portal (credit assessment, portfolio risk)
- Trust Passport (network trust score)

- Dispute evidence (indirect relationship patterns)

Edge Types & Weights:

Edge Type Weight Privacy Method Data Source

Trade frequency (last 12 months) 1.0 ZK proof (count only, no identities) shipments

Financing relationship 1.5 ZK proof (existence, no amount) financing_agreements

Co-compliance (shared auditor/certifier) 0.5 Explicit consent required tenant_verified_ids

Supply chain dependency 0.8 ZK proof (indirect, anonymised) trade_requests

Shared logistics provider 0.3 Aggregated, no consent needed service_quotations

Output for Participant (Ego Network Only):

json

```
{
  "direct_connections": 47,
  "trusted_connections": 12,
  "network_trust_score": 82,
  "indirect_exposure": "2 hops to 3 financial institutions",
  "privacy_notice": "All counts are anonymised; no counterparty identities revealed."
}
```

Display Locations:

- Financier Portal - optional card in "Portfolio & Compliance"
- Trust Passport - optional dimension (opt-in)
- Company Admin ? Network - for traders (aggregated, no identifiers)

Consent & Opt-Out:

Tenants can disable inclusion in the trust graph via Company Admin ? Privacy ? Trade Graph Visibility (off by default). Disabling does not affect sanctions detection.

1.4 Data Model Additions

sql

-- GNN risk scores (adversarial)

```
CREATE TABLE gnn_risk_scores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entity_gtid TEXT NOT NULL REFERENCES tenants(gtid) ON DELETE CASCADE,
  sanctions_proximity INTEGER,
  graph_risk_score INTEGER,
  model_version TEXT,
  inference_at TIMESTAMPTZ,
  explanation TEXT,
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
```

```

created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Institutional trade graph edges (anonymised, permissioned)
CREATE TABLE trade_graph_edges (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
from_entity_anon_id TEXT NOT NULL,
to_entity_anon_id TEXT NOT NULL,
edge_type TEXT NOT NULL, -- 'TRADE', 'FINANCING', 'COMPLIANCE', 'SUPPLY'
weight DECIMAL(5,2),
first_seen TIMESTAMPTZ,
last_seen TIMESTAMPTZ,
zk_proof_hash TEXT,
consent_granted BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Materialised view for network trust scores
CREATE MATERIALIZED VIEW network_trust_scores AS
SELECT
entity_gtid,
COUNT(DISTINCT to_entity_anon_id) AS direct_connections,
SUM(weight) AS weighted_trust_sum,
CURRENT_TIMESTAMP AS refreshed_at
FROM trade_graph_edges
GROUP BY entity_gtid;

```

1.5 Implementation Checklist

Core GNN Service

- ? Deploy gnn-risk-service (Rust + PyTorch Geometric)
- ? Train initial model on corporate graph data
- ? Integrate with Governor (sync call on trade.initiate, financing.request)
- ? Add fallback to rule-based risk scoring if GNN unavailable

Institutional Trade Graph

- ? Build anonymisation service (rotating pepper, deterministic hash)
- ? Add consent management for trust graph contribution
- ? Build Financier Portal widget for network trust score
- ? Implement opt-out toggle in Company Admin

Monitoring & Alerting

- ? Add Prometheus metrics: sgtx_gnn_risk_score, sgtx_gnn_inference_latency
- ? Set up alerting for GNN service unavailability (>5min)
- ? Configure model drift monitoring

PART 2: ADD-ON 2 - FEDERATED LEARNING NETWORK

2.0 Purpose & Design Philosophy

Train machine learning models across multiple sovereign nodes without exchanging raw trade data. Each node trains a local model on its own data (e.g., fraud detection, margin estimation, credit scoring), sends only encrypted model updates (gradients) to a secure aggregator, and receives a global model.

Key Principles:

- Privacy-preserving - No raw data ever leaves the sovereign node
- Differential privacy ($\epsilon=0.5$) applied to each update
- Zero-cost - OpenFL, Flower, PyTorch all open-source
- Federated averaging (FedAvg) or robust aggregation algorithms

AI Integration:

- A2: Local training, model inference
- A3: Model approval, drift escalation
- A4: Model distribution, access control

2.1 Architecture

Trained Models:

2.2 Data Model Additions

sql

-- Federated learning model registry

```
CREATE TABLE federated_learning_models (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  model_name TEXT NOT NULL,  
  version INTEGER,  
  global_model_hash TEXT,  
  aggregator_node_gtid TEXT,  
  round_number INTEGER,  
  accuracy_validation DECIMAL(5,4),  
  deployed_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Local training metadata per node

```
CREATE TABLE local_training_metadata (  

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
node_gtid TEXT NOT NULL,
model_name TEXT NOT NULL,
round_number INTEGER,
local_accuracy DECIMAL(5,4),
data_size INTEGER,
training_duration_seconds INTEGER,
uploaded_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

2.3 Implementation Checklist

- ? Deploy fed-learning-coordinator (Rust + OpenFL)
- ? Configure participant nodes with local-trainer sidecar
- ? Set up weekly training schedule (off-peak)
- ? Implement model drift monitoring
- ? Add Admin Portal status dashboard
- ? Configure differential privacy ($\epsilon=0.5$)

PART 3: ADD-ON 3 - CAUSAL INFERENCE ENGINE

3.0 Purpose & Design Philosophy

Identify root causes of disputes, delays, defaults, or quality failures - not just correlations. For example, "68% of the delay was due to a port strike, 22% due to carrier rerouting, 10% due to customs documentation."

Key Principles:

- Factual only - Never assigns blame; only contribution percentages
- Advisory - Used in evidence packages, never blocks trades
- Zero-cost - DoWhy, EconML all open-source

AI Integration:

- A2/A3: Causal inference (DoWhy + EconML, Python)
- A1: Groq summary generation

3.1 Architecture

Output Example:

json

```

{
  "root_causes": [
    {
      "factor": "port_strike",

```



```

"contribution": 0.54,
"description": "Port closure at Alexandria added 54% of total delay.",
"confidence_interval": { "lower": 0.45, "upper": 0.62 }
},
{
"factor": "carrier_rerouting",
"contribution": 0.32,
"description": "Carrier rerouted due to weather, adding 32%.",
"confidence_interval": { "lower": 0.28, "upper": 0.36 }
},
{
"factor": "customs_hold",
"contribution": 0.14,
"description": "Missing phytosanitary certificate caused 14% delay.",
"confidence_interval": { "lower": 0.10, "upper": 0.18 }
}
],
"model_version": "v2.1",
"groq_summary": "The delay was caused primarily by the port strike (54%), with the carrier's rerouting as a
secondary factor (32%)."
}

```

3.2 Data Model Additions

sql

-- Causal attribution results

```

CREATE TABLE causal_attribution (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
entity_id UUID NOT NULL, -- dispute_id or shipment_id
entity_type TEXT NOT NULL, -- 'DISPUTE', 'SHIPMENT'
factor TEXT NOT NULL,
contribution DECIMAL(5,4) NOT NULL,
confidence_interval JSONB,
description TEXT,
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

3.3 Implementation Checklist

- ? Deploy causal-engine (Python, DoWhy + EconML)
- ? Define causal graph (DAG) for trade disputes
- ? Integrate with dispute filing and milestone breach triggers
- ? Store results in causal_attribution table
- ? Add Groq summary generation
- ? Test with historical dispute data

PART 4: ADD-ON 4 - SELF-HEALING INFRASTRUCTURE & CHAOS ENGINEERING

4.0 Purpose & Design Philosophy

Ensure platform resilience by automatically healing failed pods, predicting hardware failures, and regularly proving system robustness through chaos experiments.

Key Principles:

- Zero-cost - K3s, Cilium, Chaos Mesh all open-source
- Predictive - LSTM model trained on Backblaze drive failure dataset
- Automated - Pod rescheduling, node cordoning, workload draining
- Safe - Chaos experiments run in staging by default; production requires multisig

AI Integration:

- A2: LSTM for hardware failure prediction
- A1: Post-mortem generation (Groq)
- A4: Orchestration

4.1 Architecture

Chaos Experiment Types:

Predictive Hardware Failure (A2, LSTM):

4.2 Data Model Additions

sql

-- Infrastructure predictions

```
CREATE TABLE infrastructure_predictions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  node_gtid TEXT NOT NULL,
  component TEXT NOT NULL,
  predicted_failure_probability DECIMAL(5,4),
  estimated_failure_date TIMESTAMPTZ,
  action_taken TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Chaos experiments

```
CREATE TABLE chaos_experiments (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
experiment_name TEXT NOT NULL,
namespace TEXT NOT NULL,
status TEXT, -- 'RUNNING', 'SUCCEEDED', 'FAILED'
started_at TIMESTAMPTZ,
finished_at TIMESTAMPTZ,
groq_summary TEXT,
logs_url TEXT,
created_by_gtid TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

4.3 Implementation Checklist

- ? Install Chaos Mesh on K3s cluster
- ? Deploy chaos-orchestrator (Rust)
- ? Deploy node-health-agent (Rust) with LSTM prediction
- ? Configure weekly chaos experiments (staging)
- ? Build Admin Portal "Infrastructure Health" dashboard
- ? Add production chaos toggle (multisig-gated)

PART 5: ADD-ON 5 - AUTOMATED PENETRATION TESTING

5.0 Purpose & Design Philosophy

Run weekly vulnerability scans against the SGTX staging environment (and optionally production with read-only scans). Critical findings block the CI/CD pipeline.

Key Principles:

- Zero-cost - OWASP ZAP, nuclei, Trivy all open-source
- CI/CD integration - Critical findings block deployment
- Automated reporting - Groq-generated summaries

AI Integration:

- A1: Groq-generated summaries
- A4: CI/CD integration

5.1 Architecture

5.2 Data Model Additions

sql

```

CREATE TABLE threat_findings (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tool_name TEXT NOT NULL,
severity TEXT NOT NULL, -- 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'

```

title TEXT NOT NULL,
description TEXT,
remediation TEXT,
cve_id TEXT,
affected_component TEXT,
finding_hash TEXT,
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);

5.3 Implementation Checklist

- ? Set up pentest-service (Rust)
- ? Configure Trivy, OWASP ZAP, nuclei, OpenVAS
- ? Schedule weekly scans (Friday 23:00 UTC)
- ? Integrate with CI/CD (critical findings block deployment)
- ? Build Admin Portal "Security" tab
- ? Add Groq-generated summary for each scan

PART 6: ADD-ON 6 - POST-QUANTUM CRYPTOGRAPHY (PQC)

6.0 Purpose & Design Philosophy

Protect long-term records (contracts, Loom hash chains, identity credentials, audit logs) against future quantum computer attacks. Adds a second layer of Dilithium3 signatures (NIST standard) and optionally Kyber1024 for key exchange.

Key Principles:

- Two-layer signature - Ed25519 (fast) + Dilithium3 (quantum-safe)
- Background re-signing - Weekly cron job re-signs records older than 1 year
- Zero-cost - liboqs, Rust oqs crate

AI Integration:

- A3: Key generation approval
- A4: Background re-signing

6.1 Architecture

6.2 Data Model Additions

sql

```
ALTER TABLE governor_decisions ADD COLUMN pqc_signature TEXT;  
ALTER TABLE contracts ADD COLUMN pqc_signature TEXT;  
ALTER TABLE shipments ADD COLUMN pqc_signature TEXT;
```

6.3 Implementation Checklist

- ? Generate Dilithium3 key pair (multisig ceremony)
- ? Implement dual-signature (Ed25519 + Dilithium3)
- ? Deploy pqc-resigner background job (weekly)
- ? Add PQC verification endpoint (/v1/verify/pqc)
- ? Publish public key at /.well-known/sgtx-keys

PART 7: ADD-ON 7 - EXPANDED ZERO-KNOWLEDGE PROOFS (ZK) & PROOF OF RESERVES

7.0 Purpose & Design Philosophy

Zero-Knowledge Proofs (ZK) enable privacy-preserving verification without revealing underlying sensitive data. SGTX implements four core use cases, plus a phased proof-of-reserves mechanism.

Use Cases:

- 1 Financier proof-of-reserves - Proves sufficient liquidity without revealing wallet balance
- 2 Confidential trade terms - Hides EXW price and logistics breakdown; buyer sees only final price
- 3 Private settlement - Proves payment occurred without revealing amount or PSP
- 4 Proof of Reserves (phased) - Initially via Big Four attestation, later via real-time ZK proofs

Key Principles:

- Optional - ZK proofs are never forced
- Client-side - Proof generation in WASM (Plonky3)
- Lightweight verification - Sub-millisecond server verification
- Zero-cost - Plonky3, WASM toolchain all open-source

AI Integration:

- A2: Proof verification
- A4: Governor integration

7.1 Architecture

Use Case 1 - Financier Proof-of-Reserves:

Before submitting a bid, a financier can optionally generate a ZK proof that their total stablecoin or fiat reserves exceed the offered amount.

Workflow:

- 1 Financier connects wallet(s) or bank account(s) via read-only API
- 2 Client-side script generates ZK proof that sum of selected reserves > amount_offered
- 3 Proof sent to SGTX with bid
- 4 Governor verifies proof; if valid, bid proceeds
- 5 No wallet address, exact balance, or chain is revealed

Use Case 2 - Confidential Trade Terms:

Seller can hide EXW price and logistics breakdown from platform; buyer sees only final quoted price.

Workflow:

- 1 Seller toggles "Make price confidential" in Quote Submission

2 System computes ZK proof that (EXW + logistics + SGTX fee) is within plausible range

3 Proof stored in seller_quotes.price_zk_proof; raw components not stored

4 Buyer sees only final total price

5 During dispute, proof can be revealed to arbitrator if needed

Use Case 3 - Private Settlement:

After settlement, payer can request a ZK proof of payment without revealing exact amount or PSP used.

Workflow:

1 Buyer clicks "Request Private Settlement Proof" in Settlement tab

2 Client-side script generates proof from PSP receipt and settlement instruction

3 Proof stored in settlement_confirmations.zk_proof

4 Buyer can share proof with auditors; they verify without seeing sensitive details

Use Case 4 - Proof of Reserves (Phased):

Phase 1 - Big Four Attestation (Initial, Months 1-12):

- Auditors: PwC, Deloitte, EY, KPMG (rotating annually)
- Content: Total assets (by currency and custodian), total liabilities, reserve ratio (?110% per constitutional rule), composition breakdown
- Publication: Signed PDF at /.well-known/reserves/attestation
- Frequency: Quarterly

Phase 2 - Real-time ZK Proofs (Mature, after >80% financing volume uses ZK-capable wallets):

- Trigger: Platform Governance Authority votes (3/5) when adoption threshold met
- Daily proof: Aggregated Merkle tree of all reserve accounts; ZK proof that total assets ? 110% of total liabilities
- Public endpoint: GET /v1/reserves/proof - returns latest ZK proof and metadata
- Privacy: Proof reveals only reserve ratio and commitment to asset list; no individual account balances disclosed

7.2 Data Model Additions

sql

```
ALTER TABLE financing_offers ADD COLUMN reserve_proof TEXT;
```

```
ALTER TABLE seller_quotes ADD COLUMN price_zk_proof TEXT;
```

```
ALTER TABLE settlement_confirmations ADD COLUMN zk_proof TEXT;
```

-- Platform reserves

```
CREATE TABLE platform_reserves (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
reporting_date DATE NOT NULL,
```

```
phase TEXT CHECK (phase IN ('ATTESTATION', 'ZK_PROOF')),
```

```
asset_total DECIMAL(20,2),
```

```

liability_total DECIMAL(20,2),
ratio DECIMAL(6,2),
attestation_pdf_url TEXT,
zk_proof TEXT,
verified BOOLEAN DEFAULT false,
verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE reserve_ratio_history (
id BIGSERIAL PRIMARY KEY,
ratio DECIMAL(6,2) NOT NULL,
checked_at TIMESTAMPTZ DEFAULT NOW()
);

```

7.3 Implementation Checklist

- ? Integrate Plonky3 and compile circuits for three use cases
- ? Add client-side WASM modules for proof generation
- ? Implement server-side verification endpoint
- ? Add UI toggles in Financier Portal, Seller Quote, Settlement
- ? Establish Phase 1 - Big Four attestation process
- ? Build daily liability aggregation
- ? Implement Phase 2 - HSM-based ZK proof generation
- ? Build reserve dashboard in Admin and Government Portals
- ? Update constitutional WASM module with reserve ratio rule

PART 8: ADD-ON 8 - CUSTOMS BOND & GUARANTEE MANAGEMENT

8.0 Purpose & Design Philosophy

This module manages all types of customs bonds, guarantees, and security deposits required for the conditional release of goods. It addresses the critical gap under Egyptian Customs Law 207/2020, Article 54 (bonds), and equivalent regulations worldwide.

Key Principles:

- Jurisdiction-aware - Different countries have different bond requirements
- AI-driven - Automatically calculates required bond amounts (A2, XGBoost)
- Non-custodial - Bonds are tracked, not held by SGTX
- Automated monitoring - Tracks bond expiry and sufficiency
- Integration-ready - Links with CBE for bank guarantee verification

AI Integration:

- A1 (Groq ? Ollama): Plain-language bond explanations, expiry reminders

- A2 (HF local ? Ollama): Bond amount calculation, risk assessment

- A4 (OPA + WasmEdge): Bond validation, enforcement

8.1 Jurisdiction-Specific Bond Requirements

8.1.1 Egypt (Customs Law 207/2020, Article 54)

8.1.2 European Union (UCC Articles 93-100)

8.1.3 United States (Customs Bond Directive 99-3510A)

8.1.4 UAE (Federal Customs Law)

8.1.5 Saudi Arabia (Customs Law)

8.1.6 United Kingdom (UK Customs Act)

8.2 Bond Calculation Engine (A2)

Input Parameters:

SGTX MASTER GLOBAL TRADE COMPLETION AMENDMENT

0. Purpose

This amendment defines the final target architecture for SGTX as a:

Global, non-custodial, relationship-controlled, sovereign-governed trade execution infrastructure connecting commercial trade, regulation, customs, government systems, logistics, financial settlement, delivery, post-clearance and evidence through one canonical USTN graph.

The objective is not to make SGTX a marketplace.

The objective is to make SGTX capable of taking an authorized trade between known parties and orchestrating everything required to execute that trade legally and operationally across jurisdictions.

1. SGTX CONSTITUTION - FINAL FORM

SGTX remains:

- non-custodial
- non-marketplace
- non-title-taking
- non-carrier
- non-customs-authority
- non-bank
- non-deposit-taking
- non-government
- AI-assisted
- Governor-governed
- OPA-enforced
- WasmEdge-enforced
- Loom-audited
- USTN-centric

- jurisdiction-aware
- relationship-controlled

SGTX does facilitate commercial/logistics services.

That means the platform may facilitate:

- approved freight forwarders
- approved LSPs
- shipping lines
- airlines
- rail operators
- ferry operators
- warehouses
- ground handlers
- terminals
- customs brokers
- laboratories
- QC/inspection providers
- insurance providers
- connected banks
- trader-saved financing institutions
- government-authorized agencies

The distinction is:

PUBLIC MARKETPLACE

X

RELATIONSHIP-CONTROLLED TRADE ORCHESTRATION

?

2. PROVIDER RELATIONSHIP MODEL

The final provider model is:

TRADER

?

APPROVED / CONNECTED / SAVED PROVIDER

?

RFQ / QUOTE

?

TRADER REVIEW

?

TRADER EXPLICIT SELECTION

?

SERVICE AGREEMENT / ADDENDUM

?

EXECUTION

A provider can become available to a trader because:

- 1 It is already approved and connected.
- 2 The trader has saved its GTID.
- 3 The trader explicitly selects it.
- 4 A government-mandated service relationship exists.

Do not introduce:

- random provider matching
- unsolicited provider recommendations
- "best provider"
- public provider rankings
- public provider marketplace
- autonomous provider selection

This preserves the blueprint's explicit non-marketplace/orchestration principle.

3. FINANCING RELATIONSHIP MODEL

Financing remains legitimate SGTX functionality.

Valid financing relationships:

TRADER

??? CONNECTED BANK

??? CONNECTED FINANCIAL INSTITUTION

??? SAVED FINANCIER GTID

Workflow:

Financing Need

?

Trader selects connected bank / saved financier

?

Financing request

?

Financier assessment

?

Offer

?

Trader acceptance

?

Financing execution

?

Disbursement

?

Repayment / settlement

Do not transform this into random public matching.

No unsolicited financier recommendations.

No public financier marketplace.

The existing blueprint's bank/PFI portal architecture remains usable, but access must be governed by relationship/GTID controls.

4. MASTER GLOBAL TRADE GRAPH

The entire SGTX platform should ultimately be represented as:

GTID

?

TRADE

?

RFQ

?

QUOTATION

?

ORDER

?

CONTRACT

?

REGULATORY SNAPSHOT

?

USTN

?

TRANSPORT GRAPH

?

CUSTOMS OPERATIONS

?

DOCUMENTS

?

GOVERNMENT REFERENCES

?

PAYMENT / FINANCE

?

PHYSICAL EXECUTION

?

DELIVERY

?

ACCEPTANCE

?

SETTLEMENT

?

RECONCILIATION

?

POST-CLEARANCE

?

CLAIMS / RETURNS

?

EVIDENCE

?

USTN CLOSED

Every downstream record ultimately references USTN.

5. GLOBAL CANONICAL DATA MODEL

Create/extend the canonical objects:

TRADE_PARTY

PRODUCT

TRADE_ITEM

RFQ

QUOTATION

ORDER

PURCHASE_ORDER

SALES_ORDER

PROFORMA

CONTRACT

INCOTERM
ORIGIN
DESTINATION
REGULATORY_PROFILE
LICENSE
PERMIT
CERTIFICATE
DOCUMENT
CUSTOMS_OPERATION
TRANSPORT_GRAPH
SHIPMENT
TRANSPORT_LEG
DUTY
TAX
GUARANTEE
PAYMENT
SETTLEMENT
INSURANCE
INSPECTION
SECURITY
DELIVERY
ACCEPTANCE
CLAIM
RETURN
POST_CLEARANCE
ACCOUNTING
EVIDENCE

Do not duplicate concepts already present in SGTX.

Extend existing models where possible.

6. GLOBAL JURISDICTION FABRIC

"Country" is insufficient as the fundamental legal object.

Create:

JURISDICTION

Support:

- sovereign country

- customs territory
- customs union
- economic union
- autonomous territory
- special administrative region
- free zone
- free port
- bonded zone
- special economic zone
- airport customs jurisdiction
- port customs jurisdiction
- tax territory
- export-control territory
- special regime

Each jurisdiction stores:

jurisdiction_id

ISO references

parent jurisdiction

customs territory

tax territory

regional bloc

customs authority

tax authority

SPS authority

TBT authority

export-control authority

transport authority

immigration authority

ports

airports

customs offices

special regimes

legal sources

effective dates

7. WORLDWIDE JURISDICTION REGISTRY

The architecture must support all countries worldwide, plus relevant customs and special jurisdictions.

Every jurisdiction begins:

NOT_ACTIVE

unless actually configured.

Possible states:

NOT_ACTIVE

COUNTRY_CONFIGURED

ADAPTER_READY

SANDBOX_CONNECTED

PRODUCTION_READY

PRODUCTION_CONNECTED

MANUAL_ONLY

PORTAL_ONLY

INTEGRATION_REQUIRED

LEGAL_AUTHORIZATION_REQUIRED

DEGRADED

DEPRECATED

Never confuse:

with:

8. GLOBAL REGULATORY SOURCE REGISTRY

Create:

REGULATORY_SOURCE

Each regulatory rule needs:

- source
- authority
- jurisdiction
- law/regulation
- article/section
- official URL
- publication date
- effective date
- expiry
- language
- hash
- verification status

- last checked

- next review

The WCO Data Model is particularly appropriate as the interoperability foundation because WCO describes it as a harmonized universal language for cross-border exchange and Single Window implementation.

9. REGULATORY SNAPSHOT

At contract/trade lock:

REGULATORY_SNAPSHOT

Capture:

- origin

- destination

- transit

- HS

- commodity

- Incoterm

- tariff

- tax

- origin rules

- applicable agreements

- licenses

- permits

- certificates

- customs procedure

- SPS

- TBT

- sanctions

- export controls

- transport restrictions

- government integration state

The snapshot must be immutable.

10. REGULATORY CHANGE MANAGEMENT

RIA should become a complete regulatory-change pipeline:

DISCOVER

?

VERIFY

?

CLASSIFY

?

IMPACT ANALYSIS

?

AFFECTED TRADES

?

POLICY UPDATE

?

SIMULATION

?

APPROVAL

?

WASM/POLICY RELEASE

?

DEPLOYMENT

?

LOOM

Never silently change production regulatory behavior.

11. PRODUCT REGULATORY PROFILE

Extend product intelligence with:

- HS6
- national tariff codes
- alternate classifications
- composition
- materials
- CAS
- brand
- model
- serial requirements
- agricultural classification
- food
- pharmaceutical
- veterinary
- chemical
- DG

- dual-use
- strategic
- CITES
- packaging
- labeling
- shelf life
- temperature
- conformity
- SPS

12. CLASSIFICATION ENGINE

Support:

- HS
- national tariff code
- statistical classification
- export classification
- dual-use
- strategic goods

AI assists classification.

AI does not become final legal authority.

Low confidence:

CONDITIONAL

Human review.

13. TARIFF ENGINE

Support:

- MFN
- preferential
- quotas
- anti-dumping
- countervailing
- safeguard
- agricultural levy
- excise
- environmental fee
- import surcharge
- other measures

Every rule must have:

- jurisdiction
- HS
- origin
- effective date
- legal source

14. ORIGIN ENGINE

Support:

- non-preferential origin
- preferential origin
- wholly obtained
- substantial transformation
- tariff shift
- regional value content
- processing rules
- cumulation
- direct shipment
- approved exporter
- origin declaration
- COO
- EUR.1/equivalent

The engine determines whether preferential tariff eligibility actually exists.

15. TRADE AGREEMENT ENGINE

Create a registry for:

- bilateral
- multilateral
- FTA
- PTA
- customs union
- regional agreements
- sectoral agreements

Store:

- parties
- dates
- tariff schedule

- product coverage
- origin rules
- quotas
- certification
- cumulation
- exclusions
- direct transport
- suspension.

16. LICENSE ENGINE

Support:

- import licenses
- export licenses
- transit licenses
- controlled-goods licenses
- strategic-goods licenses
- special product licenses

States:

NOT_REQUIRED

REQUIRED

APPLICATION_READY

SUBMITTED

PENDING

ISSUED

EXPIRED

REVOKED

REJECTED

17. PERMIT ENGINE

Support:

- SPS
- agriculture
- veterinary
- food
- pharmaceuticals
- environment
- chemicals

- communications
- strategic goods
- import
- export
- transit
- transport

Dynamic determination from product + jurisdiction + transaction.

18. CERTIFICATE ENGINE

Support:

- COO
- preferential COO
- EUR.1
- phytosanitary
- health
- veterinary
- laboratory
- analysis
- conformity
- inspection
- fumigation
- treatment
- cold treatment
- Halal
- organic
- insurance
- security.

19. SPS ENGINE

Determine requirements from:

- commodity
- HS
- origin
- destination
- season
- intended use
- mode

Support:

- plant health
- animal health
- food safety
- quarantine
- sampling
- treatment
- lab
- inspection
- release.

20. TBT ENGINE

Support:

- technical regulations
- mandatory standards
- conformity
- testing
- product registration
- labeling
- energy
- electrical safety
- EMC
- radio
- environmental requirements.

21. CONTROLLED-GOODS ENGINE

Support:

- dual-use
- military/strategic
- chemicals
- biological
- radioactive
- controlled medicine
- weapons-related
- cyber/advanced technology
- CITES

and:

- export licenses
- import licenses
- end-user statements
- end-use certificates
- re-export restrictions
- transit controls.

22. SANCTIONS ENGINE

Screen:

- trader
- seller
- buyer
- UBO
- banks
- logistics providers
- vessels
- aircraft
- ports
- relevant intermediaries

States:

CLEAR

POTENTIAL_MATCH

ENHANCED_DD

BLOCKED

AI assists.

A4 enforces.

23. CUSTOMS VALUATION ENGINE

Calculate/validate:

- transaction value
- freight
- insurance
- assists
- royalties
- commissions
- related-party adjustments
- currency

- customs valuation method

- tax base.

24. TRUE LANDED COST ENGINE

Calculate:

GOODS

+ ORIGIN COST

+ PACKAGING

+ INLAND

+ EXPORT CLEARANCE

+ CERTIFICATES

+ INSPECTION

+ INSURANCE

+ INTERNATIONAL FREIGHT

+ TRANSIT

+ DESTINATION HANDLING

+ DUTY

+ VAT/GST

+ EXCISE

+ BROKER

+ LOCAL DELIVERY

+ OTHER FEES

+ SGTX FEES

This becomes a decision-support component before trade lock.

25. ORDER MANAGEMENT

Expand commercial flow:

RFQ

? QUOTE

? NEGOTIATION

? PO

? SO

? PROFORMA

? CONTRACT

? FULFILLMENT

Support:

- partial

- amendment
- cancellation
- split shipment
- backorder
- replacement
- return.

26. INCOTERM ENGINE

Machine-read:

- cost responsibility
- risk responsibility
- transport
- insurance
- export clearance
- import clearance
- permits
- duty
- tax
- documents
- transfer point
- delivery.

The existing blueprint already ties transport configuration and Incoterms to the trade request.

27. UNIFIED DOCUMENT ENGINE

All documents remain centralized.

Classify:

COMMERCIAL

CUSTOMS

REGULATORY

TRANSPORT

FINANCIAL

SECURITY

INSURANCE

Each has legal metadata.

28. DOCUMENT CONSISTENCY ENGINE

Cross-check:

- order

- contract
- invoice
- packing
- COO
- certificates
- customs
- AWB
- B/L
- e-CMR
- RoRo documents
- permits
- insurance
- manifest
- settlement.

Check:

- party
- quantity
- weight
- value
- HS
- currency
- origin
- destination
- dates
- transport identity.

29. DOCUMENT AUTHENTICATION / LEGALIZATION

Support:

- chamber authentication
- government authentication
- foreign ministry legalization
- consular legalization
- apostille
- certified translation
- digital signature
- electronic seal

- qualified electronic signature

30. MULTI-AGENCY GOVERNMENT ENGINE

Use:

SEQUENTIAL

PARALLEL

CONDITIONAL

RISK_TRIGGERED

OPTIONAL

Example:

CUSTOMS

+ AGRICULTURE

+ FOOD

+ HEALTH

+ STANDARDS

+ SECURITY

?

RELEASE

This builds directly on the blueprint's existing Multi-Agency Workflow.

31. GLOBAL CUSTOMS ENGINE

Support:

- import
- export
- transit
- temporary import
- temporary export
- inward processing
- outward processing
- bonded warehouse
- customs warehouse
- free zone
- re-export
- re-import
- destruction
- abandonment
- drawback

- post-clearance.

32. GLOBAL SINGLE-WINDOW GATEWAY

Use:

- API
- EDI
- XML
- JSON
- UN/EDIFACT
- SFTP
- portal
- broker
- manual.

Map:

SGTX Canonical Model

?

WCO / Regional Model

?

National Model

?

Authority System

The WCO Data Model specifically supports harmonized datasets for customs and other border agencies in Single Window environments.

33. GOVERNMENT CONNECTOR STANDARD

Every connector supports:

DISCOVER

AUTHENTICATE

VALIDATE

PREPARE

SUBMIT

STATUS

AMEND

CANCEL

INSPECT

RELEASE

DOCUMENT

PERMIT

CERTIFICATE

PAYMENT

RECONCILE

34. GOVERNMENT CONNECTOR STATES

Use:

NOT_DISCOVERED

DISCOVERED

DOCUMENTED

CONTACT_REQUIRED

CREDENTIALS_REQUIRED

SANDBOX_AVAILABLE

SANDBOX_CONNECTED

CERTIFICATION_REQUIRED

CERTIFICATION_PENDING

PRODUCTION_READY

PRODUCTION_CONNECTED

DEGRADED

OUTAGE

PORTAL_ONLY

MANUAL_ONLY

DEPRECATED

35. GOVERNMENT AUTHORITATIVE STATUS

Separate:

SGTX_READY

SUBMITTED

GOVERNMENT_ACCEPTED

GOVERNMENT_REJECTED

GOVERNMENT_HOLD

GOVERNMENT_RELEASED

MANUAL_AUTHORITY_CONFIRMED

SGTX never invents government approval.

36. GOVERNMENT INTEGRATION CONTROL CENTER

The existing government portal already provides integration management, connection testing and connector logs.

Upgrade it globally.

Display:

- authority
- system
- jurisdiction
- mode
- procedure
- API
- EDI
- portal
- credentials
- certificate
- sandbox
- certification
- legal agreement
- production status
- last success
- last error
- version
- owner.

37. FOUR-DIMENSION EXTERNAL READINESS

Every integration reports independently:

TECHNICAL

LEGAL

OPERATIONAL

COMMERCIAL

This avoids saying:

"Connected"

when technically connected but legally uncertified.

38. MISSING-INTEGRATION CENTER

Every missing requirement must show:

JURISDICTION

AUTHORITY

SYSTEM

PROCEDURE

MODE

WHY_REQUIRED

API

EDI

PORTAL

SANDBOX

CREDENTIALS

CERTIFICATION

LEGAL_AGREEMENT

STATUS

PRIORITY

OWNER

NEXT_ACTION

AFFECTED_USTNs

AFFECTED_TRADE_LANES

Priority:

P0 = legally blocks

P1 = major trade-lane blocker

P2 = manual operation

P3 = optimization

P4 = optional

39. COUNTRY ACTIVATION

Country activation workflow:

SELECT JURISDICTION

?

LOAD OFFICIAL SOURCES

?

CONFIGURE CUSTOMS

?

CONFIGURE TAX

?

CONFIGURE SPS

?

CONFIGURE TBT

?

CONFIGURE LICENSES

?

CONFIGURE TRANSPORT

?

IDENTIFY GOVERNMENT SYSTEMS

?

IDENTIFY APIs/EDI/PORTALS

?

CREDENTIALS

?

SANDBOX

?

CONFORMANCE

?

LEGAL REVIEW

?

PRODUCTION AUTHORIZATION

?

ACTIVATE

?

LOOM

40. COUNTRY ACTIVATION ? PRODUCTION ACTIVATION

This rule must be enforced everywhere.

For example:

COUNTRY_CONFIGURED

+

ADAPTER_READY

+

NO PRODUCTION CREDENTIALS

must remain:

INTEGRATION_REQUIRED

not:

PRODUCTION_CONNECTED.

41. TRADE LANE PASSPORT

For every:

ORIGIN

DESTINATION

TRANSIT

COMMODITY

HS

MODE

INCOTERM

generate a:

TRADE_LANE_PASSPORT

containing:

- legal
- customs
- licenses
- permits
- certificates
- transport
- providers
- broker
- tax
- duty
- government systems
- payment
- insurance
- manual steps
- expected exceptions.

The blueprint already describes Trade Lane Passport and Corridor Registry as core TCN concepts.

42. TRADE-LANE READINESS

Calculate:

PARTY_READY

PRODUCT_READY

CLASSIFICATION_READY

ORIGIN_READY

LICENSE_READY

PERMIT_READY

CERTIFICATE_READY

CUSTOMS_READY

TRANSPORT_READY

SECURITY_READY

TAX_READY

PAYMENT_READY

INSURANCE_READY

DELIVERY_READY

GOVERNMENT_CONNECTIVITY_READY

43. ROAD CORRIDOR ENGINE

Road becomes a first-class engine:

ROAD_CORRIDOR

ROAD_LEG

BORDER

CUSTOMS

TRANSIT

GUARANTEE

DRIVER

VEHICLE

TRAILER

SEAL

WAYBILL

GPS

GEOFENCE

INCIDENT

POD

Support:

- Egypt
- Jordan
- Saudi
- UAE
- GCC
- Iraq
- Libya
- all future jurisdictions.

44. ROAD MULTI-COUNTRY WORKFLOW

Example:

Egypt

?

Export Customs

?

Border

?

Jordan Transit

?

Saudi Transit

?

UAE Import Customs

?

Final Delivery

Every leg has its own legal/customs status.

45. TIR / CUSTOMS GUARANTEE ENGINE

Generalize beyond TIR:

TIR

eTIR

CUSTOMS BOND

BANK GUARANTEE

TRANSIT GUARANTEE

COMPREHENSIVE GUARANTEE

DUTY DEFERRAL

TEMPORARY ADMISSION

WAREHOUSE GUARANTEE

46. ROAD VEHICLE / DRIVER AUTHORIZATION

Validate:

- vehicle registration

- insurance

- roadworthiness

- payload

- trailer

- reefer

- DG capability

- country permits

Driver:

- passport
- license
- visa
- permit
- international authorization
- DG authorization.

47. AIR CARGO ENGINE

First-class:

AIR_BOOKING

AIRLINE

AIRPORT

GHA

FLIGHT

MAWB

HAWB

e-AWB

CARGO-XML

ONE RECORD

SECURITY

e-CSD

DG

e-DGD

ULD

RCS

DEP

ARR

RCF

NFD

DLV

CUSTOMS

ACI AIR

WAREHOUSE

BUILD-UP

BREAKDOWN

IATA currently describes ONE Record as the air cargo standard for a single digital shipment view using a common data model and secured API. Its 2026 Cargo-XML toolkit explicitly includes mapping between

Cargo-XML and ONE Record and e-CSD-related guidance.

48. AIR CUTOFF ENGINE

Support:

- documentation cutoff
- customs cutoff
- security cutoff
- acceptance cutoff
- airline cutoff
- buildup cutoff
- ULD cutoff.

49. AIR CHARGEABLE WEIGHT

Calculate:

- actual
- volumetric
- dimensional
- chargeable.

Feed:

- quotation
- booking
- AWB
- customs
- insurance
- settlement.

50. AIR SECURITY / DG / SPECIAL CARGO

Support:

- screening
- e-CSD
- DG
- e-DGD
- pharma
- perishables
- live animals
- human remains
- valuables
- temperature controlled

- oversized
- lithium batteries.

Rules must be versioned.

51. AIR PIECE / ULD TRACKING

Track:

USTN

MAWB

HAWB

PIECE

SSCC

ULD

FLIGHT

LOCATION

SECURITY

CUSTOMS

TEMPERATURE

52. AIR MULTIMODAL

Support:

ROAD ? AIR

AIR ? ROAD

ROAD ? AIR ? ROAD

ROAD ? AIR ? AIR ? ROAD

Use Road Engine for road segments.

53. OCEAN CONTAINER ENGINE

Keep container ocean as a distinct engine:

BOOKING

VESSEL

VOYAGE

PORT

CONTAINER

VGM

B/L

e-B/L

MANIFEST

ACI

CUSTOMS

TERMINAL

GATE

TRANSSHIPMENT

DEMURRAGE

DETENTION

DELIVERY

54. RAIL ENGINE

Support:

- rail booking
- train
- wagon
- terminal
- consignment
- transit
- customs
- tracking
- interchange
- delivery.

55. NEW FIRST-CLASS ENGINE: RORO & ROLLING CARGO

This is the major addition requested now.

Do not treat RoRo as simply Ocean.

Create:

RORO & ROLLING CARGO ENGINE

The transport hierarchy becomes:

TRANSPORT ORCHESTRATOR

?

??? ROAD

??? AIR

??? OCEAN CONTAINER

??? RORO / ROLLING CARGO

??? RAIL

??? FERRY

??? MULTIMODAL

56. RORO MASTER OBJECT

Create:

RORO_SHIPMENT

under USTN.

Structure:

USTN

??? RORO_SHIPMENT

??? BOOKING

??? VOYAGES

??? PORT_CALLS

??? MANIFEST

??? RORO_UNITS

??? CUSTOMS

??? INSPECTIONS

??? YARD

??? LOADING

??? DISCHARGE

??? DELIVERY

??? PAYMENTS

??? CLAIMS

??? EVIDENCE

57. RORO UNIT IDENTITY

Every rolling unit has:

RORO_UNIT_ID

VIN

CHASSIS

REGISTRATION

MAKE

MODEL

YEAR

VEHICLE_TYPE

WEIGHT

DIMENSIONS

FUEL

BATTERY

RUNNING_STATUS

CONDITION

OWNER

EXPORTER

IMPORTER

CONSIGNEE

HS

ORIGIN

For non-vehicle rolling cargo:

- serial
- equipment ID
- dimensions
- weight
- condition.

58. RORO VEHICLE TYPES

Support:

- passenger vehicles
- trucks
- trailers
- buses
- tractors
- agricultural equipment
- construction machinery
- industrial equipment
- motorcycles
- boats on trailers
- non-running units
- oversized machinery.

59. RORO BOOKING ENGINE

Support:

- port pair
- vessel
- voyage
- number of units
- VIN
- dimensions

- weight
- running/non-running
- special handling
- hazardous status
- oversized
- preferred sailing
- delivery window
- Incoterm.

60. RORO VOYAGE MODEL

Track:

VESSEL

IMO

VOYAGE

OPERATOR

ORIGIN

DESTINATION

TRANSIT PORTS

ETD

ETA

ACTUAL DEPARTURE

ACTUAL ARRIVAL

BOOKING CUTOFF

DOCUMENT CUTOFF

GATE CUTOFF

CARGO CUTOFF

STATUS

61. RORO MANIFEST

Manifest must be unit-level.

Every unit links:

VIN

RORO_UNIT_ID

BOOKING

VOYAGE

SHIPPER

CONSIGNEE

HS

WEIGHT

DESTINATION

CUSTOMS

GATE

LOADING

62. VIN-LEVEL CUSTOMS RECONCILIATION

SGTX must answer:

- Is this VIN customs-cleared?
- Which declaration?
- Which voyage?
- Which port?
- Was it gated in?
- Was it loaded?
- Was it discharged?
- Where is it in the yard?
- Has it gated out?
- Has it been delivered?

63. RORO TERMINAL ADAPTER

Support:

- pre-advice
- booking
- gate appointment
- gate-in
- VIN scan
- inspection
- yard assignment
- parking
- stock
- loading
- ramp assignment
- discharge
- gate-out.

64. RORO YARD ENGINE

Track:

YARD

ZONE

BLOCK

ROW

SLOT

DECK

POSITION

VIN

UNIT

STATUS

ARRIVAL

STORAGE START

FREE TIME

Statuses:

EXPECTED

ARRIVED

GATE_IN

INSPECTED

PARKED

READY_FOR_LOADING

LOADED

DISCHARGED

AVAILABLE

RELEASED

GATE_OUT

65. RORO GATE ENGINE

Origin:

APPOINTMENT

? ARRIVAL

? VIN SCAN

? DOCUMENT CHECK

? CUSTOMS CHECK

? INSPECTION

? GATE-IN

Destination:

DELIVERY ORDER

? CUSTOMS RELEASE

? VIN SCAN

? CONDITION CHECK

? GATE-OUT

66. RORO INSPECTION ENGINE

Capture:

- VIN
- time
- inspector
- location
- photos
- videos
- mileage
- fuel
- battery
- keys
- tires
- glass
- mirrors
- exterior
- interior
- pre-existing damage
- new damage.

67. RORO DAMAGE COMPARISON

Compare origin versus destination.

AI A2:

POSSIBLE_DAMAGE

Human/authorized inspector:

CONFIRMED_DAMAGE

States:

NO_CHANGE

POSSIBLE_DAMAGE

CONFIRMED_DAMAGE

DISPUTED_DAMAGE

AI must never determine liability autonomously.

68. RORO SECURITY

Support:

- VIN
- RFID
- QR
- barcode
- digital seal
- GPS
- access control
- key custody
- restricted yard
- security events.

Egypt's Ministry of Transport describes digital integration between Damietta and Trieste, exchange of official/customs data and RFID-based digital-seal safety for the Egypt-Italy RoRo line.

69. RORO STOWAGE ENGINE

Inputs:

- dimensions
- weight
- destination
- port sequence
- deck
- ramp
- height
- restrictions
- vehicle type.

Outputs:

- deck
- zone
- position
- load order
- discharge order.

A2 optimizes.

A4 validates.

70. RORO CAPACITY ENGINE

Support:

- units
- linear meters
- square meters
- cubic meters
- weight
- deck
- height.

Do not assume container TEU is the appropriate capacity metric.

71. RORO CUTOFF ENGINE

Track:

BOOKING_CUTOFF

DOCUMENT_CUTOFF

CUSTOMS_CUTOFF

GATE_IN_CUTOFF

CARGO_CUTOFF

LOADING_CUTOFF

72. RORO BL ENGINE

Create:

RORO_BILL_OF_LADING

Support:

- master B/L
- house references where applicable
- shipper
- consignee
- notify party
- vessel
- voyage
- POL
- POD
- transit
- VINS
- cargo descriptions
- weight
- dimensions
- freight

- charges.

73. RORO SHIPPING-INSTRUCTION ENGINE

Generate shipping instructions automatically from:

- contract
- USTN
- units
- VINs
- customs
- voyage
- consignee
- origin
- destination.

No re-entry.

74. RORO UNIT STATE MACHINE

BOOKED

DOCUMENTS_PENDING

CUSTOMS_PENDING

READY_FOR_GATE

GATE_IN

INSPECTION_PENDING

INSPECTED

YARD

READY_FOR_LOAD

LOADED

AT_SEA

TRANSSHIPMENT

DISCHARGED

DESTINATION_YARD

CUSTOMS_HOLD

CUSTOMS_RELEASED

DELIVERY_ORDER

READY_FOR_GATE_OUT

GATE_OUT

DELIVERED

ACCEPTED

75. RORO VESSEL STATE

Separately:

SCHEDULED

BOOKING_OPEN

CUTOFF_APPROACHING

CARGO_ACCEPTING

LOADING

DEPARTED

AT_SEA

TRANSSHIPMENT

ARRIVED

DISCHARGING

COMPLETED

Never mix vessel state with cargo/unit state.

76. RORO CUSTOMS RECONCILIATION

Reconcile:

VIN

? Commercial Docs

? Customs

? Manifest

? Vessel/Voyage

? Terminal

? Release

? Delivery

77. EGYPT RORO ADAPTER

Create:

EGYPT_RORO_ADAPTER

The adapter determines:

- Nafeza applicability
- UCR applicability
- export declaration
- transit
- manifest
- shipping-agent messages
- terminal process

- customs procedures
- port requirements
- unit documentation
- vehicle documentation
- destination/transit requirements.

Do not blindly apply container-only Enhanced Export messages to RoRo.

Nafeza's July 18, 2026 Enhanced Export notice explicitly identifies its first five messages-UCR Verification, Booking Confirmation, Empty Containers, Shipment Inquiry and Manifest-for seaports/container shipping agents.

The system therefore needs mode/procedure applicability rather than one universal maritime workflow.

78. EGYPT AIR ADAPTER

Egypt's Nafeza system currently identifies an Aviation ACI system, with mandatory implementation beginning January 1, 2026, and a separate enhanced export cycle.

Therefore support:

- Nafeza
- ACI Air
- export
- import
- transit where applicable
- MAWB
- HAWB
- manifest
- customs
- certificates
- release
- government reference reconciliation.

79. EGYPT ROAD ADAPTER

Support:

- Nafeza export
- UCR where applicable
- export declaration
- transit
- customs
- SPS
- certificates
- TIR

- road waybill
- border
- vehicle
- driver
- seal.

Do not assume maritime ACI/CargoX logic applies to road exports.

80. EGYPT MODE-APPLICABILITY ENGINE

Create:

TRANSPORT_MODE_PROCEDURE_ENGINE

Modes:

ROAD

AIR

OCEAN_CONTAINER

RORO

BULK

RAIL

FERRY

MULTIMODAL

The engine determines:

- customs procedure
- government messages
- identifiers
- ACI requirements
- manifest
- declaration
- permits
- terminal requirements.

81. EGYPT RO-RO CORRIDORS

The architecture should support the existing Egypt-Europe-Gulf RoRo corridor model.

The Ministry of Transport describes the Damietta-Trieste line and multimodal onward movement by rail/road, including Gulf destinations.

Represent such a corridor as:

Egypt

?

Damietta

?

RORO

?

Trieste

?

Rail

?

Rotterdam

?

Road

?

Destination

One USTN.

Multiple mode legs.

82. RORO + GLOBAL TCN

The existing Trade Corridor Network becomes:

TCN

?

CORRIDOR

?

PORT

?

RORO SERVICE

?

VESSEL

?

VOYAGE

?

RORO UNITS

Do not create another corridor database.

83. MULTIMODAL ORCHESTRATOR

Final transport hierarchy:

TRANSPORT ORCHESTRATOR

?

??? ROAD

??? AIR

??? OCEAN CONTAINER

??? RORO / ROLLING CARGO

??? RAIL

??? FERRY

??? MULTIMODAL

Each specialist engine owns its mode-specific rules.

The Multimodal Orchestrator joins them.

84. RORO + ROAD

Typical:

TRUCK

?

RORO TERMINAL

?

RORO VESSEL

?

DESTINATION RORO TERMINAL

?

TRUCK

One USTN.

85. RORO + RAIL

Example:

RORO

?

Trieste

?

RAIL

?

Rotterdam

?

ROAD

This is explicitly aligned with the Egyptian RoRo corridor example.

86. RORO + GCC TRANSIT

Support:

Europe

?

Trieste

?

Egypt

?

Damietta

?

Indirect Transit

?

Gulf

The Egyptian Maritime Transport Sector reported in June 2026 that the Damietta-Trieste RoRo line was handling indirect-transit shipments toward Saudi Arabia, UAE, Qatar, Oman and Kuwait.

SGTX should represent those as actual multi-jurisdiction transit legs, not just "delivery."

87. GLOBAL TRANSPORT PROVIDER ADAPTER

Every approved provider gets:

- GTID
- service scope
- jurisdictions
- licenses
- insurance
- routes
- commodities
- equipment
- API/EDI/portal
- contract
- SLA
- fees.

Provider visibility remains controlled by relationship/approval.

88. CUSTOMS BROKER ENGINE

Global broker workspace supports:

- declarations
- documents
- customs questions
- inspections
- amendments
- fees
- guarantees

- releases
- transit
- post-clearance.

Broker licenses must be validated against jurisdiction/scope.

89. GLOBAL TAX ENGINE

Support:

- VAT
- GST
- sales tax
- excise
- import tax
- withholding
- environmental levy
- special tax
- e-invoice
- e-receipt
- tax payment
- refund.

Never infer:

CUSTOMS RELEASED = TAX COMPLETE

90. GLOBAL FINANCIAL EXECUTION

Support:

- bank transfer
- PSP
- open banking
- local rails
- SWIFT
- ISO 20022
- documentary collection
- LC
- guarantees
- deferred government fees
- refunds.

SGTX orchestrates instructions/status but does not become the bank.

91. TRADE FINANCE

Support:

- connected bank financing
- trader-selected financier
- LC
- documentary collection
- guarantee
- standby
- collateral
- disbursement
- repayment
- reconciliation.

92. INSURANCE

Full lifecycle:

QUOTE

? BIND

? POLICY

? CERTIFICATE

? INCIDENT

? CLAIM

? SURVEY

? SETTLEMENT

? RECOVERY

? CLOSE

93. ACCOUNTING

Support:

- AP
- AR
- landed cost
- freight
- duty
- tax
- insurance
- accrual
- settlement
- refund

- FX
- inventory
- COGS.

94. ERP

Adapters:

- SAP
- Oracle
- Dynamics
- NetSuite
- Odoo
- generic REST
- SOAP
- EDI
- SFTP.

95. CUSTOMS GUARANTEE ENGINE

General guarantee system:

- TIR
- eTIR
- customs bond
- bank guarantee
- comprehensive guarantee
- transit guarantee
- temporary admission
- duty deferral
- warehouse guarantee.

96. BONDED / SPECIAL REGIMES

Support:

- bonded warehouse
- customs warehouse
- free zone
- free port
- inward processing
- outward processing
- temporary admission
- temporary export

- duty suspension
- end-use relief
- drawback.

97. POST-CLEARANCE

Support:

- audit
- query
- correction
- reassessment
- refund
- drawback
- penalty
- appeal
- ruling
- record retrieval.

98. DELIVERY ACCEPTANCE

Final delivery requires:

- receiver
- quantity
- condition
- quality
- temperature where applicable
- POD
- document acceptance
- acceptance timestamp.

DELIVERED is not the same as ACCEPTED.

99. CLAIMS

Support:

- damage
- shortage
- quality
- temperature
- delay
- customs
- documentation

- carrier
- insurance
- warranty.

100. RETURNS / REVERSE TRADE

Support:

- rejection
- return
- warranty
- repair
- replacement
- re-export
- re-import
- destruction
- abandonment.

Use parent/child USTNs.

v13.0 INTEGRATION BANNER — PREVIOUSLY-MISSING CHANGE-SET SEGMENT (ADD-ONS 9-28 AND FINAL SUMMARY PREAMBLE)

The content that follows (from "PART 9: ADD-ON 9" through the article immediately preceding "101. FINAL EVIDENCE") was absent from the v12.0 appended amendment body. Per audit finding A-24, it has been re-integrated here in its canonical position, restoring the complete change-set sequence: Add-Ons 1-8 (already present above), then Add-Ons 9-28, then the Final Summary preamble (the all-add-on coverage table and articles 0 through 100 of the SGTX Master Global Trade Completion Amendment), and finally the existing tail of the Final Summary (articles 101 through 162) that was already present. No existing v12.0 content has been removed or rewritten; this is a pure insertion.

----- BEGIN v13.0 RE-INTEGRATED SEGMENT (Add-Ons 9-28 and Final Summary preamble) -----

PART 9: ADD-ON 9 - DEMURRAGE & DETENTION MANAGEMENT

9.0 Purpose & Design Philosophy

Demurrage and detention fees are the #1 cost driver in container shipping. This module automatically extracts carrier tariffs and tracks free time in real-time.

Key Principles:

- Carrier tariff extraction – A2 (HF Donut) extracts terms from PDF/EDI quotes
- Real-time free time monitoring – Per port, container type, carrier
- Proactive alerts – 48h, 24h, expiry notifications
- Jurisdiction-aware – Different ports have different norms
- Integration with settlement – Demurrage included in final invoice

AI Integration:

- A1 (Grok → Ollama): Alert summaries, explanations

- A2 (HF local → Ollama): Tariff extraction, congestion prediction
- A4 (OPA + WasmEdge): Settlement enforcement

9.1 Carrier Demurrage Tariff Extraction

Extracted from carrier quotations (Part 3B.3.5.3):

json

```
{
  "carrier": "MSC Egypt",
  "container": "40ft Reefer",
  "free_time": {
    "demurrage_days": 5,
    "detention_days": 7,
    "combined_days": 12
  },
  "demurrage_rates": {
    "day_1-3": 0,
    "day_4-7": 150,
    "day_8-14": 200,
    "day_15+": 250
  },
  "detention_rates": {
    "day_1-3": 0,
    "day_4-7": 100,
    "day_8-14": 150,
    "day_15+": 200
  },
  "currency": "USD",
  "source": "MSC Tariff 2026-001"
}
```

9.2 Port-Specific Free Time Norms

Country	Port	Standard Free Time	Extension Policy
Egypt	Alexandria, Damietta, Port Said	5 days	Request up to 3 days
Egypt	Safaga, Port Tawfik	7 days	Request up to 5 days
Germany	Hamburg, Bremerhaven	7 days	Up to 14 days with approval
UAE	Jebel Ali, Khalifa Port	7 days	Up to 14 days with approval
Saudi	Jeddah, Dammam, Yanbu	7 days	Up to 10 days with approval
Italy	Trieste, Livorno, Genoa	5 days	Up to 10 days with approval
USA	New York, Los Angeles	4 days	Up to 7 days with approval
UK	Felixstowe, Southampton	5 days	Up to 10 days with approval
Brazil	Santos, Paranaguá	5 days	Up to 7 days with approval
China	Shanghai, Shenzhen	7 days	Up to 14 days with approval
India	Mumbai, Chennai	5 days	Up to 7 days with approval

9.3 Demurrage Calculation Engine

```

rust
fn calculate_demurrage(
  container_type: &str,
  carrier: &str,
  port: &str,
  release_date: DateTime,
  gate_out_date: DateTime,
  free_time_days: i32,
  carrier_tariff: &CarrierTariff
) -> DemurrageCalculation {
  let days_used = (gate_out_date - release_date).num_days();
  let excess_days = days_used - free_time_days;
  if excess_days <= 0 {
    return DemurrageCalculation {
      days_used: days_used,
      excess_days: 0,
      demurrage_amount: 0,
      detention_amount: 0,
      total_amount: 0,
      status: "FREE_TIME"
    };
  }

  // Determine if demurrage or detention applies
  let demurrage_applies = // based on milestone state
  let detention_applies = !demurrage_applies;
  let demurrage = if demurrage_applies {
    calculate_tiered_amount(carrier_tariff.demurrage_rates, excess_days)
  } else { 0.0 };
  let detention = if detention_applies {
    calculate_tiered_amount(carrier_tariff.detention_rates, excess_days)
  } else { 0.0 };
  DemurrageCalculation {
    days_used,
    excess_days,
    demurrage_amount: demurrage,

```

```
detention_amount: detention,
total_amount: demurrage + detention,
status: if excess_days > 30 { "ESCALATED" } else { "DEMURRAGE_STARTED" }
}
}
```

Output Example:

json

```
{
  "container_type": "40ft Reefer",
  "carrier": "MSC Egypt",
  "port": "Alexandria",
  "release_date": "2026-06-20T08:00:00Z",
  "gate_out_date": "2026-07-01T14:00:00Z",
  "free_time_days": 5,
  "days_used": 11,
  "excess_days": 6,
  "demurrage": {
    "days": 6,
    "rates": {
      "day_1-3": 0,
      "day_4-7": 150,
      "day_8-14": 200,
      "day_15+": 250
    },
    "amount": 1050,
    "breakdown": [
      {"day_range": "day_4-7", "days": 3, "rate": 150, "amount": 450},
      {"day_range": "day_8-14", "days": 3, "rate": 200, "amount": 600}
    ]
  },
  "total_amount": 1050.00,
  "currency": "USD",
  "status": "DEMURRAGE_STARTED",
  "explanation": "Container was released on 20 Jun and gated out on 1 Jul, 11 days total. Free time is 5 days. Excess 6 days accrued demurrage of USD 1,050."
}
```

9.4 Database Schema

sql

-- Demurrage tracking

```
CREATE TABLE demurrage_tracking (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,  
  container_number TEXT NOT NULL,  
  carrier_gtid TEXT REFERENCES tenants(gtid),  
  port_unlocode TEXT NOT NULL REFERENCES ports(unlocode),  
  container_type TEXT NOT NULL,  
  free_time_days INTEGER NOT NULL,  
  release_date TIMESTAMPTZ NOT NULL,  
  gate_out_date TIMESTAMPTZ,  
  actual_days_used INTEGER,  
  excess_days INTEGER DEFAULT 0,  
  demurrage_amount DECIMAL(20,4) DEFAULT 0,  
  detention_amount DECIMAL(20,4) DEFAULT 0,  
  total_amount DECIMAL(20,4) DEFAULT 0,  
  currency TEXT DEFAULT 'USD',  
  status TEXT DEFAULT 'FREE_TIME',  
  demurrage_breakdown JSONB,  
  settled BOOLEAN DEFAULT false,  
  settled_at TIMESTAMPTZ,  
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Carrier tariff database

```
CREATE TABLE carrier_demurrage_tariffs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  carrier_gtid TEXT NOT NULL REFERENCES tenants(gtid),  
  port_unlocode TEXT NOT NULL REFERENCES ports(unlocode),  
  container_type TEXT NOT NULL,  
  free_time_days INTEGER NOT NULL,  
  demurrage_rates JSONB,
```

```

detention_rates JSONB,
currency TEXT DEFAULT 'USD',
valid_from TIMESTAMPTZ,
valid_to TIMESTAMPTZ,
is_active BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Port free time defaults
CREATE TABLE port_free_time (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
port_unlocode TEXT NOT NULL REFERENCES ports(unlocode),
container_type TEXT NOT NULL,
free_time_days INTEGER NOT NULL,
extension_days INTEGER DEFAULT 0,
extension_policy TEXT, -- 'AUTO_APPROVE', 'APPROVAL_REQUIRED', 'NOT_AVAILABLE'
carrier_specific BOOLEAN DEFAULT false,
last_updated TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(port_unlocode, container_type)
);

-- Demurrage alerts
CREATE TABLE demurrage_alerts (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
alert_type TEXT NOT NULL, -- 'FREE_TIME_48H', 'FREE_TIME_24H', 'DEMURRAGE_STARTED',
'ESCALATED'
message TEXT NOT NULL,
sent_to TEXT[] NOT NULL,
sent_at TIMESTAMPTZ DEFAULT NOW(),
acknowledged BOOLEAN DEFAULT false,
acknowledged_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_demurrage_tracking_ustn ON demurrage_tracking(ustn);

```



```
CREATE INDEX idx_demurrage_tracking_container ON demurrage_tracking(container_number);
CREATE INDEX idx_demurrage_tracking_status ON demurrage_tracking(status);
CREATE INDEX idx_demurrage_tracking_release_date ON demurrage_tracking(release_date);
CREATE INDEX idx_demurrage_alerts_ustn ON demurrage_alerts(ustn);
CREATE INDEX idx_demurrage_alerts_acknowledged ON demurrage_alerts(acknowledged);
```

9.5 API Endpoints

Method	Path	Description	GET	/v1/demurrage/{ustn}	Get current demurrage status
GET	/v1/demurrage/forecast/{ustn}	Forecast future demurrage			
POST	/v1/demurrage/alert/acknowledge	Acknowledge demurrage alert			
GET	/v1/demurrage/tariff	Get carrier tariff for port/container			
POST	/v1/demurrage/calculate	Calculate demurrage for a shipment			
GET	/v1/demurrage/port/{unlocode}	Get port free time norms			

9.6 Smart Inbox Integration

Priority	Category	Example Title	Action
95 (High)	Free Time	48h	"Container MEDU1234567 free time expires in 48h"
95 (High)	Plan Release	24h	"Container MEDU1234567 free time expires in 24h"
90 (High)	Urgent Release	Demurrage Started	"Demurrage started for USTN-EG-001 – \$150/day"
85 (High)	Escalated	Demurrage > 30 days for USTN-EG-001 – escalate	"Dispute"

9.7 Integration with Existing Blueprint

Blueprint Part	Integration
Part 3B.3.5.3 (Mode C)	Extract tariffs from carrier quotes
Part 3B.6 (Milestones)	Trigger demurrage start on release; stop on gate-out
Part 6 (Payment Orchestrator)	Include demurrage in settlement calculations
Part 8 (Container Release)	Release date triggers tracking
Part 12A.1 (Smart Inbox)	Proactive alerts
Part 10 (Dispute)	Demurrage dispute category
Part 4.7 (Transport)	Port congestion prediction (A2)

9.8 Implementation Checklist

Phase 1: Foundation

- Create demurrage_tracking table
- Create carrier_demurrage_tariffs table
- Create port_free_time table
- Create demurrage_alerts table

Phase 2: Extraction & Calculation

- Implement tariff extraction from carrier quotes (A2, HF Donut)
- Implement demurrage calculation engine
- Implement free time monitoring
- Implement port congestion prediction (A2)

Phase 3: Alerts & Integration

- Implement proactive alerts (48h, 24h, expiry)
- Integrate with Smart Inbox
- Integrate with settlement calculations
- Add demurrage dispute category (Part 10)

PART 10: ADD-ON 10 - BROKER LIABILITY & INSURANCE MANAGEMENT

10.0 Purpose & Design Philosophy

Under Egyptian Customs Law 207/2020, brokers are personally liable for declaration accuracy. This module tracks broker liability insurance, declaration errors, and performance metrics.

Key Principles:

- Mandatory liability insurance – Min EGP 500,000 (Egypt) or equivalent
- Declaration error tracking – Penalties of 5-50% of duty value
- Performance metrics – Accuracy rate, rejection rate
- Jurisdiction-aware – Different requirements by country

AI Integration:

- A1 (Groq → Ollama): Performance summaries
- A2 (HF local → Ollama): Error pattern detection
- A4 (OPA + WasmEdge): Enforcement

10.1 Jurisdiction-Specific Requirements

Jurisdiction	Liability Insurance	Penalty Range	Bond Required
Egypt	Mandatory (EGP 500,000+)	5-50% of duty	EGP 100,000+
EU	Professional indemnity	Variable	No bond
USA	Customs bond	3x duty + taxes	\$50,000+
UAE	Professional indemnity	10-100% of duty	Bank guarantee
Saudi	Professional indemnity	10-50% of duty	Bank guarantee
UK	Professional indemnity	Variable	No bond

10.2 Database Schema

sql

-- Broker liability insurance

```
CREATE TABLE broker_liability_insurance (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  broker_gtid TEXT NOT NULL REFERENCES tenants(gtid),  
  insurer TEXT NOT NULL,  
  policy_number TEXT NOT NULL,  
  coverage_amount DECIMAL(20,4) NOT NULL,  
  currency TEXT DEFAULT 'EGP',  
  valid_from TIMESTAMPTZ,  
  valid_to TIMESTAMPTZ,  
  certificate_url TEXT,  
  verified BOOLEAN DEFAULT false,  
  verified_at TIMESTAMPTZ,  
  status TEXT DEFAULT 'ACTIVE',  
  created_at TIMESTAMPTZ DEFAULT NOW())  
;
```

-- Broker declaration errors

```
CREATE TABLE broker_declaration_errors (  

```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid),
ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
error_type TEXT NOT NULL,
error_description TEXT,
penalty_amount DECIMAL(20,4),
currency TEXT DEFAULT 'EGP',
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Broker performance metrics

```

CREATE TABLE broker_performance (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
broker_gtid TEXT NOT NULL REFERENCES tenants(gtid),
total_declarations INTEGER DEFAULT 0,
accepted_declarations INTEGER DEFAULT 0,
rejected_declarations INTEGER DEFAULT 0,
error_rate DECIMAL(5,2),
average_processing_time_hours DECIMAL(5,2),
last_assessment TIMESTAMPTZ,
rating DECIMAL(3,2),
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

```

CREATE INDEX idx_broker_liability_insurance_broker ON broker_liability_insurance(broker_gtid);
CREATE INDEX idx_broker_liability_insurance_valid_to ON broker_liability_insurance(valid_to);
CREATE INDEX idx_broker_declaration_errors_broker ON broker_declaration_errors(broker_gtid);
CREATE INDEX idx_broker_declaration_errors_ustn ON broker_declaration_errors(ustn);
CREATE INDEX idx_broker_performance_broker ON broker_performance(broker_gtid);

```

10.3 API Endpoints

Method Path Description	POST /v1/broker/insurance/add Add liability insurance	
GET /v1/broker/insurance/status Get current insurance status		
POST /v1/broker/error/record Record declaration error		
GET /v1/broker/performance Get performance metrics		
POST /v1/broker/insurance/renew Renew insurance		

10.4 Integration with Existing Blueprint

Blueprint Part|Integration|Part 9.5 (CBR Portal)|Insurance upload, error tracking|Part 2.2 (Onboarding)|Insurance upload during onboarding|Part 5.7 (Customs Declaration)|Error logging on rejection|Part 12A.1 (Smart Inbox)|Expiry alerts, error alerts|Part 12C.10 (Government Portal)|Broker performance view|

10.5 Implementation Checklist

- ■ Create broker_liability_insurance table
- ■ Create broker_declaration_errors table
- ■ Create broker_performance table
- ■ Implement insurance upload and verification
- ■ Implement error logging on declaration rejection
- ■ Implement performance metrics calculation
- ■ Add Smart Inbox alerts for insurance expiry
- ■ Add Government Portal view for broker performance

PART 11: ADD-ON 11 - CUSTOMS VALUATION INTELLIGENCE

11.0 Purpose & Design Philosophy

Provides AI-driven duty estimation and market-value comparison to help traders avoid customs disputes.

Key Principles:

- AI-driven – A2 (XGBoost) estimates duty and detects deviations
- Real-time comparison – Declared value vs. market average
- Proactive alerts – Warns when deviation exceeds 20%
- Dispute-ready – Valuation history stored for audit defence

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): Duty estimation, market comparison
- A4 (OPA + WasmEdge): Validation

11.1 AI-Driven Valuation (A2, XGBoost)

Input Features:

Input|Description|Source|HS Code|Harmonized System code|Trade request|Origin Country|Country of origin|Trade request|Destination Country|Importing country|Trade request|Invoice Value|Commercial value|Trade request|Market Price|Current market price|RIA / Public data|Commodity Type|Category|Trade request|Output Example (Normal):

json

```
{
  "estimated_duty": 1500.00,
  "currency": "USD",
  "confidence": 87,
```

```
"valuation_method": "TRANSACTION_VALUE",
"comparison": {
  "market_average": 1450.00,
  "deviation": 3.4,
  "market_data_confidence": 92
},
"explanation": "Duty estimated at USD 1,500 (15% of declared value). This is within 5% of market average."
}
```

Output Example (High Deviation Alert):

json

```
{
  "estimated_duty": 1500.00,
  "declared_value": 20000.00,
  "market_average": 25000.00,
  "deviation": -20.0,
  "alert": {
    "type": "VALUATION_UNDER_DECLARED",
    "severity": "WARNING",
    "message": "Declared value (USD 20,000) is 20% below market average (USD 25,000).",
    "recommendation": "Consider reviewing valuation. Risk of customs reassessment."
  }
}
```

11.2 Database Schema

sql

-- Customs valuations

```
CREATE TABLE customs_valuations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  hs_code CHAR(6) NOT NULL,
  origin_country CHAR(2) NOT NULL,
  destination_country CHAR(2) NOT NULL,
  declared_value DECIMAL(20,4) NOT NULL,
  estimated_duty DECIMAL(20,4) NOT NULL,
  market_average DECIMAL(20,4),
  deviation_percentage DECIMAL(5,2),
```

```

valuation_method TEXT,
confidence DECIMAL(5,2),
alert_type TEXT,
alert_severity TEXT,
alert_message TEXT,
recommendation TEXT,
model_version TEXT,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Valuation disputes

```

CREATE TABLE valuation_disputes (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
declared_value DECIMAL(20,4) NOT NULL,
customs_reassessed_value DECIMAL(20,4),
dispute_reason TEXT NOT NULL,
evidence TEXT,
status TEXT DEFAULT 'PENDING',
resolved_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

```

CREATE INDEX idx_customs_valuations_ustn ON customs_valuations(ustn);
CREATE INDEX idx_customs_valuations_hs ON customs_valuations(hs_code);
CREATE INDEX idx_customs_valuations_alert ON customs_valuations(alert_type) WHERE alert_type IS NOT NULL;
CREATE INDEX idx_valuation_disputes_ustn ON valuation_disputes(ustn);

```

11.3 API Endpoints

Method	Path	Description
GET	/v1/valuation/{ustn}	Get valuation for USTN
POST	/v1/valuation/estimate	Estimate duty for a trade
POST	/v1/valuation/dispute	File valuation dispute
GET	/v1/valuation/market/{hs_code}	Get market data for HS code

11.4 Integration with Existing Blueprint

Blueprint Part|Integration|Part 4.15 (Governor PreScreen)|Add valuation gate: G1U35 – Valuation check|Part 4.3 (Product Form Agent)|Market data for HS codes|Part 10 (Dispute)|Valuation dispute category|Part 12A.2 (TCC)|Valuation alert display|Part 13.1.4 (Trade Core)|Add valuation fields to shipments|

11.5 Implementation Checklist

- ■ Create customs_valuations table
- ■ Create valuation_disputes table
- ■ Implement A2 valuation model (XGBoost)
- ■ Implement market data integration (RIA)
- ■ Implement valuation calculation endpoint
- ■ Add valuation alert to Smart Inbox
- ■ Add valuation dispute workflow (Part 10)

PART 12: ADD-ON 12 - COLD CHAIN QUALITY MANAGEMENT

12.0 Purpose & Design Philosophy

Comprehensive temperature monitoring, pre-trip inspection (PTI) management, and EU-compliant logging for perishable goods.

Key Principles:

- EU RASFF compliant – Continuous monitoring, 15-min intervals, 5-year retention
- PTI management – Pre-trip inspection certificate tracking
- Calibration tracking – Data logger calibration certificates
- Anomaly detection – A2 (LSTM) predicts deviations

AI Integration:

- A1 (Groq → Ollama): Anomaly explanations
- A2 (HF local → Ollama): Anomaly detection (LSTM), shelf-life prediction
- A4 (OPA + WasmEdge): Enforcement

12.1 Cold Chain Specifications by Commodity

Commodity	Temperature Range	Humidity	Ventilation	PTI Required
Frozen Strawberries	-18°C to -20°C	85-90%	Yes	Yes
Fresh Oranges	1°C to 4°C	85-95%	Yes (10-15 cfm)	Yes
Fresh Lemons	10°C to 14°C	85-95%	Yes (10-15 cfm)	Yes
Frozen Fish	-20°C to -25°C	85-90%	No	Yes
Fresh Meat	-1°C to 1°C	85-90%	Yes (20 cfm)	Yes
Dairy Products	2°C to 4°C	85-90%	Yes	Yes
Pharmaceuticals	2°C to 8°C	40-60%	No	Yes
Fresh Flowers	2°C to 4°C	90-95%	Yes	Yes

12.2 Database Schema

sql

-- Cold chain requirements (per product-destination)

```
CREATE TABLE cold_chain_requirements (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  hs_code CHAR(6) NOT NULL,  
  destination_country CHAR(2) NOT NULL,  
  origin_country CHAR(2) NOT NULL,  
  temperature_min DECIMAL(5,2),  
  temperature_max DECIMAL(5,2),
```

```

humidity_min DECIMAL(5,2),
humidity_max DECIMAL(5,2),
ventilation_cfm DECIMAL(5,2),
pti_required BOOLEAN,
treatment_required TEXT,
treatment_duration_days INTEGER,
certification_required TEXT,
source TEXT,
last_updated TIMESTAMPTZ DEFAULT NOW()
);

```

-- PTI certificates

```

CREATE TABLE pti_certificates (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
container_number TEXT NOT NULL,
carrier_gtid TEXT REFERENCES tenants(gtid),
inspection_date TIMESTAMPTZ NOT NULL,
valid_until TIMESTAMPTZ NOT NULL,
temperature_set_point DECIMAL(5,2) NOT NULL,
actual_temperature DECIMAL(5,2) NOT NULL,
pti_result TEXT NOT NULL, -- 'PASS', 'FAIL', 'CONDITIONAL'
pti_reference TEXT,
certificate_url TEXT,
inspector_name TEXT,
verified BOOLEAN DEFAULT false,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Cold chain monitoring (IoT data)

```

CREATE TABLE cold_chain_readings (
id BIGSERIAL PRIMARY KEY,
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
container_number TEXT NOT NULL,
temperature DECIMAL(5,2) NOT NULL,
humidity DECIMAL(5,2),
recorded_at TIMESTAMPTZ NOT NULL,
anomaly BOOLEAN DEFAULT false,

```



```

anomaly_type TEXT
);
SELECT create_hypertable('cold_chain_readings', 'recorded_at');
-- Cold chain anomalies
CREATE TABLE cold_chain_anomalies (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
container_number TEXT NOT NULL,
deviation_celsius DECIMAL(5,2) NOT NULL,
duration_minutes INTEGER NOT NULL,
severity TEXT NOT NULL, -- 'LOW', 'MEDIUM', 'HIGH'
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ,
governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Data logger calibration
CREATE TABLE data_logger_calibration (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
logger_id TEXT NOT NULL,
calibration_date TIMESTAMPTZ NOT NULL,
valid_until TIMESTAMPTZ NOT NULL,
calibration_certificate_url TEXT,
verified BOOLEAN DEFAULT false,
verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_cold_chain_requirements_hs ON cold_chain_requirements(hs_code);
CREATE INDEX idx_cold_chain_requirements_destination ON
cold_chain_requirements(destination_country);
CREATE INDEX idx_pti_certificates_container ON pti_certificates(container_number);
CREATE INDEX idx_pti_certificates_valid_until ON pti_certificates(valid_until);
CREATE INDEX idx_cold_chain_readings_ustn ON cold_chain_readings(ustn);
CREATE INDEX idx_cold_chain_readings_recorded_at ON cold_chain_readings(recorded_at DESC);
CREATE INDEX idx_cold_chain_anomalies_ustn ON cold_chain_anomalies(ustn);

```

```
CREATE INDEX idx_data_logger_calibration_logger ON data_logger_calibration(logger_id);
```

12.3 API Endpoints

Method	Path	Description
GET	/v1/cold-chain/requirements	Get cold chain requirements for HS code
POST	/v1/cold-chain/pti/register	Register PTI certificate
GET	/v1/cold-chain/status/{ustn}	Get cold chain status
POST	/v1/cold-chain/reading	Record temperature reading
GET	/v1/cold-chain/anomalies/{ustn}	Get anomalies for USTN

12.4 Integration with Existing Blueprint

Blueprint Part	Integration
Part 4.3 (Product Form Agent)	Cold chain requirements for product
Part 4.5 (Documentation)	PTI certificate required
Part 5.1 (Weight Calculation)	Temperature-sensitive weight validation
Part 13.1.25 (IoT Sensor)	Enhanced anomaly detection
Part 12A.1 (Smart Inbox)	Temperature excursion alerts
Part 10 (Dispute)	Cold chain evidence

12.5 Implementation Checklist

- Create cold_chain_requirements table
- Create pti_certificates table
- Create cold_chain_readings hypertable
- Create cold_chain_anomalies table
- Create data_logger_calibration table
- Implement A2 anomaly detection (LSTM)
- Implement PTI certificate verification
- Add Smart Inbox alerts for temperature excursions
- Add cold chain evidence to dispute package

PART 13: ADD-ON 13 - INSPECTION AGENCY ACCREDITATION

13.0 Purpose & Design Philosophy

Track ISO 17020 accreditation, scope of work, and performance metrics for QC inspection agencies.

Key Principles:

- Accreditation tracking – ISO 17020, scope, expiry
- Performance metrics – Acceptance rate, override rate, dispute rate
- Jurisdiction-aware – Different accreditation bodies per country

AI Integration:

- A1 (Groq → Ollama): Performance summaries
- A2 (HF local → Ollama): Trend analysis
- A4 (OPA + WasmEdge): Enforcement

13.1 Database Schema

```
sql
```

```
-- Inspection agency accreditations
```

```
CREATE TABLE inspection_agency_accreditations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

agency_gtid TEXT NOT NULL REFERENCES tenants(gtid),
accreditation_standard TEXT NOT NULL, -- 'ISO 17020'
accreditation_body TEXT NOT NULL,
certificate_number TEXT NOT NULL,
valid_from TIMESTAMPTZ,
valid_to TIMESTAMPTZ,
scope_of_accreditation TEXT[], -- commodity types, regions
verified BOOLEAN DEFAULT false,
verified_at TIMESTAMPTZ,
status TEXT DEFAULT 'ACTIVE',
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Agency performance metrics
CREATE TABLE inspection_agency_performance (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
agency_gtid TEXT NOT NULL REFERENCES tenants(gtid),
total_inspections INTEGER DEFAULT 0,
acceptance_rate DECIMAL(5,2),
override_rate DECIMAL(5,2),
dispute_rate DECIMAL(5,2),
average_turnaround_hours DECIMAL(5,2),
last_assessment TIMESTAMPTZ,
rating DECIMAL(3,2),
created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Accreditation bodies by jurisdiction
CREATE TABLE accreditation_bodies (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
jurisdiction_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),
body_name TEXT NOT NULL,
body_website TEXT,
standard TEXT NOT NULL, -- 'ISO 17020'
recognized BOOLEAN DEFAULT true,
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

CREATE INDEX idx_inspection_agency_accreditations_agency ON inspection_agency_accreditations(agency_gtid);

CREATE INDEX idx_inspection_agency_accreditations_valid_to ON inspection_agency_accreditations(valid_to);

CREATE INDEX idx_inspection_agency_performance_agency ON inspection_agency_performance(agency_gtid);

CREATE INDEX idx_accreditation_bodies_jurisdiction ON accreditation_bodies(jurisdiction_code);

13.2 API Endpoints

Method	Path	Description	GET	/v1/accreditation/status/{agency_gtid}	Get accreditation status
POST	/v1/accreditation/register	Register new accreditation			
GET	/v1/accreditation/performance/{agency_gtid}	Get performance metrics			
GET	/v1/accreditation/bodies	List accreditation bodies			

13.3 Integration with Existing Blueprint

Blueprint Part|Integration|Part 9.4 (QC Portal)|Check accreditation before job assignment|Part 2.2 (Onboarding)|Upload accreditation|Part 12A.1 (Smart Inbox)|Expiry alerts|Part 12C.10 (Government Portal)|Accreditation view|

13.4 Implementation Checklist

- Create inspection_agency_accreditations table
- Create inspection_agency_performance table
- Create accreditation_bodies table
- Implement accreditation check in QC job assignment
- Add expiry monitoring and alerts
- Add performance tracking

PART 14: ADD-ON 14 - CURRENCY RISK MANAGEMENT

14.0 Purpose & Design Philosophy

Real-time FX rates (ECB), hedging recommendations, natural hedging opportunities.

Key Principles:

- Real-time FX – ECB daily rates (free XML feed)
- Hedging recommendations – Based on exposure and volatility
- Natural hedging – Match inflows with outflows
- Advisory only – Never forces hedging

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): Volatility prediction
- A4 (OPA + WasmEdge): Validation

14.1 Database Schema

sql

-- Currency exposures

```
CREATE TABLE currency_exposures (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,  
  base_currency TEXT NOT NULL,  
  exposure_currency TEXT NOT NULL,  
  exposure_amount DECIMAL(20,4) NOT NULL,  
  locked_rate DECIMAL(16,8),  
  current_rate DECIMAL(16,8),  
  unrealised_gain_loss DECIMAL(20,4),  
  hedged_percentage DECIMAL(5,2),  
  hedge_type TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Hedging recommendations

```
CREATE TABLE hedging_recommendations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid),  
  currency_pair TEXT NOT NULL,  
  exposure_amount DECIMAL(20,4) NOT NULL,  
  risk_level TEXT NOT NULL,  
  recommended_hedge_percentage DECIMAL(5,2),  
  estimated_cost DECIMAL(5,2),  
  explanation TEXT,  
  natural_hedging_opportunity TEXT,  
  valid_until TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Exchange rates (ECB daily)

```
CREATE TABLE exchange_rates (  
  id BIGSERIAL PRIMARY KEY,  
  base_currency TEXT NOT NULL,  
  target_currency TEXT NOT NULL,  
  rate DECIMAL(16,8) NOT NULL,  
  recorded_at TIMESTAMPTZ DEFAULT NOW())
```

);

-- Indexes

CREATE INDEX idx_currency_exposures_ustn ON currency_exposures(ustn);

CREATE INDEX idx_hedging_recommendations_tenant ON hedging_recommendations(tenant_gtid);

CREATE INDEX idx_exchange_rates_recorded_at ON exchange_rates(recorded_at DESC);

14.2 API Endpoints

Method	Path	Description
GET	/v1/currency/rate	Get current FX rate
GET	/v1/currency/exposure/{ustn}	Get exposure for USTN
GET	/v1/currency/recommendation	Get hedging recommendation
GET	/v1/currency/history	Get FX rate history

14.3 Integration with Existing Blueprint

Blueprint Part	Integration
Part 4.9 (Commercial Settlement)	Currency selection
Part 5.6 (Invoice)	Multi-currency support
Part 6 (Payment Orchestrator)	FX rate in settlements
Part 12A.2 (TCC)	Currency risk card

14.4 Implementation Checklist

- Create currency_exposures table
- Create hedging_recommendations table
- Create exchange_rates table
- Integrate ECB daily rates (free XML feed)
- Implement A2 volatility prediction
- Add hedging recommendations to TCC
- Add currency risk card

PART 15: ADD-ON 15 - GOVERNMENT API SANDBOX

15.0 Purpose & Design Philosophy

Government APIs change without notice. This module provides a sandbox environment for testing and automated regression testing against live APIs.

Key Principles:

- Sandbox environment – Mirrored production
- Automated regression – Daily tests against live APIs
- Change detection – Alert on API changes
- Adaptive integration – Auto-update for minor changes

AI Integration:

- A1 (Groq → Ollama): Change summaries
- A2 (HF local → Ollama): API change detection
- A4 (OPA + WasmEdge): Test validation

15.1 Supported APIs

Country	API	Purpose	Sandbox	Available
Egypt	CargoX	ACI	■	Egypt
Egypt	ETA	e-Invoice	■	Egypt
EU	ICS2	Import control	■	EU
EU	TRACES	Phytosanitary	■	EU
UAE	Trade		■	UAE

Single

Window|Customs|■|Saudi|FASAH|Customs|■|USA|ACE|Customs|■|UK|CDS|Customs|■|

15.2 Database Schema

sql

-- Government API sandbox

```
CREATE TABLE government_api_sandbox (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  country_code CHAR(2) NOT NULL,  
  api_name TEXT NOT NULL,  
  sandbox_url TEXT NOT NULL,  
  production_url TEXT NOT NULL,  
  last_mock_generation TIMESTAMPTZ,  
  version TEXT,  
  is_synced BOOLEAN DEFAULT false,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Government API test results

```
CREATE TABLE government_api_test_results (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  api_id UUID REFERENCES government_api_sandbox(id) ON DELETE CASCADE,  
  test_type TEXT NOT NULL,  
  endpoint TEXT NOT NULL,  
  request_body TEXT,  
  response_body TEXT,  
  expected_status INTEGER,  
  actual_status INTEGER,  
  passed BOOLEAN,  
  diff TEXT,  
  test_run TIMESTAMPTZ DEFAULT NOW()  
);
```

-- API change detection logs

```
CREATE TABLE api_change_detection_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  api_id UUID REFERENCES government_api_sandbox(id) ON DELETE CASCADE,  
  change_type TEXT NOT NULL,  
  old_schema TEXT,
```

```

new_schema TEXT,
detected_at TIMESTAMPTZ DEFAULT NOW(),
resolved BOOLEAN DEFAULT false,
resolved_at TIMESTAMPTZ
);
-- Indexes
CREATE INDEX idx_government_api_sandbox_country ON government_api_sandbox(country_code);
CREATE INDEX idx_government_api_test_results_api ON government_api_test_results(api_id);
CREATE INDEX idx_api_change_detection_logs_api ON api_change_detection_logs(api_id);

```

15.3 API Endpoints

Method	Path	Description
GET	/v1/sandbox/status	Get sandbox status
POST	/v1/sandbox/test	Run API test
GET	/v1/sandbox/results	Get test results
POST	/v1/sandbox/sync	Sync with production

15.4 Integration with Existing Blueprint

Blueprint Part	Integration
Part 7 (Government Integration)	Sandbox for testing
Part 12C.10 (Government Portal)	Sandbox status view
Part 12C.11 (Admin Portal)	Sandbox management

15.5 Implementation Checklist

- ■ Create government_api_sandbox table
- ■ Create government_api_test_results table
- ■ Create api_change_detection_logs table
- ■ Implement sandbox environment
- ■ Implement automated regression testing
- ■ Implement change detection
- ■ Add alerting for API changes

PART 16: ADD-ON 16 - FTA PREFERENCE MANAGEMENT

16.0 Purpose & Design Philosophy

Automatically detect FTA eligibility, calculate preference rates, and manage certificates.

Key Principles:

- Auto-detection – Check if trade qualifies for FTA
- Preference calculation – Apply preferential rates
- Certificate management – EUR.1, COO, etc.
- Jurisdiction-aware – All FTAs worldwide

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): FTA detection
- A4 (OPA + WasmEdge): Validation

16.1 FTA Coverage

FTA|Countries|Preference Rate|Document Required|Egypt-EU|Egypt → EU|0%|EUR.1, Invoice declaration|Egypt-UAE|Egypt → UAE|0-5%|Certificate of Origin|Egypt-Saudi|Egypt → Saudi|0-5%|Certificate of Origin|Egypt-UK|Egypt → UK|0%|UK-Egypt FTA|AfCFTA|African countries|0-10%|AfCFTA Certificate|GAFTA|Arab League|0-10%|Certificate of Origin|EU-Mercosur|EU → Mercosur|0-10%|EUR.1|USMCA|US-Mexico-Canada|0-5%|Certificate of Origin|RCEP|Asia-Pacific|0-10%|Certificate of Origin|

16.2 Database Schema

sql

-- FTA preferences

```
CREATE TABLE fta_preferences (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fta_name TEXT NOT NULL,
  origin_country CHAR(2) NOT NULL,
  destination_country CHAR(2) NOT NULL,
  hs_code CHAR(6),
  preference_rate DECIMAL(5,2),
  product_specific_rules JSONB,
  certificate_type TEXT,
  valid_from TIMESTAMPTZ,
  valid_to TIMESTAMPTZ,
  source TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- FTA preference claims

```
CREATE TABLE fta_preference_claims (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  fta_preference_id UUID REFERENCES fta_preferences(id),
  claim_type TEXT NOT NULL,
  claim_reference TEXT,
  status TEXT DEFAULT 'PENDING',
  verified BOOLEAN DEFAULT false,
  verified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Indexes

```
CREATE INDEX idx_fta_preferences_origin_dest ON fta_preferences(origin_country, destination_country);
```

```
CREATE INDEX idx_fta_preferences_hs ON fta_preferences(hs_code);
```

```
CREATE INDEX idx_fta_preference_claims_ustn ON fta_preference_claims(ustn);
```

16.3 API Endpoints

Method	Path	Description
GET	/v1/fta/check	Check FTA eligibility
GET	/v1/fta/preference	Get preference rate
POST	/v1/fta/claim	Claim FTA preference
GET	/v1/fta/certificate	Get certificate requirements

16.4 Integration with Existing Blueprint

Blueprint Part	Integration
Part 4.3 (Product Form Agent)	FTA eligibility for product
Part 4.5 (Documentation)	FTA certificate required
Part 5.7 (Customs Declaration)	FTA claim in declaration
Part 12A.2 (TCC)	FTA savings display

16.5 Implementation Checklist

- Create fta_preferences table
- Create fta_preference_claims table
- Implement FTA detection
- Implement preference rate calculation
- Add certificate management
- Add Smart Inbox alerts for FTA opportunities

PART 17: ADD-ON 17 - PIRACY & SECURITY RISK ENGINE

17.0 Purpose & Design Philosophy

Provides maritime security intelligence, corridor scoring, and insurance premium adjustment for high-risk routes.

Key Principles:

- Real-time intelligence – IMB Piracy Reporting Centre, Maritime Security Council
- Corridor scoring – Risk level per trade route
- Insurance impact – Premium adjustment based on risk
- Advisory only – Never blocks trades; informs decisions

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): Risk scoring
- A4 (OPA + WasmEdge): Validation

17.1 Risk Levels

Risk Level	Description	Insurance Impact	Security Measures
LOW	No recent incidents	Standard premium	Standard
MODERATE	Some incidents	+10-20% premium	Enhanced vigilance
HIGH	Active risk zone	+30-50% premium	Armed guards
CRITICAL	Immediate threat	+100% premium	Avoid transit

17.2 Database Schema

```
sql
```

```
-- Maritime security incidents
```

```
CREATE TABLE maritime_security_incidents (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
incident_type TEXT NOT NULL, -- 'PIRACY', 'ATTEMPTED_PIRACY', 'HOSTAGE', 'ROBBERY'  
latitude DECIMAL(10,8),  
longitude DECIMAL(11,8),  
description TEXT,  
severity TEXT NOT NULL,  
occurred_at TIMESTAMPTZ,  
source TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Corridor security scores

```
CREATE TABLE corridor_security_scores (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
corridor_code TEXT NOT NULL REFERENCES trade_corridors(corridor_code),  
security_score INTEGER NOT NULL,  
risk_level TEXT NOT NULL,  
last_incident_at TIMESTAMPTZ,  
recommended_security_measures TEXT[],  
insurance_premium_impact DECIMAL(5,2),  
valid_until TIMESTAMPTZ,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Security advisories

```
CREATE TABLE security_advisories (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
advisory_type TEXT NOT NULL,  
region TEXT,  
affected_corridors TEXT[],  
message TEXT,  
issued_at TIMESTAMPTZ,  
valid_until TIMESTAMPTZ,  
source TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

CREATE INDEX idx_maritime_security_incidents_occurred ON maritime_security_incidents(occurred_at DESC);

CREATE INDEX idx_corridor_security_scores_corridor ON corridor_security_scores(corridor_code);

CREATE INDEX idx_security_advisories_valid_until ON security_advisories(valid_until);

17.3 API Endpoints

Method	Path	Description
GET	/v1/security/corridor/{code}	Get corridor security score
GET	/v1/security/incidents	Get recent incidents
GET	/v1/security/advisories	Get active advisories
GET	/v1/security/insurance-impact	Get insurance impact

17.4 Integration with Existing Blueprint

Blueprint	Part	Integration
Part 30 (Trade Corridors)	Security scoring for corridors	Part 4.8 (Insurance)
Premium adjustment	Part 12A.2 (TCC)	Security risk card
Part 12C.10 (Government Portal)	Security intelligence view	

17.5 Implementation Checklist

- Create maritime_security_incidents table
- Create corridor_security_scores table
- Create security_advisories table
- Integrate IMB Piracy Reporting Centre feed
- Implement A2 risk scoring
- Add security risk card to TCC
- Add insurance premium adjustment

PART 18: ADD-ON 18 - TRADE COMPLIANCE CALENDAR

18.0 Purpose & Design Philosophy

Centralised calendar for regulatory deadlines, certificate expiries, tariff changes, and other compliance events.

Key Principles:

- Centralised view – All deadlines in one place
- Proactive reminders – 30, 14, 7, 1 day before
- Auto-population – RIA detects regulatory changes
- Actionable – One-click to address each event

AI Integration:

- A1 (Groq → Ollama): Event summaries
- A2 (HF local → Ollama): Event detection
- A4 (OPA + WasmEdge): Validation

18.1 Compliance Event Types

Event Type	Description	Example	Tariff Change	Customs duty changes	CBAM reporting
deadline	Certificate Expiry	Phyto/Health/COO	expiry	Phyto expires	2026-12-15
Licence Renewal	Broker/Exporter licence	Broker licence	expires		2027-01-01
					Regulatory

Reporting|CBE/ECB/FinCEN deadlines|CBE report due 2026-07-05|Sanctions Update|New sanctions list|OFAC update 2026-06-20|Trade Agreement Change|FTA update|EU-Mercosur implementation|Bond Expiry|Customs bond expires|Bond expires 2026-12-31|

18.2 Database Schema

sql

-- Compliance calendar events

```
CREATE TABLE compliance_calendar_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid),  
  event_type TEXT NOT NULL,  
  title TEXT NOT NULL,  
  description TEXT,  
  event_date TIMESTAMPTZ NOT NULL,  
  reminder_days INTEGER[] DEFAULT '{30, 14, 7, 1}',  
  status TEXT DEFAULT 'PENDING',  
  completed_at TIMESTAMPTZ,  
  linked_ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Compliance calendar alerts

```
CREATE TABLE compliance_calendar_alerts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  event_id UUID REFERENCES compliance_calendar_events(id) ON DELETE CASCADE,  
  alert_type TEXT NOT NULL,  
  sent_at TIMESTAMPTZ DEFAULT NOW(),  
  acknowledged BOOLEAN DEFAULT false,  
  acknowledged_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Regulatory deadlines (RIA auto-populated)

```
CREATE TABLE regulatory_deadlines (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  jurisdiction_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),  
  event_type TEXT NOT NULL,  
  title TEXT NOT NULL,  
  description TEXT,
```

```

deadline_date TIMESTAMPTZ NOT NULL,
recurrence_type TEXT, -- 'YEARLY', 'QUARTERLY', 'MONTHLY'
source_regulation TEXT,
last_updated TIMESTAMPTZ DEFAULT NOW()
);
-- Indexes
CREATE INDEX idx_compliance_calendar_events_tenant ON compliance_calendar_events(tenant_gtid);
CREATE INDEX idx_compliance_calendar_events_date ON compliance_calendar_events(event_date);
CREATE INDEX idx_compliance_calendar_events_status ON compliance_calendar_events(status);
CREATE INDEX idx_regulatory_deadlines_jurisdiction ON regulatory_deadlines(jurisdiction_code);
CREATE INDEX idx_regulatory_deadlines_date ON regulatory_deadlines(deadline_date);

```

18.3 API Endpoints

Method	Path	Description
GET	/v1/compliance/calendar	Get compliance calendar
GET	/v1/compliance/upcoming	Get upcoming deadlines
POST	/v1/compliance/event/add	Add compliance event
POST	/v1/compliance/event/complete	Mark event complete

18.4 Integration with Existing Blueprint

Blueprint Part	Integration	Part	4.1 (RIA)	Regulatory deadline detection	Part	12A.1 (Smart Inbox)	Reminders	Part	12A.10 (Task Center)	Compliance tasks	Part	12C.10 (Government Portal)	Compliance dashboard

18.5 Implementation Checklist

- Create compliance_calendar_events table
- Create compliance_calendar_alerts table
- Create regulatory_deadlines table
- Implement RIA integration for deadline detection
- Implement Smart Inbox reminders
- Add compliance dashboard to Government Portal
- Add calendar view for traders

PART 19: ADD-ON 19 - CARGO INSURANCE INTEGRATION

19.0 Purpose & Design Philosophy

API integration with insurance providers, enabling premium calculation, policy issuance, and claim handling.

Key Principles:

- API integration – Connect to insurers (Allianz, Zurich, etc.)
- Premium calculation – Based on commodity, route, coverage
- Policy management – Issuance, tracking, renewal
- Claim handling – Submit and track claims

AI Integration:

- A1 (Groq → Ollama): Claim summaries
- A2 (HF local → Ollama): Premium optimisation
- A4 (OPA + WasmEdge): Validation

19.1 Insurance Providers

Provider|Jurisdiction|Coverage Types|API Available|Allianz|Global|All Risks, ICC(A/B/C), FPA, WA|■|Zurich|Global|All Risks, ICC(A/B/C)|■|AIG|Global|All Risks, Specialised|■|Egyptian Insurance|Egypt|All Risks|??|Saudi National|Saudi|All Risks|??|

19.2 Database Schema

sql

-- Insurance providers

```
CREATE TABLE insurance_providers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  provider_name TEXT NOT NULL,
  jurisdiction TEXT[],
  coverage_types TEXT[],
  api_endpoint TEXT,
  credentials_encrypted JSONB,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Insurance policies

```
CREATE TABLE insurance_policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  provider_id UUID REFERENCES insurance_providers(id),
  policy_number TEXT NOT NULL,
  coverage_type TEXT NOT NULL,
  coverage_amount DECIMAL(20,4) NOT NULL,
  premium_amount DECIMAL(20,4) NOT NULL,
  currency TEXT DEFAULT 'USD',
  valid_from TIMESTAMPTZ,
  valid_to TIMESTAMPTZ,
  policy_document_url TEXT,
  status TEXT DEFAULT 'ACTIVE',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Insurance claims

```
CREATE TABLE insurance_claims (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,  
  policy_id UUID REFERENCES insurance_policies(id) ON DELETE SET NULL,  
  claim_number TEXT,  
  claim_type TEXT NOT NULL,  
  claim_amount DECIMAL(20,4),  
  description TEXT,  
  evidence_url TEXT,  
  status TEXT DEFAULT 'PENDING',  
  resolved_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_insurance_policies_ustn ON insurance_policies(ustn);  
CREATE INDEX idx_insurance_policies_valid_to ON insurance_policies(valid_to);  
CREATE INDEX idx_insurance_claims_ustn ON insurance_claims(ustn);
```

19.3 API Endpoints

Method	Path	Description
GET	/v1/insurance/premium	Calculate premium
POST	/v1/insurance/policy/issue	Issue policy
GET	/v1/insurance/policy/{ustn}	Get policy details
POST	/v1/insurance/claim/submit	Submit claim
GET	/v1/insurance/claim/{id}	Get claim details

19.4 Integration with Existing Blueprint

Blueprint	Part	Integration	Part	4.8 (Insurance Requirements)	Provider integration	Part	5.6 (Invoice)
Insurance premium in invoice	Part 12A.2 (TCC)	Insurance card	Part 10 (Dispute)	Claim evidence			

19.5 Implementation Checklist

- Create insurance_providers table
- Create insurance_policies table
- Create insurance_claims table
- Integrate with insurers (Allianz, Zurich)
- Implement premium calculation
- Implement policy issuance
- Implement claim handling
- Add Smart Inbox alerts for policy expiry

PART 20: ADD-ON 20 - TRADE FINANCE DOCUMENTATION

20.0 Purpose & Design Philosophy

Manage LC applications, confirmations, bills of exchange, and assignment of proceeds.

Key Principles:

- Document management – LC, confirmation, bills of exchange
- Workflow tracking – Status, approvals, signatures
- Integration – Financing module

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): Document extraction
- A4 (OPA + WasmEdge): Validation

20.1 Finance Documentation Types

Document|Description|Trigger|LC Application|Letter of Credit request|Trade request|LC Confirmation|LC confirmation from bank|Financing agreement|Bill of Exchange|Payment instruction|Settlement|Assignment of Proceeds|Payment assignment|Financing|

20.2 Database Schema

sql

-- Trade finance documents

```
CREATE TABLE trade_finance_documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,  
  financing_agreement_id UUID REFERENCES financing_agreements(id) ON DELETE SET NULL,  
  document_type TEXT NOT NULL,  
  document_reference TEXT,  
  issuing_bank_gtid TEXT REFERENCES tenants(gtid),  
  beneficiary_gtid TEXT REFERENCES tenants(gtid),  
  amount DECIMAL(20,4),  
  currency TEXT,  
  valid_from TIMESTAMPTZ,  
  valid_to TIMESTAMPTZ,  
  document_url TEXT,  
  status TEXT DEFAULT 'PENDING',  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_trade_finance_documents_ustn ON trade_finance_documents(ustn);
```

```
CREATE INDEX idx_trade_finance_documents_financing ON  
trade_finance_documents(financing_agreement_id);
```

```
CREATE INDEX idx_trade_finance_documents_status ON trade_finance_documents(status);
```

20.3 API Endpoints

Method	Path	Description
POST	/v1/finance/document/create	Create finance document
GET	/v1/finance/document/{id}	Get document details
POST	/v1/finance/document/sign	Sign document
GET	/v1/finance/documents/{ustn}	Get all documents for USTN

20.4 Integration with Existing Blueprint

Blueprint Part|Integration|Part 4.9 (Commercial Settlement)|LC selection|Part 3B.5 (Financing Module)|LC confirmation|Part 6 (Payment Orchestrator)|Bill of exchange|Part 12A.2 (TCC)|Finance documents card|

20.5 Implementation Checklist

- ■ Create trade_finance_documents table
- ■ Implement LC application generation
- ■ Implement LC confirmation tracking
- ■ Implement bill of exchange management
- ■ Add document signing workflow
- ■ Integrate with financing module

PART 21: ADD-ON 21 - BACK-TO-BACK LC MANAGEMENT

21.0 Purpose & Design Philosophy

Manage chains of LCs (primary → secondary) where a buyer uses their credit line to issue an LC to a seller, who then uses it to issue a second LC to their supplier.

Key Principles:

- Chain tracking – Primary LC → Secondary LC → etc.
- Risk management – Each LC independent
- Transaction management – Coordinated execution

AI Integration:

- A1 (Groq → Ollama): Explanations
- A4 (OPA + WasmEdge): Validation

21.1 Back-to-Back LC Workflow

text

Buyer (Importer) → Primary LC → Seller (Exporter)

↓

Secondary LC → Supplier

21.2 Database Schema

sql

-- Back-to-back LC

```
CREATE TABLE back_to_back_lc (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  primary_lc_id UUID REFERENCES trade_finance_documents(id),
```

```

secondary_lc_id UUID REFERENCES trade_finance_documents(id),
buyer_gtid TEXT NOT NULL REFERENCES tenants(gtid),
seller_gtid TEXT NOT NULL REFERENCES tenants(gtid),
supplier_gtid TEXT NOT NULL REFERENCES tenants(gtid),
amount DECIMAL(20,4) NOT NULL,
currency TEXT NOT NULL,
status TEXT DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

```

CREATE INDEX idx_back_to_back_lc_primary ON back_to_back_lc(primary_lc_id);
CREATE INDEX idx_back_to_back_lc_secondary ON back_to_back_lc(secondary_lc_id);

```

21.3 API Endpoints

Method	Path	Description
POST	/v1/finance/back-to-back/create	Create back-to-back LC
GET	/v1/finance/back-to-back/{id}	Get details
GET	/v1/finance/back-to-back/chain/{ustn}	Get LC chain

21.4 Integration with Existing Blueprint

Blueprint Part	Integration
Part 20 (Trade Finance Docs)	LC management
Part 3B.5 (Financing Module)	Financing chain
Part 12A.2 (TCC)	LC chain card

21.5 Implementation Checklist

- Create back_to_back_lc table
- Implement LC chain tracking
- Implement risk management
- Add LC chain view in TCC

PART 22: ADD-ON 22 - FORCE MAJEURE HANDLING

22.0 Purpose & Design Philosophy

Handle force majeure events (natural disasters, war, pandemic) that affect trade execution.

Key Principles:

- Event detection – RIA scrapes official sources
- Claim workflow – Structured claim submission
- Contract renegotiation – Extension of deadlines
- Jurisdiction-aware – Force majeure clauses per jurisdiction

AI Integration:

- A1 (Groq → Ollama): Event summaries
- A2 (HF local → Ollama): Event detection
- A4 (OPA + WasmEdge): Validation

22.1 Force Majeure Events

Event	Type	Description	Detection	Natural	Disaster	Earthquake,	flood,	cyclone	RIA
(scraped)	War/Conflict	Armed conflict	RIA (scraped)	Pandemic	Disease	outbreak	WHO	alert	Port
Strike	Labour action	RIA (scraped)	Sanctions	New sanctions	RIA (scraped)				

22.2 Database Schema

sql

-- Force majeure events

```
CREATE TABLE force_majeure_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type TEXT NOT NULL,
  location TEXT,
  affected_countries TEXT[],
  description TEXT,
  severity TEXT NOT NULL,
  start_date TIMESTAMPTZ NOT NULL,
  end_date TIMESTAMPTZ,
  status TEXT DEFAULT 'ACTIVE',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Force majeure claims

```
CREATE TABLE force_majeure_claims (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  event_id UUID REFERENCES force_majeure_events(id) ON DELETE SET NULL,
  claimant_gtid TEXT NOT NULL REFERENCES tenants(gtid),
  reason TEXT NOT NULL,
  evidence TEXT,
  status TEXT DEFAULT 'PENDING',
  resolved_at TIMESTAMPTZ,
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Force majeure contract extensions

```
CREATE TABLE force_majeure_extensions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  claim_id UUID REFERENCES force_majeure_claims(id) ON DELETE SET NULL,
  contract_id UUID REFERENCES contracts(id) ON DELETE SET NULL,
```

```

original_deadline TIMESTAMPTZ,
extended_deadline TIMESTAMPTZ,
extension_days INTEGER,
status TEXT DEFAULT 'PENDING',
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

-- Indexes

```

CREATE INDEX idx_force_majeure_events_status ON force_majeure_events(status);
CREATE INDEX idx_force_majeure_events_start_date ON force_majeure_events(start_date);
CREATE INDEX idx_force_majeure_claims_ustn ON force_majeure_claims(ustn);
CREATE INDEX idx_force_majeure_claims_event ON force_majeure_claims(event_id);

```

22.3 API Endpoints

Method	Path	Description	GET	/v1/force-majeure/events	Get active events
POST	/v1/force-majeure/claim/file	File claim	GET	/v1/force-majeure/claim/{id}	Get claim details
POST	/v1/force-majeure/extension/request	Request extension			

22.4 Integration with Existing Blueprint

Blueprint Part|Integration|Part 4.1 (RIA)|Event detection|Part 3 (Contract)|Extension clauses|Part 10 (Dispute)|Force majeure evidence|Part 12A.1 (Smart Inbox)|Event alerts|

22.5 Implementation Checklist

- ■ Create force_majeure_events table
- ■ Create force_majeure_claims table
- ■ Create force_majeure_extensions table
- ■ Implement RIA event detection
- ■ Implement claim workflow
- ■ Implement contract extension
- ■ Add Smart Inbox alerts

PART 23: ADD-ON 23 - SHIPPER'S DECLARATION & EXPORT DOCUMENTATION

23.0 Purpose & Design Philosophy

Manage shipper's declarations, export accompanying documents (EAD), and export licence tracking.

Key Principles:

- Document management – Declaration, EAD, licence
- Compliance tracking – Expiry, renewal
- Integration – Customs declaration

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): Document extraction

- A4 (OPA + WasmEdge): Validation

23.1 Export Documentation Requirements

Document|Required For|Issuing Authority|Shipper's Declaration|All exports|Exporter|Export Accompanying Document|EU exports|Customs|Export Licence|Restricted goods|Ministry of Trade|Certificate of Origin|FTA preference|Chamber of Commerce|EUR.1|Egypt-EU FTA|Chamber of Commerce|

23.2 Database Schema

sql

-- Shipper's declarations

```
CREATE TABLE shippers_declarations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  exporter_gtid TEXT NOT NULL REFERENCES tenants(gtid),
  declaration_reference TEXT,
  declaration_date TIMESTAMPTZ,
  goods_description TEXT,
  hs_code TEXT,
  net_weight DECIMAL(10,2),
  value DECIMAL(20,4),
  currency TEXT,
  origin_country CHAR(2),
  destination_country CHAR(2),
  incoterm TEXT,
  signed BOOLEAN DEFAULT false,
  signed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Export accompanying documents (EAD)

```
CREATE TABLE export_accompanying_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
  ead_reference TEXT NOT NULL,
  issue_date TIMESTAMPTZ,
  valid_until TIMESTAMPTZ,
  issuing_authority TEXT,
  document_url TEXT,
  status TEXT DEFAULT 'ACTIVE',
```

```
created_at TIMESTAMPTZ DEFAULT NOW())
```

```
);
```

```
-- Export licences
```

```
CREATE TABLE export_licences (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
tenant_gtid TEXT NOT NULL REFERENCES tenants(gtid),
```

```
licence_number TEXT NOT NULL,
```

```
issuing_authority TEXT NOT NULL,
```

```
valid_from TIMESTAMPTZ,
```

```
valid_to TIMESTAMPTZ,
```

```
restricted_commodities TEXT[],
```

```
document_url TEXT,
```

```
status TEXT DEFAULT 'ACTIVE',
```

```
created_at TIMESTAMPTZ DEFAULT NOW())
```

```
);
```

```
-- Indexes
```

```
CREATE INDEX idx_shippers_declarations_ustn ON shippers_declarations(ustn);
```

```
CREATE INDEX idx_export_accompanying_documents_ustn ON  
export_accompanying_documents(ustn);
```

```
CREATE INDEX idx_export_licences_tenant ON export_licences(tenant_gtid);
```

```
CREATE INDEX idx_export_licences_valid_to ON export_licences(valid_to);
```

23.3 API Endpoints

Method	Path	Description
POST	/v1/export/declaration/create	Create declaration
GET	/v1/export/declaration/{ustn}	Get declaration
POST	/v1/export/ead/create	Create EAD
GET	/v1/export/licence/status	Get licence status

23.4 Integration with Existing Blueprint

Blueprint Part	Integration
Part 5.7 (Customs Declaration)	Declaration data
Part 2.2 (Onboarding)	Licence upload
Part 12A.1 (Smart Inbox)	Licence expiry

23.5 Implementation Checklist

- Create shippers_declarations table
- Create export_accompanying_documents table
- Create export_licences table
- Implement declaration generation
- Implement EAD management
- Implement licence tracking

PART 24: ADD-ON 24 - PORT & TERMINAL INTEGRATION

24.0 Purpose & Design Philosophy

Beyond the container release API, deeper integration with port and terminal systems.

Key Principles:

- EDI integration – Gate-in/gate-out
- Pre-advice – Advance notification to terminals
- Terminal OS integration – Real-time status

AI Integration:

- A1 (Groq → Ollama): Explanations
- A4 (OPA + WasmEdge): Validation

24.1 Terminal Integration Points

Integration	Description	Format	Gate-in	Container arrives at terminal	EDI (EDIFACT)	Gate-out	Container leaves terminal	EDI (EDIFACT)	Vessel Schedule	ETD/ETA updates	API/EDI	Container Status	Location updates	API/EDI	Pre-advice	Advance notification	API
-------------	-------------	--------	---------	-------------------------------	---------------	----------	---------------------------	---------------	-----------------	-----------------	---------	------------------	------------------	---------	------------	----------------------	-----

24.2 Database Schema

sql

-- Terminal integrations

```
CREATE TABLE terminal_integrations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  terminal_gtid TEXT NOT NULL REFERENCES tenants(gtid),  
  integration_type TEXT NOT NULL,  
  endpoint_url TEXT,  
  format TEXT DEFAULT 'EDI',  
  credentials_encrypted JSONB,  
  is_active BOOLEAN DEFAULT true,  
  last_test TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Gate events

```
CREATE TABLE gate_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,  
  container_number TEXT NOT NULL,  
  event_type TEXT NOT NULL, -- 'GATE_IN', 'GATE_OUT'  
  event_time TIMESTAMPTZ NOT NULL,  
  terminal_gtid TEXT REFERENCES tenants(gtid),  
  gate_reference TEXT,  
  created_at TIMESTAMPTZ DEFAULT NOW())
```


);

-- Indexes

CREATE INDEX idx_gate_events_ustn ON gate_events(ustn);

CREATE INDEX idx_gate_events_container ON gate_events(container_number);

CREATE INDEX idx_gate_events_time ON gate_events(event_time DESC);

24.3 API Endpoints

Method	Path	Description
POST	/v1/terminal/pre-advice	Send pre-advice
POST	/v1/terminal/gate-in	Record gate-in
POST	/v1/terminal/gate-out	Record gate-out
GET	/v1/terminal/status/{container}	Get status

24.4 Integration with Existing Blueprint

Blueprint Part|Integration|Part 8 (Container Release)|Gate events trigger release|Part 3B.5.1 (Pre-advice)|Pre-advice webhook|Part 13.1.5 (Ports)|Terminal mapping|Part 12A.1 (Smart Inbox)|Gate event alerts|

24.5 Implementation Checklist

- Create terminal_integrations table
- Create gate_events table
- Implement pre-advice webhook
- Implement gate-in recording
- Implement gate-out recording
- Add EDI support (EDIFACT)
- Integrate with Part 8 (Container Release)

PART 25: ADD-ON 25 - PAYMENT GUARANTEE CONFIRMATION (OPTIONAL)

25.0 Purpose & Design Philosophy

Note: This add-on is OPTIONAL. Payment guarantee verification is not mandatory. Traders can choose not to use it.

Provides optional verification of payment guarantees (LCs, bank guarantees) through SWIFT MT700/URDG 758.

Key Principles:

- Optional – Never forced
- Multiple methods – SWIFT, manual, third-party
- Integration – Bank portals

25.1 Verification Methods

Method	Description	Availability	SWIFT MT700	LC confirmation	Via bank integration	URDG 758	Bank guarantee verification	Via bank integration	Manual Upload	PDF confirmation	Always available	Third-Party	Independent verification	Optional

25.2 Database Schema

sql

-- Payment guarantee confirmations

```
CREATE TABLE payment_guarantee_confirmations (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
ustn TEXT REFERENCES shipments(ustn) ON DELETE SET NULL,
confirmation_type TEXT NOT NULL, -- 'SWIFT_MT700', 'URDG_758', 'MANUAL', 'THIRD_PARTY'
confirmation_reference TEXT,
issuing_bank_gtid TEXT REFERENCES tenants(gtid),
confirming_bank_gtid TEXT REFERENCES tenants(gtid),
confirmation_date TIMESTAMPTZ,
verification_status TEXT DEFAULT 'PENDING',
document_url TEXT,
verified BOOLEAN DEFAULT false,
verified_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Indexes

```
CREATE INDEX idx_payment_guarantee_confirmations_ustn ON
payment_guarantee_confirmations(ustn);
```

```
CREATE INDEX idx_payment_guarantee_confirmations_status ON
payment_guarantee_confirmations(verification_status);
```

25.3 Implementation Checklist

- ■ Create payment_guarantee_confirmations table
- ■ Implement manual upload (always available)
- ■ Optional: implement SWIFT MT700 integration
- ■ Optional: implement URDG 758 integration
- ■ Add to TCC as optional card

PART 26: ADD-ON 26 - DEMURRAGE DISPUTE RESOLUTION

26.0 Purpose & Design Philosophy

Demurrage disputes are common. This module provides a structured dispute resolution workflow integrated with the main dispute system (Part 10).

Key Principles:

- Structured workflow – Initiate, evidence, mediate, resolve
- Evidence package – Automatic demurrage calculation audit trail
- Integration – Part 10 dispute system

AI Integration:

- A1 (Groq → Ollama): Explanations
- A2 (HF local → Ollama): Pattern analysis

- A4 (OPA + WasmEdge): Validation

26.1 Database Schema

sql

-- Demurrage disputes (extends Part 10)

```
CREATE TABLE demurrage_disputes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ustn TEXT NOT NULL REFERENCES shipments(ustn) ON DELETE CASCADE,
  amount_disputed DECIMAL(20,4) NOT NULL,
  reason TEXT NOT NULL,
  evidence TEXT,
  status TEXT DEFAULT 'PENDING',
  resolved_at TIMESTAMPTZ,
  governor_decision_id UUID REFERENCES governor_decisions(decision_id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Indexes

```
CREATE INDEX idx_demurrage_disputes_ustn ON demurrage_disputes(ustn);
CREATE INDEX idx_demurrage_disputes_status ON demurrage_disputes(status);
```

26.2 Integration with Existing Blueprint

Blueprint Part|Integration|Part 10 (Dispute Management)|Dispute category|Part 9 (Demurrage Management)|Evidence package|Part 12A.2 (TCC)|Dispute card|

26.3 Implementation Checklist

- ■ Create demurrage_disputes table
- ■ Integrate with Part 10 dispute system
- ■ Add demurrage as dispute category
- ■ Auto-compile evidence package
- ■ Add mediation workflow

PART 27: ADD-ON 27 - (RESERVED)

This add-on is reserved for future enhancements.

PART 28: ADD-ON 28 - GLOBAL REGULATORY INTELLIGENCE & REQUIREMENTS ENGINE (GRiRE)

28.0 Purpose & Design Philosophy

The GRiRE engine provides AI-powered discovery and import of regulatory, customs, documentation, and logistics requirements for every country and territory worldwide – not just the ones manually hardcoded.

Key Principles:

- AI-driven discovery – Scrapes 500+ official sources across 195+ countries
- Self-updating – Changes detected and imported automatically

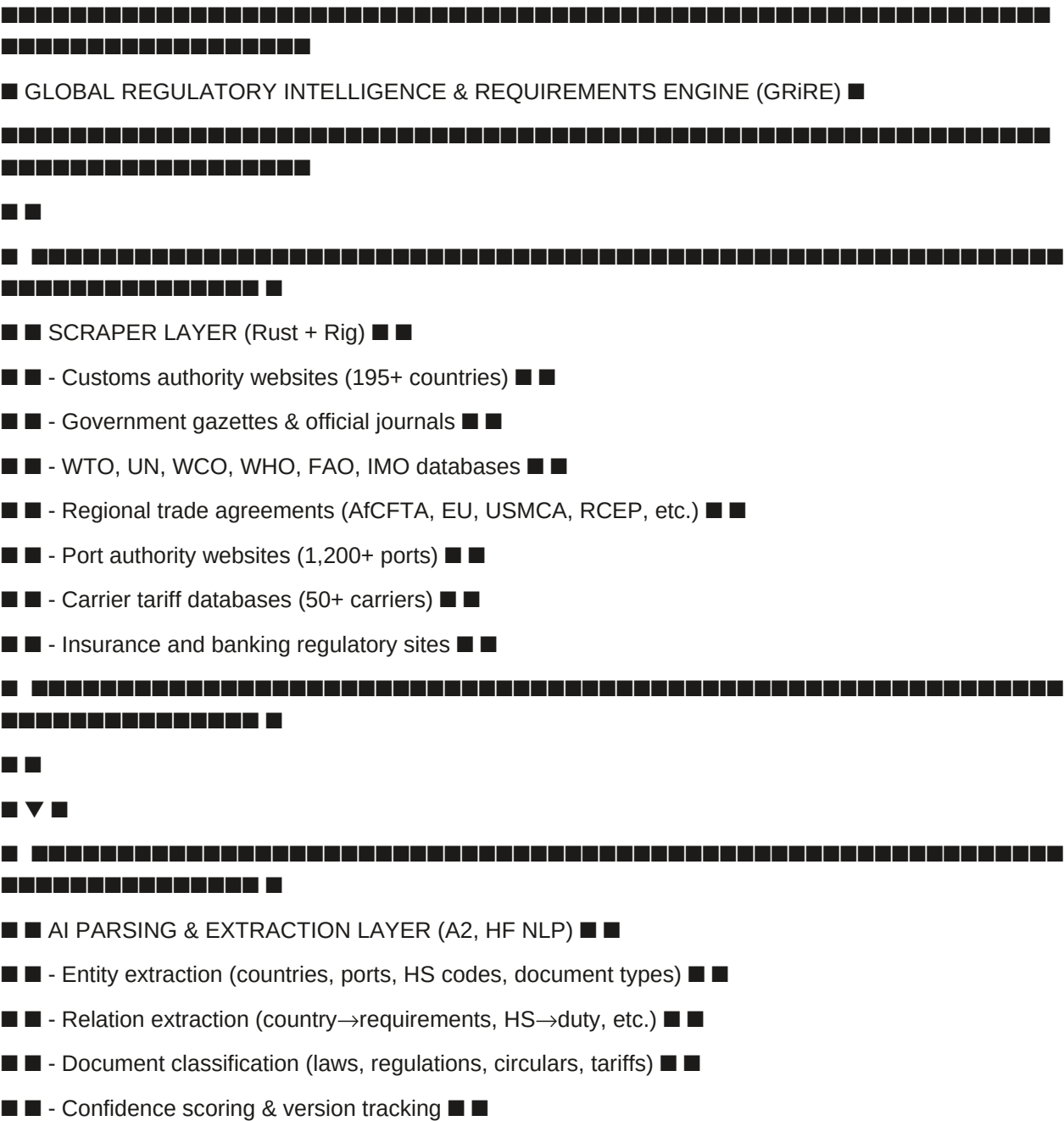
- Auto-configuration – All platform modules configured automatically
- Confidence scoring – Low-confidence items flagged for human review
- Zero-cost – All data sources are free and public

AI Integration:

- A1 (Groq → Ollama): Change summaries, explanations
- A2 (HF local → Ollama): Entity extraction, document classification, relation extraction
- A4 (OPA + WasmEdge): Validation

28.1 Core Architecture

text



11

■ ■ STRUCTURED REQUIREMENTS STORE (PostgreSQL + Vector DB) ■ ■

■ ■ - Country Regulatory Profiles ■ ■

■ ■ - HS Code Tariff Schedules ■ ■

■ ■ - Required Documents Matrix ■ ■

■ ■ - Cold Chain & Treatment Requirements ■ ■

■ ■ - Bond & Guarantee Rules ■ ■

■ ■ - Demurrage & Free Time Norms ■ ■

■ ■ - Customs Valuation Rules ■ ■

■ ■ - FTA & Preference Rules ■ ■

22

■ ▼ ■

■ ■ OUTPUT LAYER ■ ■

■ ■ - Dynamic form fields (Product Form Agent) ■ ■

■ ■ - Document requirements checklist ■ ■

■ ■ - Bond amount calculator ■ ■

■ ■ - Demurrage free time & rates ■ ■

■ ■ - Cold chain temperature specifications ■ ■

■ ■ - Duty estimation & valuation alerts ■ ■

28.2 Data Sources (500+)

Category|Sources|Coverage|Customs|WCO, national customs websites|195 countries|Tariffs|WTO Tariff Data, national tariff schedules|150+ countries|Documents|UN/CEFACT, WCO guidelines|All countries|Cold Chain|WHO, FAO, national food safety agencies|180 countries|Phyto|IPPC ePhyto Hub|180+ countries|Sanctions|UN, OFAC, EU, national lists|All countries|FTAs|WTO RTA database, national trade agreements|350+ agreements|Ports|UN/LOCODE, port authority sites|1,200+ ports|Shipping|IMO, national maritime authorities|All flag states|Demurrage|Carrier tariffs, port

websites|200+ ports|

28.3 AI Pipeline Details

A2 Models:

Task|Model|Training Data|Entity Extraction|HF NER (fine-tuned)|500,000+ regulatory documents|Document Classification|HF BERT (fine-tuned)|100,000+ regulations|Relation Extraction|HF RelationExtraction|50,000+ structured rules|Tariff Classification|XGBoost|Historical tariff data|A1 (Groq → Ollama):

- Summarises regulatory changes
- Generates plain-language explanations
- Creates Smart Inbox alerts

A4 (OPA + WasmEdge):

- Validates requirements consistency
- Enforces compliance gates

28.4 Structured Requirements Store

Core Table: Country Regulatory Profiles

sql

```
CREATE TABLE country_regulatory_profiles (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),  
  profile_version INTEGER DEFAULT 1,  
  regulatory_body TEXT,  
  customs_authority TEXT,  
  import_licence_required BOOLEAN,  
  export_licence_required BOOLEAN,  
  bond_required BOOLEAN,  
  bond_factor_default DECIMAL(5,2),  
  bond_aeo_factor DECIMAL(5,2),  
  free_time_standard_days INTEGER,  
  demurrage_currency TEXT DEFAULT 'USD',  
  document_language TEXT,  
  electronic_documents_accepted BOOLEAN,  
  original_documents_required BOOLEAN,  
  translated_documents_required BOOLEAN,  
  import_prohibitions JSONB,  
  restricted_commodities TEXT[],  
  customs_valuation_method TEXT,
```

```
tariff_currency TEXT,  
last_updated TIMESTAMPTZ DEFAULT NOW(),  
confidence_score DECIMAL(5,2),  
source_url TEXT,  
loom_hash TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

HS Code Tariff Rates:

sql

```
CREATE TABLE hs_tariff_rates (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
hs_code CHAR(6) NOT NULL,  
country_code CHAR(2) NOT NULL,  
tariff_rate DECIMAL(6,3),  
duty_type TEXT, -- 'AD_VALOREM', 'SPECIFIC', 'COMPOUND'  
unit TEXT,  
additional_taxes JSONB,  
preferential_rates JSONB,  
valid_from TIMESTAMPTZ,  
valid_to TIMESTAMPTZ,  
source TEXT,  
confidence_score DECIMAL(5,2),  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Required Documents Matrix:

sql

```
CREATE TABLE country_required_documents (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
country_code CHAR(2) NOT NULL REFERENCES jurisdictions(code),  
document_type TEXT NOT NULL,  
hs_code CHAR(6),  
commodity_category TEXT,  
required BOOLEAN,  
mandatory_for VARCHAR(20), -- 'IMPORT', 'EXPORT', 'TRANSIT'  
trigger_event TEXT, -- 'CUSTOMS', 'SETTLEMENT', 'SHIPMENT'
```

```
format_requirement TEXT, -- 'ORIGINAL', 'ELECTRONIC'  
language_requirement TEXT,  
issuing_authority TEXT,  
regulation_reference TEXT,  
last_updated TIMESTAMPTZ DEFAULT NOW()  
);
```

Cold Chain Requirements:

sql

```
CREATE TABLE cold_chain_requirements (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
hs_code CHAR(6) NOT NULL,  
destination_country CHAR(2) NOT NULL,  
origin_country CHAR(2) NOT NULL,  
temperature_min DECIMAL(5,2),  
temperature_max DECIMAL(5,2),  
humidity_min DECIMAL(5,2),  
humidity_max DECIMAL(5,2),  
ventilation_cfm DECIMAL(5,2),  
pti_required BOOLEAN,  
treatment_required TEXT,  
treatment_duration_days INTEGER,  
certification_required TEXT,  
source TEXT,  
last_updated TIMESTAMPTZ DEFAULT NOW()  
);
```

Demurrage Norms:

sql

```
CREATE TABLE demurrage_norms (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
port_unlocode TEXT NOT NULL REFERENCES ports(unlocode),  
container_type TEXT NOT NULL,  
free_time_days INTEGER,  
carrier_specific BOOLEAN,  
demurrage_rates JSONB,  
detention_rates JSONB,
```



```
currency TEXT DEFAULT 'USD',  
source TEXT,  
last_updated TIMESTAMPTZ DEFAULT NOW()  
);
```

FTA Preference Rules:

sql

```
CREATE TABLE fta_preference_rules (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
fta_name TEXT NOT NULL,  
origin_country CHAR(2) NOT NULL,  
destination_country CHAR(2) NOT NULL,  
hs_code CHAR(6),  
preference_rate DECIMAL(5,2),  
product_specific_rules JSONB,  
certificate_type TEXT,  
valid_from TIMESTAMPTZ,  
valid_to TIMESTAMPTZ,  
source TEXT,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

28.5 AI-Driven Country Discovery & Import

Workflow:

1 Scheduled Discovery (every 6 hours):

- Query UN stats for country list
- Check for new countries or status changes
- Initiate scraping for newly discovered countries

2 Intelligent Scraping:

- For each country, identify official sources:
 - Customs website
 - Trade ministry
 - WTO tariff database
 - Regional agreements
- Use AI to navigate and extract structured data

3 Data Extraction (A2):

- Customs procedures: Import/export documentation, bonds, inspections

- Tariffs: Duty rates, preferential schedules, additional taxes
- Documents: Required documents per HS code / commodity
- Cold chain: Temperature, humidity, treatment requirements
- Demurrage: Free time, charges per port
- FTAs: Agreements, rates, certificate types

4 Validation & Confidence Scoring:

- Cross-reference with multiple sources
- Assign confidence score
- Flag low-confidence items for human review

5 Storage & Versioning:

- Store with timestamp and source
- Track changes over time
- Alert on significant changes

Example: Discovery for Brazil

1 Sources detected:

- Ministry of Economy (Customs): RFB
- ANVISA (Health), MAPA (Agriculture)
- WTO tariff data
- Mercosur agreements

2 Data extracted:

- Bonds: 100% of duty, bank guarantee accepted
- Documents: Invoice, packing list, COO, phyto, health
- Cold chain: 2-8°C for pharmaceuticals, -18°C for frozen
- Demurrage: 5 days free time (Santos), 7 days (Paranaguá)
- FTA: Mercosur, EU-Mercosur (pending)

3 Confidence: 87%

- Cross-referenced with WTO data and ANVISA

Smart Inbox Alert:

text

?? NEW COUNTRY PROFILE: BRAZIL

Discovered: 2026-06-20 06:00 UTC

Confidence: 87%

Customs bond required: YES

Documents: 6 required

Demurrage norms: 2 ports added

[\[View Profile\]](#) [\[Review Low-Confidence Items\]](#) [\[Approve Profile\]](#)

Module	Auto-Configuration	Customs Bond	Bond factor, acceptance of bank guarantees	Demurrage	Free time, default rates per commodity
		per port	Cold Chain	Temperature/humidity ranges	
	Documentation	Required documents	checklist	FTA	Preference rates, certificate types
	Insurance	Recommended coverage	types	Financing	Corridor risk assessment (historical)
	Government Portal	Government node creation	(country-specific)	GTID	Country code added, regulatory body auto-populated

■ • Amount: 100% of duty (bank guarantee or cash deposit) ■

- • Issuer: Brazilian bank or international bank with presence ■

■ ?? Tariff Information: ■

- • HS 0805.10 (Oranges): 10% duty + 2% MERCOSur tax ■

■ ■

Demurrage (Santos Port): 1000

- • Free time: 5 days ■

22

■ ?? AI Insight: This trade is eligible for EU-Mercosur FTA preference. ■

22

■ [View Full Regulatory Report] [Apply Requirements] [Request Human Review] ■

A horizontal row of 18 empty square boxes, each with a thin black border, intended for the student to write their answers to the questions.

28.8 Coverage Metrics

With GRIRE|Without GRIRE|Discovery: 1-2 hours|Manual research: 2-3 weeks|AI Parsing: 2-4 hours|Configuration: 1-2 weeks|Validation: 1-2 hours (auto)|Validation: 1 week|Configuration: Instant (auto)|Total: 1-2 months|Total: < 1 day||

Blueprint Part|Integration|Part 4.1 (RIA)|GRiRE is the advanced, AI-powered version of RIA|Part 4.3 (Product Form Agent)|Requirements data|Part 4.5 (Documentation)|Required documents|Part 7 (Government Integration)|Government API discovery|Part 13 (Data Model)|Jurisdiction-specific tables|Part 31-50 (Add-Ons)|All add-ons powered by GRiRE|

Phase 1: Core Engine (Months 1-3)

Phase 2: Country Coverage Expansion (Months 4-6)

- ■ Implement auto-configuration for all modules
- ■ Add user interface for country-specific requirements
- ■ Implement confidence scoring

Phase 3: Continuous Improvement (Ongoing)

- ■ Add version tracking and change detection
- ■ Implement human review workflow
- ■ Add Smart Inbox alerts for regulatory changes
- ■ Improve model accuracy

PART 29: FINAL SUMMARY

All Add-Ons (1-28) - Complete Coverage

# Add-On	Name Priority Status 1 GNN	Risk	Engine Built-in Complete 2 Federated
Learning Built-in Complete 3 Causal	Inference	Engine Built-in Complete 4 Self-Healing	
Infrastructure Built-in Complete 5 Automated	Penetration	Testing Built-in Complete 6 Post-Quantum	
Cryptography Built-in Complete 7 Expanded	ZK	Proofs Built-in Complete 8 Customs	Bond &
Guarantee P0 Specified 9 Demurrage	&	Detention P0 Specified 10 Broker	Liability &
Insurance P0 Specified 11 Customs	Valuation	Intelligence P1 Specified 12 Cold	Chain
Management P1 Specified 13 Inspection	Agency	Accreditation P1 Specified 14 Currency	Risk
Management P1 Specified 15 Government	API	Sandbox P1 Specified 16 FTA	Preference
Management P1 Specified 17 Piracy &	Security	Risk Engine P1 Specified 18 Trade	Compliance
Calendar P1 Specified 19 Cargo	Insurance	Integration P2 Specified 20 Trade	Finance
Documentation P2 Specified 21 Back-to-Back	LC	Management P2 Specified 22 Force	Majeure
Handling P2 Specified 23 Shipper's	Declaration &	Export Docs P2 Specified 24 Port &	Terminal
Integration P2 Specified 25 Payment	Guarantee	Confirmation P3 Optional 26 Demurrage	Dispute
Resolution P3 Specified 27 (Reserved) -	- 28 GRiRE		
Engine Foundation Specified			

SGTX MASTER GLOBAL TRADE COMPLETION AMENDMENT

0. Purpose

This amendment defines the final target architecture for SGTX as a:

Global, non-custodial, relationship-controlled, sovereign-governed trade execution infrastructure connecting commercial trade, regulation, customs, government systems, logistics, financial settlement, delivery, post-clearance and evidence through one canonical USTN graph.

The objective is not to make SGTX a marketplace.

The objective is to make SGTX capable of taking an authorized trade between known parties and orchestrating everything required to execute that trade legally and operationally across jurisdictions.

1. SGTX CONSTITUTION — FINAL FORM

SGTX remains:

- non-custodial
- non-marketplace
- non-title-taking
- non-carrier

- non-customs-authority
- non-bank
- non-deposit-taking
- non-government
- AI-assisted
- Governor-governed
- OPA-enforced
- WasmEdge-enforced
- Loom-audited
- USTN-centric
- jurisdiction-aware
- relationship-controlled

SGTX does facilitate commercial/logistics services.

That means the platform may facilitate:

- approved freight forwarders
- approved LSPs
- shipping lines
- airlines
- rail operators
- ferry operators
- warehouses
- ground handlers
- terminals
- customs brokers
- laboratories
- QC/inspection providers
- insurance providers
- connected banks
- trader-saved financing institutions
- government-authorized agencies

The distinction is:

PUBLIC MARKETPLACE

X

RELATIONSHIP-CONTROLLED TRADE ORCHESTRATION

✓

2. PROVIDER RELATIONSHIP MODEL

The final provider model is:

TRADER

↓

APPROVED / CONNECTED / SAVED PROVIDER

↓

RFQ / QUOTE

↓

TRADER REVIEW

↓

TRADER EXPLICIT SELECTION

↓

SERVICE AGREEMENT / ADDENDUM

↓

EXECUTION

A provider can become available to a trader because:

- 1 It is already approved and connected.
- 2 The trader has saved its GTID.
- 3 The trader explicitly selects it.
- 4 A government-mandated service relationship exists.

Do not introduce:

- random provider matching
- unsolicited provider recommendations
- "best provider"
- public provider rankings
- public provider marketplace
- autonomous provider selection

This preserves the blueprint's explicit non-marketplace/orchestration principle.

3. FINANCING RELATIONSHIP MODEL

Financing remains legitimate SGTX functionality.

Valid financing relationships:

TRADER

■■■■ CONNECTED BANK

■■■■ CONNECTED FINANCIAL INSTITUTION

■■■■ SAVED FINANCIER GTID

Workflow:

Financing Need



Trader selects connected bank / saved financier



Financing request



Financier assessment



Offer



Trader acceptance



Financing execution



Disbursement



Repayment / settlement

Do not transform this into random public matching.

No unsolicited financier recommendations.

No public financier marketplace.

The existing blueprint's bank/PFI portal architecture remains usable, but access must be governed by relationship/GTID controls.

4. MASTER GLOBAL TRADE GRAPH

The entire SGTX platform should ultimately be represented as:

GTID



TRADE



RFQ



QUOTATION



ORDER



CONTRACT

↓

REGULATORY SNAPSHOT

↓

USTN

↓

TRANSPORT GRAPH

↓

CUSTOMS OPERATIONS

↓

DOCUMENTS

↓

GOVERNMENT REFERENCES

↓

PAYMENT / FINANCE

↓

PHYSICAL EXECUTION

↓

DELIVERY

↓

ACCEPTANCE

↓

SETTLEMENT

↓

RECONCILIATION

↓

POST-CLEARANCE

↓

CLAIMS / RETURNS

↓

EVIDENCE

↓

USTN CLOSED

Every downstream record ultimately references USTN.

5. GLOBAL CANONICAL DATA MODEL

Create/extend the canonical objects:

TRADE_PARTY
PRODUCT
TRADE_ITEM
RFQ
QUOTATION
ORDER
PURCHASE_ORDER
SALES_ORDER
PROFORMA
CONTRACT
INCOTERM
ORIGIN
DESTINATION
REGULATORY_PROFILE
LICENSE
PERMIT
CERTIFICATE
DOCUMENT
CUSTOMS_OPERATION
TRANSPORT_GRAPH
SHIPMENT
TRANSPORT_LEG
DUTY
TAX
GUARANTEE
PAYMENT
SETTLEMENT
INSURANCE
INSPECTION
SECURITY
DELIVERY
ACCEPTANCE
CLAIM
RETURN
POST_CLEARANCE

ACCOUNTING

EVIDENCE

Do not duplicate concepts already present in SGTX.

Extend existing models where possible.

6. GLOBAL JURISDICTION FABRIC

"Country" is insufficient as the fundamental legal object.

Create:

JURISDICTION

Support:

- sovereign country
- customs territory
- customs union
- economic union
- autonomous territory
- special administrative region
- free zone
- free port
- bonded zone
- special economic zone
- airport customs jurisdiction
- port customs jurisdiction
- tax territory
- export-control territory
- special regime

Each jurisdiction stores:

jurisdiction_id

ISO references

parent jurisdiction

customs territory

tax territory

regional bloc

customs authority

tax authority

SPS authority

TBT authority

export-control authority

transport authority

immigration authority

ports

airports

customs offices

special regimes

legal sources

effective dates

7. WORLDWIDE JURISDICTION REGISTRY

The architecture must support all countries worldwide, plus relevant customs and special jurisdictions.

Every jurisdiction begins:

NOT_ACTIVE

unless actually configured.

Possible states:

NOT_ACTIVE

COUNTRY_CONFIGURED

ADAPTER_READY

SANDBOX_CONNECTED

PRODUCTION_READY

PRODUCTION_CONNECTED

MANUAL_ONLY

PORTAL_ONLY

INTEGRATION_REQUIRED

LEGAL_AUTHORIZATION_REQUIRED

DEGRADED

DEPRECATED

Never confuse:

with:

8. GLOBAL REGULATORY SOURCE REGISTRY

Create:

REGULATORY_SOURCE

Each regulatory rule needs:

- source
- authority

- jurisdiction
- law/regulation
- article/section
- official URL
- publication date
- effective date
- expiry
- language
- hash
- verification status
- last checked
- next review

The WCO Data Model is particularly appropriate as the interoperability foundation because WCO describes it as a harmonized universal language for cross-border exchange and Single Window implementation.

9. REGULATORY SNAPSHOT

At contract/trade lock:

REGULATORY_SNAPSHOT

Capture:

- origin
- destination
- transit
- HS
- commodity
- Incoterm
- tariff
- tax
- origin rules
- applicable agreements
- licenses
- permits
- certificates
- customs procedure
- SPS
- TBT
- sanctions

- export controls
- transport restrictions
- government integration state

The snapshot must be immutable.

10. REGULATORY CHANGE MANAGEMENT

RIA should become a complete regulatory-change pipeline:

DISCOVER

↓

VERIFY

↓

CLASSIFY

↓

IMPACT ANALYSIS

↓

AFFECTED TRADES

↓

POLICY UPDATE

↓

SIMULATION

↓

APPROVAL

↓

WASM/POLICY RELEASE

↓

DEPLOYMENT

↓

LOOM

Never silently change production regulatory behavior.

11. PRODUCT REGULATORY PROFILE

Extend product intelligence with:

- HS6
- national tariff codes
- alternate classifications
- composition
- materials

- CAS
- brand
- model
- serial requirements
- agricultural classification
- food
- pharmaceutical
- veterinary
- chemical
- DG
- dual-use
- strategic
- CITES
- packaging
- labeling
- shelf life
- temperature
- conformity
- SPS

12. CLASSIFICATION ENGINE

Support:

- HS
- national tariff code
- statistical classification
- export classification
- dual-use
- strategic goods

AI assists classification.

AI does not become final legal authority.

Low confidence:

CONDITIONAL

Human review.

13. TARIFF ENGINE

Support:

- MFN

- preferential
- quotas
- anti-dumping
- countervailing
- safeguard
- agricultural levy
- excise
- environmental fee
- import surcharge
- other measures

Every rule must have:

- jurisdiction
- HS
- origin
- effective date
- legal source

14. ORIGIN ENGINE

Support:

- non-preferential origin
- preferential origin
- wholly obtained
- substantial transformation
- tariff shift
- regional value content
- processing rules
- cumulation
- direct shipment
- approved exporter
- origin declaration
- COO
- EUR.1/equivalent

The engine determines whether preferential tariff eligibility actually exists.

15. TRADE AGREEMENT ENGINE

Create a registry for:

- bilateral

- multilateral
- FTA
- PTA
- customs union
- regional agreements
- sectoral agreements

Store:

- parties
- dates
- tariff schedule
- product coverage
- origin rules
- quotas
- certification
- cumulation
- exclusions
- direct transport
- suspension.

16. LICENSE ENGINE

Support:

- import licenses
- export licenses
- transit licenses
- controlled-goods licenses
- strategic-goods licenses
- special product licenses

States:

NOT_REQUIRED

REQUIRED

APPLICATION_READY

SUBMITTED

PENDING

ISSUED

EXPIRED

REVOKED

REJECTED

17. PERMIT ENGINE

Support:

- SPS
- agriculture
- veterinary
- food
- pharmaceuticals
- environment
- chemicals
- communications
- strategic goods
- import
- export
- transit
- transport

Dynamic determination from product + jurisdiction + transaction.

18. CERTIFICATE ENGINE

Support:

- COO
- preferential COO
- EUR.1
- phytosanitary
- health
- veterinary
- laboratory
- analysis
- conformity
- inspection
- fumigation
- treatment
- cold treatment
- Halal
- organic
- insurance

- security.

19. SPS ENGINE

Determine requirements from:

- commodity
- HS
- origin
- destination
- season
- intended use
- mode

Support:

- plant health
- animal health
- food safety
- quarantine
- sampling
- treatment
- lab
- inspection
- release.

20. TBT ENGINE

Support:

- technical regulations
- mandatory standards
- conformity
- testing
- product registration
- labeling
- energy
- electrical safety
- EMC
- radio
- environmental requirements.

21. CONTROLLED-GOODS ENGINE

Support:

- dual-use
- military/strategic
- chemicals
- biological
- radioactive
- controlled medicine
- weapons-related
- cyber/advanced technology
- CITES

and:

- export licenses
- import licenses
- end-user statements
- end-use certificates
- re-export restrictions
- transit controls.

22. SANCTIONS ENGINE

Screen:

- trader
- seller
- buyer
- UBO
- banks
- logistics providers
- vessels
- aircraft
- ports
- relevant intermediaries

States:

CLEAR

POTENTIAL_MATCH

ENHANCED_DD

BLOCKED

AI assists.

A4 enforces.

23. CUSTOMS VALUATION ENGINE

Calculate/validate:

- transaction value
- freight
- insurance
- assists
- royalties
- commissions
- related-party adjustments
- currency
- customs valuation method
- tax base.

24. TRUE LANDED COST ENGINE

Calculate:

GOODS

+ ORIGIN COST

+ PACKAGING

+ INLAND

+ EXPORT CLEARANCE

+ CERTIFICATES

+ INSPECTION

+ INSURANCE

+ INTERNATIONAL FREIGHT

+ TRANSIT

+ DESTINATION HANDLING

+ DUTY

+ VAT/GST

+ EXCISE

+ BROKER

+ LOCAL DELIVERY

+ OTHER FEES

+ SGTX FEES

This becomes a decision-support component before trade lock.

25. ORDER MANAGEMENT

Expand commercial flow:

RFQ

→ QUOTE

→ NEGOTIATION

→ PO

→ SO

→ PROFORMA

→ CONTRACT

→ FULFILLMENT

Support:

- partial
- amendment
- cancellation
- split shipment
- backorder
- replacement
- return.

26. INCOTERM ENGINE

Machine-read:

- cost responsibility
- risk responsibility
- transport
- insurance
- export clearance
- import clearance
- permits
- duty
- tax
- documents
- transfer point
- delivery.

The existing blueprint already ties transport configuration and Incoterms to the trade request.

27. UNIFIED DOCUMENT ENGINE

All documents remain centralized.

Classify:

COMMERCIAL

CUSTOMS

REGULATORY

TRANSPORT

FINANCIAL

SECURITY

INSURANCE

Each has legal metadata.

28. DOCUMENT CONSISTENCY ENGINE

Cross-check:

- order
- contract
- invoice
- packing
- COO
- certificates
- customs
- AWB
- B/L
- e-CMR
- RoRo documents
- permits
- insurance
- manifest
- settlement.

Check:

- party
- quantity
- weight
- value
- HS
- currency
- origin
- destination
- dates
- transport identity.

29. DOCUMENT AUTHENTICATION / LEGALIZATION

Support:

- chamber authentication
- government authentication
- foreign ministry legalization
- consular legalization
- apostille
- certified translation
- digital signature
- electronic seal
- qualified electronic signature

30. MULTI-AGENCY GOVERNMENT ENGINE

Use:

SEQUENTIAL

PARALLEL

CONDITIONAL

RISK_TRIGGERED

OPTIONAL

Example:

CUSTOMS

+ AGRICULTURE

+ FOOD

+ HEALTH

+ STANDARDS

+ SECURITY

↓

RELEASE

This builds directly on the blueprint's existing Multi-Agency Workflow.

31. GLOBAL CUSTOMS ENGINE

Support:

- import
- export
- transit
- temporary import
- temporary export

- inward processing
- outward processing
- bonded warehouse
- customs warehouse
- free zone
- re-export
- re-import
- destruction
- abandonment
- drawback
- post-clearance.

32. GLOBAL SINGLE-WINDOW GATEWAY

Use:

- API
- EDI
- XML
- JSON
- UN/EDIFACT
- SFTP
- portal
- broker
- manual.

Map:

SGTX Canonical Model

↓

WCO / Regional Model

↓

National Model

↓

Authority System

The WCO Data Model specifically supports harmonized datasets for customs and other border agencies in Single Window environments.

33. GOVERNMENT CONNECTOR STANDARD

Every connector supports:

DISCOVER

AUTHENTICATE

VALIDATE

PREPARE

SUBMIT

STATUS

AMEND

CANCEL

INSPECT

RELEASE

DOCUMENT

PERMIT

CERTIFICATE

PAYMENT

RECONCILE

34. GOVERNMENT CONNECTOR STATES

Use:

NOT_DISCOVERED

DISCOVERED

DOCUMENTED

CONTACT_REQUIRED

CREDENTIALS_REQUIRED

SANDBOX_AVAILABLE

SANDBOX_CONNECTED

CERTIFICATION_REQUIRED

CERTIFICATION_PENDING

PRODUCTION_READY

PRODUCTION_CONNECTED

DEGRADED

OUTAGE

PORTAL_ONLY

MANUAL_ONLY

DEPRECATED

35. GOVERNMENT AUTHORITATIVE STATUS

Separate:

SGTX_READY

SUBMITTED

GOVERNMENT_ACCEPTED

GOVERNMENT_REJECTED

GOVERNMENT_HOLD

GOVERNMENT_RELEASED

MANUAL_AUTHORITY_CONFIRMED

SGTX never invents government approval.

36. GOVERNMENT INTEGRATION CONTROL CENTER

The existing government portal already provides integration management, connection testing and connector logs.

Upgrade it globally.

Display:

- authority
- system
- jurisdiction
- mode
- procedure
- API
- EDI
- portal
- credentials
- certificate
- sandbox
- certification
- legal agreement
- production status
- last success
- last error
- version
- owner.

37. FOUR-DIMENSION EXTERNAL READINESS

Every integration reports independently:

TECHNICAL

LEGAL

OPERATIONAL

COMMERCIAL

This avoids saying:

"Connected"

when technically connected but legally uncertified.

38. MISSING-INTEGRATION CENTER

Every missing requirement must show:

JURISDICTION

AUTHORITY

SYSTEM

PROCEDURE

MODE

WHY_REQUIRED

API

EDI

PORTAL

SANDBOX

CREDENTIALS

CERTIFICATION

LEGAL_AGREEMENT

STATUS

PRIORITY

OWNER

NEXT_ACTION

AFFECTED_USTNs

AFFECTED_TRADE_LANES

Priority:

P0 = legally blocks

P1 = major trade-lane blocker

P2 = manual operation

P3 = optimization

P4 = optional

39. COUNTRY ACTIVATION

Country activation workflow:

SELECT JURISDICTION

↓

LOAD OFFICIAL SOURCES

↓

CONFIGURE CUSTOMS



CONFIGURE TAX



CONFIGURE SPS



CONFIGURE TBT



CONFIGURE LICENSES



CONFIGURE TRANSPORT



IDENTIFY GOVERNMENT SYSTEMS



IDENTIFY APIs/EDI/PORTALS



CREDENTIALS



SANDBOX



CONFORMANCE



LEGAL REVIEW



PRODUCTION AUTHORIZATION



ACTIVATE



LOOM

40. COUNTRY ACTIVATION ≠ PRODUCTION ACTIVATION

This rule must be enforced everywhere.

For example:

COUNTRY_CONFIGURED

+

ADAPTER_READY

+

NO PRODUCTION CREDENTIALS

must remain:

INTEGRATION_REQUIRED

not:

PRODUCTION_CONNECTED.

41. TRADE LANE PASSPORT

For every:

ORIGIN

DESTINATION

TRANSIT

COMMODITY

HS

MODE

INCOTERM

generate a:

TRADE_LANE_PASSPORT

containing:

- legal
- customs
- licenses
- permits
- certificates
- transport
- providers
- broker
- tax
- duty
- government systems
- payment
- insurance
- manual steps
- expected exceptions.

The blueprint already describes Trade Lane Passport and Corridor Registry as core TCN concepts.

42. TRADE-LANE READINESS

Calculate:

PARTY_READY

PRODUCT_READY

CLASSIFICATION_READY

ORIGIN_READY

LICENSE_READY

PERMIT_READY

CERTIFICATE_READY

CUSTOMS_READY

TRANSPORT_READY

SECURITY_READY

TAX_READY

PAYMENT_READY

INSURANCE_READY

DELIVERY_READY

GOVERNMENT_CONNECTIVITY_READY

43. ROAD CORRIDOR ENGINE

Road becomes a first-class engine:

ROAD_CORRIDOR

ROAD_LEG

BORDER

CUSTOMS

TRANSIT

GUARANTEE

DRIVER

VEHICLE

TRAILER

SEAL

WAYBILL

GPS

GEOFENCE

INCIDENT

POD

Support:

- Egypt

- Jordan
- Saudi
- UAE
- GCC
- Iraq
- Libya
- all future jurisdictions.

44. ROAD MULTI-COUNTRY WORKFLOW

Example:

Egypt

↓

Export Customs

↓

Border

↓

Jordan Transit

↓

Saudi Transit

↓

UAE Import Customs

↓

Final Delivery

Every leg has its own legal/customs status.

45. TIR / CUSTOMS GUARANTEE ENGINE

Generalize beyond TIR:

TIR

eTIR

CUSTOMS BOND

BANK GUARANTEE

TRANSIT GUARANTEE

COMPREHENSIVE GUARANTEE

DUTY DEFERRAL

TEMPORARY ADMISSION

WAREHOUSE GUARANTEE

46. ROAD VEHICLE / DRIVER AUTHORIZATION

Validate:

- vehicle registration
- insurance
- roadworthiness
- payload
- trailer
- reefer
- DG capability
- country permits

Driver:

- passport
- license
- visa
- permit
- international authorization
- DG authorization.

47. AIR CARGO ENGINE

First-class:

AIR_BOOKING

AIRLINE

AIRPORT

GHA

FLIGHT

MAWB

HAWB

e-AWB

CARGO-XML

ONE RECORD

SECURITY

e-CSD

DG

e-DGD

ULD

RCS

DEP

ARR

RCF

NFD

DLV

CUSTOMS

ACI AIR

WAREHOUSE

BUILD-UP

BREAKDOWN

IATA currently describes ONE Record as the air cargo standard for a single digital shipment view using a common data model and secured API. Its 2026 Cargo-XML toolkit explicitly includes mapping between Cargo-XML and ONE Record and e-CSD-related guidance.

48. AIR CUTOFF ENGINE

Support:

- documentation cutoff
- customs cutoff
- security cutoff
- acceptance cutoff
- airline cutoff
- buildup cutoff
- ULD cutoff.

49. AIR CHARGEABLE WEIGHT

Calculate:

- actual
- volumetric
- dimensional
- chargeable.

Feed:

- quotation
- booking
- AWB
- customs
- insurance
- settlement.

50. AIR SECURITY / DG / SPECIAL CARGO

Support:

- screening
- e-CSD
- DG
- e-DGD
- pharma
- perishables
- live animals
- human remains
- valuables
- temperature controlled
- oversized
- lithium batteries.

Rules must be versioned.

51. AIR PIECE / ULD TRACKING

Track:

USTN

MAWB

HAWB

PIECE

SSCC

ULD

FLIGHT

LOCATION

SECURITY

CUSTOMS

TEMPERATURE

52. AIR MULTIMODAL

Support:

ROAD → AIR

AIR → ROAD

ROAD → AIR → ROAD

ROAD → AIR → AIR → ROAD

Use Road Engine for road segments.

53. OCEAN CONTAINER ENGINE

Keep container ocean as a distinct engine:

BOOKING

VESSEL

VOYAGE

PORT

CONTAINER

VGM

B/L

e-B/L

MANIFEST

ACI

CUSTOMS

TERMINAL

GATE

TRANSSHIPMENT

DEMURRAGE

DETENTION

DELIVERY

54. RAIL ENGINE

Support:

- rail booking
- train
- wagon
- terminal
- consignment
- transit
- customs
- tracking
- interchange
- delivery.

55. NEW FIRST-CLASS ENGINE: RORO & ROLLING CARGO

This is the major addition requested now.

Do not treat RoRo as simply Ocean.

Create:

RORO & ROLLING CARGO ENGINE

The transport hierarchy becomes:

TRANSPORT ORCHESTRATOR



■■■■ ROAD

■■■■ AIR

■■■■ OCEAN CONTAINER

■■■■ RORO / ROLLING CARGO

■■■■ RAIL

■■■■ FERRY

■■■■ MULTIMODAL

56. RORO MASTER OBJECT

Create:

RORO_SHIPMENT

under USTN.

Structure:

USTN

■■■■ RORO_SHIPMENT

■■■■ BOOKING

■■■■ VOYAGES

■■■■ PORT_CALLS

■■■■ MANIFEST

■■■■ RORO_UNITS

■■■■ CUSTOMS

■■■■ INSPECTIONS

■■■■ YARD

■■■■ LOADING

■■■■ DISCHARGE

■■■■ DELIVERY

■■■■ PAYMENTS

■■■■ CLAIMS

■■■■ EVIDENCE

57. RORO UNIT IDENTITY

Every rolling unit has:

RORO_UNIT_ID

VIN

CHASSIS

REGISTRATION

MAKE

MODEL

YEAR

VEHICLE_TYPE

WEIGHT

DIMENSIONS

FUEL

BATTERY

RUNNING_STATUS

CONDITION

OWNER

EXPORTER

IMPORTER

CONSIGNEE

HS

ORIGIN

For non-vehicle rolling cargo:

- serial
- equipment ID
- dimensions
- weight
- condition.

58. RORO VEHICLE TYPES

Support:

- passenger vehicles
- trucks
- trailers
- buses
- tractors
- agricultural equipment
- construction machinery
- industrial equipment
- motorcycles
- boats on trailers

- non-running units
- oversized machinery.

59. RORO BOOKING ENGINE

Support:

- port pair
- vessel
- voyage
- number of units
- VIN
- dimensions
- weight
- running/non-running
- special handling
- hazardous status
- oversized
- preferred sailing
- delivery window
- Incoterm.

60. RORO VOYAGE MODEL

Track:

VESSEL

IMO

VOYAGE

OPERATOR

ORIGIN

DESTINATION

TRANSIT PORTS

ETD

ETA

ACTUAL DEPARTURE

ACTUAL ARRIVAL

BOOKING CUTOFF

DOCUMENT CUTOFF

GATE CUTOFF

CARGO CUTOFF

STATUS

61. RORO MANIFEST

Manifest must be unit-level.

Every unit links:

VIN

RORO_UNIT_ID

BOOKING

VOYAGE

SHIPPER

CONSIGNEE

HS

WEIGHT

DESTINATION

CUSTOMS

GATE

LOADING

62. VIN-LEVEL CUSTOMS RECONCILIATION

SGTX must answer:

- Is this VIN customs-cleared?
- Which declaration?
- Which voyage?
- Which port?
- Was it gated in?
- Was it loaded?
- Was it discharged?
- Where is it in the yard?
- Has it gated out?
- Has it been delivered?

63. RORO TERMINAL ADAPTER

Support:

- pre-advice
- booking
- gate appointment
- gate-in
- VIN scan

- inspection
- yard assignment
- parking
- stock
- loading
- ramp assignment
- discharge
- gate-out.

64. RORO YARD ENGINE

Track:

YARD

ZONE

BLOCK

ROW

SLOT

DECK

POSITION

VIN

UNIT

STATUS

ARRIVAL

STORAGE START

FREE TIME

Statuses:

EXPECTED

ARRIVED

GATE_IN

INSPECTED

PARKED

READY_FOR_LOADING

LOADED

DISCHARGED

AVAILABLE

RELEASED

GATE_OUT

65. RORO GATE ENGINE

Origin:

APPOINTMENT

→ ARRIVAL

→ VIN SCAN

→ DOCUMENT CHECK

→ CUSTOMS CHECK

→ INSPECTION

→ GATE-IN

Destination:

DELIVERY ORDER

→ CUSTOMS RELEASE

→ VIN SCAN

→ CONDITION CHECK

→ GATE-OUT

66. RORO INSPECTION ENGINE

Capture:

- VIN
- time
- inspector
- location
- photos
- videos
- mileage
- fuel
- battery
- keys
- tires
- glass
- mirrors
- exterior
- interior
- pre-existing damage
- new damage.

67. RORO DAMAGE COMPARISON

Compare origin versus destination.

AI A2:

POSSIBLE_DAMAGE

Human/authorized inspector:

CONFIRMED_DAMAGE

States:

NO_CHANGE

POSSIBLE_DAMAGE

CONFIRMED_DAMAGE

DISPUTED_DAMAGE

AI must never determine liability autonomously.

68. RORO SECURITY

Support:

- VIN
- RFID
- QR
- barcode
- digital seal
- GPS
- access control
- key custody
- restricted yard
- security events.

Egypt's Ministry of Transport describes digital integration between Damietta and Trieste, exchange of official/customs data and RFID-based digital-seal safety for the Egypt–Italy RoRo line.

69. RORO STOWAGE ENGINE

Inputs:

- dimensions
- weight
- destination
- port sequence
- deck
- ramp
- height
- restrictions
- vehicle type.

Outputs:

- deck
- zone
- position
- load order
- discharge order.

A2 optimizes.

A4 validates.

70. RORO CAPACITY ENGINE

Support:

- units
- linear meters
- square meters
- cubic meters
- weight
- deck
- height.

Do not assume container TEU is the appropriate capacity metric.

71. RORO CUTOFF ENGINE

Track:

BOOKING_CUTOFF

DOCUMENT_CUTOFF

CUSTOMS_CUTOFF

GATE_IN_CUTOFF

CARGO_CUTOFF

LOADING_CUTOFF

72. RORO BL ENGINE

Create:

RORO_BILL_OF_LADING

Support:

- master B/L
- house references where applicable
- shipper
- consignee
- notify party

- vessel
- voyage
- POL
- POD
- transit
- VINs
- cargo descriptions
- weight
- dimensions
- freight
- charges.

73. RORO SHIPPING-INSTRUCTION ENGINE

Generate shipping instructions automatically from:

- contract
- USTN
- units
- VINs
- customs
- voyage
- consignee
- origin
- destination.

No re-entry.

74. RORO UNIT STATE MACHINE

BOOKED

DOCUMENTS_PENDING

CUSTOMS_PENDING

READY_FOR_GATE

GATE_IN

INSPECTION_PENDING

INSPECTED

YARD

READY_FOR_LOAD

LOADED

AT_SEA

TRANSSHIPMENT

DISCHARGED

DESTINATION_YARD

CUSTOMS_HOLD

CUSTOMS_RELEASED

DELIVERY_ORDER

READY_FOR_GATE_OUT

GATE_OUT

DELIVERED

ACCEPTED

75. RORO VESSEL STATE

Separately:

SCHEDULED

BOOKING_OPEN

CUTOFF_APPROACHING

CARGO_ACCEPTING

LOADING

DEPARTED

AT_SEA

TRANSSHIPMENT

ARRIVED

DISCHARGING

COMPLETED

Never mix vessel state with cargo/unit state.

76. RORO CUSTOMS RECONCILIATION

Reconcile:

VIN

↔ Commercial Docs

↔ Customs

↔ Manifest

↔ Vessel/Voyage

↔ Terminal

↔ Release

↔ Delivery

77. EGYPT RORO ADAPTER

Create:

EGYPT_RORO_ADAPTER

The adapter determines:

- Nafeza applicability
- UCR applicability
- export declaration
- transit
- manifest
- shipping-agent messages
- terminal process
- customs procedures
- port requirements
- unit documentation
- vehicle documentation
- destination/transit requirements.

Do not blindly apply container-only Enhanced Export messages to RoRo.

Nafeza's July 18, 2026 Enhanced Export notice explicitly identifies its first five messages—UCR Verification, Booking Confirmation, Empty Containers, Shipment Inquiry and Manifest—for seaports/container shipping agents.

The system therefore needs mode/procedure applicability rather than one universal maritime workflow.

78. EGYPT AIR ADAPTER

Egypt's Nafeza system currently identifies an Aviation ACI system, with mandatory implementation beginning January 1, 2026, and a separate enhanced export cycle.

Therefore support:

- Nafeza
- ACI Air
- export
- import
- transit where applicable
- MAWB
- HAWB
- manifest
- customs
- certificates
- release
- government reference reconciliation.

79. EGYPT ROAD ADAPTER

Support:

- Nafeza export
- UCR where applicable
- export declaration
- transit
- customs
- SPS
- certificates
- TIR
- road waybill
- border
- vehicle
- driver
- seal.

Do not assume maritime ACI/CargoX logic applies to road exports.

80. EGYPT MODE-APPLICABILITY ENGINE

Create:

TRANSPORT_MODE_PROCEDURE_ENGINE

Modes:

ROAD

AIR

OCEAN_CONTAINER

RORO

BULK

RAIL

FERRY

MULTIMODAL

The engine determines:

- customs procedure
- government messages
- identifiers
- ACI requirements
- manifest
- declaration

- permits
- terminal requirements.

81. EGYPT RO-RO CORRIDORS

The architecture should support the existing Egypt–Europe–Gulf RoRo corridor model.

The Ministry of Transport describes the Damietta–Trieste line and multimodal onward movement by rail/road, including Gulf destinations.

Represent such a corridor as:

Egypt



Damietta



RORO



Trieste



Rail



Rotterdam



Road



Destination

One USTN.

Multiple mode legs.

82. RORO + GLOBAL TCN

The existing Trade Corridor Network becomes:

TCN



CORRIDOR



PORT



RORO SERVICE



VESSEL



VOYAGE



RORO UNITS

Do not create another corridor database.

83. MULTIMODAL ORCHESTRATOR

Final transport hierarchy:

TRANSPORT ORCHESTRATOR



■■■■ ROAD

■■■■ AIR

■■■■ OCEAN CONTAINER

■■■■ RORO / ROLLING CARGO

■■■■ RAIL

■■■■ FERRY

■■■■ MULTIMODAL

Each specialist engine owns its mode-specific rules.

The Multimodal Orchestrator joins them.

84. RORO + ROAD

Typical:

TRUCK



RORO TERMINAL



RORO VESSEL



DESTINATION RORO TERMINAL



TRUCK

One USTN.

85. RORO + RAIL

Example:

RORO



Trieste



RAIL

↓

Rotterdam

↓

ROAD

This is explicitly aligned with the Egyptian RoRo corridor example.

86. RORO + GCC TRANSIT

Support:

Europe

↓

Trieste

↓

Egypt

↓

Damietta

↓

Indirect Transit

↓

Gulf

The Egyptian Maritime Transport Sector reported in June 2026 that the Damietta–Trieste RoRo line was handling indirect-transit shipments toward Saudi Arabia, UAE, Qatar, Oman and Kuwait.

SGTX should represent those as actual multi-jurisdiction transit legs, not just "delivery."

87. GLOBAL TRANSPORT PROVIDER ADAPTER

Every approved provider gets:

- GTID
- service scope
- jurisdictions
- licenses
- insurance
- routes
- commodities
- equipment
- API/EDI/portal
- contract
- SLA
- fees.

Provider visibility remains controlled by relationship/approval.

88. CUSTOMS BROKER ENGINE

Global broker workspace supports:

- declarations
- documents
- customs questions
- inspections
- amendments
- fees
- guarantees
- releases
- transit
- post-clearance.

Broker licenses must be validated against jurisdiction/scope.

89. GLOBAL TAX ENGINE

Support:

- VAT
- GST
- sales tax
- excise
- import tax
- withholding
- environmental levy
- special tax
- e-invoice
- e-receipt
- tax payment
- refund.

Never infer:

CUSTOMS RELEASED = TAX COMPLETE

90. GLOBAL FINANCIAL EXECUTION

Support:

- bank transfer
- PSP
- open banking

- local rails
- SWIFT
- ISO 20022
- documentary collection
- LC
- guarantees
- deferred government fees
- refunds.

SGTX orchestrates instructions/status but does not become the bank.

91. TRADE FINANCE

Support:

- connected bank financing
- trader-selected financier
- LC
- documentary collection
- guarantee
- standby
- collateral
- disbursement
- repayment
- reconciliation.

92. INSURANCE

Full lifecycle:

QUOTE

→ BIND

→ POLICY

→ CERTIFICATE

→ INCIDENT

→ CLAIM

→ SURVEY

→ SETTLEMENT

→ RECOVERY

→ CLOSE

93. ACCOUNTING

Support:

- AP
- AR
- landed cost
- freight
- duty
- tax
- insurance
- accrual
- settlement
- refund
- FX
- inventory
- COGS.

94. ERP

Adapters:

- SAP
- Oracle
- Dynamics
- NetSuite
- Odoo
- generic REST
- SOAP
- EDI
- SFTP.

95. CUSTOMS GUARANTEE ENGINE

General guarantee system:

- TIR
- eTIR
- customs bond
- bank guarantee
- comprehensive guarantee
- transit guarantee
- temporary admission
- duty deferral
- warehouse guarantee.

96. BONDED / SPECIAL REGIMES

Support:

- bonded warehouse
- customs warehouse
- free zone
- free port
- inward processing
- outward processing
- temporary admission
- temporary export
- duty suspension
- end-use relief
- drawback.

97. POST-CLEARANCE

Support:

- audit
- query
- correction
- reassessment
- refund
- drawback
- penalty
- appeal
- ruling
- record retrieval.

98. DELIVERY ACCEPTANCE

Final delivery requires:

- receiver
- quantity
- condition
- quality
- temperature where applicable
- POD
- document acceptance
- acceptance timestamp.

DELIVERED is not the same as ACCEPTED.

99. CLAIMS

Support:

- damage
- shortage
- quality
- temperature
- delay
- customs
- documentation
- carrier
- insurance
- warranty.

100. RETURNS / REVERSE TRADE

Support:

- rejection
- return
- warranty
- repair
- replacement
- re-export
- re-import
- destruction
- abandonment.

Use parent/child USTNs.

----- END v13.0 RE-INTEGRATED SEGMENT (Add-Ons 9-28 and Final Summary preamble) -----

The existing v12.0 Final Summary tail (articles 101 through 162) resumes immediately below.

101. FINAL EVIDENCE

At closure create:

FINAL_TRADE_EVIDENCE_PACKAGE

containing:

- RFQ
- quotes
- PO/SO
- contract

- regulatory snapshot
- classification
- origin
- licenses
- permits
- certificates
- customs
- transport
- government references
- payment
- bank
- settlement
- accounting
- inspection
- QC
- insurance
- GPS
- IoT
- delivery
- claims
- disputes
- post-clearance
- Governor
- Loom.

102. PHASE 10 - USTN CLOSURE

The remediation already established the correct rule.

Maintain:

USTN_CLOSED

? canClose = true

canClose = true

? seven closure conditions true

Closure requires:

1 delivery accepted

2 settlement complete

3 financial reconciliation complete

4 customs complete

5 post-clearance complete

6 disputes/claims satisfied according to closure policy

7 evidence sealed

Never allow authoritative contradictory state.

103. SEMANTIC E2E VALIDATOR

Every lifecycle stage verifies:

L1 EXISTENCE

L2 REFERENTIAL INTEGRITY

L3 STATE INTEGRITY

L4 CONSTITUTIONAL AUTHORIZATION

Not just database presence.

104. THREE CANONICAL E2E FIXTURES

Maintain:

COMPLETE

e2ePassed=true

canClose=true

USTN_CLOSED

SETTLEMENT_BLOCKED

e2ePassed=false

canClose=false

SETTLEMENT_INCOMPLETE

MULTI_BLOCKED

e2ePassed=false

canClose=false

SETTLEMENT_INCOMPLETE

POST_CLEARANCE_OPEN

EVIDENCE_NOT_SEALED

Historical fixtures are marked:

HISTORICAL_FIXTURE.

105. STATE-INTEGRITY INVARIANTS

Maintain all 12 previously defined invariants.

Especially:

USTN_CLOSED + canClose=false

= STATE_INTEGRITY_EXCEPTION

and the exception must never be silently accepted as valid production state.

106. PHASE 10 COMPLETENESS MATRIX

The final system must maintain the FINAL_COMPLETENESS_MATRIX.

Minimum columns:

Subsystem

Implemented

Database

API

OpenAPI

UI

Governor

OPA

WasmEdge

Loom

Tested

Integrated

Production

Fallback

Admin Visible

Documentation

Owner

Status

No UNKNOWN at final acceptance.

107. PRODUCTION READINESS TERMINOLOGY

Use:

CORE_READY

ADAPTER_READY

COUNTRY_CONFIGURED

SANDBOX_CONNECTED

PRODUCTION_CONNECTED

MANUAL_ONLY

PORTAL_ONLY

INTEGRATION_REQUIRED

LEGAL_AUTHORIZATION_REQUIRED

Never claim:

WORLDWIDE_INTEGRATED

without evidence.

108. PRODUCTION READINESS COVERAGE

Use:

PRODUCTION READINESS COVERAGE

rather than a generic "readiness score."

This is not:

- legal compliance percentage
- operational probability
- worldwide integration percentage.

109. EXTERNAL READINESS

Every connector separately reports:

TECHNICAL

LEGAL

OPERATIONAL

COMMERCIAL

This is essential for government integration.

110. DATA LOCALIZATION

Each jurisdiction declares:

EGYPT_ONLY

COUNTRY_ONLY

REGIONAL

APPROVED_CROSS_BORDER

GLOBAL_ALLOWED

Apply the strictest relevant rule.

Only send required data.

111. DIGITAL SIGNATURE / LEGALITY

Support:

- standard electronic signatures
- advanced electronic signatures
- qualified electronic signatures
- digital seals
- certificates
- notarization
- legalization

- apostille
- certified translations.

112. GLOBAL STANDARDS GATEWAY

Support mappings for:

- WCO Data Model
- WCO code lists
- HS
- UN/CEFACT
- UN/LOCODE
- UN/EDIFACT
- UBL
- e-CMR
- e-AWB
- e-B/L
- Cargo-XML
- ONE Record
- ISO 20022
- XAdES
- CAdES
- PAdES
- CMS
- QES
- verifiable credentials
- GS1
- EPCIS.

For air freight, keep both Cargo-XML and ONE Record: IATA describes ONE Record as the standardized digital data-sharing model, while its current Cargo-XML toolkit specifically addresses mappings between Cargo-XML and ONE Record.

113. ERP / ACCOUNTING / BANK RECONCILIATION

Final reconciliation:

SGTX

? Bank

? PSP

? Government

? Carrier

? Broker

? Insurance

? ERP

? Accounting

No unresolved material mismatch before USTN closure.

114. MANUAL FALLBACK

Any unavailable API must fall through:

API

?

EDI

?

SFTP

?

PORTAL

?

BROKER

?

MANUAL

Manual action must still be:

- authenticated
- attributable
- timestamped
- documented
- hashed
- Loom logged.

115. CONNECTOR SECURITY

All external integration connectors should support as applicable:

- mTLS
- OAuth
- API key
- HSM
- secret rotation
- certificate rotation
- signed payloads
- encryption
- replay prevention

- idempotency
- rate limiting
- network isolation
- audit.

116. CONNECTOR VERSION MANAGEMENT

Store:

SYSTEM_VERSION

API_VERSION

SCHEMA_VERSION

LEGAL_VERSION

JURISDICTION_VERSION

No government update should silently break the system.

117. REGIONAL ADAPTERS

Build reusable regional frameworks:

- EU
- GCC
- ASEAN
- AfCFTA-related environments
- ECOWAS
- EAC
- COMESA
- SADC
- other applicable regional systems.

Country overlays add national rules.

118. ALL-WORLD COUNTRY ADAPTER ARCHITECTURE

The system must support a country/jurisdiction adapter for every country worldwide.

Do not hard-code every country's law into the core.

Each adapter receives:

- customs
- tax
- SPS
- TBT
- licenses
- permits
- certificates

- transport
- security
- banking/payment
- e-invoicing
- digital-signature
- legalisation
- customs broker
- government systems
- local APIs
- EDI
- portals
- manual procedures.

119. GLOBAL COUNTRY ACTIVATION CENTER

Admin can:

- activate country
- configure authority
- add connector
- add credentials
- configure sandbox
- request certification
- track legal authorization
- activate production
- deactivate
- rollback
- view affected trade lanes.

120. FINAL INTEGRATION-GAP CENTER

The Admin Portal must expose:

JURISDICTION

AUTHORITY

SYSTEM

PROCEDURE

MODE

REQUIRED

TECHNICAL

LEGAL

OPERATIONAL

COMMERCIAL

CREDENTIAL

CERTIFICATION

LEGAL AGREEMENT

STATUS

PRIORITY

OWNER

NEXT ACTION

AFFECTED USTNs

AFFECTED LANES

No hidden external dependency.

121. GLOBAL TRADE CONTROL TOWER

Executive dashboard should show:

- active trades
- trade value
- shipments
- customs
- government holds
- logistics status
- border delays
- air delays
- ocean delays
- RoRo vessels
- RoRo units
- rail
- payment
- settlement
- claims
- insurance
- regulatory changes
- connector health
- integration gaps
- USTN closure.

122. RORO CONTROL TOWER

Specific cards:

ACTIVE RORO

UNITS

VESSELS

VOYAGES

GATE-IN

YARD

READY TO LOAD

LOADED

AT SEA

ARRIVED

CUSTOMS HOLD

DAMAGED

STORAGE EXPOSURE

GATE-OUT

DELIVERED

Search by:

- USTN

- VIN

- booking

- voyage

- vessel

- consignee.

123. AIR CONTROL TOWER

Show:

- MAWB

- HAWB

- flight

- RCS

- DEP

- ARR

- RCF

- NFD

- customs

- security

- DG
- cutoffs
- ULD
- temperature
- connection risk.

124. ROAD CONTROL TOWER

Show:

- corridor
- country
- border
- vehicle
- driver
- customs
- transit guarantee
- seal
- GPS
- route deviation
- ETA
- temperature
- border waiting
- incidents.

125. OCEAN CONTROL TOWER

Show:

- vessel
- voyage
- port
- container
- B/L
- manifest
- customs
- ETA
- gate
- transshipment
- demurrage/detention
- delivery.

126. MULTIMODAL CONTROL TOWER

Show the complete graph:

LEG 1

?

LEG 2

?

LEG 3

?

LEG 4

with each mode visually distinct but under the same USTN.

127. AI AUTHORITY - FINAL

A1

May:

- explain
- translate
- suggest
- summarize
- notify
- generate draft instructions.

A2

May:

- classify
- detect anomalies
- detect discrepancies
- predict delays
- predict connection risk
- compare images
- estimate ETA
- optimize ULD/stowage
- analyze route.

A3

May:

- escalate.

A4

OPA + WasmEdge:

- enforce policy
- validate state
- validate documents
- enforce customs gates
- enforce regulatory gates
- enforce transport gates
- enforce financial gates
- enforce closure.

A5

Forbidden.

128. SECURITY / GOVERNOR / LOOM

Every irreversible event:

Human Intent

?

Authentication

?

Authorization

?

Governor

?

OPA

?

WasmEdge

?

Decision

?

Signature

?

Loom

?

External/System Action

No bypass.

129. FINAL E2E TRADE WORKFLOW

The final complete SGTX lifecycle is:

BUYER/SELLER TRADE INTENT

?

KNOWN COUNTERPARTY

?

RFQ

?

QUOTE

?

NEGOTIATION

?

PO / SO

?

PROFORMA

?

CONTRACT

?

REGULATORY SNAPSHOT

?

PRODUCT CLASSIFICATION

?

ORIGIN

?

FTA/PREFERENCE

?

LICENSE

?

PERMIT

?

CERTIFICATE

?

INSURANCE

?

PACKING

?

TRANSPORT CONFIGURATION

?

BOOKING

?

EXPORT CUSTOMS

?

SECURITY

?

PHYSICAL EXECUTION

?

TRANSIT

?

IMPORT CUSTOMS

?

DUTY / TAX

?

INSPECTION

?

RELEASE

?

DELIVERY

?

ACCEPTANCE

?

SETTLEMENT

?

BANK/PSP RECONCILIATION

?

ACCOUNTING

?

CLAIMS/WARRANTY WINDOW

?

POST-CLEARANCE

?

RETURN/DRAWBACK/REFUND IF REQUIRED

?

FINAL EVIDENCE

?

USTN CLOSED

130. FINAL TRANSPORT ARCHITECTURE

The final transportation layer is now:

SGTX TRANSPORT ORCHESTRATOR

?

??

? ? ? ? ? ?

ROAD AIR OCEAN RORO RAIL FERRY OTHER/

CONTAINER ? FUTURE

? ? ? ? ?

??

?

MULTIMODAL GRAPH

?

USTN

RoRo is now explicitly first-class.

This is important operationally because the Egypt-Italy RoRo corridor is not just an ocean voyage: the Egyptian Ministry of Transport describes Damietta-Trieste RoRo, electronic exchange of official/customs data, RFID-based digital-seal safety, and onward rail/road movement to Rotterdam and other European destinations.

The 2026 Egyptian Maritime Transport Sector also describes indirect-transit RoRo flows toward Saudi Arabia, UAE, Qatar, Oman and Kuwait, which reinforces the need for RoRo to participate natively in SGTX's multimodal and customs-transit graph.

131. FINAL EGYPT MODE ARCHITECTURE

Egypt should not have one generic "Nafeza integration."

It should have:

EGYPT GOVERNMENT ADAPTER

?

??? EXPORT

??? IMPORT

??? TRANSIT

?

??? SEA CONTAINER

??? RORO

??? AIR / ACI AIR

??? ROAD

??? RAIL

?

??? SPS

??? HEALTH

??? AGRICULTURE

??? LAB

??? CERTIFICATES

??? TAX / ETA

??? OTHER AUTHORITIES

Nafeza currently distinguishes the Advanced Export System/UCR from ACI Air, and its current July 2026 Enhanced Export message notice specifically targets shipping agents and container shipping at seaports.

Therefore SGTX's mode-specific applicability architecture is essential.

132. FINAL WORLDWIDE STANDARDS ARCHITECTURE

SGTX CANONICAL DATA

?

??? WCO Data Model

??? UN/CEFACT

??? UN/EDIFACT

??? UN/LOCODE

??? ISO 20022

??? UBL

??? GS1

??? Cargo-XML

??? ONE Record

??? e-AWB

??? e-B/L

??? e-CMR

??? National/Regional Standards

The WCO Data Model is explicitly intended as a harmonized interoperability foundation for Customs and other cross-border regulatory agencies, which makes it the right normalization/reference layer rather than hard-coding every national schema into SGTX core.

133. FINAL PRODUCTION-READINESS MODEL

SGTX's truthful deployment states remain:

CORE_READY

ADAPTER_READY

COUNTRY_CONFIGURED

SANDBOX_CONNECTED

PRODUCTION_CONNECTED

MANUAL_ONLY

PORTAL_ONLY

INTEGRATION_REQUIRED

LEGAL_AUTHORIZATION_REQUIRED

The current Phase 10 remediation correctly moved SGTX away from claiming global integration where external certifications/credentials are still missing.

134. FINAL DEFINITION OF "COMPLETE"

SGTX is internally architecturally complete when:

ALL CORE DOMAINS EXIST

+

ALL TRANSPORT MODES EXIST

+

ALL REGULATORY CLASSES EXIST

+

ALL GOVERNMENT ADAPTER TYPES EXIST

+

ALL FINANCIAL FLOWS EXIST

+

ALL POST-TRADE FLOWS EXIST

+

E2E VALIDATION EXISTS

+

USTN CLOSURE IS CONSISTENT

+

INTEGRATION GAPS ARE VISIBLE

SGTX is externally operational only when the actual institutions have:

- authorized access
- production credentials
- certifications
- contracts/agreements where required
- operational procedures
- tested production connectors.

That distinction must remain permanent.

135. FINAL SGTX IDENTITY

The completed architecture should therefore be described as:

SGTX - Sovereign Global Trade Exchange - is a non-custodial, relationship-controlled, sovereign-governed trade execution infrastructure that connects commercial transactions with regulatory requirements, customs, government systems, approved logistics providers, approved transport operators, connected banks and trader-selected financing institutions, orchestrating the complete physical and financial trade lifecycle through a single USTN-centered, auditable execution graph.

Not a marketplace.

Not a public logistics directory.

Not an autonomous AI trader.

Not a bank.

Not a customs authority.

Not a carrier.

Not a government.

But the orchestration and execution layer connecting all of them.

And after this amendment, the transport layer is properly:

Road + Air + Ocean Container + RoRo/Rolling Cargo + Rail + Ferry + Multimodal, all beneath one transport orchestrator and one USTN graph.

One important external-standard note

I would also make the standards gateway version-aware rather than hard-coding a single WCO version. The WCO currently documents Data Model 3.x releases while its 2026 guidance describes the future transition toward Version 4. That means SGTX should store WCO_DM_VERSION as configuration and support mapping evolution rather than embedding one immutable WCO schema.

This consolidated amendment now includes the global regulatory fabric, worldwide jurisdiction system, customs/single-window architecture, road corridors, air freight, ocean container, RoRo/rolling cargo, rail, ferry, multimodal orchestration, product classification, tariff/origin/FTA, licenses/permits/certificates, SPS/TBT, controlled goods/sanctions, government adapters, tax, payment, connected-bank financing, trader-saved financiers, insurance, ERP/accounting, post-clearance, returns, claims, evidence, provider controls, integration-gap center, regulatory change management, Trade Lane Passport, USTN closure hardening, Phase 10 completeness controls, and the Egypt-specific mode-aware Nafeza architecture established throughout this conversation.

SGTX PLATFORM - MASTER CONSTITUTIONAL AMENDMENT

TRANSACTION FINALITY, MULTI-CLOCK STATE, EVENT INTEGRITY, SETTLEMENT ORCHESTRATION, RECONCILIATION, EXCEPTION, RECOVERY, EVIDENCE, FINANCIAL EXPOSURE, CROSS-BORDER & INSTITUTIONAL GOVERNANCE

Status: ACCEPTED / INTEGRATED / CANONICAL

Version: 1.0 - Accepted and Integrated

Role: Master amendment to the SGTX Blueprint

Architectural Position: Core / Constitutional

Scope: Entire SGTX transaction lifecycle and all connected execution, banking, ERP, marketplace, logistics, document, customs, compliance, financial, legal, regulatory, AI and institutional interfaces.

1. PURPOSE AND ARCHITECTURAL AUTHORITY

This document is the canonical consolidation of the accepted SGTX architectural amendments.

It supersedes fragmented or duplicative descriptions of:

- transaction finality;
- multi-clock state;
- immutable event history;
- reversal handling;
- reconciliation;
- exception management;
- recovery;
- evidence;
- financial exposure;
- bank-side settlement;
- multi-party settlement;
- counterparty state;
- document authenticity;
- cross-border jurisdiction;
- AI governance;
- institutional authority;
- USTN correlation;
- post-closure behavior.

Where a previous blueprint section conflicts with this amendment, this amendment governs unless a later formally approved constitutional amendment explicitly supersedes it.

The architecture is designed to preserve SGTX's original non-custodial, jurisdiction-aware, institutional execution philosophy while making its treatment of real-world divergence explicit and operationally enforceable.

2. FUNDAMENTAL SGTX REPOSITIONING

SGTX is no longer architected merely as a trade execution workflow.

SGTX becomes:

A sovereign, non-custodial, AI-governed trade execution, state, settlement orchestration, reconciliation, exception, recovery and evidence infrastructure.

The external-facing category is:

Primary Category

Cross-Border Trade Execution & Reconciliation Infrastructure

Functional Category

Commercial-to-Settlement Execution Layer

Technical Category

Multi-Clock, Event-Sourced Transaction Infrastructure

Strategic Category

Neutral execution and coordination infrastructure between commercial systems and regulated institutions

The core proposition is:

SGTX does not attempt to make international trade artificially linear. It maintains a governed, cryptographically verifiable execution state while continuously reconciling commercial, financial, legal, documentary, logistics, compliance and regulatory realities as they change.

SGTX therefore does not compete with the primary functions of:

- marketplaces;
- ERPs;
- banks;
- logistics providers;
- customs systems;
- regulatory systems;
- document issuers;
- courts;
- payment rails.

Instead, SGTX provides a coordination, execution-state, reconciliation and evidence layer between them.

3. FUNDAMENTAL SGTX CONSTITUTIONAL PRINCIPLE

The following is a non-negotiable architectural principle:

SGTX does not require the real world to agree. SGTX makes disagreement explicit, attributable, evidence-backed, reconcilable and governable.

Different systems may legitimately report different realities at the same time.

Example:

text

text

Execution = SETTLED

Financial = RETURNED

Legal = DISPUTED

Physical = DELIVERED

Documents = COMPLETE

Compliance = CLEARED

Reconciliation = OPEN

Exposure = REOPENED

Closure = SUSPENDED

svgs

This is not automatically an invalid transaction.

It is a multidimensional representation of real-world divergence.

SGTX must represent that divergence rather than force every dimension into one artificial universal status.

4. CORE SGTX DESIGN PHILOSOPHY

The architecture is governed by the following principles:

- 1 Trade is not inherently linear.
- 2 Settlement is not necessarily closure.
- 3 Finality is multidimensional.
- 4 Historical events are immutable.
- 5 Current state is derived from governed events and authoritative external inputs.
- 6 Reversals create new states; they do not erase history.
- 7 Reconciliation is an execution subsystem, not a reporting feature.
- 8 Recovery replaces rollback.
- 9 Unknown is a valid state.
- 10 Authority is domain-specific.
- 11 Assertions are not automatically confirmations.
- 12 Evidence integrity is distinct from legal authority.
- 13 AI never possesses independent authoritative decision rights.
- 14 Deterministic policy remains authoritative over AI recommendation.
- 15 No timeout may silently create closure.
- 16 Post-closure events remain addressable.
- 17 Financial exposure is distinct from transaction status.
- 18 No subsystem may silently redefine transaction truth.
- 19 Non-custody is an architectural property and does not by itself determine regulatory classification.
- 20 SGTX operates within the legal and regulatory permissions applicable to its actual functionality in each jurisdiction.

5. CANONICAL SGTX TRANSACTION LIFECYCLE

The former model:

text

text

REQUEST

?

MATCH

?

CONTRACT

?

EXECUTE

?

SETTLE

?

COMPLETE

svgsvg

is insufficient.

The canonical lifecycle becomes:

text

text

INTENT

?

AGREEMENT

?

OBLIGATION CREATION

?

PRE-EXECUTION VALIDATION

?

EXECUTION

?

VERIFICATION

?

PROVISIONAL SETTLEMENT

?

FINANCIAL CONFIRMATION

?

RECONCILIATION

?

FINALIZATION

?

POST-CLOSURE OBSERVATION

?

ADMINISTRATIVE CLOSURE

svgsvg

At almost every stage, controlled side paths may occur:

text

text

DISPUTE

REVERSAL

CORRECTION

LEGAL REVIEW

COMPLIANCE HOLD

REGULATORY HOLD

CUSTOMS HOLD

RECOVERY

COMPENSATING EXECUTION

RECONCILIATION

POST-CLOSURE EVENT

svgsVG

Therefore:

Settlement is not necessarily the end of the transaction lifecycle.

6. MULTI-CLOCK TRANSACTION REALITY

SGTX must maintain multiple independently meaningful transaction clocks.

6.1 Execution Clock

Represents:

What SGTx has verified and recorded through its governed execution infrastructure.

Examples:

- obligation created;
- document verified;
- milestone verified;
- settlement instruction accepted;
- execution event recorded.

6.2 Financial / Settlement Clock

Represents:

What the relevant bank, payment institution or payment rail has confirmed regarding the financial processing and settlement of funds.

Examples:

- initiated;
- authorized;
- submitted;
- processing;
- accepted;

- settled;
- partially settled;
- returned;
- rejected;
- recalled;
- reversed;
- unknown.

6.3 Legal / Contractual Authority Clock

Represents:

What the applicable contract, court, regulator or legally authoritative framework recognizes as the legal or contractual status of the transaction.

Examples:

- effective;
- suspended;
- disputed;
- legally reviewed;
- legally reversed;
- termination effective;
- authority order received.

6.4 Physical / Operational Clock

Represents:

What is occurring in the physical and operational execution environment.

Examples:

- production;
- loading;
- inspection;
- warehouse receipt;
- in transit;
- customs inspection;
- customs clearance;
- port release;
- delivery;
- acceptance.

6.5 Extended Domain Clocks

Where materially required, SGTX may maintain additional state dimensions such as:

- Documentary Clock;

- Compliance Clock;
- Regulatory Clock;
- Counterparty Clock;
- Reconciliation Clock;
- Exposure Clock.

The architecture therefore operates as:

Multi-clock, multidimensional transaction reality.

7. STATE VECTOR - NOT SINGLE STATUS

SGTX must not depend on:

text

text

transaction.status

svgs

as the complete representation of transaction reality.

A transaction instead has a governed state vector.

Example:

text

text

Execution = SETTLED

Financial = CONFIRMED

Legal = EFFECTIVE

Physical = DELIVERED

Documents = COMPLETE

Compliance = CLEARED

Reconciliation = MATCHED

Exposure = RESOLVED

Closure = CLOSED

svgs

Another valid state:

text

text

Execution = SETTLED

Financial = RETURNED

Legal = DISPUTED

Physical = DELIVERED

Documents = COMPLETE

Compliance = CLEARED

Reconciliation = OPEN

Exposure = REOPENED

Closure = SUSPENDED

svgs

The state vector is the canonical operational representation.

8. STATE VECTOR DOMAINS

The canonical state vector should support, as applicable:

text

text

EXECUTION

FINANCIAL

LEGAL

PHYSICAL_OPERATIONAL

DOCUMENTARY

COMPLIANCE

REGULATORY

COUNTERPARTY

RECONCILIATION

DISPUTE

EXPOSURE

CLOSURE

svgs

Not every transaction requires every dimension to be active.

A transaction profile determines which dimensions are required.

9. FINALITY CLASSES

SGTX must not use one universal concept of "final."

The canonical finality model is:

F0 - Intent Finality

Parties have expressed a qualified transaction intent.

F1 - Contractual Finality

The relevant contractual agreement has become effective according to its governing conditions.

F2 - Execution Finality

SGTX has cryptographically recorded the defined execution event according to applicable policy.

F3 - Financial Settlement Finality

The applicable financial/banking infrastructure has confirmed settlement according to its own rules.

F4 - Legal Finality

The applicable legal or contractual framework recognizes the relevant matter as legally final for the defined jurisdiction and issue, subject to any remaining legally available remedies.

F5 - Operational / Administrative Closure

Required reconciliation, documentary, financial, compliance, operational and closure conditions have been satisfied according to closure policy.

9.1 Finality Is Not a Mandatory Staircase

F0-F5 are finality classes/dimensions, not necessarily a single sequential staircase.

Example:

text

text

Execution = F2

Financial = F3

Legal = F1

Physical = IN_TRANSIT

Documents = COMPLETE

Closure = OPEN

svgs

This is valid.

10. SETTLEMENT IS NOT CLOSURE

SGTX must explicitly distinguish:

Settlement

from:

Closure

Financial settlement confirmation does not automatically close a transaction.

A transaction may remain open because of:

- unresolved documents;
- customs issues;
- legal disputes;
- late evidence;
- counterparty disputes;
- financial reconciliation;
- logistics obligations;
- contractual post-settlement obligations;

- regulatory requirements;
- outstanding exposure.

11. CLOSURE AS A POLICY-DERIVED CONDITION

Closure must never be treated as an arbitrary field that a developer can assign.

The architecture must contain a:

Closure Policy Engine

Conceptually:

text

text

Required Conditions

?

Closure Policy

?

State Vector Evaluation

?

All Required Conditions Satisfied?

/ \

YES NO

? ?

CLOSED OPEN / SUSPENDED /

CLOSED_WITH_EXCEPTION

svgsvg

Closure must be derived from governed requirements.

No manual or automated process may bypass closure policy.

12. IMMUTABLE CANONICAL EVENT SPINE

SGTX requires a canonical append-only event history.

Every material event must be represented as a new event.

Examples:

text

text

TRADE_CREATED

COUNTERPARTY_VERIFIED

CONTRACT_SIGNED

OBLIGATION_CREATED

DOCUMENT_ACCEPTED

DOCUMENT_REVOKED
MILESTONE_REACHED
MILESTONE_DISPUTED
PAYMENT_INSTRUCTION_CREATED
PAYMENT_ACCEPTED
PAYMENT_SETTLED
PAYMENT_PARTIALLY_SETTLED
PAYMENT_RETURN_REQUESTED
PAYMENT_RETURNED
PAYMENT_RECALLED
PAYMENT_REVERSED
COMPLIANCE_HOLD
CUSTOMS_HOLD
LEGAL_ORDER_RECEIVED
RECONCILIATION_OPENED
RECONCILIATION_RESOLVED
CORRECTIVE_EXECUTION
COMPENSATING_EXECUTION
POST_CLOSURE_EVENT
TRADE_CLOSED

svgsVG

No destructive state rewriting is permitted as the authoritative historical mechanism.

13. HISTORICAL TRUTH VS CURRENT STATE

SGTX must explicitly separate:

Historical Record

What events occurred and were recorded.

Operational State

The current derived execution state.

Financial Authority State

What the relevant financial infrastructure confirms.

Legal Authority State

What the applicable legal authority recognizes.

Economic Exposure

What financial exposure or economic position currently exists.

Counterparty Assertion

What a counterparty claims or asserts occurred.

Evidence State

What supporting evidence exists and its integrity/authority characteristics.

These dimensions must not be conflated.

14. EVENT SOURCING PRINCIPLE

The canonical model is:

text

text

EVENT SPINE

?

STATE PROJECTION

?

CURRENT OPERATIONAL VIEW

svgsvg

The event history is the historical record.

State projections are derived views.

If a projection is corrupted, it must be reconstructable from the governed event history.

Neither PostgreSQL projections, Temporal workflow state, nor ClickHouse analytics may silently become the historical authoritative record.

15. EVENT CAUSALITY

Every material event should contain or be linked to, as applicable:

text

text

event_id

transaction_id

USTN

parent_event_id

event_type

event_time

observation_time

effective_time

source_system

source_event_id

source_reference

authority

evidence_reference

previous_event_hash

event_hash

policy_version

actor

authorization_context

idempotency_key

svgsvg

Implementation may vary, but the underlying information model is mandatory.

16. THREE TIMESTAMPS

Every important externally sourced event must distinguish:

Event Time

When the event actually occurred.

Observation Time

When SGTX learned or received the event.

Effective Time

When the event became legally/operationally effective.

Example:

text

text

Payment Reversal

Event Time = Aug 10 14:02

Observation Time = Aug 11 03:21

Effective Time = Aug 10 14:02

svgsvg

These timestamps must not be collapsed merely for convenience.

17. EVENT TIME VS INGESTION ORDER

SGTX must assume that external events may arrive:

- late;
- duplicated;
- out of sequence;
- after temporary connectivity loss;
- after another related event.

Therefore:

Ingestion order is not event order.

SGTX must preserve both.

18. IDEMPOTENCY

External event processing must support:

text

text

source_system

source_event_id

event_hash

idempotency_key

received_at

svgsvg

The same external event must not create duplicate authoritative state transitions.

Duplicate delivery must be safely recognized and handled.

19. REVERSALS ARE NEW STATES - NEVER UNDO OPERATIONS

Forbidden as authoritative historical behavior:

text

text

SETTLED ? NOT_SETTLED

svgsvg

Canonical examples:

text

text

SETTLED

?

RETURN_REQUESTED

?

RETURN_CONFIRMED

svgsvg

or:

text

text

SETTLED

?

DISPUTED

?

LEGAL_REVIEW

?

LEGALLY_REVERSED

svgsvg

or:

text

text

SETTLED

?

BANK_CORRECTION

?

RECONCILIATION_REQUIRED

?

CORRECTED

svgsvg

The original settlement remains permanently recorded.

20. RECOVERY, NOT ROLLBACK

SGTX adopts:

Compensating execution rather than historical rollback.

A real-world event cannot necessarily be undone.

Examples include:

- delivered cargo;
- submitted customs filings;
- executed bank transactions;
- regulatory submissions;
- physical inspections;
- signed documents;
- counterparty actions.

Therefore recovery consists of new, governed actions.

21. CONTROLLED POST-CLOSURE MUTATION

The architecture must distinguish:

Historical immutability

Historical events never change.

Current-state mutability

A valid new event may change the current derived operational state.

Example:

text

text

CLOSED

?

POST_CLOSURE_EVENT

?

IMPACT_ASSESSMENT

?

REOPEN_RECONCILIATION

?

NEW_DERIVED_STATE

svgsvg

This is:

Immutable history + mutable derived state.

22. POST-CLOSURE OBSERVATION PERIODS (ARCHITECTURAL DEFINITION)

Different transaction classes may have different post-closure observation periods.

The post-closure observation period is determined by the applicable contract, transaction class, jurisdiction, regulatory requirement, document lifecycle, financial process and institutional policy.

No universal duration is constitutionally prescribed.

22.1 Jurisdiction-Specific Post-Closure Periods

The architecture must support the definition of post-closure periods within jurisdiction capability profiles.

Example profile structure (not constitutional values):

text

text

JURISDICTION: EGYPT

??? Post-Closure Periods

? ??? Agricultural Export: Configurable (Contract/Jurisdiction)

? ??? Industrial Export: Configurable (Contract/Jurisdiction)

? ??? Services Trade: Configurable (Contract/Jurisdiction)

? ??? Financial Settlement: Configurable (Regulatory/Bank Policy)

? ??? Customs Declaration: Configurable (Customs Authority)

? ??? Regulatory Filing: Configurable (Regulatory Authority)

? ??? Tax-Related Trade: Configurable (Tax Authority / Legal Requirement)

??? Retention Policy: Derived from Jurisdictional Legal Framework

22.2 Post-Closure Period Configuration

The period is configurable by:

- Contract terms
- Jurisdictional requirements
- Transaction class
- Customer agreement
- Regulatory mandate
- Applicable legal retention and limitation periods

22.3 Post-Closure Event Processing

Post-closure events within the observation period trigger:

- 1 Impact assessment
- 2 Reconciliation review
- 3 State re-evaluation
- 4 Potential adjustment
- 5 Recovery if required

The active period and applicable policy must be documented in the transaction twin and jurisdictional profile.

23. UNKNOWN MUST BE A VALID STATE

SGTX must be able to represent:

text

text

UNKNOWN

svgsVG

and:

text

text

UNVERIFIED

svgsVG

where authoritative evidence is insufficient.

A bank timeout does not automatically mean:

text

text

FAILED

svgsVG

and does not mean:

text

text

SETTLED

svgsvg

It may mean:

text

text

FINANCIAL = UNKNOWN

RECONCILIATION = REQUIRED

svgsvg

This is a constitutional safety principle.

24. NO FORCED CLOSURE

Incorrect:

text

text

TIMEOUT ? CLOSED

svgsvg

Correct:

text

text

TIMEOUT

?

EXCEPTION

?

RECONCILIATION

?

RESOLUTION

?

CONTROLLED CLOSURE

svgsvg

Timeout is an event.

It is not automatically a final transaction state.

25. CANONICAL EXCEPTION TAXONOMY

The platform must support, as applicable:

Financial

text

text

PAYMENT_PENDING

PAYMENT_PARTIALLY_SETTLED
PAYMENT_REJECTED
PAYMENT_RETURN_REQUESTED
PAYMENT_RETURNED
PAYMENT_RECALLED
PAYMENT_REVERSED
PAYMENT_UNKNOWN

svgsvg

Documents

text

text

DOCUMENT_MISSING
DOCUMENT_INVALID
DOCUMENT_REVOKED
DOCUMENT_EXPIRED
DOCUMENT_CONFLICT
DOCUMENT_SUPERSEDED

svgsvg

Milestones

text

text

MILESTONE_PENDING
MILESTONE_FAILED
MILESTONE_DISPUTED
MILESTONE_CORRECTED

svgsvg

Compliance / Regulatory

text

text

COMPLIANCE_HOLD
SANCTIONS_HOLD
REGULATORY_HOLD
CUSTOMS_HOLD
LEGAL_HOLD

svgsvg

Reconciliation

text

text

RECONCILIATION_REQUIRED

RECONCILIATION_DIVERGENT

RECONCILIATION_ESCALATED

RECONCILIATION_RESOLVED

svgsvg

Execution / Recovery

text

text

EXECUTION_SUSPENDED

CORRECTIVE_EXECUTION

COMPENSATING_EXECUTION

svgsvg

Closure

text

text

CLOSED

CLOSED_WITH_EXCEPTION

svgsvg

Developers must not invent uncontrolled authoritative states outside governed state policy.

26. RECONCILIATION CONTROL PLANE

Reconciliation is a core execution subsystem.

It is not a reporting feature.

It compares:

text

text

SGTX Execution State

vs

Financial / Bank State

vs

Contract State

vs

Document State

vs

Physical / Logistics State

vs

Compliance State

vs

Legal State

vs

Counterparty Assertions

vs

Economic Exposure

svgsvg

Possible outcomes include:

text

text

MATCHED

PARTIALLY_MATCHED

DIVERGENT

UNRESOLVED

EXCEPTION

svgsvg

27. RECONCILIATION QUESTIONS

SGTX should be capable of answering:

- 1 What did SGTX record?
- 2 What did the bank confirm?
- 3 What did the counterparty assert?
- 4 What evidence exists?
- 5 Which source is authoritative for each dimension?
- 6 When did each event occur?
- 7 When did each system learn about it?
- 8 What policy was applicable?
- 9 What is the current state vector?
- 10 What is the current economic exposure?
- 11 What remains unresolved?
- 12 What corrective/recovery paths are permitted?

This is a core institutional capability.

28. STATE CONFLICT PROTOCOL

When:

text

text

SOURCE A = SETTLED

SOURCE B = RETURNED

SOURCE C = DISPUTED

svgsVG

SGTX must not simply select the latest event.

Canonical protocol:

text

text

CONFLICT DETECTED

?

AFFECTED TRANSITION CONTROLLED

?

AUTHORITIES IDENTIFIED

?

EVIDENCE COLLECTED

?

TEMPORAL RELATIONSHIP ANALYZED

?

POLICY APPLIED

?

RESOLUTION PATH DETERMINED

?

AUTHORIZED / GOVERNED RESOLUTION

?

NEW AUTHORITATIVE EVENT CREATED

svgsVG

29. AUTHORITY MATRIX

Each state domain must identify its relevant authority.

DomainPrimary Authority

SGTX execution record SGTx canonical execution system

Bank settlement Relevant bank/payment infrastructure

Customs status Applicable customs authority

Tax determination Applicable tax authority

Document issuance Authorized issuer

Contractual status Applicable contract/governance

Legal determination Competent legal authority

Physical logistics event Authorized logistics/carrier evidence

Compliance determination Applicable authorized institution

Corporate accounting Relevant enterprise accounting system unless otherwise contractually defined

SGTX must not assume global authority over all dimensions.

30. AUTHORITY IS DOMAIN-SPECIFIC

There is no universal:

"SGTX always wins."

Instead:

The authoritative source depends on the specific state dimension and applicable governance framework.

This prevents:

- legal overreach;
- banking overreach;
- regulatory overreach;
- accounting overreach;
- customs overreach.

31. ASSERTION VS CONFIRMATION

SGTX must explicitly distinguish:

Assertion

A person or system claims that something happened.

Example:

Seller says payment was received.

Confirmation

An authoritative system confirms that something happened.

Example:

Bank confirms settlement.

An assertion is not automatically a confirmation.

Both may be recorded, but they have different evidentiary/authority characteristics.

32. EVIDENCE INTEGRITY VS AUTHORITY

SGTX must independently represent:

Evidence Integrity

Is the evidence:

- authentic;
- intact;
- attributable;
- temporally consistent;
- cryptographically verifiable where applicable?

Authority

Does the source have authority to determine the relevant matter?

A highly authentic cryptographic record does not automatically make its creator legally authoritative.

These dimensions must never be conflated.

33. TRUTH TRIANGULATION ENGINE

For high-value or high-risk transactions, SGTX may compare independent sources.

Example:

text

text

Bank confirmation

+

SGTX execution record

+

Contractual obligation

+

Counterparty evidence

svgs

Truth triangulation must not operate as majority voting.

It uses:

- evidence convergence;
- authority weighting;
- temporal consistency;
- provenance;
- policy;
- source reliability;
- conflict detection.

34. BANK REALITY ADAPTER LAYER

SGTX must include formal bank/payment-rail reality adapters.

Conceptually:

text

text

BANK-NATIVE STATE

?

BANK REALITY ADAPTER

?

CANONICAL SGTX FINANCIAL STATE

svgsvg

Adapters may handle:

- bank APIs;
- webhooks;
- ISO 20022;
- host-to-host;
- SFTP;
- bank-specific messages;
- bank identifiers;
- return codes;
- timeouts;
- partial responses;
- duplicate notifications;
- out-of-order events;
- bank-specific status semantics.

The original bank-native status must remain available for provenance.

35. CANONICAL FINANCIAL MODEL VS BANK-NATIVE MODEL

A bank's terminology must not become SGTX's universal internal semantics.

Example:

text

text

Bank Native State

?

Adapter

?

SGTX Canonical Financial State

svgsvg

Both must be preserved.

SGTX's canonical model is stable.

Bank-specific models remain adapter-specific.

36. ISO 20022-FIRST BANK INTEGRATION

ISO 20022 becomes a preferred banking integration standard for supported workflows.

However:

ISO 20022 is an external banking messaging standard, not SGTX's internal canonical state model.

The architecture is:

text

text

SGTX Canonical Financial Model

?

ISO 20022 Adapter

?

Bank

svgsvg

and may also support:

text

text

SGTX Canonical Financial Model

?

API Adapter

?

Bank

svgsvg

or:

text

text

SGTX Canonical Financial Model

?

H2H / SFTP Adapter

?

Bank

svgsvg

This allows jurisdiction and bank-specific integration.

As of June 21, 2026, the CBE states that banks operating in Egypt adopted ISO 20022 for SWIFT messaging in interbank financial transfers, making an ISO 20022-first Egyptian integration strategy

particularly appropriate.

37. SGTX BANK SETTLEMENT GATEWAY

Introduce formally:

SGTX Bank Settlement Gateway - BSG

The BSG is the controlled integration boundary between SGTX and participating financial institutions.

It may perform:

- instruction translation;
- schema validation;
- cryptographic/signature validation;
- USTN correlation;
- settlement instruction validation;
- beneficiary consistency validation;
- bank-side policy handoff;
- payment status ingestion;
- event correlation;
- exception handling;
- reconciliation;
- evidence capture.

The BSG does not custody customer funds.

38. BANK AUTHORITY BOUNDARY

The architecture remains:

text

text

SGTX

?

Settlement Instruction

?

Bank Settlement Gateway

?

Bank Validation

?

Bank Authorization

?

Bank Execution

?

Bank Confirmation

?

SGTX Financial State

svgs

The bank remains authoritative for its:

- funds movement;
- bank ledger;
- account status;
- authorization;
- bank-side compliance;
- payment execution;
- bank settlement confirmation.

39. NON-CUSTODIAL CONTROL-PLANE PRINCIPLE

The constitutional principle is:

SGTX provides control-plane orchestration without becoming the asset custodian.

SGTX can:

- orchestrate;
- coordinate;
- verify;
- observe;
- reconcile;
- notify;
- generate evidence;
- generate authenticated instructions;
- initiate authorized workflows.

SGTX does not automatically:

- hold pooled customer funds;
- rewrite bank ledgers;
- independently reverse external payments;
- override bank authority;
- issue legal determinations;
- act as a court;
- act as a central bank;
- become the underlying payment rail.

40. REGULATORY CLASSIFICATION PRINCIPLE

Non-custody does not itself determine regulatory status.

SGTX's regulatory treatment in any jurisdiction depends on actual:

- functionality;
- technical control;
- customer interaction;
- contractual structure;
- payment involvement;
- funds movement role;
- authorization role;
- data access;
- regulatory perimeter;
- jurisdiction.

In Egypt, the CBE's rules cover regulated payment activities including execution of payment transactions, transfers, payment initiation services and other payment services.

Therefore:

SGTX must never claim that a functionality is unregulated merely because it is non-custodial or described as "instruction-only."

Actual legal/regulatory classification must be assessed before enabling the relevant functionality in each jurisdiction.

41. REGULATORY CLASSIFICATION GATE

Before enabling a regulated or potentially regulated functionality in a jurisdiction:

text

text

SGTX Functionality

?

Regulatory Classification Assessment

?

License / Partnership / Exemption / Restriction

?

Approved Operating Model

?

Feature Enablement

svgs

No jurisdiction-specific financial functionality should be enabled solely because the software supports it.

42. JURISDICTION CAPABILITY PROFILE

Each jurisdiction should have a structured capability profile:

text

text

JURISDICTION

??? Allowed Capabilities

??? Restricted Capabilities

??? Required Licenses

??? Required Partners

??? Required Authorities

??? Payment Rails

??? Banking Standards

??? Document Standards

??? Data Restrictions

??? Tax Rules

??? Customs Rules

??? Regulatory Rules

??? Evidence Requirements

??? Authority Profiles

??? Required Approvals

svgsvg

This becomes part of the country adapter system.

43. BANK-SIDE AUTHORIZATION / MAKER-CHECKER

SGTX must never assume that a signed SGTX instruction automatically authorizes movement of funds.

The bank may require:

text

text

SGTX Instruction

?

Bank Validation

?

Maker Approval

?

Checker Approval

?

Bank Execution

svgsvg

The exact authorization model remains bank-specific.

SGTX records relevant authorization evidence but does not replace the bank's internal authorization controls.

44. BENEFICIARY VERIFICATION

Each settlement leg should contain, as applicable:

text

text

Beneficiary ID

Beneficiary Legal Name

Beneficiary Type

Account Identifier / IBAN

Bank Identifier

Jurisdiction

Currency

Amount

Payment Purpose

Underlying Obligation ID

Supporting Documents

Sanctions Status

KYC/KYB Status

Beneficiary Verification State

Authorization Context

svgsvg

The bank remains authoritative for bank-side account ownership/beneficiary controls.

45. PAYMENT LEG MODEL

A multi-party settlement must be broken into independently identifiable payment legs.

Hierarchy:

text

text

USTN

?

Settlement Instruction ID

?

Settlement Leg ID

?

Bank Transaction Reference

svgsvg

Example:

text

text

Parent Settlement

?

??? Leg 1 = Seller = SETTLED

??? Leg 2 = Logistics = SETTLED

??? Leg 3 = Customs = REJECTED

??? Leg 4 = Laboratory = PENDING

??? Leg 5 = Broker = UNKNOWN

svgsvg

Parent state:

text

text

Settlement = PARTIALLY_SETTLED

svgsvg

46. MULTI-PARTY SETTLEMENT OBLIGATION MODEL

A trade may contain multiple financial obligations:

text

text

USTN

?

??? Seller Obligation

?

??? Logistics Obligation

?

??? Customs Obligation

?

??? Laboratory Obligation

?

??? Broker Obligation

?

??? SGTX Service Fee Obligation

svgsvg

Each obligation can contain:

- obligation ID;
- beneficiary;
- amount;
- currency;
- authorization;
- contractual basis;
- document/evidence basis;
- dependency;
- settlement condition;
- reversal condition;
- dispute condition;
- recovery path;
- reconciliation state.

47. SETTLEMENT ATOMICITY POLICY

Each multi-leg settlement must have an applicable Settlement Atomicity Policy.

Supported policies may include:

ALL_OR_NONE

All eligible legs must satisfy required conditions before execution completes.

PARTIAL_ALLOWED

Eligible legs may execute independently.

SEQUENCED

A subsequent leg depends on an earlier leg.

CONDITIONAL

Execution depends on defined milestones/evidence/conditions.

HUMAN_RELEASE

One or more stages require authorized human/institutional release.

The atomicity policy does not give SGTX power to force bank execution.

It defines the transaction's permitted orchestration and recovery behavior.

48. PAYMENT DEPENDENCY GRAPH

Example:

text

text

SELLER PAYMENT

?

DOCUMENT RELEASE

?

LOGISTICS PAYMENT

?

DELIVERY

?

FINAL PAYMENT

svgsvg

The actual dependency structure is transaction-specific.

The graph becomes part of the Obligation Graph and Settlement Orchestration model.

49. SETTLEMENT INSTRUCTION LIFECYCLE

The settlement instruction is distinct from the overall trade lifecycle.

Canonical lifecycle:

text

text

DRAFT

?

VALIDATED

?

AUTHORIZED

?

SIGNED

?

SUBMITTED_TO_BANK

?

BANK_ACCEPTED

?

PROCESSING

?

PARTIALLY_EXECUTED

?

FULLY_EXECUTED

svgsvg

Controlled side paths:

text

text

REJECTED

EXPIRED

CANCEL_REQUESTED

RETURNED

RECALLED

DISPUTED

RECONCILIATION_REQUIRED

UNKNOWN

svgsvg

50. SETTLEMENT INSTRUCTION VERSIONING

Instructions are immutable versions.

Example:

text

text

SI-001 v1

?

SUPERSEDED

?

SI-001 v2

svgsvg

No prior instruction version is deleted.

Supersession is a new event.

51. PAYMENT INSTRUCTION EXPIRATION

Each instruction may contain:

- issue time;
- valid-from;
- expiration;
- requested execution date;
- value date;
- cancellation rules;
- authorization validity.

Expired instructions must not silently execute.

52. FUNDS SUFFICIENCY SNAPSHOT

Where supported, SGTX may request a bank-side funds/availability assessment.

Possible bank response:

text

text

SUFFICIENT

INSUFFICIENT

RESERVED

UNKNOWN

svgsvg

SGTX must not independently represent bank account balances as authoritative.

The bank remains authoritative.

53. PAYMENT LEG CERTIFICATE

Each completed financial leg should be capable of producing:

text

text

USTN

Settlement Instruction ID

Settlement Leg ID

Bank Transaction Reference

Amount

Currency

Beneficiary

Bank Status

Value Date

Execution Timestamp

Return Code, if applicable

Bank Evidence Reference

SGTX Event Hash

Policy Version

Reconciliation Status

svgsvg

This becomes an input to the Reconciliation Control Plane.

54. MULTI-PARTY SETTLEMENT RECONCILIATION

SGTX must support:

Leg Reconciliation

text

text

Instruction Leg

vs

Bank Transaction

vs

Beneficiary Evidence

svgsvg

Parent Settlement Reconciliation

text

text

All Legs

vs

Total Contractual Obligation

svgsvg

Commercial Reconciliation

text

text

Contract

vs

Invoices

vs

Payment Legs

svgsvg

Financial Reconciliation

text

text

Bank Confirmations

vs

SGTX Financial State

svgsvg

55. SETTLEMENT COMPLETION MATRIX

Example:

Leg Required Bank SGTX Evidence Reconciliation

Seller Yes Settled Settled Verified Matched

Logistics Yes Settled Settled Verified Matched

Customs Yes Rejected Rejected Verified Matched

Laboratory No Pending Pending - Open

Parent transaction:

text

text

PARTIALLY_SETTLED

EXCEPTION_OPEN

RECONCILIATION_OPEN

svgsvg

56. BANK REFERENCE MAPPING

Bank systems may use their own identifiers.

SGTX maintains:

text

text

USTN

?

SGTX Settlement Instruction ID

?

Settlement Leg ID

?

Bank Instruction ID

?

Bank Transaction Reference

?

External Payment Reference

svgsvg

This ensures resilient cross-system correlation even if external systems transform, truncate or replace references.

57. EXTERNAL IDENTIFIER REGISTRY

USTN is not the replacement for external identifiers.

SGTX must maintain an:

External Identifier Registry

Each identifier should support:

text

text

Identifier Type

Identifier Value

Issuing Authority

Issuer System

USTN

Related Entity

Related Event

Validity

Lifecycle Status

Source

Evidence

svgs

Examples may include:

- bank reference;
- Nafeza ACID where applicable;
- Nafeza UCR where applicable;
- CargoX reference;
- customs reference;
- ERP reference;
- marketplace reference;
- invoice number;
- bill of lading number;
- contract ID;
- document ID.

58. USTN - UNIVERSAL STATE & TRANSACTION NAMESPACE

USTN is upgraded from a simple transaction reference into:

The persistent cross-system namespace and correlation identity for a trade and its connected states, obligations, instructions, documents, logistics, regulatory references, financial events, evidence and recovery events.

USTN is not:

"the single source of truth for every external system."

It instead provides:

canonical SGTX correlation and state lineage across systems whose respective authorities remain external.

59. NAFEZA / CUSTOMS CORRELATION MODEL

SGTX must not replace jurisdiction-specific customs identifiers.

The architecture should instead correlate:

text

text

USTN

?

??? Nafeza ACID, where applicable

?

??? Nafeza UCR, where applicable

?

??? CargoX Reference, where applicable

?

??? Customs Declaration Reference

?

??? Customs Event References

svgsVG

Nafeza's current Enhanced Export System describes UCR as a unified shipment identifier used to track the shipment through completion, making external-identifier correlation an important SGTX integration pattern.

Existing Nafeza materials also require ACID to appear on relevant shipment documents in applicable ACI workflows.

SGTX should therefore act as the cross-system correlation layer rather than attempting to create a competing government identifier.

60. NAFEZA EVIDENCE GRAPH

Where legally and technically permitted:

text

text

USTN

?

Nafeza Identifier

?

Customs Event

?

Document Set

?

Obligation

?

Settlement Leg

?

Financial State

svgsvg

This connects customs reality to the trade's broader state graph.

61. GOVERNMENT PAYMENT LIMITATION

SGTX must not assume that every government fee can be directly executed as an ordinary multi-party payment leg.

Each beneficiary/authority type should be classified as:

text

text

DIRECT_SETTLEMENT_ELIGIBLE

BANK_MEDIATED

AUTHORITY_SPECIFIC_WORKFLOW

PAYMENT_PROOF_ONLY

EXTERNAL_RECONCILIATION_ONLY

svgsvg

The actual permitted method is jurisdiction-specific.

62. BANK-SIDE RISK GATEWAY

Before a settlement instruction reaches bank execution:

text

text

SGTX Bank Settlement Gateway

?

Schema Validation

?

Signature Validation

?

USTN Validation

?

Instruction Version Validation

?

Beneficiary Consistency

?

Bank Policy

?

AML / Sanctions / Fraud Controls

?

Bank Authorization

?

Payment Execution

svgsvg

SGTX does not replace bank-side compliance controls.

63. FINANCIAL POSITION & CONSEQUENCE SUBLEDGER

SGTX maintains a transaction-level financial consequence model.

It is not intended to replace the customer's corporate accounting/ERP general ledger.

Track, as applicable:

- gross commercial value;
- expected settlement;
- actual settlement;
- returned amount;
- disputed amount;
- fees;
- adjustments;
- FX consequences;
- penalties;
- compensation;
- recoverable amount;
- outstanding exposure;
- reopened exposure;
- contingent exposure.

64. FINANCIAL EXPOSURE ENGINE

Example:

text

text

Trade Value = \$100,000

Settlement = CONFIRMED

Exposure = \$0

svgsvg

After financial reversal:

text

text

Settlement = REVERSED

Exposure = \$100,000

Recovery Status = OPEN

svgsvg

SGTX therefore identifies the current economic exposure without pretending to be the enterprise accounting ledger.

65. ECONOMIC POSITION IS DISTINCT FROM TRANSACTION STATE

A transaction can be:

text

text

Execution = SETTLED

svgsvg

while:

text

text

Financial = REVERSED

Exposure = REOPENED

svgsvg

These states are not contradictory.

They represent different dimensions.

66. OBLIGATION GRAPH

A trade must be represented as a graph of obligations rather than merely a list of tasks.

Example:

text

text

Commercial Agreement

?

????????????????????

???

Payment Documents Logistics

???

Bank Customs Carrier

???

Settlement Clearance Delivery

svgsvg

Each obligation contains, as applicable:

- obligation ID;

- prerequisite;
- dependency;
- authority;
- evidence requirement;
- deadline;
- completion condition;
- reversal condition;
- dispute condition;
- recovery path;
- financial consequence.

67. DEPENDENCY GRAPH

SGTX must understand dependencies.

Example:

text

text

DOCUMENT REVOKED

?

CUSTOMS OBLIGATION AFFECTED

?

CLEARANCE BLOCKED

?

DELIVERY MILESTONE AT RISK

?

PAYMENT CONDITION AFFECTED

?

FINANCIAL EXPOSURE CHANGES

svgsVG

68. RECOVERY GRAPH

Every important transaction class must have governed recovery pathways.

Example:

text

text

DOCUMENT REVOKED

?

??? Replace Document

??? Suspend Milestone

??? Notify Authority

??? Reopen Compliance

??? Correct Obligation

??? Terminate / Alternative Contract Path

svgsvg

Recovery must be policy-driven and authority-controlled.

69. CAUSAL IMPACT ENGINE

When an event occurs, SGTX evaluates:

What else does this event affect?

Example:

text

text

PAYMENT REVERSED

?

Settlement Condition Affected

?

Financial Exposure Reopened

?

Fee Realization Affected

?

Accounting Consequence

?

Contract Condition Review

?

Counterparty Notification

?

Recovery Path

svgsvg

This creates causal intelligence.

70. EXCEPTION ENGINE

Inputs:

text

text

Unexpected Event

+

Current State Vector

+

Policy

+

Authority

+

Evidence

+

Dependencies

svgsvg

Outputs:

text

text

CONTINUE

PAUSE

ESCALATE

RECONCILE

CORRECT

COMPENSATE

REQUEST AUTHORITY

CLOSE WITH EXCEPTION

svgsvg

The Exception Engine must remain deterministic/policy-governed for authoritative actions.

71. EXCEPTION CONTAINMENT

An exception should affect only the necessary transaction scope.

Example:

A document issue should not automatically freeze an unrelated payment unless:

- the obligation graph;
- settlement dependency;
- contract;
- policy;
- jurisdiction;
- or authority

requires the dependency.

This creates:

Localized Failure Containment

rather than:

One Failure Collapses the Entire Transaction

72. EXCEPTION SEVERITY

Canonical severity model:

Severity 1

Informational.

Severity 2

Operational review.

Severity 3

Financial reconciliation required.

Severity 4

Execution suspension.

Severity 5

Legal/regulatory escalation.

Severity thresholds must be configurable by transaction class and jurisdiction.

73. EXCEPTION SLA

Exceptions may have:

- detection deadline;
- acknowledgment deadline;
- resolution target;
- escalation deadline;
- authority escalation path.

These are operational policies, not immutable universal values.

74. DOCUMENT AUTHENTICITY & VERSIONING

Significant documents must support:

text

text

Document ID

Issuer

Version

Issue Time

Effective Time

Validity Period

Signature

Authenticity Evidence

Status

Revocation Status

Superseding Document

Source

Related USTN

svgsVG

Rules:

Document supersession is not document deletion.

Revocation is an event, not erasure.

75. DOCUMENT REVOCATION FLOW

Example:

text

text

DOCUMENT VALID

?

REVOCATION RECEIVED

?

AUTHENTICITY VERIFIED

?

REVOCATION RECORDED

?

OBLIGATION IMPACT ANALYSIS

?

DEPENDENT STATES REVIEWED

?

RECONCILIATION / RECOVERY

svgsVG

76. COUNTERPARTY REALITY & EVIDENCE LAYER

SGTX must explicitly represent what each relevant party claims occurred.

Track, as applicable:

- counterparty assertion;
- acknowledgment;
- consent;

- evidence;
- dispute;
- contractual obligation;
- response deadline;
- authority;
- timestamps;
- provenance.

This creates an explicit:

Counterparty Reality Model

without automatically treating counterparty assertions as truth.

77. CROSS-BORDER & JURISDICTION REALITY LAYER

SGTX must model:

- governing law;
- applicable jurisdictions;
- transaction location;
- asset location;
- bank location;
- counterparty location;
- customs jurisdiction;
- tax jurisdiction;
- regulatory jurisdiction;
- legal-recognition requirements;
- evidence requirements;
- electronic-document rules;
- authority mapping;
- cross-border constraints;
- conflict-of-law considerations.

78. NO UNIVERSAL LEGAL ORACLE

SGTX must not model "the legal system" as a simplistic external oracle.

Instead:

Legally authoritative determinations become authoritative external inputs into the applicable SGTx state model.

SGTX:

- records them;
- validates their provenance;
- associates them with the affected transaction;

- applies permitted operational consequences;
- preserves the historical event.

SGTX does not replace legal authority.

79. AI GOVERNANCE

Constitutional rule:

AI assists; AI never possesses independent authoritative decision rights.

AI may:

- classify;
- summarize;
- detect anomalies;
- detect contradictions;
- identify missing evidence;
- predict reconciliation problems;
- prioritize cases;
- recommend actions;
- assemble investigation packages;
- identify patterns;
- generate structured summaries.

AI may not independently:

- finalize material settlement;
- reverse a financial transaction;
- rewrite authoritative records;
- resolve a legal dispute;
- override a bank;
- override a regulator;
- delete evidence;
- bypass policy;
- change authority;
- convert a recommendation into an unauthorized material action.

80. AI RECOMMENDATION GATEWAY

Canonical flow:

text

text

AI Recommendation

?

Evidence Check

?

Deterministic Policy Check

?

Authority Check

?

Human / Institutional Approval if Required

?

Execution

svgs

Never:

text

text

AI

?

Independent Authority

?

Uncontrolled Authoritative Execution

svgs

Low-risk automation may execute automatically only where deterministic policy explicitly authorizes the action independently of AI authority.

The AI remains an input, not the governing authority.

81. RISK-TIERED HUMAN / INSTITUTIONAL INVOLVEMENT

Tier 0

Deterministic automation.

Tier 1

Automation + monitoring.

Tier 2

AI recommendation + deterministic policy validation.

Tier 3

Institutional approval required.

Tier 4

Human/legal/regulatory authority required.

The transaction type and action determine the tier.

82. USTN LINEAGE

Canonical lineage:

text

text

USTN

?

Parent Event

?

Child Event

?

Exception Event

?

Compensation Event

?

Resolution Event

svgsvg

Every material downstream event must remain traceable through USTN.

83. EVENT LINEAGE & CORRELATION

A material event may be linked to:

- trade;
- contract;
- obligation;
- settlement instruction;
- settlement leg;
- external bank transaction;
- document;
- logistics event;
- customs identifier;
- dispute;
- legal event;
- evidence;
- recovery event.

USTN remains the principal cross-domain namespace.

84. POLICY VERSIONING

Every material policy-driven decision must identify:

text

text

Policy Name

Policy Version

Decision

Evidence

Authority

Timestamp

Actor/System

svgsvg

Example:

text

text

SettlementPolicy v3.7

Decision = ALLOWED

Evidence = ...

Authority = ...

svgsvg

Historical policy decisions are never rewritten simply because a new policy later becomes active.

85. POLICY TIME TRAVEL

Authorized institutional users must be able to ask:

What policy was in force when this event occurred?

and separately:

What would the current policy say today?

These are different analytical questions.

86. REPLAY MODE

Given:

text

text

USTN

+

Event History

+

Policy Versions

+

Authority Inputs

+

Evidence References

svgsvg

SGTX should be able to reconstruct:

What did the system know at that moment, and why did it produce that state?

Replay supports:

- audits;
- investigations;
- legal review;
- regulator examination;
- incident response;
- debugging;
- AI validation;
- policy testing.

87. PROOF-OF-EXECUTION CERTIFICATE

For applicable milestones SGTX should be able to generate a structured certificate containing:

text

text

USTN

Contract Reference

Execution State

Financial State

Legal State

Physical/Operational State

Evidence References

Event Timestamps

Authority Sources

Cryptographic Event Root

Policy Version

Reconciliation Status

Exceptions

Settlement References

svgsvg

This becomes an institutional integration artifact.

88. SETTLEMENT CERTIFICATE ? CLOSURE CERTIFICATE

Settlement Certificate

Confirms:

The defined settlement condition was satisfied according to the specified evidence and applicable financial authority.

Closure Certificate

Confirms:

Required operational, financial, documentary, compliance, reconciliation and closure conditions have been satisfied according to the applicable closure policy.

These certificates must remain separate.

89. TRANSACTION TWIN

Introduce:

SGTX Transaction Twin

The Transaction Twin is the live logical representation of:

- obligations;
- actors;
- dependencies;
- documents;
- financial state;
- legal state;
- execution state;
- physical state;
- compliance state;
- evidence;
- exceptions;
- exposure;
- recovery paths;
- closure conditions.

It is not a metaverse or speculative virtual-world product.

It is the structured operational representation of the transaction.

90. DISPUTE PACKET

When a dispute occurs, SGTX should be able to assemble:

- transaction history;
- timeline;
- contract references;
- applicable clauses;
- documents;
- signatures;

- bank/payment events;
- logistics evidence;
- authority events;
- reconciliation discrepancies;
- state changes;
- event reasons;
- supporting evidence;
- AI-generated summary;
- source references.

The AI-generated portion is advisory; source evidence remains authoritative.

91. RECOVERY VAULT

Introduce an evidence/recovery repository.

It is not a financial custody account.

It contains or references:

- event history;
- state transitions;
- evidence;
- authority determinations;
- policies;
- reconciliation cases;
- dispute packages;
- corrective actions;
- recovery actions;
- certificates.

This becomes the forensic backbone.

92. EVIDENCE-FIRST PRINCIPLE

No material state transition should occur without the evidence or authority required by applicable policy.

Permitted provisional states are the explicit exception.

Where evidence is incomplete, SGTX must prefer:

text

text

PROVISIONALLY_VERIFIED

svgsvg

or:

text

text

UNKNOWN

svgsvg

over unjustified certainty.

93. PROVISIONAL STATES

Where policy permits:

text

text

PROVISIONALLY_VERIFIED

PROVISIONALLY_SETTLED

svgsvg

may be used.

These are not equivalent to definitive confirmation.

A provisional state must identify:

- missing evidence;
- validation deadline;
- authority source;
- conditions for conversion to definitive status.

94. NO SILENT REPAIR

Every correction must produce an explicit event containing, as applicable:

text

text

Previous State

New State

Reason

Authority

Evidence

Actor/System

Timestamp

Policy Version

Parent Event

svgsvg

No hidden database repair may silently become authoritative truth.

95. STATE INTEGRITY

SGTX should provide a non-authoritative measure of how strongly the current state is supported by evidence.

Potential factors:

- source agreement;
- evidence completeness;
- authority validity;
- temporal consistency;
- document integrity;
- payment confirmation;
- reconciliation completeness.

This is separate from:

- financial risk;
- credit risk;
- legal judgment.

96. RECONCILIATION CONFIDENCE

SGTX may maintain an operational evidence-consistency measure.

Example:

text

text

100 = Fully reconciled

85 = Minor non-critical evidence pending

60 = Material divergence

30 = Critical unresolved divergence

0 = Contradictory / unusable evidence

svgs

This is a prioritization indicator.

It does not replace deterministic state.

97. DIVERGENCE INDEX

The Divergence Index measures how far transaction dimensions have diverged.

Example:

text

text

Execution = SETTLED

Financial = RETURNED

Legal = DISPUTED

Divergence = HIGH

svgs

Where required dimensions align:

text

text

Divergence = NONE / LOW

svgsVG

This remains an operational analytics metric, not authoritative transaction truth.

98. TRANSACTION HEALTH

Operational dashboard:

text

text

GREEN

YELLOW

ORANGE

RED

BLACK

svgsVG

It may consider:

- financial divergence;
- legal issues;
- documents;
- compliance;
- logistics;
- reconciliation;
- deadlines;
- exposure.

Transaction Health never replaces the canonical state vector.

99. RESPONSIBILITY & LIABILITY MODEL

SGTX must not make absolute liability assumptions from technical architecture.

Responsibility follows:

- data provenance;
- contractual control;
- authorization;
- validation responsibility;
- system control;
- actor responsibility;

- applicable law.

Examples:

text

text

SGTX-generated data error

? potentially SGTX responsibility

Buyer-supplied wrong beneficiary

? potentially buyer responsibility

Bank execution error

? potentially bank responsibility

External regulatory change

? contract/law dependent

Credential compromise

? responsibility depends on control failure and applicable law

svgs

A formal contractual responsibility matrix is required for institutional deployments.

100. INSTRUCTION RESPONSIBILITY MATRIX

For each material settlement field, define:

Data Originator Validator Authoritative Party Executor

Amount Contract/authorized party SGTX/Bank Contracting parties/bank as applicable Bank

Beneficiary Authorized commercial party Bank/SGTX workflow Bank/account authority Bank

Currency Contract/authorized instruction Bank Contract/bank rules Bank

Payment Purpose Authorized party Bank/SGTX Applicable framework Bank

Sanctions Result Bank / authorized screening source Bank Bank/regulatory framework Bank

Value Date Instruction/Bank Bank Bank/payment system Bank

Tax Amount Applicable source Relevant authority/bank Tax framework Authorized channel

The actual responsibility mapping is jurisdiction- and product-specific.

101. FINANCIAL SAFETY BOUNDARY

SGTX can:

- orchestrate;

- verify;

- reconcile;

- observe;

- record;

- coordinate;

- generate evidence;
- create authenticated instructions;
- initiate authorized workflows.

SGTX does not inherently:

- custody;
- become a bank;
- replace corporate accounting systems;
- override banks;
- make legal judgments;
- become a regulator;
- become a central bank;
- unilaterally execute regulated payment activity where authorization is required.

Actual functionality remains subject to applicable law and regulatory classification.

102. BANK INSTITUTIONAL IDENTITY

Replace simplistic bank subtypes with:

text

text

ENTITY

?

INSTITUTION TYPE

?

REGULATORY STATUS

?

JURISDICTION

?

CAPABILITIES

svgsvg

Example:

text

text

Institution Type = BANK

Regulatory Status = APPROPRIATELY AUTHORIZED

Jurisdiction = EGYPT

Capabilities =

PAYMENT

SETTLEMENT

FX

TRADE_SERVICES

svgsvg

The actual regulatory status must be established from appropriate authoritative sources.

SGTX must not invent regulatory classifications.

103. REGULATORY STATUS VERIFICATION

For institutional onboarding:

text

text

Institution

?

Regulatory Authority

?

Authorization Status Verification

?

Capability Verification

?

Jurisdiction Profile

?

SGTX Enablement

svgsvg

In Egypt, the CBE maintains licensing and oversight frameworks for payment system operators and payment service providers under the Central Bank and Banking System Law No. 194 of 2020.

104. CROSS-BORDER REALITY

SGTX country/jurisdiction adapters should cover:

- identity/KYB;
- tax;
- customs;
- regulatory authorities;
- trade documents;
- logistics;
- banking/payment status;
- FX;
- compliance;
- reconciliation;

- document recognition;
- evidence requirements;
- authority mappings;
- cross-border rules;
- regulatory classification;
- permitted operating model.

105. COUNTRY CAPABILITY ADAPTERS

A country adapter must not merely be:

text

text

Country = Egypt

svgsvg

It should be:

text

text

COUNTRY PROFILE

+

JURISDICTION PROFILE

+

CAPABILITY MATRIX

+

AUTHORITY MAP

+

EVIDENCE PROFILE

+

PAYMENT RAIL PROFILE

+

DOCUMENT PROFILE

+

CUSTOMS PROFILE

+

REGULATORY PROFILE

svgsvg

106. BANK-SIDE / PAYMENT INTEGRATION STANDARD

SGTX must support a bank-approved integration spectrum:

text

text

ISO 20022

Bank APIs

Host-to-Host

SFTP

Treasury Connectivity

ERP Connectivity

Other Approved Institutional Interfaces

svgsvg

The architecture must not make one transport mandatory for all institutions.

107. MARKETPLACE / ERP INTEGRATION

Marketplaces and ERPs should not have to understand the entire SGTX internal architecture.

They consume stable canonical outputs such as:

text

text

TRADE_ACCEPTED

EXECUTION_STARTED

OBLIGATION_COMPLETED

SETTLEMENT_CONFIRMED

SETTLEMENT_PARTIALLY_COMPLETED

EXCEPTION_OPENED

RECONCILIATION_REQUIRED

DISPUTE_OPENED

TRANSACTION_CLOSED

POST_CLOSURE_EVENT

svgsvg

and machine-readable state queries.

108. ERP / MARKETPLACE OUTPUT MODEL

An external system should be able to request:

text

text

Current Execution Status

Financial Status

Exception Status

Reconciliation Status

Closure Status

Exposure Status

Evidence References

Proof-of-Execution

Settlement Certificates

svgsvg

without importing the entire internal event history.

Full history remains available through controlled authorized access.

109. INTEROPERABILITY EVENT CONTRACT

External systems consume a stable canonical event interface:

text

text

Marketplace / ERP

?

SGTX Integration API

?

Canonical Event Contract

?

SGTX

?

Execution / State / Settlement / Reconciliation

svgsvg

Internal implementation details should not unnecessarily leak into the external contract.

110. SETTLEMENT ORCHESTRATION CONTROL PLANE

Formalize:

SGTX Settlement Orchestration Control Plane

This is the logical bridge between:

Obligation Graph

and:

Bank Settlement Gateway

Architecture:

text

text

Contract / Obligation Graph

?

Settlement Orchestration

?

Multi-Leg Settlement Instruction

?

Bank Settlement Gateway

?

Bank

?

Financial State

?

Reconciliation

svgsvg

It does not custody funds.

111. EXISTING TECHNOLOGY STACK ALIGNMENT

The accepted architecture should remain aligned with the existing SGTX stack.

NATS JetStream

Use for:

- durable event streams;
- event propagation;
- integration events;
- asynchronous notifications;
- external event ingestion.

NATS is transport/event infrastructure, not the ultimate historical authority by itself.

Temporal

Use for:

- long-running workflows;
- retries;
- compensation;
- recovery;
- reconciliation workflows;
- disputes;
- human approval;
- post-closure workflows;
- timeout orchestration.

Temporal workflow state is not the canonical historical event record.

PostgreSQL

Use for:

- canonical relational metadata;
- state projections;
- authority mappings;
- policy references;
- obligation metadata;
- transaction projections;
- settlement instruction metadata;
- reconciliation cases.

ClickHouse

Use for:

- historical analytics;
- reconciliation analytics;
- exception analytics;
- operational intelligence;
- institutional reporting;
- trend analysis.

ClickHouse is not the authoritative transactional state store.

112. SETTLEMENT GRAPH HIERARCHY

The data model must distinguish:

text

text

TRADE

?

??? CONTRACT

?

??? OBLIGATIONS

?

??? SETTLEMENT INSTRUCTIONS

? ?

? ??? SETTLEMENT LEG 1

? ??? SETTLEMENT LEG 2

? ??? SETTLEMENT LEG N

?

??? DOCUMENTS

??? LOGISTICS

??? CUSTOMS

??? COMPLIANCE

??? DISPUTES

??? RECOVERY EVENTS

svgsvg

This prevents duplication and keeps parent/child relationships explicit.

113. CLOSURE WITH EXCEPTION

Where policy permits closure despite defined non-critical unresolved matters:

text

text

CLOSED_WITH_EXCEPTION

svgsvg

must preserve:

- exception ID;
- severity;
- outstanding obligation;
- financial exposure;
- authority;
- reason;
- required post-closure action;
- responsible party;
- deadline.

Closure with exception is not equivalent to full closure.

114. POST-CLOSURE EVENT PROCESSING

Late evidence or authority events must follow:

text

text

POST-CLOSURE EVENT

?

AUTHENTICITY / AUTHORITY CHECK

?

IMPACT ASSESSMENT

?

DEPENDENCY ANALYSIS

?

RECONCILIATION

?

CURRENT STATE UPDATE

?

RECOVERY / CORRECTION IF REQUIRED

svgsvg

Historical events remain immutable.

115. POLICY-GOVERNED RECOVERY

Recovery actions must never be invented by an AI system or arbitrary application code.

Recovery paths are predefined through:

- policy;
- obligation graph;
- dependency graph;
- authority;
- contract;
- jurisdiction.

116. PROTECTED STATE TRANSITIONS

Material state transitions must pass through:

text

text

Event Validation

?

Authority Validation

?

Evidence Validation

?

Policy Validation

?

Dependency Validation

?

State Transition

?

Event Append

?

Projection Update

?

Reconciliation Trigger

svgsVG

No material authoritative transition may bypass the control path.

117. NO SILENT AUTHORITY ESCALATION

A component must not acquire authority merely because it receives an external event.

Example:

text

text

Bank message received

svgsVG

does not automatically mean:

text

text

SGTX may alter legal state

svgsVG

The system must determine the event's domain and authority.

118. NO SILENT STATE COLLAPSE

The system must not convert:

text

text

Execution = SETTLED

Financial = UNKNOWN

Legal = DISPUTED

svgsVG

into:

text

text

Transaction = FAILED

svgsVG

unless a governed policy explicitly defines that derived condition.

Multidimensional divergence must remain visible.

119. TRANSACTION HEALTH VS AUTHORITATIVE STATE

The following are operational analytics:

- Transaction Health;
- Divergence Index;
- Reconciliation Confidence;
- State Integrity Score.

They must never overwrite or replace:

- state vector;
- event history;
- external authority state.

120. FINANCIAL RISK VS STATE INTEGRITY

Keep these separate.

Financial Exposure

How much economic exposure exists?

State Integrity

How strongly is the current state supported by evidence?

Reconciliation Confidence

How well do relevant systems agree?

Transaction Health

What is the operational condition?

A trade can have:

text

text

High Financial Exposure

+

High State Integrity

svgsvg

or:

text

text

Low Financial Exposure

+

Low State Integrity

svgsvg

These are different concepts.

121. SGTX TRANSACTION CONSTITUTION

The following are non-negotiable constitutional rules:

- 1 No silent state mutation.
- 2 No destructive rollback of authoritative history.
- 3 External authority remains authoritative within its domain.
- 4 AI cannot independently create unauthorized material authoritative state.
- 5 Every material correction is traceable.
- 6 Every reversal is a new event.
- 7 Financial, legal, execution and physical states remain distinct.
- 8 Unknown is a valid state.
- 9 Authority must be explicit.
- 10 Evidence must support material state transitions.
- 11 Reconciliation is a first-class lifecycle.
- 12 Post-closure events remain addressable.
- 13 No subsystem may silently redefine transaction truth.
- 14 No forced closure on timeout.
- 15 Assertions are not automatically confirmations.
- 16 Evidence integrity is distinct from legal authority.
- 17 Policy version must be traceable.
- 18 Historical events remain immutable.
- 19 Current operational state is derived from governed events.
- 20 Recovery uses compensating execution, not historical erasure.
- 21 USTN is a canonical namespace, not universal external authority.
- 22 Bank settlement remains bank-authoritative.
- 23 Regulatory classification depends on actual functionality.
- 24 ISO 20022 is an external interoperability standard, not the SGTX canonical state model.
- 25 Multi-leg settlement requires an explicit atomicity policy.
- 26 Settlement instruction state is distinct from trade state.
- 27 Settlement leg state is distinct from parent settlement state.
- 28 Closure is derived through closure policy.
- 29 Evidence may be provisional, but provisional status must be explicit.
- 30 A timeout cannot be interpreted as successful settlement without authoritative evidence.
- 31 Observations, Assertions, and Confirmations are distinct event types with distinct authority characteristics.
- 32 Commands and Events are distinct architectural constructs. A Command is a request for action; an Event is a record that an action occurred or was accepted/rejected. No command becomes an

authoritative event without passing through the governed control path.

122. SGTX BANK / FINANCIAL BOUNDARY

The bank controls:

text

text

Funds

Bank Ledger

Account Authorization

Bank Compliance

Payment Execution

Settlement Confirmation

Bank-Side Risk Controls

svgsvg

SGTX controls or coordinates, within its authorized scope:

text

text

Transaction Orchestration

Obligation State

Execution State

USTN Correlation

Settlement Instruction State

Settlement Leg State

Evidence

Reconciliation

Exception Workflow

Recovery Orchestration

Certificates

Cross-System State

svgsvg

Neither side silently replaces the other's authority.

123. SGTX INSTITUTIONAL CONTROL BOUNDARY

SGTX is:

- execution infrastructure;
- state infrastructure;
- orchestration infrastructure;

- reconciliation infrastructure;
- evidence infrastructure;
- exception/recovery infrastructure.

SGTX is not inherently:

- commercial bank;
- central bank;
- customer-funds custodian;
- escrow institution;
- payment rail;
- legal adjudicator;
- regulatory authority;
- ERP replacement;
- marketplace replacement;
- autonomous AI sovereign.

Actual regulatory treatment depends on jurisdiction and implemented functionality.

124. IMPLEMENTATION PRIORITY FRAMEWORK

Calendar estimates in prior reviews are not contractual project commitments.

Implementation follows dependency-based architecture waves.

P0 - Constitutional Core

Implement:

- multi-clock architecture;
- state vector;
- finality classes;
- canonical event spine;
- USTN lineage;
- immutable history;
- authority matrix;
- policy versioning;
- three timestamps;
- idempotency;
- out-of-order event handling;
- unknown/provisional states;
- state conflict protocol;
- assertion vs confirmation;
- evidence vs authority separation;

- no-forced-closure rules;
- closure policy model;
- settlement/transaction state separation.
- Observation vs Assertion vs Confirmation distinction.
- Command vs Event architectural pattern.

P1 - Settlement & Execution Integrity

Implement:

- Settlement Orchestration Control Plane;
- Bank Settlement Gateway;
- settlement instruction lifecycle;
- settlement leg model;
- settlement atomicity policy;
- bank reality adapters;
- ISO 20022 adapters;
- reversal engine;
- compensating execution;
- recovery engine;
- exception engine;
- reconciliation control plane;
- document authenticity/versioning;
- counterparty reality/evidence;
- late evidence;
- post-closure events.

P2 - Trade Intelligence & Financial Consequences

Implement:

- obligation graph;
- dependency graph;
- recovery graph;
- causal impact engine;
- truth triangulation;
- Financial Position & Consequence Subledger;
- Financial Exposure Engine;
- dispute packet;
- payment-leg reconciliation;
- settlement completion matrix.

P3 - Institutional Assurance

Implement:

- Proof-of-Execution Certificate;
- Settlement Certificate;
- Closure Certificate;
- Transaction Twin;
- Replay Mode;
- Policy Time Travel;
- State Integrity;
- Reconciliation Confidence;
- Divergence Index;
- Transaction Health;
- recovery vault.

P4 - Cross-Border Expansion

Implement:

- jurisdiction intelligence;
- capability-based jurisdiction profiles;
- regulatory classification gate;
- country profiles;
- payment-rail adapters;
- customs profiles;
- tax profiles;
- regulatory profiles;
- electronic-document recognition;
- evidence profiles;
- authority profiles;
- external identifier registry;
- cross-border conflict handling.

124A. IMPLEMENTATION TIMELINE (Planning Estimate)

This section is provided as a planning roadmap for internal estimation and resource planning. It is not a contractual commitment.

The following estimates are based on the assumption of parallel workstreams and are contingent on:

- Team capacity and availability
- Number of parallel workstreams
- Architectural dependencies being resolved according to the specified order
- External bank integration timelines

- Regulatory approval processes
- Testing and certification cycles

These estimates should be used for high-level planning only and must be regularly updated as actual engineering velocity and institutional dependencies become known.

124B. DATA RETENTION AND ARCHIVING (Architecture Definition)

SGTX retention is policy-driven and jurisdiction-aware.

Minimum legal retention requirements are determined from the applicable law, regulation, contract, regulatory instruction and litigation/dispute hold applicable to the specific data class and jurisdiction.

Where multiple requirements apply, SGTx applies the legally required retention rule identified by the applicable jurisdictional profile and does not infer a universal retention period from a single statute.

124B.1 Jurisdictional Retention Profile Architecture

The architecture must support the definition of retention policies within jurisdiction capability profiles.

Example profile structure (not constitutional values):

text

text

JURISDICTION: EGYPT

??? Data Retention Policies

? ??? Event History: Determined by applicable tax/commercial law

? ??? State Vector History: Determined by applicable tax/commercial law

? ??? Settlement Instructions: Determined by applicable tax/commercial law

? ??? Bank Confirmations: Determined by applicable AML/regulatory law

? ??? Customs Declarations: Determined by applicable customs law

? ??? ...

??? Legal Hold Procedures

??? Regulatory Hold Procedures

??? Disposition Process

124B.2 Archiving Strategy

The Recovery Vault is the primary archiving mechanism.

It stores or references:

- Complete event history
- All state transitions
- Evidence packages
- Authority determinations
- Policy versions
- Reconciliation cases
- Dispute packages

- Corrective actions
- Recovery actions
- Settlement certificates
- Closure certificates

124B.3 Data Disposition

At the end of the applicable retention period, records enter a governed disposition process that evaluates:

- Legal holds
- Regulatory holds
- Contractual requirements
- Evidentiary requirements
- Applicable data-protection rules

Following this evaluation, records may be subject to:

- Deletion
- Anonymization
- Archival
- Other permitted disposition

Records must not be automatically anonymized or deleted unless explicitly permitted by the applicable legal, regulatory, and contractual frameworks.

124C. BANK ONBOARDING REQUIREMENTS

124C.1 Legal Agreements Required

- Master Services Agreement
- Data Processing Agreement (GDPR/PDPL compliant)
- Information Security Schedule
- Operational SLA
- Incident/Breach Procedures
- Liability Framework
- Business Continuity/Disaster Recovery
- Audit Rights
- Subprocessor Provisions
- Change Management Procedure
- Regulatory Cooperation Provisions

124C.2 Technical Integration Requirements

- Connectivity validation (API, ISO 20022, H2H, SFTP)
- Message format validation (ISO 20022, bank-native)
- Certificate validation (mTLS, PKI)
- Idempotency testing

- Replay protection testing
- Latency testing
- Throughput testing
- Exception handling testing
- Reconciliation testing

124C.3 Testing Requirements

- Connectivity testing
- Unit testing (all message types)
- Integration testing
- End-to-end testing (full transaction lifecycle)
- Disaster recovery testing
- Security penetration testing
- Load testing
- Exception testing

124C.4 Operational SLA (Targets)

This section specifies SGTX technical delivery targets, not bank/external settlement SLAs.

SGTX distinguishes three latency domains:

L1 - SGTX Internal Processing

- API acceptance / validation latency
- Internal system processing

L2 - Bank Gateway Delivery

- Time to transmit and receive gateway acknowledgment

L3 - External Financial Execution

- Bank/payment rail settlement duration
- External compliance screening
- Correspondent routing
- Batch/rail constraints

L3 is external and must not be presented as an SGTX deterministic latency commitment.

124D. REGULATORY IMPACT ASSESSMENT (Preliminary Alignment)

This section represents a preliminary architecture alignment with known regulatory frameworks.

SGTX's actual regulatory classification cannot be confirmed purely from the architecture document. The operative question is:

What functionality will SGTX actually perform in each jurisdiction?

The final regulatory treatment depends on actual:

- Functionality
- Technical control

- Customer interaction
- Contractual structure
- Payment involvement
- Funds movement role
- Authorization role
- Data access
- Regulatory perimeter
- Jurisdiction

124D.1 CBE Regulatory Alignment (Preliminary)

The architecture aligns with known elements of CBE regulations under Law No. 194 of 2020, which covers payment system operators and payment service providers, including activities such as payment execution, transfers, payment initiation, and account-information services.

Key alignment points:

- Non-custodial control-plane principle preserves bank authority over funds
- Multi-clock state separation enables proper financial reporting
- Bank Settlement Gateway provides clear regulatory boundary
- Regulatory Classification Gate ensures jurisdiction-specific compliance

124D.2 FATF Recommendations

- Authority matrix ensures clear responsibility
- Evidence-first principle supports audit trails
- State vector supports proper record-keeping
- Reconciliation control plane supports transaction monitoring

124D.3 AML/CFT Compliance (Architecture)

The architecture supports:

- Screening integration
- Suspicious Activity Report generation
- Beneficiary verification
- Sanctions status tracking
- PEP screening

The regulated institution retains its own responsibility for compliance. SGTX provides:

- Screening result integration
- Evidence collection and correlation
- Workflow enforcement
- Exception routing

124D.4 Data Protection Compliance

The architecture supports:

- Data localisation mandates
- PDPL compliance
- Cross-border data transfer controls
- Data retention policies
- Evidence integrity requirements

124E. BUSINESS CONTINUITY AND DISASTER RECOVERY (Targets)

This section defines targets for SGTX internal system resilience and disaster recovery.

124E.1 RTO/RPO Targets

These are engineering targets subject to explicit durability/quorum definitions and operational testing.

124E.2 Data Replication

The architecture supports:

- Active-active replication for selected critical services
- Synchronous replication for Event Spine
- Asynchronous replication for analytics (ClickHouse)

The exact replication topology is implementation-specific and must be defined in the detailed infrastructure specification.

124E.3 Failover Architecture

The architecture supports:

- Automatic failover within defined targets
- Manual failover with multi-sig approval
- Regular failover testing

124E.4 Disaster Recovery Testing

The architecture supports:

- Regular DR testing
- Quarterly failover testing
- Annual full failover exercise
- DR drill reporting for audit purposes

125. DEPENDENCY RULE

P0 must establish the data/state/authority model before P1-P4 implementations attempt to build independent state logic.

No feature may create its own competing:

- transaction state;
- authority model;
- event history;
- reconciliation semantics;
- identity namespace.

All components must consume the canonical constitutional models.

126. DESIGN RULE: "ONE CONSTITUTION, MANY ADAPTERS"

SGTX should have:

One

- canonical event model;
- canonical state model;
- canonical obligation model;
- canonical authority model;
- canonical evidence model;
- canonical USTN model.

But many:

- bank adapters;
- country adapters;
- payment-rail adapters;
- ERP adapters;
- marketplace adapters;
- customs adapters;
- document adapters;
- logistics adapters.

Adapters translate external reality into the canonical SGTX model without erasing the external native representation.

127. DESIGN RULE: "TRANSLATION, NOT TRANSFORMATION"

SGTX should translate external semantics into canonical state while preserving original external semantics.

For example:

text

text

Bank Native Status = XYZ

?

SGTX Adapter

?

Canonical Financial State = PROCESSING

svgsVG

But the original:

text

text

XYZ

svgsvg

remains preserved.

This is critical for institutional auditability.

128. DESIGN RULE: "ORCHESTRATION, NOT CUSTODY"

SGTX coordinates financial activity without assuming ownership or custody of the underlying funds.

129. DESIGN RULE: "EVIDENCE, NOT CLAIM"

SGTX should prefer:

text

text

Verified evidence

svgsvg

over:

text

text

Unverified assertion

svgsvg

and:

text

text

Unknown

svgsvg

over:

text

text

False certainty

svgsvg

130. DESIGN RULE: "RECOVERY, NOT ERASURE"

The system moves forward through:

- correction;
- compensation;
- recovery;
- new authorization;
- reconciliation.

It does not erase historical reality.

131. DESIGN RULE: "AUTHORITY, NOT MAJORITY"

When sources conflict, SGTX does not simply count votes.

It evaluates:

- authority;
- provenance;
- evidence integrity;
- temporal consistency;
- contractual relevance;
- policy.

132. DESIGN RULE: "STATE IS A VECTOR"

No single global status may silently collapse multiple real-world dimensions into one oversimplified conclusion.

133. DESIGN RULE: "CLOSURE IS EARNED"

A transaction becomes closed because closure policy conditions are satisfied, not because a workflow reached its last screen or because a timer expired.

134. FINAL CANONICAL SGTX ARCHITECTURE

text

text

COMMERCIAL ECOSYSTEM

Marketplace | ERP | Corporate | Logistics | Bank

?

?

SGTX EXECUTION API

?

????????????????????????????

? ?

OBLIGATION ENGINE AUTHORITY ENGINE

? ?

????????????????????????????

?

SGTX TRANSACTION TWIN

?

????????????????????????????

? ?

EXECUTION ENGINE SETTLEMENT ORCHESTRATION

?

MULTI-LEG INSTRUCTION

?

?

BANK SETTLEMENT GATEWAY

?

??

???

API ISO 20022 H2H/SFTP

???

??

?

BANK SYSTEMS

?

BANK CONTROL

?

PAYMENT RAIL

?

?

EXTERNAL EXECUTION

?

?

CANONICAL EVENT SPINE

?

??

????

EXECUTION FINANCIAL LEGAL PHYSICAL DOCUMENT

CLOCK CLOCK CLOCK CLOCK STATE

????

??

?

?

STATE VECTOR

?

??

????

RECONCILIATION EXCEPTION OBLIGATION EXPOSURE CLOSURE

CONTROL PLANE ENGINE GRAPH ENGINE POLICY

?????

???

???

CAUSAL IMPACT DEPENDENCY ?

ENGINE GRAPH ?

???

?????????????????

??

STATE CONFLICT ?

PROTOCOL ?

??

?????????????????????????????????????

????

DISPUTE RECOVERY CORRECTION ?

ENGINE ENGINE ENGINE ?

????

?????????????????????????????????????

??

EVIDENCE LAYER ?

??

?????????????????????????????????????

????

DISPUTE PACKET PROOF OF EXECUTION RECOVERY VAULT ?

??

?????????????????????????????????????

?

SETTLEMENT / CLOSURE

CERTIFICATES

?

?

INSTITUTIONAL OUTPUT

svgsvg

135. CANONICAL TRANSACTION HIERARCHY

The canonical object hierarchy is:

text

text

TRADE

?

??? CONTRACT

?

??? OBLIGATION GRAPH

?

? ??? COMMERCIAL OBLIGATION

? ??? DOCUMENT OBLIGATION

? ??? LOGISTICS OBLIGATION

? ??? CUSTOMS OBLIGATION

? ??? COMPLIANCE OBLIGATION

? ??? FINANCIAL OBLIGATION

?

??? SETTLEMENT INSTRUCTION

? ?

? ??? SETTLEMENT LEG 1

? ??? SETTLEMENT LEG 2

? ??? SETTLEMENT LEG N

?

??? PAYMENT / BANK EVENTS

?

??? DOCUMENTS

?

??? LOGISTICS EVENTS

?

??? CUSTOMS EVENTS

?

??? COMPLIANCE EVENTS

?

??? LEGAL EVENTS

?

??? RECONCILIATION CASES

?

??? DISPUTES

?

??? RECOVERY EVENTS

?

??? CLOSURE EVENTS

svgsvg

Everything remains correlated through USTN and event lineage.

136. CANONICAL SETTLEMENT HIERARCHY

text

text

TRADE

?

FINANCIAL OBLIGATION

?

SETTLEMENT INSTRUCTION

?

SETTLEMENT LEG

?

BANK-SIDE INSTRUCTION

?

BANK TRANSACTION

?

PAYMENT RAIL EVENT

?

BANK CONFIRMATION

?

SGTX FINANCIAL STATE

?

RECONCILIATION

?

PARENT SETTLEMENT STATE

svgsvg

137. CANONICAL REVERSAL MODEL

text

text

ORIGINAL EVENT

?

EXTERNAL REVERSAL / DISPUTE / CORRECTION

?

NEW EVENT

?

AUTHORITY / EVIDENCE CHECK

?

STATE VECTOR UPDATE

?

FINANCIAL EXPOSURE UPDATE

?

CAUSAL IMPACT ANALYSIS

?

RECONCILIATION

?

RECOVERY / COMPENSATION IF REQUIRED

svgsvg

History remains intact.

138. CANONICAL DISPUTE MODEL

text

text

DISPUTE ASSERTION

?

AUTHORITY / EVIDENCE CLASSIFICATION

?

DISPUTE OPEN

?

AFFECTED OBLIGATIONS IDENTIFIED

?

AFFECTED FINANCIAL EXPOSURE IDENTIFIED

?

EVIDENCE PACKAGE ASSEMBLED

?

RECONCILIATION

?

AUTHORIZED RESOLUTION

?

NEW EVENT

?

RECOVERY / CORRECTION / CLOSURE

svgsvg

SGTX records and orchestrates the process; it does not independently adjudicate legal disputes.

139. CANONICAL LATE-EVIDENCE MODEL

text

text

TRADE CLOSED

?

LATE EVENT RECEIVED

?

AUTHENTICITY / AUTHORITY CHECK

?

POST-CLOSURE EVENT

?

CAUSAL IMPACT ANALYSIS

?

RECONCILIATION

?

CURRENT STATE EVALUATION

?

RECOVERY / CORRECTION IF REQUIRED

svgsvg

140. CANONICAL MULTI-PARTY PAYMENT MODEL

text

text

USTN

?

??? Seller Obligation

? ??? Leg A

?

??? Logistics Obligation

? ??? Leg B

?

??? Customs Obligation

? ??? Leg C

?

??? Laboratory Obligation

? ??? Leg D

?

??? Broker Obligation

? ??? Leg E

?

??? SGTX Fee Obligation

??? Leg F

svgsvg

Every leg can independently have:

text

text

PENDING

AUTHORIZED

SUBMITTED

PROCESSING

SETTLED

PARTIALLY_SETTLED

REJECTED

RETURNED

RECALLED

REVERSED

UNKNOWN

svgsvg

The parent transaction derives its aggregate settlement state from the governed leg states and settlement policy.

141. CANONICAL COMMERCIAL / BANK / SGTX DIVISION

Commercial Parties

Determine:

- commercial agreement;
- commercial obligations;
- contractual data;
- beneficiary instructions;
- accepted conditions.

SGTX

Provides:

- orchestration;
- execution state;
- obligation graph;
- correlation;
- evidence;
- reconciliation;
- exceptions;
- recovery workflow;
- certificates.

Bank

Controls:

- customer account;
- funds;
- banking authorization;
- bank-side compliance;
- payment execution;
- bank ledger;
- financial confirmation.

Government / Regulatory Authority

Controls:

- regulatory decisions;
- customs determinations;
- tax determinations;
- legally authoritative actions.

This separation is fundamental.

142. EXTERNAL INSTITUTION OUTPUTS

SGTX should expose institutional views such as:

Marketplace View

- trade status;
- execution status;
- exception state;
- settlement confirmation;
- delivery state.

ERP View

- transaction status;
- financial state;
- obligation completion;
- exposure;
- reconciliation;
- evidence references.

Bank View

- USTN;
- settlement instructions;
- legs;
- authorization;
- contractual basis;
- beneficiary information;
- reconciliation status;
- required evidence.

Regulatory / Authority View

- applicable transaction identifiers;
- authorized evidence;
- relevant events;
- required documents;
- applicable obligations;
- state history where authorized.

143. SECURITY AND CONTROL PRINCIPLES

All authoritative transaction actions must support, as applicable:

- authenticated actors;
- authorization context;
- least privilege;
- versioned policies;
- cryptographic integrity;

- immutable event history;
- idempotency;
- replay protection;
- auditability;
- evidence provenance;
- role separation;
- maker/checker where required;
- institutional access control.

144. BUSINESS CONTINUITY PRINCIPLE

SGTX must be capable of operating safely during:

- bank connectivity failure;
- customs connectivity failure;
- ERP downtime;
- marketplace downtime;
- delayed external events;
- incomplete webhook delivery;
- network interruption.

Failure of connectivity must not create artificial success.

Where state cannot be verified:

text

text

UNKNOWN

svgsvg

or:

text

text

RECONCILIATION_REQUIRED

svgsvg

must be preferred over false confirmation.

145. OFFLINE / DELAYED EXTERNAL EVIDENCE

Where courthouse, bank, customs or other external connectivity fails:

SGTX may continue activities that policy explicitly permits without external confirmation.

However:

Lack of connectivity must never be converted into fabricated authoritative evidence.

Pending operations remain:

text

text

PENDING

UNKNOWN

PROVISIONAL

svgsvg

until required evidence arrives.

146. OBSERVABILITY REQUIREMENTS

Operational observability must distinguish:

- event ingestion;
- event processing;
- state projection;
- reconciliation;
- bank connectivity;
- document verification;
- external identifier correlation;
- exception queues;
- recovery workflows.

Critical observability metrics should include:

- reconciliation backlog;
- unresolved divergence;
- stale financial states;
- unknown-state age;
- exception age;
- settlement-leg failure rate;
- duplicate event rate;
- late-event rate;
- post-closure event rate.

147. TESTING PRINCIPLE

The SGTX test strategy must be:

Failure-path-first, not happy-path-first.

Tests must explicitly include:

- partial settlement;
- duplicate payment event;
- out-of-order payment event;

- delayed bank confirmation;
- payment reversal;
- bank timeout;
- document revocation;
- document supersession;
- milestone dispute;
- customs hold;
- legal event after closure;
- contradictory source assertions;
- invalid evidence;
- authority mismatch;
- policy version change;
- late evidence;
- compensation;
- recovery;
- closure with exception;
- prohibited AI action;
- unauthorized state transition;
- corrupted projection and replay from event spine.

148. ADVERSARIAL TESTING

Every critical state transition should have tests asking:

What happens if the external world does the opposite of what SGTX expects?

Examples:

text

text

Expected = SETTLED

Actual = RETURNED

Expected = DOCUMENT_VALID

Actual = REVOKED

Expected = DELIVERED

Actual = LOST / DISPUTED

Expected = BANK_ACCEPTED

Actual = UNKNOWN

Expected = LEGAL_EFFECTIVE

Actual = COURT_STAY

svgsvg

The expected behavior is always governed state transition, not silent correction.

149. AI SAFETY TESTING

AI must be tested against:

- hallucinated evidence;
- fabricated authority;
- incorrect document interpretation;
- conflicting sources;
- prompt injection from documents;
- malicious counterparty content;
- recommendation escalation;
- attempted direct state manipulation.

AI outputs must never bypass deterministic policy and authority controls.

150. INSTITUTIONAL DEPLOYMENT GATE

A new institution should pass:

text

text

Identity

?

Jurisdiction

?

Regulatory Status

?

Capabilities

?

Security

?

Connectivity

?

Authority Mapping

?

Data Contract

?

Operational SLA

?

Testing

?

Certification

?

Production Enablement

svgsvg

151. REGULATORY / BANK CHANGE MANAGEMENT

When a jurisdiction or bank changes:

- payment message format;
- regulatory rule;
- required data;
- authority;
- document;
- authentication;
- API;
- settlement requirement;

SGTX must update its adapter/policy profile without rewriting historical transaction events.

Historical policy versions remain intact.

152. MASTER DESIGN INVARIANT

The following must remain true regardless of country, bank, ERP, marketplace or payment rail:

External systems may change. SGTX's canonical transaction semantics and constitutional controls must remain stable.

Adapters absorb external change.

153. WHAT THIS ARCHITECTURE DOES NOT MEAN

This architecture does not turn SGTX into:

- a commercial bank;
- a central bank;
- a sovereign currency issuer;
- a custodian;
- an escrow institution;
- a payment rail;
- an accounting ERP;
- a marketplace replacement;
- a legal court;
- a regulator;
- a generalized blockchain;

- an autonomous AI decision system.

SGTX remains the controlled infrastructure layer between commercial intent and real-world execution, subject to jurisdiction-specific regulatory treatment based on implemented functionality.

154. FINAL SGTX ARCHITECTURAL PROPOSITION

SGTX is not a system designed to force a trade into one simplistic lifecycle.

It is infrastructure that maintains a:

cryptographically verifiable, authority-aware, multidimensional representation of a trade throughout its entire life.

That includes:

- intent;
- agreement;
- obligations;
- execution;
- settlement;
- partial settlement;
- financial reversal;
- legal dispute;
- document revocation;
- customs intervention;
- logistics failure;
- reconciliation;
- recovery;
- financial exposure;
- late evidence;
- post-closure events.

SGTX therefore answers not only:

"Did the trade execute?"

but also:

"What happened?"

"When did it happen?"

"When did SGTX learn about it?"

"Who or what had authority?"

"What evidence supports it?"

"Which systems agree?"

"Which systems disagree?"

"What financial exposure exists?"

"What obligations are affected?"

"What recovery paths are permitted?"

"What remains unresolved?"

"What is the legally and financially recognized state?"

155. FINAL SGTX DESIGN PHILOSOPHY

The architecture is ultimately governed by these principles:

1. Trade is not linear.

SGTX models the real world rather than pretending it is synchronized.

2. Settlement is not always closure.

A financial settlement does not automatically close the entire transaction.

3. Finality is multidimensional.

Execution, financial, legal and physical finality may differ.

4. History is immutable.

Reversals create new events.

5. State is derived.

Current state is calculated from governed events and authoritative inputs.

6. Reconciliation is execution.

It is a core operational subsystem.

7. Unknown is safer than false certainty.

SGTX must be allowed to say:

We do not yet know.

8. Authority is domain-specific.

SGTX does not replace banks, courts, regulators, customs, issuers or counterparties.

9. AI assists; policy and authorized institutions govern.

AI does not become sovereign authority.

10. Recovery replaces rollback.

The system moves forward by recording what happened and executing a permitted corrective path.

11. USTN correlates; it does not override.

USTN is the Universal State & Transaction Namespace, not the external world's universal authority.

12. The bank controls money.

The bank remains responsible for funds movement and banking authorization.

13. SGTX controls orchestration.

SGTX provides execution coordination, state, reconciliation, evidence, exception and recovery infrastructure within its authorized scope.

14. Closure is earned.

Closure occurs only when governed closure conditions are satisfied.

15. External reality remains authoritative.

SGTX integrates with the real world; it does not attempt to replace it.

156. FINAL MASTER ARCHITECTURAL STATEMENT

SGTX is a sovereign, non-custodial, AI-governed cross-border trade execution and reconciliation infrastructure that maintains a cryptographically verifiable, authority-aware, multidimensional transaction state across commercial, financial, legal, physical, documentary, compliance and regulatory domains.

It does not require these domains to agree at the same moment. It preserves their differences, records their provenance, reconciles their divergence, calculates economic consequences, orchestrates permitted execution and recovery, and maintains an immutable causal history throughout the transaction lifecycle.

SGTX does not erase reversals, disputes, corrections or late evidence. It records them as new governed events. It does not replace banks, courts, regulators, customs authorities, document issuers, ERPs or marketplaces. It provides the infrastructure layer through which those systems can remain authoritative within their respective domains while participating in one coherent transaction lifecycle.

SGTX is therefore not merely an execution engine. It is a governed transaction-state, settlement-orchestration, reconciliation, exception, recovery and evidence infrastructure for real-world cross-border trade.

157. MASTER IMPLEMENTATION INVARIANT

Before any SGTx feature, connector, workflow, AI capability, payment function, banking integration, country adapter or developer implementation is approved, it must answer:

text

text

1. Which transaction domain does it affect?
2. Which clock does it affect?
3. Which state-vector dimension does it affect?
4. Who is authoritative for that dimension?
5. What evidence is required?
6. What event records the action?
7. What USTN/lineage relationships are created?
8. Can the event arrive late or out of order?
9. Can it be duplicated?
10. Can it be reversed?
11. What financial exposure can it create?
12. What obligations can it affect?
13. What downstream dependencies can it affect?
14. What exception states can result?
15. What recovery paths are permitted?
16. Can it create a post-closure event?
17. Which policy version governs it?

18. What role, if any, can AI play?
19. What human/institutional authorization is required?
20. What jurisdiction/regulatory classification applies?
21. Does it preserve the non-custodial boundary?
22. Does it preserve external institutional authority?
23. Can the complete causal history be reconstructed?
24. Can the current state be rebuilt from the event history?
25. Can the transaction be safely reconciled if external systems disagree?

svgs

A feature that cannot answer these questions is not yet constitutionally ready for production SGTx.

158. MASTER GOVERNANCE RULE

No future amendment may introduce a new:

- transaction state;
- settlement state;
- authority model;
- financial truth model;
- document truth model;
- event history;
- identifier namespace;
- exception path;
- AI authority;
- recovery mechanism

without explicitly mapping it to this constitutional architecture.

Duplicate concepts must be merged.

Conflicting concepts must be resolved.

New functionality must extend the canonical model rather than create a parallel model.

159. MASTER BASELINE STATUS

This document is the canonical SGTx Transaction Finality, Multi-Clock State, Event Integrity, Settlement Orchestration, Reconciliation, Exception, Recovery, Evidence, Financial Exposure, Cross-Border and Institutional Governance baseline.

The following are therefore considered integrated and accepted:

- Three-Clock / Multi-Clock Architecture;
- Physical/Operational Clock;
- State Vector;
- Finality Classes;
- Settlement-versus-Closure distinction;

- Closure Policy Engine;
- Immutable Canonical Event Spine;
- Event Sourcing;
- Event Causality;
- Three-Timestamp Model;
- Idempotency;
- Out-of-Order Event Handling;
- Reversal-as-New-State;
- Recovery-not-Rollback;
- Exception State Taxonomy;
- Unknown State;
- No Forced Closure;
- Controlled Post-Closure Mutation;
- Post-Closure Observation (with jurisdiction-specific period definition);
- Reconciliation Control Plane;
- State Conflict Protocol;
- Authority Matrix;
- Domain-Specific Authority;
- Assertion-versus-Confirmation;
- Evidence Integrity-versus-Authority;
- Truth Triangulation;
- Bank Reality Adapter;
- Bank Settlement Gateway;
- ISO 20022-first banking integration;
- Bank/API/H2H/SFTP adapter model;
- Bank-side authorization boundary;
- Non-Custodial Control-Plane principle;
- Regulatory Classification Gate;
- Jurisdiction Capability Profiles;
- Counterparty Reality & Evidence;
- Document Authenticity & Versioning;
- Cross-Border Reality Layer;
- Obligation Graph;
- Dependency Graph;
- Recovery Graph;

- Causal Impact Engine;
- Exception Engine;
- Exception Containment;
- Exception Severity;
- Exception SLAs;
- Financial Position & Consequence Subledger;
- Financial Exposure Engine;
- Leg-Level Settlement State;
- Settlement Leg ID;
- Settlement Instruction Lifecycle;
- Settlement Instruction Versioning;
- Settlement Atomicity Policy;
- Payment Dependency Graph;
- Funds Sufficiency Snapshot;
- Payment Leg Certificate;
- Bank Reference Mapping;
- External Identifier Registry;
- USTN as Universal State & Transaction Namespace;
- Nafeza/Customs Identifier Correlation;
- AI Governance;
- AI Recommendation Gateway;
- Risk-Tiered Human/Institutional Involvement;
- USTN Event Lineage;
- Policy Versioning;
- Policy Time Travel;
- Replay Mode;
- Proof-of-Execution Certificate;
- Settlement Certificate;
- Closure Certificate;
- Transaction Twin;
- Dispute Packet;
- Recovery Vault;
- Evidence-First Principle;
- Provisional States;
- No Silent Repair;

- State Integrity;
- Reconciliation Confidence;
- Divergence Index;
- Transaction Health;
- Responsibility / Liability Matrix;
- Institutional Output Model;
- Technology Stack Alignment;
- Constitutional Governance;
- Failure-path-first testing;
- Regulatory Classification by actual functionality;
- Bank-controlled funds execution;
- Cross-system state reconciliation;
- Observation vs Assertion vs Confirmation distinction.
- Command vs Event architectural pattern.

No previously accepted SGTX architectural principle is intentionally removed by this consolidation. Concepts that previously appeared separately have been merged into this single canonical framework.

159.1 CANONICAL DISTINCTION: OBSERVATION, ASSERTION, CONFIRMATION

SGTX must explicitly distinguish three types of events:

Observation

SGTX observed an external condition or message.

Example:

Bank webhook received.

An observation is a record of receipt. It does not imply authenticity or authority by itself.

Assertion

A person or system claims that something happened.

Example:

Counterparty says payment was received.

An assertion is a claim. It has evidentiary value but is not authoritative confirmation.

Confirmation

An authoritative system confirms that something happened.

Example:

Bank confirms settlement.

A confirmation is authoritative within the relevant domain.

These must be distinct event types in the event spine.

The reconciliation engine uses this distinction to evaluate evidence strength and authority.

159.2 CANONICAL DISTINCTION: COMMAND VS EVENT

SGTX must explicitly distinguish:

Command

A request to cause an action.

Example:

Submit Settlement Instruction

A command is a request. It is not yet authoritative history.

Event

A record that the action occurred, was accepted, or was rejected.

Example:

SETTLEMENT_INSTRUCTION_SUBMITTED

An event is authoritative history.

A Command is not an Event.

The flow is:

text

text

COMMAND

?

Policy / Authority / Evidence Validation

?

EVENT (if accepted)

svgsVG

or:

text

text

COMMAND

?

Policy / Authority / Evidence Validation

?

REJECTION_EVENT

svgsVG

An API request itself must not become authoritative history unless it passes through the governed control path.

This prevents unauthorized state mutations and preserves the integrity of the event spine.

160. CANONICAL DATA MODEL

The canonical data model is defined in the SGTx Canonical Data Model Specification, which is a supporting document to this Constitution.

That specification includes:

- State Vector Store
- Event Spine
- Authority Matrix
- Finality Classes
- Settlement Instructions
- Settlement Instruction Versions
- Settlement Legs
- Reconciliation Cases
- Obligation Graph
- Financial Exposure
- External Identifier Registry
- Exception Events
- Transaction Twin

The implementation of these schemas may use various database technologies, but must adhere to the constitutional principles defined in this document.

161. CANONICAL API ENDPOINTS

The canonical API contract is defined in the SGTX API Contract Specification, which is a supporting document to this Constitution.

That specification defines:

- State Management APIs
- Event Management APIs
- Settlement Management APIs
- Reconciliation Management APIs
- Evidence Management APIs
- Authority Management APIs
- Transaction Twin APIs
- Certificate APIs
- Exception Management APIs
- Bank Settlement Gateway APIs

The API contract must enforce the Command vs Event pattern and the Observation vs Assertion vs Confirmation distinction.

162. IMPLEMENTATION MAPPING (Logical Domains)

The constitutional concepts map to logical ownership domains.

END OF INTEGRATED AMENDMENT BODY

PART D — DOCUMENT METADATA (v13.0)

This final part records the structural metadata of the v13.0 master document and a change log summarising the additions and expansions applied during this integration. Consistent with the integration discipline, no deletions have been performed; all changes are additions, insertions, or expansions of existing material.

Total line count of the final document: 88,825 lines (paragraph-level, excluding table-cell internal wrapping)

Approximate page count: approximately 771 pages (range 701–858); estimated at ~500 words per A4 page, 11–12 pt body, standard margins

Total word count (approximate): 385,852 words (approximate; includes body, amendment, and table cell text)

Change Log (v13.0)

Change 1: Added Part A — fresh, properly-ordered Contradiction & Consistency Audit (v13.0 Re-audit).

A new twenty-four-finding audit (A-01 through A-24) was prepended to the master document. Findings A-01 through A-23 consolidate and re-order (into ascending logical sequence) the audit carried in the v12.0 source, and expand each finding with an explicit Integration Impact statement. Finding A-24 is a new finding that records the completeness gap in the v12.0 appended amendment body and its resolution. A summary register table was added immediately after the audit introduction.

Change 2: Added Part B marker and integration banner.

A "PART B — PRESERVED v12.0 MASTER BLUEPRINT" heading and integration banner were added to clearly delimit the preserved v12.0 content from the new Part A audit. The original v12.0 audit header, blueprint body, and appended amendment body are preserved exactly as received.

Change 3: Re-integrated the previously-missing change-set segment (Add-Ons 9-28 and Final Summary preamble).

Per audit finding A-24, the segment of the source RTF comprising Add-Ons 9 through 28 and the preamble of the Final Summary (the all-add-on coverage table and articles 0 through 100 of the SGTX Master Global Trade Completion Amendment) was inserted in its canonical position, immediately preceding the existing article "101. FINAL EVIDENCE". This restores the complete change-set sequence while preserving every line of the v12.0 source. The insertion is bounded by clearly-labelled integration banners.

Change 4: Added Part D — Document Metadata.

A final Document Metadata part was appended, recording the total line count, approximate page count, approximate word count, and this change log.

Change 5: Expanded audit with Integration Impact statements.

Each of the twenty-three pre-existing audit findings (A-01 through A-23) was expanded with an explicit Integration Impact statement describing how the resolution is reflected in the merged master document. No original resolution language was removed; the expansions are purely additive.

Change 6: Preserved all original content.

No paragraph, table, heading, or list item from the v12.0 source document was deleted or rewritten. The v12.0 audit header (including its original reverse-ordered A-23 through A-01 presentation), the complete Main Blueprint body, and the complete appended amendment body (Add-Ons 1-8 and the Final Summary tail, articles 101-162) are all preserved verbatim. All 1,438 tables and all 46,765 original paragraphs are

retained.

Source Manifest

Source 1 (Main Blueprint, base of integration):
SGTX_PLATFORM_MASTER_BLUEPRINT_INTEGRATED_v12.0.docx. Logical paragraph lines
extracted: 76,122. Logical-text SHA-256:
c263f527f3966dab01d3cffc87e7d2747d01c017ca7a786d1964f09018087d42.

Source 2 (Updates / Modifications / Edits): sgtx add ons and modifications.rtf. Change-set text lines
extracted: 7,582. Raw-RTF SHA-256:
87181d220df82a485485eec4b9896031910c79afa6332516ef2afac4682c5b72.

Output: SGTX_PLATFORM_MASTER_BLUEPRINT_INTEGRATED_v13.0.docx. Integration date:
2026-08-24. Integration method: line-by-line preservation with additive insertions; no deletions performed.

End of v13.0 master document. This document supersedes v12.0 as the canonical integrated master
baseline; the v12.0 source remains fully preserved within it for traceability.

APPENDICES

Appendix A — Change Log v13.1 → v14.0

Change	Rationale
Three-layer separation (L0/L1/L2)	Separate immutable principles from mutable architecture from testable specs
Over-claim elimination	'production-ready' → CORE_READY; 'complete' → 'specified'
28-add-on status matrix	Explicit deployment-state vocabulary per add-on
4-dimension external readiness	TECHNICAL/LEGAL/OPERATIONAL/COMMERCIAL independently reported
Command/Event taxonomy	Command (intent) ≠ Event (fact) — enforced in API contract
32-point constitution	All 32 points listed explicitly with [L0] tags
Audit trail preserved	A-01 through A-24 as governing resolutions
No calendar claims	P0-P4 waves are dependency-driven
Full content preservation	ALL v13.1 paragraphs included — nothing deleted

Appendix B — Source Manifest

Source	Type	Pages/Lines	SHA-256
SGTX_v13.1_FINAL.docx	Integrated Edition	2,312 pages / 259,960 words	(see document metadata)
v12.0 baseline	Main Blueprint	~76,122 lines	c263f527f3966dab01d3cfc87e7d2747d01c01
sgtx add ons and modifications.rtf	Change-Set	~7,582 lines	87181d220df82a485485eec4b9896031910c79